

现代 GPU 编程 · 面向 ML 系统 (工程师版)

SUB 现代 GPU 编程 · 面向 ML 系统 (工程师版)

基于 mlc.ai 公开教材《Modern GPU Programming for MLSys》的中文原创精读笔记
仅供学习 · 非逐字翻译 · 以原文为准

Contents

现代 GPU 编程 · 面向 ML 系统 (中文精读笔记)	11
目录	11
建议阅读路径	13
版权与免责声明	14
第 0 章 · 写给纯新手:GPU & ML Infra 极简入门	15
0.1 先说清楚: 这本书在讲什么, 你为什么需要先读这一章	15
0.2 一个贯穿全书的比喻:GPU 是一座” 超大工厂”	15
0.3 谁来干活: 线程的层级	16
0.4 东西放哪: 内存的层级	17
0.5 到底在算什么 (一): 矩阵乘 GEMM	18
0.6 到底在算什么 (二):Tensor Core 与 MMA	18
0.7 怎么喂数据: 异步搬运、流水线与” 重叠”	18
0.8 注意力 (Attention) 是什么, 为什么在硬件上难	19
0.9 这本书的两个” 新东西”:Blackwell 与 TIRx	19
0.10 术语速查表 (读后面遇到生词就回来翻)	20
0.11 该怎么读这本书	20
小结	21
延伸阅读	21
第 01 章 · GPU 执行模型	23
一、执行层级: 为什么不是” 一锅线程”	24
二、内存空间: 容量与速度的取舍	26
三、计算引擎:CUDA 核与张量核	28
四、GEMM 数据流水线: 把拼图拼起来	29
小结	31
延伸阅读	32
术语对照	32
第 02 章 · 什么决定 Kernel 的速度	35
一、屋顶线模型: 两条路取更慢的那条	36
二、常见 workload 的算术强度	37
三、算术强度低的时候怎么办	39
四、优化阶梯: 从” 可能” 到” 做到”	40
五、重叠是最大的杠杆	41
六、占用率与资源压力	42
小结	43
延伸阅读	44

术语对照	44
第 03 章 · 数据布局与其记号	47
一、形状-步长模型 (The Shape-Stride Model)	48
二、Tile 布局 (Tile Layout)	49
三、命名轴 (Named Axes)	50
四、分布式布局 (Distributed Layout)	52
五、Swizzle 布局 (Swizzle Layout)	54
小结	56
延伸阅读	57
术语对照	57
第 04 章 · 各代 GPU 的 Tensor Core 操作数布局	59
引子: 为什么“逻辑公式没变”还会算错	59
一、两条从未离场的约束	60
二、Ampere: 跨 warp lane 的寄存器片段	61
三、Hopper:wgmma、共享内存描述符与 swizzle 格式	65
四、Blackwell:tcgen05 与 TMEM	67
五、一个反复出现的片段:m8n8	70
六、贯穿三代的主线 (The Throughline)	70
小结	71
延伸阅读	71
术语对照	72
第 05 章 · 异步数据搬运:TMA	73
0. 为什么需要 TMA: 从「线程自己搬」说起	73
1. 一个线程发起, 硬件搬运整块 tile	74
2. Swizzle 布局: 不只是「搬到」, 还要「摆对」	75
3. 3D TMA: 在一次拷贝里同时完成「分块」和「swizzle」	76
4. 完成机制 (一): 加载用 mbarrier	77
5. 完成机制 (二): 存储用 commit group + wait group	78
6. 为什么 TMA 对流水线 (pipelining) 至关重要	79
小结	80
延伸阅读	80
术语对照	80
第 06 章 · 特殊内存:TMEM(Tensor Memory)	83
引子: 寄存器不够用,Blackwell 才另开了一条路	83
TMEM 是一个二维地址空间	84
分配: 把 TMEM 当成像 SMEM 一样要做预算的资源	85
读写 TMEM: 三条专用的异步通路	86
把整条数据通路串起来:TMEM 就在 Tensor Core 数据流的正中间	89
小结	89
延伸阅读	89

术语对照	90
第 07 章 · 异步协调:mbarrier	91
7.1 为什么需要一个显式的”完成信号”	91
7.2 mbarrier 到底是什么	92
7.3 相位追踪 (Phase Tracking)	94
7.4 同步规则 (Synchronization Rules)	96
小结	97
延伸阅读	98
术语对照	98
第 08 章 · 进阶: 集群启动控制 (CLC)	99
8.1 从静态调度到动态调度: 为什么需要 CLC	99
8.2 两条指令: 请求与查询	101
8.3 工作窃取循环	102
8.4 与常驻 GEMM 的关系	103
小结	104
延伸阅读	104
术语对照	105
第 09 章 · TIRx 入门	107
一、为什么需要 TIRx: 把「隐藏的决策」翻到台面上	107
二、运行环境与依赖	108
三、第一个 kernel: 单次 MMA 的 GEMM	109
四、三要素回读:scope / layout / dispatch	114
五、编译是怎么发生的	115
六、接下来去哪	116
小结	117
延伸阅读	117
术语对照	117
第 10 章 · TIRx Layout API	119
10.1 为什么需要 Layout API: 让记号变成对象	119
10.2 先看几个例子 (Layouts by Example)	120
10.3 交互演示 (Interactive Demo) 要点	121
10.4 TileLayout: 主力仿射布局	121
10.5 具名轴 (Named Axes): 名字本身就是语义	123
10.6 前向映射 (Forward Mapping):layout.apply()	124
10.7 案例 A: 张量核寄存器 tile	125
10.8 案例 B:Blackwell 张量内存 (TMEM)	126
10.9 缩放因子布局 (Scale Factor Layouts): 复制的真正用武之地	127
10.10 现成布局 (Ready-Made Layouts)	127
10.11 SwizzleLayout 与 ComposeLayout: 仿射之外的换序	128
10.12 为什么要 swizzle:bank 冲突的来由	129

10.13 swizzle 变换的数学形式	129
10.14 如何选择 swizzle 参数	130
10.15 元素的 bank 与 line	130
10.16 完整演算: 对 (8, 64) float16 tile 施加 128B swizzle	130
10.17 设计原理 (Design Rationale)	131
小结	132
延伸阅读	132
术语对照	132
第 11 章 · 构建分块 GEMM(Step 1-3)	135
本章导读: 从一个正确的 Tile 开始 (Building a Tiled GEMM)	135
GEMM: 问题定义与数据通路 (GEMM)	137
优化路线图 (Optimization Path)	139
第 1 步: 单 Tile 顺序 GEMM(Step 1: Sequential Single-Tile GEMM)	141
第 2 步:K 维循环累加 (Step 2: K-Loop Accumulation)	147
第 3 步: 空间分块, 多 CTA 并行 (Step 3: Spatial Tiling / Multi-CTA)	151
习题精解 (Exercises)	155
小结	159
延伸阅读	159
术语对照	160
第 12 章 · 用 TMA 为 GEMM 做流水线 (Step 4-6)	163
为什么基础版 GEMM 在浪费时间	164
Step 4:TMA 异步 Load	164
Step 5: 软件流水线 (PIPE_DEPTH=2)	168
Step 6: 持久化 kernel + tile 调度器	171
练习 (原书 Exercises)	173
小结	173
延伸阅读	174
术语对照	174
第 13 章 · 用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)	177
引言: 用 Warp 专门化与 Cluster 把 GEMM 推到极致 (Scaling GEMM with Warp Specialization and Clusters)	177
第 7 步:Warp 专门化 + 流水线 (Step 7: Warp Specialization + Pipeline)	180
第 8 步: 用 2-CTA Cluster 协同算一个大 Tile(Step 8: 2-CTA Cluster)	186
第 9 步: 多消费者 Warp 专门化 (Step 9: Multi-Consumer Warp Specialization)	191
端到端的优化成果 (End-to-End Result)	196
习题与自测 (Exercises)	199
小结	203
延伸阅读	203
术语对照	204

第 14 章 · Flash Attention 4	207
本章导读: 为什么 Attention 不是「再来一遍 GEMM」(Overview)	207
算法的形状 (Algorithm Shape)	209
Tile-原语图 (Tile-Primitive Graph)	211
Warp 的角色与作用域 (Warp Roles and Scopes)	214
怎么读后面的代码片段 (Reading the Fragments)	216
两个 MMA 阶段: 被 softmax 串起来的双矩阵乘 (The Two MMA Phases)	218
TMEM 的布局与复用 (TMEM Layout and Reuse)	222
barrier 如何把各个角色「接线」起来 (How Barriers Connect the Roles)	225
流水线结构: 同一时刻到底谁在干活 (Pipelining Structure)	228
重缩放与写回 (Rescaling and Writeback)	231
因果掩码 (Causal Masking)	234
GQA 支持 (GQA Support)	236
Tile 调度 (Tile Scheduling)	238
编译并验证 (Compile and Verify)	241
与 GEMM 的区别 (Differences from GEMM)	243
课后练习 (Exercises)	245
小结	247
延伸阅读	248
术语对照	248
第 15 章 · 参考资料 (附录)	251
15.1 这一部分是什么	251
15.2 三类需求 → 三个去处	251
15.3 两个”不在此处, 但你可能要找”的指针	252
15.4 建议的使用方式 (阅读顺序)	253
小结	253
延伸阅读	254
术语对照	254
第 16 章 · 调试 Warp 专门化 Kernel	255
16.1 心法: 把异步 kernel 看成一组「交接」	256
16.2 调试前: 先排除运行环境	256
16.3 标准调试流程 (Workflow)	257
16.4 「四要素表」: 可迁移的调试工作表	258
16.5 当编译失败时	259
16.6 检查生成的代码: 让 CUDA 当地图	259
16.7 何时必须重启 Python	260
16.8 症状地图 (Symptom Map)	260
16.9 死锁 (Deadlocks)	261
16.10 崩溃与 context 毒化	263
16.11 Step 7 参考骨架	263
16.12 结果错误 (Wrong Results)	264

16.13 正确但慢 (Correct but Slow)	265
16.14 提交一个好的 Issue	265
小结	266
延伸阅读	266
术语对照	266
第 17 章 · 编译器内部 (概览)	269
17.1 这一页是什么: 写给贡献者的入口	269
17.2 子页面一览: 目前只有「TIRx 下降流水线」	270
17.3 速览:TIRx 下降流水线在做什么	270
17.4 建议的阅读顺序	272
小结	272
延伸阅读	273
术语对照	273
第 18 章 · TIRx Lowering 流水线	275
18.1 这条流水线解决什么问题	275
18.2 它处在编译流程的哪个位置	276
18.3 流水线的 18 个 pass(逐个看)	277
18.4 深入 LowerTIRx: 核心下降是怎么发生的	279
18.5 一个完整的小例子:scale kernel 的下降全程	280
18.6 自己动手复现每个阶段	281
小结	282
延伸阅读	282
术语对照	283
第 19 章 · TIRx 语言参考 (概览)	285
19.1 这一页是什么	285
19.2 五个子页面分别讲什么	285
19.3 建议的阅读顺序	290
小结	290
延伸阅读	291
术语对照	291
第 20 章 · 解析器工具 (Parser utilities)	293
一、先搞清楚: 什么叫「解析期工具」	293
二、T.meta_var 一把 Python 值 inline 进 IR	294
三、@T.inline 一内联函数	295
四、@T.meta_class 一解析期状态对象	297
五、T.constexpr 一编译期 kernel 参数	298
六、四个工具的横向对比	299
小结	299
延伸阅读	300
术语对照	300

第 21 章 · 数据类型与表达式	301
21.0 为什么要把”类型”拆成两层?	301
21.1 表达式的 dtype	302
21.3 指针 (handle)	305
小结	307
延伸阅读	308
术语对照	308
第 22 章 · 缓冲区与内存	309
一、缓冲区的统一心智模型	309
三、共享内存 (shared memory)	312
四、寄存器 (registers)	315
六、缓冲区方法 (Buffer APIs)	318
第 23 章 · 控制流	321
一、if / else: 分支	321
二、统一控制流 vs 发散控制流 (核心)	323
三、循环家族	325
第 24 章 · CUDA C++ / PTX intrinsics	329
24.1 为什么需要「逃生通道」	329
24.2 调用后端内建函数 (backend intrinsics)	330
24.4 内联原始 CUDA(Inlining raw CUDA)	335
小结	336
延伸阅读	337
术语对照	337
术语对照表 (全书总表)	339
A · 架构与执行模型	339
B · 性能模型	339
C · 数据布局 (Layout)	340
D · Tensor Core 与 MMA	340
E · 内存层级与特殊内存	341
F · 异步搬运与协调	341
G · CLC 与调度	342
H · Warp 专门化与调试	342
I · TIRx 语言与控制流	343
J · 解析期 / 元编程	343
K · 编译器内部与下降	344
L · 参考与生态	344
M · 关键算法	344

现代 GPU 编程 · 面向 ML 系统 (中文精读笔记)

这是基于 mlc.ai 公开教材《Modern GPU Programming for MLC Sys》(© 2026 MLC Community) 撰写的 **中文原创精读笔记 / 学习导读**, 仅用于学习目的。

本笔记是对原书的转述、归纳与讲解, 并非原书的逐字翻译; 书中所有结论、代码示例、设计取舍均以原文为准。阅读原文请访问官方网站:

[<https://mlc.ai/modern-gpu-programming-for-mlsys/>](https://mlc.ai/modern-gpu-programming-for-mlsys/)

这份笔记就想带你走通一条路: 从“GPU 到底是怎么跑代码的”, 一直走到“怎么亲手写出真正快的内核”。怎么走? 顺着三条主线——先看 **Blackwell 架构**, 搞清楚硬件长啥样; 再学 **TIRx 编程模型**, 知道拿什么语言去写; 最后是 **GEMM 和 Flash Attention 实战**, 把前面学的东西全拿出来用一遍。末尾还塞了一整套编译器内部和语言参考, 你随时翻回来查就行。

前置知识

前置知识: 完全没 GPU 基础? **别直接从第 1 章开始**——先花 20 分钟读第 0 章·写给纯新手:GPU & ML Infra 极简入门。它用大白话把 warp、内存层级、GEMM、Tensor Core 这些后面满天飞的词先讲一遍, 有了画面感, 后面会顺很多。

目录

下面表格里,「原文链接」点过去是 mlc.ai 的官方页面; 章节标题点过去是本仓库里对应的笔记。

新手必读

章节	标题	一句话简介
00	写给纯新手:GPU & ML Infra 极简入门	零基础总入口: 用大白话讲清线程层级 / 内存层级 / GEMM / Tensor Core / 重叠, 先建立画面再读正文。

第一部分 · 硬件、内存与执行模型

章节	标题	一句话简介	原文链接
01	GPU 执行模型	从 SIMT、warp / CTA / cluster 到 Tensor Core、TMA、TMEM, 建立现代 GPU 的整体心智模型。	原文

02	什么决定 Kernel 的速度	用屋顶线模型（算术强度定胜负）建立「先诊断、再优化」的工作流，核心杠杆是重叠。	原文
03	数据布局与其记号	形状 / 步长 / 视图 / 分块 / 命名轴 / swizzle 的统一记号，是后续所有 layout 讨论的语言。	原文
04	各代 GPU 的 Tensor Core 操作数布局	从 Ampere 到 Blackwell,MMA 操作数 / 累加器在寄存器与 TMEM 中的迭代布局演变。	原文
05	异步数据搬运:TMA	张量内存加速器: 用张量映射描述符 + tile 坐标 + mbarrier 实现异步批量搬运与多级缓冲。	原文
06	特殊内存:TMEM(Tensor Memory)	Blackwell 专供 Tensor Core 累加的二维 (Lane × Col) 片上内存及其使用约束。	原文
07	异步协调:mbarrier	到达计数 + 相位位 + 字节计数三机一体的异步屏障，是所有流水线的协调中枢。	原文
08	进阶: 集群启动控制 (CLC)	用持久化核 + 工作窃取式 tile 调度棒干尾部 SM，理解关键路径与尾部行为。	原文

第二部分 · TIRx 编程模型

章节	标题	一句话简介	原文链接
09	TIRx 入门	IR 层级的 GPU 编程 DSL，用最小 GEMM 讲透 scope / layout / dispatch 三大要素。	原文
10	TIRx Layout API	把纸面布局记号变成编译器对象:TileLayout / SwizzleLayout / ComposeLayout。	原文

第三部分 · GEMM 实战 (Step 1→9)

章节	标题	一句话简介	原文链接
11	构建分块 GEMM(Step 1-3)	从最朴素的分块 GEMM 起步，搭好寄存器 / 共享内存 / MMA 的基本骨架。	原文
12	用 TMA 为 GEMM 做流水线 (Step 4-6)	引入 TMA + mbarrier + 双缓冲 / 持久化 kernel，让搬运与计算重叠起来。	原文
13	用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)	用 warp 专门化拆分生产者 / 消费者，并借 cluster 跨 CTA 协作冲击峰值。	原文

第四部分 · Flash Attention 4

章节	标题	一句话简介	原文链接
14	Flash Attention 4	把前述全部机制 (TMA / TMEM / mbarrier / warp 专门化) 综合到一个完整的 FA4 内核中。	原文

第五部分 · 参考与附录

章节	标题	一句话简介	原文链接
15	参考资料 (附录)	一张「需求 → 去处」导航表, 告诉你查语言特性 / 看下降流水线 / 调内核时该翻哪一页。	原文
16	调试 Warp 专门化 Kernel	把内核看成一组「交接」, 用「角色 / 存储 / 交接 / 生命周期」四要素表对照生成的 CUDA 排错。	原文
17	编译器内部 (概览)	面向贡献者: 讲清 <code>tvm.compile(tir_pipeline="tirx")</code> 内部的下降流水线与关键阶段。	原文
18	TIRx Lowering 流水线	高层 TIRx 经 18 个模块级 pass 下降为 host/device 函数, 核心是 LowerTIRx。	原文
19	TIRx 语言参考 (概览)	语言特性分五块 (解析器工具 / 数据类型 / 缓冲区 / 控制流 / 内建) 的导读与速查。	原文
20	解析器工具 (Parser utilities)	<code>T.meta_var</code> / <code>@T.inline</code> / <code>@T.meta_class</code> / <code>T.constexpr</code> 四大解析期元编程工具。	原文
21	数据类型与表达式	<code>dtype</code> / <code>type</code> / <code>PrimExpr</code> / 指针 / 向量化访存的语义与下降到 CUDA 的方式。	原文
22	缓冲区与内存	<code>buffer</code> = 指针 + 元数据; 静态 / 动态共享内存、竞技场分配、TMEM 列偏移。	原文
23	控制流	统一 vs 发散控制流、集体操作、 <code>elect_sync</code> 、各类同步与循环修饰符。	原文
24	CUDA C++ / PTX intrinsics	后端内建函数: 选举 / 具名屏障 / 栅栏 / 异步代理 / 蝶形归约等逃生口。	原文

建议阅读路径

每个人来这儿的起点都不一样, 所以我给你列了几条路。你先看看哪条最像自己, 照着走就行:

- **路线 A · 从零开始打基础 (最适合新手):** 干脆从 **01 一路读到 24**。第一部分先把硬件和执行模型啃明白, 心里有个谱; 第二部分学会怎么用 **TIRx** 把这些写成代码; 到了第三、第四部分的实战, 再把前面那些机制一个一个串起来。第五部分你不用从头读到尾, 当成查字典的工具书, 需要了再翻。
- **路线 B · 上来就想写 GEMM:** 你要是 **CUDA** 和 **GPU** 架构本来就熟, 那别绕弯子, 直接跳到 **第 11 章**动手写分块 **GEMM**。写着写着碰到不认识的东西——比如 **TMA**、**mbarrier**、**TMEM**、**warp** 专门化——再回头翻 **第 05 到 08 章**补一补就好。
- **路线 C · 只想弄懂性能优化是怎么个思路:** 先读 **第 02 章**, 把屋顶线模型和”重叠”这两件事吃透, 养成”先诊断、再动手”的习惯; 然后看 **第 08 章** (持久化核和尾部行为) 和 **第 13 章** (**warp** 专门化、**cluster**), 慢慢就摸清怎么把硬件的性能压榨干净了。
- **路线 D · 想给 TIRx 或编译器贡献代码:** 直接进 **第 17、18 章**, 先把编译器内部和下降流水线看懂; **第 19 到 24 章**那套语言参考放手边, 要查语义随时翻。
- **要是你正在抓那种莫名其妙的 bug(数据被写坏、死锁、数据竞争):** 别慌, 先翻 **第 16 章** (调试 **Warp** 专门化 **Kernel**), 照着那张四要素交接表一项一项对过去——这类问题十有八九就是某个交接没对齐。

提示

提示: 每章的核心概念都配了中英对照和一句话解释, 想查的话翻本仓库的 **术语对照表**就行。

版权与免责声明

- 原教材《Modern GPU Programming for ML Sys》的版权是 © 2026 MLC Community 的。
- 这个仓库是**非官方的中文学习笔记**, 东西都是社区里学习的人自己整理的, 难免有理解不到位的地方; 只要和原文对不上, **就以原文为准**。
- 这份笔记只用来学习和交流, 千万别拿去做商业用途。

第 0 章 · 写给纯新手:GPU & ML Infra 极简入门

本章是中文笔记新增的”地基”章，原书没有对应章节。如果你会写代码 (Python / C++ / 后端 / 数据都行) 但几乎没碰过 GPU，**请从这一章开始**。读完它，你再翻后面任何一章，那些满天飞的 warp、SMEM、tile、MMA 才会有画面感，而不是一堆吓人的黑话。

本章要点

本章要点 (TL;DR)

- GPU 不是”一颗更快的 CPU”，而是一座**超大并行工厂**：几万个很弱的小工人（线程）被编成班组一起干活。
- 三件事撑起整本书：**谁来干活**（线程层级:thread / warp / lane / warpgroup / CTA / cluster）、**东西放哪**（内存层级:寄存器 / 共享内存 / 全局内存 / 张量内存）、**到底在算什么**（矩阵乘 GEMM 和注意力 Attention）。
- GPU 跑得快的秘诀只有一个词：**重叠 (overlap)**——让”搬数据”和”做计算”同时进行，谁也别等谁。
- 这本书讲的是 NVIDIA 最新的 **Blackwell** 架构，以及一种用来写 GPU 内核的语言 **TIRx**。这两个名字现在记不住没关系，本章末尾会点一下。
- 这一章只求”听个大概、建立画面”。每个概念后面都有专章细讲，**看不懂的细节先放过，有印象就行**。

0.1 先说清楚：这本书在讲什么，你为什么需要先读这一章

这本书讲的是怎么把 GPU 榨干、写出”飞快”的计算内核——尤其是深度学习里最吃算力的两个运算：**矩阵乘法 (GEMM)** 和 **注意力 (Attention)**。

问题在于：原书默认你已经懂 GPU 是怎么工作的，所以上来就是 warp、共享内存、Tensor Core……对一个没写过 GPU 的人，这就像没学过加减乘除就被拉去听微积分。

所以这一章的任务很简单：**用大白话，把后面会反复出现的那套基础词汇，一次性给你讲明白。不求深，只求你心里有画面**。

0.2 一个贯穿全书的比喻:GPU 是一座”超大工厂”

把 GPU 想象成一座工厂，你大概就抓住八成了：

- **工人 = 线程 (thread)**：数量极多 (几万个)，但每个都很弱、只会干很简单的活。
- **班组 = 线程的层级**：工人不是一盘散沙，而是 32 人一个**小班 (warp)**、几个小班一个**车间 (CTA)**……一层层编起来，方便协作。

- **仓库 = 各级内存:** 有的仓库就在工位手边 (快但极小), 有的在车间角落 (中等), 有的在厂区外的大仓库 (巨大但远)。
- **专用机床 = Tensor Core:** 普通工人 (CUDA 核) 啥都能干但慢; 而工厂里有几台专门做” 矩阵乘” 的高速机床, 一开起来吞吐量甩开普通工人一个数量级。
- **传送带 = 数据搬运引擎 (TMA):** 专门负责把原料从大仓库运到工位, 且运的时候不占用工人的手。

关键

关键: 工厂效率高不高, 不在于某个工人多快, 而在于**机床别停**。只要传送带能一直把原料及时送到, 高速机床就能一直满负荷转。这就是后面整本书在折腾的事。

0.3 谁来干活: 线程的层级

GPU 不会把几万个线程当成一个扁平的大池子, 而是一**层层嵌套**地组织起来。为什么? 因为” 协作” 发生在不同的尺度上——有时两个线程要交换一个值, 有时一整个车间要共用一块数据。每一层, 都是为了让” 某个尺度的协作” 变得便宜。

从小到大, 六层:

结构图

连接关系:

- Grid 网格 / (一次 kernel 的全部线程) → Cluster 集群 / (一组车间, 可跨多个 SM)
- Cluster 集群 / (一组车间, 可跨多个 SM) → CTA / Block 车间 / (跑在一个 SM 上, 调度的基本单位)
- CTA / Block 车间 / (跑在一个 SM 上, 调度的基本单位) → Warpgroup 班组 / (4 个 warp = 128 线程)
- Warpgroup 班组 / (4 个 warp = 128 线程) → Warp 小班 / (32 个线程, 同步执行)
- Warp 小班 / (32 个线程, 同步执行) → Thread 工人 / (最小执行单位)

逐个认识一下 (这张表是全书最该记住的一张):

名字	是什么	大白话
线程 / thread	最小的执行单位, 有自己的” 程序计数器” 和私有寄存器	一个工人
lane(通道)	warp 里某个线程的 编号 (0-31)	工人在小班里的 座位号 。注意:lane 不是新硬件, 就是线程的编号
warp(线程束)	32 个线程 捆在一起, 同一时刻发同一条指令	一个 32 人 小班, 动作整齐划一
warpgroup(线程束组)	4 个 warp = 128 个线程	4 个 小班拼成的班组 (较新的架构才有)
CTA / block(线程块)	一批线程, 跑在 同一个 SM (下面解释)上, 共享一块” 共享内存”	一个车间
cluster(集群)	一组 CTA, 可以 分布在不同 SM 上 还能互相协作	几个车间联手
grid(网格)	一次 kernel 启动的 所有线程	整个厂这一单活的全部人手

关键

关键: 这几层是”谁装谁”的包含关系,但**每一层具体装几个,有的硬件定死、有的你自己定**。
硬性规定: 一个 warp 永远是 32 个线程, 一个 warpgroup 永远是 4 个 warp(= 128 线程)。
可一个 **CTA(车间) 里到底放几个 warp, 是你写 kernel 时自己定的**——比如一个 128 线程的 CTA = 4 个 warp = 1 个 warpgroup; 一个 256 线程的 CTA = 8 个 warp = 2 个 warpgroup。所以别把 warpgroup 当成 CTA 和 warp 中间一个固定的硬层级, 它说白了就是”把相邻的 4 个 warp 凑成一组”的叫法, 方便某些指令一次指挥 128 个线程。

还有两个一定会撞见的词:

- **SM(流式多处理器 / Streaming Multiprocessor):**GPU 里真正干活的”车间厂房”。一颗 GPU 由很多个 SM 拼成, 一个 CTA 跑在一个 SM 上。
- **SIMT(单指令多线程):**warp 里 32 个线程**发同一条指令**, 但各自拿自己的数据。这就是为什么 GPU 适合”对一大堆数据做同样的事”。

注意

注意:”同发一条指令”不代表它们必须走同一个 if 分支。每个 lane 可以被单独”屏蔽”掉, 所以同一个 warp 里有的 lane 走 if、有的走 else 也行——只是这时硬件得分两趟跑, 慢一些 (这叫**分支发散**)。

0.4 东西放哪: 内存的层级

GPU 的内存不是一块, 而是**好几层**, 典型规律是: **越靠近工人的越快、但越小; 越远的越大、但越慢**。

名字	在哪	谁能用	特点	工厂比喻
寄存器 / Register(RF)	紧贴计算单元	每个线程私有	最快、最小	工人手里握着的几个零件
共享内存 / SMEM	SM 片上	一个 CTA 内共享	很快、不大	车间中间的公用工作台
全局内存 / GMEM(也叫 HBM)	显存	所有线程都能访问	很大、相对慢	厂区外的大仓库
张量内存 / TMEM	SM 片上 (Blackwell 新增)	给 Tensor Core 放累加结果用	专用	高速机床旁边的专属料台

数据的典型流向就是: **从大仓库 (GMEM) 搬到工作台 (SMEM)→ 工人/机床计算 → 结果再写回大仓库**。

这里有两个性能上极其要命、后面反复出现的概念, 先混个脸熟:

- **访存合并 (coalescing):** 一个 warp 的 32 个线程同时读数据时, 如果它们要的地址**正好连续**, 硬件能把 32 次访问合并成 **1 次**; 如果地址东一个西一个, 就可能裂成 32 次。差距几十倍。
- **bank 冲突 (bank conflict):** 共享内存被切成 32 个”**bank(存储体)**”。32 个线程如果各读各的 bank, 能并行; 如果挤到**同一个 bank 的不同地址**, 硬件只能排队一个个来。这也是为什么后面要费劲折腾数据的”摆放方式”(布局、swizzle)。

关键

关键: GPU 编程里大量的“奇技淫巧”，归根结底都是在回答同一个问题——怎么把数据摆好，让 32 个线程读得又快又不打架。

0.5 到底在算什么 (一): 矩阵乘 GEMM

深度学习里最核心、最吃算力的运算，就是**矩阵乘法**。专业点叫 **GEMM**(General Matrix Multiply, 通用矩阵乘)，本质就是你高中学的 $C = A \times B$ 。

为什么它是主角？因为神经网络里的全连接层、卷积、注意力，落到底层几乎全是矩阵乘。GPU 跑得快不快，很大程度上就看 GEMM 跑得快不快。

关键概念:tile(分块)。一个矩阵可能是 4096×4096 ，一次根本算不完、也放不进快内存。怎么办？**切块**。把大矩阵切成一个个固定大小的小方块（比如 128×128 ），每次只搬一小块、算一小块。这个小方块就叫 **tile**。整本书你会看到无数次 tile——记住它就是“大矩阵切出来的小方块”。

还有一个公式你会反复看到： $D = A \cdot B + C$ 。它读作“乘了再加”：

- A、B: 要相乘的两块小矩阵 (tile);
- C: 之前已经累加的中间结果;
- D: 这一步算完的新结果。

为什么要“加上 C”？因为大矩阵乘被切成很多小块后，**最终结果是很多小块乘积累加起来的**。那个一直在累加的 C/D，就叫**累加器 (accumulator)**。它放哪儿，其实是个大学问：累加器往往不小，而且要从第一块一直累到最后一块，**全程都得占着地方**。要是霸占着每个线程那点**稀缺的寄存器**，就没几个寄存器干别的了。所以最新的 Blackwell 干脆给它单开了一块专用片上内存——就是 0.4 里提到的 **TMEM**，专门用来摆累加器，把宝贵的寄存器腾出来。（这条线索，第 6 章会细讲。）

0.6 到底在算什么 (二):Tensor Core 与 MMA

GPU 里有两类“算手”：

- **CUDA 核 (CUDA Core)**: 通用的标量计算单元，啥都能算（加减乘除、判断、寻址），但做矩阵乘相对慢。
- **Tensor Core(张量核)**: 专门做矩阵乘的固定功能硬件。它一次就能吞下两块 tile，直接算出 $D = A \cdot B + C$ ，吞吐量比 CUDA 核高一个数量级以上。

Tensor Core 做的这个“小块矩阵乘加”操作，有个专门的名字：**MMA**(Matrix Multiply-Accumulate, 矩阵乘累加)。你还会看到 **wgmma**、**tcgen05** 这种词——它们是不同架构上发起 MMA 的具体指令，现在只需知道“哦，这是在让 Tensor Core 算矩阵乘”就够了。

关键

关键: 现代 GPU 之所以能跑大模型，核心就是 Tensor Core。后面大半本书的折腾，本质都是为了让 Tensor Core 一刻不停地有数据可算。

0.7 怎么喂数据: 异步搬运、流水线与“重叠”

Tensor Core 再快，**没数据可算也只能干瞪眼**。所以“怎么及时把数据喂上”和“怎么算”一样重要。

- **同步 vs 异步**: 老办法是让计算线程自己去搬数据——但它搬的时候就没法算了，白白浪费。新办法是**异步搬运**: 线程只下个”搬运指令”，活儿交给专门的硬件引擎，自己继续算。
- **TMA(张量内存加速器 / Tensor Memory Accelerator)**: 就是这个专门的搬运引擎，负责在大仓库 (GMEM) 和工作台 (SMEM) 之间**成批、异步**地搬 tile。
- **mbarrier(异步屏障)**: 既然搬运是异步的，就得有个东西通知”搬完了，可以用了”。mbarrier 就是这个”信号灯”，负责在搬运方和计算方之间对暗号。
- **流水线 / 重叠 (pipeline / overlap)**: 这是**全书的灵魂**。与其”搬完 → 算完 → 写完”一步步串着来 (同一时刻只有一个引擎在忙)，不如让它们**像流水线一样错开重叠**: 这边在算第 k 块，传送带已经在搬第 k+1 块，收尾引擎在写第 k-1 块。三个引擎同时忙，整体就快得多。

结构图

分组:

- **串行**: 一个等一个 (慢): 「搬第 k 块」「算第 k 块」「写第 k 块」「搬第 k+1 块」
- **重叠**: 三个引擎同时忙 (快): 「搬 k+1 块」「算 k 块」「写 k-1 块」

关键

关键: 记住这一句，后面就不会迷路——**慢内核和快内核的差距，几乎全在”重不重叠”上。**

0.8 注意力 (Attention) 是什么，为什么在硬件上难

Transformer 大模型的核心运算是**注意力 (Attention)**。抛开数学，它干的事是: 对每个词，去看序列里所有其他词、算出”该关注谁多一点”，再据此把信息加权汇总。

落到计算上，它是**两次矩阵乘，中间夹一个 softmax**:

1. $S = Q \cdot K^T$: 算出一个**分数矩阵 (score matrix)**，形状是 $n \times n$ 。序列一长，这个矩阵就大得吓人。
2. **softmax(S)**: 把一行分数变成一组**权重**。做法是: 每个分数先取指数 $\exp$ ，再除以这一行的总和。效果是——分数大的被进一步拉大、所有权重都为正、且一行加起来正好等于 1(所以叫”归一化”，你可以理解成”把一行分数换算成百分比”)。
3. $O = \text{softmax}(S) \cdot V$: 用权重把信息加权汇总成输出。

难点在哪? 那个巨大的分数矩阵 S 如果老老实实存进显存，**又占地方又慢**。更麻烦的是 softmax 本身: 它要除以”**一整行的总和**”，还要先找出这一行的最大值 (为了数值稳定)。这就好像逼着你**先把一整行的分数都算出来、攒齐了**，才能开始归一化——那不就等于要把 S 整个存下来吗?

著名的 **Flash Attention** 就是来破这个两难的。它的核心思路是: **绝不把完整的 S 存下来**，而是一块一块地流式过 K/V ，**边算边维护两个”运行中的统计量”**——当前见过的最大值、当前的累计总和; 每来一块新数据，就**按比例修正**之前的结果。这样既不用存下整个 S，又能算出和”完整 softmax”完全一样的答案。这套技巧的硬件实现，正是本书第 14 章的高潮。

现在你只需记住: **注意力 = 两个矩阵乘 + 中间一个 softmax，且要想办法把那个大分数矩阵实际存出来。**

0.9 这本书的两个”新东西”:Blackwell 与 TIRx

最后两个名字, 扫个盲就行:

- **Blackwell**:NVIDIA 最新一代 GPU 架构 (代表卡如 B200)。GPU 架构是一代代演进的:**Ampere(A100)**→ **Hopper(H100)**→ **Blackwell(B200)**。一代比一代多出新硬件 (比如 Hopper 带来了 TMA 和 warpgroup,Blackwell 带来了 TMEM)。本书专门讲 Blackwell, 所以会出现很多”只有最新卡才有”的特性。
- **TIRx**: 本书用来写 GPU 内核的**编程语言/工具**, 是嵌在 Python 里的一种 DSL(随 Apache TVM 一起发布)。它的特别之处是: **让你直接、明确地点名硬件** (这块数据放 SMEM、这步用 Tensor Core 算……), 而不像高层框架那样把硬件细节藏起来。第 9 章开始专门讲它。

0.10 术语速查表 (读后面遇到生词就回来翻)

English	中文	一句话理解
thread	线程	一个最小的工人
lane	通道	线程在 warp 里的座位号 0~31
warp	线程束	32 个线程的小班, 同步执行
warpgroup	线程束组	4 个 warp = 128 线程
CTA / block	线程块 / 车间	跑在一个 SM 上的一批线程
cluster	集群	可跨 SM 协作的一组 CTA
grid	网格	一次 kernel 的全部线程
SM	流式多处理器	GPU 里真正干活的”车间/厂房”
SIMT	单指令多线程	一个 warp 同发一条指令、各算各的数据
Register / RF	寄存器	每个线程私有、最快最小的存储
SMEM	共享内存	一个 CTA 内共享的片上快内存
GMEM / HBM	全局内存 / 显存	所有线程可见、又大又慢
TMEM	张量内存	Blackwell 给 Tensor Core 放累加器的专用内存
coalescing	访存合并	连续地址的访问合并成一次, 极大提速
bank conflict	bank 冲突	多线程挤同一存储体, 被迫排队
GEMM	通用矩阵乘	$C = A \times B$, 深度学习的算力主角
tile	分块 / 小方块	大矩阵切出来的固定大小小块
MMA	矩阵乘累加	Tensor Core 干的活: $D = A \cdot B + C$
accumulator	累加器	一直累加中间结果的那块数据
Tensor Core	张量核	专做矩阵乘的高速固定功能硬件
CUDA Core	CUDA 核	通用标量计算单元, 啥都能算但慢
TMA	张量内存加速器	异步成批搬 tile 的专用引擎
mbarrier	异步屏障	协调异步搬运/计算的”信号灯”
overlap / pipeline	重叠 / 流水线	让搬、算、写同时进行, 快的关键
epilogue	收尾阶段	算完后把结果写回去的那一段
Attention	注意力	两个矩阵乘夹一个 softmax
swizzle	换序摆放	打乱数据摆放以避开 bank 冲突
Blackwell	(架构代号)	NVIDIA 最新一代 GPU 架构
TIRx	(编程语言)	本书用来写 GPU 内核的 Python 内嵌 DSL

提示

提示: 更全的术语对照见术语对照表。

0.11 该怎么读这本书

- **完全没基础**: 先把这一章看完 (细节不懂没关系), 再从第 1 章顺着往下读。每读到一章卡壳, 回这里的速查表瞄一眼。
- **会一点 CUDA**: 可以快速扫过本章, 直接进第 1 章; 遇到 Blackwell 专属的 TMA / TMEM / warpgroup 再回头查。
- **只想看 GEMM / Attention 实战**: 也建议先看完本章和第 1、2 章, 否则第 11-14 章会很吃力。

完整的阅读路线见首页 README。

小结

这一章不指望你记住所有细节, 只要在脑子里搭起三根支柱就够了:

1. **谁干活**: 线程被编成 warp(32)→ warpgroup(128)→ CTA(车间)→ cluster, 跑在一个个 SM 上。
2. **东西放哪**: 寄存器 (私有最快)→ 共享内存 SMEM(车间共用)→ 全局内存 GMEM(又大又慢), 外加 Blackwell 的 TMEM。
3. **在算什么、怎么算快**: 核心是矩阵乘 GEMM 和注意力 Attention; 让 Tensor Core 不停转、让搬运和计算**重叠**, 就是全书的主线。

带着这三根支柱, 翻开第 1 章吧——你会发现那些黑话, 忽然就有画面了。

延伸阅读

- 下一章: 第 1 章 · GPU 执行模型 (把本章的”谁干活、东西放哪”讲得更细)
- 速查: 术语对照表

第 01 章 · GPU 执行模型

原文:GPU Execution Model —Modern GPU Programming for ML Sys

本章要点

本章要点 (TL;DR)

- GPU 把数以千计的线程组织成一个**嵌套层级**: 线程 (thread)→ 线程束 (warp)→ 线程束组 (warpgroup)→ CTA → 集群 (cluster)→ 网格 (grid)。每一层都是为了让”某个尺度上的协作”变得廉价而存在。
- 一个核函数 (kernel) 的工作流, 本质上是把数据在多个**内存空间**之间搬运并暂存的过程: 寄存器 (RF)、共享内存 (SMEM)、全局内存 (GMEM), 以及 Blackwell 新增的张量内存 (TMEM)。典型数据通路是 $GMEM \rightarrow SMEM \rightarrow () \rightarrow SMEM \rightarrow GMEM$ 。
- 算力被拆成两类引擎: 通用的 **CUDA 核 (CUDA core)** 负责索引、逐元素运算、归约、控制流; **张量核 (Tensor Core)** 是固定功能单元 (fixed-function), 以”块 (tile)”粒度一次性完成 $D = A \cdot B + C$, 吞吐量比 CUDA 核高一个数量级以上。
- 专门的数据搬运引擎 (如 TMA) 负责喂数据给计算引擎。计算引擎与搬运引擎是**相互独立、异步运行**的。
- 把以上拼成一条 GEMM 流水线后, 快慢的全部差距都在于 **重叠 (overlap)**: 让搬运、计算、收尾 (epilogue) 三个引擎**同时忙起来**, 而不是一个等一个。

前置知识

前置知识: 这是全书的基础章, 几乎不要求额外背景——只要你知道 GPU 大致是”成千上万个线程一起算”、矩阵乘法 (GEMM) 是 ML 的核心运算就够了。没把握的话, 先翻一下第 0 章 · 极简入门。本章会默认你已经认识这些词, 并从这里开始把它们一层层讲清楚。

这是全书第一部分「硬件、内存与执行模型」的头一章。它就想干一件事: 在你脑子里画出一张地图。

怎么画? 分两步。第一步, 把执行层级、内存空间、计算引擎、搬运引擎这几块拼图**一块一块**讲明白。第二步, 用一条 GEMM(通用矩阵乘) 流水线把它们**串成一条线**, 让你亲眼看见数据和执行是怎么在硬件里跑起来的。

关键

关键: 全书就围着一个核心观点转——后面几乎所有的优化技巧, 说到底都在回答同一个问题:”怎么把活儿合理地铺到这几块拼图上?” 所以别把这一章当成可看可不看的背景。它是后面每一章都要用的一套词汇和坐标系, 先打好这个底, 后面才走得顺。

先看一眼硬件长啥样。GPU 里最核心的计算单元叫**流式多处理器 / Streaming Multiprocessor**, 简称 SM。原文用一个交互式演示, 把 Blackwell 架构下一个 SM 内部的主要部件都摆了出来

来: 线程束、线程束组、共享内存、张量内存, 还有张量核和 TMA 引擎。我们用一张静态图表达同样的东西, 你就把它当成这一章的”地图总览”来看:

结构图

分组:

- **Blackwell SM (流式多处理器)**
- **执行资源:** 「warp $\times$ 4 = warpgroup」 「warp $\times$ 4 = warpgroup」 「...(多个常驻 CTA)...」
- **片上内存:** 「寄存器文件 RF (每线程)」 「共享内存 SMEM (每 CTA)」 「张量内存 TMEM (每 CTA)」
- **计算引擎:** 「CUDA 核 (通用 SIMT ALU)」 「Tensor Core (tcgen05, 块粒度 MMA)」
- **搬运引擎:** 「TMA (批量异步拷贝)」 「GMEM $\rightleftharpoons$ SMEM」

注意

注意: 一颗 GPU 是由一大堆这样的 SM 拼起来的, 这些 SM 共用一块设备级的全局内存 (HBM)。这一章我们盯着看的, 是**单个 SM 内部**, 以及**几个 SM 凑在一起协作** (也就是集群) 时的执行和内存模型。

一、执行层级: 为什么不是”一锅线程”

1.1 嵌套层级的设计动机

很多人第一反应是这样: GPU 不是有成千上万个线程嘛, 那它们大概就是一大锅平铺的线程吧? **还真不是**。GPU 把这些线程套了好几层, 组成了一个嵌套的层级。

干嘛这么费劲? 因为**线程之间的协作, 远近不一样**。举几个例子:

- 挨着的两个线程, 想交换一下寄存器里的值——这是很”近”的协作。
- 同一个块里的 128 个线程, 想合用一小块暂存数据——这是中等距离的协作。
- 趴在两个不同 SM 上的两个块, 想共享操作数 (operand, 就是参与运算的输入数据, 比如矩阵乘里的 A、B 矩阵)——这就”远”了。

要是只有一锅平铺的线程, 硬件就只能拿”最贵的那套”通信手段去对付所有情况, 哪怕很多协作其实近在眼前。GPU 反着来: **分层, 每一层专门把某个距离上的协作做得又快又省**。下面这张图, 就是 Blackwell 上整个层级的样子。

结构图

连接关系:

- Grid 网格 / (整个 kernel 的所有线程) $\rightarrow$ Cluster 集群 / (一组协作的 CTA, 可跨 SM)
- Cluster 集群 / (一组协作的 CTA, 可跨 SM) $\rightarrow$ CTA / Thread Block / (硬件调度的基本单位, 跑在单个 SM 上)
- CTA / Thread Block / (硬件调度的基本单位, 跑在单个 SM 上) $\rightarrow$ Warpgroup 线程束组 / (连续 4 个 warp = 128 线程)
- Warpgroup 线程束组 / (连续 4 个 warp = 128 线程) $\rightarrow$ Warp 线程束 / (32 个线程, SIMT 执行)

- Warp 线程束 / (32 个线程,SIMT 执行) → Thread 线程 / (标量执行单元, 有独立 PC 和寄存器)

1.2 逐层拆解

我们从最小的标量单位起步，一层一层往上爬。

层级	规模	关键特性
线程 (thread)	1	标量执行单元; 有自己独立的程序计数器 (PC) 和寄存器 (register, 线程私有、最快的小块存储); 在所属 warp 内用 lane ID(通道号, 即线程在 warp 里的 0-31 编号) 标识。
线程束 (warp)	32 线程	以 SIMT (single instruction, multiple threads, 单指令多线程) 方式执行。32 个 lane 一起发射同一条指令, 但各自持有独立寄存器, 且可被单独屏蔽 (mask)。
线程束组 (warpgroup)	4 个 warp = 128 线程	Hopper 引入, 作为发射 warp-group 级 MMA (MMA = 矩阵乘加, 一次算 $D = A \cdot B + C$; wgmma 即由 warpgroup 发射的那种) 的单位; 在 Blackwell 上又多了一个角色——它是访问 TMEM 的协作单位,128 个线程一起把一块 TMEM 搬进/搬出寄存器。
CTA (Cooperative Thread Array, 即 CUDA 里的 thread block)	多个 warp	硬件调度的 基本单位 ; 一个 CTA 跑在单个 SM 上, 并在 SM 内拥有一块私有的共享内存。多个 CTA 可同时常驻同一 SM, 此时它们瓜分该 SM 的共享内存容量。
集群 (cluster)	一组 CTA	一组协作的 CTA, 成员可以分布在 不同 SM 上。集群内的 CTA 之间可以互相同步, 并能读写彼此的共享内存——这就是 分布式共享内存 (DSMEM) 。

说到 SIMT, 有个地方特别容易绕进去, 这里专门拎出来讲。

关键

关键:warp 里 32 个 lane ”发射同一条指令”, 乍一听好像它们必须齐步走、只能走同一条分支——其实不用。每个 lane 有自己的寄存器, 还能被单独屏蔽掉, 所以同一个 warp 里, 不同 lane 完全可以走不同的分支。那代价是什么? 一旦分支发散, 硬件没法同时跑, 只能把这几条分支**轮着串行跑一遍**, 跑哪条就把不走这条路的 lane 先屏蔽掉。正是这种”允许同一 warp 内部分叉”的本事, 把 SIMT 和纯 SIMD 从根上区分开了。

1.3 ”作用域 (scope)”: 同一个操作由谁来发起

想读懂 Blackwell, 这里有个关键观点, 先记牢。

早期架构上事情很简单：一个核函数里的各种操作，基本都是”同一组线程”在发起。可到了 Blackwell，玩法变了——**关键操作不再由同一组线程发起了**。为什么？因为每种操作都有它**最顺手的粒度**：有的活儿一个线程做最合适，有的得拉上一整个 warpgroup 一起干：

操作	由谁执行（作用域/scope）
TMA 拷贝（TMA = 张量内存加速器，专门在 GMEM 与 SMEM 之间批量异步搬数据的硬件）	由 单个线程 发起，随后交给专门硬件去完成。
TMEM → 寄存器加载	按 warpgroup 分布：4 个 warp 协作，每个 warp 搬运 TMEM 块的一个切片。
tcgen05 MMA	由一个被选中（ elected ）的线程提交。
集群级 MMA(2-CTA)	一次横跨两个 CTA 。

”具体由哪组线程来执行某个操作”，书里给这件事起了个名字，叫这个操作的作用域 / **scope**。

关键

关键:scope 是全书反复出现的三大设计要素之一——作用域 / **scope**、布局 / **layout**、派发 / **dispatch**。往后每碰到一个 GPU 操作，你都该先问一句:”它的 scope 是啥？单个线程？一个 warp？一个 warpgroup？还是跨 CTA？”把这个想明白了，你才知道同步该怎么写、数据该怎么分。layout 和 dispatch 是另外两位，后面的章节会一点点补齐。

二、内存空间：容量与速度的取舍

2.1 为什么有这么多种内存

线程再快，手上没数据也只能干等着。所以光有计算单元不顶用，还得有地方搁数据。

这里有条躲不开的物理铁律：**没有哪种内存能既大又快**。容量和速度，你只能挑一头，或者在中间找个折中。既然不存在完美的内存,GPU 索性多备几种，每种在”大”和”快”这根取舍曲线上各占一个位置。说白了，一个核函数忙活的事，核心就是**让数据在这几种内存之间来回倒腾**。

内存空间	归属	角色	说明
全局内存 (GMEM)	设备级 (device-wide)	张量的持久存储	大容量 HBM，所有 SM 共享；离计算最远，延迟最高。
共享内存 (SMEM)	每 CTA(单个 SM 内)	块 (tile) 的暂存区	低延迟的” 便笺本 (scratchpad) ”；B200 上每 SM 最多 228 KB。(tile/分块：把大矩阵切成的小方块,GPU 一次处理一块。)
张量内存 (TMEM)	每 CTA	MMA 累加器存储	Blackwell 新增；由 tcgen05 使用。(累加器/accumulator：矩阵乘一块块累加结果时，暂存在中间和的那块地方。)
寄存器文件 (RF)	每线程	标量与每线程的块片段 (fragment，即一块大 tile 切给单个线程、暂存在它寄存器里的那一小片)	最快；存放收尾 (epilogue) 阶段和临时值。

这张表别当成四个互不相干的条目来看。你从上往下顺一遍就会发现，它其实勾出了一条路——数据从最远最慢的 GMEM 出发，一步步往计算单元身边凑。本书里几乎每个核函数的数据通路，都是这个走法：

结构图

要是这个 kernel 用了张量核，那 **TMEM** 就卡在这条路的中间——计算正进行的时候，累加器就停在它那儿。

2.2 重点理解 **TMEM:Blackwell** 独有的设计

四种内存里，前三种你多多少少都打过照面，就 **TMEM** 是 **Blackwell** 全新的玩意儿，以前压根没有它这一号。它的完整细节留到后面「Tensor Cores: tcgen05」那章再讲，但它当初为啥被造出来，现在就值得弄明白——因为这背后是一笔很经典的权衡。

先看它要解决啥问题：早期 GPU 把那个巨大的 MMA 累加器直接塞进寄存器。问题就出在这儿——累加器本身就大，寄存器又是稀缺到不行的资源。累加器占得越多，能匀给别处的寄存器就越少；寄存器一吃紧，能同时常驻的线程数（也就是占用率 / occupancy）就被压下来了。这成了一个甩了好多年都甩不掉的瓶颈。

Blackwell 的解法：把 tcgen05 算出来的累加器，从寄存器里挪出去，搬到一块叫 **TMEM** 的新地方。TMEM 是个 CTA 作用域的二维便笺本，规格是 **128 个 lane × 最多 512 个 32 位列**（每 CTA 一块），物理上就长在 SM 上。当然天下没白吃的午餐——代价是：核函数进收尾阶段之前，得自己动手把 **TMEM** 里的数据显式读回寄存器，系统不会自动给你搬。

结构图

分组：

- 早期架构：「寄存器文件 RF / 累加器（巨大）与其他值 / 抢占稀缺寄存器」
- **Blackwell**：「寄存器文件 RF / （收尾时显式读回）」**TMEM** ($128 \times \leq 512$ 列) / 累加器停在这 / （显式分配 / 释放）」

连接关系：

- **TMEM** ($128 \times \leq 512$ 列) / 累加器停在这 / （显式分配 / 释放） $\xrightarrow{\text{显式加载（按 warpgroup 分布）}}$ 寄存器文件 RF / （收尾时显式读回）

就这”多出来的一步”，牵出了两个会贯穿全书的后果：

1. **读 TMEM 得自己显式来，而且是按 warpgroup 分着读：**一个 warpgroup 的 4 个 warp 一起协作完成（还记得上一节那个 scope 吗？这就是活生生的例子）。
2. **TMEM 不像寄存器那样系统帮你自动打理：**你得亲手分配，用完亲手释放。

注意

注意：把累加器从寄存器挪到 TMEM，是笔很典型的买卖——花一点写代码的麻烦，换硬件资源彼此解耦。好处是寄存器再也不被那个大累加器占满了；代价是程序员得自己惦记 TMEM 的生死（分配、释放），还得记得那次显式读回。这种”拿复杂度换性能”的权衡，后面你会一遍

遍碰到。

2.3 跨集群的分布式共享内存 (DSMEM)

回想一下: 整个层级里, 就**集群这一层**, 成员能横跨好几个 SM。正是这个”够得着别的 SM” 的本事, 让它有了一项别的层级都没有的内存能力。

它解决啥问题? 一个 CTA 趴在单个 SM 上, 平时只能用自家这个 SM 的共享内存。可单个 CTA 的 SMEM 预算就那么一丁点, 处理大块数据时往往要么**得放下更多操作数**, 要么想把已经搬进来的数据多用几遍——这些事, 一个 CTA 光靠自己根本兜不住。

Hopper 的答案叫线程块集群 / thread block cluster: 把一组 CTA 拴在一块, 让它们协作得比平时更近一步。它们能一起同步, 还能读写对方的共享内存。这种”互相读写对方 SMEM” 的能力, 就叫**分布式共享内存 / DSMEM(Distributed Shared Memory)**。到了 Blackwell, 集群不但留了下来, 还更上一层楼, 添了**动态调度** (见后续「集群启动控制 / Cluster Launch Control」) 和 **2-CTA 协作式 MMA**。

具体怎么玩? 一个 CTA 可以直接指名道姓地去访问对端 (peer)CTA 的共享内存。一个线程能”点名”对端 SMEM 里的某个位置, 把一块 tile 从自己的 SMEM 整批拷过去, 等字节全都送到了, 再**升起一面完成屏障 / barrier**(barrier/mbarrier: 一种同步信号, 用来告诉别人”我这步干完了, 你可以接着干了”)招呼对方一声 (见后续「Async Coordination: mbarriers」)。第三部分那个 2-CTA 集群 GEMM, 就是靠这套机制在两个 CTA 之间共享操作数 tile, **全程不用把数据绕到全局内存再绕回来**——省了一大圈冤枉路。

原文用交互演示展示了 2-CTA 集群多出来的这一下”DSMEM 跳转”。我们用图说一遍同样的意思: 每个 CTA 各攥着 A、B 的一半, 再通过集群去读对端的那半 B(走 DSMEM), 最后两个 CTA 合力憋出一个 256×256 的输出 tile:

结构图

分组:

- **SM 0:** 「CTA 0 / 持有 $A_0 + B_0$ 」
- **SM 1:** 「CTA 1 / 持有 $A_1 + B_1$ 」

连接关系:

- CTA 0 / 持有 $A_0 + B_0$ $\xrightarrow{\text{DSMEM 读取对端 B}}$ CTA 1 / 持有 $A_1 + B_1$
- CTA 1 / 持有 $A_1 + B_1$ $\xrightarrow{\text{DSMEM 读取对端 B}}$ CTA 0 / 持有 $A_0 + B_0$
- CTA 0 / 持有 $A_0 + B_0 \rightarrow 256 \times 256$ 输出 tile / (由这一对 CTA 协作产出)
- CTA 1 / 持有 $A_1 + B_1 \rightarrow 256 \times 256$ 输出 tile / (由这一对 CTA 协作产出)

关键

关键:DSMEM 的价值, 一句话——它开了一条**绕开全局内存的近道**。以前两个块想共享数据, 只能各自跑一趟 GMEM 把它读出来 (流量白白翻倍, 延迟还高); 现在直接 SMEM 拷给 SMEM, 又快又省。后面要讲的 2-CTA 协作式 MMA 和 TMA 多播, 底下踩的都是它。

三、计算引擎: CUDA 核与张量核

线程, 加上它们辛辛苦苦搬来的数据, 最后都得在算术单元上”会合”, 真正开算。这里有个关键: 一个 SM 给你的不是一种引擎, 而是**两种压根不一样**的数学引擎。它俩各管一摊、互相搭台, 而这分工几乎把每个核函数该怎么写都定下来了。

引擎	类型	职责
CUDA 核 (CUDA core)	通用 SIMD ALU	跑标量/向量指令: 索引算术、逐元素运算、归约、控制流——也就是围绕矩阵重活的”胶水逻辑”。
张量核 (Tensor Core)	固定功能单元 (fixed-function)	以 块 (tile) 粒度做稠密矩阵的乘法, 一条指令算出 $D = A \cdot B + C$ 。

3.1 为什么这个分工如此重要

道理很直白: 张量核的算术吞吐量, 比 CUDA 核高出一大截, 在 FLOP/s 上差着 10 倍甚至更多。所以那些稠密线性代数 (GEMM、卷积、注意力 / attention——Transformer 里那个靠矩阵乘算 token 之间关联度的核心算子), 只有搬到张量核上跑, 才有戏逼近硬件的峰值。

关键

关键: 顺着这个差距, 能推出一条贯穿全书的”第一性原理”——**想拿性能, 基本就是一件事: 把张量核喂饱, 别让它停**。这话很硬: CUDA 核哪怕优化到头, 也补不回那 10 倍的吞吐窟窿。这也就解释了, 为啥后面那么多章节都在死磕同一个问题: 怎么把数据够快地递到张量核嘴边, 让它一刻都别闲着。

3.2 代际变化: 张量核怎么被编程、结果停在哪

一代 GPU 升到下一代, 张量核这东西本身一直都在。真正在变的是另外两件事: 你怎么给它编程, 还有它算出来的结果搁在哪儿。

- **Hopper:** 引入了异步的 warpgroup MMA ——`wgmma.mma_async`(由 warpgroup 发射, 见 §1.2)。
- **Blackwell:** 第五代张量核 `tcgen05`, 把累加器放进 **TMEM** 而非寄存器 (见 §2.2)。本书用专门一章「Tensor Cores: tcgen05」讲它。

3.3 集群对计算引擎的两项扩展

前面说集群把内存撑大了 (DSMEM)。其实它对计算引擎也动了两处手脚, 这两点在后面的 GEMM 章节里会一再冒头:

1. **2-CTA 协作式 MMA:** 让两个 CTA 各掏出自己那份 SMEM 操作数, 拼成一个更大的张量核 MMA tile 一块算。
 2. **TMA 多播 / multicast:** 让搬运引擎只加载一次, 就把同一块 GMEM tile 一股脑分发给好几个 CTA, 省掉”每个 CTA 各加载一遍”那份白白浪费的流量。
- 这两处, 底下都是踩着前面那个分布式共享内存 (DSMEM) 来的。

四、GEMM 数据流水线: 把拼图拼起来

到这儿, 几块拼图我们一块一块都摆出来了。现在终于能把它们拼到一起——拿一条最典型的 **GEMM(通用矩阵乘) 流水线**, 看看数据和执行是怎么配合着流起来的。原文用交互演示给了一条**三级 GEMM tile 流水线**, 你点一下某个动作 (比如 `tma load`), 它在硬件单元之间走过的那条数据路径就会被高亮出来。

4.1 单个 GEMM tile 的三个阶段

一块 GEMM tile 从头走到尾, 要过三关。下面这张表, 把每一关的 `scope`(谁来发起) 和数据往哪儿走都标清楚了:

阶段	scope(谁发起)	数据通路
① Load 加载	单个线程发起 TMA	GMEM --TMA--> SMEM(记录预期字节数, 字节到齐后屏障翻转)
② Compute 计算	一个被选中的线程发起 tcgen05 MMA	SMEM -- --> Tensor Core -- --> TMEM(算完后发信号给屏障)
③ Epilogue 收尾	整个 warpgroup 协作	TMEM -- --> -- dtype--> SMEM --TMA store--> GMEM

一关一关说:

- 1. **加载 / Load:** 用一次 **TMA 拷贝** (详见后续「Async Data Movement: TMA」), 把一块 A 或 B 操作数 tile 从 GMEM 搬进 SMEM。这趟拷贝由一个线程发起, 它还会先报个数: 这趟一共要到多少字节。然后字节一点点落地,TMA 引擎边搬边报进度, 直到说好的字节全到齐, 完成屏障才翻一下, 等于跟大家说一声”数据齐活了”。
- 2. **计算 / Compute:** 用一次 **tcgen05 MMA**(详见后续「Tensor Cores: tcgen05」), 从 SMEM 把操作数 tile 读出来, 把乘积累加进一块 **TMEM tile**。这一关由一个被选中 (**elected**) 的线程发起, 算完了给屏障递个信号。
- 3. **收尾 / Epilogue:** 整个 **warpgroup** 一起上, 把 TMEM 里的累加器读回寄存器, 把结果转成输出要的数据类型, 最后写回 GMEM——一般是先在 SMEM 里中转一下, 再发一次 TMA store。

注意

注意: 回头对一下 §1.2 那张 `scope` 表, 你会发现一个挺妙的巧合: 这三关刚好各用了一种不同的作用域——加载是**单个线程**发起, 计算是**单个被选中的线程**提交, 收尾是**整个 warpgroup**一起干。前面那句”Blackwell 的关键操作不再由同一组线程发起”, 这就是它最实打实的例子。

4.2 真正的胜负手: 重叠 (overlap)

照上面这个写法, 三关看着像是**规规矩矩排队、一个接一个跑下来**的。可这里藏着全章最要紧的一句话: **慢 kernel 和快 kernel 的差距, 全在”重叠 / overlap”这三个字上**。

- **朴素 (naive)kernel:** 真就老老实实按顺序来——加载, 等; 计算, 等; 存储。坏就坏在, 每个引擎等上一个干完的那段时间里, 自己都在**干瞪眼、白白闲着**。

- 快速 kernel: 把这三步流水线 (pipeline) 起来——张量核正算第 k 块的同时,TMA 引擎已经在取第 $k+1$ 块了,收尾那头还在收拾 (drain) 第 $k-1$ 块。这么一来,同一时刻三个引擎全都在忙,谁都没闲下来。

朴素 (串行, 引擎大把闲着) ——横轴是时间 ($t_0 \rightarrow t_9$), 每个时刻只有一个引擎在干活, 剩下的都在等:

引擎	t0	t1	t2	t3	t4	t5	t6	t7	t8
TMA	[load k]					[load k+1]			
Tensor		[mma k]					[mma k+1]		
Epilog			[ep k]					[ep k+1]	

流水线 (重叠, 三引擎一起忙) ——同一个时刻三个引擎都在干活:

引擎	t0	t1	t2	t3	t4
TMA	[load k]	[load k+1]	[load k+2]	[load k+3]	...
Tensor		[mma k-1]	[mma k]	[mma k+1]	...
Epilog			[ep k-2]	[ep k-1]	[ep k]

关键

关键: 三个异步引擎想重叠着跑, 就得有人盯着, 保证它们能安全地把活儿交给下一个——这正是后续「Async Coordination: mbarriers」要讲的屏障 (barrier) 加相位 (phase) 模型。第三部分那个“GEMM 阶梯 (GEMM ladder)”, 就是一级一级地把朴素 kernel 改造成深度流水线 kernel, 而这整套改造踩的地基, 就是 mbarrier/phase 模型。这也是本书为啥要专门拨一章讲异步协调——没有它, 重叠根本无从谈起。

小结

这一章把读懂整本书要用的坐标系搭好了。要是只让你记三句话, 就记这三句:

- 执行是分层的。线程 $\rightarrow$ warp $\rightarrow$ warpgroup $\rightarrow$ CTA $\rightarrow$ 集群 $\rightarrow$ 网格, 每一层都把某个距离上的协作变便宜。Blackwell 带来的关键转变是: 不同操作各有各的“作用域 / scope”——TMA 由单线程发起,tcgen05 MMA 由单个被选中的线程提交,TMEM 读回和 epilogue 按 warpgroup 分着干, 集群 MMA 跨两个 CTA。scope 是本书三大设计要素 (scope / layout / dispatch) 里打头的那个。
- 数据是分层暂存的。GMEM(大而慢) $\rightarrow$ SMEM(快便笺本) $\rightarrow$ 寄存器 (最快); 用张量核的 kernel 里,TMEM 还夹在中间替你拿着累加器。TMEM 是 Blackwell 独有的设计, 它拿“自己分配、自己释放、还得显式按 warpgroup 读回”这点麻烦, 换来把那个大累加器从稀缺的寄存器里解放出来。再往上, 集群这一层又解锁了 DSMEM, 让 CTA 之间能绕开 GMEM 直接共享数据。
- 算力被拆成几个各干各的引擎, 胜负全押在重叠上。CUDA 核管胶水逻辑, 张量核以 10 倍以上的吞吐扛矩阵重活,TMA 这类搬运引擎专门负责喂数据。把它们拼成 GEMM 的 load $\rightarrow$ MMA $\rightarrow$ epilogue 三级流水线之后, 真正的性能秘诀就一句: 让搬运、计算、收尾这三个异步引擎一起忙起来——而它们之间能安全交接, 靠的就是 mbarrier/phase 模型。

一句话收尾: 本书后面几乎所有的优化, 翻来覆去都是同一件事的不同版本——怎么把活儿铺到这几块拼图上, 再让各个引擎重叠着一起跑。

接下来推荐读的章节 (都是本书后面的内容):

- **Tensor Cores: tcgen05** ——详解 tcgen05 计算与张量内存 (TMEM)。
- **Async Data Movement: TMA** ——讲解基于 TMA 的异步数据搬运。
- **Async Coordination: mbarriers** ——介绍协调这些引擎的 mbarrier 与相位 (phase) 模型。

延伸阅读

- 原文 (英文):GPU Execution Model —Modern GPU Programming for ML

建议: 原文里有好几个交互式演示 (Blackwell SM 全貌、执行层级逐层高亮、2-CTA 集群的 DSMEM 跳转、GEMM 三级流水线的数据路径追踪)。这份笔记用静态图把同样的内容画了出来, 但要说”点一下某个动作, 亲眼看它在硬件单元之间怎么流”那种直观劲儿, 静态图到底差点意思。想找那个感觉, 建议配着原书的交互演示一起看。

术语对照

中文	English / 缩写	含义速记
线程	thread	标量执行单元, 有独立 PC 与寄存器
线程束	warp	32 个线程, SIMT 执行
线程束组	warpgroup	4 个 warp = 128 线程; 发射 wgmma、访问 TMEM 的协作单位
协作线程阵列	CTA / thread block	硬件调度基本单位, 跑在单个 SM 上
集群	cluster	一组可跨 SM 协作的 CTA
网格	grid	一个 kernel 的全部线程
流式多处理器	SM	GPU 的基本计算核心模块
单指令多线程	SIMT	warp 内同发一条指令、各 lane 可独立屏蔽
全局内存	GMEM	设备级大容量 HBM, 所有 SM 共享
共享内存	SMEM	每 CTA 的低延迟片上便笺本 (B200 <=228 KB/SM)
张量内存	TMEM	Blackwell 新增, 每 CTA 的 128x<=512 列累加器便笺本
寄存器文件	RF	每线程最快存储, 放标量与块片段
分布式共享内存	DSMEM	集群内 CTA 互相读写对端 SMEM 的能力
CUDA 核	CUDA core	通用 SIMT ALU, 做胶水逻辑
张量核	Tensor Core	固定功能单元, 块粒度 $D=A \cdot B+C$
张量内存加速器	TMA	异步批量数据搬运引擎 (GMEM $\rightleftharpoons$ SMEM)
矩阵乘加	MMA	matrix multiply-accumulate
通用矩阵乘	GEMM	稠密矩阵乘法
收尾阶段	epilogue	把累加器转 dtype 并写回的尾部阶段
重叠	overlap	让多个异步引擎同时忙起来

屏障	barrier / mbarrier	协调异步引擎交接工作的同步原语
作用域	scope	执行某操作的那组线程

第 02 章 · 什么决定 Kernel 的速度

原文:What Makes a Kernel Fast

本章要点

本章要点 (TL;DR)

- **屋顶线模型 (roofline model)** 给一个核函数 (kernel) 设定了**性能天花板**, 这个天花板要么由**内存带宽 (memory bandwidth)** 决定, 要么由**算力吞吐 (compute throughput)** 决定——取两者中更慢的那条路。
- 决定“撞上哪个天花板”的, 是**算术强度 (arithmetic intensity, AI)**: 每搬运一字节数据, 能做多少有用的算术运算 (FLOP/byte)。
- **算术强度低** → **内存受限 (memory-bound)**。出路是: 少搬字节、多复用数据、做算子融合 (fusion)、用更小的 dtype——本质都是“把 kernel 往屋顶线右边推”。
- **算术强度高** → 可以做到**算力受限 (compute-bound)**。此时核心任务变成“别让张量核 (Tensor Core, 专做矩阵乘加的硬件单元, 见第 0 章) 闲着”。
- 在现代 GPU kernel 里, 最大的杠杆是 **重叠 (overlap)**: 只要依赖图允许, 就让 TMA(硬件托管的批量数据搬运引擎)、张量核、收尾 (epilogue, 把结果转格式并写回的尾部阶段)、写回这几件事**同时在跑**。
- 一个性能数字脱离了天花板就没有意义。先**估算算术强度、定位天花板、判断是内存受限还是算力受限**, 再去**优化真正卡脖子的那个资源**——这是贯穿全书的优化 workflow。

前置知识

前置知识: 读这一章前, 最好先懂**内存层级 (寄存器 / 共享内存 / 全局内存)**、**Tensor Core(张量核)**、**重叠 (overlap)**, 以及第 01 章讲的**执行层级**。没把握的话, 先翻一下第 0 章·极简入门, 以及第 01 章。本章会默认你已经认识这些词。

第 01 章带你认了认硬件: 有哪几层执行单元、有哪些内存、有哪些专门算数和搬数据的引擎, 等于先画了张地图。这一章我们聊个更实在的问题——一个 **kernel 到底快不快, 是什么说了算?**

先说结论: 不是什么玄学技巧。它是一套能让你“拿数字算账”的框架, 叫**屋顶线模型**。把它学会, 你还没动手写代码, 就能先判断出劲该往哪儿使。

关键

关键: 一个性能数字, 光盯着它本身看, 啥也说明不了。你得**拿它跟天花板比**, 它才有意义。举个例子你就懂了。330 TFLOP/s, 听着是不是挺唬人? 可假设这颗 GPU 能稳稳跑到 2 PFLOP/s 左右 (稠密 fp16/bf16 张量核), 那这数字**连六分之一的算力都没摸到**, 大半个芯片都在那儿干瞪眼呢。说白了, 没有天花板当参照, 你压根分不清: 这 kernel 是已经榨到硬件极限了, 还是还有一大把性能躺着没人用。

那这天花板是从哪儿冒出来的？这正是屋顶线模型的活儿。它的思路特简单：一个 kernel 干的事，无非两类——搬字节和做算术。然后看是谁在拖后腿。要是数据喂不上来，那带宽就是你的上限；要是数据反复用、算术活儿也够重，这才轮到算力吞吐来当上限。

注意

注意（本章数字约定）：跟第 01 章一样，我们还是拿 **NVIDIA B200** 当例子，而且天花板都取个整方便推算：稠密 fp16/bf16 张量核吞吐算 **2 PFLOP/s** 上下，HBM3e 带宽算 **8 TB/s** 上下。这些数会随着型号、时钟、功耗墙、怎么测来回变，所以你就把它们当成**数量级**（order-of-magnitude）的参考，别当成数据手册上一分不差的常数。

一、屋顶线模型：两条路取更慢的那条

前面说了，每个 kernel 都干两件事：搬数据、做算术。屋顶线模型的精髓就一句话——**这两条路谁慢，谁就是你的上限**。

- **算力天花板 / compute ceiling**：就是硬件算术能力的顶。比如 B200 上跑张量核 GEMM(通用矩阵乘，即稠密大矩阵相乘)，这个顶就是张量核的吞吐；而要是个逐元素 (elementwise) 的小 kernel，这个顶可能是 CUDA 核 (通用算术核，逐个线程做标量运算) 的吞吐，也可能是别的某个功能单元。
- **内存天花板 / memory ceiling**：它等于**带宽 × 算术强度**。每搬一字节只算那么一丁点，内存带宽就成了瓶颈；反过来，一字节能撑起一大堆运算，那内存基本就不卡你了。

把这两条凑一块儿，就是屋顶线那个基本不等式：

$$\text{FLOP/s} \leq \min(\text{FLOP/s}, \text{带宽} \times \text{算术强度})$$

其中算术强度定义为：

$$\text{算术强度} = \frac{\text{FLOP}}{\text{字节}} \quad (\text{单位: FLOP/byte})$$

注意

注意：一提算术强度，你先得说清楚是按哪一级内存算的。按 HBM 画屋顶线，字节就指 HBM 字节；按 L2 算，就是 L2 字节；按 SMEM 算，就是共享内存字节。这一章如果不特别说明，默认都在聊 **HBM 屋顶线**。

1.1 读懂这张图：斜屋顶、平屋顶与脊点

先教你怎么看这张图。**横轴是算术强度 (FLOP/byte)，纵轴是你跑到的性能**。图上有两条”屋顶”：

$$\begin{aligned} \text{斜屋顶: } & \text{性能} = \text{带宽} \times \text{算术强度} \\ \text{平屋顶: } & \text{性能} = \text{算力天花板} \end{aligned}$$

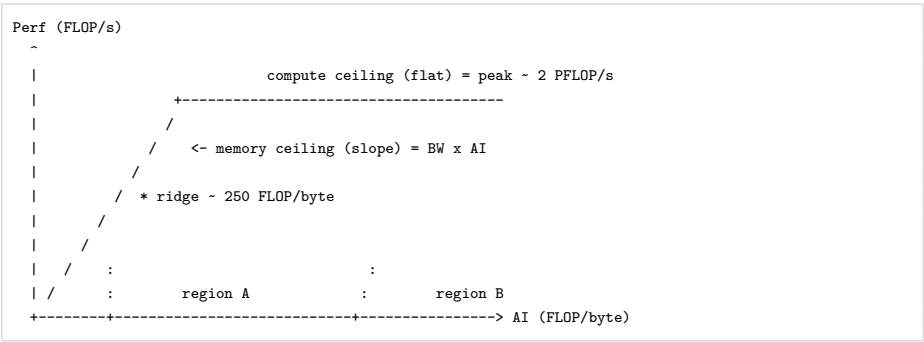
这两线会撞到一起，撞上的那个点叫**脊点 / ridge point**：

= FLOP/s /

脊点就是那个临界值——算术强度到了这儿，你正好从内存受限切到算力受限。拿 B200 取整后的数字代进去算算：

~ = 2000 TFLOP/s / 8 TB/s
~ = 250 FLOP/byte

把斜屋顶、平屋顶、脊点这三样画一块儿，差不多就是下面这样：



图注：横轴是**算术强度 AI**(FLOP/byte)，纵轴是**可达性能** (FLOP/s)。

标记	含义
memory ceiling(斜线)	内存屋顶 = 带宽 × 算术强度
compute ceiling(平线)	算力屋顶 = 峰值 FLOP/s ≈ 2 PFLOP/s
ridge(*)	脊点 ≈ 250 FLOP/byte, 两条屋顶相交处
region A(脊点左侧)	内存受限区: 被带宽卡住
region B(脊点右侧)	算力受限区: 可被算力卡住

关键

关键：屋顶线真正值钱的，不是这张图长啥样，而是它一眼就能告诉你——到底是哪个资源在卡你。

- 算术强度**落在脊点左边** → **内存受限**：这会儿你再怎么打磨算术指令也是白搭，因为字节根本喂不上来。
- 算术强度**落在脊点右边** → **有机会做到算力受限**：这会儿抠那几个零碎字节也没用，你该干的是把算力单元喂饱，把性能顶到平屋顶上去。

一句话：内存受限的 kernel，不会因为你”数学指令写得漂亮”就变快；算力受限的 kernel，也不会因为你”省了几个不相干的字节”就变快。**所以优化的第一步永远是：先看清楚自己落在脊点哪一边。**

二、常见 workload 的算术强度

这里有个特别重要的发现：算术强度多半是算法自己定的，跟你怎么写实现关系不大。这有啥好处？好处大了——你连一行 kernel 都还没写，就能先把它算术强度估个八九不离十，提前知道劲该往哪儿使。下面挑三类最典型的 workload 看看。

Workload	算术强度	在脊点哪一侧	优化目标
逐元素 / 归约 (GELU、RMSNorm)	低 (常数级)	远在左侧，内存受限	逼近内存带宽屋顶
GEMM(矩阵乘)	高 (随规模增长)	右侧，算力受限	喂饱张量核 + 重叠
注意力 (Attention)	居中 (取决于实现)	看情况	先抬高 AI，再做调度

2.1 逐元素与归约：天生内存受限

逐元素 kernel(像 GELU) 和归约式 kernel(像 RMSNorm) 有个通病：它们要读写老大多一个张量，可摊到每个元素上，就做那么两三次 FLOP。

搬一大堆、算没几下，算术强度能不低吗？所以它们远远地趴在脊点左边。这类 kernel 你别惦记着去够张量核屋顶，踏踏实实去贴内存带宽屋顶就对了。换个说法，你要问的全是很“机械”的访存问题：

```
-          (coalesced)?
-          ?
-          (producer)      (consumer)      ?
- dtype      ?
-          TMA      ?
```

关键

关键：一个 kernel 要是既没法复用、又没法融合，那内存屋顶就是它实打实的头顶，谁也别想往上抬。这时候你的目标不是去“超过”屋顶——超不过去——而是把它“贴满”(怎么贴，第四节细说)。

2.2 GEMM：算术强度随规模增长

GEMM 是另一头的极端。它的算术强度问题越大、它越高。为啥？因为一块 tile(大矩阵切出来的小方块) 搬进来，它能被反复拿去做好多次乘加 (MMA，矩阵乘加运算)——加载一回，用上好多次，FLOP/byte 可不就上去了。

拿个方阵 fp16 矩阵乘来说 ($M = N = K$)，理想情况下算术强度差不多是：

AI $\approx$ $2N^3 / (3 \cdot 2N^2)$
 $= N / 3$ FLOP/byte

注意

注意 (这个估算有几个前提)：它假设 A、B 各只读一遍、C 只写一遍、beta = 0、片上复用一点不浪费，而且没有任何多余的元数据、padding 或者冗余流量。实际 kernel 搬的数据比这个理想值多，不过拿来“估个量级”还是够使的。

公式别背，搞懂它咋来的就行。先看分母 $3 \cdot 2N^2$ ：三块矩阵 (A、B、C)，每块 N^2 个元素，每个元素 2 字节 (fp16)。再看分子 $2N^3$ ：这是整个矩阵乘的浮点运算总数——每个输出元素得做 N 次乘加，一次乘加算 2 个 FLOP。

代进去 $N = 4096$ 试试：

```
AI == 4096 / 3 == 1365 FLOP/byte
```

1365 早把 B200 的脊点 (约 250 FLOP/byte) 甩到左边老远了。所以结论很清楚：大 GEMM 在 HBM 屋顶线底下，是算力受限的。既然如此，它要优化就别去省 HBM 流量了，该干的是：把张量核使起来、喂饱它，再让搬数据和算数据重叠着跑——这么干才摸得到算力屋顶。

关键

关键：这正好解释了一个看着挺拧巴的现象——朴素 GEMM 算术强度明明很高，却可能慢得要命。咋回事？因为算法准许你跑得快，不等于你的实现真的跑得快。代码写得糙一点，张量核照样能大段大段地闲着没事干。说到底，算术强度只是给你画了个天花板，够不够得着，那是另一码事 (看第三节那个“优化阶梯”)。

2.3 注意力：介于两者之间的混合问题

注意力 (Attention, Transformer 里让每个 token 看其他 token 的核心算子) 就卡在前面这俩中间。它的算术强度不是个定数，好几样东西都能让它变：序列长度、头维度 (head dimension)、tiling、掩码 (masking)，还有最要命的一样——你有没有把中间张量物化 (materialize) 到内存里去。

标准注意力最疼的地方，就在分数矩阵 / score matrix(query 和 key 两两点积得到的中间大矩阵)。kernel 要是把分数矩阵先写进 HBM、回头再读回来，那等于让一个巨大的中间张量在内存里白白跑了个来回。Flash Attention(书里拿 Flash Attention 4 举例) 的高明就在这儿：它把要用的 tile 摞在片上，这趟 HBM 往返直接省了，算术强度“噌”一下就上去了。

关键

关键：所以优化注意力是个“两步走”的活——前半段是屋顶线问题，后半段是调度问题。第一步先改算法，把流向 HBM 的字节砍下来 (也就是把 AI 抬上去)；第二步再调度 kernel，让剩下的搬运和计算重叠起来。你品一品就会发现，这不就是把 GEMM 那套打法直接搬过来用嘛。

三、算术强度低的时候怎么办

一个 kernel 落在脊点左边，它就是内存受限的。这会儿张量核、CUDA 核很可能大段大段地干等着，因为卡你脖子的是字节，不是算术指令。咋办？两条路。

3.1 第一条路：抬高算术强度 (高杠杆)

这条路更划算，因为它能把 kernel 往算力受限那边推。具体有三招。

第 1 招：算子融合 (fusion)——这招最关键。算术强度低，最常见的病根就是“一个中间张量刚写进 HBM，下一个算子转头又把它读回来”。怎么治？把上游的 producer 和下游的 consumer 塞进

同一个 **kernel**。这么一来，中间张量就一直待在寄存器 (register, 线程私有的最快存储)、SMEM(共享内存, 一个 block 内线程共用的片上内存) 或者 TMEM(Blackwell 上专给张量核累加器用的片上内存) 里，**那趟 HBM 往返直接没了**。几个典型例子：

```
- GEMM +          epilogue
- normalization
-                ( Flash Attention)
```

第 2 招：为复用而分块 (blocking for reuse)。道理简单得很：一块 tile 搬进来一次，趁它还没被踢出去，能多用几次就多用几次，这样每个字节就摊上了更多算术活儿。GEMM 算术强度高，靠的就是这个。别的 workload 只要也有”同一块 tile 反复用”的机会，这招照搬就行。

第 3 招：让每个数变小 (换更小的 dtype)。从 fp32 往下降到 fp16、fp8 乃至 fp4，一箭双雕：流量小了，FLOP/byte 还顺带抬高了。

注意

注意：实际省下来的，通常没有 dtype 比例算出来的那么多。因为低精度格式常常得拖着元数据、缩放因子 (scale factor)，要么就得多走一道转换——**block-scaled fp8/fp4** 就是这样。话虽如此，换更小的 dtype 还是把 kernel ”往屋顶线右边推”最干脆的办法之一。

3.2 第二条路：认了，就把内存屋顶贴满

不是每个 kernel 都救得回来。有些就是**没东西可融、也没复用可榨**：一次纯拷贝、一个简单的逐元素操作、对着一个大张量扫一遍做归约——它们生下来就是内存受限，你改不了。

关键

关键：碰上这种，别惦记着去”打败”屋顶，想办法把它**贴满 (saturate)** 就行。怎么贴？

```
-
-
-      (coalesced)
-      tile      TMA
-
-              (in flight),
-      ,         dtype
```

关键

关键：一个内存受限的 kernel 一旦把内存屋顶贴满了，你再去抠算力那边的优化就是白费功夫。这时候想让它更快，只剩一条路：**回头改算法，让它少搬点字节**。

四、优化阶梯：从”可能”到”做到”

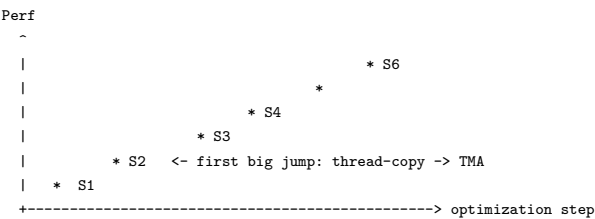
屋顶线只跟你说什么有可能做到，它可不管够到那个极限有多费劲。这两码事你得分清楚。

打个比方，一个大 fp16 GEMM 理论上是算力受限的。可这话的意思无非是”HBM 屋顶不是主要拦路虎”，绝不是说你随手写个实现就能摸到张量核屋顶。从”理论上能到”到”实际真跑到”，中间隔着一道坎；想填平它，你得用对指令、摆对布局 (layout)、做对暂存 (staging)，还得把同步和调度都安排明白。

关键

关键: 这道坎, 本书第三部分在 B200 上拆成了一级一级的台阶 (就是后面「用 Warp 专精与集群扩展 GEMM」那一章)。这里有个点你留意: 每往上迈一级, 底层算法压根没动, 变的只是 tile 怎么算、怎么调度。

GEMM 优化阶梯: 性能一级一级往上走, 可**算法自始至终没变**, 变的只是 tile 怎么算、怎么调度。



台阶	名称	说明
S1	同步 tiled 基线	thread-copy 分块路径, 作为起点
S2	TMA 路径	第一次能测出来的大跳:thread-copy → TMA
S3	Warp 专精	分离 producer / consumer 角色
S4	CTA 集群	更大的协作 tile 与多播
S6	多消费者执行	顶部: 多个消费者同时取用结果

- 第一次能测出明显提速的大跳, 是从 **thread-copy** 分块路径换到 **TMA** 路径。TMA 干的是这事儿: 那种”规则的 GMEM → SMEM tile 搬运”, 本来压在 CTA 线程身上, 现在交给硬件托管的批量拷贝去喂张量核。线程解放了, 搬运还更快了。
- 再往后的提速, 基本全靠重叠和调度:TMA 提前把后面要用的 tile 预取进共享内存;tcgen05.mma 异步地算;epilogue 这边同时排干 (drain) 上一批结果;最后再用 **软件流水线 (software pipelining)** 加 **warp 专精 (warp specialization)** 把这几段编排到一块儿, 让好几个硬件引擎一起开工。

注意

注意: 别想当然觉得”每爬一级都得比上一级快”。像 warp 专精这种步子, 可能会先把资源砸进一个”眼下并不马上提速”的结构里。可只要它能**解锁那些更简单结构压根做不到的重叠**, 这一步就走对了——它是在给后面铺路呢。

五、重叠是最大的杠杆

假设一个 GEMM 已经是算力受限、张量核也用上了, 那还差点啥? **多半就差在”空档”上**——总有些引擎在那儿干等, 时间白白溜走了。

来看看一个朴素 kernel 是咋跑的:

```
load tile k
compute tile k
```

```
store tile k
load  tile k+1
compute tile k+1
store tile k+1
```

这种一步压一步的排法，会把硬件晾在那儿大把空转：加载在跑的时候，张量核干瞪眼；张量核在算的时候，拷贝引擎没事干；轮到写回排干，前面俩可能又都在等。说白了，任何一个时刻，基本就一个引擎在干活。

流水线化的 kernel 就两样了，它让互不相干的几个阶段同时开跑：

```
load  tile k+1    <-
compute tile k     <-
store  tile k-1    <-
```

朴素（串行，引擎大量闲置）——任一时刻只有一个引擎在干活：

引擎	t0	t1	t2	t3	t4	t5
TMA	[load k]			[load k+1]		
Tensor		[mma k]			[mma k+1]	
Store			[st k]			[st k+1]

流水线（重叠，多引擎齐忙）——同一时刻多个引擎都在干活：

引擎	t0	t1	t2	t3
TMA	[load k+1]	[load k+2]	[load k+3]	...
Tensor		[mma k]	[mma k+1]	...
Store			[st k-1]	[st k] ...

这就是本书后面 Blackwell kernel 结构的主心骨：**TMA** 负责异步搬数据，**tcgen05.mma** 负责异步算张量核，**epilogue** 和写回管输出这头，再拿 **mbarrier**（异步屏障对象，用来等”某件事干完了”再放行）这么个对象把各个阶段拴到一起，让每个消费者只在它真要用数据的那一刻，才去等一下。

关键

关键：这里你得想明白一件事——重点不是把依赖消掉，而是绕着依赖来排活儿。依赖是消不掉的：tile k 的 MMA 非得等 tile k 加载完才能动；tile k 的 epilogue 非得等 MMA 算完才能读累加器。可妙就妙在——tile k+1 的加载往往可以趁着 tile k 的 MMA 还没算完就先跑起来；tile k-1 的写回往往可以同时在那儿排干。正是因为有这些”能并行的空档”，后面好几章才专门来讲各种异步机制：

机制	作用	对应后续章节
TMA	GMEM → SMEM 的搬运	Async Data Movement: TMA
mbarrier	标记加载完成、做资源交接	Async Coordination: mbarriers
tcgen05	异步张量核计算	Special Memory: TMEM
TMEM	存放长生命周期的累加器	Special Memory: TMEM
warp 专精	分离 producer / consumer 角色	用 Warp 专精与集群扩展 GEMM
集群 (cluster)	更大的协作 tile 与多播 (multicast)	同上

这些机制长得各不一样，可它们都在伺候同一个调度目标：让有用的活儿同时跑在好几条硬件路径上，而不是挤在一条上。

六、占用率与资源压力

关键

关键: 藏延迟的法子不止”重叠”这一招。还有个更老、更通用的机制,叫 占用率 (occupancy)。

先说占用率是个啥: 就是同一时间常驻在一个 SM(流式多处理器,GPU 上独立调度执行的计算核心,见第 0 章) 上的活儿有多少。它咋帮你藏延迟呢? 是这样: 一个 warp(线程束,32 个线程的小班,见第 0 章) 卡住了 (stall), 调度器立马切到另一个已经准备好的 warp 去跑。只要你手头总攒着一池子”招手就来”的独立 warp, 延迟就这么神不知鬼不觉地盖过去了。

那占用率被啥卡着? 被单个 SM 的资源卡着, 主要这四样: 寄存器、共享内存、warp 槽位、CTA 槽位。一个 kernel 要是每个线程吃一大堆寄存器, 或者每个 CTA(协作线程阵列, 即一个 block, 会被整体调度到同一个 SM 上) 啃掉一大块共享内存, 那它占用率一般就高不了——因为一个 SM 上根本塞不下几个 CTA、几个 warp。

这里有个反常识的地方: 不少现代张量核 kernel 是成心”大手大脚”用资源、成心把占用率压低的。多级共享内存流水线把 SMEM 吃掉, 大块的寄存器片段 (fragment, 张量核运算时操作数在各线程寄存器里的分片) 把寄存器吃掉,TMEM 分配把张量内存吃掉,warp 专精更狠, 一整个 warp 一整个 warp 地划走, 专门给 producer 或 consumer 角色用。图啥呢?

注意

注意 (这是成心做的取舍): 因为这些 kernel 换了套藏延迟的玩法。它们不再靠”一大堆不相干的 warp 常驻”来藏, 而是改成在少数几个常驻 CTA 内部, 用明摆着的重叠去藏。所以你会看到个挺有意思的结果——一个占用率很低的 kernel, 只要它的流水线能让 TMA、张量核、写回都没闲着, 照样能跑得飞快。

这两套玩法没有谁绝对更高明: 有些 kernel 因为访存不规则、或者腾不出地方做显式重叠, 就只能吃高占用率这碗饭; 另一些则非得靠深度暂存加专精不可, 因为只有这样才能喂得饱张量核。所以你真正该问的, 从来不是”占用率高不高”, 而是——正在干活的那些硬件单元, 到底喂饱了没有?

小结

这一章说来说去就一句话: 先看病, 再下药。后面整本书都会一遍遍绕回这三个诊断问题:

- kernel ?

- ?

- ?

情况不一样, 答案也就很清楚了:

Kernel 类型	诊断	优化方向
内存受限	落在脊点左侧, 被带宽卡住	减少字节 + 提升带宽利用: 融合、合并、向量化、TMA、更小 dtype
算力受限 GEMM	落在脊点右侧	先上张量核, 再做重叠: 暂存操作数、发异步 MMA、填满流水线、不卡住计算路径地排干结果

Flash Attention	先抬 AI 再调度	先把分数 / 概率 tile 留在片上抬高 AI; 之后复用 GEMM 那套重叠工具 (tiled 搬运、SMEM 暂存、异步计算、资源交接)
-----------------	-----------	---

这么一来, 就有了一套能照着做的优化流程:

估算术强度 → 定位屋顶 → 判断是内存受限还是算力受限 → 只去优化那个真正设定天花板的资源。

少了这一步, 优化 kernel 就成了拍脑袋瞎猜; 有了它, 你每动一处都说得清为什么——要是把算术强度抬上去了, 要是把内存路径往带宽峰值推近了, 要是把算力屋顶底下那点空闲时间给砍掉了。

延伸阅读

- 原文 (英文):[What Makes a Kernel Fast —Modern GPU Programming for ML](#) Sys
- 第 01 章「GPU 执行模型」: 这一章接着用了那里定下的 B200 取整天花板 (≈ 2 PFLOP/s、 ≈ 8 TB/s), 还有执行层级、内存空间、计算和搬运引擎这些硬件底子。

建议: 原文配了一张 B200 屋顶线图 (把内存屋顶、算力屋顶、脊点都标出来了), 还有一张 GEMM 优化历程图 (从同步 tiled 基线, 一路经过 TMA、warp 专精、CTA 集群, 走到多消费者执行的实测点)。这份笔记用静态示意图做了对应的表达, 坐标轴比例只是示意, 真要看准实测数值, 请以原书图表为准。

术语对照

中文	English / 缩写	含义速记
屋顶线模型	roofline model	用”内存 / 算力取更慢”给 kernel 设上限
算术强度	arithmetic intensity (AI)	每搬一字节做多少 FLOP(FLOP/byte)
内存受限	memory-bound	落在脊点左侧, 被带宽卡住
算力受限	compute-bound	落在脊点右侧, 被算力卡住
脊点	ridge point	两条屋顶相交处 = 峰值 FLOP/s ÷ 带宽
内存带宽	memory bandwidth	单位时间能搬多少字节 (B200 HBM3e ≈ 8 TB/s)
算力吞吐	compute throughput	单位时间能做多少 FLOP(B200 fp16 ≈ 2 PFLOP/s)
算子融合	fusion	把 producer/consumer 合进一个 kernel, 消除 HBM 往返
为复用而分块	blocking for reuse	一块 tile 加载一次、复用多次
物化	materialize	把中间张量真正写进内存
分数矩阵	score matrix	注意力里的中间矩阵, 易成 HBM 流量大户

闪存注意力	Flash Attention	把 tile 留片上、避免 HBM 往返以抬高 AI
合并访存	coalesced access	相邻线程访问相邻地址，带宽利用高
软件流水线	software pipelining	用代码把多阶段排成重叠流水线
Warp 专精	warp specialization	把 warp 分工成 producer / consumer 角色
重叠	overlap	让多个异步引擎同时忙起来
占用率	occupancy	一个 SM 上常驻的工作量，靠多 warp 藏延迟
通用矩阵乘	GEMM	稠密矩阵乘法
矩阵乘加	MMA	matrix multiply-accumulate
收尾阶段	epilogue	把累加器转 dtype 并写回的尾部阶段

第 03 章 · 数据布局与其记号

原文:[Data Layout and Its Notation](#)

本章要点

本章要点 (TL;DR)

- **数据布局 (data layout / 数据布局)** 干的事, 就是告诉你某个逻辑下标对应的数, 到底躺在物理上的哪个位置。别小看它: 访存能不能合并 (coalescing)、会不会撞 bank、某个硬件引擎读不读得了一块 tile, 全看它怎么摆。同一批数字, 光是换个摆法, 在同一块 GPU 上性能就能差一个数量级。
- 全书就用一套紧凑记号描述所有布局: $S[(\text{shape}) : (\text{strides})]$, 说白了就是「形状: 步长」做点积。这玩意儿你早就见过——PyTorch/NumPy 里的 `shape + stride` 就是它, 这里只是把它搬到了 GPU 上。
- 给步长贴上 **命名轴 (named axes / 命名轴)** (像 `@m`、`@laneid`、`@reg`、`@gpuid_x`、`@TLane` 这些), 同一套记号就能描述数据在内存、寄存器、warp 通道、TMEM、乃至整个 GPU 网格上是怎么铺开的。
- 再加一个 **复制项 (replication term / 复制项)** $R[n : \text{stride}]$, 就能表达「同一份数据被广播或拷贝到好几个地方」。分布式分片里的副本是它, Blackwell block-scaled MMA 在 TMEM 里的 `warpx4` 广播也是它。
- **Swizzle(地址异或重排 / swizzle)** 专门用来消掉共享内存的 bank 冲突, 让同一块 tile 按行读、按列读都能跑得快。注意它不属于仿射布局 $S[\dots]$, 而是单独叠在上面的一层非仿射变换。

前置知识

前置知识: 读这一章前, 最好先懂 warp 与 lane(通道)、寄存器 vs 共享内存、内存 bank 与 bank 冲突, 以及 Tensor Core 大概是干嘛的。没把握的话, 先翻一下第 0 章·极简入门。本章会默认你已经认识这些词。

平时写机器学习程序, 我们脑子里基本只有张量的「逻辑形状」, 比如一个 (M, N) 的矩阵。可形状只说清了一件事: 一共有多少个数。至于这些字节摆在内存的哪个角落, 它一个字都没提。偏偏硬件最在意的就是这个。

这一章要补的, 就是这块拼图。**数据布局 (data layout)** 回答的问题很直接: 逻辑下标 $(i, j, \hat{e})$ 对应的那个元素, 到底存在哪儿? 答案可能是全局内存, 可能是共享内存, 可能是寄存器, 也可能是某种专用的硬件存储。

那物理摆放为什么这么要紧? 原文给了三条理由, 都挺具体, 记住就够了。

1. **访存能不能合并:** 一个 warp(一个 warp = 32 个线程组成的小班, 见第 0 章) 里 32 个通道 (lane, 即线程在 warp 内的编号) 同时发 load。要是它们读的地址刚好连成一片, 硬件一把就能合成 1 次内存事务; 可要是地址东一个西一个, 就可能炸成 32 次。这差距, 大得吓人。

2. **会不会撞 bank**: 这些地址落在不同的内存 bank(共享内存被切成的若干「存储体」) 上, 就能并行处理; 可要是挤进同一个 bank 的不同地址, 硬件没辙, 只能一个一个排队来, 串行化。
3. **引擎读不得了**: 一块 tile(大矩阵切出来的小方块) 的字节怎么排, 直接决定 Tensor Core(GPU 里专门算矩阵乘的硬件单元, 见第 0 章) 能不能把它当成一个合法的操作数 (参与运算的输入数据) 读进去。

这一章的讲法是一层一层往上垒: 先搭最朴素的「形状-步长」模型, 再一点点往上加东西 (tile、命名轴、复制项), 最后才到 swizzle。讲到后面你会发现, 整章翻来覆去其实就是 **同一个想法换着花样讲**。

一、形状-步长模型 (The Shape-Stride Model)

咱们从最简单的布局讲起。说到底, 描述一个布局只要两样东西: 一个 **形状 (shape)**, 加上一组对应的 **步长 (stride)**。写成记号长这样:

```
S[(shape) : (strides)]
```

想知道某个逻辑下标存在哪儿? 把「下标」和「步长」做一次 **点积**就完事了。拿一个行主序 (row-major) 的 4×4 矩阵来说:

```
S[(4, 4) : (4, 1)]      addr(i, j) = i.4 + j.1
```

步长 (4, 1) 是啥意思? 其实特好懂。行下标 *i* 每加 1, 地址就往前蹦 4 步——因为一行有 4 个元素嘛。列下标 *j* 每加 1, 地址只往前挪 1 步。就这么个经典的 shape/stride 模型, 只是写法更紧凑 (你把它当成 CuTe 记号的行主序简化版就行)。

你其实早就用过它

别把它当成什么新鲜玩意儿。只要你写过 PyTorch 或 NumPy, 你早就在用这套东西了。这些库里的张量, 说白了就是「一个 shape 加一个 stride, 盖在一段扁平的存储缓冲区上面」:

```
import torch
t = torch.arange(12).reshape(3, 4)
t.shape      # torch.Size([3, 4])
t.stride()   # (4, 1)  <-   S[(3, 4) : (4, 1)]
```

一旦你这么去看张量, 很多「重排」操作为什么 **压根不碰数据**, 一下就通了。它们干的事无非是改写步长, 然后在同一段存储上给你返回一个 **视图 (view)**。最典型的就是转置 / permute:

```
tt = t.permute(1, 0)          #      t.T
tt.shape                      # torch.Size([4, 3])
tt.stride()                   # (1, 4)  <-      ,
tt.data_ptr() == t.data_ptr() # True,
```

`t.permute(1, 0)` 不过就是同一片内存上的 `S[(4, 3) : (1, 4)]`。换句话说, **所谓转置, 就是把步长换了换, 一个字节都没挪**。`reshape` / `view` 作用在连续张量上也是同样的套路——给老存储套上一个新 shape、新 stride, 如此而已。

注意

注意:NumPy 的行为跟这一模一样,唯一的区别是它的 `.strides` 按「字节」算,而 PyTorch 的 `.stride()` 按「元素」算。

零拷贝推理的边界

下面这张图把「逻辑视图」和「物理存储」的关系串了起来:

结构图

连接关系:

- 逻辑下标 (i, j) $\xrightarrow{\text{下标} \cdot \text{步长 (点积)}}$ 线性地址 addr
- 线性地址 addr $\rightarrow$ 扁平存储缓冲区 [0][1][2][3][4]...[11]

GPU 上的布局也是这么回事。一块 tile 的映射,不管映到内存,还是用后面要讲的命名轴映到通道和寄存器,骨子里都是「固定缓冲区上的一条步长规则」。所以 **重排一块 tile,多数时候只是换了个布局,并没有真去拷贝数据**。

关键

关键: 零拷贝这套说法,只有在「单一线性地址空间上的逻辑视图」里才干干净净地成立。到了 GPU 上,它只在一种情况下管用:新视图跟现有的字节排布、跟所有权的安排是兼容的。反过来,只要你动了「哪个线程、哪个寄存器拥有某个元素」,或者改了 SMEM 的 `swizzle`,那就 **躲不掉**,必须老老实实搬数据,靠 `load`、`store`、`shuffle`、`ldmatrix`、`transpose` 这些操作来搬。后面好几章的优化,根子都扎在这儿。

二、Tile 布局 (Tile Layout)

前面讲的都是「整个张量」的布局。可现实里,GPU kernel 几乎从不会一口气啃下整块矩阵,而是把它切成一个个更小的 **tile(分块)**,丢给硬件的不同部分分头去加载、变换、计算。

好消息是: **搞 tiling,不需要学任何新概念**。它还是一个布局,无非就是多写几个维度罢了。

举个例子,把一个 8×8 矩阵切成若干 2×4 的小块。这样我们就得到一个 **4 维**布局,坐标是 (tile_row, row_in_tile, tile_col, col_in_tile);步长得挑得让每个 tile 内部保持连续:

```
S[(4, 2, 2, 4) : (16, 4, 8, 1)]
```

映射就分两步。第一步,把逻辑 (i, j) 拆成一个四元组 (i//2, i%2, j//4, j%4),也就是分出「外坐标」和「内坐标」;第二步,拿这四个数跟步长做点积。还拿 8×8 切 2×4 来说,拆法是这样:

逻辑下标	拆分 (外, 内)	含义
i(行,8 行)	(i//2, i%2)	第几个 tile 行 (共 4 个) / tile 内第几行 (共 2 行)
j(列,8 列)	(j//4, j%4)	第几个 tile 列 (共 2 个) / tile 内第几列 (共 4 列)

那步长 (16, 4, 8, 1) 又是怎么定下来的? 就盯住一条约束: 让每个 tile 里头那 8 个元素在物理上连着放 (也就是按 tile 一块一块地铺, 而不是整体一刀切的行主序)。咱们挨个看:

```

,      tile      8      :
col_in_tile (stride 1) : 0->1->2->3,tile      4
row_in_tile (stride 4) : +4      tile      (      4      )
                        ->      2x4 tile      8      [0..7]

,      tile      :
tile_col      (stride 8) : +8      tile(      tile = 8      )
tile_row      (stride 16) : +16      tile(      2      tile = 16      )

:

      tile      ,
shape stride      ,      /
```

注意

注意: 原书在这儿放了个交互演示, 随便点一个格子就能看到它的 tiled 下标和物理地址。静态笔记没法还原这种点来点去的效果, 不过上面的步长拆解已经把「逻辑 (i,j) → 拆分 → 地址」整条规则讲透了。想找找手感, 可以去原书那个交互演示里玩一玩。

三、命名轴 (Named Axes)

到目前为止,S[...] 里每个步长指的都是「线性内存里的一个偏移」, 我们也一直默认「地址」就等于内存里的位置。可 GPU 上的数据, 能待的地方 **远不止内存这一处**。除了内存, 一块 tile 还可能散在 warp 通道 (lane) 上、散在线程寄存器 (register, 每个线程私有、访问最快的小存储) 上, 甚至散在 TMEM(Tensor Memory,Blackwell 新增的、专给 Tensor Core 用的片上存储) 的通道和列上。

那怎么用 **同一套记号**把这些情况一网打尽? 作者这招挺巧: 给每个步长系数贴一个「轴标签」, 标清楚它是在哪个空间里挪动。

标签	含义
@m	普通内存 (memory)
@laneid	warp 通道索引, 即 thread_index % warp_size
@reg	每通道私有的寄存器 (register)
@warpid	warp 索引
@TLane / @TCol	TMEM 的通道 / 列坐标

贴好标签, 一个行主序的 8×16 内存 tile 就写成这样:

```
S[(8, 16) : (16@m, 1@m)]
```

(两个步长都带 @m, 说明它们都指向线性内存。)

读法速成: 手把手念一行 S[...]

你可能已经被后面那种 S[(8,4,2):(4@laneid,1@laneid,1@reg)] 的写法吓到了。别慌——它再花哨, **念法永远是固定的几步**。先把套路记下来, 后面所有表达式都照这个模子读就行。

关键

关键:`S[( ): ( )]` 就念成一句话——「这块数据有这么大 (形状); 每个下标 +1, 就在某个空间里走这么远 (步长)」。

第 1 步: 看中括号。`S[ ... ]` 就代表”一个布局”。中间的冒号 : 把它劈成两半: 左边是形状, 右边是步长, 两边的项一一对应。

第 2 步: 读形状。`(8, 4, 2)` 就是说这块数据是 $8 \times 4 \times 2$, 三个维度, 没别的。

第 3 步: 读步长里的 `N@axis`(这是命门)。`N@axis` 念作:「这一维的下标每 +1, 就沿 `axis` 这个轴走 `N` 步」。关键全在 `@` 后面那个轴——它告诉你”走在哪个空间里”:

轴标签	走在哪
<code>@m</code>	内存地址上
<code>@laneid</code>	warp 的 32 个通道 (lane) 上
<code>@reg</code>	某个通道自己的寄存器上
<code>@TLane / @TCol</code>	TMEM 的通道 / 列上

第 4 步: 同一个轴出现多次, 就把它们加起来。要是有两个维度的步长都带 `@laneid`, 说明这俩合伙决定”落在哪个 lane”——把它们各自的「下标 $\times$ 步长」加到一起, 就是最终的 lane 号。

把这套套到那个吓人的例子上:`S[(8, 4, 2) : (4@laneid, 1@laneid, 1@reg)]`。假设我想知道逻辑元素 (`i=2, j=3, r=1`) 到底住哪:

- 前两维都带 `@laneid`, 合起来算 `lane:lane = 2 $\times$ 4 + 3 $\times$ 1 = 11;`
- 第三维带 `@reg:reg = 1 $\times$ 1 = 1。`
- 结论: 元素 `(2,3,1)` 住在第 11 号 lane 的 1 号寄存器里。

(顺手验证一下: 第一维 $8 \times$ 第二维 $4 = 32$, 正好等于一个 warp 的 32 个 lane——所以这块数据刚好摊满一整个 warp, 每个 lane 手里攥 2 个元素。)

最后一块拼图: `+ R[n : stride]`。有时你会看到 `S[...]` + `R[...]`, 那个 `R` 是复制 (replication): 把前面 `S[...]` 定好位置的那份数据, 再沿某个轴抄 `n` 份, 相邻两份隔 `stride` 那么远。比如 `+ R[4 : 32@TLane]` 念作”沿 TMEM 通道轴复制 4 份、每份隔 32 个通道”, 也就是第 0、32、64、96 号通道拿的是同一个值。

记住这套念法, 后面 `@TLane`、`@gpuid_x`、`R[...]` 再怎么排列组合, 你都能一个字一个字拆开读懂, 而不用死背结论。

标签真正发光的时刻: 数据跨线程分布

光给内存 tile 贴个 `@m`, 你还看不出标签有啥用。它真正出彩, 是在描述「数据散在好多个线程上」、而不是「老老实实躺在内存里」的时候。看这个例子:

```
S[(8, 4, 2) : (4@laneid, 1@laneid, 1@reg)]
```

这个布局不再指向线性内存了, 它把「行、列」映射到了 **lane ID** 和 **每个通道里的寄存器上**:

- 前两维 (形状 8 和 4) 的步长都带 `@laneid`。也就是说, 这俩逻辑维度合起来决定「数据落在 warp 的哪个通道」(`lane = 0 $\cdot$ 4 + 1 $\cdot$ 1`, 一共 $8 \times 4 = 32$ 个 lane, 正好把一个 warp 填满)。
- 最后一维 (形状 2) 的步长带 `@reg`。意思是这一维顺着「同一个通道内的寄存器」展开, 说白了就是每个 lane 手里攥着 2 个元素 `r0` 和 `r1`。

结构图

连接关系:

- 逻辑张量 (8, 4, 2) → @laneid 维 (8×4=32): 决定落在哪个 lane
- 逻辑张量 (8, 4, 2) → @reg 维 (2): 决定在该 lane 的哪个寄存器
- @laneid 维 (8×4=32): 决定落在哪个 lane → warp 的 32 个通道
- @reg 维 (2): 决定在该 lane 的哪个寄存器 → 每通道私有寄存器 r0 / r1

关键

关键: 这恰好就是 Tensor Core 的「寄存器片段 (register fragment, 即一块矩阵被拆开、分散存放在各个线程寄存器里的那一份份小碎片)」布局, 后面《Tensor Core Operand Layouts Across GPU Generations》那一章会反复用到。同一套 S[...] 记号, 既能描述内存布局, 又能描述「哪个通道、哪个寄存器拿着哪个元素」——这套记号的本事, 就在这儿。

注意

注意: 原书在这儿也配了交互演示, 点一下格子就能看到它是由哪个 lane / register 持有的。想把「逻辑元素 ↔ 物理 lane/reg」这层对应关系摸熟, 去原书玩一玩这个演示。

四、分布式布局 (Distributed Layout)

命名轴最妙的一点是: 它能在系统的好几个层级上, 用同一种方式描述「数据放在哪」, 甚至能描述「跨好几台设备」的摆放。前面我们用它讲了单卡内部的 lane 和 register, 现在把同一个想法往外推, 推到一整个 GPU 网格 (mesh) 上:

- 像 @gpubid_x / @gpubid_y 这种轴, 说的是「数据落在网格里的哪台 GPU 上」;
- 有了它俩, 这套记号就能描述分布式训练 / 推理里的 分片 (sharding) 模式了。

用 R[...] 描述复制

命名轴几乎啥都能描述, 但还缺一块: **复制 (replication)**, 也就是「同一份数据被拷到好几个地方」这种情形。为此作者添了个新记号:

```
R[n : stride]
```

R 标明这是一个复制维度, n 是复制几份, stride 是相邻两份副本之间隔多远 (也照样能带轴标签)。比如 R[2 : 1@gpubid_x], 意思就是「沿 @gpubid_x 轴复制 2 份」。

把分片和复制拼一块儿, 一个表达式就能一口气干两件事: 先在 2×2 的 GPU 网格上把张量分片, 再沿某一轴复制:

```
S[(2, 4, 8) : (1@gpubid_y, 8@m, 1@m)] + R[2 : 1@gpubid_x]
```

下面这张图, 画的就是这个「分片 + 复制」模式落在 2×2 网格上是什么样:

横轴 @gguid_x 是复制方向, 纵轴 @gguid_y 是分片方向。网格上每台 GPU 手里拿着啥, 看下表:

	@gguid_x = 0	@gguid_x = 1(复制方向)
@gguid_y = 0	GPU(0,0): 分片 A	GPU(1,0): 分片 A 副本
@gguid_y = 1	GPU(0,1): 分片 B	GPU(1,1): 分片 B 副本

- S[...] 里的 1@gguid_y → 张量沿 y 轴在两台设备间「分片」(分片 A / 分片 B);
- R[2 : 1@gguid_x] → 这个分片再沿 x 轴「复制」一份给配对的设备 (副本)。

注意

注意: 原书的交互演示能点格子看它由哪台 (或哪几台) 设备持有, 还能在「完全分片 / 分片 + 副本 / 分片 + 偏移」三种布局之间来回切。静态图没法还原这种切换, 想看个明白就去原书体验一下。

4.1 kernel 内的复制:TMEM 中的 Scale Factor

复制维度 R[...] 可不光是给「多台设备」用的。同样这套写法, 也能描述 **单个 kernel 内部** 发生的事——硬件把数据 **跨通道广播**出去。Blackwell(NVIDIA 的最新 GPU 代号,Hopper 的下一代) 的 block-scaled MMA(MMA = 矩阵乘加,Tensor Core 干的核心活) 就是个绝佳的例子。

它的缩放因子 (scale factor) 存在 TMEM 里。妙就妙在: 一个 **128 行**的缩放向量, 实际上只吃掉了 **32 个 TMEM 通道**。

- 逻辑行 r → TMEM 通道 r % 32;
- r // 32 沿列方向展开。

接着, 这存好的 32 个 TMEM 通道, 会沿着 TMEM 的 **TLane 轴复制一遍**, 从 32 个铺成 128 个。这么一弄, 读这份数据的 warpgroup(4 个 warp 凑成的一组, 共 128 个线程) 里 **每一个 warp**, 都能在自己那个 32 通道的 TMEM 窗口里摸到一份副本。这就是一次 **warp_x4** 广播, 用复制维度写出来就是:

S[(32, ẽ) : (1ẽTLane, ẽ)] + R[4 : 32ẽTLane]

这就给出了 4 份副本, 相邻两份隔 32 个 TMEM 通道。换句话说,TMEM 通道 1、1+32、1+64、1+96 拿的是同一个缩放值。

存储阶段:128 逻辑行 → 32 TMEM 通道

- 行 r → TMEM lane r % 32, 列偏移看 r // 32;
- 总共只动用了 32 条通道 (lane 0 …lane 31)。

广播阶段:warp_x4 复制 (R[4 : 32ẽTLane])

通道 1、1+32、1+64、1+96 拿的是同一份缩放值; 每个 warp 的 32 通道窗口各自分到一份副本:

	warp0	warp1	warp2	warp3
TMEM 通道范围	0..31	32..63	64..95	96..127

关键

关键: 跟 GPU 网格那回事一样, 复制维度本身一个新数据都不带。它只是声明「同一个值, 同时坐在 4 个 TMEM 通道上」, 就像 @gpuid_x 把一行广播到整个 GPU 网格那样。真正去读它的, 是这些 warp 里的线程。

注意

注意: 每列内部的字节怎么打包 (scale_vec 的 1X/2X/4X 模式), 还有 cta_group::2 的拆分, 这些留到《Tensor Core Operand Layouts Across GPU Generations》再细说。原书在这儿也有交互演示, 把「压紧塞进 32 个 TMEM 通道」和「warp_x4 广播到 128 个读取通道」这两步连着演给你看, 值得一看。

要是你熟悉 CuTe, 本章的记号这么理解就行: 它就是 CuTe 的一个 **行主序变体**, 只不过多塞了「显式的硬件命名轴」和「专门的复制结构」。

五、Swizzle 布局 (Swizzle Layout)

这一章的最后一个布局, 是专门为治一个具体的硬件毛病而生的。

问题:bank 冲突

GPU 的 **共享内存 (shared memory, SMEM)** 在物理上被切成了好几个 **内存 bank**。访问最快的情形, 是几个通道恰好落在 **不同的 bank** 上。可一旦多个通道挤去访问 **同一个 bank 里的不同地址**, 硬件就没招了, 只能把这些访问 **一个接一个排队来**。这就是 **bank 冲突 (bank conflict)** 要交的学费。

在张量程序里, 这个坑几乎绕不过去, 因为内存访问往往不是干干净净的线性。处理矩阵时, 我们老是要同时读同一块 tile 的 **某一行**和 **某一列**。这就把人逼进了一个两难:

布局	行访问	列访问
利于行的布局	高效	产生 bank 冲突
利于列的布局	产生 bank 冲突	高效

Swizzle(混洗 / 异或重排) 这招, 就是冲着打破这个两难来的。

思路: 用 XOR 把地址打散

Swizzle 的核心, 就是 **把地址映射重排一下**。最典型的玩法, 是拿列索引跟行索引做一次 **异或 (XOR)**。这么一来, **无论你按行读还是按列读, 地址都会被甩到不同的 bank 上**。

下面拿一个 8×8 tile, 把「朴素行主序」和「XOR swizzle」摆一块儿对比 (假设有 8 个 bank, 看某一整列各自落进哪些 bank):

读「第 c 列」那 8 个元素, 各自落进哪个 bank:

行	朴素行主序 (bank = c)	XOR swizzle(bank = c XOR row)
---	------------------	-------------------------------

行 0	bank c	bank (c XOR 0)
行 1	bank c	bank (c XOR 1)
...	...(全是同一个 bank)	...
行 7	bank c	bank (c XOR 7)
结果	8 个元素全落进同一个 bank → 串行化成 8 个周期	同一列被打散到 8 个不同 bank → 1 个周期读完

关键

关键:swizzle 给的「无冲突保证」是 **带条件的**。只有当「元素宽度、swizzle 模式、访问模式」三者对得上 (也就是凑成某个引擎 descriptor 期望的那一组) 时, 它才成立; **并不是**随便什么元素宽度、什么对齐方式都灵。一句话: 模式选错, 保证作废。

真实硬件: 分段 + atom

8×8 只是个方便讲解的玩具例子。真实 GPU 的 bank 多得多, 一般是 32 个。为了在真实规模上也好用,swizzle **不会把整块 tile 当成铁板一块来处理**, 而是先把内存切成一小段一小段 (segment), 再在 **每一段内部**各自施加 swizzle 模式。

最常见的是 SWIZZLE_128B, 它围着 **128 字节的段**来组织。这么安排,「行/列重排」这套把戏就跟 32-bank 的内存系统天然对上了 (128 B = 32 bank × 4 B)。它的读取规则大致是:

```
physical_sector = logical_sector XOR row
```

在每个 128 字节段里头, 光靠这一条规则, 就能把每一列都打散到不同的 bank 上。

不同的 swizzle 模式 (atom 大小)

把上面的思路再往前推一步: 硬件定一个又小、又会反复出现的 **atom(原子单元)**, 在它身上施加置换; 不同的 swizzle 模式, 挑的 atom 大小不一样; 整块 tile 就拿选好的 atom 一格一格平铺 (tile) 出来。

模式	atom 形状	适用条件 (连续维度至少)
SWIZZLE_128B	8 × 128 B	128 字节 (fp16 下即 64 个元素)
SWIZZLE_64B	8 × 64 B	64 字节
SWIZZLE_32B	8 × 32 B	32 字节
16 B 交错 (interleaved) 模式	原书未给出具体 atom 形状	用于更小的连续维度

该选哪个模式?

一条经验法则:**tile 能填满哪个最大的 atom, 就挑哪个**。

- 一个 N 字节的 atom, 要求 tile 的 **连续维度**至少有 N 字节, 而且得是 N 的整数倍。
- 所以 SWIZZLE_128B 只有在一行至少跨 128 字节 (也就是 64 个 float16 元素) 时才用得上。
- 而一旦用得上, 它就是首选。它那个 8 × 128 B 的 atom 刚好铺满一整条 128 字节的 bank line, 所以能 **一把就把一列打散到全部 32 个 bank**; 在 fp16 下,8 行 8 列读起来都不撞 bank。
- 要是问题形状把连续维度逼得很小,128 B atom 填不满, 那就退一档, 换 SWIZZLE_64B 或 SWIZZLE_32B ——原则不变, 就是「一行能盖得住的最大 atom」。

Swizzle 与 S[...] 记号的关系

关键

关键: 这些被置换过的地址, 你 **根本不用自己去手算**。另外有一点得说清楚:**swizzle 不属于 S[...] 那个仿射映射**。它是单独叠在上头的、自成一体、非仿射的一层。

具体来说:S[...] 布局先把元素安顿到一个线性内存 (@m) 地址,swizzle 再把这个地址置换一下。在 TIRx 的 Layout API 里, 写成:

```
ComposeLayout(swizzle, tile)
```

你要操心的只有一件事: **给每一个会碰这块 tile 的算子, 选一个统一的模式**。剩下的, 全甩给这个「组合后的布局」去摆平就行。

结构图

连接关系:

- 逻辑下标 $\xrightarrow{\text{S[...] 仿射映射}}$ 线性 @m 地址
- 线性 @m 地址 $\xrightarrow{\text{swizzle 非仿射置换}}$ 实际 SMEM 地址

与 tiling、TMA 的会合

这个「组合后的布局」, 恰好也正是 **硬件要去填的那个东西**。swizzle 和 tiling 就在这儿碰上头:

- 一个 **TMA descriptor** 本身就是多维的, 所以单个三维「盒子 (box)」一次就能把两件事说清:tile 的 atom 怎么平铺, **外加每个 atom 内部怎么 swizzle**;
- 这样一次 TMA load, 就能 **一个 atom 一个 atom 地把 tile 铺开**, 顺手在写进共享内存的同时把 **swizzle 也做了**, 压根不用再单独跑一遍 swizzle(详见《Async Data Movement: TMA》)。

注意

注意: 至于每个引擎到底「要哪种」swizzle, 这是 **看代际 (generation)** 而定的, 正好就是下一章的内容。

小结

这一章一层一层往上垒, 最后搭出了一套贯穿全书的布局语言:

- 形状-步长模型 S[(shape):(strides)]** 是地基——逻辑下标跟步长做点积, 就得到物理地址。它其实就是 PyTorch/NumPy 张量背后那套 shape+stride。转置、reshape 为啥能零拷贝? 因为它们只改步长、不碰数据。
- Tile 布局**没添任何新概念, 无非就是把下标拆成「外坐标 / 内坐标」、再多写几个维度。
- 命名轴 (@m、@laneid、@reg、@warpid、@TLane/@TCol、@gguid_x/y)** 把同一套记号推到了内存、寄存器、通道、TMEM, 乃至整个 GPU 网格, 让「数据在硬件资源上怎么铺」也能精确写出来。

4. 复制项 `R[n:stride]` 描述「同一份数据被广播到好几处」, 分布式分片里的副本是它, Blackwell block-scaled MMA 在 TMEM 里的 `warpx4` 广播也是它。复制维度本身不带新数据。
5. `Swizzle` 拿 XOR 把地址重排, 消掉 SMEM 的 bank 冲突, 让按行读、按列读都快。它是叠在 `S[...]` 上头的一层非仿射变换 (`ComposeLayout(swizzle, tile)`), 按 `atom(8×N B)` 分段施加; 挑法是「能填满的最大 `atom` 优先」, 默认就用 `SWIZZLE_128B`。

最后, 记牢贯穿全章那句话: 同样一批数字, 换个物理摆法, 在同一块 GPU 上跑出来的性能差一个数量级。而 `S[...]` + 命名轴 + `R[...]` + `swizzle`, 就是用来精确说清、并攥住这种物理摆放的家伙事儿。

本章局限说明: 原书一共有 7 处交互演示 (tile 拆分、lane/reg 分布、GPU 网络分布、TMEM warp_x4 广播、8×8 bank 冲突对比、SWIZZLE_128B 分段、各 swizzle 格式 atom 浏览)。这份笔记已经用 ASCII 图和表格把它们的核心要点重搭出来了, 但点击切换、逐周期步进这类交互, 静态文档实在还原不了。想真正吃透, 最好配着原书的交互演示一起看。

延伸阅读

- 原文章节: [Data Layout and Its Notation](#)
- 相关后续章节 (原书内部引用):
 - *Tensor Core Operand Layouts Across GPU Generations* ——寄存器片段、block-scaled MMA、`scale_vec`、`cta_group::2`
 - *Async Data Movement: TMA* ——TMA descriptor 如何一次性完成 `atom` 平铺 + `swizzle`
 - *TIRx Layout API* ——`ComposeLayout(swizzle, tile)` 的实际接口
- 概念背景: CuTe (本章记号是其行主序变体)、PyTorch/NumPy 的 `stride` 模型

术语对照

中文	English
数据布局	data layout
形状	shape
步长	stride / strides
视图	view
分块 / 瓦片	tile / tiling
命名轴	named axes
warp 通道	lane / laneid
寄存器	register / reg
复制 (广播)	replication
分片	sharding
GPU 网格	GPU mesh
缩放因子	scale factor
访存合并	coalescing
内存 bank	memory bank
bank 冲突	bank conflict
共享内存	shared memory (SMEM)
混洗 / 异或重排	swizzle / swizzling
原子单元	atom

仿射 (映射)	affine (map)
张量核心	Tensor Core
寄存器片段	register fragment

第 04 章 · 各代 GPU 的 Tensor Core 操作数布局

原文:Tensor Core Operand Layouts Across GPU Generations

本章要点

本章要点 (TL;DR)

- 从 Ampere 到 Hopper 再到 Blackwell, Tensor Core 干的活始终是同一件: $D = A \cdot B + C$ (矩阵乘累加 / MMA)。真正在变的, 是操作数怎么喂进 Tensor Core、支持哪些 tile 形状和数据类型、以及累加器 (accumulator) 落在哪里。
- Ampere: 操作数是分布在 warp 32 个 lane 上的寄存器片段 (register fragment)。用 `ldmatrix` 把共享内存 (SMEM) tile 搬进寄存器, `mma.sync` 在寄存器之间计算, 累加器全程驻留在寄存器里。
- Hopper: `wgmma` 可以通过矩阵描述符 (matrix descriptor) 直接从 SMEM 读操作数, 描述符里写明硬件期望的 swizzle 格式。但累加器仍然在寄存器里。
- Blackwell: 输入侧保留 SMEM 操作数路径, 但把累加器搬进 TMEM (Tensor Memory); block-scaled MMA 的缩放因子 (scale factor) 也要经 TMEM 暂存。
- 两条贯穿所有世代的内存约束从未消失: 全局内存合并访问 (coalescing) 和共享内存 bank 冲突 (bank conflict)。布局不是 Tensor Core kernel 的“装饰”, 它本身就是指令接口的一部分——任何一环对不上, 硬件照样运行, 只会读错字节或读得慢。

前置知识

前置知识: 读这一章前, 最好先懂第 3 章的布局记号 (就是 `S[(8,4):(...)]` 那套写法), 以及这几个词: warp (一个 warp = 32 个线程的小班)、lane (warp 里每个线程的编号, 0-31)、寄存器片段 (register fragment, 一块 tile 被拆散摊在各个 lane 的寄存器里)、MMA (矩阵乘累加, $D = A \cdot B + C$)、TMEM (Blackwell 新增的张量内存)。没把握的话, 先翻一下第 0 章·极简入门, 以及第 3 章《数据布局》。本章会默认你已经认识这些词。

引子: 为什么“逻辑公式没变”还会算错

你远远地看 Tensor Core (GPU 里专门做矩阵乘法的硬件单元), 会觉得它特别稳: 喂进去 A、B 两块 tile (大矩阵切出来的小方块), 带上累加器 C (存放累加结果的那块数据), 吐出来一个 D。这个样子打 Volta 那会儿就没变过。可你凑近一看就会发现, 这个运算周围的种种细节, 其实一直在变。

这一变, 就给我们惹出两个很实在的麻烦。

1. 同一个 kernel, 在这一代上飞快, 换一代可能就拖泥带水了。为什么? 因为数据进出 Tensor Core 走的路换了。

2. 布局要是摆错了,kernel 干脆给你算出个错答案。注意,哪怕你写的逻辑明明白白还是 $D = A \cdot B + C$, 结果照样能错。

这是为什么?说白了,Tensor Core 根本不认什么”抽象矩阵”,它认的是按某个固定硬件布局摆好的操作数。在它眼里,第 0 个元素天经地义就该躺在某个 lane 的某个寄存器槽里,第 1 个该在另一个位置,后面以此类推。你的数据只要没按它想要的样子摆,它就会照样去乘,只不过把”压根不该相乘的两个元素”给乘到一块儿去了。

所以这一章,我们就顺着这根「布局契约 / layout contract」的线,把三代硬件挨个走一遍。描述契约要用到的那套布局记号,出自《数据布局》那一章;Blackwell 的 TMEM 细节比较多,单独搁在《Special Memory: TMEM》一章里讲。

一、两条从未离场的约束

其实早在 Tensor Core 出现之前,就有两条最朴素的内存约束在悄悄左右着 GPU kernel 的布局。它们不挑对象,对哪个 kernel 都管用,哪怕这个 kernel 跟 Tensor Core 八竿子打不着。我们先把这两条掰扯清楚,后面 Tensor Core 那套故事才好接得上。

1. 全局内存合并访问 / global memory coalescing

先建立个直觉。一个 warp(32 个线程的小班)里有 32 个 lane(每个线程的编号),它们会同时伸手去读全局内存 (GMEM, 显存,GPU 上最大也最慢的那层内存)。这时内存系统最乐意看到的,是这 32 个地址挨得近近的,凑在少数几个连续、对齐的内存段里。

- 地址又连续又对齐 → 一次、顶多几次内存事务就全搬回来了。
- 地址东一个西一个 → 同样这点数据,硬要拆成一大堆内存事务才搬得完,带宽和时间双双浪费。

2. 共享内存 bank 冲突 / shared memory bank conflict

共享内存 (SMEM, 一个 block 内线程共享的高速片上内存) 被切成 32 个 bank(存储体,可同时独立访问的小仓库)。麻烦就出在这儿:一个 warp 里好几个 lane 去读的地址各不相同,却偏偏都落在同一个 bank 上,这时硬件没法一口气服务它们,只能让它们排着队一个一个来,也就是串行化。

注意

注意一个看着特别老实、人畜无害的”扁平共享内存数组”,也可能因为 bank 怎么映射而慢得离谱。慢,慢的不是数据本身,而是物理地址到 bank 的那套映射关系。

解药:swizzle / 地址混洗

那共享内存这边怎么救?常用的招数叫 swizzle / 地址混洗(故意打乱数据在内存里的物理摆放,把访问摊开到不同 bank)。思路其实很简单:逻辑 tile 我们一个字节都不挪,只是给物理地址的映射偷偷加一道排列,把访问横着摊到好几个 bank 上,别再一窝蜂全挤一个 bank。

结构图

连接关系:

- 物理地址 / (打散到 32 个 bank) → 读写都能 / 避开 bank 冲突

图:swizzle 只改物理摆放, 不改逻辑坐标。

Tensor Core 额外加的第三条约束

Tensor Core kernel 在上面两条之上, 还要再背一条: 操作数得摆成 **Tensor Core** 指令自己想要的那个布局。本章接下来的主线, 就是看这” 第三条约束” 怎么从 Ampere 一路演变到 Hopper、再到 Blackwell。

关键

关键一句话先记住: 前两条 (合并访问、bank 冲突) 三代都跑不掉; 真正在变的是第三条, 以及” 布局这件脏活到底归谁干”——是 kernel 自己一行行写出来, 还是直接甩给硬件描述符去管。

二、Ampere: 跨 warp lane 的寄存器片段

Ampere 这代 GPU 上, 挑大梁的 Tensor Core 指令是 warp 级的 `mma.sync.aligned.m16n8k*` 这一家子。想搞懂这一代, 你只要盯死一个问题: 这条指令到底在哪儿读、又往哪儿写? 答案就俩字——**寄存器**。

A、B, 连同 C/D 累加器, 统统是摊在 warp 那 32 个 lane 上的 **per-thread 寄存器片段** (寄存器 / register 是每个线程私有、速度最快的那点存储)。共享内存在这里说白了就是个中转站。换句话说,MMA 还没开跑, 操作数 tile 就得先从 SMEM 搬进寄存器, 而且得严丝合缝地摆成指令要的那个片段布局。

Ampere 的数据通路

步骤	数据通路	说明
1	SMEM --ldmatrix--> registers	把 tile 搬进片段
2	regs --mma.sync--> registers	寄存器到寄存器计算
3	regs -- store--> SMEM	累加器写回

结构图

连接关系:

- 共享内存 SMEM $\xrightarrow{\text{ldmatrix} / (\text{warp 协作})}$ 寄存器片段 / (32 lane 上的 fragment)
- 寄存器片段 / (32 lane 上的 fragment) $\xrightarrow{\text{mma.sync}}$ 累加器寄存器片段
- 累加器寄存器片段 $\xrightarrow{\text{普通 store} / (\text{无专用反向指令})}$ SMEM / GMEM

图:Ampere 的 Tensor Core 通路, 输入靠 `ldmatrix`, 输出靠普通 `store`。

Ampere 布局这套故事, 大半都能从这条通路里顺出来:**kernel 得先把 tile 用一种” 方便高效加载” 的姿势写进 SMEM, 然后靠 ldmatrix 把它捏成 mma.sync 想吃的那种寄存器片段。**

2.1 Ampere Tensor Core 到底期望什么

Ampere Tensor Core 读的那种寄存器片段, 是拿 8×8 子 tile 当最小积木一块块拼出来的。这些 8×8 的小单元, 就是 `ldmatrix` 加载、MMA 吃进去的基本单位。

我们拿个具体例子上手: `mma.m16n8k16`, 输入是 `fp16/bf16`, 累加用 `fp32`。它的累加器 tile 是 16×8 , 会按一个雷打不动的固定模式推到 32 个 lane 上。别急, 这个模式我们一步步拆。

先看 C/D 累加器。lane 1 手里攥着的行是:

```
1 / 4
1 / 4 + 8
```

它攥着的列是:

```
2 * (1 % 4)
2 * (1 % 4) + 1
```

公式你别背, 记住结论就够了: 每个 lane 手里都攥着 4 个 `fp32` 累加值。这 4 个值, 来自两行——上下两个“8 行半区”各贡献一行——每行又取相邻的两列。还有个特好记的规律: 连着的 4 个 lane, 正好把某一行的全部 8 列凑齐。

下面这张表, 把 16×8 累加器到底怎么分给各个 lane 摆了出来 (每格写的是它归哪个 lane)。就拿 lane 0-3 这一组说: 它们的 $1/4 = 0$, 所以管的是第 0 行 (再加上第二个寄存器对应的第 8 行):

行	列 0, 列 1	列 2, 列 3	列 4, 列 5	列 6, 列 7	备注
行 0	L0	L1	L2	L3	行 = $1/4$, lane 0-3 占第 0 行
行 1	L4	L5	L6	L7	lane 4-7 占第 1 行
...	...	...	...	...	每 4 个 lane 覆盖一行
行 7	L28	L29	L30	L31	lane 28-31 占第 7 行
行 8	L0	L1	L2	L3	行 = $1/4 + 8$, 同一 lane 的第二个寄存器
...	...	...	...	...	
行 15	L28	L29	L30	L31	

(每格里的那个 lane, 同时握着这一格对应的两列; 比如行 0 的“列 0, 列 1”格里写着 L0, 意思就是 lane 0 握着第 0 行的第 0、1 两列。)

说明: lane 0 同时持有 (行 0, 列 0)、(行 0, 列 1)、(行 8, 列 0)、(行 8, 列 1) 四个值; lane 1 持有列 2/3; 以此类推。每个 lane 拿 4 个 `fp32` 值, 连续 4 个 lane 刚好拼出某行的 8 列; 32 个 lane 覆盖上半区 (行 0-7) 与下半区 (行 8-15)。

A 和 B 的摆法跟累加器大同小异, 只是各有各的侧重:

- **A 操作数:** M 这一侧的行怎么切, 跟上面一模一样; K 维度则沿着 $1 \% 4$ 和 lane 手里那几个寄存器铺开。对 `fp16/bf16` 来说, 一个 32-bit 寄存器里塞 2 个 K 值。
- **B 操作数:** K 的摆法得跟 A 对上, 然后把 N 这一侧沿着 lane 组和寄存器铺开。

关键

关键具体数字会随指令形状、数据类型变来变去，但有条原则万变不离其宗：**Tensor Core** 要的就是一个特定的 **per-lane** 寄存器片段。值只要没待在对的寄存器、没排成对的模式，指令就会把不该乘的元素乘到一块儿。

用布局记号写出来,m8n8 片段长这样 (带上” 命过名的 lane 轴”):

```
S[(8, 4, 2) : (4@laneid, 1@laneid, 1@m)]
```

这堆符号别被它唬住，一项一项翻成人话就清楚了：

- 两个 @laneid 项凑一块儿，讲的是行、列的碎片怎么撒到各个 lane 上；
- 最后那个 @m 项，讲的是在某一个 lane 内部，值该塞进哪个寄存器槽。

2.2 ldmatrix: 从 SMEM 到寄存器片段的桥

ldmatrix 要解决的事儿很明确：怎么把数据从共享内存里，变成 Tensor Core 想要的那个寄存器片段？它就是架在这俩中间的那座桥。它是一条 warp 协作式加载——整组 lane 一起上，一条指令就能把一个、或者好几个 8×8 的 16-bit 矩阵，从 SMEM 搬进 mma.sync 想要的那种分布式寄存器布局。

指令长这样 (后面还能选挂一个 .trans 限定符):

```
ldmatrix.sync.aligned.m8n8.x1.shared.b16    //    1    8x8
ldmatrix.sync.aligned.m8n8.x2.shared.b16    //    2
ldmatrix.sync.aligned.m8n8.x4.shared.b16    //    4
```

这里头有个关键约定：**每一行的基地址，是由某个 lane 负责给出的**。具体说，第 m 个矩阵的第 r 行，它的基地址来自 lane $m*8 + r$ 。这么一算就是：

形态	加载矩阵数	用到的 lane(提供行地址)
.x1	1	lane 0-7
.x2	2	lane 0-15
.x4	4	lane 0-31

加载完，结果**直接就落进 MMA 片段**里了。在最基础的 8×8 情形下, lane 1 拿到手的，正好就是 Tensor Core 指望它持有的那一对行/列。

为什么非它不可反过来想一下：你要是不用 ldmatrix，改拿一堆普通的 per-lane ld.shared 去凑，那就得自己一字不差地把那套散射模式手工拼出来——又烦又特别容易出错。ldmatrix 的好处，就是把” 共享内存到片段” 这套重排，压成一条 warp 协作指令一把搞定。

那 .trans 是干啥用的？它会在加载的时候捎带手把每个 8×8 矩阵转个置。什么时候用得上？就是当操作数在 SMEM 里存的方向，跟 MMA 指令想要的方向正好拧着的时候，靠它来掰正。

结构图

分组:

- **SMEM**: 「8x8 16-bit tile / (行基址由 lane 提供)」

连接关系:

- 8x8 16-bit tile / (行基址由 lane 提供) $\xrightarrow{\text{ldmatrix} / (.trans \text{ 可转置})}$ warp 寄存器片段 / (Tensor Core 期望布局)

图 (等价于原书 *ldstmatrix.svg*): *ldmatrix* 把 8×8 SMEM tile 装进 warp 寄存器片段; *Ampere* 上反方向只能用普通 *store*, 专用的 *stmatrix* 要到 *Hopper* 才出现。

2.3 把 Ampere 片段写回去

mma.sync 算完之后, 累加器还是一个寄存器片段。收尾阶段 (epilogue, 矩阵乘算完后做缩放、写回等收尾工作的那一段) 要干的, 就是把它从片段里捞出来。

注意

注意这儿有个坑: *Ampere* 上压根没有 *ldmatrix* 的专用反向指令。所以 kernel 只能拿普通的 per-thread store 凑合, 有时还得搭上 warp shuffle 或者局部重排, 先把片段理顺了, 再写进 SMEM/GMEM, 凑成一个有用的布局。

这套模型, 好处是够简单, 代价是把一大摊子布局脏活全甩给了 kernel: 输入侧用 *ldmatrix* 造片段, 中间计算指令读写寄存器片段, 输出侧又靠普通 *store* 把片段一点点写出去。

2.4 Ampere 上的 swizzle

Ampere kernel 天生就甩不掉共享内存 swizzle。为什么? 一句话就能讲明白: 同一块 SMEM tile, 写进去是一种访问模式, 读出来又是另一种访问模式。这两种模式一打架, bank 冲突就找上门了。

来看个典型场景。从全局内存按行往 tile 里灌数据, 行主序 (row-major) 让这个写既能合并、又对 bank 友好, 看着挺美。可坏就坏在读这一步: *ldmatrix* 读这块 tile 的时候, 走的可能是沿着列, 或者横跨好几个 8×8 子 tile。在朴素行主序下, 这些读会一股脑全撞进同一个 bank。

给你算笔账就懂了——一块 (8, 64) 的 float16 tile, 它一行有多大:

$$64 * 2 = 128$$

128 字节, 恰好就是一整条共享内存 bank line。坏就坏在这儿: 沿着某个固定列往下走, 每跨一行就往前蹦 128 字节, bank 索引绕一圈又绕回原点。结果呢, 8 行全塌进同一个 bank, 变成 8 路冲突。

关键

关键你大概会想: “那我改成列主序不就完了?”——没那么便宜。改列主序通常只是把冲突挪了个地方: 列式读是好了, 可行式写又变差了。说白了就是按下葫芦浮起瓢。

真正管用的解法是 **XOR swizzle**: 让物理列的位置, 跟着行号一起动。最简单的一版长这样:


```
physical_col = logical_col XOR row
```

妙就妙在：逻辑 tile 半个字节没动，光是把物理摆放重排了一下，就让行式写和 Tensor Core 的读模式俩一块儿躲开了 bank 冲突。下面这张表，把三种方案摆一块儿对比一下：

方案	行式写（从 GMEM 填充）	列式读（给 ldmatrix）
朴素行主序	合并、bank 友好	塌到同一 bank,8 路冲突
朴素列主序	写变差	读变好
XOR swizzle	仍合并、bank 友好	散到多个 bank

沿同一列 col_c 往下读时，两种方案落到的 bank 对比如下：

访问	朴素行主序（每行 +128B,bank 索引循环重复）	XOR swizzle(physical_col = logical_col XOR row)
row0 col_c	bank k	bank (k XOR 0)
row1 col_c	bank k(冲突!)	bank (k XOR 1)
...	...	...
row7 col_c	bank k(8 路冲突)	散开，无冲突

图（等价于原书 *swizzle_conflict.svg*）：朴素行主序下行写横铺、列读撞 bank；XOR swizzle 在保住合并行写的同时，把列读打散到各 bank。

承上启下这里记住一点：在 Ampere 上，这套 swizzle 基本上全靠你手写共享内存的索引计算来实现。到了往后的世代，它会摇身一变，变成硬件引擎直接读的**描述符格式**里的一部分。这正是接下来 Hopper 最值得看的地方。

三、Hopper:wgmma、共享内存描述符与 swizzle 格式

Hopper 这代，动刀子动的是 Tensor Core 通路的**输入侧**。它不再死磕”每个操作数都得先用 ldmatrix 塞进寄存器”，而是让 wgmma 直接从共享内存里读操作数。

- **B 操作数**：从一个 SMEM 矩阵描述符读。
- **A 操作数**：既能从 SMEM 描述符读，也能从寄存器读——这就对应 .ss 和 .rs 两种形态。

注意

注意可别误会，这只是把”SMEM 来源操作数”那个显式的 ldmatrix 步骤给省了，布局要求一条没少。Tensor Core 照旧要求操作数按精确的 SMEM 格式摆好。真正变的是说明的方式：现在这个格式是通过矩阵描述符，直接说给硬件听的，不再埋在 kernel 手写的那堆索引计算里。

3.1 Hopper Tensor Core 期望什么

先讲讲这个描述符是干啥的。Hopper 的 **SMEM 矩阵描述符**，你就把它当成一块 SMEM 矩阵 tile 的一份小巧说明书。它的活儿，就是告诉 wgmma：逻辑操作数坐标该怎么换算成实际的 SMEM 地址。说明书里写着这么几个字段：

字段	作用
start address(起始地址)	tile 在 SMEM 的基址
leading dimension offset(主维偏移)	沿主维前进的步长
stride dimension offset(步维偏移)	沿另一维前进的步长
swizzle mode(swizzle 模式)	选定 atom 形状 + atom 内的 XOR 排列
base offset(基偏移)	附加偏移

关键

关键同样这几个字段，具体该怎么读，得看操作数的 **major mode / 主序**。对 **K 主序**的 tile，一个步长是沿 K 走、另一个沿 M 走；换成 **MN 主序**的 tile，这俩角色正好掉个个儿。

上面那个 **swizzle 模式**字段，取值就是下面这几种”SMEM 描述符格式”里挑一个：

SWIZZLE_NONE
SWIZZLE_32B
SWIZZLE_64B
SWIZZLE_128B

swizzle 模式一选，就**两件事**当场拍板：

- 1. 描述符用多大的 **atom 形状 (atom shape)**；
- 2. 在每个 atom 内部，到底施加哪一种 **XOR 排列**。

举个例子：**128 字节 swizzle 模式**会把操作数看成一张网格，格子是 **8 行 × 128 字节**的 atom,swizzle 就在每个 atom 内部各管各的。

结构图

连接关系：

- 选中某个 atom / (如 8 行 x 128B) $\xrightarrow{\text{swizzle mode / 选 atom 内字节位置}}$ 最终 SMEM 字节地址

图 (等价于原书 *smem_descriptor.svg*): 描述符的 *stride* 选 *atom*,*swizzle* 选 *atom* 内的字节位置。

3.2 数据是谁写进去的?TMA 与 wgmma 必须”对暗号”

描述符再聪明，字节也总得有人先按规矩码好。这活儿一般交给 **TMA / Tensor Memory Accelerator**(Hopper 起的硬件搬运引擎，专门成块地把数据从 GMEM 搬进 SMEM)，由它来把这块 SMEM tile 填上。这里有条要命的规矩：**TMA 用的 swizzle 格式，必须跟后面 wgmma 描述符用的一模一样**。这就跟对暗号一个道理：

- TMA 写进去的要是 128 字节的 swizzled tile，那 **wgmma** 描述符也得按 128 字节的 swizzled tile 来读。
- 一旦**描述符和数据对不上**,Tensor Core 读到的就是一堆被打乱的操作数。

结构图

连接关系：

- SMEM tile / (128B swizzled) $\xrightarrow{\text{wgmma 描述符 (读) / 必须也 =128B}}$ Tensor Core

图:TMA 写入与 *wgmma* 读取必须命名同一种 *swizzle* 格式, 否则读到乱码。

这正是 Hopper 比 Ampere 最关键的那一步转变。*swizzle* 不再窝在某段手写的 SMEM 索引计算里了。Hopper 把它提拔成了一等公民——一种有名有姓的描述符格式: 写 tile 的 TMA load 和读 tile 的 *wgmma*, 引用的是同一份格式, 自然也就对得上暗号。

3.3 Hopper 的输出仍用寄存器

Hopper 虽然把输入路径翻新了, 可有一样东西没动: **累加器还是住在寄存器里**。

wgmma 把累加器写进 per-thread 寄存器片段。片段多大、占几个寄存器, 看指令形状——比如 `m64nNk16,N` 越大, 累加器吃掉的寄存器就越多。可骨子里跟 Ampere 没两样: 收尾阶段到手的, 还是一个寄存器片段。

所以 Hopper 是个混血儿——输入是新的, 输出是旧的:

阶段	Ampere	Hopper	变化
输入操作数	寄存器片段 (经 <code>ldmatrix</code>)	SMEM 描述符直读 (<i>swizzle</i> 由硬件描述)	改了
累加器/输出	寄存器片段	寄存器片段	没变

换句话说, 到了 Hopper 这代, **输出累加器仍然是个寄存器布局的活儿**。这个输出侧的故事, 得等到 Blackwell 才会被重写。

四、Blackwell:tcgen05 与 TMEM

数据操作数这一边,Blackwell 基本照搬了 Hopper 那套 SMEM 描述符的路子:A、B 还是在 SMEM 里按 Tensor Core 想要的布局摆好; 某些模式下,A 操作数 (operand, 送进 Tensor Core 参与运算的一块数据) 甚至能直接从 **TMEM**(Tensor Memory,Blackwell 新增、专给 Tensor Core 暂存累加器的一块片上内存) 读。

真正翻天覆地的变化, 落在累加器身上。搁以前, 累加器是个”成天泡在寄存器里”的片段;Blackwell 的 `tcgen05.mma` 干脆把它改成写进 **Tensor Memory / TMEM**:

- 计算阶段: 累加器就待在 **TMEM** 里。
- 收尾阶段: 再用 `tcgen05.ld` 把它从 TMEM 装回寄存器。

结构图

连接关系:

- TMEM / (累加器驻留于此) $\xrightarrow{\text{tcgen05.ld}}$ 寄存器片段 / (交给 epilogue)

图:Blackwell 把累加器的”家”从寄存器搬到了 *TMEM*。

这么一搬, 等于把**输出布局这个难题, 从寄存器挪到了 TMEM**。于是 kernel 现在多了几桩活儿要干:

1. 分配 **TMEM**;

- 2. 挑对 **TMEM** 布局;
- 3. 等 **MMA** 算完;
- 4. 走对应的 `tcgen05.ld` 路径, 把累加器片段取回来, 交给收尾阶段。

注意

注意至于 `cta_group::1` 和 `cta_group::2` 怎么把累加器拆到一个或两个 CTA(cooperative thread array, 也就是一个线程 block) 上, 那是《Tensor Cores: tcgen05》那章的事, 这里就不展开了。本章里跟前几代差得最远的, 其实是下面要讲的 block-scaled 缩放因子布局。

4.1 TMEM 里的缩放因子布局 (scale factor layout)

先说说这个缩放因子是从哪冒出来的。像 **mxfp8**、**nvfp4** 这类 block-scaled MMA 模式, 会平白多出一类**缩放因子操作数**。所以这时候除了 A、B,MMA 还得再读两块东西:

```
SFA(M, SFK)
SFB(N, SFK)
```

这里的 **SFK**, 说的是 K 方向上**缩放块 (scale block)** 一共有几个。

关键

关键一句话先记住: 数据操作数 A、B 住 **SMEM**, 缩放因子住 **TMEM**。住的地方不一样, 搬运的路线当然也跟着不一样。

那为什么搬运得分两步? 因为 **TMA** 只管从 **GMEM** 搬到 **SMEM**, 没本事直接往 **TMEM** 里写。所以缩放因子得绕个弯, 分两步走:

步骤	数据通路	说明
第一步	GMEM --TMA--> SMEM	TMA 从 GMEM 搬到 SMEM
第二步	SMEM --tcgen05.cp--> TMEM	再从 SMEM 拷进 TMEM

只有等这次拷贝干完, 缩放因子才算真到了位——也就是落进了 `tcgen05.mma` 指望读到它们的那块内存空间。

结构图

连接关系:

- SMEM $\xrightarrow{\text{tcgen05.cp}}$ TMEM
- TMEM $\xrightarrow{\text{读取}}$ `tcgen05.mma`

图: 缩放因子的两步搬运路径 (数据 A/B 是 *GMEM*→*SMEM* 一步直达, 缩放因子要多绕一步进 *TMEM*)。

4.2 缩放因子的 TMEM 坐标:warpx4 广播

缩放因子在 TMEM 里怎么摆,用的是 TMEM 自己那套硬件坐标 **Lane** 和 **Col**(在 TIRx 布局记号里写成 **TLane** 和 **TCol**)。

它的核心套路一句话就能说清: 把一份缩放数据先压实成一小份,再复制四份铺满整个空间。说细点,一个 **128 行**的缩放向量先被压实进一个 **32-lane** 组,然后原封不动复制到 TMEM 的四个 **32-lane** 窗口里。写成布局记号就是:

```
S[(32, sf_per_mma) : (1@TLane, 1@TCol)] + R[4 : 32@TLane]
```

符号不用抠那么细,对着这两项看就行:

- 分片项 (**shard**): 管的是摆好那份基础的 32 行组——**TLane = r,TCol = s**;
- 复制项 (**replica**): 管的是在 lane 偏移 0、32、64、96 这四个位置各放一份副本——**TLane = r + 32*q(q 取 0、1、2、3),TCol = s** 不动。

这套就叫 **warpx4 广播模式**: 同一份压实的缩放因子组,被复制到各处,这样整个 128-lane 的 TMEM 空间里,它哪儿都能看见。

同一个 32 行缩放组被广播 4 份,铺满 128-lane 的 TMEM 空间:

TMEM 窗口	TLane 0 ...31	32 ...63	64 ...95	96 ...127
内容	基组	副本	副本	副本
副本编号	q=0	q=1	q=2	q=3

图:warpx4 把压实的 32-lane 缩放组,复制到四个 32-lane 窗口,覆盖 128-lane TMEM 空间。

4.3 TCol 单元内部的字节打包 (scale_vec 模式)

广播管的是”放到哪些 lane”,可每个 32-bit 的 TCol 单元里头,还有一层更细的讲究——这 4 个字节里到底装了几个缩放值。这就看 **scale_vec** 模式怎么定了:

模式	4 字节单元内的打包方式
1X	一个缩放值,广播填满整个 32-bit 单元
2X	打包两个缩放值,每个各复制一份
4X	打包四个 K-block 缩放值

4 字节 (32-bit)TCol 单元内部,三种模式的字节摆放:

模式	字节 0	字节 1	字节 2	字节 3	含义
1X	S0	S0	S0	S0	一个值,广播
2X	S0	S0	S1	S1	两个值,各复制
4X	S0	S1	S2	S3	四个 K-block 值

图 (等价于原书 sf_scale_vec.svg):scale_vec 字节打包的三种模式。

注意

注意这种字节打包, 在 Ampere 和 Hopper 上根本找不到对应物。道理也简单: 那两代压根就没有给 tcgen05 block-scaled MMA 用的那种 TMEM 缩放因子操作数, 既然东西都没有, 自然也谈不上怎么打包。

4.4 cta_group::2 下缩放因子跟着数据走

到了 cta_group::2, 你记住一条原则就够了: 缩放因子永远跟着它要缩放的那块数据走。拆开看就是:

- **SFA** 缩放的是 A。A 既然按 M 维在两个 CTA 之间切开, 那 SFA 也跟着按 M 切, 好对上各个 CTA 手里的那些 A 行。
- **SFB** 缩放的是 B。B 既然被参与计算的两个 CTA 半区共用, 那 SFB 就多播 (multicast, 一份数据同时发给多个目标) 给这两个 CTA。
(想看更细的, 翻《Tensor Cores: tcgen05》。)

五、一个反复出现的片段:m8n8

三代一路讲下来, 周围的内存路径换了一茬又一茬, 可有个老面孔始终赖着没走:m8n8 风格的寄存器片段。只不过它每一代出场的地方不太一样。

世代	m8n8 片段扮演的角色
Ampere	ldmatrix 构建它, 好让 mma.sync 读取 (输入侧)
Hopper	wgmma 把累加器写成寄存器片段, 交给 epilogue(输出侧)
Blackwell	计算阶段累加器在 TMEM; 但 tcgen05.ld 在 epilogue 处理与存储之前, 把它装回寄存器片段

关键

关键片段从来没消失过, 变的只是它露脸的地方。早些的世代里, 累加器整个计算阶段都泡在片段里; 到了 Blackwell, 它基本只在 TMEM 和收尾阶段交接的那个当口才冒一下头。

六、贯穿三代的主线 (The Throughline)

结构图

分组:

- **Ampere**: 「输入: 显式构建寄存器片段 / (ldmatrix)」 「swizzle: 主要靠 kernel 手写索引数学」 「累加器: 寄存器」
- **Hopper**: 「输入: SMEM 描述符直读 / (wgmma)」 「swizzle: 具名描述符格式 / (TMA 与 wgmma 共享)」 「累加器: 寄存器」
- **Blackwell**: 「输入: SMEM 操作数 (+ 部分 TMEM A)」 「累加器: 搬进 TMEM / (tcgen05.mma / tcgen05.ld)」 「block-scaled: 缩放因子经 TMEM 暂存」

连接关系:

- Ampere → Hopper

把三代横向对比成一张表:

维度	Ampere	Hopper	Blackwell
主力 MMA 指令	<code>mma.sync.aligned.m16n8k*</code>	<code>wgmma(.ss/.rs)</code>	<code>tcgen05.mma</code>
输入操作数来源	寄存器片段 (经 <code>ldmatrix</code>)	SMEM 描述符直读	SMEM 描述符 (+ 部分 TMEM A)
swizzle 谁负责	kernel 手写索引数学	硬件描述符格式 (TMA + wgmma 共享)	硬件描述符格式
累加器位置	寄存器	寄存器	TMEM
输出取回	普通 <code>store</code>	epilogue 读寄存器片段	<code>tcgen05.ld</code> 从 TMEM 取回
缩放因子 (block-scaled)	无	无	经 <code>tcgen05.cp</code> 暂存到 TMEM

关键

核心论断千万别以为有了描述符就高枕无忧了。描述符根本没替你把布局这活儿免掉，它只是把契约写清楚了而已。kernel 该兜的底还得自己兜：数据搬运路径、内存布局、Tensor Core 指令，这三样必须**严丝合缝地对齐**。写 swizzled SMEM tile 的那个 TMA 描述符、读这块 tile 的那个 MMA 描述符、还有挂在 buffer 上的那个布局——它们说的得是同一种物理摆放，一个字都不能差。

而且最要命的一点是：就算有一环没对上，硬件也**照跑不误**，一声不吭，不报错。它就默默地给你读错字节，或者慢吞吞地读。正因为这样，**布局压根不是 Tensor Core kernel 上面可有可无的装饰**，它本身就是指令接口 (instruction interface) 的一部分。

小结

- Tensor Core 的”高层语义”三代都没动 (还是 $D = A \cdot B + C$)，可操作数进出走哪条道、累加器最后落在哪儿，每代都在变。而这恰恰决定了你的 kernel 算得对不对、跑得快不快。
- Ampere: 一切围着**寄存器片段**打转。`ldmatrix` 造片段、`mma.sync` 算、普通 `store` 写回;swizzle 靠手写索引计算 (典型就是 `physical_col = logical_col XOR row`) 来躲 bank 冲突。
- Hopper:`wgmma` 改成拿 **SMEM 矩阵描述符**直接读操作数,swizzle 也升级成了有名有姓的描述符格式 (TMA 写和 `wgmma` 读必须对上暗号); 不过累加器还窝在**寄存器里**。
- Blackwell: 输入侧接着用 SMEM 描述符,但**累加器搬进了 TMEM**(`tcgen05.mma` 写、`tcgen05.ld` 取回);block-scaled MMA 的缩放因子得经 `tcgen05.cp` 分两步挪进 TMEM, 还用上了 `warpX4` 广播 + `scale_vec` 字节打包这套独门布局。
- 两条老约束——**全局内存合并和共享内存 bank 冲突**——从头到尾都在。三代怎么演变,说到底就是把”**第三条约束 (Tensor Core 操作数布局)**”从 kernel 的手工活,一点点交给硬件描述符契约去管。描述符让契约更清楚了,可一分钱没替你出: 布局,自始至终都是指令接口的一部分。

延伸阅读

- 原文 (英文):[Tensor Core Operand Layouts Across GPU Generations](#)
 - 相关章节 (本书): 《数据布局 (Data Layout)》——本章布局记号的来源; 《Special Memory: TMEM》——Blackwell TMEM 细节; 《Tensor Cores: tcgen05》——cta_group::1/2 累加器拆分与缩放因子多播细节。
-

术语对照

中文	English
矩阵乘累加	MMA / matrix-multiply-accumulate
寄存器片段	register fragment
线程束 / 通道	warp / lane
共享内存	SMEM(shared memory)
全局内存合并访问	global memory coalescing
共享内存 bank 冲突	shared memory bank conflict
地址混洗	swizzle
矩阵描述符	matrix descriptor
主序 (K 主序 / MN 主序)	major mode(K-major / MN-major)
张量内存	TMEM(Tensor Memory)
缩放因子 (块缩放)	scale factor(block-scaled)
收尾阶段	epilogue
协作线程阵列	CTA(cooperative thread array)
张量内存加速器	TMA(Tensor Memory Accelerator)
多播	multicast

第 05 章 · 异步数据搬运:TMA

原文:[Async Data Movement: TMA](#)

本章要点

本章要点 (TL;DR)

- TMA(Tensor Memory Accelerator / 张量内存加速器) 就是一个专门搬数据的硬件引擎: 你只要派「一个线程」喊一嗓子, 引擎就在后台把一整块矩形 tile 在全局内存 (GMEM) 和共享内存 (SMEM) 之间搬好。搬的时候不挡道, 整个 CTA 该干嘛干嘛。
- 整件事靠一份「张量映射描述符 (tensor-map descriptor)」说了算: 形状、步长、每个元素几个字节、tile 多大, 还有最关键的 **swizzle(交织) 模式**, 全写在里头。
- 加载 (load) 的时候,TMA 顺手就把 **swizzle** 做了: 字节往 SMEM 一落, 就已经摆成 Tensor Core 爱吃的样子, 省掉单独再重排一遍。前提是描述符、SMEM 布局、后面的 MMA 三家对布局得说好同一套规矩。
- 怎么知道搬完了? 加载靠 **mbarrier**(还会盯着字节数), 存储 (store) 靠 **commit group** 加 **wait group**——这俩管的是不同的「交接点」。
- TMA 真正的看家本领是**流水线 (pipelining)**: 让引擎在后台先把「下一块 tile」搬进来, 同时让 Tensor Core 在前台算「这一块 tile」, 中间用 **barrier** 把两边对上。

前置知识

前置知识: 读这一章前, 最好先懂 GMEM/SMEM(显存与片上共享内存)、tile(把大矩阵切出来的小方块)、GEMM(通用矩阵乘)、还有 **mbarrier**(异步屏障)。没把握的话, 先翻一下第 0 章 · 极简入门,mbarrier 的细节可以看第 7 章。本章会默认你已经认识这些词。

0. 为什么需要 TMA: 从「线程自己搬」说起

想搞懂 TMA, 咱们先看看没它的时候活是怎么干的。

先问一句: 一个 GEMM 或者注意力 (attention) 核函数, 跑得快不快, 到底卡在哪儿? 卡在 **Tensor Core**(GPU 里专门算矩阵乘的硬件单元, 见第 0 章) 有没有数据吃。流水线一旦填满, 这类核函数基本都是**计算受限 (compute-bound)** 的, 也就是算力已经榨干了。可这有个前提——下一块操作数 (operand, 即参与矩阵乘的输入数据, 如 A、B 矩阵)tile 得及时端上来。端晚了,Tensor Core 就只能干瞪眼空转。

过去的做法是让线程自己去搬 tile。每个线程算好地址, 从 GMEM 发 load, 再把值 store 到 SMEM。能用是能用, 但暗里要付两笔账:

1. **指令预算白花了:** 本该拿去算数的 warp(一个 warp = 32 个线程的小班, 见第 0 章) 指令, 现在全耗在算地址、做拷贝这些「跑腿记账」上。
2. **把计算 warp 的指令流搅浑了:** 这批 warp 本来一门心思该喂 Tensor Core, 结果被迫掺进一大堆搬运指令。

关键

关键: 这两点凑一块, 问题就清楚了——「搬数据」和「做计算」在抢同一份硬件 (指令发射、寄存器——线程私有的高速小存储, 见第 0 章)。TMA 的招数很干脆: **把搬运整个外包给一个专职硬件引擎**, 计算 warp 腾出手来, 专心算就行。

下面这张图把两条路摆一块比一比:

结构图

分组:

- **旧方式: 线程自己搬:**「每个线程 / 算地址 + load + store」「SMEM」「占用 warp 指令 / 占用寄存器 / 污染计算指令流」
- **TMA 方式: 硬件搬:**「一个线程 / 发起一次拷贝指令」「TMA 引擎 / (异步搬整块 tile)」「SMEM」「其余线程继续干活 / 不占计算指令流」

连接关系:

- TMA 引擎 / (异步搬整块 tile) → SMEM

注意

注意: 原文这儿有个交互演示 (*TMA copying a tile from global memory to shared memory*), 你能切换 **swizzle** 模式, 把鼠标停在源单单元格上, 看它最后落到 SMEM 的哪个位置。静态笔记没法还原这种交互, 想直观感受就去翻原书; 后面我会用 ASCII 和 Mermaid 图把要点画给你看。

1. 一个线程发起, 硬件搬整块 tile

1.1 发起线程做什么、不做什么

一次 TMA 拷贝, 是从一个**发起线程 (issuing thread)**开头的。但注意, 这线程**不会**去一个个遍历 tile 里的元素。它干的活更像是「下单」——递给硬件一份「拷贝说明书」, 真正搬运的力气活, 全甩给 TMA 引擎。

这份「说明书」的主体, 就是 **张量映射描述符 (tensor-map descriptor)**。它写清了这么几件事:

描述内容	含义
tensor shape	全局张量的形状
strides	各维度步长 (用于在 GMEM 中寻址)
element size	单个元素的字节大小
tile shape	要读取的 tile 形状
swizzle mode	写入 SMEM 时采用的交织模式

光有描述符还不够。真要发起一次拷贝, 还得现补上两个「运行时坐标」:

- 这回从全局张量的哪儿取 tile(也就是 tile 坐标 / tile coordinates)。
- 取来的 tile 放到 SMEM 的哪个地址。

这么理解就顺了：描述符回答的是「这张量长啥样、怎么排的」，坐标回答的是「这一次到底搬哪块、搬哪去」。前者一锤定音、固定不变，后者每次都可能不一样。

指令一发出去，拷贝就**异步**跑起来了。发起线程接着往下走它的CTA(一个线程块 block，即一组协作线程，见第 0 章)里其他线程也不用停下来等。这会儿搬运全归 TMA 引擎管了，不再是一串普普通通的 load/store 在那儿循环。

1.2 同一个逻辑操作，两条实现路径

「把这块 tile 拷过去」这件事，核函数现在有两种写法：

结构图

连接关系：

- 逻辑操作：「拷贝这块 tile」→ 线程拷贝 (thread copy) / 线程协作 load/store
- 逻辑操作：「拷贝这块 tile」→ TMA 拷贝 (TMA copy) / 一个线程发起，硬件搬运
- 线程拷贝 (thread copy) / 线程协作 load/store → 优点：对每次访问有直接控制 / 缺点：吃线程指令和寄存器
- TMA 拷贝 (TMA copy) / 一个线程发起，硬件搬运 → 优点：适合大而规整的 tile / (Tensor Core 操作数 tile 的天然选择)

这两条路，同步的规矩不一样，跑出来的性能也不一样。到底走哪条，这件事本身就是一个分发 (dispatch) 决策。原文把整件事拆成三个互不打架的概念，这么一拆就清楚多了：

- 布局 (layout)：核函数想要内存排成啥样。
- 作用域 (scope)：哪些线程、哪些 CTA 来掺和这次拷贝。
- 分发 (dispatch)：这次拷贝用普通线程代码做，还是丢给 TMA 做。

2. Swizzle 布局：不只是「搬到」，还要「摆对」

2.1 为什么搬到位还不够

把 tile 搬进 SMEM，其实只算干完一半。为啥？因为 Tensor Core 不光要 SMEM 里装的是**对的值**，还要这些值**摆成对的样子**。摆得不对，读起来就慢——最常见的坑就是 SMEM bank 冲突 (SMEM 被切成多个存储体即 bank，多个线程同时挤同一个 bank 就被迫排队，见第 0 章)。

这时候 **TMA swizzle(交织)** 就上场了。TMA 往 SMEM 写 tile 的同时，能顺手把 **SMEM 的地址模式重排 (permute)** 一下。逻辑上看，这 tile 还是规规矩矩一个矩形；可它在 SMEM 里到底落在哪些格子，早就被交织打散了。

关键

关键：swizzle 用哪种模式，本身就写在描述符里。描述符配好了，发起线程根本不用自己动手 **swizzle**——字节落进 SMEM 那一刻，引擎已经替你交织妥当了。

2.2 「一致约定」是硬性前提

swizzle 是省事，但有一条铁律不能破：**三家对布局得统一口径**。哪三家？看下图。

结构图

这条铁律有多硬？你想想：TMA 拿一种 **swizzle** 把数据写进去，MMA(矩阵乘累加，Tensor Core 执行的核心指令) 却拿另一种 **swizzle** 去读。这时候硬件才**不错**，它就老老实实照你说的做。结果呢？字节全摆错位，算出来的东西跟着错，而且一声不吭——半点提示都没有。

举个例子。假设核函数声明某个操作数 **tile** 按 **128 字节 swizzle 布局**来存，那 TMA 描述符就得用对得上的 **swizzle 模式**，MMA 分发也得照着同一套 SMEM 排布去预期。这正是布局记号 (layout notation) 的价值——它可不只是个「记账的」：DSL 里写的布局，必须跟 TMA 描述符、跟 Tensor Core 指令用的硬件布局，**严丝合缝对上**。

换个角度想：**swizzle 不动逻辑 tile**。后面 MMA 吃的，还是同一个逻辑 A/B tile。swizzle 只管这 tile **怎么跨着 SMEM bank 在物理上摆**。一句话：它改的是「某个逻辑元素摆在物理哪个位置」，不是「这元素本身是啥」。

3. 3D TMA: 在一次拷贝里同时完成「分块」和「swizzle」

3.1 问题:Tensor Core 想要的是「分块进 swizzle 原子」的布局

普通的 2D TMA 拷贝，搬的是一块**扁平的 2D tile**。可 Tensor Core 想要的 SMEM 布局往往没这么省心——它要 tile 被一个个**塞进 swizzle 原子 (atom)** 里。这里说的原子，就是《数据布局及其记号》那章讲过的 8×128 字节原子。

扁平 tile 跟「塞进原子」的布局对不上，咋办？TMA 的解法挺妙：**给描述符再加一个维度**。

3.2 3D TMA 的寻址方式:(group, row, col)

3D TMA 把 SMEM 这块地方当成一个三维盒子，用 (group, row, col) 来点位置：

- **group**: 走到第几个原子 (就是在一个个原子之间跨步)。
- **row / col**: 定位**某个原子内部**的具体位置。

有了这三维，一次 **3D 拷贝**就能一口气把两件事都办了：

1. **分块 (tile)**: 把 tile 一个原子一个原子地铺开。
2. **swizzle**: 在每个原子里头做交织。

最妙的是落地的结果：数据搬进 SMEM 的时候，**已经是 MMA 想要的布局了**，不用再额外跑一趟分块或者 swizzle。

结构图

连接关系：

- 3D TMA 拷贝 / 寻址 = (group, row, col) $\rightarrow$ atom 0 / (内部已 swizzle)
- 3D TMA 拷贝 / 寻址 = (group, row, col) $\rightarrow$ atom 1 / (内部已 swizzle)
- 3D TMA 拷贝 / 寻址 = (group, row, col) $\rightarrow$ atom 2 / (内部已 swizzle)
- atom 0 / (内部已 swizzle) $\rightarrow$ SMEM: 已是 MMA 期望布局
- atom 1 / (内部已 swizzle) $\rightarrow$ SMEM: 已是 MMA 期望布局
- atom 2 / (内部已 swizzle) $\rightarrow$ SMEM: 已是 MMA 期望布局

注意

注意：原文配了个交互演示 *a 3D TMA copy, addressed as (group, row, col), tiling into swizzled shared memory*。想直观体会一下这个寻址过程，去原书点点看。

3.3 swizzle 格式的选择：取决于 tile 能不能「填满」原子

那到底用哪种 swizzle 格式呢？这事跟分块是绑死的。背后就两条道理：

- swizzle 越宽，越能把一列数据摊到更多 bank 上，读起来越不容易撞车。所以只要塞得下，128 字节 swizzle 永远是默认首选。
- 但是，N 字节的原子，要求 tile 的「连续维度」得把它填满。要是某个 tile 因为形状受限太瘦、填不满 128 字节，那 128 字节 swizzle 就用不成了，只能往下退，退到 64 字节，甚至 32 字节。

经验法则：在 tile 能填满的前提下，挑最大的那个 swizzle。

原文举了个特别直观的例子：拿一个 16×16 tile 直接铺 128 字节 swizzle，会撞车；非得把它切成对得上原子的 16×8 分组，才能做到无冲突 (conflict-free)。

下面用个简化的 ASCII 图，帮你建立「填满 vs 填不满」的直觉：

结构图

连接关系：

- 连续维度 / (字节数) $\xrightarrow{\text{填满 128 字节}}$ 128B swizzle / (散布到最多 bank，默认)
- 连续维度 / (字节数) $\xrightarrow{\text{只填 64 字节}}$ 降级到 64B swizzle
- 连续维度 / (字节数) $\xrightarrow{\text{只填 32 字节}}$ 降级到 32B swizzle
- 16×16 tile + 128B swizzle $\xrightarrow{\text{直接铺}}$ 有 bank 冲突
- 16×16 tile + 128B swizzle $\xrightarrow{\text{切成 } 16 \times 8 \text{ 分组 (匹配原子)}}$ conflict-free

swizzle 格式	适用场景	bank 散布
128 字节	连续维度能填满 128B (默认首选)	最广
64 字节	tile 偏小、填不满 128B 时降级	中等
32 字节	tile 更小时进一步降级	较窄

4. 完成机制 (一)：加载用 mbarrier

4.1 为什么「发出指令」不等于「可以读」

别忘了，拷贝是异步的。这就意味着：指令发出去，数据可不一定到位。消费者不能一看 TMA 指令发出去了，就急吼吼跑去读 SMEM tile——得等引擎真把字节写完了，这 tile 才安全、才能读。

那怎么知道引擎写完没？对 TMA 加载来说，报「干完了」这个信儿的，是 mbarrier (细节看《异步协调:mbarriers》那章)。

4.2 标准时序

时序图 (按时间顺序)

1. 备注:1. 初始化/复用流水线阶段的 *mbarrier*
2. 发起线程 → *mbarrier*:2. *mbarrier.arrive.expect_tx(bytes)* / (告知预期字节数 + 完成自身 *arrive*)
3. 发起线程 → TMA 引擎:3. 发起 TMA *load*
4. TMA 引擎 → *mbarrier*:4. 字节到达时做 *complete-tx* 更新
5. 备注: 满足两条件才翻转 *phase*: / *arrival* 计数满足且待传字节数归零
6. 消费者 (MMA 路径) → *mbarrier*:5. 等待 *barrier* 的目标 *phase*
7. 备注: 等待完成 → *SMEM tile* 就绪, MMA 可安全读取

4.3 关键 API:*mbarrier.arrive.expect_tx*

设置字节数, 用的就是这么一条操作:

```
mbarrier.arrive.expect_tx(bytes)
```

这一条指令, 一下子干了两件事:

1. 记下这次预期要传多少字节;
2. 顺带替发起线程在 *barrier* 上完成 *arrive*(到达)。

关键

关键: 可别误会, 调了这条指令并不等于 *barrier* 就完成了。它还得等 TMA 引擎回话:「该到的字节都齐了」。

barrier 的 **phase(相位)** 非得两个条件都凑齐才翻转:

1. *arrival* 计数到位;
2. 待传字节数 (*pending byte count*) 清零。

接下来消费者就守在这个 *barrier* 上等。一等到目标 *phase*, 就说明 *SMEM tile* 已经备好了, 这时候 MMA 路径才能放心去读。

原文这儿还配了张同步流程图 (*tma_sync_flow.png*), 意思跟上面那张时序图一模一样: 生产者 是 TMA 引擎, 消费者是 MMA 路径 (或者随便哪段要读 *SMEM tile* 的代码), *barrier* 就是横在它俩中间那个说一不二的交接点。这跟别处异步「生产者-消费者」交接, 用的是同一套 *barrier* 模型。

5. 完成机制 (二): 存储用 *commit group* + *wait group*

5.1 为什么 *store* 的完成机制不一样

TMA 存储正好是倒过来: 把数据从 *SMEM* 搬回 *GMEM*。它一样是异步的, 可报完成的那套机制换了。为啥换? 根子上是「交接点」变了——看下面这张对照表就懂了。

	TMA 加载 (load)	TMA 存储 (store)
数据方向	GMEM → SMEM	SMEM → GMEM
典型场景	喂给同核函数内的消费者 (MMA)	把最终结果写出到 GMEM
核函数关心什么	SMEM tile 何时就绪可读	何时可以安全复用 SMEM 缓冲区 / 确认搬运排空
完成机制	mbarrier(带字节计数)	commit group + wait group

说白了就是俩边操心的东西不一样。加载是要把一个 SMEM tile 亮给后面的消费者看, 所以得靠 mbarrier 精确地通知一声。存储则是把最终数据写出去, 一般压根没有哪个核内消费者在等它——核函数真正惦记的是另一码事:「啥时候能安全地把 SMEM 缓冲区拿回来重用, 或者说这串存储到底啥时候算完」。

5.2 用法

用起来是这么个流程: 核函数发出一个或者好几个 store, 把它们提交 (commit) 成一组, 过一会儿再来等 (wait) 这一组排空 (drain)。等待一返回, 在核函数看来这组 store 就全干完了, 它们占的那块 SMEM 也就能放心拿回来重用了。

原文拿一段很精炼的对照, 把这两套机制摆在一块总结:

TMA load:	mbarrier
TMA store:	commit group wait group

理解: 这两套机制说到底干的是同一件事, 只不过交接点不一样。加载是让一个 SMEM tile 对后面的消费者亮相; 存储是确认往外的传输真的完事了——这样核函数才敢重用源头那块存储, 或者拿「store 已经排空」这个事实接着往下走。

6. 为什么 TMA 对流水线 (pipelining) 至关重要

前面铺垫了这么多,TMA 真正大显身手的时刻, 是它当上流水线里一环的时候。

核心思路一句话就够:Tensor Core 正算着「当前 tile」, 核函数就抢先把「下一块 tile」的加载发出去。加载在后台闷头跑, 计算在前台连轴转, 等「下一块 tile」终于轮到被算了,barrier 就负责把这两条线接上。

一个典型的 GEMM 循环就翻来覆去用这套结构, 它有个名字, 叫多级缓冲 (multi-stage buffering):

结构图

分组:

- SMEM 多级缓冲:「Stage 0 / 正被 MMA 消费」「Stage 1 / 正被 TMA 填充」

连接关系:

- Stage 0 / 正被 MMA 消费 → Tensor Core (MMA) / 前台: 算当前 tile
- TMA 引擎 / 后台: 搬未来 tile → Stage 1 / 正被 TMA 填充

循环每往前走一步, 各个 stage 的角色就轮一次班 (rotate)。规矩就两条:

- **MMA 想读某个 stage 之前**, 先等这个 stage 的 load barrier(确认数据齐了);
- **TMA 想覆盖某个 stage 之前**, 先确认上一个消费者已经用完它了 (别把人家还没读的数据给盖了)。

关键

关键: 这也就讲明白了, 为啥在 Blackwell、Hopper(NVIDIA 较新的两代 GPU 架构,Hopper 即 H100 那代,Blackwell 更新) 风格的核函数里,**TMA 和 mbarrier 几乎总是成对出现**。TMA 给的是异步拷贝引擎, 管搬;mbarrier 给的是**精确判断「字节啥时候就绪」**的本事, 管通知。一个搬一个通知, 缺哪个都转不起来。

小结

- **TMA 就是个硬件拷贝引擎:** 一个线程喊一声、硬件异步把整块矩形 tile 搬好, 把搬运彻底从计算 warp 的指令流里拎出去,Tensor Core 就不会被搬运拖后腿了。
- **张量映射描述符是核心那层抽象:** 形状、步长、元素大小、tile 形状、swizzle 模式全装在里头; 它把「一次拷贝」从「一长串 load/store 循环」拎成「一份说明书 + 喊一声发起」。
- **swizzle 在加载路上顺手就把布局重排了:** 数据落进 SMEM 那一刻, 已经摆成 MMA 想要的样子。但描述符、SMEM 布局、MMA 这三家必须对布局说好同一套规矩, 不然硬件会「正确地」给你算出错的结果。
- **3D TMA (group, row, col)** 一次拷贝里就把「按原子分块」和「原子内 swizzle」一块办了;swizzle 格式 (128/64/32 字节) 选哪个, 看 tile 的连续维度能不能填满对应原子, 经验法则则是挑能填满的最大那个。
- **报完成分两套:** 加载用带字节计数的 mbarrier(arrival 到位且字节清零,phase 才翻转), 存储用 commit group + wait group。它俩对着的是不同的交接点。
- **TMA 的终极价值落在流水线上:** 前台算、后台搬、barrier 把两边对上——这正是现代 GEMM、attention 核函数能把 Tensor Core 喂得满满当当的关键。

延伸阅读

- 原文:[Async Data Movement: TMA —Modern GPU Programming for ML](#) Sys
- 相关章节 (原文中引用): 《What Makes a Kernel Fast》(核函数性能受限因素)、《Data Layout and Its Notation》(数据布局与 swizzle 原子)、《Async Coordination: mbarriers》(mbarrier 的异步协调模型)。

术语对照

中文	English
张量内存加速器	Tensor Memory Accelerator (TMA)
张量映射描述符	tensor-map descriptor
tile 坐标	tile coordinates
交织 (模式)	swizzle (mode)
swizzle 原子	swizzle atom
共享内存	shared memory (SMEM)

全局内存	global memory (GMEM)
异步屏障	mbarrier
相位	phase
字节计数追踪	byte-count tracking
提交组 / 等待组	commit group / wait group
矩阵乘累加	MMA (matrix multiply-accumulate)
协作线程阵列	CTA (Cooperative Thread Array)
通用矩阵乘	GEMM
计算受限	compute-bound
流水线	pipelining
多级缓冲	multi-stage buffering
bank 冲突	bank conflict
分发 (决策)	dispatch
作用域	scope

第 06 章 · 特殊内存:TMEM(Tensor Memory)

原文:Special Memory: TMEM

本章要点

本章要点 (TL;DR)

- **TMEM(Tensor Memory, 张量内存)** 是 Blackwell 架构才有的一块片上内存, 专门伺候新一代 Tensor Core 指令族 `tcgen05`。每个 SM 上一块, 长成 **128 个 Lane(行) × 最多 512 个 Col(列)** 的二维便笺 (scratchpad), 每个 Col 宽 32 位。
- 它存在的头号任务, 就是给寄存器减负: 以前 (Hopper 及更早) MMA 的累加器 (accumulator) 得全程赖在寄存器里; Blackwell 让 `tcgen05.mma` 把累加器直接写进 TMEM, 这样不挤爆寄存器也能开更大的 Tensor Core tile。
- **TMEM 靠「二维坐标 (Lane, Col)」寻址**, 不是共享内存那种一个线性字节偏移就完事。在本书的 TIRx 布局记号里, 这两根硬件轴写作 `TLane` 和 `TCol`。
- **TMEM 得自己显式分配、显式释放** (每个 CTA、按 32 列一档), 它不像寄存器那样编译器替你派好——把它当成跟共享内存 (SMEM) 一样、需要「精打细算做预算」的资源。
- 普通的 `ld.shared/st.shared` 根本碰不到 **TMEM**。数据想进想出, 只能走专用、而且异步的 `tcgen05` 指令: `tcgen05.ld(TMEM → 寄存器)`、`tcgen05.st(寄存器 → TMEM)`、`tcgen05.cp(SMEM → TMEM)`, 还得配上各自的完成/等待机制。

前置知识

前置知识: 读这一章前, 最好先懂寄存器 vs 共享内存、MMA / 累加器、warpgroup、Tensor Core 这几个概念。没把握的话, 先翻一下第 0 章 · 极简入门, 也可随时查术语对照表。本章会默认你已经认识这些词。

引子: 寄存器不够用, Blackwell 才另开了一条路

要搞懂 TMEM 为什么会存在, 咱们先问一个老问题: **矩阵乘算出来的累加结果, 到底该放哪儿?**

在 Hopper 和更早的 GPU 上, 答案很简单——放**寄存器里**。Tensor Core(专做矩阵乘累加的硬件单元)的累加器 (accumulator, 也就是矩阵乘累加结果的暂存) 就待在寄存器里, 整个流程也好理解:

1. MMA(矩阵乘累加 Matrix-Multiply-Accumulate) 指令一算完, 就吐出一个**寄存器片段 (register fragment, 累加器在每个线程寄存器里的那一份)**;
2. 接下来整个计算阶段 (compute phase), 内核就让这个片段一直赖在寄存器里;

3. 等到收尾的 `epilogue`(收尾阶段, 负责类型转换/逐元素/写回) 阶段, 内核再把片段读出来, 做类型转换 (`cast`)、做逐元素操作, 最后写回去。

这一套听着挺顺。可问题来了: **寄存器又少又金贵, 而且是一个线程一个线程死死分好的**。这就埋下了一个绕不过去的矛盾。

关键

关键:Tensor Core 就好「大 tile」(tile = 从大矩阵切出来的小方块) 这一口——tile 开得越大, 它越能跑满、吞吐越高。可 tile 一大, 累加器片段也跟着膨胀。片段一膨胀, 它在每个线程的寄存器里就占得越多, 把线程本来要用的其他变量全挤出去了。说白了, 「想用大 tile 提吞吐」和「累加器全塞寄存器」这两件事, 直接顶上了。

这个矛盾, 画张表就一目了然。寄存器文件每个线程的容量是死的, 同一块寄存器文件, 在大小两种 tile 下被占成什么样, 对比一下:

场景	累加器片段占用	留给其他工作变量的空间	结果
小 tile	较小	充足	舒服
大 tile	几乎占满整个寄存器文件	被严重挤压	溢出 / 降低占用率

Blackwell 就是冲着这个矛盾下的刀。思路很直接: 既然寄存器装不下大累加器, 那干脆别让累加器一直占着寄存器。所以 `tcgen05.mma` 不再把累加器留在寄存器里, 而是直接写进一块新加的片上内存——也就是 **TMEM**。这块内存, 早期的 NVIDIA GPU 根本没有。

这么一改, 好处立竿见影:**Blackwell 能开更大的 Tensor Core tile 了, 还不用拿每个线程的寄存器去硬扛整个累加器**。那代价呢?TMEM 可不像寄存器那样白送给你。编译器不会顺手帮你管好它, 这些活儿得内核自己干:

- 自己去**分配 (allocate)** TMEM;
- 用**对的布局 (layout)** 去寻址;
- 用**对的指令**把数据搬进搬出;
- CTA 用完了, 记得把它**释放 (free)** 掉。

下面我们一条一条来看。

TMEM 是一个二维地址空间

先记一句话:**TMEM 不是一条扁平的字节数组, 它是一张二维网格**。共享内存那套「给个线性字节偏移就能定位」的玩法, 在这儿行不通。硬件给 TMEM 的两根坐标起了名字, 一根叫 **Lane**, 一根叫 **Col**:

维度	含义	规模	备注
Lane	行 (纵向)	128 个 Lane 行	对应 TIRx 记号里的 TLane 轴
Col	列 (横向)	最多 512 个 Col 列	每个 Col 是一个 32 位 的列, 对应 TIRx 记号里的 TCol 轴

这个形状可不是拍脑袋定的:`tcgen05.mma` 往里写累加器, 用的就是这套二维结构。所以你想定位 TMEM 里的某个位置, 得给一对坐标——一个 Lane、一个 Col, 而不是一个字节偏移。

下面这张表就是这张二维网格 (对应原书的 `tmem_grid` 图): 纵着看是 Lane(128 行, 对应 TIRx 的 `TLane` 轴), 横着看是 Col(最多 512 列, 对应 `TCol` 轴, 每个 Col 宽 32 位)。每个格子, 就是一个 (Lane, Col) 坐标。

	Col 0	Col 1	Col 2	...	Col N-1
Lane 0				...	
Lane 1				...	
Lane 2				...	
...				...	
Lane 127				...	

- 竖着的行:128 个 Lane, 也就是 `TLane` 轴。
- 横着的列: 最多 512 个 Col, 也就是 `TCol` 轴, 每个 Col 宽 32 位。

在 TIRx 布局记号里怎么写

在 **TIRx**(也就是本书用的那套张量 IR / DSL) 里声明一块 TMEM 缓冲区, 得给它配一个能把这两根硬件坐标说清楚的布局。在布局记号里 (完整定义在原书「Data Layout and Its Notation」那一章), 我们把 TMEM 的 Lane 轴叫 `TLane`, Col 轴叫 `TCol`。

注意

注意:TLane / TCol 不过是布局轴的名字, 并不是要替换官方硬件术语 Lane / Col。起这两个名, 纯粹是想在 DSL 里把 TMEM 的两个维度写明白, 顺便跟其他张量布局用上同一套写法。

举个例子, 一个累加器 tile 可以这么写:

```
S[(128, N) : (1@TLane, 1@TCol)]
```

把这个记号拆开看就清楚了:

- `(128, N)`——这个 tile 有 **128 行** (沿 Lane 维度)、**N 列** (沿 Col 维度)。
- `(1@TLane, 1@TCol)`——这是步长 (**stride**): 往下走一行, 就在 `TLane` 轴上前进 1; 往右走一列, 就在 `TCol` 轴上前进 1。
- 合起来看: **这就是最直来直去的一种布局**——挨着的行沿 `TLane` 排, 挨着的列沿 `TCol` 排, 中间不搞任何重排、不搞交错。

关键

关键: 千万别把 TMEM 当成「Tensor Core 背后那个看不见的仓库」。它是整个 tile 布局里堂堂正正的一员: 内核得给它起名、从里头分配列, 还得用一个跟 `tcgen05` 指令读写方式严丝合缝的布局。布局一旦对不上, 数据就错位了。

分配: 把 TMEM 当成像 SMEM 一样要做预算的资源

内核要想用 TMEM, 得先里头占块地方。这点跟寄存器完全两码事: 寄存器是编译器替你分好的, TMEM 你得自己张口去要。

分配的规矩不多, 记住这几条就行:

- 以 CTA(线程块, 即一个 thread block) 为单位申请: 由 CTA 里某一个 warp(一个 warp = 32 个线程的小班, 见第 0 章) 出面, 要一段 TMEM 列区间。
- 32 列起步: 你要多少列, 硬件都会向上取整到 32 的倍数。
- 要完会给你一个基址 (base TMEM address): 往后所有 tcgen05 指令, 都靠这个基址找到你占下的那块地方。
- 用完务必释放 (free): CTA 结束的时候把地还回去。

想真正理解它, 最好的办法就是把 TMEM 当成和共享内存一个性质的、CTA 级别、得做预算的资源:

结构图

分组:

- 一个 CTA 的资源预算
- SMEM 预算 (必须塞进 SMEM 容量内): 「TMA 暂存的矩阵操作数」「中间缓冲」「...」
- TMEM 预算 (必须塞进 TMEM 容量内): 「MMA 累加器 (列)」 「block-scaled 缩放因子」「临时暂存 (staging)」

这么一来, TMEM 就成了内核做资源规划 (resource planning) 时绕不开的一项。具体怎么取舍, 都是很实在的账:

决策	收益	代价
用更大的累加器 tile	提升 Tensor Core 吞吐	消耗更多 TMEM 列
启用 block-scaled MMA(分块缩放)	支持更细粒度的量化缩放	需要额外 TMEM 空间放缩放因子

这些用途加到一块, 全都得挤进那点有限的 TMEM 预算里——道理跟你必须把所有共享内存缓冲塞进 SMEM(共享内存 Shared Memory, CTA 内片上共享的快内存) 预算里, 一模一样。

注意

注意: TMEM 的总量是有天花板的 (每个 SM(流式多处理器, GPU 上一个独立的计算核心) 就一块, 128 × 512 个 32 位列)。它和占用率 (occupancy)、SMEM 用量这几样凑在一起, 共同决定了一个内核能开多大 tile、能同时跑几个 CTA。

读写 TMEM: 三条专用的异步通路

前面那句话再敲一遍黑板: 普通的 ld.shared / st.shared 根本够不着 TMEM。TMEM 是块独立的地址空间, 数据想进想出, 只能走专门的 tcgen05 指令。这样的通路总共三条。

下图把 Blackwell Tensor Core 的整条数据通路都画出来了, 你一眼就能看到 TMEM 正好卡在中间:

结构图

连接关系:

- 全局内存 GMEM $\xrightarrow{\text{TMA}}$ 共享内存 SMEM
- 共享内存 SMEM $\xrightarrow{\text{tcgen05.cp} / (\text{批量拷贝缩放因子})}$ TMEM / 128 Lane × 最多 512 Col
- 共享内存 SMEM $\xrightarrow{\text{tcgen05.mma} / \text{读 SMEM 操作数, 累加写入 TMEM}}$ TMEM / 128 Lane × 最

多 512 Col

- 寄存器 Registers $\xrightarrow{\text{tcgen05.st} / (\text{寄存器} \rightarrow \text{TMEM})}$ TMEM / 128 Lane $\times$ 最多 512 Col
- TMEM / 128 Lane $\times$ 最多 512 Col $\xrightarrow{\text{tcgen05.ld} / (\text{TMEM} \rightarrow \text{寄存器})}$ 寄存器 Registers
- 寄存器 Registers $\xrightarrow{\text{epilogue:cast}/\text{逐元素}/\text{写回}}$ 输出到 GMEM

整条链路一句话串下来:**TMA**(Tensor Memory Accelerator, 负责 GMEM↔SMEM 异步大块搬运的硬件) 把矩阵操作数 (operand, 即参与矩阵乘的输入矩阵) 暂存进 SMEM → **tcgen05.mma** 读这些操作数, 把结果累加进 TMEM(要是 block-scaled, 缩放因子也搁 TMEM 里)→ 计算阶段一结束,**tcgen05.ld** 把累加器搬回寄存器 → **epilogue** 做转换, 把最终结果写出去。

通路一:**tcgen05.ld**(TMEM → 寄存器)

这条用得最勤,**epilogue** 必走这条。MMA 阶段把累加器算在了 TMEM 里, 可 **epilogue** 要的偏偏是一个**寄存器片段**——数据只有回到寄存器, 才方便做类型转换、逐元素操作、最后写回。

这条通路有个挺要紧的脾气: 它是「分着干」的。

关键

关键: 在 DSL 这一层, 一次 TMEM 加载是整个 **warpgroup**(4 个 warp 合成的更大协作单位, 共 128 个线程) 凑一块儿分头干的。它会被拆 (lower) 成四条 warp 级的 **tcgen05.ld**, 一个 warp 摊一条。每个 warp 管 128 个 Lane 行里的 32 行, 四个 warp 一拼, 正好把整个 Lane 维度 (也就是布局记号里的 TLane 轴) 铺满。

整个 TLane 维度一共 128 行, 一个 **warpgroup**(4 个 warp) 分着扛, 每个 warp 发一条 **tcgen05.ld**, 各管 32 行:

warp	负责的 Lane 行	发出的指令
warp 0	Lane 0 .. 31	tcgen05.ld
warp 1	Lane 32 .. 63	tcgen05.ld
warp 2	Lane 64 .. 95	tcgen05.ld
warp 3	Lane 96 .. 127	tcgen05.ld

这条指令本身还带着一大家子**加载形状 (load shape)**, 什么 .16x64b、.16x128b、.16x256b、.32x32b、.16x32bx2 等等, 外加一个**重复因子 (repeat factor)**, 从 .x1 一路到 .x128。你挑哪个形状, 就决定了这一趟读几列 TMEM、每个线程能分到几个寄存器。

不过最该上心的, 是读回来的**寄存器片段到底长啥样**。在常见的 **epilogue** 路径里, 规律是这样:

关键

关键:**lane 1**(线程在 warp 里的编号) 拿到的值, 来自 **TMEM 第 1 / 4 行的那两列**。

这有啥讲究? 讲究就在于: 它给出的形态, 跟早期 GPU 上 MMA 直接暴露的那种「一个 lane 一份累加器片段」长得一模一样 (细节见原书「Tensor Core Operand Layouts Across GPU Generations」)。这份前后一致, 可太值钱了。

注意

注意: 正因为片段的样子没变,Blackwell 的 epilogue 能直接捡起 Ampere mma 或 Hopper wgmma 时代那套「寄存器级 cast + 写回」的老代码接着用——哪怕计算阶段累加器其实是住在 TMEM 里的。说白了:TMEM 不过是给累加器换了个住处,可一旦搬回寄存器,上层软件几乎不用动。

把 1 / 4 这个映射画出来,大概是这个样子 (对应原书 tcgen05_ldst 那张图,m8n8 片段): 好几个 lane 共用同一个 TMEM 行,每个 lane 从这行里取 2 列 (也就是 2 个值) 进寄存器。

TMEM 行 (row = 1 / 4)	由哪些 lane 读取	每个 lane 拿到
row 0	lane 0, 1, 2, 3	该行的 2 列 (2 个值)
row 1	lane 4, 5, 6, 7	该行的 2 列 (2 个值)
row 2	lane 8 .. 11	该行的 2 列 (2 个值)
...	...	...

通路二:tcgen05.st(寄存器 → TMEM)

tcgen05.st 就是 tcgen05.ld 反过来: 线程手里已经攥着一个寄存器片段,想把它塞回 TMEM。

啥时候用得上? 比方说某些操作数或者中间值,得先在寄存器里过一道手 (staging),再写进 TMEM,留给后面某条 tcgen05 指令使。

通路三:tcgen05.cp(共享内存 → TMEM)

tcgen05.cp 走的是批量拷贝 (bulk copy) 这条路,干得最多的活儿就是搬 block-scaled MMA 的缩放因子。流程两步:

- 1. 先用 TMA 或者普通线程代码,把缩放数据 (scale data) 先备到共享内存里;
- 2. 再让 tcgen05.cp 把它搬成 Tensor Core 想要的那种 TMEM 布局。

三条通路有个共同点: 全是异步的

关键

关键:tcgen05.ld、tcgen05.st、tcgen05.cp 这三条全是异步 (asynchronous) 的。也就是说,指令很可能在数据还没真搬完的时候就先返回了。所以在你用这份结果、或者回头再用那块内存之前,一定得用对的完成机制把它等住 (细节见原书「Async Coordination: mbarriers」)。

这里特别容易踩坑: 到底怎么等,得看你用的是哪条指令。单拉个表:

指令	完成 / 等待机制
tcgen05.ld	通过 tcgen05.wait::ld 等待完成
tcgen05.st	通过 tcgen05.wait::st 等待完成
tcgen05.cp	像 tcgen05.mma 一样,通过 commit group(提交组)+mbarrier(异步屏障,用来等异步操作完成的同步对象)完成

还有一点别忘了:

注意

注意: 要是数据得从一组线程交到另一组线程手上 (producer → consumer), 那光「等」还不够, 内核多半还得补一道栅栏 (fence), 接收方才能按对的顺序看到那些已经写好的数据。栅栏一漏, 接收方就可能读到旧数据, 或者读得乱七八糟。

把整条数据通路串起来:TMEM 就在 Tensor Core 数据流的正中间

最后退一步, 咱们从头到尾捋一遍:TMEM 就坐在 Blackwell Tensor Core 数据通路的正中间。

阶段	谁在动	数据从哪到哪
1. 预取操作数	TMA	GMEM → SMEM
2. (block-scaled) 准备缩放因子	TMA/线程 + tcgen05.cp	GMEM/SMEM → TMEM
3. 矩阵乘累加	tcgen05.mma	读 SMEM 操作数, 累加写入 TMEM
4. 取回累加器	tcgen05.ld	TMEM → 寄存器
5. 尾声处理与写回	epilogue(寄存器级 cast/逐元素)	寄存器 → GMEM

这套设计的来龙去脉, 一句话就能讲透: 寄存器文件想做大, 物理上难、成本上也贵, 与其硬磕,Blackwell 干脆另加一块专给 Tensor Core 用的二维便笺内存, 把「装大累加器」这副担子从寄存器卸下来, 扔给了 TMEM。代价就是: 本来编译器替你打理的累加器存储, 现在得内核亲自动手去分配、寻址、搬运、释放。可回报也实打实——不用牺牲寄存器, 就能把 tile 开得更

小结

- TMEM 是 Blackwell 专属、tcgen05 专用的片上二维内存 (每 SM 一块,128 Lane × ≤512 Col, 每 Col 32 位)。它来到世上就一个目的: 把大 tile 压在寄存器上的那份压力卸掉——让累加器不必一直霸着寄存器。
- 它靠二维坐标 (Lane, Col) 寻址 (TIRx 记号里写作 TLane / TCol)。它是 tile 布局里的正式成员, 不是躲在背后的后备仓库; 布局必须跟 tcgen05 指令的读写方式对得上。
- 它得跟 SMEM 一样自己显式分配、显式释放 (以 CTA 为单位、按 32 列向上取整、返回基址), 还得算进内核的资源预算里。
- 数据进出 TMEM 就三条异步专用通路:tcgen05.ld(搬回寄存器,epilogue 的主路径, 一个 warp-group 拆成四个 warp,lane 1 ↔ 行 1/4)、tcgen05.st(从寄存器写回 TMEM)、tcgen05.cp(从 SMEM 批量拷贝, 常拿来搬缩放因子)。三条各等各的 (wait::ld, wait::st, commit group + mbarrier), 跨线程交接时还得补栅栏。
- 还有一条值得记牢的「前后一致»: 累加器虽然搬进了 TMEM, 但 tcgen05.ld 把它取回寄存器之后, 片段的样子和 Ampere/Hopper 时代一个样, 所以 epilogue 那套 cast 和写回的代码, 几乎能原封不动地接着用。

延伸阅读

- 原文:Special Memory: TMEM —Modern GPU Programming for MLSys
- 相关章节 (原书内部引用):
 - Data Layout and Its Notation(布局记号 TLane/TCol 的定义)

- Tensor Core Operand Layouts Across GPU Generations(各代 Tensor Core 操作数/片段布局的连续性)
- Async Coordination: mbarriers(异步完成与 mbarrier 协调机制)

术语对照

中文	English	说明
张量内存	TMEM(Tensor Memory)	Blackwell 专属、tcgen05 专用的片上二维内存
张量核心	Tensor Core	执行矩阵乘累加的专用硬件单元
累加器	accumulator	MMA 矩阵乘累加的结果存储
寄存器片段	register fragment	累加器/操作数在每个线程寄存器里的那一份
线程束	warp	32 个线程组成的调度单位
warp 组	warpgroup	4 个 warp 组成的更大调度/协作单位
线程块	CTA(Cooperative Thread Array)	协作线程阵列, 即一个 thread block
共享内存	SMEM(Shared Memory)	CTA 内共享的片上内存
全局内存	GMEM(Global Memory)	设备主存
矩阵乘累加	MMA / GEMM	Matrix-Multiply-Accumulate / 通用矩阵乘
张量内存加速器	TMA(Tensor Memory Accelerator)	负责 GMEM↔SMEM 异步大块搬运的硬件
分块缩放 MMA	block-scaled MMA	带分块缩放因子的低精度 MMA
尾声阶段	epilogue	MMA 后做 cast/逐元素/写回的阶段
占用率	occupancy	SM 上并发活跃 warp/CTA 的比例
内存栅栏	fence	保证跨线程写入可见顺序的同步原语
异步屏障	mbarrier	用于异步操作完成协调的屏障对象

第 07 章 · 异步协调:mbarrier

原文:[Async Coordination: mbarriers](#)

本章要点

本章要点 (TL;DR)

- **TMA 和 Tensor Core 都是异步的**: 发出一条指令, 不等于这条指令已经做完。发起线程把活儿往硬件引擎一塞就走了, 真正的搬数据、算矩阵在后台跟你的程序并行跑。所以别再靠”代码写在前面”就一口咬定数据备好了。
- **mbarrier 就是那个”完成信号”**: 它把五花八门的异步交接, 捏成一个特别简单的套路——干活的人 **arrive**(到达), 用数据的人 **wait**(等待)。
- 一个 **mbarrier** 内部记两笔账: **到达计数器** (还差几个人报到), 还有专门给 TMA 用的**字节计数** (还差多少字节没搬完)。两笔账都归零, 这一轮才算结。
- 每个屏障还带一个 **相位位 / phase bit**, 每过一轮翻一下。就靠它, 同一个屏障才能在循环里反复用, 而不会把”上一轮做完了”误当成”这一轮做完了”。
- 流水线 GEMM(矩阵乘, 深度学习里最核心的算子) 里, 屏障要几个, 看的是**可复用的流水级 (stage) 有几个**, 跟 K 维一共有多少个 tile 没关系。每个 stage 配一个屏障, 再加上寄存器里的一个相位值, 就齐活了。

前置知识

前置知识: 读这一章前, 最好先懂**异步搬运 (TMA)**、**warp/线程**、**共享内存 (SMEM)**, 以及**流水线 / 重叠 (overlap)** 这几个概念。没把握的话, 先翻一下第 0 章 · 极简入门。本章会默认你已经认识这些词。

7.1 为什么需要一个显式的”完成信号”

想搞懂 **mbarrier**, 我们得先弄明白: 它到底在替我们擦什么屁股?

现代 GPU(尤其是 Hopper / Blackwell 这一代, 英伟达较新的两代数据中心架构) 上, 最累的两类活儿都是**异步**干的:

- **TMA / Tensor Memory Accelerator**: 专门在全球内存 (global memory, GPU 的”主存”, 大但慢) 和共享内存 (SMEM, 一个 block 内线程共用的片上快速内存) 之间搬大块张量 (tile, 大矩阵切出来的小方块)。
- **Tensor Core 上的 tcgen05 MMA**(Tensor Core 是专做矩阵乘的硬件单元; MMA = 矩阵乘累加): 专门把矩阵乘的累加结果写进 TMEM(Tensor Memory, Blackwell 上专给 Tensor Core 用的一块片上内存)。

”异步”是个啥意思? 说白了就是: kernel 发出一条 TMA load 之后, 发它的那个线程**根本不会停下来等**。它把活儿往硬件引擎手里一塞, 扭头自己接着往下跑。真正的搬运, 是在后台跟你程序的后半段**同时**进行的。MMA 也是一个道理。

这么干的好处一目了然: **搬数据和算数据能叠在一起跑 (overlap)**, 谁也不耽误谁——想把 GPU 的算力榨干, 全靠这个。可天下没有免费的午餐, 它也带来一个绕不开的大麻烦:

关键

关键: 异步说白了就是——“代码写在前面”不代表“事情已经做完”。前面那条异步操作还没收尾, 后面的代码就可能抢跑了。换句话说, **你再也不能靠“谁写在前面”来判断数据准没准备好**。

不加约束的话, 马上就会冒出三种经典的数据竞争 (race):

出错场景	后果
TMA 还在往 SMEM tile 写数据,MMA 就开始读这块 tile	MMA 读到 残缺的数据
Tensor Core 还没把累加器 (accumulator, 边乘边累加结果的那块存储) 写完, 尾声阶段 (epilogue) 就去读 TMEM	epilogue(收尾阶段, 把算好的结果写回的那段代码) 读到 错误的值
kernel 等在了一个永远不会满足的条件上	kernel 永远卡住, 推进不下去

那怎么办? 办法很朴素: **每有一处异步交接 (handoff), 就在那儿插一个明确的完成信号**, 让 kernel 知道“这步真做完了”。**mbarrier** 就是这个信号。它的玩法特别统一:

```
(producer):          ->  barrier    "arrive"( )
(consumer):          ->  barrier    "wait"( )
```

就这一套”到达 / 等待”, 能同时管三种交接:

- 1. **TMA → MMA:** 这块 tile 搬好了没?
- 2. **MMA → epilogue:** 累加器算完了没, 能读了没?
- 3. **跨流水级复用缓冲区:** 这块 buffer 腾出来了没, 能拿去覆写了没?

注意

注意:mbarrier 可不是那种”用一次就扔”的一次性标志位。它身上带一个相位位, 每完成一轮就翻一下。正是靠它, 同一个屏障才能在几百次循环里反复复用, 而不会把”上一轮做完了”当成”这一轮做完了”。

7.2 mbarrier 到底是什么

mbarrier 是 memory barrier(内存屏障) 的缩写。你就把它想成一个摆在共享内存 (SMEM) 里的小账本, 专门记同步状态。这本账上主要有两栏:

- **到达计数器 / arrival counter:** 这一轮还差几个人没报到。
- **相位位 / phase bit:** 现在轮到第几轮了 (用 0、1 来回切)。

结构图

分组:

- **mbarrier(位于 SMEM):**「到达计数器 / arrival counter / (还差几个到达)」 「相位位 / phase bit / (0 / 1 交替)」

原书这里有个交互演示：能看到 `mbarrier` 的到达计数器、相位位，还能点 `init / arrive / wait` 三个操作，挨个字段单独看。这份静态笔记用上面那张图做了等价表达，但想体会动态过程，还是去原书的在线演示玩一玩最直观。

7.2.1 init: 初始化

屏障这辈子从 `init` 开始。这一步, `kernel` 只要交代清楚一件事：这一轮总共要等几个人报到（也就是期望到达数）。初始化一完成：

- 屏障停在 相位 0；
- 计数器被设成你说的那个期望到达数。

从这一刻起，屏障就开始“点名等人”了。它得等所有该报到的生产者、用资源的人都喊一声“我干完了”，这一轮才算凑齐。

7.2.2 arrive: 到达（三条不同的路径）

每来一次“到达”，屏障还要等的活儿就少一点。但有个点你得记住：不同角色“到达”的姿势是不一样的——这个区别特别关键。原书一共讲了三条到达路径。

路径一: TMA 的 `tx-count` 到达（字节预算）

说到 TMA load，它常用的报到方式是 `tx-count` 到达。代码长这样：

```
//          :expect_tx
mbarrier.arrive.expect_tx(bytes);
```

别看就一行，它一口气干了两件事：

1. 替发起线程在屏障上报一次到；
2. 顺手告诉屏障：这趟 TMA 引擎大概要搬多少字节（也就是先记一笔字节预算）。

关键

关键：光是“发起线程报了到”，屏障还不算完成。它还得等 TMA 引擎把这笔字节账“还清”——真把那么多字节搬到位才行。所以相位要翻转，得同时满足两条：普通到达计数归零，而且待传输的字节计数也归零。

这也是为什么不能把 `expect_tx` 想成简单的“又报了一次到”。它真正的活儿，是给这次异步拷贝先挂一笔字节预算；之后硬件每搬完一批，就靠 `complete-tx` 把对应的字节扣掉。一直扣到字节归零、到达也归零，这个屏障才算彻底交差。

时序图（按时间顺序）

1. 发起线程 → `mbarrier.arrive.expect_tx(bytes)`
2. 备注：到达计数 -1 / 同时登记 `byte` 预算 = `bytes`
3. 发起线程 → TMA 引擎：发起异步拷贝（立即返回）
4. TMA 引擎 → `mbarrier.complete-tx`（每搬完一批就扣减字节）

5. 备注: 当到达计数 $==0$ 且字节计数 $==0$ / 相位翻转, *wait* 才放行

路径二: Tensor Core 的 commit 到达

换到 Tensor Core 这边, 报到的方式就完全是另一套了。记牢一句话: 你发一条 `tcgen05 MMA`, 它自己是不会去碰任何屏障的。你得手动把一次屏障到达, 挂到 MMA 的”提交 (commit)”路径上, 比如:

```
//          : barrier          MMA    commit
tcgen05.commit.mbarrier::arrive;
```

挂好之后, 等这一批提交出去的 MMA 全算完, Tensor Core 就会替你去屏障上报到。

注意

注意: 要是 kernel 忘了挂这个 commit 到达, 那等在这个屏障上的消费者就会傻等一辈子, 永远等不到。这是写 `tcgen05` 特别容易栽的一个坑。

路径三: 普通线程的直接到达

普通线程也能自己直接去屏障上报到。这一般用在两种场合:

- 这个普通线程本身就扮演生产者;
- 一伙线程想宣布: 我们把某个资源用完了。

举个最常见的例子: 某个消费者把一块 SMEM 缓冲区读完之后, 可以在屏障上”到达”一下, 意思就是告诉生产者——这块 `buffer` 我腾出来了, 你拿去复用吧。

7.2.3 wait: 等待 (消费者这一侧)

`wait` 是同一套协议里消费者要干的事。消费者会死等, 直到屏障转到”这一轮该到的那个相位”, 然后才敢去读数据、或者复用这个屏障护着的资源。

把上面三条到达路径凑一块看, 本章最要紧的一个洞见就浮出来了:

关键

关键: 异步硬件不光是”自己闷头跑在程序前面”, 它还会反过来通过屏障, 把’我干完了’回报给程序。TMA 报的是”SMEM tile 准备好了”; Tensor Core 报的是”TMEM 结果算好了”; 普通线程报的是”这块 `buffer` 我不占了”。情况五花八门, 最后全被收成同一个形状: 生产者 `arrive`, 消费者 `wait`。

7.3 相位追踪 (Phase Tracking)

7.3.1 为什么必须有相位

屏障一般不会用一次就重新分配一个。一个做了流水线 (pipelined) 的 K 维循环, 可能要把同一种交接重复几百遍。每次迭代都新建一个 SMEM 屏障? 既不现实, 也太浪费。所以真实做法是: kernel 只攥着固定的一小撮屏障, 随着循环往前推, 翻来覆去地复用它们。

那复用怎么才不出乱子？全靠**相位位**。

屏障每凑齐当前这一轮的全部到达，就把**相位翻一下**：0 变 1，1 变 0，循环往复。wait 这边则会盯着看：**消费者期望的那个相位，跟现在对上了没**。这个期望相位，kernel 自己存在**寄存器**（register，每个线程私有的最快存储）里。某个 stage 等到一轮之后，kernel 在拿它跑下一轮之前，会**顺手把本地存的相位值也翻一下**。

结构图

分组：

- 同一个被复用的 barrier：「迭代 0 / 等待相位 0 / 完成后翻转到 1」「迭代 1 / 等待相位 1 / 完成后翻转到 0」「迭代 2 / 等待相位 0 / ...」「迭代 3 / 等待相位 1 / ...」

原书这里又是个**交互演示**：一个屏障跨好几次流水迭代被反复用，每完成一轮，你就能看到相位位翻一下。上图是它的静态版，**想看动起来的样子，配合原书交互演示一起看**。

7.3.2 没有相位会出什么乱子

设想一个屏障刚给一次 TMA load 用过，而且已经完成了。下一轮循环又拿**同一个屏障来复用，可偏偏没追踪相位**——这下消费者一瞅，发现屏障是”完成”状态，就乐呵呵地以为”新的 load 已经就绪了”，冲上去读数据。可它看到的其实是**上一轮**遗留的完成状态，新数据压根还没搬过来呢。

相位位干的就是把这两轮**隔开**这件事。下面这张表是**最简单的情形：只有一个屏障，每次迭代都用它**，所以期望相位每轮都翻一次。

迭代	期望相位
迭代 0	相位 0
迭代 1	相位 1
迭代 2	相位 0
迭代 3	相位 1
...	0/1 交替继续

注意

注意：”每迭代翻一次相位”是有前提的——前提是只有一个屏障在那儿连轴复用。下一节你就会看到，一旦有好几个 stage、好几个屏障轮班上，某个屏障得隔好几次迭代才轮到一回。所以**单盯着某一个屏障**，它的相位翻得没那么勤——千万别把这两种情形的相位序列混为一谈。

7.3.3 真实流水线里的记账：按 stage 来记

真实流水线里，这本账一般是一个 stage 一本（per stage）地记。kernel 手里攥着：

- 固定几个 **SMEM 流水级**（stage）；
- 跟 stage **一对一配好的**固定几个屏障；
- 寄存器里存的一小撮**相位值**。

循环往前推的时候，**每个逻辑迭代都会被派到某个物理 stage 上**。相位值干的活儿，就是告诉 wait：你这会儿等的，到底是这个物理屏障的**哪一轮**。

```
      : 0 1 2 3 4 5 ...
stage: s0 s1 s0 s1 s0 s1 ( 2 )
      : p0 p0 p1 p1 p0 p0 ( stage )
```

关键

关键: 这一下就讲通了后面那个问题: 为什么”用 TMA 做流水线化 GEMM”的代码, 用不着给每个 K tile 都配一个屏障? 因为它只要每个可复用 stage 配一个屏障, 再搭上相位追踪, 就够使了。

- **stage** 索引管的是: 这回该用哪块 SMEM 缓冲区、哪个屏障;
- **相位值**管的是: 把”这个 stage 这一次用”和”上一次用”区分开。

试一试 (让你的 agent 陪你练): 丢给它一个**两级流水线**, 让它追四次迭代。每次迭代都把这几样列出来:stage 索引、本地相位值、屏障啥时候翻转, 还有——要是复用 stage 之前忘了翻相位, 会捅出什么篓子。

7.4 同步规则 (Synchronization Rules)

把屏障和相位这套机制吃透之后, 你会发现 Tensor Core kernel 里的同步其实很有规律, 基本就是”照章办事”。这套章程一句话就能交代清楚:

只要一条执行路径生产了数据、或者腾出了某个资源, 而另一条路径接下来要用, 那这次交接就必须白纸黑字写出来。

原书把它分成三种常见情形。

情形一: 线程代码 → 异步引擎

假设线程往 SMEM 写了点东西, 紧跟着一条 **TMA store** 或 **MMA** 就要来读这块 SMEM。这时候 kernel 得保证: 线程写进去的内容, 在引擎读之前已经是看得见的。想做到这点, 得靠合适的线程级同步, 或者一道栅栏 (fence)。

至于具体用哪条指令, 要看这次交接的作用域 (scope), 但道理从头到尾就一个:

注意

注意: 生产线程没写完之前, 引擎绝对不许看到那块 SMEM 缓冲区。

情形二:TMA → MMA

TMA load 是异步往一块 SMEM tile 里灌数据的。就算看到”TMA 指令已经发出去了”,MMA 也别想当然地认为 tile 备好了。所以:

- TMA 操作必须绑一个 mbarrier;
- MMA 动这块 tile 之前, 必须先在那个屏障上 wait。

情形三:MMA → epilogue

tcgen05 MMA 是异步把结果写进 TMEM 的。Tensor Core 没把活儿干完,epilogue 就别急着去读累加器。所以:

- MMA 的 **commit** 路径要在一个完成屏障上 **arrive**;
- epilogue 读 TMEM 之前,先在那个屏障上 **wait**。

结构图

分组:

- 情形二:TMA 到 MMA:「TMA load / (异步填充 SMEM tile)」「mbARRIER」「MMA 读取 tile」
- 情形三:MMA 到 epilogue:「tcgen05 MMA / (异步写 TMEM)」「mbARRIER」「epilogue 读取 TMEM」

连接关系:

- mbARRIER $\xrightarrow{\text{wait 通过后}}$ MMA 读取 tile
- mbARRIER $\xrightarrow{\text{wait 通过后}}$ epilogue 读取 TMEM

原书这里又有个交互演示: 一个 TMA load 通过 **mbARRIER** 发出完成信号,MMA 路径等屏障放行了才去读 SMEM tile。Tensor Core → epilogue 那次交接长得一模一样,唯一的区别是:那边的到达,是 **Tensor Core** 的 **commit** 路径来报的,不是 TMA。想看它动起来,去翻原书的交互演示。

7.4.1 同一套机制,也能拿来”回收资源”

屏障可不只会发”数据备好了”的信号,它一样能发”资源腾出来了”的信号:

- 一个 SMEM stage,旧 tile 的所有消费者还没用完之前,不许覆写;
- 一块 TMEM 区域,上一个占用的人还没读写完之前,不许复用。

到了这两种场合,arrive 和 wait 的意思就换了个讲法:

- **arrive** = ”这资源我用完了”;
- **wait** = ”好了,这资源现在可以放心交给下一个 stage 了”。

关键

关键: 这才是读流水线 GEMM kernel 里那堆同步语句的正确打开方式——那一片 **wait** 和 **arrive** 绝不是到处乱撒的防御代码。每一条都对着一次明明白白的所有权交接 (**ownership transfer**): 一块 tile 备好了、一个累加器能读了、或者一块 buffer 能回收了。把这些交接点一个一个认出来,再绕的控制流也就读顺了。

小结

- TMA 和 Tensor Core 的**异步**让搬数据和算数据能叠着跑,代价是”代码先后顺序”再也证明不了数据就绪了。所以**每一处异步交接,都得配一个明确的完成信号**。

- **mbarrier** 是个摆在 **SMEM** 里的硬件同步对象，主要记两笔账：到达计数器和相位位；碰上 TMA，还多记一笔字节计数 (**tx-count**)。完成的条件是：到达归零 而且 字节归零。
- 三条到达路径各玩各的：TMA 用 **expect_tx**(顺手设字节预算)、Tensor Core 用 **tcgen05.commit.mbarrier** 了挂就死等)、普通线程可以直接 **arrive**(常拿来发”资源腾出来了”的信号)。
- **相位位**让同一个屏障在循环里能放心复用：每过一轮翻一次，消费者把期望相位存在寄存器里、每轮也翻一下，就不会把旧的完成当成新的完成。
- 真实流水线按 **stage** 记账：stage 有几个，屏障就几个，跟 K tile 一共多少个没关系；**stage** 索引管选缓冲区和屏障，相位值管区分这一轮和上一轮。
- 把所有 **wait** / **arrive** 都当成所有权交接来读（数据就绪 / 累加器可读 / buffer 可回收），再复杂的 kernel，控制流也读得明明白白。

延伸阅读

- 原文:[Async Coordination: mbarriers —Modern GPU Programming for ML](#) Sys(含本章三处交互演示:mbarrier 状态视图、相位翻转、TMA→MMA 完成信号)
- 相关章节:**Async Data Movement: TMA**(异步数据搬运)、**Tensor Cores: tcgen05**(张量核心)、**Pipelining GEMM with TMA**(用 TMA 做流水线化 GEMM)——想搞懂最后那章里”为啥屏障数 = stage 数”，得先把本章啃下来。

术语对照

中文	English
内存屏障 / 异步屏障	mbarrier (memory barrier)
到达计数器	arrival counter
相位位	phase bit
字节计数 / 事务计数	tx-count / byte count
期望字节预算	expect_tx
完成事务回报	complete-tx
到达 / 等待	arrive / wait
提交 (MMA 工作组)	commit
异步数据搬运引擎	TMA (Tensor Memory Accelerator)
张量核心	Tensor Core
张量内存	TMEM (Tensor Memory)
共享内存	SMEM (shared memory)
矩阵乘累加	MMA (Matrix-Multiply-Accumulate)
尾声阶段	epilogue
流水级	stage
程序顺序	program order
访存与计算重叠	overlap
所有权转移	ownership transfer

第 08 章 · 进阶：集群启动控制 (CLC)

原文: [Advanced: Cluster Launch Control](#)

本章要点

本章要点 (TL;DR)

- **常驻核 / persistent kernel**: 传统做法是「一个 CTA(一个线程块,GPU 调度的基本单位) 算一块输出 tile(大矩阵切出来的小方块), 算完就走人」。常驻核换了个思路——只起一小批活得久的 CTA / 集群 (cluster, 一起启动、能互相同步的一组 CTA), 让它们循环不停地一块接一块吞 tile。于是核心问题就剩一句: **手上这块算完了, 下一块打哪来?**
- **集群启动控制 / Cluster Launch Control (CLC)**: Blackwell(NVIDIA 较新的 GPU 架构代号, 继 Hopper 之后) 新加的硬件机制, 作用是让一个已经在跑的集群, 在运行时向硬件的网格调度器 / grid scheduler 讨还没启动的集群坐标。说白了, 它就是一条**硬件级的工作窃取 (work-stealing)** 通道。
- CLC 对外就给两条 PTX 指令: 一条异步发请求, 一条读回结果。完成信号靠 **mbarrier**(内存屏障, 异步操作干完后用来报信的同步对象) 报, 沿用 TMA(异步搬数据的硬件引擎) 那套 barrier + phase 模型, **不另搞任何新的等待方式**。
- 它最大的好处是**改善尾部行为 (tail behavior)**: 当各块 tile 算得有快有慢、或者 tile 数除不尽 SM 数时 (SM = 流式多处理器,GPU 上真正干活的核心, 见第 0 章), 先干完的集群马上就能接着拉新活儿, 不用在那儿干瞪眼空等。
- 注意别把它跟「线程块集群」搞混: 集群打 Hopper 起就是**启动单位**了;CLC 是 Blackwell 补上的本事, 让这些**集群坐标**的调度从死的变成活的。

前置知识

前置知识: 读这一章前, 最好先懂 CTA / cluster(集群)、tile(分块)、SM、关键路径与尾部、常驻核 (persistent kernel) 这几个概念。没把握的话, 先翻一下第 0 章 · 极简入门。本章会默认你已经认识这些词。

8.1 从静态调度到动态调度: 为什么需要 CLC

想搞懂 CLC, 得先把**常驻核**这个执行模式弄明白。这套玩法, 前面《用 warp 特化与集群扩展 GEMM》那一章是一步步搭起来的。

先看传统 GPU kernel 怎么干活。它把 CUDA 网格 / grid(一次启动的全部 CTA 的总集合) 当成一锤子买卖: 有多少个 tile, 就一口气启动多少个 CTA, 一个 CTA 包一个输出 tile, 算完拍屁股走人。常驻核不这么玩, 它换了个思路:

- 它只起一小批**活得久**的 CTA 或者集群。数量大概调到「一个 SM 上差不多蹲一个干活的」这个量级——但要说清楚, 它**并不靠**严格的 1:1 映射, 不需要这种保证。

- 每个集群算完一个 tile, 就接着算下一个, 算完再下一个, 就这么转圈, 一直转到整个输出空间全算完为止。

核一旦变成常驻的, 整个调度问题就被压成了一句话:

关键

关键: 一个集群算完当前 tile 之后, 下一个 tile 从哪里来?

静态公式调度: 简单但尾部会拉胯

最省事的答案是**静态公式 / static formula**: 拿 CTA id 直接算出第一个 tile 的坐标, 往后每跳一步, 都按一个固定的 **网格步长 / grid stride** 往后挪。

```
tile_id = cta_id
while tile_id < num_tiles:
    compute_tile(tile_id)
    tile_id += grid_size      #                tile
```

这招好不好用, 得看情况:

- 写起来简单。哪个 CTA 该算哪几块 tile, 在 kernel 真正开跑之前就**全排明白了**。
- 要是每块 tile 算起来都差不多累, 而且 tile 总数又恰好能被 SM 数整除, 那它跑得挺香。
- 可问题就出在「活儿在开干之前就钉死了」这件事上。万一有几块 tile 算得特别慢, 或者最后剩下的几块怎么也分不匀, 就会撞上一个很尴尬的场面: 一些 SM 早早把自己那份干完了, 只能在旁边干瞪眼; 另一些还在那儿吭哧吭哧啃尾巴。

CLC: 把分配推迟到运行时再决定

CLC 就是来收拾这个烂摊子的。它改了一件事: 不再开跑前就把整张排班表钉死, 而是让一个正在跑的常驻集群, 跑着跑着、在运行时向**硬件网格调度器**现要一份活儿——要的是「另一个还没启动的集群」那份。

结构图

分组:

- **分组**: 「CTA 0: tile 0,4,8...」「早干完 → 空等」「CTA 1: tile 1,5,9...」「还在啃尾巴」
- **CLC 动态调度 (运行时讨要)**: 「集群干完手上 tile」「向调度器 / try_cancel」「接管该坐标继续算」「退出循环」

连接关系:

- 向调度器 / try_cancel $\xrightarrow{\text{成功: 拿到坐标}}$ 接管该坐标继续算
- 向调度器 / try_cancel $\xrightarrow{\text{失败: 队列空了}}$ 退出循环
- 接管该坐标继续算 → 集群干完手上 tile

这一要, 结果无非两种:

- **要到了** → 当前集群就「接管」那个集群坐标, 掉头去算它对应的 tile。
- **要不到** → 说明没活儿可偷了, 循环到这儿就收工。

注意

注意:CLC 和「线程块集群」根本是两码事, 千万别混。线程块集群说的是一起启动、能做集群级同步、能访问分布式共享内存的那么一组 CTA, 它是 **Hopper** 带来的 (见《GPU 执行模型》)。CLC 则是 **Blackwell** 才加上的本事, 管的是「这些集群坐标到底怎么调度」这件事, 让它从死的变成活的。换句话说, **集群一直就是启动的单位;CLC 干的只是让一个正在跑的集群去取消一次还没发生的启动, 然后把那份坐标接过来。**

这里有个心智模型特别关键, 先记牢:CLC **不是**从某个软件任务队列里发一个抽象任务给你。它干的事很实在——**把一次还没发生的集群启动取消掉 (cancel a pending cluster launch)**, 再把那个集群本来该有的坐标转手给来要活儿的人。后面那一堆协议细节, 追到根上, 都是从这一句话长出来的。

8.2 两条指令: 请求与查询

CLC 对外就给你**两条 PTX 指令**: 一条异步发请求 (`try_cancel`), 一条读回响应 (`query_cancel`)。

这里有个小地方得说一句: 读响应的 `query_cancel` 又拆成两个变体, 一个查「成没成」, 一个取坐标。所以下面你会看到 3 个具体指令名。别被这个数字吓着——说到底, 要干的就「发请求」和「查结果」这两件事。

请求指令:`clusterlaunchcontrol.try_cancel.async`

维度	说明
语义	请求调度器 取消 一个待定集群的启动, 并把那个集群的坐标返回给调用方
响应位置	写入 共享内存 , 是一条 16 字节 的记录
同步性	异步 ——指令发出后 不等待 响应到达
完成信号	通过 mbarrier 报告, 沿用与 TMA 相同的 barrier + phase 模型; 响应到达由 barrier 的字节数完成 (byte-count completion) 来通知

关键

关键:CLC **没另搞一套等待机制**, 它直接沿用你早就熟的那一套。流程是这样:kernel 发出请求, 把它挂到一个 barrier 上, 之后要读响应之前, 先 **wait** 一下这个 barrier。响应到没到? 靠 barrier 的「字节数完成 / **byte-count completion**」来报信。这跟 TMA 这类异步硬件操作, 完全是一个路数 (见《异步协调:mbarriers》)。

查询指令:`clusterlaunchcontrol.query_cancel.*`

等 barrier 触发 (fired) 了, kernel 就拿查询指令去读结果。这一步得分两小步走:

第一小步:`query_cancel.is_canceled` ——它给你一个**谓词 / predicate**, 说白了就是一个真/假的判断, 告诉 kernel 这次取消到底成没成。

- 谓词是 **true** → 调度器真找着了 **一个待定的集群启动**, 把它取消了, 坐标也给你了。
- 谓词是 **false** → 没活儿剩了, 拿不到了。

第二小步 (只有 `is_canceled` 为 `true` 时才做):`query_cancel.get_first_ctaid` ——把被取消那个集群的**第一个 CTA id** 取出来。这个 CTA id 其实是个坐标向量, 一般按 (x, y, z) 读出来。拿到手之后, kernel 再把它**解码**成接下来要算的那块输出 tile。

```
//      :
mbarrier_wait(clc_barrier, phase)          // 1)
if query_cancel.is_canceled(response):     // 2)
    (x, y, z) = query_cancel.get_first_ctaid(response)
    next_tile = decode_tile_coord(x, y, z)  //      true
else:
    break                                  //      false ->      ,
```

注意

注意: 这个协议里没拿「哨兵 `tile id / sentinel`」这种魔法数字来表示「没活儿了」。kernel 纯粹是**靠谓词来分支**: 谓词 `true`, 坐标就有效; 谓词 `false`, 工作窃取循环就到头。

为什么这么设计? 还得绕回 CLC 到底在干啥: 硬件不是从软件队列里发个抽象任务给你, 而是在**取消一次还没发生的集群启动**。既然是这样, 「成功的响应」自然就捎着一个**货真价实的集群坐标**; 而「失败的响应」无非就是说启动队列已经掏空了。这么一想, 全都顺了。

8.3 工作窃取循环

有了这两条指令, 整个常驻调度器就缩成了一个**很短的循环**。而这个循环里最妙的一笔, 在于**请求该摆在哪个位置**。先卖个关子, 往下慢慢看。

循环的结构

时序图 (按时间顺序)

1. 备注: 手上有「当前 `tile`」要算
2. 常驻集群 → 网格调度器:1. 为「下一个 `tile`」发 `try_cancel`(异步)
3. 备注:2. 在请求飞行途中, 先算「当前 `tile`」
4. 网格调度器 → mbarrier: 响应就绪 → 字节数完成
5. 常驻集群 → mbarrier:3. 算完后, wait 响应 barrier
6. 常驻集群 → 常驻集群:4. `query is_canceled?`
7. 常驻集群 → 常驻集群:5a. `get_first_ctaid` → 解码坐标 → 作为下一个 `tile`, 继续
8. 常驻集群 → 常驻集群:5b. 没活儿了 → 退出

用大白话列出来, 就这五步:

1. 先为「可能还有的下一个 `tile`」发一发 `try_cancel`。
2. 趁这个请求还在路上飞着 (**in flight**), 赶紧把当前这块 `tile` 算了。
3. 算完, wait 那个响应 barrier。
4. 查一下取消到底成没成。
5. 成了就拿返回的坐标接着算, 没成就退出。

为什么要「先请求，再计算」？

关键

关键: 集群才不会傻等着当前 tile 算完了, 再去要下一份活儿。它的套路是先问、后算。这么一来, 「向调度器发请求」和「干正经计算」这两件事就叠 (overlap, 让等待和计算同时进行、互相遮掩) 到一块儿了——等你把当前 tile 算完, 下一个 tile 的答案多半早就在那儿等着了, 你几乎不用干等。

这个道理, 跟常驻核在别处用异步拷贝 (TMA)、用 Tensor Core(GPU 里专门做矩阵乘的加速单元)barrier 时完全是一个意思, 核心就一句话: 别让又慢又长的高延迟操作直愣愣地卡在关键路径 / critical path 上 (关键路径 = 决定整体快慢、省不掉的那条最长链路)。CLC 无非是把这套老办法挪到了 tile 调度上——早早地把下一份活儿要上, 手头先把当前这份算了, 真要用结果时再去取。

下面拿一张时间轴表来说 (简化过): 行是两个并行的角色, 列是时间往前走,t0 → t3。

角色 \ 时间	t0	t1	t2	t3
集群 (关键路径)	发 try_cancel(异步, 不阻塞)	算当前 tile	算当前 tile	wait barrier(此时多半已就绪, 几乎不阻塞)
调度器 (后台)	收到请求	在后台找下一块	响应就绪 → 字节数完成	—

看明白了吧: 发请求和算当前 tile 在 t1-t2 完全叠在一起, 所以等集群算完、走到 t3 去 wait 的时候, 响应早就准备好了, 基本不卡。

8.4 与常驻 GEMM 的关系

《用 warp 特化与集群扩展 GEMM》那一章 (GEMM = 通用矩阵乘法, 深度学习里最核心的算子), 主线讲解用的是静态调度器。为啥用它? 理由很实在: 静态调度器好讲——下一个 tile 拿循环状态一算就出来了。比如那章里有个叫 ClusterPersistentScheduler2D 的调度器, 就是按网格步长的路子, 在输出 tile 空间上一块一块往下分的。

而 CLC, 就是这套静态分配的动态版替身。这儿要重点强调一句: 外层循环一个字都没动——每个常驻集群干的还是那套老活儿「算一块输出 tile, 再往下挪一块」。真正变的, 就只有『下一个 tile 打哪来』这一件事:

维度	静态调度器	CLC 动态调度
下一个 tile 来源	公式算出 (grid-stride)	硬件工作窃取返回
决策时机	启动时一次性定死	运行时按需分配
tile 计算体 (tile body)	相同的常驻 GEMM 主体	完全相同
尾部不均时	早干完的 SM 空等	早干完的集群继续拉活
tile 成本不均时	假设开跑前的分配「足够好」	不需要这个假设

在两种场景下 CLC 优势最明显

1) 启动的尾巴 (tail of the launch) 跑到收尾这段, 静态排班下剩的活儿往往七零八落: 有些 SM 早把分到的 tile 全啃光了, 有些却还压着好几块。换成 CLC, 先干完的集群直接掉头去要另一个待

定集群的坐标——只要启动队列里还有活儿，先收工的人就能不停地拉新 tile 来干，不会在那儿闲着。

2)tile 一块比一块累有些 GEMM tile 走的代码路径跟别人不一样，可能是碰上了边界 (boundaries)、掩码 (masking)、稀疏 (sparsity)、分组调度 (grouped scheduling)，也可能是围着主矩阵乘做了点融合 (fused) 计算。静态排班的麻烦在哪？它得假设：在还没真跑、压根看不出这些成本差异的时候，分配就已经分得够好了。而 CLC 压根不需要这个假设——它是等某个集群真闲下来了，才再给它派活儿。

在 TIRx 里怎么暴露

就冲这些好处，在 TIRx 里可以把 CLC 包装成一个动态 tile 调度器。对编程模型来讲，tile 的计算根本不用动。tile 主体还是静态调度器用的那套常驻 GEMM 主体。变的只有调度器，它从原来的

「按公式算出我的下一个 tile 坐标」

变成了

「向硬件要下一个可用的集群坐标」。

到头来的效果是：还是那个常驻循环，只不过分活儿的法子从「启动时就钉死的排班」，换成了「硬件实时说了算」。

小结

- **常驻核 + CLC** 这一套组合，把 GPU 上的 tile 调度从「编译/启动期就钉死」往后挪到了「运行时由硬件现派」。它要治的病很清楚：负载不均，导致收尾时一帮人干瞪眼空转。
- CLC 这套实现相当克制：就两条 PTX 指令，外加白用现成的 mbarrier 完成模型。它没另起炉灶造什么新的等待原语，而是把「异步发请求 + barrier 等响应」这套在 TMA、Tensor Core 上早验证过的老范式，原封不动搬到了调度上。
- 它的协议是靠谓词驱动，不是靠哨兵值。为啥？还是那句话——CLC 的本质是取消一次还没发生的集群启动，不是从软件队列里取任务。所以成功就直接给你真坐标，失败就说明队列空了。
- 它设计上最漂亮的一手，是「先请求、后计算」这个循环安排：把调度延迟悄悄塞进当前 tile 的计算时间里，不让它压在关键路径上。这跟整套异步 GPU 编程的思路是一脉相承的。
- 从工程角度看，CLC 对上层（比如 TIRx）几乎是零打扰：tile 计算体一个字不动，只要把调度器从「套公式」换成「问硬件」，就白捡一个硬件级的动态负载均衡。

一句话：CLC = 给常驻核接上一条硬件工作窃取通道，让先干完的集群在运行时「偷」走还没启动的集群坐标，这样一来，tile 成本或数量不均时，收尾的性能就能明显好上一截。

延伸阅读

- 本章原文：[Advanced: Cluster Launch Control —Modern GPU Programming for ML](#)
- 相关章节 (同书):
 - 《GPU 执行模型》——线程块集群、分布式共享内存的来历 (Hopper)

- 《异步协调:mbarriers》——barrier + phase + 字节数完成模型,CLC 的完成信号复用了它
- 《异步内存:TMA》——同样的「异步请求 + barrier 等待」范式
- 《用 warp 特化与集群扩展 GEMM》——常驻 GEMM 与静态 ClusterPersistentScheduler2D 的来龙去脉

术语对照

中文	English / 缩写
集群启动控制	Cluster Launch Control(CLC)
常驻核	persistent kernel
线程块集群	thread block cluster
工作窃取	work-stealing
网格调度器	grid scheduler
网格步长	grid stride
静态调度器	static scheduler
动态 tile 调度器	dynamic tile scheduler
尾部行为	tail behavior
关键路径	critical path
谓词	predicate
哨兵值	sentinel
内存屏障	mbarrier
相位 (位)	phase (bit)
字节数完成	byte-count completion
输出分块	output tile
线程块	CTA
矩阵乘	GEMM / MMA
张量内存访问	TMA

第 09 章 · TIRx 入门

原文: [Introduction to TIRx](#)

本章要点

本章要点 (TL;DR)

- **TIRx**(张量 IR neXt / **T**ensor **I**R ne**X**t) 是一种嵌在 Python 里的 DSL, 专门用来在 **IR**(中间表示) 这一层写 GPU kernel。它的妙处在于: 你照样能直接点名硬件——线程、SMEM、TMEM、屏障、`tcgen05` MMA 一个不少; 但你点的这些东西, 全都以**结构化 IR**的样子存着, 所以编译器看得懂, 能帮你下降 (lower)、检查、调度。
- 一个 kernel 到底怎么跑, 关键就那么几件事。TIRx 把它们**全摆到台面上**, 归成三大设计要素: **作用域 (scope)**——谁来干; **布局 (layout)**——数据块放哪; **派发 (dispatch)**——走哪条硬件路。
- 整章就盯着一个**能跑的最小 GEMM**(只做一次 MMA): 算一个 128×128 的输出 tile, 公式是 $D = A \cdot B^T, K=64$, 而且整个矩阵乘就用一句 `Tx.gemm_async` 写完。
- 别看 kernel 小, 三大要素它一个不落全演示到了。说白了, 全书后面的章节, 做的就是「把这三个要素放大到更大规模」这一件事。
- 编译就一句 `tvm.compile(mod, target="cuda", tir_pipeline="tirx")`, 核心的下降 pass 叫 **LowerTIRx**——它挨个把 tile 原语按各自的 scope/layout/dispatch 契约「兑现」成真指令。整个过程**对你不藏一点东西**:IR 能看, 最后生成的 CUDA C 也能看。

前置知识

前置知识: 读这一章前, 最好先懂 GPU 的线程层级 (grid $\rightarrow$ CTA/block $\rightarrow$ warpgroup $\rightarrow$ warp $\rightarrow$ 线程)、内存层级 (GMEM/SMEM/寄存器, 以及 Blackwell 上的 TMEM)、还有 Tensor Core / MMA 大概是干嘛的。没把握的话, 先翻一下第 0 章 · 极简入门。本章会默认你已经认识这些词。

一、为什么需要 TIRx: 把「隐藏的决策」翻到台面上

前面第一部分 (Part I), 我们把硬件是什么讲透了。可光认识硬件还不行——你想让它真干活, 总得有个写代码的法子才行。

最直接的法子, 就是裸写 CUDA 或者 PTX。事实上, 不少高性能 kernel 就是这么手搓出来的。那它差在哪? **差在: 决定这个 kernel 怎么跑的那几件关键事, 在裸代码里你几乎看不出来。**比如这几个问题:

- 哪些线程在执行某个操作?
- 每一块数据 tile 究竟住在哪个存储层级?
- 这条计算实际走的是哪条硬件路径 (普通线程拷贝?TMA(硬件异步批量搬数据的引擎)?Tensor Core(专做矩阵乘的硬件单元)?)?

裸写 CUDA/PTX 的时候,这几个选择都散落在哪了呢? 埋在 `intrinsic` 的一长串参数里, 埋在手算的地址里, 还有一堆「大家都这么写」的约定俗成里。于是就成了这样: 别人来读你的代码, 基本得靠「考古」, 才能猜出你当初到底想干嘛。

那 TIRx 是怎么破这个局的? 它没走「把硬件细节都藏起来、让你写得省心」那条路——那是高层框架的玩法。TIRx 偏不: 它照样让你直接点名硬件, 但把上面那三类决策, 从「藏在代码里」改成了「明明白白写在 IR 上」。

关键

关键: 一句话说明 TIRx 的思路——底层控制力一点不丢, 同时还让编译器看得懂。道理很简单: 决策只要明着写出来, 编译器就能拿它去检查 (check)、下降 (lower)、调度 (schedule)。

这三类决策,TIRx 给起了三个名字, 后面整本书都会用到:

设计要素	它回答的问题	在代码里体现为
作用域 (scope)	谁 (哪些线程) 来执行这个操作?	CTA(一个线程块, 见第 0 章)/warpgroup(4 个 warp = 128 线程)/ 单个 elected 线程等前缀与 guard
布局 (layout)	操作数 tile(大矩阵切出来的小方块) 住在哪里、以什么方式排布?	SMEM(片上共享内存) 的 swizzle 布局、TMEM 的 TLane/TCol、寄存器视图等
派发 (dispatch)	走哪条硬件路径执行?	dispatch="tcgen05"、拷贝是普通线程还是 TMA

这三件事, 本章不想干巴巴地讲概念。咱换个法子: 直接搬一个完整、能跑的 kernel 出来看。先让它跑起来, 再回头一行一行读, 看看 scope、layout、dispatch 分别是怎么把这段代码捏成现在这样的, 以及它最后又是怎么编出来的。

注意

注意: 本章就盯着「一个 kernel + 三个要素」, 故意把焦距收得很窄。这个 kernel 用到的张量布局模型, 留到后面《TIRx 布局 API》专门讲; 完整的语言特性, 放在《TIRx 语言参考》里。

二、运行环境与依赖

想真把本章的例子跑起来, 你得有一块 **Blackwell** 架构的 GPU(目标是 `sm_100a`, 比如 B200)。TIRx 编译器是跟着 Apache TVM 一起发的, 就在 `wheel` 包里的 `tvm.tirx` 模块, 安装的时候记得配上 CUDA 版的 PyTorch:

```
pip install apache-tvm
```

装完跑一行命令, 确认能正常 `import` 就行:

```
python -c "import tvm, tvm.tirx; print(tvm.__version__)"
```

注意

注意: 这套环境配好, 全书的例子都能跑。手头没有 Blackwell GPU 也别慌——代码是跑不了, 但你完全可以当「阅读理解」来读。本章真正值钱的地方, 本来就在于搞懂这一个 kernel 是怎么对应到 scope、layout、dispatch 上的。

三、第一个 kernel: 单次 MMA 的 GEMM

咱要看的这个 kernel, 是把 GEMM(通用矩阵乘 $C = A \cdot B$) 砍到不能再砍的最小版——可即便这么小, 它还是刚好能驱动起一个 Tensor Core。规格很清楚:

- 计算单个 128×128 的输出 tile;
- 公式为 $D = A \cdot B^T$, 其中 $K = 64$;
- 整段计算从头到尾用一个 `Tx.gemm_async` tile 操作表达。

3.1 「一个 tile 操作 $\neq$ 一条硬件指令」

这一点想明白了, 你就抓住 TIRx 的价值了。`Tx.gemm_async` 看着就是「一句话」, 可它并不会变成一条硬件指令。为啥? 咱算笔账:

- 硬件 MMA(矩阵乘累加, Tensor Core 的核心操作) 的 **K-atom**(K 方向原子粒度) 是 16;
- 而我们这个 tile 的 $K = 64$;
- 所以编译器会把它下降成一小串沿 K 步进的 `tcgen05.mma` 指令 ($64 / 16 = 4$ 步)。

关键

关键:DSL 值钱的地方就在这儿——你写的是「tile(数据块)」, 不是「一串指令」。怎么沿 K 拆、怎么生成对的 `tcgen05.mma` 序列, 那都是编译器的活, 程序员根本不用管。

光发一个 MMA 还不够, kernel 周边还得干一圈杂活。顺着数据流走一遍就清楚了: 先把 SMEM(共享内存) 和 TMEM(张量内存 / Tensor Memory) 分配好, 把 A、B 从 GMEM(全局内存) 搬进 SMEM, 接着把 tile MMA 发到 TMEM 累加器 (accumulator, 边算边累加结果的那块存储) 里算, 算完再经寄存器 (RF) 把结果读回来, 最后写出去。

这 kernel 虽小, 它可是后面《构建分块 GEMM》那条优化路上的**第 1 步 (Step 1)**。到那一章, 我们会带着完整讲解再回来找它。

3.2 起手式: 统一的 import

每个 TIRx kernel 开头那几行 import 都大同小异, 先扫一眼混个脸熟:

```
import tvm
from tvm.script import tirx as T                                # T:TIRx              (device_entry Buffer ptx )
from tvm.script.tirx import tile as Tx                         # Tx:tile              (cta.copy gemm_async wg.copy_async ê)
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_shared_layout, SwizzleMode #
↪ TMA/swizzle
from tvm.tirx.layout import TileLayout, S, Tlane, TCol, tid_in_wg # DSL:      Tlane/TCol
↪ tid_in_wg
```

记住它们各管一摊就行: **T 管「语言/硬件层」**——入口声明、buffer、PTX intrinsic 这些; **Tx 管「tile 操作层」**——拷贝、矩阵乘这些; **tvm.tirx.layout** 则是一套专门用来描述布局的小 DSL。

3.3 kernel 的整体骨架

整个 kernel 被装进一个小 builder `hgemm_v1(M, N, K)` 里: 你喂它一个问题规模, 它吐给你一个 `PrimFunc`。本章挑的是 `M=N=128, K=64`, 这个尺寸跑起来正好只有一个输出 **tile**。第一版能让你「一口气读完」, 靠的就是这个小尺寸。

下面把它拆成几块, 每块只摘最要紧的那几行 (完整源码看原文)。

(a) 数据类型与分块常量

```
a_type = b_type = d_type = tvn.DataType("float16") # A/B/D      fp16
acc_type = tvn.DataType("float32")                #            fp32(      )

BLK_M, BLK_N, BLK_K = 128, 128, 64 #      CTA
MMA_M, MMA_N, MMA_K = 128, 128, 16 #      :      MMA tile      ,      K-atom=16
```

注意

注意: `MMA_M/N/K` 在这儿纯粹是「注释」, 就为了告诉你底层硬件那个 `MMA tile` 长啥样。它们压根不会传进 `gemm_async`——`gemm_async` 自己会从操作数 `tile` 和累加器 `tile` 把 `MMA` 形状推出来。既然用不上, 后面几步索性就把这几个常量删了。这也透着 `TIRx` 的一点小脾气: 形状只要编译器能推出来, 你就别手动喂。

(b) 为 A、B 准备 swizzled SMEM 布局

```
A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_ATOM, (BLK_M, BLK_K))
B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_ATOM, (BLK_N, BLK_K))
```

布局 (layout) 这个要素, 头一回露面就在这儿: A、B 在 `SMEM` 里用的是 **128B swizzle** 布局。Swizzle(混排, 把数据在 `SMEM` 里打乱地址摆放) 是干嘛使的? 两件事: 一来躲开 `SMEM` 的 bank 冲突 (多个线程挤同一个存储体, 被迫排队), 二来满足 `tcgen05.mma` 对操作数怎么摆的要求。它到底怎么混排, 《`TIRx` 布局 API》里会细说; 眼下你记住一句话就够了: **这就是 `tcgen05.mma` 想要的布局**。

(c) 进入 device 入口、计算线程坐标

```
@T.prim_func
def kernel(A: T.Buffer((M, K), a_type),
           B: T.Buffer((N, K), b_type),
           D: T.Buffer((M, N), d_type)):
    T.device_entry() #      device      kernel
    bx, by = T.cta_id([M // BLK_M, N // BLK_N]) # CTA      (      grid      1x1)
    wg_id = T.wargroup_id([1]) #      wargroup -> wg_id      0(      )
    warp_id = T.warp_id_in_wg([4]) #      wargroup      warp      (0-3)
    lane_id = T.lane_id([32]) #      warp      lane      (0-31)
```

关键

关键: 这几行, 就是作用域 (scope) 的那套「坐标系」。cta_id / warpgroup_id / warp_id_in_wg / lane_id(lane = warp 内的线程编号,0-31) 一层套一层, 从「整个网格(grid, 所有 CTA 的总集)」一路定位到「某一个线程」。后面凡是要判断「这活儿归谁干」(比如 if warp_id == 0), 全靠这套坐标说话。

第一步故意让 M 等于 BLK_M、N 等于 BLK_N, 这样 grid 就是 1×1, 每个 CTA 的 tile 偏移(m_st, n_st) 自然全是 0。要等到 Step 3 往后, 才会把它扩大到更大的 M/N。

(d) SMEM 分配: 用 pool 显式排布

```
pool = T.SMEMPool() # ,
tmem_addr = pool.alloc((1,), "uint32") # TMEM
mma_bar = pool.alloc((1,), "uint64", align=8) # MMA (mbarrier), 8B
pool.move_base_to(1024) # 1024,
Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout) # A SMEM, swizzle
Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout) # B SMEM, swizzle
pool.commit() # ,
```

这一段最能看出 TIRx「直接点名硬件」是个什么味儿:SMEM 不是背着你偷偷分配好的, 而是你拿一个 SMEMPool 亲手摆出来的。还有个点要留意——Asmem/Bsmem 在分配的那一刻, 就绑上了上一步定义的那套 swizzle 布局。布局这个要素, 到这儿又落地了一回。

(e) 屏障 + TMEM 初始化 (只让 warp 0 干)

```
if warp_id == 0: # : warp 0
    if lane_id == 0: # : lane 0
        T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1) # mbarrier, 1
        T.ptx.tcgen05.alloc(T.address_of(tmem_addr), n_cols=512, cta_group=1) # TMEM(512 )

T.ptx.fence.proxy_async("shared::cta") # fence: SMEM
T.ptx.fence.mbarrier_init() # mbarrier fence
T.cuda.cta_sync() # CTA ,
```

注意

注意: 一看到 T.ptx.*, 你就该知道: 这是在直接发 PTX intrinsic。这正是 TIRx 和高层框架彻底分家的地方——mbarrier(异步操作完成时用来同步等待的内存屏障)、tcgen05.alloc、fence 这些底层概念, 它一个都不躲, 只不过把它们包成了结构化的、编译器看得懂的 IR 节点罢了。

(f) 声明 TMEM 累加器视图

```
tmem = T.decl_buffer(
    (128, 512), "float32", scope="tmem", allocated_addr=tmem_addr[0],
    layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]) # ->TLane, ->TCol
)
```

这一行, 大概是布局要素最有代表性的一处了。TMEM 累加器是用 TileLayout 来描述的:128 行映射到 TLane(lane 维),512 列映射到 TCol(列维)。

这里的 TLane/TCol 到底是啥？它们是 TIRx 专门给 TMEM 这种特殊存储起的名字，叫**命名轴 / named axes**。注意了：它们可不是你平时理解的那种「行」「列」，而是跟 Tensor Core 硬件布局一一对应的逻辑轴。

(g) 加载：全员协作把 GMEM 拷到 SMEM

```
m_st = T.meta_var(bx * BLK_M) # CTA ( 0)
n_st = T.meta_var(by * BLK_N) # CTA ( 0)
phase_mma: T.int32 = 0 # mbarrier (phase)

Tx.cta.copy(Asmem[:, :], A[m_st:m_st + BLK_M, :]) # CTA :128
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st + BLK_N, :])
T.cuda.cta_sync() #
```

关键

关键: Tx.cta.copy 是 CTA 作用域的拷贝——CTA 里 128 个线程全员上，一块儿搬数据。这是 scope 要素的典型样本：同样是「拷贝」俩字，作用域一换，干活的线程是哪一群也跟着换了。

(h) 计算：单个 elected 线程发射 MMA

```
if warp_id == 0:
    if T.ptx.select_sync(): # warp
        Tx.gemm_async(
            tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
            accum=False, # , ( )
            dispatch="tcgen05", # : Blackwell Tensor Core
            cta_group=1
        )
        T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1) # ,
T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) # MMA
```

这是整段 kernel 的核心，也是三要素难得同框的一幕。一个一个看：

- **scope:** if warp_id == 0 再加上 elect_sync()，意思就是由选举出来的那一个线程来发射。凭啥一个线程就够了？因为下降出来的每条 tcgen05.mma，本身就是一次协作式 (cooperative) 的 MMA 启动——真正的计算是 Tensor Core 硬件一起完成的，软件这边只要派一个线程「点个火」启动它就行。
- **layout:** 操作数 Asmem/Bsmem 走 swizzle SMEM 布局，累加器 tmem 走 TLane/TCol 布局——这些前面早铺好了。
- **dispatch:** dispatch="tcgen05"，明明白白选了 Blackwell Tensor Core 这条路。

(i) 写回: TMEM → 寄存器 → GMEM

```
Dreg = T.alloc_local((BLK_N,), acc_type) # fp32 ( )
Dreg_f16 = T.alloc_local((BLK_N,), d_type) # fp16
Dreg_wg = Dreg.view(128, BLK_N,
    layout=FileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)])) # 128 warpgroup
↪
Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N]) # warpgroup :128
T.ptx.tcgen05.wait.ld() # TMEM load
```



```
Tx.cast(Dreg_f16[:,], Dreg[:,]) # fp32 -> fp16
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id) #
Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:,]) # GMEM
```

关键

关键:Tx.wg.copy_async 是 warpgroup 作用域——warpgroup 的 128 个线程把 TMEM 累加器一行一行 (row by row) 读回来。这里的门道在 Dreg_wg 这个视图: 它靠 tid_in_wg 把「行」对应到「warpgroup 里的某个线程」, 于是每个线程刚好分到一行片段 (row fragment)。这是 layout 和 scope 配合得最漂亮的一幕——布局回答「哪个线程拿哪一行」, 作用域回答「这 128 个线程一块儿上」。

(j) 释放 TMEM

```
T.cuda.cta_sync()
if warp_id == 0:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1) #
    T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_group=1) # TMEM
```

TMEM 这东西金贵, 既然当初是你手动分配的, 用完就得自己手动还回去——这又是「直接点名硬件」这股风格的一处体现。

3.4 整体数据流示意

读到这儿, 各块代码都过了一遍, 咱用一张图把它们串成一条完整的数据流 (顺手标上每一段是谁在干活):



图说: 顺着这条数据流走, 一共冒出来三种不一样的作用域——CTA 级拷贝 (128 线程全员搬,GMEM→SMEM)、单个 elected 线程发射 MMA(协作式硬件 MMA,SMEM→TMEM)、warpgroup 级读回 (128 线程逐行取累加器,TMEM→RF)。同一个 kernel 里, 不同的操作各挑各的作用域, 这就是 scope 要素活生生的样子。图里颜色是按存储层级分的: 灰 =GMEM、蓝 =SMEM、橙 =TMEM、绿 = 寄存器。

3.5 先跑起来: 用 torch 对拍验证

在逐行细抠 kernel 之前, 先干件让人踏实的事——确认它真能跑, 而且算得对。流程很短:

```
import torch

target = tvm.target.Target("cuda")      #          sm_100a, arch
device = torch.device('cuda')

M, N, K = 128, 128, 64
kernel = hgemm_v1(M, N, K)
with target:
    # tir_pipeline="tirx"      TIRx          ;          Executable
    ex = tvm.compile(tvm.IRModule({"main": kernel}), target=target, tir_pipeline="tirx")

A_tensor = torch.randn(M, K, dtype=torch.float16, device=device)
B_tensor = torch.randn(N, K, dtype=torch.float16, device=device)
D_tensor = torch.zeros(M, N, dtype=torch.float16, device=device)

ex.mod(A_tensor, B_tensor, D_tensor)    #          torch      ,

D_ref = (A_tensor.float() @ B_tensor.float()).T.half()      # torch
torch.testing.assert_close(D_tensor, D_ref, rtol=2e-2, atol=1e-2)
print("PASS")
```

注意

注意: 这段里有两个对开发者特别贴心的小设计。一是 **arch 自动探测**: 你只写 `target="cuda"` 就够了, 编译器会自己上设备上摸出 `sm_100a` 这类具体架构。二是 `ex.mod(...)` 直接吃 **torch 张量**, 中间你不用手动转一道。还有一点, 容差为啥给到 `rtol=2e-2, atol=1e-2` 这么松? 因为 `fp16` 算起来本身就带数值误差, 卡得太死反倒容易误判。

四、三要素回读:scope / layout / dispatch

kernel 跑通了, 现在咱换个角度回头再读一遍, 边读边问自己一个问题: 它每一行, 到底「拍了什么板」? 说白了, 整个 kernel 不过就是沿着三个设计要素做的一连串选择。每个操作都在回答同样三个问题——谁来跑、数据 住哪、怎么执行——这三个答案, 恰好就是 `scope`、`layout`、`dispatch`。

关键

关键: 原文这块有个交互演示——你点一下 `Scope / Layout / Dispatch` 按钮, kernel 里归这个要素管的代码行就会高亮起来。静态笔记没法复现这个交互, 不过下面三张表已经把「每个要素管哪几行」列清楚了, 效果是一样的。

4.1 Scope: 谁来执行?

操作	作用域	参与线程	为什么是这个作用域
<code>Tx.cta.copy(...)</code>	CTA	全部 128 线程	GMEM→SMEM 是大批量搬运, 全员一起上最快

<code>Tx.gemm_async(...)</code>	单个 elected 线程	1 个线程发起	下降出的每条 <code>tcgen05.mma</code> 本身就是 协作式 MMA 启动 ，真正算的是 Tensor Core 硬件，软件派一个线程点火就够
<code>Tx.wg.copy_async(...)</code>	warpgroup	warpgroup 的 128 线程	TMEM 读回是按行切的，每个线程拿一行片段最顺

4.2 Layout: 每块 tile 住在哪、怎么排？

tile	存储层级	布局	说明
A、B 操作数	SMEM	128B swizzle 布局	<code>tcgen05.mma</code> 想要的就是这个摆法，顺带躲开 bank 冲突
累加器	TMEM	TLane / TCol 命名轴布局	跟 Tensor Core 硬件的累加布局一一对应
寄存器读回视图 <code>Dreg_wg</code>	RF	行映射到 tid_in_wg	让 warpgroup 里每个线程刚好「占一行」

4.3 Dispatch: 走哪条硬件路径？

操作	dispatch 选择	含义
<code>Tx.gemm_async(..., dispatch="tcgen05", ...)</code>	tcgen05	走 Blackwell Tensor Core 这条路
拷贝操作 (本章)	普通线程拷贝	第一版就用最朴素的线程拷贝
拷贝操作 (后续章节)	TMA	后面的 GEMM 步骤会把拷贝换成 TMA ，可周围的 scope 和 layout 一点不动

关键

关键:dispatch 想换就换，这是 **TIRx** 的一大长处。想「换条硬件路」(比如线程拷贝 → **TMA**)？这是个**局部、正交**的小改动：只动 **dispatch**，作用域不碰，布局也不碰。正因为「三要素互不打架」这个设计，**kernel** 从第一版一路升级到生产级，才能拆成一小步一小步稳稳地走，而不是每次都推倒重来。

动手练 (让你的 agent 搭把手): 从第一个 **kernel** 里挑三行——一行拷贝、一行 **MMA**、一行 **TMEM** 读回。让 **agent** 给每行贴上 **scope / layout / dispatch** 三个标签，然后你对着代码里的 **guard**(像 `if warp_id == 0`)、**buffer** 布局，还有 **dispatch=** 参数，核一核它贴得对不对。

五、编译是怎么发生的

其实前面跑测试那会儿，已经悄悄编译过一次了。这里咱凑近瞧瞧，那一步背后到底干了啥。做法很短：把 **PrimFunc** 塞进 **IRModule**，扔给 `tvm.compile(...)` 就完事：

```
target = tvm.target.Target("cuda")
ex = tvm.compile(tvm.IRModule({"main": kernel}), target=target, tir_pipeline="tirx")
```

`tir_pipeline="tirx"` 这个参数, 启动的就是 **TIRx** 下降流水线。它的核心 pass 是 **LowerTIRx**:

结构图

连接关系:

- `IRModule` $\rightarrow$ `LowerTIRx` / 按 `scope/layout/dispatch` 契约 / 把每个 `tile` 原语兑现成指令
- `LowerTIRx` / 按 `scope/layout/dispatch` 契约 / 把每个 `tile` 原语兑现成指令 $\rightarrow$ `host/device` 拆分
- `host/device` 拆分 $\rightarrow$ `finalize`
- `finalize` $\rightarrow$ `Executable` / (可直接调用)

- **LowerTIRx** 才是真正的主角: 它一个一个啃 `tile` 原语, 读懂每个原语的 `scope / layout / dispatch` 契约。换句话说, 前面咱聊的那三个设计要素, 就是在这一步被「兑现」成一条条真指令的 (比如 `Tx.gemm_async`, 正是在这儿被展开成沿 `K` 步进的 `tcgen05.mma` 序列)。
- 再往后是两步常规活儿:`host/device` 拆分加一个 `finalize`, 最后产出个能直接启动的模块。
- 顺带提一句, 你也可以把 `tvm.compile` 写在 `with target:` 块里头, 这样 `kernel` 会自动继承外层的 `target` 上下文,`target` 就不用再传一遍了。

5.1 一切都可检视

这套流程有个特别招人喜欢的脾气: 它对你不藏一点东西。编译结果你能从两个层级上看:

```
kernel.show()                #          TIRx(TVMScript    )
print(kernel.script())        #          ,

#          CUDA C      :
print(ex.mod.imports[0].inspect_source())
```

注意

注意: 这种「往上能读 **IR**, 往下能读生成的 **CUDA C**」的透明度, 不管是调 bug 还是学东西, 都太管用了——高层的意图 (**IR**) 你看得到, 它最后落成啥样的底层代码, 你照样看得到。本节只是个「速写」; 完整的下降故事 (所有 pass、tile 原语的 `dispatch` 怎么解析、`host/device` 怎么拆) 留到《编译器内部》那一章再讲。

六、接下来去哪

就这么一个 `kernel`, 已经够咱认全 `scope`、`layout`、`dispatch` 了, 还顺手看着它编译、跑起来。那接下来去哪? 这三个设计要素, 加上 `kernel` 本身, 各自都拽出了一个后续章节:

后续章节	内容	什么时候去看
TIRx 布局 API	TileLayout 、命名轴、 swizzle 这套张量布局模型——本章 A/B /累加器怎么摆, 全建在它上面	要是你觉得 <code>layout</code> 是三要素里最摸不透的一个, 就从这儿入手

TIRx 语言参考	完整的语言特性：解析器工具、数据类型、buffer 与内存、控制流、线程同步	当你想要一本完整的「词典」，而不只是「导览」时
构建分块 GEMM	把本章 kernel 当 Step 1，一路加上 K 循环累加、空间分块、TMA、warp 专门化 (warp specialization), 最后建成真正能用的 kernel	想看同样这三个要素怎么扩到生产级 kernel? 这是最顺的下一站

小结

- TIRx 是一种「IR 层级」的 GPU 编程 DSL: 它特别在哪? 一边照样让你直接点名硬件 (线程、SMEM、TMEM、屏障、tcgen05 MMA), 一边又把决定 kernel 怎么跑的那几个关键决策抬到结构化 IR 上, 这样编译器才有得检查、有得下降、有得调度。
- 全章的认知抓手, 就是 scope / layout / dispatch 三大设计要素: 分别回答「谁来干」「数据住哪」「走哪条硬件路」。这三个问题抓住了, 你就有了一把通读任意 TIRx kernel 的万能钥匙。
- 一个只做一次 MMA 的最小 GEMM, 就把三要素一次性演全了:Tx.cta.copy(CTA 作用域加载)→ Tx.gemm_async(dispatch="tcgen05")(单线程发射、走 tcgen05 路径)→ Tx.wg.copy_async(warpg 逐行读回)。
- 核心思路是「写 tile, 不写指令»:Tx.gemm_async 就一句,LowerTIRx 会沿 K-atom=16 把它展开成好几条 tcgen05.mma——这些你一条都不用手写。
- 三要素互不打架, 正是 TIRx 好演进的根子: 后面想把线程拷贝换成 TMA, 只动 dispatch 就行,scope 和 layout 一个都不用动。
- 编译流程 tvml.compile(..., tir_pipeline="tirx") 全程透明:IR 能看 (.show() / .script()), 生成的 CUDA C 也能看 (inspect_source())。
- 说到底, 本书后面那些章节, 干的就是一件——把这 kernel 里的三个要素「放大到真实规模」。

延伸阅读

- 原文章节:Introduction to TIRx
- 全书主页:Modern GPU Programming for MLsys
- 相关后续章节 (原书内):TIRx 布局 API、TIRx 语言参考、构建分块 GEMM、编译器内部
- 原书的交互演示 (Scope/Layout/Dispatch 行高亮) 无法在本笔记中复现, 建议直接到原文页面体验。

术语对照

中文	English	说明
张量 IR neXt	TIRx (Tensor IR neXt)	本章主角:IR 层级的 GPU 编程 Python DSL
作用域	scope	三要素之一: 哪些线程执行某操作
布局	layout	三要素之一:tile 数据住在哪、怎么排
派发	dispatch	三要素之一: 走哪条硬件路径

中间表示	IR (Intermediate Representation)	编译器可处理的结构化程序表示
下降	lower / lowering	把高层 IR 翻译为更低层指令的过程
共享内存	SMEM (Shared Memory)	CTA 内共享的片上内存
张量内存	TMEM (Tensor Memory)	Blackwell 上专供 Tensor Core 累加的内存
全局内存	GMEM (Global Memory)	设备全局显存
寄存器文件	RF (Register File)	每线程私有寄存器
线程块	CTA (Cooperative Thread Array)	一组协作线程, 对应 CUDA block
warp 组	warpgroup	4 个 warp(128 线程) 组成的协作单元
线程束	warp	32 个 lane 组成的执行单元
通用矩阵乘	GEMM	General Matrix Multiply
矩阵乘累加	MMA (Matrix Multiply-Accumulate)	Tensor Core 的核心操作
Blackwell Tensor Core 路径	tcgen05	第 5 代 Tensor Core 指令族 (tcgen05.mma 等)
混排 / 搅动	swizzle	SMEM 地址重排, 避免 bank 冲突
命名轴	named axes (TLane / TCol)	布局 DSL 中与硬件对应的逻辑轴
协作式	cooperative	一条指令由整组线程协同完成
屏障	mbarrier	异步操作完成同步的内存屏障
张量内存访问	TMA (Tensor Memory Accelerator)	硬件异步批量拷贝引擎
warp 专门化	warp specialization	让不同 warp 承担不同角色的优化手法

第 10 章 · TIRx Layout API

原文:[TIRx Layout API](#)

本章要点

本章要点 (TL;DR)

- 上一章我们在纸上发明了一套记号来描述布局。这一章干的事说白了就一句话: 把那套纸面记号原样搬进编译器, 变成你能 new 出来的对象。这样一来, 你在纸上画的布局, 跟内核里敲的代码, 几乎就是同一行字。
- 要记的东西不多, 就三个: **TileLayout**(仿射布局)、**SwizzleLayout**(拿 XOR 给共享内存换序)、**ComposeLayout**(把 swizzle 叠到 tile 上面)。
- 整套设计的魂就一句话: 一个布局, 把一个逻辑坐标, 映射到一个甚至好几个物理坐标。方向记牢——是「逻辑 → 物理」, 而且物理那头是一个集合。为什么非得是集合? 因为数据要广播 (broadcast) 的时候, 只有“一对多”才说得通。
- TileLayout** 分三步走, 有点像填快递单: **shard / 分片** (写作 $S[\dots]$, 公式里记成 D) 先把基础位置定下来; **replica / 复制** (写作 $R[\dots]$, 记成 R) 再把它抄送到别的地方; **offset / 偏移** (记成 O) 最后整体挪个位置。一句话公式: $L(x) = \{ D(x) + r + O \mid r \in R \}$ 。
- 布局里的轴 (axis) 可不是匿名的维度编号, 每一条都是有名有姓的真实硬件坐标, 比如 **laneid**、**warpid**、**TLane**、**TCol**、**m**、**Bank**。另外配了几个现成的构造器 (**tmem_datapath_layout**、**tcgen05_atom_layout**、**wg_local_layout**), 把高频的硬件布局打包好让你直接拿来用——但别紧张, 它们吐出来的还是普通 **TileLayout**, 没藏什么黑魔法。

前置知识

前置知识: 读这一章前, 最好先懂第 3 章的布局记号 (shape/stride 那一套)、第 9 章的 TIRx 基础, 以及 warp / lane、TMEM、swizzle 这几个词。没把握的话, 先翻一下第 0 章·极简入门, 以及第 3、9 章。本章会默认你已经认识这些词。

10.1 为什么需要 Layout API: 让记号变成对象

先把上一章那套记号在脑子里过一遍。它就三块拼起来: 一个 tile(大矩阵切出来的小方块) 形状, 一组挂在具名轴上的步长 (stride), 外加一个可选的复制项。那个复制项是专门留给“被拷贝、而不是被切开”的数据用的。这一章要干的, 就是把这套记号坐实成编译器真能跑的 API。

好处一句话就能说明白: 纸上写的, 跟代码里写的, 一模一样。比方说你在文档里写下这么一行:

```
S[(128, 256) : (1@TLane, 1@TCol)]
```

这一行可不是写给人看的说明，它真的会构造出一个 `TileLayout` 对象。这个对象能挂 (attach) 到 `buffer` 身上。一旦挂上去，凡是碰这个 `buffer` 的 `tile` 操作，都能直接从布局里读出”这块数据该摆在硬件的哪个角落”。

好处在哪？摆放规则只写一次、只检查一次，后面编译器反反复复地用。你不用在每个算子里都把”每个元素住哪儿”重抄一遍。

那布局是啥时候挂上去的？两个时机：要么从内存池里分配的时候，要么声明 `buffer` 的时候。

```
pool.alloc(shape, dtype, layout=layout)          #
T.decl_buffer(shape, dtype, scope=scope, layout=layout) #      buffer
```

从这一刻起，`buffer` 就自己”背着”物理摆放信息到处走了。`tile` 操作再也不用一遍遍啰嗦每个元素在哪。

所有布局对象都住在同一个模块里：

```
from tvm.tirx.layout import (
    TileLayout, SwizzleLayout, ComposeLayout, #
    S, R, # shard / replica
    laneid, warpid, tid_in_wg, #
    TLane, TCol, m, # TMEM
    tcgen05_atom_layout, tmem_datapath_layout, #
)
```

关键

关键：整套 API 就围着一个念头转——一个逻辑下标，对应的未必是单个物理地址，而是一组挂在具名轴上的物理坐标，也就是一个集合。大多数时候这集合里就一个元素。可一旦用上复制，同一个逻辑元素就会在好几个物理位置同时露面。布局之所以拆成 `shard / replica / offset` 三块，正是为了把这件事支撑起来：`shard` 把元素摆好，`replica` 把它拷到别处，`offset` 再整体平移一下。

10.2 先看几个例子 (Layouts by Example)

别急着抠机制，先看几个典型例子，把手感找到再说。

例 1: TMEM(Blackwell 上专给张量核用的特种内存，见第 0 章)里的累加器 (accumulator，矩阵乘里不断累加部分和的那块寄存器/内存)——只摆放，不复制。

```
acc = TileLayout(S[(128, 256) : (1@TLane, 1@TCol)])
```

这行怎么读？很直白：逻辑的行扔到 `TLane`，逻辑的列扔到 `TCol`。顺嘴提一句命名：TMEM 那一章里硬件坐标叫 `Lane` 和 `Col`，到了 TIRx 记号里就改写成 `TLane`、`TCol`，其实是同一个东西。

例 2: 块缩放 (block-scaled) MMA(matrix-multiply-accumulate，张量核上的矩阵乘累加指令)的缩放因子布局——这回用上了复制。

```
scale_factor_layout = TileLayout(
    S[(32, sf_per_mma) : (1@TLane, 1@TCol)] + R[4 : 32@TLane]
)
```


怎么读?shard 先在 TMEM 里放下一个 32 行的组。replica 接着以 32 个 lane(通道, 即 warp 内的线程编号 0-31, 见第 0 章) 为间隔, 把这个组重复 4 份。这么一来, 这个 32 行的组就把 128 个 lane 的整片 TMEM 铺满了。

例 3: 张量核 (Tensor Core, GPU 上专做矩阵乘的硬件单元) 寄存器片段 (fragment, 一个 tile 分散存到各线程寄存器里的那一小片)——散落在 lane 和 warp(线程束, 一个 warp = 32 个线程的小班, 见第 0 章) 上。

```
frag = TileLayout(
    S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
)
```

这里有两点得留个心。第一, 同一条物理轴可以出现不止一次——你看, 这里就有两个不同的 iter 都往 laneid 上塞数据。第二, 要是你没写明往哪条轴放 (例子里最后那个光秃秃的 1), 那它默认就落到内存轴 m 上。

例 4: 真实内核里, 常见的硬件布局一般直接调构造器拿。

```
acc = tmem_datapath_layout("D", 128, 256)
ld = tcgen05_atom_layout("32x32b", (128, 64), "float32")
```

这些构造器返回的依然是普通的 TileLayout 对象。它们不过是替你把常用布局打了个包, 不是另起炉灶搞出来的新东西。所以你想干啥都行: 检视 (inspect) 它返回的布局、把它跟别的布局拼起来, 或者碰上特别古怪的形状, 索性退回去手写底层的 S[...] / R[...]。

10.3 交互演示 (Interactive Demo) 要点

原文在这儿放了个交互式演示。你可以挑一个预置布局, 改改逻辑形状, 改改 S / R 项, 再选个 dtype 和 swizzle 模式, 然后点中一个元素, 看它到底落到了哪个、或者哪几个物理坐标上。

可惜这是个交互组件, 静态笔记里搬不过来。不过没关系, 下面这张“数据流”图就把它在背后干的活儿概括清楚了——而且这张图基本就是后面整套 API 的精确版, 值得你记住:

结构图

连接关系:

1. 展平 flatten / (行主序 row-major) → 2. 按 shard extents 切分 / 得到 c0, c1, ...
2. 按 shard extents 切分 / 得到 c0, c1, ... → 3. 各分量乘步长累加到具名轴 / ck*sk @ ak, 再加 offset
3. 各分量乘步长累加到具名轴 / ck*sk @ ak, 再加 offset → 有 replica?
- 有 replica? $\xrightarrow{\text{无}}$ 单一物理坐标 / {...}
- 有 replica? $\xrightarrow{\text{有}}$ 枚举所有副本组合 / 得到坐标集合 {...}, {...}

注意

注意: 这个演示最值钱的地方, 就是它把一件事画给你看了: 同一列访问, 不加 swizzle 的时候全挤在一个 bank 里; 一旦配上对路的 swizzle, 立马就散到不同 bank 上去了。有条件的话, 强烈建议去原书亲手玩一玩。

10.4 TileLayout: 主力仿射布局

TileLayout 是你最常打交道的布局对象, 它属于**仿射 (affine)** 那一类——说白了就是” 乘个步长再加起来” 这种规规矩矩的变换。它的写法跟正文记号完全是一套:

```
TileLayout(S[shape : strides]) # shard
TileLayout(S[shape : strides] + R[replica_shape : replica_stride]) # shard + replica
TileLayout(S[shape : strides] + R[...] + offset) # shard + replica + offset
```

- **S** 项就是 **shard spec**(分片规格), 你可以一句话读出来:” 拿一个这种形状的逻辑 tile, 按这些步长, 把它摊到具名轴上去”。
- 要是某个值得同时出现在好几个地方, 就拿 **R** 项把 **shard** 扩一扩。
- 最后还能再叠一个可选的 **offset**。

再往下挖一层: 这几个部件底层都靠一种叫 **iter** 的东西来表示。一个 **iter** 就是一个三元组:

```
(extent, stride, axis)
```

你可以把一个 **iter** 想象成” 在某条具名轴上, 按固定步长走一趟”。这三个数:**extent** 是走几步 (也就是有多少个位置),**stride** 是每步迈多远,**axis** 是在哪条硬件坐标上走。下面咱们把 **shard** / **replica** / **offset** 三块挨个拆开看。

10.4.1 Shard(分片, 记作 D)

shard 就是 **S[...]** 写出来的那一块。它干的活儿是: 把逻辑下标摊到一个或几个 **iter** 上**切开**, 再算出**基础物理坐标**。看个例子:

```
S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
```

这里有 4 个 **shard iter**,**extent** 依次是 8, 2, 4, 2。它们的步长把数据分别送到 **laneid**、**warpid**、又一次 **laneid**、还有默认内存轴 **m** 上。

别觉得陌生, 这其实就是你平时那套” **形状-步长**” 规则的加强版。唯一不一样的地方是: 步长现在挂到了**具名硬件轴**上, 而不再是挂在一个扁平的内存地址上。

10.4.2 Replica(复制, 记作 R)

replica 说的是同一个逻辑元素的**额外物理副本**。它有个要命的特点:**replica iter** 跟逻辑下标一点**关系都没有**。换句话说, 不管你访问的是哪个逻辑元素,**replica** 都照着同一个套路在硬件空间里多铺几份。比如:

```
R[2 : 4@warpid] # warpid , 4 warp
```

关键

关键: 别把复制看成图省事的小聪明, 它是在**老老实实在地照搬硬件本来的样子**。有些数据天生就得在 **warp**、**lane** 或者内存区域之间广播 (**broadcast**) 出去。而” 逻辑 → 物理” 这个方向, 恰好天生就能把这事说清楚——因为它本来就允许一个逻辑元素对应一组物理坐标。

10.4.3 Offset(偏移, 记作 0)

offset 就是一个固定坐标, 它会加到**每一个**结果上。比如:

```
5@warpid  #          warpid          5
```

offset 通常拿来干这么几件事: 把 tile 挪到指定的起点; 给某个独占用途圈一块地方; 或者描述一个”紧贴在另一个 tile 后头”的 tile(俩人共用同一块资源)。

10.4.4 三部分如何协同

一个布局会**按固定的顺序**走这三步, 次序乱不得:

- 1. **shard** 先把基础坐标算出来;
- 2. **replica** 再把这个坐标抄成零个或多个额外副本;
- 3. **offset** 最后把每个坐标都挪一挪。

对逻辑坐标 **x**, 最终结果就是:

$$L(x) = \{ D(x) + r + 0 \mid r \text{ in } R \}$$

这公式不用背, 记住”定位 → 抄送 → 平移”这三步就够了。要是没有 replica,**R** 里就只剩一个零偏移, 结果是个**只装一个元素的集合**; 要是有了 replica, 那每个副本位置都会摊上一个坐标。一个写全乎的布局长这样:

```
layout = TileLayout(  
    S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)] # shard:      tile  
    + R[2 : 4@warpid]                                # replica:    4    warp  
    + 5@warpid                                         # offset:      warpid=5  
)
```

从左往右读就行:shard 把逻辑 tile 摆好,replica 在隔着 4 个 warp ID 的地方又放一份副本,offset 把整体挪到从 **warpid = 5** 起步。要是你手头已经有现成的 iter 对象, 也可以直接拼:

```
TileLayout.from_iters(shard, replica, offset)
```

不过大多数代码还是爱用 **S[...] / R[...]** 记号, 因为它读起来更像数学公式, 一眼就懂。

10.5 具名轴 (Named Axes): 名字本身就是语义

前面一直在念叨”具名轴”, 这里专门掰开讲讲。布局里的轴**不是匿名的维度编号**, 每一条都对应一个真实的硬件坐标, 或者编译器层面的某个摆放坐标。常见的轴有这么几类:

类别	轴名	含义
网格轴 (grid)	bx, by, bz	把工作分布到不同 CTA(cooperative thread array, 即一个 block, 见第 0 章)上

集群轴 (cluster)	cbx, cby, cbz	在一个 CTA cluster(一组能协同的 block 集群) 内部摆放工作
线程轴 (thread)	tx, warpid, laneid, wgid, tid_in_wg, wid_in_wg	描述在 CTA 或 warpgroup(几个 warp 凑成的协作组) 内部的归属
内存轴	m	默认的线性内存轴 (寄存器槽常用它)
二维便签	P, F	用于二维 scratchpad 式摆放
共享内存	Bank	命名共享内存的 bank
TMEM 轴	TLane, TCol	TIRx 对 TMEM 的 Lane / Col 坐标的命名

关键

关键: 轴的名字本身就是布局的一部分。为啥这么强调? 因为两个数值一样的坐标, 硬件含义可能差着十万八千里:1@tx 不等于 1@tid_in_wg,1@laneid 也不等于 1@TLane。布局把这层意思明明白白写死, 你就不用再靠上下文去猜”这一维到底指啥”了。

10.6 前向映射 (Forward Mapping):layout.apply()

所谓”求值一个布局”, 意思就是: 你扔给它一个逻辑坐标, 它给你算出对应的物理落点。要用的 API 是:

```
layout.apply(*coord)
```

- 没复制的时候, 你拿到一个坐标字典;
 - 有复制的时候, 你拿到一组坐标字典。
- 坐标字典说白了就是把轴名映到一个整数位置, 长这样:

```
{"laneid": 7, "warpid": 2, "m": 1}
```

整个求值就四步, 跟前面那张数据流图一一对得上:

- 展平 (flatten):** 先把逻辑坐标按行主序压成一个一维下标。对形状 (S0, S1, ..., Sr-1) 里的坐标 (x0, ..., xr-1), 公式是:
“ flat = x0*S1*S2*...*Sr-1 + x1*S2*...*Sr-1 + ... + xr-2*Sr-1 + xr-1 “
- 切分 (split):** 把上一步那个一维下标, 按 shard 的 extents (e0, ..., en-1) 重新切成几段, 得到分量 c0, ..., cn-1(依然是行主序)。
- 累加 (accumulate):** 每个分量乘上自己的步长, 加到对应的轴上去。具体讲, 要是第 k 个 shard iter 是三元组 (ek, sk, ak), 那分量 ck 就往轴 ak 上贡献 ck*sk。同一条轴上的贡献全加起来, 最后再补上 offset。
- 复制 (replica):** 把 replica iter 施加上去。每个 replica iter 都贡献一个跟逻辑坐标无关的额外偏移; 要是有好几个 replica iter, 布局就把它们所有的组合都枚举出来。

注意

注意: 这里藏着一个特别好用的推论——**布局压根不用把输入形状写死**。它只认一件事: 逻辑 tile 的元素总数, 得等于 shard extents 的乘积。这个数一对上, 光靠”展平 + 切分”就能把整个映射定义得清清楚楚。正是这一点, 让布局变得相当通用。

10.7 案例 A: 张量核寄存器 tile

光说不练假把式, 咱们拿前面的公式跑一个完整的例子。假设有个逻辑 (8, 16) 的 tile, 要分到 2 个 warp(每个 warp 32 个 lane) 上, 每个 lane 自己拿一小段寄存器 (register, 每个线程私有、最快的存储, 见第 0 章) 片段。寄存器槽就用默认内存槽 `m` 来表示。布局长这样:

```
layout = TileLayout(
    S[(8, 2, 4, 2) : (4@laneid, 1@warpid, 1@laneid, 1)]
    + R[2 : 4@warpid]
    + 5@warpid
)
```

随便挑个逻辑元素 (`i`, `j`), 咱们按四步走一遍:

第 1 步行主序展平: `flat = 16*i + j`

第 2 步按 shard extents (8, 2, 4, 2) 切分:

```
c0 = i
c1 = floor(j / 8)
c2 = floor(j / 2) mod 4
c3 = j mod 2
```

第 3 步累加到各轴 (4@laneid, 1@warpid, 1@laneid, 1 对应 `m`):

```
laneid = 4*c0 + c2
warpid = c1
m      = c3
```

加上 offset 5@warpid:

```
laneid = 4*i + floor(j/2) mod 4
warpid = floor(j/8) + 5
m      = j mod 2
```

第 4 步施加 replica `R[2 : 4@warpid]` (在 `warpid` 上加 0 或 4):

```
laneid = 4*i + floor(j/2) mod 4
warpid = floor(j/8) + 5 + 4*r,   r in {0, 1}
m      = j mod 2
```

捋一捋结果: shard 把 tile 放到了 warp 5 和 6 上, replica 又把它拷到 warp 9 和 10。所以同一个逻辑元素, 最后落在了**两组 warp 位置**上:

结构图

连接关系:

- shard 落点 / warpid = 5, 6 $\xrightarrow{+4 \text{ (replica)}}$ replica 副本 / warpid = 9, 10

关键

关键: 这个例子正好讲明白了为啥这套模型非得用”物理坐标的集合”不可。倒过来想一下: 假如你用”物理坐标 $\rightarrow$ 逻辑坐标”的函数, 复制根本就写不出来——因为好几个物理点都会映回同一个逻辑点, 这压根就不是一个合法的函数。反过来, 用”一个逻辑坐标 $\rightarrow$ 多个物理坐标”, 一点障碍都没有。这就是 API 死活要选”逻辑 $\rightarrow$ 物理”这个方向的根本原因。

10.8 案例 B:Blackwell 张量内存 (TMEM)

上个例子用的是线程轴。但同一套布局模型, 其实拿来描述**内存摆放**也照样行——轴不一定非得是线程轴, 换成内存轴一样玩得转。TMEM 靠硬件 Lane 和 Col 坐标来寻址, 在 TIRx 记号里就写成 TLane、TCol。看这个布局:

```
layout = TileLayout(  
    S[(2, 128, 112) : (112@TCol, 1@TLane, 1@TCol)]  
)
```

要是逻辑形状正好就是 (2, 128, 112), 那就省事了: 切出来的分量恰好就是逻辑坐标本身, 不用再折腾换算。对元素 (a, 1, c):

```
TLane = 1  
TCol = 112*a + c
```

- extent 是 128、步长 1@TLane 的那个 iter, 负责把 128 行 TMEM Lane 填满;
- extent 是 2(步长 112@TCol) 和 extent 是 112(步长 1@TCol) 这两个 iter 凑一块儿, 一共盖住 224 列, 也就是 $TCol \in [0, 224)$ 。

TLane \ TCol	[0, 111]	[112, 223]
TLane=0	a=0 段 (112 列)	a=1 段 (112 列)
TLane=1	a=0 段 (112 列)	a=1 段 (112 列)
...	...	...
TLane=127	a=0 段 (112 列)	a=1 段 (112 列)

图注:128 行 TMEM Lane 全部填满;TCol 共 224 列, 左半 [0, 111] 是 a=0 段, 右半 [112, 223] 是 a=1 段。

注意

注意: 这 224 列可不是手滑写错了, 是故意的——TMEM 布局不一定非得凑成 2 的幂。举个例子, 一个块缩放 FP8 GEMM(general matrix multiply, 通用矩阵乘法) 就可能专门挑 224 列的累加器: 因为要是塞满 256 列,TMEM 就装不下了, 没法把两个累加器阶段再加上缩放因

子全放进去。这种”不规整”的形状,Layout API 张口就能写——这正是它”支持通用形状”这个设计目标在发挥作用。

10.9 缩放因子布局 (Scale Factor Layouts): 复制的真正用武之地

前面那个累加器是**纯摆放**: 一个逻辑累加器元素, 只对一个 TMEM 坐标, 一对一。可块缩放 MMA 里的**缩放因子 (scale factor)** 就两码事了——同一个物理组, 经常得让好几个 warp 窗口同时都能看见。这下复制可就大显身手了:

```
scale = TileLayout(  
    S[(32, sf_per_mma) : (1@TLane, 1@TCol)] # shard:    TMEM        32  
    + R[4 : 32@TLane]                       # replica:    32 lane,    4  
)
```

对逻辑缩放坐标 (r, s):

```
shard:      TLane = r ,      TCol = s  
replica    :      TLane = r + 32*q ,      TCol = s ,      q in {0,1,2,3}
```

这么一来, 这个 32 行组就在 TMEM lane 的 0-31、32-63、64-95、96-127 这四个窗口里同时现身了:

lane 窗口	TLane [0, 31]	TLane [32, 63]	TLane [64, 95]	TLane [96, 127]
内容	SF	SF	SF	SF
replica 偏移	原本 (+0)	+32	+64	+96

图注: 四个 warp 大小的 lane 窗口看到的是同一份缩放因子组 (SF), 通过 replica 在 TLane 上分别偏移 0 / 32 / 64 / 96。

这其实就是「跨 GPU 世代的张量核操作数布局」那一章讲的 **warp_x4 广播模式**: 四个 warp 大小的 TMEM lane 窗口, 看到的都是同一份缩放因子组。

当然, 真实场景里这个布局还要再复杂一些。这个”原子”会跟外层 iter(沿 M 行和 K 缩放因子组铺开) 拼到一起。另外, 看缩放因子是什么 dtype, 好几个缩放因子还可能挤进同一个 32 位 TCol 单元——比方说 fp8 缩放因子, 就能把 4 个值打包塞进一个 32 位列单元。除此之外还有两个可选的 iter: 一个”步长为 0 的复用”iter, 一个”流水深度”iter, 分别用来描述跨多次 MMA 的缩放复用, 以及双缓冲。

关键

关键: 这里最该记住的一点是——同一个 TileLayout 模型, 把两种压根不一样的情况一锅端了。累加器是 TMEM 里的”单点摆放”; 缩放因子是同一片 TMEM 地址空间里的”复制摆放”。就一套模型, 两类需求全搞定。

10.10 现成布局 (Ready-Made Layouts)

写内核的时候, 谁也不乐意每个硬件布局都从头手敲。所以 TIRx 替那些天天用到的布局备好了现成的构造器, 直接调就完事:

构造器	返回内容	关键参数说明
<code>tmem_datapath_layout(datapath, rows, cols)</code>	<code>tcgen05.mma</code> 写出的 TMEM 累加器布局	<code>datapath</code> 选择行摆放模式:"D" 对应 M=128 的恒等式摆放,"F" 对应 M=64 的散布式摆放
<code>tcgen05_atom_layout(instr_shape, tensor_shape, dtype)</code>	由一个 <code>tcgen05.ld</code> / <code>tcgen05.st</code> 原子搬运的寄存器 <code>tile</code> 布局	<code>instr_shape</code> 如 <code>.32x32b</code> 、 <code>.16x64b</code> 、 <code>.16x128b</code> 等
<code>wg_local_layout(cols, rows=128)</code>	<code>warpgroup</code> 本地寄存器 <code>tile</code>	通常每个线程一行, 落在 <code>tid_in_wg</code> 上

这里有个关于 `tcgen05_atom_layout` 的小门道值得留意: 它在”DSL 层” 和”降级 (lowering) 层” 长得不是一个样。在 DSL 这一层, 它就是个 `warpgroup` 分布式 `tile`; 可一到降级阶段, 它就被拆成四条 `warp` 协同 (`warp-collective`) 的 `tcgen05.ld` / `tcgen05.st` 指令, 一个 `warp` 摊一条, 每条管自己那 32 个 TMEM lane。

结构图

连接关系:

- DSL 层:`tcgen05_atom_layout(...)` / 一个 `warpgroup` 分布式 `tile` → `warp 1`: `tcgen05.ld/st` (lane 32-63)
- DSL 层:`tcgen05_atom_layout(...)` / 一个 `warpgroup` 分布式 `tile` → `warp 2`: `tcgen05.ld/st` (lane 64-95)
- DSL 层:`tcgen05_atom_layout(...)` / 一个 `warpgroup` 分布式 `tile` → `warp 3`: `tcgen05.ld/st` (lane 96-127)

注意

注意: 这些 helper 不过是替你省掉手写常见硬件映射的功夫, 它们并没有把模型藏起来——每个 helper 吐出来的, 都是拿前面那套 S / R 部件拼成的普通 `TileLayout`。

10.11 SwizzleLayout 与 ComposeLayout: 仿射之外的换序

`TileLayout` 是仿射的: 它能描述具名轴上的步长、复制和偏移。线程片段、TMEM `tile`、紧凑缩放因子布局这一大类摆放, 它都拿得下。

可有一样它就是搞不定, 那就是共享内存 (SMEM, 一个 `block` 内线程共享的片上内存, 见第 0 章) 的 `swizzle`(换序, 把数据在存储体间打乱摆放以避免冲突)。`swizzle` 是用来躲开 `bank` 冲突 (`bank conflict`, 多个 `lane` 同时挤一个存储体的不同地址, 被迫排队串行) 的, 它根本不是仿射那种步长模式, 而是对线性共享内存地址来一次基于 `XOR` 的置换 (`permutation`)。这种重排硬塞进仿射模型里, 别扭得很。

所以 TIRx 索性把 `swizzle` 单拎出来做成一个布局对象, 再让它跟 `tile` 布局组合着用:

```
swizzle = SwizzleLayout(...) #
layout = ComposeLayout(swizzle, tile) # swizzle tile
```


执行顺序一目了然: 先让 **tile** 布局把线性内存地址算出来, 再让 **swizzle** 把这个地址重排一遍。把这两层拆开, 比硬把 XOR 置换往仿射模型里塞, 要干净利落得多。

结构图

连接关系:

- 线性内存地址 $m \xrightarrow{\text{SwizzleLayout(XOR 置换)}} \text{换序后地址 } \text{addr}$
- 换序后地址 $\text{addr} \rightarrow \text{实际共享内存落点} / (\text{bank 被打散})$

10.12 为什么要 swizzle:bank 冲突的来由

得先把 bank 冲突是怎么来的讲清楚, 你才明白 swizzle 到底在治啥病。

共享内存被切成 **32 个 bank**, 每个 bank 的字 (word) 是 4 字节。规矩是这样: 一次访问里, 要是好几个 lane 同时去碰**同一个 bank 里的不同地址**, 硬件就只能排着队一个一个来, 这次访问就被**串行化 (serialized)** 了, 慢一大截。

麻烦就麻烦在: 一个老老实实的行主序 tile, **从结构上**天生就会撞上这种冲突。咱们拿一个 (8, 64) 的 float16 tile、行主序布局来瞧瞧:

```
TileLayout(S[(8, 64) : (64@m, 1@m)])
```

逻辑元素 (i, j) 的线性地址是 $m = 64 \cdot i + j$ 。一行有 64 个 float16, 也就是 128 字节——**正好是一整条共享内存 bank line**。问题就卡在这儿: 假设一个 warp 沿着某一列 (j 固定) 往下读, 每往下挪一行, 地址就整整跨过一条 128 字节的 line。跨完一条 line, bank 索引又绕回原处, 于是这一列读下来, **所有行全挤在同一个 bank 上**, 这冲突是跑不掉了。

那 swizzle 怎么治? 思路是: **让地址的低位去依赖更高的行位**。这么一搞, 原本死活落在同一个 bank 的那一行, 就被打散到不同 bank 上去了。

10.13 swizzle 变换的数学形式

思路讲明白了, 咱们把 swizzle 的数学形式摆出来。别一看位运算就发怵, 先弄清每个参数是干啥的。一个 SwizzleLayout 由三个整数管着:

```
per_element = M #
swizzle_len = B # XOR
atom_len = S # XOR
```

输入是线性元素地址 m , 变换分几步走。先拿大白话过一遍: **m 最低的 M 位原封不动** (这是为了让一小撮连续元素继续挨在一起); 剩下的高位拽下来当临时值 $x = m \gg M$; 接着把 x 里 $[S, S+B)$ 那段位组**异或 (XOR)** 到 $[0, B)$ 这段上; 最后再把刚才没动的低 M 位拼回去。写成伪代码就是:

```
mask = (1 << B) - 1
low = m & ((1 << M) - 1) # M ,
x = m >> M #
x2 = x ^ ((x >> S) & mask) # [S, S+B) XOR [0, B)
addr = (x2 << M) | low # M
```

注意

注意: 这套变换要成立, 前提是 $S \geq B$ 。还有一点务必记牢: **swizzle 不会动 tile 里到底有哪些逻辑元素**, 它只动这些元素落到共享内存的哪个位置。MMA 读到的照样是同一个逻辑 tile, swizzle 不过是把背后那套物理 bank 排布优化了一下罢了。

10.14 如何选择 swizzle 参数

好消息是: 平时你基本不用亲自去定这三个参数——它们会根据 **dtype** 和共享内存 **swizzle** 模式自动推出来。常见的 swizzle 模式就 32 字节、64 字节、128 字节这三种。

不过道理还是值得搞明白。per_element(也就是 M) 怎么选? 原则是: 让一个向量大小的小组保持连续。拿 float16 说, 一个 16 字节向量能装 8 个元素, 所以 $M = \log_2(8) = 3$ 。再配上 128 字节 swizzle, 布局就这么写:

```
SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)
```

这套参数一箭双雕: 既保住了 16 字节向量组不被拆散, 又足够把更大范围的共享内存地址模式搅乱, 从而把列方向的 bank 冲突消掉。于是, 一个带 swizzle 的共享内存分配就长这样:

```
tile    = TileLayout(S[(8, 64) : (64@m, 1@m)])
swizzle = SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)
layout  = ComposeLayout(swizzle, tile) # SMEM buffer
```

关键

关键: 绝大多数代码根本不该自己去手算这些参数——dtype 和 descriptor 模式一般早就把它们定死了。你真正得操心的就一件事: TIRx 布局里的 **swizzle**、**TMA descriptor**、还有 **MMA** 的预期, 这三家必须对得上、保持一致。

10.15 元素的 bank 与 line

怎么验证一个 swizzle 是不是真管用? 办法挺简单: 把 swizzle 之后的元素地址换算回它落在哪个 bank, 看看是不是散开了。设 **addr** 是 swizzle 后的元素地址, **b** 是每个元素的字节数:

```
byte = addr * b #
bank = floor(byte / 4) mod 32 # 32 bank, bank 4
line = floor(byte / 128) # 128 bank line
```

对 float16, $b = 2$, bank 公式就简化成 $\text{bank} = \text{floor}(\text{addr} / 2) \bmod 32$ 。下面那段演算用的就是这个。

10.16 完整演算: 对 (8, 64) float16 tile 施加 128B swizzle

理论讲完了, 咱们把整件事从头到尾算一遍, 看看冲突是不是真没了。还是那个行主序 float16 tile:
`m = 64*i + j`, 套上 `SwizzleLayout(per_element=3, swizzle_len=3, atom_len=3)`。把它代进变换公式:

```
x      = m >> 3
addr = ((x ^ ((x >> 3) & 7)) << 3) | (m & 7)
```

令 $q = \text{floor}(j/8)$ 、 $r = j \bmod 8$, 可简化为:

```
addr = 64*i + 8*(q XOR i) + r
```

盯住列 $j = 0$ 看 (这时 $q = 0$ 、 $r = 0$), 得到 `addr = 72*i`。咱们一行一行地算 bank(用 `bank = floor(addr/2) mod 32`):

行 i	addr = 72*i	bank = floor(addr/2) mod 32
0	0	0
1	72	4
2	144	8
3	216	12
4	288	16
5	360	20
6	432	24
7	504	28

看结果: 这一列现在散在 8 个互不相同的 bank 上, 冲突没了。

再对比一下不加 swizzle 的情形: 同一列的地址是 `m = 64*i`, 于是 `bank = floor(64*i / 2) mod 32 = 0`——每一行全挤在 bank 0, 整次访问被彻底串行化。差距一眼就看出来了。

情况 (列 j=0)	8 行 i 落到的 bank	结果
无 swizzle	全部塞进 bank 0	串行化, 约 8x 慢
有 128B swizzle	散到 bank 0, 4, 8, 12, 16, 20, 24, 28	无冲突

注意

注意: 可别把这个”无冲突”当成包治百病的护身符, 它是有前提的: 你得照着 swizzle 当初的设计意图来用。dtype、swizzle 宽度、访问形状, 统统得跟 TMA、MMA 的 descriptor 模式对上。一个 128 字节 float16 swizzle, 是专门冲着特定的 16 字节行块和张量核访问模式设计的——它可不保证你随便来个什么共享内存访问都能无冲突。

10.17 设计原理 (Design Rationale)

学到这儿, 咱们回头看看整套 Layout API 背后那三条设计取舍。你会发现, 前面每个例子其实都在替它们做注解:

1. **支持通用形状**。硬件 tile 可不总是 2 的幂。全局张量、共享内存阶段、TMEM 累加器、缩放因子缓冲，常常因为容量见顶或者算法上的取舍，长成“不规整”的形状。布局模型把这种形状当家常便饭来对待（还记得 10.8 那个 224 列的累加器吧）。
1. **映射方向是「逻辑坐标 → 物理坐标」**。这个方向为啥要紧？因为复制太常见了：一个逻辑元素可能同时住在好几个物理位置。”逻辑 → 物理”这个方向，正好能把这点直接写成一个坐标集合（回想一下 10.7 那个 warp 复制）。
1. **硬件轴明着写出来**。布局不玩匿名维度，也不靠”回头看上下文”去猜每一维是啥意思。`tx`、`tid_in_wg`、`laneid`、`warpid`、`TLane`、`TCol` 彼此的区别，统统白纸黑字写进布局本身。

最后还有一条很要紧的分工：**布局不管合法性、也不管可行性，那不是它一个人扛的活儿**。布局就管一件事——“数据搁哪儿”；至于“某个操作能不能合法、又高效地用上这种摆放”，那是上层 tile 原语 (primitive) 拍板的事。这么一分工，Layout API 既保持了小巧，又把足够的信息喂给编译器，让它去派发 (dispatch) 真正的硬件操作。

小结

- TIRx Layout API 把上一章那套纸面布局记号**坐实成了对象**，核心就三个类：`TileLayout`、`SwizzleLayout`、`ComposeLayout`。
- `TileLayout` 是仿射布局，分三步走——**shard(S, 定位) + replica(R, 抄送) + offset(0, 平移)**，底层全靠 (`extent`, `stride`, `axis`) 这种 **iter** 三元组来表示。一句话公式 $L(x) = \{ D(x) + r + 0 \mid r \in R \}$ 抓住了魂：一个逻辑坐标 → 一组物理坐标。
- `layout.apply()` 的求值就四步：**展平 → 按 shard extents 切分 → 乘步长累加到具名轴再加 offset → 枚举 replica 组合**。它不把输入形状写死，只要”逻辑元素数 = shard extents 的乘积”就行。
- 轴是**具名硬件坐标**，名字本身就带着意思（记牢 `1@laneid` $\neq$ `1@TLane`）。两个案例（张量核寄存器 tile、Blackwell TMEM）说明同一个模型既能描述线程摆放、也能描述内存摆放；缩放因子布局则演示了 replica 怎么自然地把 `warp4` 广播表达出来。
- `swizzle` 是 **XOR 置换、不属于仿射**，所以单拎出来做成 `SwizzleLayout`，再用 `ComposeLayout` 叠到 tile 上。它只动物理落点、不动逻辑内容，靠”低位依赖高位”把 bank 打散，从而消掉列访问的 bank 冲突——但前提是得跟 TMA / MMA 的 descriptor 模式严丝合缝地对上，否则白搭。
- 三条设计哲学串起来：**支持通用（非 2 的幂）形状、逻辑 → 物理的集合映射、把具名硬件轴明着写出来**。布局只管”搁哪儿”，合法性交给上层 tile 原语去判断。

延伸阅读

- 原文：[TIRx Layout API —Modern GPU Programming for ML Sys](#)
- 前置章节：《数据布局及其记号 (Data Layout and Its Notation)》——本章记号的来源
- 相关章节：《特种内存:TMEM(Special Memory: TMEM)》《跨 GPU 世代的张量核操作数布局 (Tensor Core Operand Layouts Across GPU Generations)》

术语对照

中文	English	说明
仿射布局	affine layout	<code>TileLayout</code> 表达的步长/复制/偏移类布局
分片 (规格)	shard / shard spec	由 <code>S[...]</code> 构建, 产出基础物理坐标
复制 (规格)	replica / replica spec	由 <code>R[...]</code> 构建, 生成与逻辑无关的额外副本
偏移	offset	加到每个结果上的固定坐标
迭代器 (三元组)	iter	<code>(extent, stride, axis)</code> , 描述一条具名轴上的带步长行走
具名轴	named axis	真实硬件坐标或编译器摆放坐标, 名字即语义
前向映射	forward mapping	逻辑坐标 $\rightarrow$ 物理坐标 (集合) 的求值
换序 / 拌序	swizzle	基于 XOR 的共享内存地址置换, 用于避开 bank 冲突
bank 冲突	bank conflict	多 lane 访问同一 bank 不同地址导致的串行化
块缩放 (MMA)	block-scaled (MMA)	带缩放因子的矩阵乘累加
缩放因子	scale factor	块缩放 MMA 中的缩放系数
广播	broadcast	同一数据在 warp/lane/内存区域间被复制
降级	lowering	DSL 层布局编译为具体硬件指令的过程
张量内存	TMEM (Tensor Memory)	Blackwell 上由 Lane/Col 寻址的特种内存
线程束 / warp	warp	32 个 lane 的执行单元
warp 组	warpgroup	多个 warp 组成的协作单元

第 11 章 · 构建分块 GEMM(Step 1-3)

原文:构建分块 GEMM(Step 1-3)

本章要点

本章要点 (TL;DR)

- GEMM($D = AB^T$) 是几乎所有深度学习算子的底层原语; 本章只追求**正确**, 性能留给后两章, 先建立一个可信的基线 (baseline)。
- 采用「一次只改一个决策」的增量式构建:Step 1 单 Tile 顺序 GEMM 建立基线契约,Step 2 加 K 维循环累加,Step 3 用多 CTA 网格做空间分块。
- 把每一步都读成对同一份「契约三要素」的编辑——**scope(由谁执行)/ layout(操作数 tile 布局)/ dispatch(走哪条派发路径)**, 大多数步骤只动其中一项。
- 基线数据通路恒定不变:GMEM $\rightarrow$ SMEM $\rightarrow$ tcgen05.mma $\rightarrow$ TMEM $\rightarrow$ GMEM; 后续优化只改「每一跳怎么做得更高效」, 从不增删跳本身。
- 两个最隐蔽的正确性陷阱: 协作拷贝后 MMA 读 SMEM 前必须 `cta_sync`(等齐 + 发布写入); 复用 `mbarrier` 时 K 循环里必须 `phase_mma ^= 1`, 否则 `try_wait` 会提前放行, 静默算错。
- 累加器在 TMEM 中用 `fp32` 保存「运行和」压住舍入误差, 写回时才收窄 (`cast`) 为 `fp16`;Step 3 暂不复用相邻 CTA 的 operand, 带宽浪费留待持久化调度解决。

前置知识

前置知识: 读这一章前, 最好先懂 GPU 的执行/存储层级 (warp、CTA、SM、SMEM/GMEM) 和矩阵乘法的基本概念 (GEMM、tile、MMA、Tensor Core)。没把握的话, 先翻一下第 0 章 · 极简入门, 以及第 9 章的 TIRx scope / layout / dispatch 模型。本章会默认你已经认识这些词。

本章导读: 从一个正确的 Tile 开始 (Building a Tiled GEMM)

这一章是整本书的转折点。前面几章一直在讲抽象——TIRx 的 scope / layout / dispatch 这套模型。从这里开始, 我们要动真格的: 把这套模型落到一个真实的内核 (kernel) 上, 亲手写出第一个正确的分块 GEMM(tiled GEMM)。

为什么整本书都围着 GEMM 转

GEMM(通用矩阵乘法 / General Matrix Multiplication) 是现代机器学习里最核心的一个计算。你看, 线性层、注意力里的各种投影、还有一大堆卷积的实现, 扒到底层全是一次矩阵乘法。说白了, GPU 上的时间, 绝大部分都花在 GEMM 上。

正因为它这么重要,「算得对的 GEMM」和「算得快的 GEMM」之间的差距就特别要命。这个差距是什么? 其实就是「让芯片大半算力干等着」和「把芯片喂得饱饱的」之间的差距。同样一段乘法, 写得好能把硬件榨干, 写得差就让 Tensor Core(GPU 里专做矩阵乘加的硬件单元, 见第 0 章)大量闲着。

注意

注意: 本章的目标只有一个字——**对**。性能 (performance) 留给后面两章。这里我们要的, 是一个能算出正确答案的最小可信基线 (baseline)。

为什么不一步到位, 而要分步搭建

一个能跑满硬件的 GEMM 内核, 会一下子把好几件难事全压你身上: 内存搬运、K 维累加、分块 (tiling)、还有 Tensor Core 的调度。一上来就写这种内核, 等于逼你**同时调试这一整套东西**。更要命的是, 你手里连个可信的对照都没有, 真出了错, 根本说不清是哪一环坏的。

所以更稳的走法是: 先从「能算对的最小内核」开始, 然后**一次只改一个决策**, 慢慢往上长。每一步只动一个地方, 一出问题就知道准是这步引进来的。这种一点一点搭起来的做法, 就是本章——乃至后面两章——从头到尾的套路。

结构图

分组:

- **不可取的做法: 一步到位:**「内存搬运 + K 累加 + 分块 + Tensor Core 调度 / (同时出错, 无从对比)」
- **本章采用的做法: 增量构建:**「最小正确内核」「处理完整 K 维」「处理完整尺寸矩阵」

连接关系:

- 最小正确内核 $\xrightarrow{+ \text{ K 维累加 }}$ 处理完整 K 维
- 处理完整 K 维 $\xrightarrow{+ \text{ 跨 CTA 的空间分块 }}$ 处理完整尺寸矩阵

本章的三步路线图

本章从一个 128×128 的输出 tile(大矩阵切出来的小方块, 见术语对照表) 起步, 一步步长成能算完整尺寸矩阵的内核。三步各管一摊:

步骤	名称	引入的核心变化
Step 1	单 Tile 顺序 GEMM	只算一个输出 tile, 建立后续一切修改所基于的基线
Step 2	K 维循环累加	增加沿 K 维度的循环累加, 处理完整的收缩 (contraction) 维度
Step 3	跨 CTA 的空间分块	在多个 CTA(Cooperative Thread Array, 一组协作的线程 / 即线程块, 共享 SMEM, 见第 0 章) 之间做空间分块, 覆盖完整矩阵

Step 1 搭的是基线, 后面每一步都只是在它上面「改一个地方」而已。

把每一步读成「对一份契约的编辑」

理解这条路线，有个特别好使的角度：把每一步都当成在改**同一份契约 (contract)**。这份契约说白了就三条：

条款	含义
scope(作用域)	哪个执行单位来跑这个操作
layout(布局)	操作数 tile 使用哪种布局
dispatch(分派)	走哪条分派路径来真正执行它

大多数步骤，主要也就动这三条里的一条。所以本章每一步开头，我都会先给你一张小卡片，点明这步到底改了哪一条，顺带把「想安全复用，还得补哪些同步」标清楚。

注意

注意：「scope / layout / dispatch」这条主线，会一路贯穿到后面两章。你要是把每个新内核都读成「它在上一个内核上改了契约里的哪一条」，会比把它当成一个全新内核轻松太多。

本章在三章优化路径中的位置

本章是「同一条 GEMM 优化路径」三连章里的**头一章**。这三章共用一个内核，一层叠一层、特性越来越多，而不是每章都推倒重来：

结构图

连接关系：

- 本章：构建正确的分块 GEMM / (线程拷贝 + 朴素数据路径) → 下一章：用 TMA 做流水线 / (TMA 异步搬运 + 软件流水线)

- 本章：把一个正确的分块内核搭出来，然后**就停手**——性能一概不碰。
- 下一章 (用 TMA 做 GEMM 流水线)：把朴素的线程拷贝换成 TMA(Tensor Memory Accelerator, 一个专管异步搬数据的硬件单元, 不占线程), 再用流水线 (pipelining, 让搬数据和算数据这两件事在时间上叠着干、不互相干等) 让搬数据和算数据重叠起来。
- 再下一章 (用 Warp 特化与 Cluster 扩展 GEMM)：再往上加 warp 特化 (warp specialization, 让不同的 warp 各干一种活, 如搬数据 / 算 / 写回分开) 和 CTA cluster(把多个 CTA 编成一组协作的「集群」)。

每一章都踩在前一章的肩膀上，内核一路**攒特性**，而不是从零再来一遍。这就是「一次改一个决策」放大到章节这个尺度的样子。

GEMM: 问题定义与数据通路 (GEMM)

GEMM 就是稠密矩阵相乘，它埋在几乎每一个深度学习算子的底下。线性层、注意力里的各种投影、很多卷积的实现，扒到最后都是一次矩阵乘法。正因为它无处不在，**只要把 GEMM kernel 写快，好处就会摊到整个模型上**——这也正是本章要从零搭一个分块 GEMM 的理由。

本章采用的形状约定

本教程的所有例子都算 $D = AB^\top$ ，三个矩阵的形状和元素定义如下：

矩阵	形状	含义
A	$M \times K$	左操作数 (operand)
B	$N \times K$	右操作数
D	$M \times N$	输出 (accumulator 最终写出的结果)

逐元素的定义是沿着收缩维 K 求和：

$$D[m, n] = \sum_k A[m, k] \cdot B[n, k]$$

注意

注意：这个转置 $B^\top$ ，不是我们额外多做的一步，它是数据本来就那么存出来的结果。看 B 的形状： $N \times K$ ，也就是「 N 行，每行 K 个数」。这恰好就是线性层权重平时的摆法。所以沿 K 维做收缩时，顺着内存把 B 读下来，读到的天然就是 $B^\top$ ，根本不用挪一个字节。换句话说，选 $D = AB^\top$ 这个写法，是为了贴着真实权重布局、让访存顺手，而不是平白多算一次转置。

性能度量:TFLOPS

全衡量一个 kernel 快不快，统一看吞吐量 **TFLOPS**(每秒万亿次浮点运算)。算法很简单：一次「乘加」记 2 次浮点运算 (乘一次、加一次)，再拿总浮点运算量除以墙钟时间 (wall-clock time)：

$$\text{TFLOPS} = \frac{2 \times M \times N \times K}{t_{\text{seconds}} \times 10^{12}}$$

这个公式有几个点值得记住：

- 分子 $2 \times M \times N \times K$ 是**算法本身定死的浮点运算量**，只跟问题多大有关，你 kernel 怎么写它都不变。
- 分母里能动的只有 t_{seconds} 这一个量。所以后面所有优化，归根结底都是一件事：**同样多的运算，用更短的时间跑完**，TFLOPS 自然就上去了。
- 至于除以 10^{12} ，纯粹是把单位从 FLOPS 换成 TFLOPS。

GEMM 数据通路 (GEMM Data Path)

本章后面**每一项优化**，说到底都在回答两个问题：**数据现在待在哪块存储里，以及它怎么搬过去**。所以动手写代码之前，先把数据流捋清楚，绝对值得。

往大处看，Blackwell(NVIDIA 的一代 GPU 架构，本章代码即面向它；前几代是 Hopper、Ampere) 上的一个 GEMM kernel 翻来覆去就干两件事：

- 搬 (move)：**在不同层级的存储之间搬 tile(数据分块)；
- 算 (compute)：**对搬进来的 tile 做矩阵乘累加。

下面这张图，跟着一个 tile 从输入一路走到输出，把它路过的每一层存储都标出来。这是一条**基线 (baseline) 通路**。后面每一步优化，改的都只是这条链上「某一跳具体怎么走」，但**这些跳本身，一个都不增、一个都不删**。

结构图

连接关系:

- GMEM / (全局显存) $\xrightarrow{\text{① 搬入操作数 A、B 的 tile}}$ SMEM / (共享内存)
- SMEM / (共享内存) $\xrightarrow{\text{② MMA 消费 SMEM 里的操作数}}$ tcgen05.mma / (MMA 计算单元)
- tcgen05.mma / (MMA 计算单元) $\xrightarrow{\text{③ 把 accumulator (累加结果) 写入}}$ TMEM / (张量内存)
- TMEM / (张量内存) $\xrightarrow{\text{④ epilogue 把 TMEM 读回}}$ registers / (寄存器)
- registers / (寄存器) $\xrightarrow{\text{⑤ 把结果存回 GMEM}}$ GMEM / (全局显存)

把这条从左到右的链记死, 它的关键节点是:

阶段	动作	源 → 目的地
① 加载操作数	操作数 tile 进片上	GMEM → SMEM
② MMA 计算	tcgen05.mma 消费 SMEM 里的操作数	SMEM → MMA 单元
③ 写累加器	MMA 把累加结果 (accumulator) 写进张量内存	MMA 单元 → TMEM
④ epilogue(尾处理)	把累加结果读回寄存器	TMEM → 寄存器
⑤ 存回结果	输出写回全局显存	寄存器 → GMEM

这里冒出来的几个名词, 后面会反复用, 先有个直观印象:

- **GMEM / 全局显存:** 容量最大、离计算最远、带宽也最吃紧, 输入和最终输出都放它这儿。
- **SMEM / 共享内存:**CTA(线程块 / Cooperative Thread Array) 内部共享的片上高速存储, 操作数 tile 先搬到这儿,MMA 才好读。
- **tcgen05.mma:**Blackwell 上跑 MMA(矩阵乘累加 / Matrix Multiply-Accumulate) 的指令, 真正干乘加活的就是它。
- **TMEM / 张量内存 (Tensor Memory):**Blackwell 新加的一层存储, 专门放 MMA 累加器,MMA 算完的结果直接写这里。
- **epilogue / 尾处理段:**kernel 收尾的那一段, 负责把 TMEM 里的累加结果搬回寄存器 (顺手做点缩放、激活之类), 再写出到 GMEM。

注意

注意: 后面 Step 1 / Step 2 / Step 3 的所有优化, 都不会动「GMEM → SMEM → MMA → TMEM → 寄存器 → GMEM」这条链的整体形状。它们改的只是**每一跳怎么走得更快**——比如换 TMA 异步搬、用 mbarrier 搭流水线、让多个 CTA 在空间上分担。先把这条不变的主干刻进脑子, 再看每一步在哪一跳上做文章, 一下就清楚了。

优化路线图 (Optimization Path)

前面讲的那条「朴素数据通路 (plain data path)」——把操作数从 GMEM 搬到 SMEM、用 Tensor Core 算出来、再写回 GMEM——确实能算出**对**的答案。可对不等于快。这条最省事的通路, 几乎让整块芯片都在干等: 搬数据的时候计算单元闲着, 算数据的时候拷贝引擎闲着, 大半硬件都没喂饱。

接下来要干的, 说白了就是: 在「数据要走哪几层内存」这条主链不动的前提下, 一项一项把 Blackwell 的硬件本事补齐, 每次只加一个特性, 把那些空转的硬件填满。**每个特性都对应一个 TIRx 的**

tile primitive(tile 原语)。这样就不用从零重写 kernel, 只同一份代码上「一次改一处」地往前推。

注意

注意: 这一小节其实是后面两章的「目录预览」。本章 (Step 1-3) 只管把正确的分块 GEMM 搭出来; 下面这六个优化特性, 会在《用 TMA 做 GEMM 流水线》和《用 warp 特化与 cluster 扩展 GEMM》里一个一个落地。读到这儿, 你心里有个「接下来往哪走」的地图就够了。

为什么要分步走, 而不是一步到位

朴素通路最大的好处, 是它**靠得住**: 它给的是一个正确结果, 后面所有优化都拿它当对照基准 (ground truth)。要是一上来就写个能跑满芯片的 kernel, 你就得同时盯着内存搬运、累加、分块、Tensor Core 调度这一大坨, 而且手里连个可信的参照都没有, 错了都不知道找谁对。

所以这里的打法是: **先弄一个能算对的最小 kernel, 然后每次只加一个特性**。每加一项, 都能单独验两件事——「它有没有把答案搞坏」「它有没有带来该有的提速」。问题被圈在一个小范围里, 排查起来才不抓瞎。

六个优化特性概览

下面这六项, 按加入的先后排好, 正是后面两章要走的路。它们是一层叠一层, 不是谁替换谁:

顺序	特性	核心作用	解决的问题
1	TMA 异步搬运 (TMA async movement)	用 Blackwell 的硬件拷贝通路在 GMEM ↔ SMEM 之间搬运 tile, 并用 mbarrier 跟踪拷贝完成	让数据搬运不再占用线程, 可与计算解耦
2	软件流水线 (Software pipelining)	用多个 SMEM stage(多缓冲), 让”取下一个 K tile”与”对当前 K tile 做 Tensor Core 计算”在时间上重叠	消除”搬完才能算、算完才能搬”的串行等待
3	持久化调度 (Persistent scheduling)	维持一个固定数量的 CTA 池, 每个 CTA 通过 tile scheduler 连续处理多个输出 tile, 而不是”一个 tile 启动一个 CTA”	减少 CTA 启动/销毁开销, 提高占用率
4	Warp 特化 (Warp specialization)	把生产者 (producer)、MMA 消费者 (MMA consumer)、写回 (write-back) 三种角色拆分到不同的 warpgroup 上	让不同职责的 warp 各司其职、并行流转
5	CTA 集群 (CTA clusters)	让两个 CTA 协作, 共同处理一个更大的 Blackwell MMA tile	突破单 CTA 的 tile 规模上限, 提升单次 MMA 的有效尺寸
6	多消费者执行 (Multi-consumer execution)	用多个消费者 warpgroup 同时计算 tile 的不同部分	进一步提高计算密度 (compute density), 榨干 Tensor Core

这些特性是如何叠加的

这六项，你可以当成对同一条数据通路的「一层层加料」。从最素的「线程拷贝 + 单 stage + 单 CTA 单 tile」起步，一路堆到「硬件异步搬运 + 多缓冲流水 + 持久化 + 角色特化 + 跨 CTA 协作 + 多消费者并行」：

结构图

连接关系：

- 朴素通路 (本章 Step 1-3) / 线程拷贝 GMEM→SMEM, 单 stage, 单 CTA 处理单 tile → + TMA 异步搬运 / (搬运交给硬件拷贝通路,mbarrier 跟踪完成)

注意

注意：整条路线都绕着同一组「契约三要素」转——**scope**(谁来执行)、**layout**(操作数 tile 用什么布局)、**dispatch**(走哪条派发路径)。后面每一步，基本都只是在这份契约上改一处：动一个主要的点，再顺带交代「为了安全复用还得补哪些同步」。本章的 Step 1 负责把这份契约的**基线版本**立起来，之后的优化全是在这个基线上「打补丁」。

第 1 步：单 Tile 顺序 GEMM(Step 1: Sequential Single-Tile GEMM)

从零搭一个 GEMM，聪明的做法不是一上来就奔着高性能去，而是先写一个**能把整条硬件数据通路完整跑一遍、却简单到一个循环都没有**的版本。Step 1 就是这么个「最小可运行内核」：它只算一个 128×128 的输出 tile,K 也只有 64。规模小到什么都不用循环——数据通路上的每一段 (GMEM、SMEM、TMEM、寄存器) 都恰好只出现一次。这样我们就能把每一跳 (hop) 单独看个明白，不必同时跟循环、流水线缠在一起。

这一步确立了什么：基线契约

写代码之前，先把这个内核的「形状」说定。它定下来的，就是后面每一步都要沿用的基线 (baseline)：

维度	约定
范围 (Scope)	单个 warpgroup(4 个 warp = 128 个线程组成的执行单位；一个 warp 是 32 个线程的小班，见第 0 章) 顺序地 走完整条通路，一个阶段接一个阶段，中间不重叠
数据布局 (Layout)	A、B 两个操作数 tile 放在 SMEM；累加器 (accumulator) 放在 TMEM；结果经由寄存器 (register) 暂存后写回
派发方式 (Dispatch)	同步的 Tx.copy 负责搬运数据；tcgen05 负责执行 MMA(矩阵乘加)

注意

注意：这里特意挑了「同步搬运 + 顺序执行」，而不是 TMA / 流水线。就是想先把**正确性和数据通路的全貌**讲透。性能优化是后面一项一项加的，不归 Step 1 管。

单 Tile 的数据流向 (Single-Tile Dataflow)

基线契约定好之后，下一件要钉死的，是一个 **tile** 沿这条通路走的先后顺序。第一个内核把核心 GEMM 数据通路完整走一遍——也就是经典的 $\text{GMEM} \rightarrow \text{SMEM} \rightarrow \text{TMEM} \rightarrow \text{寄存器} \rightarrow \text{GMEM}$ ——外面一个循环都不套。它先分配工作内存，加载操作数，算出乘积，写回结果，最后收拾干净走人。

整个流程长这样：

结构图

连接关系：

- Asmem in SMEM / swizzled 布局 $\rightarrow$ tcgen05 MMA / 单线程发起 tile op
- Bsmem in SMEM / swizzled 布局 $\rightarrow$ tcgen05 MMA / 单线程发起 tile op
- tcgen05 MMA / 单线程发起 tile op $\rightarrow$ fp32 accumulator / 128x128 in TMEM
- fp32 accumulator / 128x128 in TMEM $\xrightarrow{\text{tcgen05.ld} / \text{异步加载}}$ 寄存器 / 每线程 1 行
- 寄存器 / 每线程 1 行 $\xrightarrow{\text{cast fp32} \rightarrow \text{fp16}}$ D in GMEM / fp16 输出

对应的五步是：

1. **分配 (Allocate)**: 开出 SMEM(用 pool allocator 池式分配器)、TMEM(tcgen05.alloc), 再加一个 mbarrier(内存屏障)。
2. **加载 (Load)**: 128 个线程一起协作, 把 A、B 两个 tile 从 GMEM 拷到 SMEM(同步 Tx.copy)。
3. **计算 (Compute)**: 挑一个线程出来发起 Tx.gemm_async + tcgen05.commit; 其他所有线程在 mbarrier 上等待结果出来。
4. **写回 (Writeback)**: warpgroup 把结果从 TMEM 读进寄存器; 每个线程把 fp32 转 (cast) 成 fp16, 再写到 GMEM。
5. **释放 (Deallocate)**: 把 TMEM 还回去。

第一个内核的四个部件 (Four Pieces)

完整内核就几十行, 但拆成四块来读更好啃: **内存分配、同步加载、MMA 派发、写回**。这里用到的 API, 名字全来自 Part II 讲过的 TIRx tile primitive(分块原语)。

部件一: 内存分配

内核一开头, 先给操作数划出共享内存, 再给 TMEM 地址和 mbarrier 各留一个槽位:

```
pool = T.SMEMPool()
tmem_addr = pool.alloc((1,), "uint32")          # TMEM      (4      )
mma_bar = pool.alloc((1,), "uint64", align=8)    # mbarrier(8      ,      8      )
pool.move_base_to(1024)                          #              1024
Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout) # 128x64 fp16
Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout) # 128x64 fp16
pool.commit()
```

这里有两个细节, 值得多想一下:

- `pool.move_base_to(1024)` 为什么要「跳到偏移 1024」? 它把两块大的操作数 tile(Asmem、Bsmem)推到 1024 这个偏移之后, 把前面的低地址区留给那几个小元数据 (TMEM 地址槽、

mbarrier)。为什么非这么干？因为 TMA 和 `tcgen05.mma` 对操作数的对齐有讲究，这么大的操作数 `tile`，必须落在一个干净的边界上。

- **layout=A_layout 为什么非得是 swizzled(交错) 布局？**它向 `tma_shared_layout` 要的，是一种 `swizzle` 过的 SMEM 摆法，这样 TMA 和 `tcgen05.mma` 都能直接读。这正是 Part II 一再强调的「布局即契约 (layout-as-contract)」：布局不是随便摆的，它得同时对上搬运硬件和计算硬件的格式胃口。

部件二：同步加载

缓冲区开好了，操作数还得真搬进 SMEM。第一版，我们就让 CTA(协作线程阵列 / Cooperative Thread Array) 自己的线程去拷：

```
Tx.cta.copy(Asmem[:, :], A[:, :]) # CTA          ,
Tx.cta.copy(Bsmem[:, :], B[:, :])
T.cuda.cta_sync()                  #              ,          SMEM
```

因为这里就一个 `tile`($M = N = 128, K = 64$)，把整个 A 和 B 拷过去，加载就算干完了。`Tx.cta.copy(...)` 让整个 CTA 一起协作拷，每个线程管自己那一片。

紧跟着的 `T.cuda.cta_sync()` 一身二职：它既等每个线程都拷完，又把它们写进共享内存的内容发布 (**publish**) 出去。这样后面 MMA 去读 `Asmem`、`Bsmem` 时，看到的才是一个填满的完整 `tile`，而不是只填了一半的缓冲区。

注意

注意：这种「线程自己来拷」的做法，是我们头一个要换掉的东西。下一章(用 TMA 做 GEMM 流水线)就把它换成 TMA。这里先留着它，纯粹是为了让 Step 1 简单直白。

部件三:MMA 派发

操作数现在已经躺在 SMEM 里了，可以发起 MMA 了。注意：发起它的，只用挑出来的那一个线程：

```
if warp_id == 0:                # :      warpgroup      warp 0
    if T.ptx.select_sync():      # :      warp          lane
        Tx.gemm_async(tmем[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
                      accum=False, dispatch="tcgen05", cta_group=1)
        T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)
```

这两层嵌套的 `if`，是在一层层把「发起者」往小里掐：外层 `if warp_id == 0` 先只留下 `warpgroup` 的 `warp 0`，内层 `if T.ptx.select_sync()` 再在这个 `warp` 里挑出唯一一个活跃 `lane`(线程)。两层一叠，最后就剩一个线程去跑 `Tx.gemm_async` 和 `tcgen05.commit`。

这里有个特别容易理解错的地方，得先掰开说清楚：

注意

注意：「一个线程发起」绝不等于「一个线程在做乘法」。计算照样是一个完整的、`tile` 级别的 MMA。硬件会照着 SMEM 操作数布局 and TMEM 累加器布局，一起协作把整个 `tile` 的乘法做掉。关键就一句：`Tx.gemm_async` 是一个 `tile 操作 (tile operation)`，不是一条硬件指令。

为什么这么说？因为 $K = 64$ 的 tile，比硬件 MMA 的 K 原子 ($\text{MMA_K} = 16$) 宽多了。所以这一个 tile op 会被降级 (lower) 成一小串沿 K 维往前走的原始 `tcgen05.mma` 指令，整个 warpgroup 一起协作把每一条都驱动起来。

那为什么发起这串指令，只用一个线程？因为底层每一条 `tcgen05.mma` 本身就是一个一发起就协作的操作：发一次，就能驱动整个 tile MMA 里的那个 K 原子。要是让 128 个线程都去发这串序列，同一份活就被重复发了 128 遍——纯属浪费。

最后，`accum=False` 这个标志是告诉 MMA：直接覆盖 (overwrite) TMEM 目标，而不是往里累加 (add into)。这里我们要的就是覆盖，因为压根没有上一轮的部分和 (partial sum) 要接着用。

部件四：写回 (Writeback)

乘积现在躺在 TMEM 里，可调用方要的，是回到 GMEM、而且是 fp16 的结果。所以收尾阶段 (epilogue) 得把结果经寄存器带下来，顺手把类型转了：

```
Dreg = T.alloc_local((BLK_N,), acc_type)      # fp32
Dreg_f16 = T.alloc_local((BLK_N,), d_type)    # , fp16
Dreg_wg = Dreg.view(128, BLK_N,               # warpgroup
                    layout=TileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)]))
Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N]) # :TMEM -> ( tcgen05.ld)
T.ptx.tcgen05.wait.ld()                        # ,
Tx.cast(Dreg_f16[:, :], Dreg[:, :])           # fp32 -> fp16
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id) #
Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :])
```

累加器为什么用 fp32？MMA 在 TMEM 里留下的，是一个 128×128 的 fp32 累加器 tile。这个 fp32 是特意选的。GEMM 要沿 K 维把一大堆乘积加起来，用更高的精度存这个「运行和 (running sum)」，能把本来会越来越大的舍入误差摀住。但输出 D 是 fp16，所以这些值不能直接出去：得先落到寄存器，在那儿收窄 (narrow) 成 fp16，然后才能写到 GMEM。

两个寄存器缓冲各管什么？Dreg 是个每线程一份、长度为 BLK_N 的缓冲；Dreg_wg 则是同一批寄存器换了个布局后的 warpgroup 级视图 (view)：

```
TileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)])
```

这个布局把 tile 的第一维摊到 warpgroup 的线程上：线程 0 拿第 0 行，线程 1 拿第 1 行，一直到第 127 行。第二维则留在每个线程自己的寄存器缓冲里，所以每个线程攥着自己那一行的全部列。warpgroup 有 128 个线程，tile 有 128 行，于是这个 128×128 的输出正好一人一行，分得清清楚楚。

照这个视图把累加器读出来，就是 `Tx.wg.copy_async(Dreg_wg, tmem)` 干的活；它会降级成 Blackwell 的 TMEM 加载通路 `tcgen05.ld`。这个加载是异步的，所以在任何线程去碰 Dreg 之前，都得先等 `T.ptx.tcgen05.wait.ld()` 返回。不然线程就会读到加载还没填好的寄存器。

等这个 wait 返回，每个线程私有的 `Dreg[:, :]` 里，就装着它那一行逻辑输出的 fp32 值了。线程把它们收窄成 fp16 存进 `Dreg_f16`，再算出自己该写哪个全局行：

```
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
```


然后写到 `D[m_thr, n_st:n_st + BLK_N]`。这些行在四个 warp 之间分得很整齐:

Warp	负责写的行
warp 0	第 0-31 行
warp 1	第 32-63 行
warp 2	第 64-95 行
warp 3	第 96-127 行

完整内核 (Complete Kernel)

现在把四个部件拼成一个能跑的完整内核 ($M = N = 128, K = 64$)。先看导入:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_shared_layout, SwizzleMode
from tvm.tirx.layout import TileLayout, S, TLane, TCol, tid_in_wg
```

内核照后面各步统一的 `hgemm_vX(M, N, K)` 样式包起来。Step 1 跑 $M = N = 128, K = 64$, 所以整个启动 (launch) 里就一个输出 `tile`。下面是内核主体, 关键的地方都加了中文注释:

```
def hgemm_v1(M, N, K):
    a_type = tvm.DataType("float16"); b_type = tvm.DataType("float16")
    d_type = tvm.DataType("float16"); acc_type = tvm.DataType("float32")
    BLK_M, BLK_N, BLK_K = 128, 128, 64
    # MMA_M/N/K           MMA tile           ;           gemm_async
    # (           /           tile           MMA           ),
    MMA_M, MMA_N, MMA_K = 128, 128, 16
    #   A B       swizzled  SMEM           ,TMA   tcgen05.mma
    A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_ATOM, (BLK_M, BLK_K))
    B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_ATOM, (BLK_N, BLK_K))

    @T.prim_func
    def kernel(A: T.Buffer((M, K), a_type),
               B: T.Buffer((N, K), b_type),
               D: T.Buffer((M, N), d_type)):
        T.device_entry()
        #   tile           :M=BLK_M, N=BLK_N,           grid   1x1;   CTA   tile
        # (m_st, n_st)           0 Step 3+           M/N
        bx, by = T.cta_id([M // BLK_M, N // BLK_N])
        wg_id = T.warpgroup_id([1])           #   warpgroup, wg_id   0(           )
        warp_id = T.warp_id_in_wg([4]); lane_id = T.lane_id([32])

        # --- SMEM           (           ) ---
        pool = T.SMEMPool()
        tmem_addr = pool.alloc((1,), "uint32")
        mma_bar = pool.alloc((1,), "uint64", align=8)
        pool.move_base_to(1024)
        Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
        Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
        pool.commit()

        # ---           + TMEM           (   warp 0) ---
        if warp_id == 0:
```

```

        if lane_id == 0:
            T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1) # 1
            T.ptx.tcgen05.alloc(T.address_of(tmем_addr), n_cols=512, cta_group=1)
            T.ptx.fence.proxy_async("shared::cta") # SMEM
            T.ptx.fence.mbarrier_init() # mbarrier
            T.cuda.cta_sync()

        # TMEM ( alloc tmem_addr)
        tmem = T.decl_buffer((128, 512), "float32", scope="tmem",
            allocated_addr=tmem_addr[0],
            layout=TileLayout(S[(128, 512) : (1@TLane, 1@TCol)]))
        m_st = T.meta_var(bx * BLK_M); n_st = T.meta_var(by * BLK_N)
        phase_mma: T.int32 = 0

        # --- : GMEM -> SMEM( )---
        # tile , Step 3 diff
        Tx.cta.copy(Asmem[:, :], A[m_st:m_st + BLK_M, :])
        Tx.cta.copy(Bsmem[:, :], B[n_st:n_st + BLK_N, :])
        T.cuda.cta_sync()

        # --- : MMA( )---
        if warp_id == 0:
            if T.ptx.select_sync():
                Tx.gemm_async(tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
                    accum=False, dispatch="tcgen05", cta_group=1)
                T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)
            T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) # MMA

        # --- :TMEM -> -> GMEM( )---
        Dreg = T.alloc_local((BLK_N,), acc_type)
        Dreg_f16 = T.alloc_local((BLK_N,), d_type)
        Dreg_wg = Dreg.view(128, BLK_N,
            layout=TileLayout(S[(128, BLK_N) : (1@tid_in_wg, 1)]))
        Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N])
        T.ptx.tcgen05.wait.ld()
        Tx.cast(Dreg_f16[:, :], Dreg[:, :])
        m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
        Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :])

        # --- TMEM ---
        T.cuda.cta_sync()
        if warp_id == 0:
            T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
            T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_group=1)
        return kernel

```

注意

注意:mbarrier.init(..., 1) 的期望计数 (arrival count) 给的是 1, 因为要等的事件只有「MMA 完成」这一个。tcgen05.commit 会在整串 MMA 跑完后给这个 mbarrier 发个信号, 全体线程则卡在 mbarrier.try_wait 上, 等它翻 phase(相位)。

配套的编译与校验脚手架

后面每个 GEMM 步骤, 编译、运行、自检的套路都一样。所以这套脚手架在这儿**完整写一遍**, 之后就只贴内核本身。想跑哪个后续步骤, 把它 `hgemm_vX` 和对应的问题规模换进来就行。

```
import torch
target = tvn.target.Target("cuda"); device = torch.device('cuda')
M, N, K = 128, 128, 64
kernel = hgemm_v1(M, N, K)
with target:
    ex = tvn.compile(tvn.IRModule({"main": kernel}), target=target, tir_pipeline="tirx")

A_tensor = torch.randn(M, K, dtype=torch.float16, device=device)
B_tensor = torch.randn(N, K, dtype=torch.float16, device=device)
D_tensor = torch.zeros(M, N, dtype=torch.float16, device=device)
ex.mod(A_tensor, B_tensor, D_tensor)          # torch

D_ref = (A_tensor.float() @ B_tensor.float().T).half()
#           :           K           ,           K
torch.testing.assert_close(D_tensor, D_ref, rtol=2e-2, atol=1e-2)
print("PASS")
```

注意

注意: 有个坑容易踩——一个新的 Python 会话只能编译一个步骤, 换步骤之前得重启。原因是这些例子复用了内部名称, 而编译器又攥着一份按会话 (per-session) 走的状态。

Step 1 到 Step 3 都故意用很小的规模 (这里 128×128 , Step 3 用 256^3), 就是为了让最开始这几次讲解好跟。后面「用 Warp 特化与 Cluster 扩展 GEMM」一节末尾那张跨步骤端到端结果表则反过来: 它把**每一个**步骤 (包括 Step 1 这套算法) 都放到统一的 $M = N = K = 4096$ 规模下跑一遍, 这样各步骤之间的加速比才能直接比。

单 Tile 内核的局限 (Limits)

这个内核是**对的**——这正是 Step 1 想要的。但它的「对」只在一个特别窄的设定下才成立。有四个局限是**故意**埋进去的, 后面的优化路径会一个一个把它们拆掉:

#	局限	由哪一步解决 (方向)
1	只处理 单个 K tile , 无法在大 K 上做收缩 (contract)	引入 K 维循环
2	只处理 单个输出 tile , M、N 被钉死在 128	推广到多 tile / 网格 (Step 3)
3	用 同步 的 GMEM $\rightarrow$ SMEM 拷贝, 而非 TMA	下一章换成 TMA
4	数据搬运与计算 不重叠 , 两者永远不会同时进行	引入流水线 (pipelining)

第 2 步:K 维循环累加 (Step 2: K-Loop Accumulation)

第 1 步只能啃一个宽 64 的 K 切片 (K tile), 可真实矩阵的收缩维度 (contraction dimension) 常常远不止 64。第 2 步要补的, 就是这个最直接的短板: 输出 tile 还是只算一块, 但让 K 维能横跨任意多个 64 宽的小块。

思路其实很素: 把「加载 → MMA → 等待」这一套照 K 的块数重复跑, 每次 MMA 都累加进同一个 TMem 槽位。难点不在算, 在同步。同一个 mbarrier 跨迭代反复用, 就会冒出本章头一个真正的正确性陷阱: 代码要是把相位 (phase) 跟丢了,wait 可能在它该等的那次 MMA 还没算完时就提前放行, 于是不声不响地算出错误结果。下面我们就把这个错怎么发生、又怎么躲开, 讲个透。

这一步改变了什么: 复用既有布局

维度	第 2 步的变化
作用范围 (Scope)	不变, 仍然是单个 warpgroup。
布局/复用 (Layout/reuse)	同一对 SMem tile 和同一个 TMem 累加器槽位在同一个 K-loop 中反复复用。不再分配任何新存储: 操作数 tile 像「流水」一样穿过这对固定缓冲区, 累加状态始终待在同一个 TMem 槽位里。
同步 (Synchronization)	被复用的 MMA barrier 必须在每个 K 块上正确推进相位, 否则后续的 wait 会观察到更早一次的完成信号。
调度 (Dispatch)	不变。

说到底, 第 2 步没添一个字节的新存储, 它全部的复杂度就压在一件事上: 「怎么安全地把一个 mbarrier 反复用」。

K-loop 的运行机制

第 1 步只在一个宽 64 的 K tile 上做收缩。这回我们保留它那唯一的输出 tile, 但放开 K, 让它按矩阵真实的需要爱伸多长伸多长。做法是以 BLK_K=64 为步长沿 K 往前走, 每一轮把下一块 A、B 的 K 切片加载进 SMem, 然后发起一次 Tx.gemm_async。

把这些零碎小块「缝」成一个完整点积, 靠的就是 accum 这个标志:

```
Tx.gemm_async(tmem[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
              accum=(i != 0),          # 0 :accum=False,          TMem
              dispatch="tcgen05", cta_group=1)
```

- 第一块 (i==0):accum=False, 把 TMem 累加器初始化 (直接写进去, 旧值丢掉)。
- 后面每一块 (i!=0):accum=True, 把这一块的乘积加到已经躺在 TMem 里的运行总和上。

同步: 相位翻转 (phase flip) 才是真正的考点

关键

关键: 先记住一句话——同一个 mbarrier 被反复用, 所以软件得自己记住「这一轮我该等它翻到哪一相」, 记错了就等错。

每次 MMA 完成, 我们用的都是同一个 mbarrier。想安全地反复用它, 核心就一件事: 盯住「我们现在到底在等哪个相位」。

mbarrier 内部带一个 **1 比特的相位** (0 或 1)。每凑齐一次预期的到达 (arrival), 这个相位就翻到另一个值。微妙就微妙在 `try_wait` 的判定条件上:

注意

注意:`try_wait(bar, phase)` 会一直堵着, 直到 `barrier` 的内部相位跟你传进去的 `phase` 不一样为止。换句话说, 你传进去的那个参数, 代表的是「你预期马上要离开的那个旧相位」, 而不是「你想等到的那个新相位」。

下表把三次迭代的相位变化完整列出来:

K 迭代	等待前本地的 phase_mma	try_wait 在等什么	等待后本地的更新
0	0	barrier 翻到 1	phase_mma = 1
1	1	barrier 翻到 0	phase_mma = 0
2	0	barrier 翻到 1	phase_mma = 1

让这张表一直对得上的, 就是循环里那行毫不起眼的代码:

```
T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) # :barrier != phase_mma
phase_mma ^= 1 # ,
```

下面这张表, 把「本地 `phase_mma`」和「`barrier` 真实相位」怎么一块儿往前走, 摆得明明白白:

迭代	等 待 前 本 地 phase_mma	MMA 完成后 barrier 翻转	try_wait 判定	放行	^=1 后本地更新
迭代 0	0	0 → 1	try_wait(.,0):0 ≠ 1	放行	1
迭代 1	1	1 → 0	try_wait(.,1):1 ≠ 0	放行	0
迭代 2	0	0 → 1	try_wait(.,0):0 ≠ 1	放行	1

要是把 `phase_mma ^= 1` 删了会怎样? 这就是这一步最该警惕的 bug:

- 第二轮迭代还是去调 `try_wait(bar, 0)`;
- 可 `barrier` 在第一次 MMA 之后早翻到相位 1 了;
- 于是 `try_wait` 一进来就发现「当前相位 `1 ≠ 参数 0`」, 当场返回——可这会儿第二次 MMA 压根还没算完;
- 内核接着就去读一个**半成品**的累加器, 报出一个错误答案, 却一声不吭、不抛任何错。

结构图

连接关系:

- 错误时序 (缺少 `phase_mma ^= 1`) / 迭代 1:`try_wait(bar, 0) → barrier 当前相位 = 1(上一轮 MMA 留下的)`

注意

注意: 这是一个能**正常编译**、**正常跑**、却给出错误数值的 bug——不崩、不报错。正因为它这么藏得深，这一行相位翻转才值得花这么多笔墨。一句话: 复用 `mbarrier` 的时候, 「盯住相位」永远是头等大事。

完整内核

完整内核说穿了就是「第 1 步 + K-loop + 相位翻转」叠到一起。导入部分跟之前一模一样:

```
import tvm
from tvm.script import tirx as T
from tvm.script.tirx import tile as Tx
from tvm.tirx.cuda.operator.tile_primitive.tma_utils import tma_shared_layout, SwizzleMode
from tvm.tirx.layout import TileLayout, S, TLane, TCol, tid_in_wg
```

内核包在 `hgemm_v2(M, N, K)` 里。网格 (grid, 本次启动里所有 CTA 排成的阵列; [1, 1] 即只有一个 CTA) 还是 [1, 1], 因为我们依旧只算一个输出 tile。变大的只有它的 **K 跨度**:

```
BLK_M, BLK_N, BLK_K = 128, 128, 64
K_TILES = K // BLK_K          # K          64
```

下面只摘跟第 2 步直接相关的核心结构, 长实现精简掉了:

1) 初始化 (照搬第 1 步): 分配 **TMEM**、初始化 **mbarrier**, 再做一次跨 CTA 同步。

```
if warp_id == 0:
    if lane_id == 0:
        T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1) # mbarrier          =1
        T.ptx.tcgen05.alloc(T.address_of(tmemb_addr), n_cols=512, cta_group=1) #          TMEM

T.ptx.fence.proxy_async("shared::cta")
T.ptx.fence.mbarrier_init()
T.cuda.cta_sync()

phase_mma: T.int32 = 0 #          ,          0
```

2) K-loop 主体 (这步的核心): 每一块都走一遍「加载 → 同步 → MMA → 等待 → 翻相位」。

```
for i in T.serial(K_TILES):
    #          #          ,          K          A/B
    #          i K          SMEM(          )
    Tx.cta.copy(Asmem[:, :], A[:, i*BLK_K:(i+1)*BLK_K])
    Tx.cta.copy(Bsmem[:, :], B[:, i*BLK_K:(i+1)*BLK_K])
    T.cuda.cta_sync()

    if warp_id == 0:
        if T.ptx.elect_sync():
            #          lane          MMA
            Tx.gemm_async(tmemb[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
                accum=(i != 0), #          ,
                dispatch="tcgen05", cta_group=1)
```

```
T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1) # MMA          barrier

T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma) #          MMA
phase_mma ^= 1 #          (          !)
```

3) 写回 (跟第 1 步一字不差): 从 TMEM 取出累加结果, cast 成 fp16, 写回 D, 最后把 TMEM 释放掉。

```
Tx.wg.copy_async(Dreg_wg[:, :], tmem[:, :BLK_N]) # TMEM ->
T.ptx.tcgen05.wait.ld()
Tx.cast(Dreg_f16[:, :], Dreg[:, :]) # fp32 -> fp16
Tx.copy(D[m_thr, n_st : n_st + BLK_N], Dreg_f16[:, :]) #
...
if warp_id == 0:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=1)
    T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_group=1) #          TMEM
```

整个内核的数据流, 大致可以这么概括:

结构图

分组:

- **K-loop(i = 0 .. K_TILES-1):** 「A/B 第 i 块 → SMEM(固定缓冲) / (Tx.cta.copy + cta_sync)」 「TMEM 累加器槽 (跨迭代复用)」 「try_wait(phase_mma) + phase_mma ^= 1」

连接关系:

- A/B 第 i 块 → SMEM(固定缓冲) / (Tx.cta.copy + cta_sync)
 gemm_async(accum = i!=0) 累加进同一个 → TMEM 累加器槽 (跨迭代复用)
- TMEM 累加器槽 (跨迭代复用) → try_wait(phase_mma) + phase_mma ^= 1
- 全局内存 A, B → A/B 第 i 块 → SMEM(固定缓冲) / (Tx.cta.copy + cta_sync)
- K-loop(i = 0 .. K_TILES-1) 循环结束后, TMEM 中是完整点积 → 寄存器 → cast → fp16 → 全局内存 D

你看, 第 2 步在结构上对第 1 步的改动小得可怜: 套一层 K-loop、把 accum 改成看块号、再加上 phase_mma ^= 1 那一行。可偏偏就是这一行, 划开了「能跑」和「能跑对」这道分水岭。

第 3 步: 空间分块, 多 CTA 并行 (Step 3: Spatial Tiling / Multi-CTA)

第 2 步用 K-loop 搞定了收缩维度 (K 维) 的累加, 可输出还是被钉死在一个 128 x 128 的单 tile 上。真实的输出矩阵, 可比一个 tile 大太多了。所以基础内核的最后一块拼图, 就是让 M、N 这两个维度被许多 tile 一起铺满。

第 3 步的核心动作就一个: 开一个二维的 CTA 网格 (grid), 让每个 CTA 包一个输出 tile, 然后让 GPU 把所有 tile 并行算出来。例子特意挑了 M=N=K=256, 正好切成 2x2 共 4 个 tile——既让 tile 索引的逻辑「不那么一眼就看穿」, 又不至于多到把逻辑淹了。

这一步到底改了什么

把内核拆成三个关注点来看，第 3 步只动了其中一个：

关注点	与第 2 步相比	说明
作用域 (Scope)	改变	从单个 CTA 变成二维 CTA 网格，每个 CTA 拥有一个 128 × 128 的输出 tile
布局 (Layout)	不变	单个 CTA 内部的 SMEM → TMEM → 寄存器 (register) 数据通路和第 2 步完全相同
分发 (Dispatch)	不变	MMA 的指令分发方式照旧

换句话说，单个 CTA 内部的计算逻辑一行都没动，变的只是「开几个 CTA」和「每个 CTA 算哪一块」。这恰恰是分块 GEMM 设计漂亮的地方:tile 级别的实现是一个能反复用的原子，要往空间上铺开，只要给它套上对的坐标偏移就行。

网格映射：每个 CTA 找到自己那一块

网格的形状直接由分块方式推出来。既然一个 CTA 对一个 128 x 128 的输出 tile，那一共要的 CTA 数就是：

```
grid = [M // BLK_M, N // BLK_N]
```

跟第 2 步比，真正多出来的活只有一件：告诉每个 CTA，它该算矩阵的哪一块切片。
设 CTA 的二维坐标是 (bx, by)，那它负责的输出区域就是：

```
D[bx * BLK_M : (bx + 1) * BLK_M,      #      :  bx
   by * BLK_N : (by + 1) * BLK_N]      #      :  by
```

为了算出这块结果，这个 CTA 的 K-loop 会反复加载 A 对应的「行带」和 B 对应的「列带」：

```
A[bx * BLK_M : (bx + 1) * BLK_M, k : k + BLK_K]  # A      CTA
B[by * BLK_N : (by + 1) * BLK_N, k : k + BLK_K]  # B      CTA
```

为什么 by 选的是 B 的”行”，却变成了 D 的”列”

这一点最容易把人绕晕，根子在于内核守的是 $D = A @ B.T$ 这个约定——B 是先转置、再参与乘法的：

- bx 一下选中 A 的行和 D 的行，这俩直接对上，好理解。
- by 选的是 B 的行。可 B 要转置 (.T)，这些行一转，就变成了列，于是对到了 D 的列上。

下表把 2x2 网格里各个 CTA 归谁管，摆得清清楚楚（拿 M=N=256、BLK=128 举例）。行方向是 A 的行 (bx)，列方向是 B.T 的列 (by)：

A 的行方向 (bx) \ B.T 的列方向 (by)	by=0	by=1
bx=0	CTA (0,0) —D[0:128, 0:128]	CTA (0,1) —D[0:128, 128:256]

bx=1	CTA (1,0) —D[128:256, 0:128]	CTA (1,1) —D[128:256, 128:256]
------	------------------------------	--------------------------------

每个 CTA 各自跑一遍完整的 K-loop:CTA(bx,by) = $\sum_k A[bx, k] @ B[by, k].T$ 。

注意

注意:bx 是行索引、by 是列索引, 但 by 是绕了 B.T 一圈才映射到 D 的列。要是你把 A、B 的切片偏移搞混, 矩阵乘法在数值上不会报错, 却会给你算出一个完全错的结果——这是多 CTA 分块里最常见的索引 bug。

一个 tile 一个 CTA: 能用, 但费

「一个 tile 配一个 CTA」是能跑通的最简单映射, 可它**特别费内存带宽**:

- 同一行上的所有 CTA(bx 相同、by 不同), 会各自从头到全局内存 (GMEM) 里去拉一模一样的 A 行带。
- 同一列上的所有 CTA(by 相同、bx 不同), 也各自重新加载一模一样的 B 列带。

说白了, 相邻 CTA 明明已经拉进来的数据, 彼此之间一点没省着用。这步先忍着这份浪费。后面《用 TMA 做 GEMM 流水线》的第 6 步会上**持久化调度 (persistent scheduling)** 回头收拾它, 让这些被多个 CTA 共用的操作数尽量「赖在 L2 里保持热乎」, 少往 GMEM 跑重复的活。

和你的 agent 一起练: 就用 M=N=K=256、BLK_M=BLK_N=128、BLK_K=64 这套设置, 让它盯着 CTA (1, 0) 和 CTA (0, 1) 跑。每个 CTA 都写清楚 m_st、n_st、每轮 K 迭代加载的 A/B 切片、还有最后写回的 D 区域。再答一道: 既然内核算的是 $D = A @ B.T$, 到底是 B 的哪些行, 转过身变成了 D 的列?

完整内核: 在第 2 步上只改两处

第 3 步的内核骨子里还是第 2 步, 就动了**两处**:

1. **网格形状:** 从 [1, 1] 改成 [M // BLK_M, N // BLK_N]。
2. **每个 CTA 的偏移:** 加载和写回的时候, 都加上本 CTA 自己的 m_st 和 n_st。

内层的 K-loop 和写回 (writeback) 逻辑**一个字没改**。下面只把体现这两处改动的关键行挑出来, 配上中文讲解;TMEM 分配、mbARRIER 初始化这些样板代码跟第 2 步一样, 这里就略了。

关键改动一: 二维网格 + 计算本 CTA 偏移

```
@T.prim_func
def kernel(
    A: T.Buffer((M, K), a_type),
    B: T.Buffer((N, K), b_type),
    D: T.Buffer((M, N), d_type),
):
    T.device_entry()
    #          :      CTA          128x128          tile
    bx, by = T.cta_id([M // BLK_M, N // BLK_N])
    ...
    #      CTA          tile      D          /
```

```
m_st = T.meta_var(bx * BLK_M) # CTA tile
n_st = T.meta_var(by * BLK_N) # CTA tile
```

这里 `T.cta_id(...)` 顶掉了第 2 步那个写死的 `[1, 1]` 网格, 返回当前 CTA 的二维坐标 `(bx, by)`; `m_st`、`n_st` 再把坐标翻译成本 CTA 的输出 tile 在全局矩阵里的起始行 / 列偏移 (注意是元素下标的偏移, 不是字节偏移)。

关键改动二: K-loop 里给 A/B 切片加偏移

```
for i in T.serial(K_TILES):
    # A:          CTA          m_st:m_st+BLK_M,K          i
    Tx.cta.copy(Asmem[:, :], A[m_st:m_st+BLK_M, i*BLK_K:(i+1)*BLK_K])
    # B:          CTA          n_st:n_st+BLK_N(          D          ),K          i
    Tx.cta.copy(Bsmem[:, :], B[n_st:n_st+BLK_N, i*BLK_K:(i+1)*BLK_K])

    T.cuda.cta_sync()

    if warp_id == 0:
        if T.ptx.select_sync():
            # accum=(i != 0): 0 , 2
            Tx.gemm_async(tmем[:, :BLK_N], Asmem[:, :], Bsmem[:, :],
                          accum=(i != 0), dispatch="tcgen05", cta_group=1)
            T.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)

    T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)
    phase_mma ^= 1
```

跟第 2 步对一下, 这里就一处变了: A、B 切片在 M/N 方向的起点从 0 换成了 `m_st`、`n_st`。K 维照样靠循环变量 `i` 往前滑, `accum=(i != 0)` 的累加语义、`gemm_async` 的 MMA 调用、`mbarrier` 的等待握手, 统统原样不动。

关键改动三: 写回到正确的输出 tile

```
#          D : tile + warp + lane
m_thr = T.meta_var(m_st + warp_id * 32 + lane_id)
#          n_st , CTA
Tx.copy(D[m_thr, n_st:n_st+BLK_N], Dreg_f16[:])
```

写回这块也只动了偏移: 行偏移从 `warp_id * 32 + lane_id` 变成 `m_st + warp_id * 32 + lane_id`, 列范围从 `0:BLK_N` 变成 `n_st:n_st+BLK_N`。`warpgroup` 里的线程照旧各写一行, 只是现在落点是本 CTA 在 D 里那块专属区域, 而不是全局的左上角了。

整体数据流回顾

结构图

分组:

- **K-loop ($i = 0 \dots K_TILES-1$):** 「同步拷贝 A[m_st 行带, i] → SMEM (Tx.cta.copy)」 「同步拷贝 B[n_st 行带, i] → SMEM (Tx.cta.copy)」 「gemm_async 累加到 TMEM (accum = i!=0)」 「mbarrier 等待 MMA 完成」

连接关系:

- 同步拷贝 $A[m_st \text{ 行带}, i] \rightarrow \text{SMEM (Tx.cta.copy)} \rightarrow$ 同步拷贝 $B[n_st \text{ 行带}, i] \rightarrow \text{SMEM (Tx.cta.copy)}$
- 启动 $[M//\text{BLK_M}, N//\text{BLK_N}]$ 个 CTA(本例 $2 \times 2 = 4$ 个), 全部并行 $\rightarrow$ 每个 CTA(bx,by): $m_st = bx * 128, n_st = by * 128$

你看, 第 3 步只靠一层简单的坐标偏移, 就把一个「会算一个 tile」的内核, 扩成了「在 GPU 上一口气铺满整张输出矩阵」。这就是 GPU GEMM 的基本骨架: **tile 内部那条精细的数据通路一动不动, 往空间上铺开就全靠网格 + 偏移**。带宽浪费先搁着, 留给持久化调度去收拾; 但功能上, 这已经是一个能算任意 M、N 大小输出的完整分块 GEMM 了。

习题精解 (Exercises)

这一节用三道题, 把 Step 1-3 的核心要点串起来。三道题看着零散, 其实正好卡在从单 Tile 到 K 循环、再到多 CTA 这条路上三个最容易翻车、也最值得讲透的地方: **共享内存 (SMEM) 的可见性同步、mbarrier 相位 (phase) 的正确性陷阱**, 还有 **多 CTA 网格下的数据复用**。下面每道题都给完整推导和道理, 不只是丢个结论给你。

注意

注意: 做这几道题, 建议对着本章 Step 1-3 那三份完整 kernel 一起看。三个版本内层结构几乎一样, 差别就在「K 循环」和「网格 / 偏移」这两处, 所以同一套同步道理, 会在三步里一遍遍冒出来。

第 1 题: 为什么 Tx.gemm_async 读 SMEM 前必须先 cta_sync?

题目: 在 Step 1-3 里, Tx.copy(准确说是 Tx.cta.copy) 把 A、B tile 从全局内存 (GMEM) 搬进 SMEM, 搬完才做 MMA。那为什么在 Tx.gemm_async 读这些 SMEM tile 之前, kernel 非得插一句 T.cuda.cta_sync()?

核心答案: 因为这次拷贝是 **CTA 里一堆线程一起协作干的**, 而 MMA 又是 **挑一个线程发起、却要把整个 tile 都读进去的**。少了 cta_sync, 发起 MMA 的那个线程什么保证都没有——它不知道别的线程是不是已经把各自那一片写完了、写的东西对自己是不是已经能看见。结果 MMA 很可能读到一个「只填了一半」的缓冲区。

想说透这件事, 得拆成两层看:

1) Tx.cta.copy 是「大家分着搬」, 不是哪个线程一个人扛

```
Tx.cta.copy(Asmem[:, :], A[m_st:m_st + BLK_M, :]) # CTA    128
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st + BLK_N, :])
T.cuda.cta_sync()                                #      +
```

Tx.cta.copy 让整个 CTA 一块儿搬: 128 个线程, 每人管 Asmem / Bsmem 的一小块。也就是说, 拷完之后, Asmem 这块 SMEM 里不同的区域, 是被 **不同线程** 分头写进去的。

2)MMA 由挑出来的一个线程发起，却要读「整块」tile

```
if warp_id == 0:
    if T.ptx.select_sync():
        # CTA
        Tx.gemm_async(tmemb[:, :BLK_N], Asmem[:, :], Bsmem[:, :], ...)
```

发起 MMA 的就一个 lane。可 Tx.gemm_async 是个 tile 级操作: 硬件 MMA 得把 Asmem、Bsmem 这两整块都读进去做协作矩阵乘。也就是说，这一个线程发起的指令，吃的是另外 127 个线程刚写下去的数据。

3) 两个保证，少一个都不行

线程之间不会自动对齐，所以 T.cuda.cta_sync() 在这儿一身二职:

它提供的保证	没有它会怎样
等齐 (同步): 阻塞到 CTA 内所有线程都跑完拷贝	慢的线程可能还没搬完,MMA 就开始读
公布 (可见性 / fence): 让所有线程对 SMEM 的写入对其他线程可见	即使线程都「跑过了」拷贝语句, 写入也可能还停在缓存/写缓冲里,MMA 读到旧值

第二条尤其要命: 哪怕没有竞速, 现代 GPU 的内存模型也不担保一个线程写进 SMEM 的东西, 另一个线程立马就能看见。必须有一个同步 / 栅栏点, 把这些写入「发布」出去。cta_sync 既是 barrier(等齐), 又顺手把发布写入这件事一起办了。

一句话总结: 写 SMEM 的是「一帮人」, 读 SMEM 的是「一个人」。这种跨线程的生产—消费关系, 必须靠 cta_sync 一边等齐、一边公布。不然 MMA 读到的就是一块只填了一半、还可能根本看不全的缓冲区, 然后悄没声地算错。

注意

注意: 这条同步是「先写后读」型 (SMEM 写入 → MMA 读取)。本章后面的 K 循环里还有一条反方向的依赖: 拷下一轮数据之前, 得先确认上一轮 MMA 已经把旧数据读完了, 不然就会「新数据盖掉了还没读完的 buffer」。Step 2/3 还是用同一个 cta_sync 把两个方向一起挡住; 真把这两类依赖拆开、做成流水线, 那是后面 TMA 章节的事了。

第 2 题: 删掉 phase_mma ^= 1 会发生什么?

题目: 在 Step 2 里, 要是把 K 循环里的 phase_mma ^= 1 删掉, 会怎样?kernel 还会乖乖等每一次 MMA 吗, 还是某次 wait 会提前放行?

核心答案:kernel 不会再正确地等了。从第二轮迭代起,try_wait 会当场提前返回, 根本没等这轮的 MMA 算完。于是 kernel 读到一个只累加了部分 K 的「半成品」累加器, 给出错误结果——而且一个错都不报。

想搞懂为什么, 得先把 mbarrier 的相位语义讲清楚。这是整章头一个真正的正确性陷阱。

1)mbarrier 的相位只有 1 个比特, 而且语义「反直觉」

一个 mbarrier 内部带一个 1 比特的相位 (0 或 1)。每凑齐一次预期的「到达 (arrival)」, 相位就翻到另一个值。而等待原语的判断条件是:

```
T.ptx.mbarrier.try_wait(bar, phase)  #          barrier          !=          phase
```

注意这儿反直觉的地方:try_wait(bar, phase) 堵着的条件, 是 barrier 内部相位不等于你传进去的 phase。所以你传进去的那个参数, 应该是「你预期马上就要被翻走的那个旧相位」, 而不是你想等到的那个新相位。

2) 正确情形: 相位一轮一轮翻

每轮迭代里,phase_mma ^= 1 这一行让本地记的相位跟着 barrier 一块儿翻, 两边始终咬合:

K 迭代	wait 前的本地 phase_mma	try_wait 实际在等什么	wait 后本地更新为
0	0	barrier 翻到 1	phase_mma = 1
1	1	barrier 翻到 0	phase_mma = 0
2	0	barrier 翻到 1	phase_mma = 1

3) 删掉翻转后: 第二轮就出事

要是删了 phase_mma ^= 1,phase_mma 就永远卡在 0。一轮一轮跟一下:

```
0:      try_wait(bar, 0)  barrier          =0,          ->          ,
      0      MMA      barrier      1

1:      try_wait(bar, 0)  barrier          1      (          ),
      1 != 0 ->          != phase          -> wait          ,
      1      MMA!
```

也就是说: 第一轮碰巧还对 (初始相位本来就是 0), 可从第二轮起, 每次 try_wait 都瞬间放行——因为本地一直拿 0 去比一个早就变成 1 的 barrier。kernel 不卡死, 会一路跑到底, 然后从 TMEM 读出一个还没把所有 K chunk 累加完的累加器, 给你个错答案。

这就是这类 bug 最阴的地方: 能编译、能跑、不报错, 数值还「看着差不离」(尤其 K 小、误差被相对容差放过去的时候), 实则是个不声不响的错误。phase_mma ^= 1 这一行毫不起眼, 却是整个 K 循环正确性的命门。

注意

注意: 为什么得本地自己记一个相位, 不能让硬件自动管? 因为同一个 mbarrier 被跨迭代复用——硬件就一个 1 比特状态在 0/1 之间来回翻, 软件必须自己盯住「我现在该等的是哪一相」。复用 barrier 的好处是零额外存储, 代价就是这份相位账, 得你自己记对。

第 3 题:Step 3 启动多少 CTA? 相邻 CTA 之间复用了哪些 tile?

题目:M=N=4096、BLK_M=BLK_N=128 的情况下,Step 3 会启动多少个 CTA? 相邻 CTA 之间, 哪些 operand tile 在逻辑上其实是能共用的?Step 3 有没有把这种复用用上?

1)CTA 数量:1024 个

Step 3 的网格直接由 tiling 定下来, 一个 CTA 管一个 128×128 输出 tile:

```
bx, by = T.cta_id([M // BLK_M, N // BLK_N]) #          ,          CTA          tile

代入数字:

tile      = 4096 / 128 = 32
           = 32 x 32
CTA        = 32 x 32 = 1024
```

你可以画成一张 32×32 的输出 tile 网格, 一个格子就是一个 CTA。同一行 (bx 相同) 的 CTA 共用同一条 A 行带; 同一列 (by 相同) 的 CTA 共用同一条 B 列带:

bx \ by	by=0	by=1	...	by=31
bx=0	CTA (0,0)	CTA (0,1)	...	CTA (0,31)
bx=1	CTA (1,0)	CTA (1,1)	...	CTA (1,31)
...	...	...	...	...
bx=31	CTA (31,0)	CTA (31,1)	...	CTA (31,31)

- 每一行 (如 bx=0 这一整行的 32 个 CTA) 共用同一条 A 行带。
- 每一列 (如 by=0 这一整列的 32 个 CTA) 共用同一条 B 列带。

2) 逻辑上可复用的 operand tile

回忆每个 CTA 加载的切片:CTA (bx, by) 在每一轮 K 里加载

```
A[bx*BLK_M : (bx+1)*BLK_M, k : k+BLK_K] # A          ,          bx
B[by*BLK_N : (by+1)*BLK_N, k : k+BLK_K] # B          (          D          ) ,          by
```

关键就一句:A 的切片只看 bx,B 的切片只看 by。于是天然就冒出两类复用机会:

复用方向	谁共享	共享的内容	在 4096 例子里被重复加载的次数
同一行 (bx 相同)	整行 32 个 CTA	相同的 A 行带 (所有 K chunk)	同一份 A tile 被 32 个 CTA 各加载一次
同一列 (by 相同)	整列 32 个 CTA	相同的 B 列带 (所有 K chunk)	同一份 B tile 被 32 个 CTA 各加载一次

换句话说, 理论上每一片 A 从 GMEM 读一次, 就够喂整行 32 个 CTA; 每一片 B 读一次, 就够喂整列。绝大部分 GMEM 流量, 本来是能省下来的。

3) Step 3 有没有把这种复用用上?——没有

没用上。Step 3 走的是「一 tile 一 CTA」这种最素的映射, 每个 CTA 各干各的, 自己的 A 行带、B 列带都从 GMEM 重新拉一遍, 彼此之间一点不共享。结果就是: 同一份 A tile, 同一行的 32 个 CTA 各读了一遍; 同一份 B tile, 同一列的 32 个 CTA 各读了一遍, 大把 GMEM 带宽白扔了。

原文对这点态度很明确: 这份浪费是故意先留着的。Step 3 要的只是把 M、N 铺开、让多 tile 并行跑起来, 把正确性和网格映射讲清楚。复用的事, 留给后面的「持久化调度 (persistent scheduling)」(TMA 章节的 Step 6) 去办——靠让共享的 operand 常驻 L2 缓存, 把相邻 CTA 重复读 GMEM 的开销压下去。

注意

注意: 这里要把两件事分清: 「逻辑上有没有复用机会」和「kernel 实现到底兑没兑现」。复用机会是 GEMM 结构自带的 (A 切片只看 bx、B 切片只看 by), 它一直都在; 但 Step 3 只是让一堆 CTA 把同一份数据各从 GMEM 搬一遍, 压根没去兑现它。想明白这点, 你才能懂后面为什么还要持久化调度这类优化, 以及它们到底在省什么。

小结

本章从一个 128×128 、 $K = 64$ 的单输出 tile 起步, 亲手把抽象的 TIRx scope / layout / dispatch 模型, 落到一个真能跑的分块 GEMM 内核上, 然后顺着「一次只改一个决策」的路子, 分三步把它养大:

- **Step 1(单 Tile 顺序 GEMM):** 用一个 warpgroup 顺序走完 $\text{GMEM} \rightarrow \text{SMEM} \rightarrow \text{MMA} \rightarrow \text{TMEM} \rightarrow \text{GMEM}$ 这条完整通路, 把后面一切修改都依着的基线契约立起来。几个关键点: swizzled SMEM 布局 (布局即契约)、一个线程发起 tile 级 MMA (发起 $\neq$ 一个线程在算)、还有 fp32 累加器写回时收窄成 fp16。
- **Step 2(K 维循环累加):** 不添一个字节新存储, 套上 K-loop, 用 $\text{accum}=(i!=0)$ 把零散的 K 块缝成完整点积。真正的难点在于反复复用同一个 mbarrier 时的相位记账: 少了 $\text{phase_mma} \wedge = 1$, 就会不声不响地算错。
- **Step 3(空间分块 / 多 CTA):** 单 CTA 内部的计算一行不动, 只靠二维网格 + 坐标偏移, 就把内核铺到任意 M、N。重点搞懂两件事: $D = A B^{\sim\{\text{top}\}}$ 下 by 怎么经转置对到 D 的列, 以及「一 tile 一 CTA」带来的、特意先留着的那份带宽浪费。

最值钱的收获, 是一套能搬到别处用的方法论: 把每个新内核都读成「它在上一个内核上改了契约里的哪一条」, 同时对跨线程的生产—消费依赖 (SMEM 可见性、mbarrier 相位) 死盯不放——这类 bug 能编译、能跑、不报错, 却悄没声地给你错数值。

延伸阅读

- 原文: [构建分块 GEMM\(Step 1-3\)](#)
- 下一章《用 TMA 做 GEMM 流水线》: 把本章朴素的线程拷贝换成 TMA (Tensor Memory Accelerator), 并用多 SMEM stage 的软件流水线让数据搬运与计算重叠; 持久化调度 (Step 6) 回头解决本章 Step 3 暂留的 operand 复用 / GMEM 带宽浪费问题。
- 再下一章《用 Warp 特化与 Cluster 扩展 GEMM》: 引入 warp 特化 (producer / MMA consumer / writeback 分到不同 warpgroup) 与 CTA cluster (两个 CTA 协作完成更大的 MMA

tile), 并在统一的 $M = N = K = 4096$ 规模下给出跨步骤端到端加速比对比。

- 本书 **Part II(TIRx tile primitive)**: 本章用到的 `Tx.copy` / `Tx.gemm_async` / `tma_shared_layout` / `TileLayout` 等原语词汇表的来源, 建议回看以理解「布局即契约」。

术语对照

英文	中文	简要说明
GEMM (General Matrix Multiply)	通用矩阵乘法	稠密矩阵相乘, 本章计算 $D = AB^T$; 深度学习几乎所有算子的底层原语
Tile	分块 / 瓦片	矩阵被切成的小块 (本章输出 tile 为 128×128), 内核以 tile 为单位搬运与计算
Tiling / Spatial tiling	分块 / 空间分块	把大矩阵切成多个 tile, Step 3 用多 CTA 网格在空间上并行覆盖完整输出
Contraction dimension (K)	收缩维度	矩阵乘法中被求和消去的维度; Step 2 沿 K 循环累加
Baseline	基线	最小可信的正确实现, 作为后续所有优化的对照基准 (ground truth)
Scope	作用域	契约三要素之一: 由哪个执行单位 (warpgroup / CTA / 网格) 来跑操作
Layout	布局	契约三要素之一: 操作数 tile 在内存中如何摆放 (如 swizzled SMEM)
Dispatch	分派 / 派发	契约三要素之一: 走哪条派发路径真正执行操作 (如 tcgen05)
CTA (Cooperative Thread Array)	协作线程阵列 / 线程块	一组协作线程, 共享 SMEM; Step 3 一个 CTA 负责一个输出 tile
Warpgroup	warp 组	128 个线程 (4 个 warp) 组成的执行单位, 本章顺序走完整条通路
GMEM (Global Memory)	全局显存	容量最大, 离计算最远的存储, 输入与最终输出所在地
SMEM (Shared Memory)	共享内存	CTA 内共享的片上高速存储, 操作数 tile 先搬到此供 MMA 读取
TMEM (Tensor Memory)	张量内存	Blackwell 新增、专门存放 MMA 累加器的存储层级
MMA (Matrix Multiply-Accumulate)	矩阵乘累加	真正执行乘加的运算; tcgen05.mma 是 Blackwell 上的对应指令
Accumulator	累加器	沿 K 维累加部分和的缓冲, 本章在 TMEM 中以 fp32 保存以压制舍入误差
Swizzle / Swizzled layout	交错布局	一种 SMEM 摆放方式, 使 TMA 与 tcgen05.mma 都能直接读取
mbarrier (Memory barrier)	内存屏障	用于跨线程 / 异步同步的硬件屏障, 内部带 1 比特相位
Phase (flip)	相位 (翻转)	mbarrier 的 1 比特状态; 复用时需本地翻转 phase_mma ^= 1 才能正确等待
try_wait	等待原语	阻塞直到 barrier 内部相位!= 传入参数; 参数是「预期要离开的旧相位」
cta_sync	CTA 同步	一身二职: 等齐所有线程 + 发布 (publish) 它们的 SMEM 写入
Epilogue	尾处理段	内核收尾阶段, 把 TMEM 累加结果读回寄存器、收窄类型后写回 GMEM

accum flag	累加标志	<code>gemm_async</code> 参数: <code>False</code> 覆盖 (初始化)、 <code>True</code> 累加进已有 <code>TMEM</code> 值
TFLOPS	每秒万亿次浮点运算	吞吐量度量; 一次乘加算 <code>2</code> 次浮点运算, $2MNK/(t \times 10^{12})$
TMA (Tensor Memory Accelerator)	张量内存加速器	<code>Blackwell</code> 硬件异步拷贝通路, 下一章用以替换本章的线程拷贝
Persistent scheduling	持久化调度	固定 <code>CTA</code> 池连续处理多个 <code>tile</code> , 后续用以复用 <code>operand</code> 、减少 <code>GMEM</code> 重复读

第 12 章 · 用 TMA 为 GEMM 做流水线 (Step 4-6)

原文: [Pipelining GEMM with TMA](#)

本章要点

本章要点 (TL;DR)

- 上一章写出的“正确”分块 GEMM(通用矩阵乘法, 深度学习里最核心的计算) 让最贵的张量核心 (Tensor Core, 芯片上专门干矩阵乘的计算单元) 大部分时间在**空等**: 线程搬一块、张量核心算一块、线程再搬下一块, 两件本可以并行的事被排成了串行。
- 本章不改数据通路 (tile, 即大矩阵切出来的小方块; 以及 layout、数学都对), 只改**何时做、由谁调度**, 核心思想只有一个: **让一个硬件引擎跑在另一个引擎前面** (异步交接)。
- **Step 4**: 把 GMEM(全局内存, 显存里那块大空间)→SMEM(共享内存, 片上那块小而快的) 的搬运交给 TMA(Tensor Memory Accelerator) 硬件, 一个线程下一条命令即可, 用 mbarrier(memory barrier) 按字节数判断搬完没有——但**每次 load 后仍然等**, 所以还没真正重叠。
- **Step 5**: 把 SMEM 双缓冲 (PIPE_DEPTH=2 的环形缓冲), 给“下一块”腾出落脚点并预取 (prefetch), 建立起流水线骨架——但单 warpgroup(4 个 warp、共 128 线程一组, 见第 0 章) 主循环**仍先等当前 MMA 再发下一个 load**, 真正的 load/compute 重叠要到 Step 7 的 warp 专门化 (warp specialization) 才到来。
- **Step 6**: 把 kernel 改成**持久化 (persistent)**, 固定开一池 CTA(线程块, 一组协同的线程, 等同 block; 约一 SM 一个, SM 即流多处理器, GPU 上的计算核心, 见第 0 章), 用 tile 调度器 (tile scheduler) 让每个 CTA 连续处理多个输出 tile, 既摊薄每 tile 的启动开销, 又能挑一个让操作数留在 L2 里的 tile 顺序。

前置知识

前置知识: 读这一章前, 最好先懂第 11 章那个分块 GEMM(把大矩阵切成小方块来算), 以及 TMA(搬数据的专用硬件)、mbarrier(异步完成的“门门”)、双缓冲/流水线 (让搬运和计算两条线错开跑) 这几个词。没把握的话, 先翻一下第 0 章·极简入门, 再回头看第 11 章。本章会默认你已经认识这些词。

这一章属于「第三部分·GEMM 实战 (Step 1→9)」。**上一章我们盯着“算得对不对”, 这一章换个问题: “算得满不满?”——说白了, 就是别让最贵的那块硬件闲着。**

读之前, 先把终点记在心里。这三步, 一步干一件事:

- Step 4: 让专门的拷贝硬件 (TMA) 替线程搬数据;
- Step 5: 给数据腾出双缓冲的落脚点;

- Step 6: 把整个 launch 重塑成持久化形态。

这里有个点要先说清楚: 这三步从头到尾,SMEM、TMEM(张量内存,Blackwell 上专门放 MMA 累加器的片上内存)、寄存器 (register, 每个线程私有的最快存储) 的 layout 一行都没改过。真正算新东西的, 就一个——**硬件单元之间的异步交接**。剩下的全是老零件, 换个用法而已。

为什么基础版 GEMM 在浪费时间

先把问题摊开讲。张量核心 (Tensor Core) 是整块芯片上**最贵的家伙**, 理应一刻不停地转。可上一章那个写法本身没错的分块 GEMM, 偏偏让它大半时间都在干等。

为什么? 因为这个 kernel 是” 一个一个轮着来” 的:

1. 线程把一块 tile 拷进共享内存 (SMEM);
2. 张量核心把这块嚼完;
3. 线程拷下一块;
4. 张量核心又在等。

每一步都得等上一步收工。你可仔细想想: **搬下一块 tile 和 算手里这块**, 根本是两套不同的硬件在干, 完全可以同时进行啊!

要把这条缝补上, 不用动任何数据通路——tile、layout、数学, 现成的全够用。要改的就两样: **什么时候做, 和谁来调度**。

下面拿两张表对一对, 看看” 串着跑” 和” 我们想要的重叠跑” 差在哪 (行 = 硬件引擎, 列 = 时间步, 格子 = 那一刻在干什么):

基础版 (串行, 轮流上场):

引擎	t0	t1	t2	t3	t4
搬运 (拷贝硬件)	load k0	—	load k1	—	load k2
计算 (张量核心)	—	mma k0	—	mma k1	—

拷贝硬件和张量核心从来不一起干活, 各自都有一半时间在空等。

理想版 (重叠, 各司其职):

引擎	t0	t1	t2	t3	t4
搬运 (拷贝硬件)	load k0	load k1	load k2	load k3	...
计算 (张量核心)	—	mma k0	mma k1	mma k2	...

算 k0 的工夫,k1 就已经搬好了, 张量核心几乎一直在转。

关键

关键: 本章三步是**渐进**的。Step 4 把搬运交给硬件 (但还是等);Step 5 搭出流水线骨架并预取 (但单 warpgroup 还是等); 真正” 两条线同时跑满” 的重叠, 要到下一章 Step 7 用 warp 专门化把 TMA 和 MMA 拆成生产者/消费者两个角色才实现。本章打的是地基。

注意

注意: 从 Step 4 开始, 所有例子都跑在完整规模 $M=N=K=4096$ 上 (而非 Step 1-3 的小尺寸), 端到端计时统一放在下一章末尾的 *End-to-End Result* 表里。

Step 4:TMA 异步 Load

这一步只改了” 派发方式”

这一步就动一个地方：原来是线程一块一块同步地拷，现在换成 **TMA load**。别的全不动。拿原书那个三维度框架一对照，就一目了然：

维度	Step 4 的状态
Scope(范围)	不变，仍是一个 warpgroup
Layout(布局)	不变，同样的 SMEM / TMEM / 寄存器 tile
Dispatch(派发)	变了:GMEM→SMEM 的 load 从同步 Tx.copy 改为 TMA 引擎

同步拷贝 vs. TMA 命令: 执行模型的本质差别

代码就改那么几行，可背后的执行模型**根本是两码事**。一句话先抓住区别：同步拷贝是线程自己撸起袖子搬,TMA 拷贝是线程喊一嗓子、剩下的让硬件去搬。

- 同步的 Tx.copy: 这活儿是 **CTA 线程亲自干的**。算地址、发 load/store, 全得线程一条条指令来，搬的整个过程线程都被占着，脱不开身。
- TMA 拷贝: **一个线程发一条命令**，完事。后面的搬运全归 TMA 硬件，自己独立去跑。算地址、合并访存 (coalescing, 把相邻线程的访问拼成一笔大传输)、swizzle(打乱地址摆放以避免 bank 冲突) 这些琐事，早就写进 TMA 描述符 (descriptor) 里了,TMA 引擎照着做就行。

两段代码摆一块儿，差别看得清清楚楚。

改前 (Step 3):128 个线程一起上手拷，再用一句 cta_sync 让 SMEM 的写入对所有线程都可
见:

```
Tx.cta.copy(Asmem[:, :], A[m_st:m_st+BLK_M, i*BLK_K:(i+1)*BLK_K]) # 128
Tx.cta.copy(Bsmem[:, :], B[n_st:n_st+BLK_N, i*BLK_K:(i+1)*BLK_K])
T.cuda.cta_sync()
```

改后 (Step 4): 一个线程发起 TMA load, 硬件搬完没搬完，交给 mbarrier 去盯:

```
tid = warp_id * 32 + lane_id # warpgroup 0..127
if tid == 0: # TMA
    Tx.copy_async(Asmem, A[...], dispatch="tma")
    Tx.copy_async(Bsmem, B[...], dispatch="tma")
    T.ptx.mbarrier.arrive.expect_tx(tma_bar, byte_count) # mbarrier
T.ptx.mbarrier.try_wait(tma_bar, phase) # MMA SMEM TMA
```

为什么是 tid == 0, 而不是 elect_sync()?

这是这一步最容易栽跟头的地方。乍一看，挑线程用哪种写法都行；真不是。

问题出在” 到底挑几个”。**elect.sync** 是**每个 warp 挑一个活跃 lane**(warp 内的线程编号,0–31)。可一个 warpgroup 有 4 个 warp(线程束,32 个线程的小班, 见第 0 章), 这么一来 **elect_sync()** 就挑出了**四个线程**, 四个全冲进了 load 协议。

坏就坏在这。TMA 协议要往 mbarrier 里**报一个预期字节数**, 而这个数必须**不多不少报一次**。报了四遍, 计数立马乱套,**wait** 就再也等不到那个正确的释放点了。更阴的是, 这是个**不声不响的正确性 bug**——程序照跑不崩, 就是悄悄给你算错。

所以对的做法是: 按 warpgroup 的全局 id, 精准点中一个线程——**tid == 0**。

关键

关键: 图里写”Elected Thread(被选中的线程)” 指的是启动 TMA 的那个线程, 在我们的代码里就是 `tid == 0`, 不是 `elect_sync()` 选出来的 lane。两者别混淆。

Load 完成怎么知道:mbarrier + 字节数握手

换成 TMA, 其实一口气改了两件事: **谁来发起拷贝** (这个看代码就懂), 以及**怎么判断拷完了** (这件事最容易被忽略, 可一错就是静默错误)。下面重点讲后一件。

- 用 `Tx.cta.copy` 的时候:CTA 线程一起拷, 后面补一句 `cta_sync()` 就够了。
- 换成 TMA,`cta_sync()` 就不顶用了。为什么? 因为 `cta_sync()` 只管等 CTA 自己那帮线程、只给它们的 SMEM 写入排个序。可那笔正在途中的 TMA 传输, 它压根不知道——tile 还在半道上呢, 它就乐呵呵地返回了。

正确的修法, 是把”搬完了”这件事明明白白地写出来:

1. 被选中的那个线程, 先用 `arrive.expect_tx(total_bytes)` 告诉 mbarrier: 这回要等够多少字节。
2. TMA 引擎把这么多字节搬齐之后, 对应的 `mbarrier.try_wait(phase)` 才放行。
3. **非得到这一步**,SMEM tile 才能交给 MMA(矩阵乘加 / matrix multiply-accumulate, 也就是张量核心干的那摊活) 去算。

下面用 Mermaid 把原图重画一遍 (TMA Async Load 的这套握手):

时序图 (按时间顺序)

1. 被选中线程 (`tid==0`) → TMA 引擎:`copy_async(A...), copy_async(B...)` 发起搬运
2. 被选中线程 (`tid==0`) → mbarrier (`tma_bar`):`arrive.expect_tx(byte_count)` 声明预期字节数 (只能一次)
3. TMA 引擎 → Asmem/Bsmem: 自主把 tile 搬入 SMEM
4. TMA 引擎 → mbarrier (`tma_bar`): 搬够 `byte_count` 后通知 mbarrier
5. 备注:`try_wait(phase)` 在此处阻塞
6. mbarrier (`tma_bar`) → 张量核心 (MMA): 字节凑齐, 释放 (同时携带 SMEM 可见性)
7. 张量核心 (MMA) → Asmem/Bsmem: 安全读取 SMEM, 开始计算

注意

注意:mbarrier 的释放本身就携带了 SMEM 的可见性, 所以释放之后不需要再补一道 fence。这点在完整 kernel 的注释里也强调了。

Store 侧: 走同样硬件, 但用另一套等待协议

这里有两套同步协议, 脑子里千万别混——load 走一套,store 走另一套, 等的方式完全不一样:

	Load 侧	Store 侧
跟踪方式	mbarrier + 字节数	commit group + wait group

发起	一个线程 <code>copy_async(..., dispatch="tma")</code>	一个线程 <code>copy_async(D[...], Dsmem, dispatch="tma")</code>
等待	<code>arrive.expect_tx</code> $\rightarrow$ <code>try_wait(phase)</code>	<code>cp_async.bulk.commit_group()</code> $\rightarrow$ <code>cp_async.bulk.wait_group(0)</code>
保护对象	MMA 读 SMEM 前数据已到	旧 store 排空前 Dsmem 不可复用

store 这边的流程是这样: 线程先把 fp16 的结果写进 Dsmem, 同步一下; 然后挑一个线程启动 `Tx.copy_async(D[...], Dsmem, dispatch="tma")`; 最后用 `commit_group() + wait_group(0)` 卡在那儿, 等到 store 全部排空为止。

最后这一等, 可不能省。道理很简单: 上一笔 store 还没走完,Dsmem 就还占着呢, 你没法拿它去装下一块 tile。

Try with your agent(原书练习): 为一个 K tile 追踪 Step 4 的 load/store 同步——哪个线程启动每条 TMA 命令、哪个 mbarrier 或 commit group 跟踪完成、哪个等待保护 MMA 读 Asmem/Bsmem、哪个等待保护 Dsmem 的复用? 为什么这里 `elect_sync()` 是错误的线程选择?

完整 kernel 的结构与关键点

完整的 kernel, 就是把 TMA load/store 塞进 Step 3 的骨架里, 别的地方一概不碰。下面不照抄那一长串源码, 只挑出真正扛着 TMA 语义的那几行。

K 循环的主体长这样。这里要特别留意: 它每一轮都要等, 所以还谈不上真正的重叠:

```
for k in range(K_TILES):
    k_st = T.meta_var(k * BLK_K)
    if tid == 0:
        tma_load(k_st)
        T.ptx.mbarrier.try_wait(tma_bar.ptr_to([0]), phase_tma)
    if tid == 0:
        mma(accum=k != 0)
        T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)
    phase_tma ^= 1
    phase_mma ^= 1
```

关键在那两个 `@T.inline` 辅助函数上。tma_load 里发两条 `copy_async`, 顺手把字节数报出去:

```
@T.inline
def tma_load(k_st):
    tma_config = T.meta_var({"dispatch": "tma", "cta_group": 1,
                             "mbar": tma_bar.ptr_to([0])})
    Tx.copy_async(Asmem[:, :], A[m_st:m_st+BLK_M, k_st:k_st+BLK_K], **tma_config)
    Tx.copy_async(Bsmem[:, :], B[n_st:n_st+BLK_N, k_st:k_st+BLK_K], **tma_config)
    T.ptx.mbarrier.arrive.expect_tx(
        tma_bar.ptr_to([0]),
        (BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE) # A B fp16
```

整段 kernel 里, 真正扛着 TMA 语义的就五个配置点。把这五个记住, 这一步也就拿下了:

1. **TMA config**:{"dispatch": "tma", "cta_group": 1, "mbar": tma_bar.ptr_to([0])}——它告诉 Tx.copy_async 走 TMA, 并且用 tma_bar 来汇报 load 完成。
2. **字节数**: $(BLK_M * BLK_K + BLK_N * BLK_K) * 2$, 这是 A、B 两块 fp16 操作数 (operand, 参与运算的输入数据)tile 加起来的字节数, 喂给 arrive.expect_tx(...).
3. **mbarrier 初始化**:init(tma_bar.ptr_to([0]), 1), 给 TMA load 立起那道完成屏障。
4. **@T.inline:tma_load(...)** 和 **mma(...)** 是辅助函数, 编译期直接展开进 kernel 主体, 所以能随手用周围 kernel 里的变量。
5. **TMA store 同步**:epilogue(收尾阶段,K 循环算完后把结果写回的那段) 先把 fp16 行写入 Dsmem, 再用 fence.proxy_async 和 warpgroup_sync 把这些线程的写入对 TMA store 通路理顺; 接着 store 用 commit_group() / wait_group(0) 等 SMEM→GMEM 这趟传输跑完。

关键

关键:Step 4 的提速**不是靠重叠** (它每个 load 后都还在等)。提速纯粹来自**数据通路换了**:TMA 接管了批量传输, 把 CTA 线程从”用指令带宽一块块搬 tile”里解放出来。光这一项, 就足以让性能往前挪一截。一句话——把搬运从关键路径上拿掉, 这就是 Step 4 的全部价值。

所以眼下的局面是: **零件配对了, 节奏还没踩上**。load 和 MMA 还是一个接一个轮着来。下一步, 我们 TMA 通路一动不动, 光去重排时间表。

Step 5: 软件流水线 (PIPE_DEPTH=2)

为什么 Step 4 没法重叠? 瓶颈在”存储”

两个引擎明明各干各的、互不相干,Step 4 怎么就是重叠不起来? 卡点不在硬件, 在**地方不够放**。

你想象一下: 手头只有一对 SMEM tile。下一个 load 想动身, 可它**没地方落脚**。当前的 MMA 还没把这对 tile 读完, 它就不能开搬, 一搬就把人家正用着的数据给盖了。说白了, 就是缺第二块空地。

Step 5 的招就是**双缓冲 (double buffering)**: 多准备一块格子, 这个存储冲突自然就化解了。不过有句话得先说在前头: 这时候单 warpgroup 的主循环**还是先等当前 MMA、再发下一个 load**。区别只在于, 现在有了”两块分开的格子”, 可以预取、可以轮着复用了。

维度	Step 5 的状态
Scope	不变, 仍是一个 warpgroup
Layout	变了: 单对 SMEM tile 变成 PIPE_DEPTH 级的环形缓冲 (ring buffer)
Dispatch	不变, 仍是 TMA load + tcgen05 MMA(Blackwell,NVIDIA 较新的一代 GPU 架构, 这一代张量核心的 MMA 指令族, 累加器 (accumulator, 逐块累加部分乘积的那块结果存储) 落在 TMEM 上); 本步加了预取和阶段复用, 完整重叠仍留给 Step 7

流水线骨架长什么样

PIPE_DEPTH=2 说白了就是: 开两个 SMEM 阶段 (stage), 让 load 通路和 MMA 通路各有各的格子用。

下面这张图,画的是双缓冲撑起的那套流水线结构 (PIPE_DEPTH=2),它不是这个单 warp-group kernel 真实跑起来的轨迹。别会错意——这一步只是先把预取搭出来,真要跑满重叠,得等 Step 7。每个 stage 就是一块 SMEM 格子,K tile 轮着用它:

结构图

分组:

- **stage 0 (一块 SMEM 格子):** 「k0 → k2 → k4 → ...」
- **stage 1 (另一块 SMEM 格子):** 「k1 → k3 → k5 → ...」

连接关系:

- 启动: 两次 TMA load 先填满两个 stage (prefetch) → stage 1 (另一块 SMEM 格子)
- stage 0 (一块 SMEM 格子) $\xrightarrow{\text{等 stage}} \text{跑 MMA} \rightarrow \text{等 MMA 读完} \rightarrow \text{空出后 load } k + \text{PIPE_DEPTH}$
每轮循环: 轮转复用两个 stage
- stage 1 (另一块 SMEM 格子) → 每轮循环: 轮转复用两个 stage

注意

注意: 这还不是并发的 TMA/MMA 时间表; 它只是建立了环形缓冲结构,Step 7 会把这个结构拆成生产者/消费者两个角色,届时 TMA 和 MMA 才真正同时跑。

相对 Step 4 的四处改动

Step 5 的代码跟 Step 4 比,就差这四处:

1. Asmem 和 Bsmem 前头多挂了一个 PIPE_DEPTH 维度,这样每个 stage 都有自己那份 SMEM 存储。
2. tma_bar 从一个变成一个数组,一个 stage 配一个 mbarrier。
3. 主 K 循环开跑前,先把头两个 stage 预取好。
4. K 循环里用 `stage = k % PIPE_DEPTH` 轮转: 等当前 stage → 在它上面跑 MMA → 再把这个 stage 拿去 load `k + PIPE_DEPTH`。

流水线机制三件套

下面三段都是示意伪代码,只为把机制讲清楚 (为了读着清爽,if tid == 0 守卫和 try_wait 的完整写法都略掉了); 带守卫的真实版本,看后面”完整 kernel”那一节。

(1) 预取 (Prefetch): 主循环还没开跑,先把头 PIPE_DEPTH 个 stage 给 load 进来,这样循环第一轮一开始,数据就已经在那儿等着了:

```
for s in range(min(PIPE_DEPTH, K_TILES)):
    tma_load(s, s * BLK_K)
```

(2) 主循环: 每个 K tile,先等它的 stage 备好、在上面把 MMA 跑了,然后趁这个 stage 刚腾空,马上拿去 load PIPE_DEPTH 之外的那块:

```

stage = k % PIPE_DEPTH
wait(tma_bar[stage], phase_tma) #           stage    load
mma(stage, accum)               #           stage
wait(mma_bar[0], phase_mma)     #    MMA           stage
phase_mma ^= 1
tma_load(stage, next_k * BLK_K) #           stage

```

(3) 相位管理 (Phase management): 这块最容易把人绊倒，但规则其实就一句话——一个 **barrier** 多久翻一次相位，全看它有几个槽位。槽位多少不一样，翻的节奏自然就不一样。我们这两个 **barrier** 一对照，正好说明问题：

- **mma_bar**(就一个 **barrier**): MMA 累加器只占一个 TMEM 槽，所以 **mma_bar** 也只有一个，每一轮都会被重新摸到。一个每轮都摸的 **barrier**，当然每轮都翻相位。

“python phase_mma ^= 1 # “

- **tma_bar**(PIPE_DEPTH 个元素的数组): 每个 stage 配一个 **barrier**。某个 stage 的那个 **barrier**，得把整圈环绕完才会再被摸到一次。所以 **phase_tma** 只在 **stage** 下标绕回 0 的那一下翻一次：

“python if stage == PIPE_DEPTH - 1: phase_tma ^= 1 # “

关键

关键: 把“相位翻转节奏 $\propto$ barrier 槽位数”这条规律记牢。**mma_bar** 每轮回访 $\rightarrow$ 每轮翻；**tma_bar[stage]** 每 PIPE_DEPTH 轮才回访同一个 $\rightarrow$ 绕圈才翻。这是 **mbarrier** 相位语义最直接的推论。

Try with your agent(原书练习): 取 PIPE_DEPTH=2、K_TILES=5，逐 **k** 追踪主循环——列出每轮的 **stage**、传给 **try_wait** 的 **phase_tma/phase_mma**、以及是否发出新的预取。**phase_tma** 究竟在哪里翻？为什么最后两轮没有预取？(提示：**next_k = k + PIPE_DEPTH < K_TILES** 不成立时就不再预取。)

完整 kernel: 关键差异

完整 kernel 把 Step 4 那条 TMA load/store 通路原封不动地留着，外面只是又裹了一层分阶段缓冲和相位逻辑。结构上的不同就集中在三处，下面各挑几行最能说明问题的。

双缓冲的 layout——第一维就是流水线 **stage**:

```

PIPE_DEPTH = 2
A_layout = tma_shared_layout(a_type, SwizzleMode.SWIZZLE_128B_ATOM,
                             (PIPE_DEPTH, BLK_M, BLK_K)) #           stage
B_layout = tma_shared_layout(b_type, SwizzleMode.SWIZZLE_128B_ATOM,
                             (PIPE_DEPTH, BLK_N, BLK_K))

```

barrier 初始化——MMA 一个、TMA 每 stage 一个:

```

if warp_id == 0 and lane_id == 0:
    T.ptx.mbarrier.init(mma_bar.ptr_to([0]), 1) #           MMA barrier

```

```
for s in range(PIPE_DEPTH):
    T.ptx.mbarrier.init(tma_bar.ptr_to([s]), 1)           #      stage      TMA barrier
```

主循环里多了”发下一笔预取”和”绕圈翻相位”两步:

```
for k in range(K_TILES):
    stage = k % PIPE_DEPTH
    T.ptx.mbarrier.try_wait(tma_bar.ptr_to([stage]), phase_tma) #      stage      load
    if tid == 0:
        mma(stage, accum=(k != 0))
    T.ptx.mbarrier.try_wait(mma_bar.ptr_to([0]), phase_mma)
    phase_mma ^= 1
    next_k = k + PIPE_DEPTH
    if next_k < K_TILES:
        #
        if tid == 0:
            tma_load(stage, next_k * BLK_K)
    if stage == PIPE_DEPTH - 1:
        #      phase_tma
        phase_tma ^= 1
```

epilogue(TMEM → 寄存器 → Dsmem → TMA → GMEM) 跟 Step 4 一模一样, 这里就不再展开了。

Step 6: 持久化 kernel + tile 调度器

从”优化单 tile 内”到”优化跨 tile”

前面那些功夫, 都花在优化单个 tile 内部的活儿上。Step 6 把镜头往后一拉, 换了个尺度看——跨 tile 优化。

先回顾 Step 5 怎么干的: 每个 128×128 输出 tile, 起一个 CTA。对 4096×4096 的输出, 那就是 1024 个互不相干的 CTA。每个 CTA 都得自己掏一遍启动成本, 干完自己那块就直接散伙、灰飞烟灭。

Step 6 换个活法: 只起一个固定大小的 CTA 池, 让每个 CTA 轮着去啃好几个 tile。这么一来, 能换来两样好处:

- 1. 摊薄启动开销: TMEM 分配、barrier 初始化、调度器状态这些事, 原来要在 1024 个一次性 CTA 上重复 1024 遍, 现在每个 CTA 只做一次, 这一次的成本摊到它经手的大约 7 个 tile 上。
- 2. 更聪明的 tile 顺序: tile 怎么分配挪进了 kernel 内部, 调度器就能挑一个让操作数能复用的顺序来跑。

维度	Step 6 的状态
Scope	变了: 固定一池持久化 CTA, 每个通过调度器循环处理多个输出 tile
Layout	不变, 同样的 per-tile SMEM/TMEM/寄存器通路
Dispatch	不变

持久化调度: grid 大小贴合硬件, 而非问题

持久化 kernel 的核心想法, 一句话: grid 多大, 看硬件, 不看问题。

换句话说, 它就起 SM_COUNT 个 CTA(大致一个 SM 一个), 管你输出 tile 总共多少个。图的就是件事: 让每个 SM 手里一直有话, 别让它闲着。

注意

注意: 这里说”大致”是有讲究的——精确的 1:1 驻留**没人保证**, 具体得看占用率 (occupancy), 还得看硬件怎么调度 CTA。

这一章的目标硬件是 B200,SM_COUNT=148。于是就开 148 个 CTA, 各自循环去处理 ClusterPersistentScheduler 派给它的那些 tile。

第二个好处, 来自调度器挑的那个**处理顺序**。这里把 l2_group_size 设成 8, 意思是把相邻的 tile 凑成一组、一个挨一个地处理。下表是输出 tile 的网格布局 (行 = M 方向, 列 = N 方向), 每格就是一个 128×128 的输出 tile:

M 方向\ N 方向	N=0	N=1	N=2	N=3
M=0	T0	T1	T2	T3
M=1	T4	T5	T6	T7

表里的复用关系很清楚: 同一行带 (同一 M 行) 上的那几个 tile, 用的是同一份 A 行块; 同一列带 (同一 N 列) 上的, 用的是同一份 B 块。

道理就在这儿: 既然同一行带里的 tile 共用一份 A、同一列带里的共用一份 B, 那把这些沾亲带故的 tile 一个挨一个地处理, 操作数就能**一直赖在 L2 里**, 不用一趟趟跑回 HBM 重新拉。这份复用红利, Step 3 当时其实就摆在桌面上了, 只是没顺手吃掉。

接进调度器, 几行就够:

```
bx = T.cta_id([SM_COUNT]) # 1D grid, CTA SM( 2D grid!)
```

```
tile_scheduler = ClusterPersistentScheduler2D(  
    "ts",  
    num_m_tiles=M // BLK_M,  
    num_n_tiles=N // BLK_N,  
    l2_group_size=8, # 8 tile  
    num_clusters=SM_COUNT)  
tile_scheduler.init(bx)
```

一个容易漏掉的正确性后果: 相位初始化要进循环

一个 CTA 要循环啃好几个 tile, 这就牵出一个**特别容易漏的坑**: 每个 tile 都得跑一遍自己**全新的 K 循环**, 而它的 barrier 相位, 必须**从一个干净的、已知的状态起步**。

- Step 5 里, 一个 CTA 就管一个 tile, 所以 phase_tma、phase_mma 开头初始化一次就完事, 没毛病。
- 到了 Step 6 就不行了。这俩初始化得**挪进 while tile_scheduler.valid()** 循环里头, 让每个 tile 都从一个跟它自己 TMA/MMA 节奏对得上的相位重新起步, 而不是把上一个 tile 留下的残值给接了过来。

```
while tile_scheduler.valid():  
    phase_tma: T.int32 = 0 # tile 0  
    phase_mma: T.int32 = 0  
    ...  
    tile_scheduler.next_tile() # tile
```

完整 kernel: 结构上就是 Step 5 套了个外层 tile 循环

从结构上讲, Step 6 不过是把 Step 5 那整条流水线套进一个 tile 级的外层循环罢了。新加的依赖就一个——调度器本身:

```
from tvn.tirx.lang.tile_scheduler import ClusterPersistentScheduler2D
```

骨架长这样 (内层那个 K 循环跟 Step 5 一字不差):

```
SM_COUNT = 148 # NVIDIA B200 SM

# ... Step 5 SMEM/barrier/TMEM ( CTA )...

while tile_scheduler.valid():
    m_st = T.meta_var(tile_scheduler.m_idx * BLK_M) # tile
    n_st = T.meta_var(tile_scheduler.n_idx * BLK_N)
    phase_tma: T.int32 = 0
    phase_mma: T.int32 = 0
    # === : Step 5 ===
    # prefetch PIPE_DEPTH stage
    # K :wait stage -> mma -> wait mma -> k+PIPE_DEPTH ->
    # epilogue:TMEM -> RF -> Dsmem -> TMA -> GMEM
    T.cuda.cta_sync()
    tile_scheduler.next_tile() # tile

# TMEM tile ( CTA )
```

关键

关键: 注意 TMEM 的 alloc/dealloc 在整个 while 循环之外——它们随 CTA 生命周期一次性分配和释放, 不在每个 tile 上重做。这正是”摊薄启动开销”的具体体现。

练习 (原书 Exercises)

- 1. Step 4: arrive.expect_tx 用的字节数是 $(BLK_M * BLK_K + BLK_N * BLK_K) * 2$ 。如果这个字节数偏小或偏大, mbarrier 会等什么?(思路: 偏小则字节没凑齐它就提前释放, MMA 读到不完整数据; 偏大则永远凑不齐, wait 死等不放。)
- 2. Step 5: 为什么每个 SMEM stage 需要自己的 TMA barrier, 而不能两个 stage 共用一个 tma_bar?(思路: 两个 stage 的 load 在途时间不同, 共用会让”哪个 stage 好了”分不清, 相位语义也会乱。)
- 3. Step 6: 4096×4096 输出、BLK_M=BLK_N=128, 有多少个输出 tile? SM_COUNT=148 时每个持久化 CTA 平均处理几个?(算一下: 每边 $4096/128 = 32$ 个, 共 $32 \times 32 = 1024$ 个 tile; $1024/148 \approx 6.9$, 即平均约 7 个。)

小结

这一章的主线就一句话: 让一个硬件引擎跑在另一个前头。下面三步, 就是把这句话一点点落到地上的过程:

步骤	改的维度	核心动作	是否真重叠
Step 4	Dispatch	同步拷贝 → TMA 命令;mbarrier+ 字节数跟踪 load,commit/wait group 跟踪 store	否 (每 load 后仍等); 提速来自把搬运移出关键路径
Step 5	Layout	SMEM 双缓冲 (PIPE_DEPTH=2 环形缓冲)+ 预取; 相位翻转节奏 $\propto$ barrier 槽位数	否 (单 warpgroup 仍等); 搭好流水线骨架
Step 6	Scope	持久化 kernel(一 SM 一 CTA)+ tile 调度器;l2_group_size 控操作数 L2 复用; 相位初始化进循环	否; 摊薄启动开销 + 跨 tile 操作数复用

几条最该揣走的认知:

- **TMA 的价值有两层**:Step 4 先吃到的是”把搬运从线程的指令带宽里摘出去”;至于真正的 load-/compute 重叠, 那是后话 (Step 7 的 warp 专门化)。
- **两套同步协议别串了**:load 用 mbarrier + expect_tx 报字节数;store 用 commit group + wait group。拿 cta_sync() 去等 TMA load, 是个不声不响的错。
- **挑线程用 tid == 0, 别用 elect_sync()**: 后者每个 warp 挑一个,4 个线程会把字节数重复报四遍, 直接把 mbarrier 的计数搞坏。
- **相位翻转记个口诀**:barrier 每轮都回访, 就每轮翻 (mma_bar); 每绕一圈才回访一次, 就绕圈翻 (tma_bar[stage])。
- **持久化换来两份红利**: 摊薄每个 tile 的启动成本, 外加用 tile 顺序换 L2 复用; 代价是相位初始化和 prefetch 都得搬进 tile 外层循环里头。

下一章 (从 Step 7 起) 会上 warp 专门化, 把 TMA 和 MMA 拆成生产者、消费者两个角色。到那会儿, 这一章搭好的这套环形缓冲, 才真正两条线一起跑满。

延伸阅读

- 原文:[Pipelining GEMM with TMA —Modern GPU Programming for ML](#) Sys
- 本章末尾提到的端到端计时, 见下一章 *Scaling GEMM with Warp Specialization and Clusters* 的 *End-to-End Result* 表。

术语对照

中文	English	说明
张量核心	Tensor Core	芯片上最贵、最该跑满的计算单元
共享内存	SMEM (shared memory)	tile 在片上的落脚点
线程束	warp	32 个 lane 一组
warpgroup	warpgroup	4 个 warp 一组, 共 128 线程
张量内存加速器	TMA (Tensor Memory Accelerator)	专门搬 GMEM↔SMEM tile 的硬件引擎
内存屏障	mbarrier (memory barrier)	用字节数/到达数跟踪异步完成
相位	phase	mbarrier 的翻转位, 区分轮次

双缓冲	double buffering	PIPE_DEPTH=2 的两阶段环形缓冲
环形缓冲	ring buffer	多 stage 轮转复用的 SMEM 结构
预取	prefetch	主循环前先填满头几个 stage
持久化 kernel	persistent kernel	grid 按硬件 (SM 数) 定,CTA 循环处理多 tile
tile 调度器	tile scheduler	在 kernel 内分配 tile 并挑复用友好的顺序
warp 专门化	warp specialization	把 TMA/MMA 拆成生产者/消费者 (Step 7, 下一章)
提交组 / 等待组	commit group / wait group	TMA store 的完成跟踪机制
张量内存	TMEM (tensor memory)	tcgen05 MMA 累加器所在的片上内存

第 13 章 · 用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)

原文:用 Warp 专门化与 Cluster 扩展 GEMM(Step 7-9)

本章要点

本章要点 (TL;DR)

- 上一章那个流水线 GEMM, 还是把「加载 / 计算 / 写回」三件事全压在一个 warpgroup 上, 三个硬件引擎 (TMA、Tensor Core、写回路径) 只能轮流空转。本章用 Step 7-9 三步, 一级一级把「谁跟谁协作」的范围撑大, 一路逼近业界最高的吞吐。
- **Step 7(Warp 专门化 + 流水线)**: 把 warp 拆成 TMA 生产者 / MMA 消费者 / 写回三个并发角色, 靠 4 个方向相反的 barrier(tma2mma、mma2tma、mma2ld、ld2mma) 和 PipelineState 串起来; 生产者和消费者的初始 phase 必须反着, 否则不是死锁就是静默的数据损坏。
- **Step 8(2-CTA Cluster)**: 把两个 CTA 凑成 cluster, 靠 DSMEM 跨 CTA 共享操作数, 用一条 cta_group=2 的协同 MMA 算出 256×256 的 tile, 把**算术强度**大约翻一倍——这是真收益, 不是把活儿重分了一遍。
- **Step 9(多消费者 Warp 专门化)**: 再添一个 MMA 消费者, 拿「不同的 $A \times$ 同一块 B」把每个 CTA 的计算密度翻倍, cluster 输出长到 512×256 , 是本教程里最稠密、最优的那个 GEMM 变体。
- 三步只改「谁协作」, 不动「数据怎么摆」: SMEM / TMEM / 寄存器布局都沿用前两章的契约。端到端从朴素的 70 ms 一路压到 0.094 ms(约 744 $\times$), 跟 cuBLAS 打平; 收益越往后越小, 是逼近计算受限之后的边际递减。

前置知识

前置知识: 读这一章前, 最好先懂 warp 专门化、cluster(集群)、TMEM(Tensor Core 专用累加器内存)、mbarrier/phase(屏障与相位) 这几样, 它们是本章三步优化的地基。没把握的话, 先翻一下第 0 章 · 极筒入门, 以及本章直接接力的第 11-12 章 (TMA 流水线、tcgen05 MMA、TMEM 累加器的契约都来自那里)。本章会默认你已经认识这些词。

引言: 用 Warp 专门化与 Cluster 把 GEMM 推到极致 (Scaling GEMM with Warp Specialization and Clusters)

上一章遗留的瓶颈: 一个 warpgroup 包揽所有事

上一章我们用 TMA(张量内存加速器 / Tensor Memory Accelerator) 把 GEMM(矩阵乘法) 做成了流水线, 跑得已经挺快了。可它骨子里有个毛病。

毛病出在哪？整条流水线就只有一个 **warpgroup**(4 个 warp、128 个线程凑成的一支队伍；一个 warp / 线程束 = 32 个线程的小班，见第 0 章)，却要它一个人按顺序干完三件压根不是一回事的活——

- 1. **发起搬运 (load)**: 用 TMA 把下一块输入 tile(大矩阵切出来的小方块) 从全局内存搬进共享内存 (SMEM)。
- 2. **跑矩阵乘 (MMA)**: 让 Tensor Core(专做矩阵乘累加的硬件单元，见第 0 章) 在这块数据上做矩阵乘累加。
- 3. **写回 (writeback)**: 把累加结果从 TMEM(Tensor Memory,Tensor Core 专用的累加器内存) 排空，写回内存。

就算用软件流水线把这三步在时间上叠了一点，根子上的问题还在: **三种硬件引擎 (TMA 单元、Tensor Core、写回路径) 全堵在同一队线程身上**。这队线程就成了三个引擎的独木桥，谁想干活都得排队过，自然谁也快不起来。

症状: 引擎轮流空转

这瓶颈长啥样？把三个引擎的忙闲按时间步摆开，一眼就看明白了 (行是引擎，列是时间步，表示在干活，空白表示闲着):

引擎	t0	t1	t2	t3	t4	t5
TMA 单元	忙			忙		
Tensor Core		忙			忙	
写回路径			忙			忙

一队线程串着驱动三个引擎，任何时刻总有引擎在干等别人——三个永远凑不到同一拍上一起忙。

- Tensor Core 在算的时候,TMA 单元闲着;
- 结果往内存里排空的时候, 轮到 Tensor Core 闲着;
- 每个引擎想动一下, 都得隔着**同一组线程**去等别的引擎腾位置。

那怎么破? 思路就一句话: **别再让一队线程包圆了所有活**。让不同的 warp 各管一摊, 引擎之间才能真正同时转起来。

三步走: 把「协作范围」一步步做大

这一章用三步，一点一点把「谁跟谁协作」的范围撑大。每一步都拆掉一个串行瓶颈，一路逼近业界最高的吞吐。

步骤	名称	做了什么	协作范围	输出 tile
Step 7	Warp 专门化 + 流水线	把 warp 划分成生产者 / 消费者 / 写回三种角色	单个 CTA 内部	128×128
Step 8	2-CTA Cluster	把两个 CTA 组成一个 cluster, 跨彼此的 SMEM 共享操作数	跨 2 个 CTA	256×256

Step 9	多消费者 Warp 专门化	再加一个 MMA 消费者，让一份暂存 tile 喂两倍的算力	跨 2 个 CTA，双消费者	512×256
--------	---------------	--------------------------------	----------------	---------

注意

注意：这里的 CTA(Cooperative Thread Array, 即一个 thread block) 是一个线程块;cluster 则是 Hopper/Blackwell 架构引入的、把多个 CTA 绑成一组、允许它们互相访问对方 SMEM 的更上层结构。

把三步看成同一个套路的不同尺度

要看懂这三步，最好的角度是：它们其实是**同一个套路，被一次次放大**而已。

结构图

分组：

- **Step 7: 单个 CTA (输出 128×128)：**「warpgroup A: TMA 搬 + MMA 算」「warpgroup B: writeback 写回」
- **Step 8: 2-CTA Cluster (一个 256×256 tile 横跨两者，沿 cbx 轴展开)：**「CTA 0: SMEM 半块」「CTA 1: SMEM 半块」
- **Step 9: 多消费者 (cluster 输出 512×256，最稠密变体)：**「CTA 0: 消费者 1 + 消费者 2」「CTA 1: 消费者 1 + 消费者 2」

连接关系：

- CTA 0: SMEM 半块 $\xrightarrow{\text{操作数 tile 横跨两个 CTA 的 SMEM}}$ CTA 1: SMEM 半块
- CTA 0: 消费者 1 + 消费者 2 $\xrightarrow{\text{一份暂存的 B tile 被两个消费者复用}}$ CTA 1: 消费者 1 + 消费者 2
- Step 7: 单个 CTA (输出 128×128) $\xrightarrow{\text{协作范围跨出单个 CTA}}$ Step 8: 2-CTA Cluster (一个 256×256 tile 横跨两者，沿 cbx 轴展开)
- Step 8: 2-CTA Cluster (一个 256×256 tile 横跨两者，沿 cbx 轴展开) $\xrightarrow{\text{计算密度再翻倍}}$ Step 9: 多消费者 (cluster 输出 512×256，最稠密变体)

- **Step 7:** 整条流水线还窝在一个 CTA 里。TMA 和 MMA 合用一个 warpgroup，写回甩给另一个 warpgroup。这是角色分工里最小、最朴素的样子。
- **Step 8:** 协作范围头一回跨出单个 CTA。两个 CTA 搭伙产出一个 256×256 的 tile，这块 tile 横跨它俩的 SMEM。
- **Step 9:** 再往死里榨一把算力密度。cluster 的输出长到 512×256，每一块暂存的 B tile 都被两个消费者轮着用，于是我们就拿到了教程里计算最稠密的那个版本。

贯穿全章的不变量：变的是谁协作，不是数据怎么摆

整章有一样东西**自始至终没动**:SMEM、TMEM 和寄存器的布局，还是老老实实守着前两章定下的那些契约。说白了，这一章改的是**谁跟谁协作**，而不是**数据怎么摆**。

注意

注意:Step 8 是协作范围头一次伸到单个 CTA 之外。也正因为这样, 它的操作数 (operand, 即喂给矩阵乘的输入矩阵 A、B)tile 被劈成两半, 分别放进两个 CTA 的共享内存, 布局沿着 cluster 的 cbx 轴 (也就是 cluster 在 x 方向上给 CTA 的编号) 横跨两个 CTA。读 Step 8 的代码时, 脑子里一定得绷着这根弦: 同一个逻辑 tile, 物理上是散在两个 CTA 的 SMEM 里的。

第 7 步:Warp 专门化 + 流水线 (Step 7: Warp Specialization + Pipeline)

到目前为止, 我们的 GEMM 内核都是「单 warpgroup」结构: 同一批线程顺着「加载 → 计算 → 写回」走一遍。这写法有处要命的浪费——线程在搬数据 (TMA) 的时候, Tensor Core 干瞪眼; 线程在跑 MMA 的时候, TMA 引擎又闲着。两个金贵的硬件引擎, 愣是凑不到一块儿同时忙。

这一步的核心就一个词:**warp 专门化 / warp specialization**。它要治的病是这样的——与其让一支队伍轮着把所有活都干了, 不如一件活配一支**专门的 warp**, 让这些 warp **一起跑**, 再拿一套软件流水线把它们串起来。这是整条 GEMM 路上动静最大的一次架构改造, 后面第 8、9 步全是踩着它的肩膀往上走的。本节基准统一用 $M=N=K=4096$ 。

本步改了什么、没改什么

维度	第 6 步 (及之前)	第 7 步 (本步)
作用范围 (Scope)	一个 warpgroup 顺序走 load → MMA → writeback	拆成 3 个并发角色 :TMA 生产者 / MMA 消费者 / 写回, 用 full/empty barrier 衔接
布局 (Layout)	多级 SMEM stage + TMEM 累加器	不变 (沿用第 6 步)
指令派发 (Dispatch)	TMA 加载、tcgen05 MMA	不变

说白了, 多级 SMEM 流水线和持久化调度器 ClusterPersistentScheduler2D 都是从第 5-6 步原样搬过来的, **真正新增的只有一件事: 把活按角色拆开**。这一节会一项一项讲清楚:

- warp 专门化: 把不同的活分给不同的 warp / warpgroup
- 高层 barrier 抽象:TMABar、TCGen05Bar、MBarrier
- PipelineState: 自动管 stage(槽位) 和 phase(相位)
- warpgroup_sync 的 barrier ID: 让同步能精确到「一个 warpgroup」这个粒度

从顺序到并发: 为什么要拆

讲角色和 barrier 之前, 先把 warp 专门化到底要干掉哪个瓶颈看明白。下面拿时间步表格, 把「专门化前」(第 4-6 步那一路) 和「专门化后」(第 7 步) 的引擎利用率摆一块儿比 (行是引擎, 列是时间步, 格子是那一刻在干啥)。

专门化前 (单 warpgroup 轮着干)——搬数据时 Tensor Core 闲, 算的时候 TMA 闲, 两个引擎永远错着峰:

引擎	t0	t1	t2	t3	t4
----	----	----	----	----	----

TMA 引擎	load k=0		load k=1		load k=2
Tensor Core		MMA k=0		MMA k=1	

专门化后 (生产者/消费者并发)——TMA 一边预取下一片,MMA 一边在算当前片, 两个引擎同时满载:

引擎	t0	t1	t2	t3	t4
TMA 生产者 (warp 3)	load k=0	load k=1	load k=2	load k=3	...
MMA 消费者 (warp 0)		MMA k=0	MMA k=1	MMA k=2	...
写回 (WG 0)				writeback	...

第 5、6 步靠双缓冲和持久化调度把基线往上抬了抬, 但它们始终没把「加载」和「计算」拆成生产者、消费者两个独立角色。第 7 步的专门化打破了「轮流」这个魔咒:TMA 生产者 (warp 3) 趁着 MMA 消费者 (warp 0) 还在算当前片, 就抢先把下一片的加载发出去了, 于是两个引擎谁也不用干等谁。

那 load 和 MMA 之间怎么交接班? 靠两个 barrier:

- **tma2mma**(TMA → MMA): 跟 MMA 说「你等的那片 SMEM 数据到位了, 开算吧」。
- **mma2tma**(MMA → TMA): 跟 TMA 说「这块缓冲我读完了, 你拿去装下一片」。

注意

注意: 时间线里有个细节乍一看像 bug——**mma2tma** 的释放箭头会「跨一个 stage」。别慌, 这是**环形缓冲 (ring buffer)** 的正常现象。**PIPE_DEPTH=2** 的时候只有两块 SMEM 缓冲 (stage 0、stage 1):TMA 把 k=0 装满 buffer 0,k=1 装满 buffer 1。等 MMA 算完 k=0、读完 buffer 0, 它就发个 **mma2tma**, 意思是「buffer 0 空了」。可这会儿真正惦记着 buffer 0 的是谁? 是 **k=2**——因为 buffer 1 还被 k=1 占着呢。所以这个释放箭头会一路指到 k=2。说白了, 跨一个 stage 纯粹是因为环上就两个槽, 转一圈正好回到自己。

谁来干每件活:warp 角色分工

时间线讲清了为什么要拆; 接下来说谁来干。当 **WG_NUMBER=2**(内核用两个 warpgroup, 下表简写 WG) 时, 三件活这么分:

角色	位置	职责
TMA 生产者	WG 1,warp 3	持续用 TMA 加载 A、B 的 tile
MMA 消费者	WG 1,warp 0	数据一就绪立刻跑 MMA
写回 (Writeback)	WG 0(全部 4 个 warp)	读 TMEM 结果, 写回 GMEM

4 个 barrier: 前向报「就绪」, 后向报「释放」

三个并发角色, 要配 4 个 barrier。这 4 个怎么记? 它们正好分成方向相反的两组:

- **前向** (TMA → MMA → 写回) 传的是「数据**就绪**」: 意思是「你等的那片到了」。
- **后向** (写回 → MMA → TMA) 传的是「缓冲**释放**」: 意思是「你惦记的那个槽又腾空了」。

命名规则是 2 (source2destination), 记住这条, 名字自己就能读懂——tma2mma 就是「TMA 给 MMA 发信号」那个 barrier。

Barrier	类型	方向	含义
tma2mma	TMABar	TMA → MMA	「SMEM 数据已就绪」
mma2tma	TCGen05Bar	MMA → TMA	「SMEM 缓冲可复用」
mma2ld	TCGen05Bar	MMA → 写回	「TMEM 结果已就绪」
ld2mma	MBarrier	写回 → MMA	「TMEM 已空, 可写下一片」

为什么每个 barrier 的类型不一样? 因为类型取决于「生产者拿什么方式告诉别人自己干完了」:

- **TMA 加载** 用 TMABar, 它本质是一个带字节计数 / **byte counting** 的 mbarrier:TMA 硬件把该传的字节全搬到位以后, **硬件自己**就在 barrier 上 arrive 了。消费者一行轮询都不用写, 就知道数据到了。
- **TMA 存储 (store)** 使不了这招——store 是把数据往外写, 根本没有「该通知谁」这个对象。所以它只能退回到 `cp_async.bulk.commit_group() + wait_group(0)`: 谁发起的存储, 谁自己等自己的写入排空。
- **MMA** 用 TCGen05Bar,MMA 一算完, 由 `tcgen05.commit()` 这条指令在 barrier 上发信号。

注意

注意: 这里埋了个第 8 步的伏笔——现在所有 arrive 都传 `cta_mask=0`, 因为单 CTA 内核里没有别的 CTA 要通知。等第 8 步拼成 cluster, 正是这个参数会变成非零, 摇身一变成了「叫醒协作 CTA」的开关。

PipelineState: 自动管理 stage 与 phase

4 个 barrier 负责告诉各角色什么时候某块缓冲就绪了。但还差一样: 得有人记着眼下轮到哪块缓冲。这份记账活就交给 PipelineState。环形缓冲要同时盯两件事:

1. **stage**: 现在在第几个槽位;
2. **phase**: 现在在等这个槽位 barrier 的哪个「相位」(0 还是 1)。

这两个量你要是手动一块儿维护, 那正是流水循环里最容易出 off-by-one(差一) 的地方——偏偏这里只要差一个, 整个内核就**死锁**。PipelineState 的活, 就是把这两量绑一块儿自动往前推, 省得你操这份心:

```
tma_ps = PipelineState(PIPE_DEPTH, phase=1) #           :           phase=1,
# tma_ps.stage =           stage
# tma_ps.phase =           phase(0    1)
tma_ps.advance()                               #           stage(           phase)
```

初始 phase 设成几, 决定了某个角色「第一次 wait」时是直接放行还是当场卡住。而流水线两头的正确答案**正好相反**——这就是最容易翻车的地方:

- **phase=1(生产者)**: 第一次 `wait(phase=1)`,barrier 这会儿还停在 phase 0,0 != 1, 所以**立即放行**。这正合我们的意——缓冲一开始就是空的, 生产者本来就该马上去填。

- **phase=0(消费者)**: 第一次 `wait(phase=0), barrier` 也是 `phase 0, 0 == 0`, 所以**卡住**。这也正合意——还没数据呢, 消费者没东西可读, 就该老老实实等生产者先 `arrive`。

注意

注意: 两端要是设了同样的起始 `phase`, 等着你的就是死锁, 或者更糟——**静默的数据损坏** (算出错的结果你还看不出来)。这一个选择, 必须做对。

warpgroup_sync 的 barrier ID: 按 warpgroup 同步

专门化带来一个特别容易踩的同步坑: 一旦各个 `warpgroup` 走的代码路径不一样, 我们熟得不能再熟的 `cta_sync()` 就会**死锁**。为啥? 因为 `cta_sync()` 用的是硬件 `barrier #0`, 而且死死要求 **CTA 里所有线程**都到齐才放行; 可在某个 `warpgroup` 的分支里, 只有一部分线程会走到这, 人数永远凑不齐。

我们要的是一个「只管一个 `warpgroup`」的 `barrier`。GPU 一共给了 16 个命名 `barrier`(ID 0-15), 所以内核改用 `warpgroup_sync(10)`——它只同步一个 `warpgroup` 内部的线程。要是多个 `warpgroup` 都得各自独立同步 (比如第 9 步的多消费者), 它们就用 `warpgroup_sync(wg_id + 10)` 各拿一个不同的 ID, 省得撞到同一个硬件 `barrier` 上去。

完整内核实现

这一步用 `PIPE_DEPTH=2`——这是「还能让 `load` 和 `compute` 叠起来」的最小深度 (更深能多藏点内存延迟, 代价是更吃 `SMEM`; 这个取舍后面「当第 7 步出问题」一节细聊)。把前面这些零件——角色、4 个 `barrier`、`PipelineState`、`warpgroup` 级同步——拼到一块儿, 核心结构长这样 (完整源码挺长, 这里只留关键骨架 + 中文讲解):

1) 分配与初始化——4 个 `barrier`、`SMEM` 缓冲、`TMEM` 累加器都在这儿建好:

```
BLK_M, BLK_N, BLK_K = 128, 128, 64
PIPE_DEPTH = 2
WG_NUMBER = 2

# 4 barrier:
tma2mma = TMABar(pool, PIPE_DEPTH) # TMA->MMA, mbarrier
mma2tma = TCGenO5Bar(pool, PIPE_DEPTH) # MMA->TMA, tcgenO5.commit
mma2ld = TCGenO5Bar(pool, 1) # MMA->
ld2mma = MBarrier(pool, 1) # ->MMA, mbarrier
# ...
ld2mma.init(128) # WG 0 128 arrive, 128
```

2) `TMEM` 分配 + `fence` + `cta_sync`——这一段所有线程还走在同一条路上, 所以可以放心用 `cta_sync()`; 角色的分叉是在它之后才发生的:

```
if wg_id == 0 and warp_id == 0:
    T.ptx.tcggenO5.alloc(T.address_of(tmем_addr), n_cols=512, cta_group=1)
T.ptx.fence.proxy_async("shared::cta")
T.ptx.fence.mbarrier_init()
T.cuda.cta_sync() # CTA
```

3)WG 1 = TMA 生产者 (warp 3)+ MMA 消费者 (warp 0)——这是整条流水线的心脏。盯一眼: 生产者和消费者的 PipelineState 起始 phase 是反着的:

```

if wg_id == 1:
    if warp_id == 3:          # === TMA      ===
        tma_ps = PipelineState(PIPE_DEPTH, phase=1) #      phase=1:    wait
        if T.filter(lane_id, T.ptx.select_sync()): #      TMA
            while tile_scheduler.valid():
                for k in range(K_TILES):
                    mma2tma.wait(tma_ps.stage, tma_ps.phase) #      MMA
                    tma_load(k * BLK_K) #      A B      TMA copy_async
                    tma2mma.arrive(tma_ps.stage, #      arrive(      )
                                   (BLK_M*BLK_K + BLK_N*BLK_K) * F16_SIZE)
                    tma_ps.advance()
                tile_scheduler.next_tile()

    elif warp_id == 0:        # === MMA      ===
        mma_ps = PipelineState(PIPE_DEPTH, phase=0) #      phase=0:    wait
        ld_ps = PipelineState(1, phase=1)
        if T.filter(lane_id, T.ptx.select_sync()):
            while tile_scheduler.valid():
                ld2mma.wait(ld_ps.stage, ld_ps.phase) #      TMEM
                ld_ps.advance()
                for k in range(K_TILES):
                    tma2mma.wait(mma_ps.stage, mma_ps.phase) #
                    Tx.gemm_async(tmем[:, :BLK_N],
                                   Asmem[mma_ps.stage], Bsmem[mma_ps.stage],
                                   accum=(k != 0), dispatch="tcgen05", cta_group=1)
                    mma2tma.arrive(mma_ps.stage, cta_group=1, cta_mask=0) #      TMA
                    mma_ps.advance()
                mma2ld.arrive(0, cta_group=1, cta_mask=0) #
                tile_scheduler.next_tile()

```

4)WG 0 = 写回——读 TMEM、转 fp16、再用 TMA 存回 GMEM(细节下一节展开):

```

elif wg_id == 0:
    wb_ps = PipelineState(1, phase=0)
    while tile_scheduler.valid():
        mma2ld.wait(wb_ps.stage, wb_ps.phase) #      MMA
        wb_ps.advance()
        Tx.wg.copy_async(reg_wg[:, :], tmem[:, :BLK_N]) #      TMEM ->      (warpgroup      )
        T.ptx.tcgen05.wait.ld()
        ld2mma.arrive(0, cta_id=0, pred=True) #      128      arrive:TMEM
        Tx.cast(reg_f16[:, :], reg[:, :]) #      fp32 -> fp16
        Tx.copy(Dsmem[warp_id*32 + lane_id, :], reg_f16[:, :])
        T.ptx.fence.proxy_async("shared::cta")
        T.cuda.warpgroup_sync(10) #      warpgroup      (      cta_sync!)
        if warp_id == 0 and lane_id == 0:
            Tx.copy_async(D[...], Dsmem[:, :], dispatch="tma") #      TMA store
            T.ptx.cp_async.bulk.commit_group()
            T.ptx.cp_async.bulk.wait_group(0) #      store      mbarrier
            T.cuda.warpgroup_sync(10)
        tile_scheduler.next_tile()

```


注意

注意: 想跑某一步的内核, 直接复用第 1 步那套 `compile / run / check` 脚手架, 把 `hgemm_v1` 换成 `hgemm_v7/v8/v9` 就行。clustered 的那几步要求 `M`、`N` 是它 cluster tile 的整数倍 (第 8 步是 256×256 , 第 9 步是 512×256), 所以 128×128 这种小尺寸一个 tile 都凑不出来。还有一点: 每个全新的 Python session 只编译一个步骤, 切换之前得重启内核——这些内核复用了内部命名, 编译器又攥着 per-session 的状态。

收尾阶段 (写回) 细节

第 7 步的 epilogue(收尾, 即把累加结果转格式、写回内存的最后一段) 能写得相当省事: 因为只有 `BLK_N=128` 列, 写回 warpgroup 一趟就能把整块 TMEM tile 读进寄存器 (register, 每个线程私有的最快存储, 数量有限, 见第 0 章), 然后发一条 TMA store 收工。第 8、9 步就没这福气了 (这也正是它俩后面非得搞「分块写回」的原因)。第 7 步的写回顺序是这样:

1. 等 `MMA::mma2ld.wait(phase)`。(第 8、9 步会在这儿加一个 `fence.after_thread_sync()` 当保守的双保险; 不过 MMA 完成的那个 `mbarrier` 本身已经把顺序兜住了, 大多数内核——包括 CUTLASS——都把它省了, 所以第 7 步也省。)
2. 读 TMEM → 寄存器: 每线程 128 个 fp32, warpgroup 范围, 用 `Tx.copy_async + T.ptx.tcgen05.wait.1`
3. 通知 `MMA::ld2mma.arrive(0, cta_id=0, pred=True)` (全 128 线程都 arrive), 意思是 TMEM 空出来了, 可以装下一片。这里两个 kwarg 在 clustered 步骤里会反复出现: `cta_id` 指明「给哪个 CTA 那份 barrier 副本发信号」(0 就是本 CTA 的本地 barrier; 第 8 步的协作 arrive 改用 `cta_mask` 去指 CTA-0); `pred` 是逐线程的谓词, 管这个线程到底算不算 arrive (这里是 `True`, 所以每个写回线程都计进 arrival 总数)。
4. `fp32 → fp16` (在寄存器里转好)。
5. 寄存器 → Dsmem, 再用 `fence.proxy_async("shared::cta") + warpgroup_sync(10)` 把写入刷出去。
6. TMA store: Dsmem → GMEM, 用 `cp_async.bulk.commit_group() + wait_group(0)`。

注意

注意: 第 8、9 步为啥不能也「一趟读完」? 卡在寄存器压力下。第 8 步 `BLK_N=256`、第 9 步每个消费者 `MMA_N=256`, 意味着每个线程得让 256 个 fp32 ($256 \times 4 = 1024$ 字节) 同时占着寄存器, 有溢出 (spill) 到本地内存的风险, 还会逼出更大的 Dsmem 缓冲。所以那两步把写回切成一块 `EPI_N` 列 (`EPI_N=64`) 一块: 每轮迭代只让 `EPI_N` 个 fp32 活在寄存器里, 配一条对应的小 TMA store——拿「多发几条 store 指令」换「寄存器预算一直宽裕」。

另外还有两个实现要点值得记一下:

- 持久化内核 (persistent kernel): `bx = T.cta_id([SM_COUNT])`——一个 SM (流多处理器, GPU 上真正执行线程的核心, 见第 0 章) 一个 CTA, 在一个个 tile 之间循环。
- L2 友好调度: `ClusterPersistentScheduler2D` 按缓存局部性把 tile 的顺序重排了一遍。
- 「warp 专门化 + 软件流水线」这一对组合, 是高性能 GEMM 内核的通用套路, CUTLASS 那一派的设计也是这么来的。

当第 7 步出问题：常见坑与流水线深度调优

第 7 步是头一个让 TMA 加载、tcgen05 MMA、写回三个同时在飞的 GEMM 内核。下面这些故障到第 8、9 步会原样再演一遍：

- barrier 计数对不上 (比如 ld2mma.init 没设成 128);
- 角色守卫 (role guard, 也就是 if wg_id == ... / if warp_id == ...) 放错了位置;
- 漏了 fence;
- staging 缓冲在 TMA store 还没排空的时候就被人拿去复用了。

流水线深度调优。第 7 步用的是最小值 PIPE_DEPTH=2。把它调到 4 或 6, 能让 TMA 生产者跑得离 MMA 消费者更远、多藏点内存延迟, 代价是更吃 SMEM——而 SMEM 拢共就那么多,B200 每个 SM 只有 228 KB。按 BLK_M=BLK_N=128, BLK_K=64, fp16 算笔账：

项目	占用
每个流水线 stage(A + B)	$(128 \times 64 + 128 \times 64) \times 2 = 32 \text{ KB}$
Dsmem 写回 staging 缓冲 (固定)	32 KB
PIPE_DEPTH=2 合计	约 $2 \times 32 + 32 = 96 \text{ KB}$
PIPE_DEPTH=4 合计	约 160 KB
PIPE_DEPTH=6 合计	约 224 KB(紧贴 228 KB 上限)

注意

注意：想比 PIPE_DEPTH=6 再深, 就只能重新设计写回的 staging 策略, 否则 SMEM 直接爆掉。

到这儿,warp 专门化让一个 CTA 里头的线程协作了起来。下一步 (第 8 步) 要把这种协作捅过 CTA 的边界——让两个 CTA 一块儿啃一个更大的 tile。

第 8 步：用 2-CTA Cluster 协同算一个大 Tile(Step 8: 2-CTA Cluster)

第 7 步让 TMA、MMA、写回这几个引擎在流水线里叠了起来, 可每个 CTA 还是各干各的: 每个 CTA 独立算自己那块 128×128 的输出 tile, 谁也蹭不到邻居已经搬进来的操作数,B 矩阵被一遍又一遍地重复加载。第 8 步就是要打破这种老死不相往来的局面。

思路是: 把两个 CTA 凑成一个 cluster / 集群, 让它俩能互相读对方的共享内存 (SMEM)。这么一来, 一条协同的 tcgen05 MMA 指令就能算出横跨两个 CTA 的一整块 256×256 输出 tile, 而同一份 B 搬进来, 现在能喂两倍的 MMA 计算。本步矩阵规模还是 M=N=K=4096。

注意

注意：这一步真正赚到的不是「把活儿重新分一遍」, 而是实打实地把**算术强度 / arithmetic intensity**——也就是每搬一字节操作数能做多少次计算——给提上来了。下文会专门讲它为什么大约翻了一倍。

这一步改了什么：作用域 + 布局 + 派发

这次改动, 从三个维度就能概括：

维度	第 7 步 (单 CTA)	第 8 步 (2-CTA Cluster)
----	---------------	-----------------------

协作作用域 (Scope)	单个 CTA 内部协作	一个 cluster 内的两个 CTA 协作
数据布局 (Layout)	操作数全在本 CTA 的 SMEM	操作数分散在两个 CTA 的 SMEM; 由 CTA-0 持有共享的完成 barrier(通过 remote_view)
指令派发 (Dispatch)	MMA 用 cta_group=1	MMA 带上 cta_group=2 / cta_mask, 让 tcgen05 作为一条跨 2-CTA 的协同指令运行

这一步会用到的关键技术点:

- **CTA cluster:** 多个 CTA 合伙算一个更大的 tile。
- **跨 CTA 读 SMEM:** 用 `map_shared_rank` 去读邻居 CTA 的共享内存 (也就是 DSMEM, 分布式共享内存 / distributed shared memory)。
- **cta_group=2 的协同 MMA:** 在 256×256 的 cluster tile 上做一次协同矩阵乘。
- **跨 CTA 发 barrier 信号:** 用 `cta_mask` 一次把信号发给 cluster 里两个 CTA 的 barrier。

cluster tile 的形状与拆分

整套优化全靠一个硬件能力撑着:`cta_group=2` 的时候,MMA 不再只能读自己这个 CTA 暂存 (stage) 的操作数 tile, 而是被允许把**两个 CTA** 的一块儿读了。每个 CTA 加载存储版 B 的一个 128 行切片, 转置之后这 128 行就摇身变成 128 个**逻辑输出列**; 协同 MMA 再把两个切片「缝」成一个完整的操作数。

提醒一句: 本教程把 GEMM 存成 $D = A @ B.T$, 其中**存储版的 B 形状是 $N\times K$** (注意是 $N\times K$, 不是 $K\times N$, 所以才要 $B.T$)。两个 CTA 凑成 cluster 后, 这块 256×256 的 tile 拆得特别干净:

- **A 沿行方向 (竖着) 切:**CTA-0 拿 A0(第 0~127 行),CTA-1 拿 A1(第 128~255 行)。竖着擻起来就是 [A0; A1], 一共 256 行。
- **存储版的 B 按行切:**CTA-0 加载 B 第 0~127 行,CTA-1 加载 B 第 128~255 行。因为算式里用的是 $B.T$, 这两段「存储行切片」一转置, 就成了逻辑右操作数的两段「128 列切片」。
- 有了 `cta_group=2`,MMA 硬件会靠跨 CTA 的共享内存访问, 从**两个 CTA** 的 SMEM 里同时读 B, 这样它眼里就是一整条完整的逻辑输出列跨度。
- 结果就是: 两个 CTA 合力算出**一块 256×256 的输出 tile**, 每个 CTA 负责写回其中一条 128×256 的横向条带 (row stripe)。

下面这张表画的是这俩 CTA 的 A、B 切片怎么拼成 256×256 的 cluster tile(每一行对应一个 CTA 负责的那条输出行条带):

持有方	A 按行切	存储版 B($N\times K$) 按行切	B.T 后的逻辑列	输出 tile 256×256 (列 0-255)
CTA-0	A0 行 0-127	B0 行 0-127	逻辑列 0-127	写出第 0-127 行 $\times$ 整 256 列
CTA-1	A1 行 128-255	B1 行 128-255	逻辑列 128-255	写出第 128-255 行 $\times$ 整 256 列

注意:MMA(`cta_group=2`) 会把两个 CTA 的 B 一块儿读, 把两段 128 列的逻辑切片缝成完整的 256 列; 两个 CTA 各写各的那条 128 行条带, 但每条都横跨整整 256 输出列。

再说明一下: 每个 CTA 手里只有自己那段 A 行切片、和一段存储版 B 行切片, 可靠着 DSMEM, 它还能跨 cluster 把对方那段 B 切片也读到。两段切片转置后合起来, 正好把整条输出列范围盖满, 于是两个 CTA 合力憋出一块完整的 256×256 输出 tile。

为什么这是真收益, 而不是把活儿挪了挪

这里值得停下来算笔账。每个 CTA 还是只加载 $128 \times K$ 的 A 和 $128 \times K$ 的 B, 所以整个 cluster 暂存的操作数大概是单 CTA 的 2 倍; 可它产出的是 256×256 的 tile, 对应的输出 FLOPs 大约是 128×128 tile 的 4 倍 (面积涨了 4 倍)。

搬运量翻 2 倍, 计算量翻 4 倍——一除就明白了: 每搬一字节操作数, MMA 干的计算量大约翻了倍。凭啥能这样? 因为每个 CTA 的 B 切片, 通过协同 MMA 被「借」到了对方 CTA 的 A 切片上复用了一遍。换句话说, **算术强度大约翻倍**。对一个还偏内存受限 (memory-bound) 的 kernel 来讲, 这正是最想要的那根杠杆。后面 End-to-End 性能表里那个约 $2.2 \times$ 的加速, 根子就在这儿: 同样多的字节, 喂了更多的计算。

tile 地址计算

工作单元现在变成了 cluster, tile 调度器 (scheduler) 也就得按 **cluster tile** 来数了。它吐出来的每个 (m_idx, n_idx) 都指着一整块 256×256 的区域, 再由 cluster 里两个 CTA 把这块分着干。那怎么把一个 cluster 坐标翻译成每个 CTA 实际该加载的切片? 代码长这样:

```
m_st = (m_idx * CTA_GROUP + cbx) * BLK_M # CTA
n_st = (n_idx * CTA_GROUP + cbx) * BLK_N # CTA      MMA      B
```

两个 CTA 啃的是同一块 256×256 cluster tile, 而 cbx (CTA 在 cluster 里的位置, 0 或 1) 这一个坐标, 同时在两个轴上挑出本 CTA 该管的那份。注意 num_m_tiles = M // 256、num_n_tiles = N // 256 数的是 cluster tile, 不是单个 CTA 的 tile。

注意

注意: 乍一看 cbx 在 m_st 和 n_st 里都冒出来了, 像是「行偏移漏到列里去了」。其实两处都没错, 只是意思不一样, 得分开看:

- 写回路径上, cbx 只管 M 轴: 每个 CTA 攥着一条自己的 128 行条带 (CTA-0 写 m_idx*256 往后的 128 行, CTA-1 写紧挨着的下一段 128 行), 但两个 CTA 都写整整 256 输出列。这就是为什么写回的列坐标取自 cluster 的 n_idx (n_st_epi = n_idx*256 + no*128, 里头没有 cbx), 而不是取自各 CTA 的 n_st。
- n_st 里之所以带上 cbx, 是因为每个 CTA 往 MMA 里喂的是不一样的存储版 B 行切片: 这里的 cbx 是个**加载偏移**, 不是这个 CTA 的输出列偏移。

相比第 7 步的代码改动

跟第 7 步比, 改动集中在六处, 每一处都对得上前面「cluster 契约」里的某个要点:

```
# 1. Cluster      :      CTA      cluster
cbx, cby = T.cta_id_in_cluster([CTA_GROUP, 1]) # cbx = CTA      cluster      (0      1)
```

```

# 2.      MMA(          cta_group=1)
Tx.gemm_async(..., cta_group=2)

# 3.      CTA          :          (rank=1)  Bsmem
B_remote = T.ptx.map_shared_rank(Bsmem, cta_id=1)

# 4.      CTA barrier:   CTA-0    barrier      cluster
tma2mma_cta0 = T.decl_buffer(
    [CTA_GROUP], "uint64",
    data=T.ptx.map_shared_rank(tma2mma.ptr_to([0]), 0),
    scope="shared")

# 5.      mma2tma / mma2ld arrive:   cta_mask=0(   CTA,    7   )
#          cta_mask=3(          cluster      CTA)
mma2tma.arrive(mma_ps.stage, cta_group=CTA_GROUP, cta_mask=3)
mma2ld.arrive(0, cta_group=CTA_GROUP, cta_mask=3)

# 6.          cluster_sync      cta_sync
T.cuda.cluster_sync()

```

作用域变成 cluster 后的几个关键变化

这六处改动，说到底都出自同一个转变：协作范围从单个 CTA 撑大到了整个 cluster。下面就把这个「撑大」落到实处，一条条说清楚——每个 CTA 怎么知道自己在哪、cluster 拿谁的 barrier 来协调、协同 MMA 究竟由哪个 CTA 发。

- **cluster 内的 CTA 编号:**cbx 告诉每个 CTA 它在 cluster 里的位置 (0 或 1)。CTA-0 管 A 的第 0~127 行,CTA-1 管第 128~255 行。
- **远程 barrier 视图 (remote view):**cluster 里每个 CTA 都有自己的 SMEM、自己的 barrier。那就冒出一个很自然的问题:CTA-1 要等 CTA-0 产出的东西，它该去碰谁的 barrier? 答案是——挑 CTA-0 的 barrier 当唯一的协调点，让 cluster 里随便哪个 CTA 都能访问它。`map_shared_rank(tma2mma.ptr_to([0]), 0)` 返回一个指向 CTA-0 barrier、全 cluster 都看得见的指针，在 TIRx 封装层里就是 `tma2mma.remote_view(0)`。打这以后，每一次 arrive 和 wait，指的都是 CTA-0 那一份 barrier。
- **MMA 只由 CTA-0 发:**好多人会把 `cta_group=2` 脑补成「两个引擎同时点火、各跑各的」，其实不是。只有 CTA-0 发一条 `tcgen05.mma`，然后硬件驱动一条协同 MMA 横跨两个 CTA：从两个 SM 的 SMEM 读操作数，把累加结果写进两个 SM 的 TMEM。CTA-1 一条 MMA 都不发。(每个 SM 只有一个 `tcgen05` 引擎，所以 `cta_group=2` 是一条跨 SM 的 MMA，不是两个引擎并排各忙各的。) 这就是代码里为啥要拿 `if cbx == 0`：把 MMA 守起来。
- **多播 arrive(multicast arrive):**`tcgen05.commit(..., cta_group=2, cta_mask=3)` 同样只由 CTA-0 发，但它一发就同时给两个 CTA 的 barrier 都发了信号。`cta_mask=3`(二进制 11) 的意思就是 CTA-0 和 CTA-1 都是目标。
- **ld2mma 的初始化计数:**`init(128 * CTA_GROUP)`——因为两个 CTA 的写回 warpgroup(各 128 个线程) 都会来 arrive，所以等待计数得 $\times 2$ 。

实现要点

下面是 **hgemm_v8** 的关键骨架。完整源码很长, 这里精简成几段有代表性的代码, 加上中文讲解 (长的实现部分用注释带过, 不逐行抄):

```
def hgemm_v8(M, N, K):
    CTA_GROUP = 2
    BLK_M, BLK_N, BLK_K = 128, 128, 64
    MMA_M, MMA_N = 256, 256          # MMA          256x256    cluster tile
    F16_SIZE = 2                      # fp16          2

    @T.prim_func
    def kernel(A, B, D):
        bx = T.cta_id([SM_COUNT])
        cbx, cby = T.cta_id_in_cluster([CTA_GROUP, 1]) # CTA    cluster

        # ---- barrier          ----
        ld2mma.init(128 * CTA_GROUP) # CTA          arrive,    x2

        # ---- TMEM          ,          cta_group ----
        if wg_id == 0 and warp_id == 0:
            T.ptx.tcgen05.alloc(..., n_cols=512, cta_group=CTA_GROUP)

        # ---- tile          cluster tile          ----
        tile_scheduler = ClusterPersistentScheduler2D(
            "ts", num_m_tiles=M // 256, num_n_tiles=N // 256, #          //256
            l2_group_size=8, num_clusters=SM_COUNT // CTA_GROUP)
        tile_scheduler.init(bx // CTA_GROUP)
        m_st = (m_idx * CTA_GROUP + cbx) * BLK_M
        n_st = (n_idx * CTA_GROUP + cbx) * BLK_N

        # ---- CTA barrier          :          CTA-0 ----
        tma2mma_cta0 = tma2mma.remote_view(0)
```

生产者 (TMA) 和消费者 (MMA) 的核心逻辑, 能拎出三条要点:

```
# ---- TMA          (warp 3):          CTA          A B          ----
# arrive          CTA          :
tma2mma_cta0.arrive(
    tma_ps.stage,
    CTA_GROUP * (BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE) # xCTA_GROUP

# ---- MMA          (warp 0):          CTA-0    MMA ----
if cbx == 0: #          :          MMA          CTA-0
    Tx.gemm_async(
        tmem[:, :MMA_N], Asmem[...], Bsmem[...],
        accum=(k != 0), dispatch="tcgen05", cta_group=CTA_GROUP)
    mma2tma.arrive(mma_ps.stage, cta_group=CTA_GROUP, cta_mask=3) #          CTA
```

写回阶段得把 256 列掰成两块 128 列分头处理:

```
# ----          warpgroup:256          2    x128          ----
for no in T.unroll(2): #          256          ,
    Tx.wg.copy_async(reg_wg[:, ], tmem[:, no*128:(no+1)*128]) #          TMEM
```

```
Tx.cast(reg_f16[:,], reg[:,]) # fp32 -> fp16
#           cluster      n_idx,      cbx:
n_st_epi = n_idx * 256 + no * 128
Tx.copy_async(D[m_st:m_st+BLK_M, n_st_epi:n_st_epi+128],
              Dsmem[:, :], dispatch="tma") # TMA store
ld2mma.arrive(0, cta_id=0, pred=True) # MMA:TMEM

# ---- :cluster_sync      cta_sync,      CTA      TMEM ----
T.cuda.cluster_sync()
if warp_id == 0:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=CTA_GROUP)
    T.ptx.tcgen05.dealloc(tmem_addr[0], n_cols=512, cta_group=CTA_GROUP)
```

为支持 2-CTA 而做的调整汇总

调整项	具体改动	原因
输出宽度	CTA_GROUP=2,MMA_N = BLK_N * CTA_GROUP = 256	cluster tile 列宽翻倍
等待计数	ld2mma.init(128 * CTA_GROUP)	两个 CTA 的写回 warpgroup 都要 arrive
TMA arrive 字节数	CTA_GROUP * (BLK_M*BLK_K + BLK_N*BLK_K) * F16_SIZE	完成信号要把两个 CTA 加载的字节都算进去
TMEM 申请/释放	tcgen05.alloc 与 tcgen05.dealloc 都用 cta_group=2	协同分配/回收跨两个 SM 的 TMEM
写回分块	256 输出列拆成两块各 128 列	一次读全部 256 列 TMEM 会超出寄存器容量 (第 9 步进一步缩到 EPI_N=64)
结尾同步	用 cluster_sync() 取代 cta_sync()	确保所有 CTA 都干完后, 才回收 TMEM

性能与排错

多出来的这点算术强度, 直接反映在墙钟时间上: 第 8 步在 4096³ 规模下跑到 **0.104 ms**, 比第 1 步那个 70 ms 的朴素算法 (同规模) 快了约 **676×**(见 End-to-End 表)。这会儿 kernel 已经开始往计算受限 (compute-bound) 那头靠了, 这正好给第 9 步埋下伏笔——下一步要再添一个 MMA 消费者, 让更多 Tensor Core 计算同时在飞 (in-flight)。

注意

注意: 要是你发现第 8 步**反倒比第 7 步还慢**, 那基本可以断定是某条新加的 cluster 契约填岔了一点点。优先查这三处:

1. TMA arrive 的字节数是不是 CTA_GROUP * (BLK_M*BLK_K + BLK_N*BLK_K) * F16_SIZE;

2. 调度器的维度是不是 num_m_tiles=M//256, num_n_tiles=N//256(对应 256×256 的 cluster tile);

3. 写回是不是发了**两次** TMA store(每块 128 列一次), 而且每次都在 Dsmem 被重新占用前排空 (drain) 了。

cluster 把复用拉到了跨 CTA 这个层面; 而压轴的 第 9 步掉头往里走, 给生产者再配一个 MMA 消费者, 在**每个 CTA 内部**接着榨计算密度。

第 9 步: 多消费者 Warp 专门化 (Step 9: Multi-Consumer Warp Specialization)

到第 8 步,MMA 单元已经喂得挺饱:TMA 在后台流水搬数据,MMA 几乎一刻不闲地算。可还藏着一处浪费——一个 **consumer warp** 消化一块暂存好 (staged) 的 **B tile**, 速度终归是有上限的; 而这块 B tile 其实就静静躺在 SMEM 里, 谁想读都能读。第 9 步抓的就是这个空子: 再添一个 MMA consumer, 让它拿另一块 **A** 去乘同一块 **B**。这么一来, 每个 CTA 的计算密度直接翻倍,cluster 的输出 tile 也从 256×256 长成了 512×256。规模还是 M=N=K=4096。

这一步的核心, 三句话就说完了:

维度	第 8 步 → 第 9 步的变化
角色范围 (Scope)	一个 MMA consumer 变成两个, 靠 warp_id 区分
数据布局 (Layout)	一块暂存的 B tile 被两个 consumer 复用;A 多出一个「consumer 轴」
调度 (Dispatch)	不变

本节盯三件事:

- 多个 MMA warp(consumer) 怎么把吞吐顶上去;
- 多个写回 (writeback)warpgroup 怎么各用各的独立 barrier slot;
- 这套结构正是本教程里**最优那个 GEMM 变体**用的骨架。

为什么只共享 B, 不共享 A

要看懂这一步, 关键得抓住整个设计赖以立住的那个**非对称 / asymmetry**: 两个 consumer 各用自己的 A block, 去乘同一块 staged B tile。

凭啥共享 B、不共享 A? 因为两个 consumer 管的是不同的 **M 行条带 (row stripe)**——它们的 A block 是货真价实两份不同的数据; 可对这两条 M 条带来说, 要乘的 B 却**一模一样**。于是:

- 一次 B 的 TMA load, 现在喂了 2× 的 MMA 工作量;
- 折算下来, **每个有用 FLOP 摊到的 B 加载成本砍了一半**。

结构图

连接关系:

- B tile (同一份, 共享) → A_block[0] (M 行条带 0)
- B tile (同一份, 共享) → A_block[1] (M 行条带 1)
- A_block[0] (M 行条带 0) → consumer0
- A_block[1] (M 行条带 1) → consumer1
- consumer0 各写各自的累加器列区间 → TMEM [0:256]
- consumer1 各写各自的累加器列区间 → TMEM [256:512]

注意

注意: 之所以能共享 B, 根子上是因为两个 consumer 在 N 方向落在同一个 tile, 只在 M 方向错开。换句话说, 这是**唯一**说得通的共享法——原文的 Exercise 3 就是让你自己把这事想透 (反过来要是共享 A、各算各的 B, 那两份 B 是不同数据, 压根没法共享)。

多消费者的角色布局

添了第二个 consumer,kernel 里要安排的角色也跟着多了:MMA warp 从一个变俩, 还得再配一个写回 warpgroup, 去把多出来那份累加器排空 (drain)。配置上设 NUM_CONSUMER=2、WG_NUMBER=3, 于是整个 kernel 摊到三个 warpgroup(下表简称 WG):

Warpgroup	Warp	角色
WG 2	warp 0	MMA consumer 0:Asmem[... , 0] × B → TMEM 列 [0:256]
WG 2	warp 1	MMA consumer 1:Asmem[... , 1] × B → TMEM 列 [256:512]
WG 2	warp 3	TMA producer: 每个 stage 加载 2 块 A + 1 块 B
WG 0	全部	consumer 0 的写回: 读 TMEM [0:256]
WG 1	全部	consumer 1 的写回: 读 TMEM [256:512]

看得出来,WG 2 内部还是「生产者和消费者住一块儿」的格局 (producer 在 warp 3, 两个 consumer 在 warp 0/1), 而 WG 0、WG 1 各包一个 consumer 的写回。这种一对一绑定 (consumer $i \leftrightarrow$ 写回 WG $i \leftrightarrow$ TMEM 列区间 i), 是后面所有 barrier 设计的地基。

下面这张图把数据流串起来看:

结构图

分组:

- **Warpgroup 2:** 「warp 3 / TMA producer」 「warp 0 / MMA consumer 0」 「warp 1 / MMA consumer 1」

连接关系:

- warp 3 / TMA producer $\xrightarrow{2 \times A + 1 \times B \text{ 入 SMEM}}$ SMEM / Asmem(s,0) / Asmem(s,1) / Bsmem
- SMEM / Asmem(s,0) / Asmem(s,1) / Bsmem $\xrightarrow{A0 + B \text{ 共享}}$ warp 0 / MMA consumer 0
- SMEM / Asmem(s,0) / Asmem(s,1) / Bsmem $\xrightarrow{A1 + B \text{ 共享}}$ warp 1 / MMA consumer 1
- warp 0 / MMA consumer 0 $\xrightarrow{\text{累加}}$ TMEM 列 [0:256]
- warp 1 / MMA consumer 1 $\xrightarrow{\text{累加}}$ TMEM 列 [256:512]
- TMEM 列 [0:256] → WG 0 写回
- TMEM 列 [256:512] → WG 1 写回
- WG 0 写回 → 全局内存 D
- WG 1 写回 → 全局内存 D

相比第 8 步具体改了什么

要支持第二个 consumer,kernel 只动了几处地方, 而且每一处改动都能追溯到同一个事实: 现在每个 stage 要喂饱、再排空两块 A、两段 TMEM, 而 B 还是共享一份。改动归得起三类: 多暂存一块 A、

给每个 consumer 配一个独立的 barrier slot、为长高到 512×256 的 cluster tile 调整 tile 寻址。

改动点	第 8 步	第 9 步
A 的暂存形状	(PIPE_DEPTH, BLK_M, BLK_K)	(PIPE_DEPTH, NUM_CONSUMER, BLK_M, BLK_K), 每 stage 2 块 A
TMA arrive 字节数	1×A + 1×B	CTA_GROUP * (NUM_CONSUMER*BLK_M*BLK_K + BLK_N*BLK_K) * F16_SIZE(多算一块 A)
MMA 选数据	固定	用 warp_id 选 A block 和 TMEM 列区间
mma2tma 初始化	init(1)	init(NUM_CONSUMER), 每 stage 等两个 consumer 都 arrive
mma2ld / ld2mma 深度	1	depth=NUM_CONSUMER, 每个 consumer 一个独立 slot
M 方向 tile 地址	(m_idx*CTA_GROUP + cbx)*BLK_M	(m_idx*NUM_CONSUMER*CTA_GROUP + cbx)*BLK_M(多一个 NUM_CONSUMER 因子)
tile 调度器	num_m_tiles = M//256	num_m_tiles = M//256//NUM_CONSUMER(cluster tile 变 512×256)
写回	一次性读全部列	按 EPI_N 分块读, 减少同时存活在寄存器里的回读值

注意

注意:mma2tma.init(NUM_CONSUMER) 此处改动很要命——B tile 是两个 consumer 共用的, 所以 TMA 必须等两个 consumer 都用完 (各 arrive 一次, 合起来两次) 才能放心覆写这个 stage 的 SMEM。少等一个, 就撞数据竞争。

关键代码片段精读

下面只挑跟「多 consumer」直接相关的几行核心, 完整实现请看原文。

(1) A 多出一根 consumer 轴,B 仍是单份

```
# A      :[           , consumer , M, K]           stage      A
Asmem = pool.alloc((PIPE_DEPTH, NUM_CONSUMER, BLK_M, BLK_K), a_type, layout=A_layout)
# B      :           ,           consumer
Bsmem = pool.alloc((PIPE_DEPTH, BLK_N, BLK_K), b_type, layout=B_layout)

mma2ld = TCGen05Bar(pool, NUM_CONSUMER) # depth=2:      consumer      slot
ld2mma = MBarrier(pool, NUM_CONSUMER)    # depth=2:      consumer      slot
```

(2) TMA producer: 每个 stage 搬两块 A、一块 B

```
@T.inline
def tma_load(k_offset):
    m_st_c1 = T.meta_var(m_st + CTA_GROUP * BLK_M) #           A      M      :      consumer 0
    ↪ CTA_GROUP*BLK_M=256
    Tx.copy_async(Asmem[tma_ps.stage, 0, :, :], [m_st:m_st+BLK_M, ...], ...) # A block 0
```

```
Tx.copy_async(Asmem[tma_ps.stage, 1, :, :], A[m_st_c1:m_st_c1+BLK_M, ...], ...) # A block 1
Tx.copy_async(Bsmem[tma_ps.stage, :, :], B[n_st:n_st+BLK_N, ...], ...) # B
# arrive A
tma2mma.cta0.arrive(tma_ps.stage,
    CTA_GROUP * (NUM_CONSUMER * BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE)
```

(3) MMA consumer: 用 warp_id 路由 A block 与 TMEM 列

```
elif warp_id < NUM_CONSUMER: # warp 0/1 consumer
...
Tx.gemm_async(
    tmem[:, warp_id * MMA_N : warp_id * MMA_N + MMA_N], # warp_id TMEM
    Asmem[mma_ps.stage, warp_id, :, :], # warp_id A
    Bsmem[mma_ps.stage, :, :], # B,
    accum=(k != 0), dispatch="tcgen05", cta_group=CTA_GROUP)
mma2tma.arrive(mma_ps.stage, cta_group=CTA_GROUP, cta_mask=3) # stage, TMA
...
mma2ld.arrive(warp_id, cta_group=CTA_GROUP, cta_mask=3) # , warp_id WG
```

(4) 写回: 按 wg_id 认领自己那段 TMEM, 并分块回读

```
elif wg_id < NUM_CONSUMER: # WG 0/1 consumer
    while tile_scheduler.valid():
        mma2ld.wait(wg_id, wb_ps.phase) # consumer slot
        ...
        for i in T.unroll(MMA_N // EPI_N): # 256 4 EPI_N=64
            col_st = T.meta_var(wg_id * MMA_N + i * EPI_N) # wg_id TMEM
            Tx.wg.copy_async(reg_wg[:,], tmem[:, col_st:col_st+EPI_N])
            ... # cast fp16 -> SMEM -> TMA GMEM
        ld2mma.arrive(wg_id, cta_id=0, pred=True) # MMA consumer:TMEM
```

注意

注意: 写回为啥要把 256 列拆成 4 个 EPI_N=64 的小块一块块来? 因为从 TMEM 回读 (read-back) 出来的值, 得先落到寄存器、再做 cast。一口气把 256 列全读进来, 会让一大堆回读值同时挤在寄存器里, 把预算撑爆; 切成小块以后, 每轮迭代只让少量回读值活在寄存器里, 寄存器压力就好拿捏多了。

两对 barrier 怎么「按消费者」对接

第 9 步有个容易看走眼的细节:mma2ld 和 ld2mma 并不是给每个 consumer 各建一个独立对象, 而是各自就一个对象, 只不过内部 depth=NUM_CONSUMER(也就是 2 个 slot)。两条「MMA ↔ 写回」的通路, 靠 slot 下标来分:

Barrier	方向	slot 0	slot 1	MMA 侧按...索引	写回侧按...索引
mma2ld	MMA → 写回	接 warp 0 ↔ WG 0	接 warp 1 ↔ WG 1	warp_id	wg_id

1d2mma	写回 → MMA	接 WG 0 ↔ warp 0	接 WG 1 ↔ warp 1	warp_id	wg_id
--------	----------	--------------------	--------------------	---------	-------

一句话:MMA 侧用 `warp_id` 选 slot, 写回侧用 `wg_id` 选 slot。又因为角色布局上 consumer i 正好绑死写回 WG i, 两边的下标自然对得上——slot 0 永远连着「consumer 0 ↔ WG 0」,slot 1 永远连着「consumer 1 ↔ WG 1」。这套「一个对象 + 多个 slot」的写法, 比「一个 consumer 配一个对象」更省 SMEM; 往后想扩展到更多 consumer, 把 `depth` 调大就完事。

端到端的优化成果 (End-to-End Result)

从 Step 1 一路打磨到 Step 9, 我们终于能把所有招数串成一条完整的链子, 回头看看: 从最朴素的实现走到「跟 cuBLAS 打平」, 中间到底跨过了多大一道坎。这一节不讲新东西, 就是回过头拿一组能横向比的基准数据, 把整章的脉络捋顺——每一步到底快了多少、为什么越往后收益越薄、还有哪几项技术才是真正的大头。

基准条件: 让数字能比的前提

性能数字只有拿「同一把尺子」量出来, 才谈得上比较。本节所有数据都来自同一组受控条件:

- 硬件:NVIDIA B200
- 问题规模: $M = N = K = 4096, \text{fp16}$
- 时钟锁频 (locked clocks), 避免动态频率波动干扰
- 计时方式:1000 次迭代的计时基准 (timed benchmark)

注意

注意: 这组数字请当成「特定条件下的一次 B200 参考跑分」来看, 别拿它当排行榜成绩。每一步里嵌的那个 `{.python .input}` 基准单元格算「冒烟基准 (smoke benchmark)」——看趋势、判断方向对不对挺好使, 但不足以拿它去宣称摸到了峰值性能。

完整的加速链路

下表把从朴素基线到 warp 专门化 cluster kernel 的每个里程碑都摆出来, 拿 cuBLAS 当参照:

步骤	技术	耗时	相对加速
1	同步加载 + MMA(Sync load + MMA)	70 ms	1×
2	K 维循环累加 (K-loop accumulation)	—	处理大于单个 tile 的 K
3	空间分块 (Spatial tiling)	53.6 ms	约 1.3×
4	TMA 异步加载 (TMA async load)	0.49 ms	约 142×
5	软件流水线 (Software pipeline)	—	加载与计算重叠
6	持久化 kernel(Persistent kernel)	—	L2 缓存局部性
7	warp 专门化 (Warp specialization)	0.23 ms	约 309×

8	双 CTA cluster(2-CTA cluster)	0.104 ms	约 676×
9	多消费者 (Multi-consumer)	0.094 ms	约 744×
—	cuBLAS(参照)	0.094 ms	约 744×

表里带破折号那几行 (Step 2、5、6), 是为了把结构讲明白才列上去、但没单独计时的步骤。它们的价值在于「让后面的步骤变得可能」, 而不在于自己能拉出一个独立的耗时数。

关键澄清: 那个 70 ms 到底是啥

这一行最容易被读岔, 值得单拎出来说说。

70 ms 不是《构建分块 GEMM》那一节里「单 tile 的 Step 1 kernel」跑在 4096³ 上的成绩——那个 kernel 一辈子只算一个 128×128 的 tile, 而且只在小规模下跑。

70 ms 其实是一个朴素的全尺寸基线: 它沿用同样「顺序、单 tile」那套思路, 但把问题放大到完整的 4096³。换句话说, Step 1-3 在《构建分块 GEMM》里是拿小规模 (128×128、256³) 讲的, 图的是一开始讲解尽量简单; 而本表里的 Step 1、Step 3 那两行, 是它们对应的全尺寸基准版。

这点理清很要紧, 因为整条加速链之所以能「端到端横着比」, 靠的正是包括 70 ms 在内的每一个数, 都测自同一个 M = N = K = 4096 的规模。

别把 142× 这个数读歪了

从 Step 1 到 Step 4 那约 142×, 是表里最唬人的一跳。但它究竟是谁的功劳, 得看清楚:

结构图

连接关系:

- Step 1(单 128×128 tile, grid 1×1, 软件拷贝) 这一跳同时叠加了多项变化: K 维循环 (处理完整 K) + 空间分块 (把输出分块)
Step 4 (完整的分块并行 kernel + TMA)

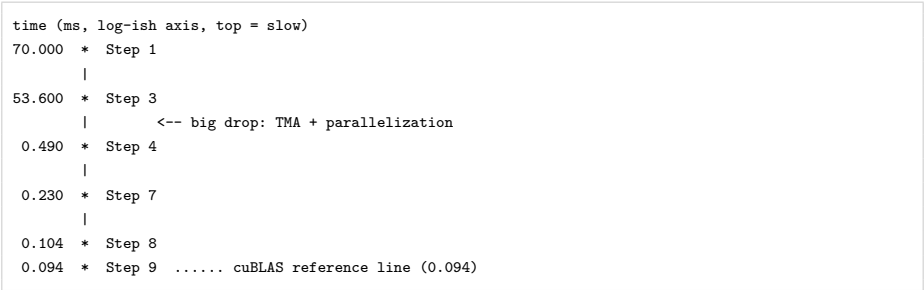
也就是说, 142× 是「从一个 grid(网格, 一次内核启动里全部 CTA 的总集合, 见第 0 章)1×1 的单 tile kernel, 一路做到带 K 循环、空间分块、众多 CTA 并行、还用上 TMA 的完整 kernel」的**综合效果**, 不是 TMA 一个人的功劳。真想把 TMA 的贡献单独拎出来, 得拿两个只在拷贝机制上有区别、其余全一样的全尺寸 kernel 去对比。

四项技术撑起了几乎全部收益

整条链子上, 真正起决定作用的就这四项技术:

- TMA 异步数据搬运 (TMA Async Data Movement):** 拿硬件拷贝引擎换掉软件拷贝 (Step 1 → Step 4, 约 142×, 前面说了这是综合效果)。
- 软件流水线 + warp 专门化 (Software Pipelining + Warp Specialization):** 给加载和计算各分一个专职角色, 让两者叠起来 (Step 4 → Step 7, 约 2.2×)。
- CTA Cluster:** 靠双 SM 协作的 MMA, 让 B-tile 在多个 CTA 之间复用得更多 (本基准下 Step 7 → Step 8, 约 2.2×)。
- 多消费者 (Multi-Consumer):** 用两个 MMA warp 把计算密度顶上去 (Step 8 → Step 9, 约 10%)。

把这四项贡献分别标在各自实测的里程碑点上, 就能看到一条曲线: 从「同步分块 kernel」一级一级往下掉, 最后贴近 cuBLAS 的参照线。



图注: 纵轴是耗时 (ms, 可当作对数刻度, 越往上越慢), 自上而下每个点对应一个实测里程碑——Step 1 同步加载 + MMA(70 ms)、Step 3 空间分块 (53.6 ms)、Step 4 TMA 异步加载 (0.49 ms)、Step 7 warp 专门化 (0.23 ms)、Step 8 双 CTA cluster(0.104 ms)、Step 9 多消费者 (0.094 ms);Step 3 到 Step 4 那段「断崖式落差」来自 TMA + 并行化; 最底下那条虚线是 cuBLAS 参照线 (0.094 ms),Step 9 已经跟它打平。

注意

注意: 这条曲线只标了「实测的里程碑点」, 中间设计时的几步 (2、5、6) 没画上去, 但它们是后面每一步的结构性铺垫。

为什么收益越往后越小

看上面那张表会发现: 加速越往下越小。这背后是**结构性的原因**, 不是后期优化「不够卖力」。

道理是这样:

- 前期那几步打的是「内存瓶颈」: TMA 换掉软件拷贝、cluster 拉高算术强度。而 70 ms 里绝大部分时间偏偏就耗在内存搬运上, 所以打这些靶子回报最高。
- 到 Step 8, kernel 已经贴近 cuBLAS 了 (0.104 vs 0.094 ms, 差约 10%), 也快摸到**计算受限 (compute-bound)**。这意味着几乎没剩多少内存停顿 (memory stall) 还能拿来遮掩了。
- Step 9 的多消费者重叠, 无非是把那点所剩无几的缝隙再捡一捡。

所以, 最后那约 10% 的收益, 正是「快撞上计算天花板」时该有的样子: 它是一个差不多已经解决的问题的**边际递减 (diminishing return)**, 而不是优化使不上劲的信号。

可以拿一张对比表概括这种「瓶颈搬家」:

阶段	主要瓶颈	优化手段	收益量级
Step 1 → 4	内存搬运 (软件拷贝、低并行)	TMA + 分块并行	极大 (约 142×)
Step 4 → 7	加载与计算无法重叠	软件流水线 + warp 专门化	中等 (约 2.2×)
Step 7 → 8	B-tile 复用不足	CTA cluster(双 SM 协作)	中等 (约 2.2×)
Step 8 → 9	残余内存停顿 (已接近计算受限)	多消费者重叠	小 (约 10%)

这些技术会原样带到下一章

本章搭起来的这整套家当——TMA 加载、tcgen05 MMA、TMEM 回读，还有 warp 专门化的 barrier——会原封不动地搬到下一章。

Flash Attention(注意力机制，大模型里算”每个词该关注哪些词”的核心算子) 会把这些全部复用，然后再加一道难度：它不再是干巴巴地重复单个 MMA，而是在两个 MMA 阶段之间塞进一步在线 softmax(online-softmax;softmax 是把一行分数归一化成概率的操作)。换句话说，我们这一章磨出来的「warp 专门化 + 异步流水线」骨架，正好是下一章应付更复杂数据流的起跑线。

习题与自测 (Exercises)

这一小节是对 Step 7-9 的回扣练习。三道题都不是”背公式”那种，而是逼你回到设计动机上：初始 phase 为啥不能瞎设、TMA 字节数为啥要乘 CTA_GROUP、为啥共享 B 而不是共享 A 才是唯一对的选法。这三件事想通了，就等于把整章的流水线契约 (pipeline contract) 在脑子里跑了一遍。下面一题一题给原创讲解和参考答案，最后再附上”拉着你的 agent 一起做”的追踪练习法。

注意

注意：做题时会反复用到一个心智模型——每个 mbarrier 都有”阶段 (phase)”和”计数 (count / byte count)”这两层状态。wait 比的是 phase,arrive 推的是 count;count 攒满了 phase 就翻转 (0↔1)，正等着这个 phase 的那一方才被放行。下面所有推理都搭在这个模型上。

第 1 题：把两端 phase 都设成 0，会怎么死锁？

题目：在 Step 7 里，要是把 TMA 和 MMA 两个 PipelineState 的初始 phase 都设成 0，会发生什么？把死锁场景画出来。

先回顾正确设定。原本 Step 7 里两端是反着的：

```
tma_ps = PipelineState(PIPE_DEPTH, phase=1) # : phase=1
mma_ps = PipelineState(PIPE_DEPTH, phase=0) # : phase=0
```

mbarrier 刚 init 完，物理相位都是 0。wait(phase) 的规矩是：当前相位 == 期望 phase 就卡住，!= 就放行。于是：

角色	初始 phase	首次 wait 看到的相位	比较结果	行为	是否正确
生产者 TMA	1	0	0 != 1	立即放行	对：缓冲区是空的，应立刻开始装填
消费者 MMA	0	0	0 == 0	阻塞	对：还没数据，应该等生产者到达

也就是说，这个一边放行、一边卡住的相位差，正是流水线能启动的前提：生产者先开跑，消费者后头等。

那两端都改成 phase=0 会咋样？生产者这边也会在第一次 wait 时撞上 0 == 0 而卡住。看生产者循环里这一句就明白：

```
mma2tma.wait(tma_ps.stage, tma_ps.phase) # " "
tma_load(k * BLK_K) # TMA load
```

如果 `tma_ps.phase=0`, 生产者头一拍就卡在 `mma2tma.wait` 上, 死等”缓冲区被 MMA 释放”的信号。可缓冲区压根没填过, MMA 自然也没读过、更谈不上释放——于是 `mma2tma.arrive` 永远不会被叫起来。

把这个**死锁闭环**画成流程图, 一目了然 (俩人互相等对方先动手, 形成循环等待):

结构图

连接关系:

- TMA 生产者 / `wait(mma2tma)` 等缓冲区可复用 / (`phase=0`, 阻塞) / 从不执行 `tma_load` 等 `mma2tma.arrive` (缓冲区释放), 但它从不发生 → MMA 消费者 / `wait(tma2mma)` 等数据就绪 / (`phase=0`, 阻塞) / 从不执行 `gemm`
- MMA 消费者 / `wait(tma2mma)` 等数据就绪 / (`phase=0`, 阻塞) / 从不执行 `gemm` 等 `tma2mma.arrive` (数据就绪), 但它从不发生 → TMA 生产者 / `wait(mma2tma)` 等缓冲区可复用 / (`phase=0`, 阻塞) / 从不执行 `tma_load`

俩人互相等对方先动手 → 永久死锁。

- TMA 等 MMA 发 `mma2tma`(缓冲区释放), 才肯装数据;
- MMA 等 TMA 发 `tma2mma`(数据就绪), 才肯计算;
- 这俩 `arrive` 谁都不会先发生, 因为它们各自都卡在 `wait` 上动不了。

这就是个典型的**循环等待 (circular wait)**, 死锁的四个必要条件全凑齐了。根子上的毛病是: 把生产者的初始 `phase` 设错, 等于**不认”缓冲区一开始是空的、可以直接写”这个事实**——你硬逼着生产者也去等一个”释放”信号, 可头一拍根本没东西需要释放。

注意

注意: 反过来, 要是两端都设成 `phase=1`, 事情更糟——它可能不是干脆利落死锁, 而是**静默的数据损坏**: 消费者第一次 `wait(phase=1)` 看到相位是 `0, 0 != 1`, 直接放行, 于是它跑去读一块还没填过的垃圾缓冲区。死锁至少会卡住, 你能察觉; 静默错误却会算出错的结果还不露馅。所以原文才反复念叨, 这一个选择”值得做对”。

第 2 题: `cta_group=2` 时, TMA `arrive` 字节数为啥要乘 `CTA_GROUP`?

题目: Step 8 里 `cta_group=2`, TMA `arrive` 的字节数是 `CTA_GROUP * (BLK_M*BLK_K + BLK_N*BLK_K) * F16_SIZE`。既然每个 CTA 只装自己那份数据, 为啥还要乘 `CTA_GROUP`?

关键先讲清: 这个字节数不是”我装了多少”, 而是”**barrier 要攒够多少字节才算满**”。TMABar 是一个带字节计数 (byte-count) 的 mbarrier: TMA 硬件每搬完一笔数据, 就把对应的字节数 `arrive` 到 barrier 上; 累计字节数一够到设定的门槛, barrier 相位翻转, 等待方 (MMA) 就被放行。所以这个数字是 barrier 的”满载线”, 它必须等于**所有把它当就绪信号的那批 load 的字节总和**。

那这个总和为啥要跨两个 CTA 来算? 因为 Step 8 用的是一枚集群级的共享 barrier——CTA-0 的 barrier 被选成唯一的协调点, 两个 CTA 的 TMA load 全都 `arrive` 到它身上:


```
tma2mma_cta0 = tma2mma.remote_view(0) # CTA CTA-0 barrier

# CTA TMA load tma2mma_cta0 barrier
Tx.copy_async(Asmem[...], A[...], cta_group=CTA_GROUP, mbar=tma2mma_cta0.ptr_to([stage]))
Tx.copy_async(Bsmem[...], B[...], cta_group=CTA_GROUP, mbar=tma2mma_cta0.ptr_to([stage]))
```

而协作式 MMA(cta_group=2) 得等两个 CTA 的 A、B 全到齐才能开算——它要跨两个 SM 的 SMEM 读操作数，拼成一块 256×256 的 tile。所以这枚 barrier 的“满”，就必须定义成:CTA-0 的 (A+B) 加上 CTA-1 的 (A+B) 全都落了地。

拿一张表，把”每个 CTA 装多少”和”barrier 要等多少”分清楚：

量	数值	含义
单 CTA 装的字节	$(BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE$	这个 CTA 自己的 A 片 + B 片
barrier 满载门槛	$CTA_GROUP * (\quad)$	集群里两个 CTA 的数据都到齐

忘了乘 CTA_GROUP 会咋样？那 barrier 的门槛就只设成了单 CTA 的字节数。于是随便哪个 CTA 的 load 先干完，字节数就提前攒满、相位提前翻转,MMA 被叫早了——这时另一个 CTA 的操作数八成还没搬到,MMA 读到半成品，结果就是竞态加错误输出（还时对时错，极难复现）。这正是原文末尾”Step 8 反倒比 Step 7 慢 / 错”排查清单里头一条要查的点。

注意

注意: 别被”每个 CTA 装自己的数据”带偏成”每个 CTA 各等各的”。这里是一枚共享 barrier 等两份数据, 不是”两枚 barrier 各等一份”。乘 CTA_GROUP 编码的就是”协作”这件事本身——把两个 CTA 的就绪条件捏成一个原子的放行信号。Step 9 又添了第二块 A, 门槛跟着变成 $CTA_GROUP * (NUM_CONSUMER * BLK_M * BLK_K + BLK_N * BLK_K) * F16_SIZE$, 逻辑一模一样:barrier 门槛 = 所有挂在它上头的 load 的字节总和。

第 3 题: 为什么共享 B 而不是 A 才对？

题目:Step 9 里每个 consumer 处理不同的 M 行, 却用同一块 B tile。为什么共享 B(而不是 A) 才是对的选法？

先看数学——共享什么, 是 GEMM 的结构定死的。本教程把 GEMM 写成 $D = A @ B.T$ 。输出 tile 上某个位置 $D[m, n]$, 是拿 A 的第 m 行去点乘 B 的第 n 行 (转置后就是第 n 列) 得来的。Step 9 让两个 consumer 各算一条不同的 M 行条带, 但它们落在同一段 N 列范围里:

结构图

连接关系:

- B tile ($N \times K$) / 两个 consumer 共用同一份 $\xrightarrow{\text{共享同一份 B}}$ consumer 0 / A block $0 \times B \rightarrow D$ 行 [0:128] (M 行段 0)
- B tile ($N \times K$) / 两个 consumer 共用同一份 $\xrightarrow{\text{共享同一份 B}}$ consumer 1 / A block $1 \times B \rightarrow D$ 行 [128:256] (M 行段 1)

- 两个 consumer 覆盖不同的 M 行, 所以它们的 A block 是两份不同的真数据 (行 0-127 对行 128-255), 没法共享——硬要共享就会用错行、算错结果。
- 两个 consumer 覆盖同一段 N 列, 而 B 只跟 N、K 有关、跟 M 没关系, 所以两个 consumer 看到的 B 完全一样, 天生就能共用同一份 SMEM 拷贝。

为什么这是”唯一可行”的共享方向? 把四种可能全摆出来, 就一种立得住:

共享方案	两个 consumer 算的是	A 是否相同	B 是否相同	能否共享
共享 B,A 各异 (Step 9 的选择)	不同 M 行、同 N 列	否	是	正确
共享 A,B 各异	同 M 行、不同 N 列	是	否	这是”沿 N 切”, 得另设计;A 相同才可共享
都共享	完全相同的 tile	是	是	没意义 (两个 consumer 算同一块, 白干)
都不共享	完全不同的 tile	否	否	退化回两个独立 GEMM, 没有复用收益

Step 9 的目标, 就是在已经装进 SMEM 的那份 B 上榨出更多 MMA:B tile 一装好, 本来就静静躺在 SMEM 里, 谁都能读。让第二个 consumer 拿一块不同的 A 去乘同一块 B, 一次 B load 就喂了 2× 的 MMA 工作,B 的”每个有效 FLOP 摊到的加载成本”直接砍半。这正是把 kernel 从访存受限推向计算受限的那根杠杆。

注意

注意: 你大概会想”那反过来共享 A、让两个 consumer 各算不同 N 列, 行不行”。数学上确实能切 (沿 N 方向切), 可那是另一套布局——而且在本教程的设定里,B 早就因为集群被劈成两半、跨 CTA 共享掉了,N 方向的复用空间在 Step 8 的 cluster 那一步就吃干净了。Step 9 之所以挑在 M 方向叠 consumer、共享 B, 一来跟现成的 cluster 布局不打架, 二来正好白捡了”B 跟 M 无关”这个数学便宜。所以”共享 B”压根不是随手一挑, 而是被 $D = A @ B.T$ 的结构加上前面几步的布局一块儿锁死的、唯一顺理成章的选择。

拉着你的 agent 一起做: 追一个 K-tile 穿过四道 barrier

原文给了个特别好的动手练习, 这里说说怎么做、要盯住哪些点。

做法: 把 Step 7 的 kernel 整段甩给你的 agent, 让它追某一个 K-tile(比如 k=0 这一拍) 挨个穿过四道 barrier——tma2mma、mma2tma、mma2ld、ld2mma。每到一道 barrier, 就逼它回答四个问题:

1. 谁在等 (who waits): 哪个角色卡在这道 barrier 的 wait 上?
2. 谁来到 (who arrives): 哪个角色、用啥指令在它上面 arrive?
3. 哪块 tile 变得能安全读了 (what becomes safe to read):arrive 之后, 等待方现在能读哪块数据 / 哪段 TMEM?
4. 哪个缓冲区间跟着能复用了 (which buffer is reusable): 这次握手放掉了哪个 SMEM/TMEM slot, 下一拍能重新占用?

自检答案 (拿来核对 agent 答得对不对):

Barrier	谁等	谁到达	之后可安全读	随之可复用
tma2mma(前向, 数据就绪)	MMA 消费者	TMA 硬件按字节计数到达	该 stage 的 Asmem/Bsmem	—(刚被填满, 还不能复用)
mma2tma(后向, 缓冲释放)	TMA 生产者	MMA 用 tcgen05.commit 到达	—(这是释放信号, 不产生新数据)	该 stage 的 Asmem/Bsmem, 供 k+2 拍的 load 复用
mma2ld(前向, 结果就绪)	Writeback warp-group	MMA 用 tcgen05.commit 到达	TMEM 累加结果	—(结果刚算完, 等读出)
ld2mma(后向, TMEM 释放)	MMA 消费者	128 个 writeback 线程全部到达	—(释放信号)	TMEM, 供下一个输出 tile 复用

注意

注意: 盯住”前向 = 数据就绪、后向 = 缓冲释放”这条主线, 就不会乱。有个细节 agent 特别容易答岔——mma2tma 的跨一级释放: 因为 PIPE_DEPTH=2 的环形缓冲就两个槽, MMA 算完 k=0(用的是 buffer 0) 后发出的 mma2tma, 放行的不是想用 buffer 1 的 k=1, 而是想再用一次 buffer 0 的 k=2。让 agent 把环形缓冲那两个 slot 老老实实画出来, 它自己就能看出这个”跳一拍”。另一个考点是 cta_mask: Step 7 里是 arrive(..., cta_mask=0)(没别的 CTA 要通知), 到 Step 8 就变成 cta_mask=3(二进制 11, 一下叫醒集群里两个 CTA)——这正是单 CTA 流水线怎么平滑长成集群协作的那个关键参数。

小结

- **一条主线:** 一步步把「协作范围」做大。本章从「单个 warpgroup 串着驱动三个引擎」这个瓶颈起步, 用 Step 7→8→9 三步, 把协作的边界从「单 CTA 内的角色分工」撑到「跨 2 个 CTA 的 cluster」, 再撑到「同一 CTA 内多消费者复用同一份 B」。每一步都拆掉一个串行瓶颈。
- **Step 7 拿到的关键:** warp 专门化让 TMA、MMA、写回三个引擎真正并起来跑; 搞懂 4 个 barrier「前向报就绪、后向报释放」的意思, 以及 PipelineState 里生产者 (phase=1) 和消费者 (phase=0) 起始相位必须反着, 这是整套流水线契约的核心。
- **Step 8 拿到的关键:** cluster 真正赚的是**算术强度翻倍**——同样的字节喂更多计算。协同 MMA 只由 CTA-0 发一条 cta_group=2 指令、横跨两个 SM; barrier 用 cta_mask=3 多播、用 remote_view 指向 CTA-0 的共享 barrier; TMA arrive 字节数要乘 CTA_GROUP。
- **Step 9 拿到的关键:** 共享 B 而不是共享 A, 是 $D = A \otimes B$ 的数学结构加上前面 cluster 布局一块儿锁死的、唯一顺理成章的选择; mma2ld / ld2mma 用「一个对象 + 多个 slot(depth=NUM_CONSUMER)」按消费者对接, 既省 SMEM 又好扩。
- **端到端来看:** 四项技术 (TMA、软件流水线 + warp 专门化、CTA cluster、多消费者) 撑起了几乎全部加速; 收益越往后越小, 是「瓶颈从访存受限搬到计算受限」之后正常的边际递减, 不是优化没劲了。这套「warp 专门化 + 异步流水线」骨架会原样带到下一章的 Flash Attention。

延伸阅读

- 本章原文: [用 Warp 专门化与 Cluster 扩展 GEMM\(Step 7-9\)](#)

- 上一章: 用 TMA 与 tcgen05 MMA 构建流水线化 GEMM(Step 4-6)——本章直接复用了那一章的多级 SMEM 流水线、TMEM 累加器布局和持久化调度器 ClusterPersistentScheduler2D, 建议先把那一章的 SMEM/TMEM 契约读透, 再回头看本章的角色拆分。
- 下一章:Flash Attention——会原封不动地复用本章的 TMA 加载、tcgen05 MMA、TMEM 回读和 warp 专门化 barrier, 再在两个 MMA 阶段之间塞进在线 softmax(online-softmax)。
- 对照参考:CUTLASS 中的 warp-specialized GEMM 设计与 Hopper/Blackwell 的 cluster(CGA)、TMA、tcgen05 编程模型文档。

术语对照

英文	中文	简要说明
Warp Specialization	Warp 专门化	把不同任务 (加载/计算/写回) 各自指派给专属 warp 并发执行, 而非让一队线程轮流干所有事
Warpgroup	Warpgroup	由 4 个 warp、共 128 个线程组成的协作单元
CTA (Cooperative Thread Array)	CTA / 线程块	一个 thread block, 本章中常作为持久化内核里每个 SM 上的工作单元
Cluster	Cluster / 集群	Hopper/Blackwell 引入的结构, 把多个 CTA 绑成一组、允许互相访问对方 SMEM
DSMEM (Distributed Shared Memory)	分布式共享内存	cluster 内跨 CTA 可访问的共享内存, 通过 map_shared_rank 读取邻居 CTA 的 SMEM
TMA (Tensor Memory Accelerator)	张量内存加速器	硬件拷贝引擎, 异步搬运 tile 并能按字节计数自动在 barrier 上 arrive
Tensor Core / MMA	Tensor Core / 矩阵乘累加	执行矩阵乘累加的硬件单元; 本章用 tcgen05 指令派发
TMEM (Tensor Memory)	张量内存	Tensor Core 专用的累加器内存,MMA 写入、写回阶段回读
Producer / Consumer	生产者 / 消费者	TMA 加载数据为生产者,MMA 计算为消费者
Barrier (mbarrier)	屏障	同步原语; 具有 phase(相位) 与 count/byte-count(计数) 两层状态
Phase	相位	mbarrier 的 0/1 状态; 计数攒满后翻转, 等待该 phase 的一方被放行
Stage	槽位 / 阶段	环形 (多缓冲) 流水线中的缓冲槽, 由 PIPE_DEPTH 决定数量
PipelineState	流水线状态	把 stage 与 phase 绑在一起自动推进, 避免手动维护导致的差一 (off-by-one) 死锁
cta_group=2	协同组大小为 2	让 tcgen05 MMA 作为一条跨 2-CTA 的协同指令运行, 读两个 CTA 的 SMEM
cta_mask	CTA 掩码	指定 arrive 信号要多播给哪些 CTA 的 barrier(如 3 = 二进制 11, 通知两个 CTA)
remote_view / map_shared_rank	远程视图	把某个 CTA 的 barrier/SMEM 指针映射成全 cluster 可见
Arithmetic Intensity	算术强度	每搬运一字节操作数所能做的计算量;cluster 把它约翻倍

Memory-bound / Compute-bound	访存受限 / 计算受限	性能瓶颈在数据搬运还是在计算; 本章把内核从前者推向后者
Persistent Kernel	持久化内核	每个 SM 一个常驻 CTA, 在多个 tile 之间循环, 改善 L2 局部性
Epilogue / Writeback	收尾 / 写回	读 TMEM、转 fp16、用 TMA store 写回 GMEM 的阶段; 高列宽时按 EPI_N 分块以控寄存器压力
Multi-Consumer	多消费者	多个 MMA 消费者复用同一块 staged B tile, 提升计算密度
Ring Buffer	环形缓冲	多 stage 复用的缓冲结构;mma2tma 释放会「跳一拍」正源于此

第 14 章 · Flash Attention 4

原文:Flash Attention 4

本章要点

本章要点 (TL;DR)

- 别把 Attention 当成「再写一遍 GEMM」。它是**两段矩阵乘**中间夹着一段 softmax: 先算分数 $S = Q \cdot K^T$, 再算输出 $O += P \cdot V$, 中间那段 softmax 跑在 CUDA Core 上。难就难在这段 softmax——你嘴上说在优化 Attention, 十有八九是在跟它较劲。
- 在线 softmax(online softmax) 一句话说白了: 从不把整个 S 算出来摆那儿, 而是把 K/V 一块一块喂进来, 边喂边加。每一行它都盯着三个「运行态」: `row_max`、`row_sum` 和输出 O 。麻烦在于, 只要 `row_max` 一变大, 之前加进 O 的东西尺度就错了, 得先**重缩放 (rescale)**再往下加——这正是 GEMM 永远不用操心的事。
- TIRx 把 kernel 拆成 4 个 warpgroup, 按「干什么活」分工: WG3 专门驱动硬件引擎 (TMA load、MMA、TMA store), WG0/WG1 各管一个 Q stage 的 softmax, WG2 做 correction(校正) 和 epilogue(收尾)。记住一条就够: 所有 MMA 都只从 WG3 warp 0 发出来。
- S 、 P 、 O 三块数据全挤在同一块 128×512 的 **TMEM** 里。靠一个 `fp16` 视图别名 (`tmem_as_fp16`) 硬给 P 腾出地方。这么挤还能不出错, 全靠一组 `mbarrier(s_ready、p_o_rescale、p_ready_2、o_ready、softmax_corr)` 把时序管得死死的。
- **因果掩码** (整块跳过 + 跨对角线逐行掩码)、**GQA** (Q 加载、 O 存储两端用 3D 视图零拷贝打包/解包)、**tile 调度** (非因果用 Linear、因果用 LPT)——这三样都只在「边界」上动手脚, 计算核心几乎一行没碰。
- 最关键的一点: 底层那套 TIRx 契约 (tile 原语 + scope + layout + barrier) 和 GEMM 一模一样。FA4 不是另起炉灶, 只是在「广度」上铺得更开——多了些 tile, 多了些交接。

前置知识

前置知识: 读这一章前, 最好先懂 TMA / TMEM / mbarrier(第 5-8 章) 和 GEMM 的 tile 流水线 (第 11-13 章), 以及 attention / softmax 的基本算法。没把握的话, 先翻一下第 0 章 · 极简入门, 再回顾一下 GEMM 那几章。本章会默认你已经认识这些词。

本章导读: 为什么 Attention 不是「再来一遍 GEMM」(Overview)

前面几章, 我们已经把一台 GPU 拆到了最底层: TMA(在 GMEM ↔ SMEM 之间异步搬数据的硬件引擎) 负责在显存 (GMEM, 全局内存, 大但慢) 和共享内存 (SMEM, 片上的小块快速内存) 之间搬数据, `tcgen05` 指令驱动 Tensor Core(专做矩阵乘的硬件单元) 做 MMA(矩阵乘累加, $C += A \cdot B$), 累加器 (accumulator, 边算边攒结果的那块内存) 放在 TMEM(Tensor Memory, Blackwell 新增、专给 MMA 用的片上内存) 里, 再加上各种 `mbarrier`(线程间同步用的「门闩」) 做同步。到了第三部分, 我们又学会了怎么把这些零件拼成一个高性能的 GEMM(通用矩阵乘)。

现在到了 Flash Attention, 这些积木终于要一起上场了。说白了,Attention 就是「决定一个 Transformer 跑不跑得起来」的那个核心 kernel, 也是前面所有功夫第一次必须拧成一股绳的地方。可它偏偏不是把 GEMM 照抄一遍那么简单——它带着一个 GEMM 从来没碰过的结构性难题。

核心矛盾：两个 MMA 中间夹着一坨「真活儿」

普通 GEMM 就是「同一个 MMA 一遍遍重复」：每一步只往累加器里加结果, 累加器是什么意思, 从头到尾没变过。Attention 不一样。它是两个 MMA, 而且这俩中间还夹着一段非得在 CUDA Core 上做的、实打实的计算：

结构图

连接关系：

- $Q, K, V \xrightarrow{TMA} SMEM$
- $SMEM \rightarrow \textcircled{1} \text{ Score MMA (Tensor Core): } S = Q \cdot K^T$
- $\textcircled{1} \text{ Score MMA (Tensor Core): } S = Q \cdot K^T \rightarrow \textcircled{2} \text{ Softmax (CUDA Core): } S \rightarrow P$ （新难点在这里）
- $\textcircled{2} \text{ Softmax (CUDA Core): } S \rightarrow P \text{（新难点在这里）} \rightarrow \textcircled{3} \text{ Value MMA (Tensor Core): } O += P \cdot V$

中间这段 softmax 为什么难缠？两个原因：

1. 它跑在 CUDA Core 上（普通通用计算核心，区别于专做矩阵乘的 Tensor Core），正好卡在两个 Tensor Core MMA 中间。求指数 (exp) 和按行归约 (row-wise reduction) 都直接压在关键路径上。Tensor Core 想接着算第二个 MMA？对不起，得先等 softmax 把 S 揉成 P。
2. 它逼着你回头去改已经算好的结果。这正是 GEMM 从来不用面对的：GEMM 的累加器只加不改；可 Attention 的输出累加器 O，会因为新的 K/V 流进来，被迫整体「换个比例」。

也正因为这样，有个听上去反直觉、但很实在的结论：**嘴上说优化 Attention 的人，其实绝大多数时间都在优化 softmax**——要么换种写法表达 exp，要么想方设法让 softmax 和 MMA 叠在一起 (overlap, 让两件事在时间上同时进行、互相藏延迟) 跑，而不是干等着它算完。

那个 GEMM 没有的「twist」：running max 一变，O 就失真

想看懂后面所有代码，先抓住这一条主线就够了——盯着一个 tile，跟着它在 kernel 里从头走到尾：

阶段	输入	计算单元	产物	含义
输入加载	Q、K、V	TMA	SMEM 中的输入 tile	从 GMEM 搬到 SMEM
① Score MMA	Q、K	Tensor Core	分数 tile S(在 TMEM)	$S = Q \cdot K^T$
② Softmax	S	CUDA Core	分子 tile P	把分数变成（未归一化的）权重
③ Value MMA	P、V	Tensor Core	更新输出累加器 O	$O += P \cdot V$

单看这张表,Attention 就像「两个矩阵乘法粘在一起」。可 Flash Attention 的在线 softmax(online softmax) 会带来一个 GEMM 永远碰不到的麻烦：

注意

注意: 只要那个滑动的 softmax 最大值 (running max) 一变, 之前加进 0 的结果当场就处在错误的比例上。你得先拿新最大值把旧 0 重新缩放 (rescale) 一遍, 下一次 Value MMA 才能安全地往里加。

说白了,Flash Attention 靠「分块一步步算 + 维护 running max / running sum」来躲开「一次性把整个 S 摆出来」这件事; 代价就是: 每来一块新 K/V, 只要发现了更大的最大值, 就得把历史累加值整体打个折, 让先来的块和后来的块站到同一个尺度上。下面这段伪代码就点出了这条「rescale」主线 (具体的 TMEM/MMA 细节先略掉):

```
#      softmax      :m      running max,l      running sum(      ),0
for kv_block in key_value_blocks:      #      K/V
    S = Q @ kv_block.K.T      # (1) Score MMA(Tensor Core)
    m_new = max(m, rowmax(S))      #      running max
    P = exp((S - m_new) / sqrt(d))      # (2) Softmax(CUDA Core,exp      )
    alpha = exp((m - m_new) / sqrt(d))      #      ->
    l = l * alpha + rowsum(P)      #
    O = O * alpha + P @ kv_block.V      # (3)      rescale      0,      Value MMA
    m = m_new
O = O / l      #
```

看 $O = O * \alpha + \dots$ 这一行, 这就是前面反复念叨的那个 twist——它把「回头改已经算好的结果」这件 GEMM 压根不用做的事, 变成了每来一块 K/V 都要做一遍的日常动作。

本章的目标与组织方式

先把话说清楚: 本章不会从零把 Flash Attention 算法重新推一遍。算法部分只讲到「够你看懂 kernel」为止; 真正下功夫讲的, 是那块全新的内容——这套算法怎么落地成 TIRx, 也就是怎么把它映射到具体的硬件原语和 warp 角色上。

所以本章是这么个讲法:

1. 先跟着一个 tile 把整条数据通路走完 (就是上面那条 $Q, K, V \rightarrow S \rightarrow P \rightarrow O$, 外加 rescale 这个 twist), 把「算法到底要干什么」讲透。
2. 再看 TIRx 怎么把每个阶段分给不同的 warpgroup, 然后用 warp 角色分工、在线 softmax 的重缩放、因果掩码 (causal masking)、GQA(分组查询注意力) 这些机制, 把各个阶段一段段接线串起来。

这条主线一句话就能说完:GEMM 教会我们怎么把数据搬进来、怎么喂给 Tensor Core 累加;Attention 在这之上又逼我们多学一手——怎么在两个 MMA 之间塞进一段 softmax, 还要在 running max 一变就回头修正已经算好的输出。接下来所有 TIRx 代码, 要解决的就是这件事。

算法的形状 (Algorithm Shape)

把 tile(数据块) 往各级存储里摆之前, 得先搞清楚这些 tile 到底为哪个算法服务。换句话说, 内存布局不是拍脑袋设计的——它就是算法的计算流程往硬件上一投影的结果。所以这一节先把 Flash Attention 的核心计算逻辑讲明白, 再回头看每个 tile「住哪儿」, 因为「住哪儿」直接决定了你后面要写什么样的 layout 代码和同步 (barrier) 代码。

说到一个 query 块 (query block),Flash Attention 要算的,就是标准的缩放点积注意力 (scaled dot-product attention):

$$O = \text{softmax}(QK^\top / \sqrt{d}) V$$

为什么不能照着公式直算

要是真照公式字面去做, 流程就是: 先把完整的分数矩阵 (score matrix) $S = QK^\top$ 算出来, 对它做 softmax, 再乘 V 。可这偏偏是**唯一不能用的**做法, 原因很简单: 整个 S 实在太大了。

我们算笔账。序列长度 $\text{seq} = 4096$ 的时候, 每个注意力头 (head) 的 S 差不多有 16M 个元素, 用 fp32 存大概 **64 MB**。这个量级比片上的 SMEM(shared memory, 共享内存) 大了好几个数量级, 也远远超过一块 128×512 的 TMEM(tensor memory)。一句话——片上根本没地方搁它。

注意

注意: 这里真正卡脖子的不是「算得慢」, 而是「放不下」。Flash Attention 的核心思路就一条: 永远不把完整的 S 实打实地摆出来 (materialize)。

流式更新与「重缩放」的由来

既然没法一口气算完整个 S ,Flash Attention 就换个打法: 把 K/V 一块一块 (block) 流式喂进来, 边读边算。问题来了——只看到局部数据, 怎么保证结果还对? 办法是给每一行维护三个「运行态 (running state)」, 用它们来概括「到目前为止看过的所有块」:

运行态	含义
row_max	到目前为止见过的最大分数 (每行一个)
row_sum	softmax 分母的累加值 (每行一个)
0	输出累加器 (output accumulator)

难点在哪? **每来一个新块, row_max 都可能变大**。一旦它变大, 之前用旧 max 算出来的那一堆结果就「量错尺子」了——它们是按旧尺度归一化的。所以在把新块的贡献加进来之前, 得先把旧的运行态「拉」到新尺度上。这就是 Flash Attention 里那个有名的**在线重缩放 (online rescaling)**。

单个块的更新逻辑可以写成如下伪代码:

```
S = Q_block @ K_block.T          #          tile(          )
m_new = max(row_max, rowmax(S))   #
scale = exp((row_max - m_new) / sqrt(d)) #    ->
P = exp((S - m_new) / sqrt(d))     #          softmax          (numerator)
row_sum = row_sum * scale + rowsum(P) #      :          ,
O = O * scale + P @ V_block        #      :      0      ,
row_max = m_new                   #
```

这里有个挺妙的设计: **同一个 scale 因子干了两份活**——它既缩放了运行分母 row_sum, 又缩放了运行输出 O 。为什么这么干就对? 因为有了这个统一的因子, 先来的块和后来的块最后都被换算到同一个尺度上, 这样它们才能正确地加到一起。

工程上的两处优化:exp2 与延迟归一化

用 `exp2` 顶替 `exp`。上面的伪代码为了好读,写的是自然指数 `exp` 加上明晃晃的 `/sqrt(d)`。但真正的 kernel 走的是更省的路子:它把 `1/sqrt(d)` 和 `log2(e)` 这两常量提前揉成一个常量 `scale_log2 = log2(e)/sqrt(d)`,然后对着原始分数直接用硬件的 `exp2` 求所有指数,靠的是这个恒等式:

```
exp(x / sqrt(d)) = exp2(x . scale_log2)
```

动机很朴素:在这类硬件上,`exp2` 就是比自然指数 `exp` 快。
归一化是故意往后拖的。这点得说清楚:伪代码里的 `P` 不是最终归一化好的注意力矩阵,它只是当前这一块 `K/V` 的 `softmax` 分子。真正的归一化被有意往后推——非要等最后一块处理完了, kernel 才写出 `0 / row_sum` 当最终输出。这么做就是为了省掉「每块都除一遍」的反反复折腾。

每个 tile 「住在哪儿」

对 TIRx 来说,知道算法算什么只算搞懂了一半;另一半是每个 tile 在 kernel 跑起来的时候待在哪种存储里。后面这一半才是决定 layout 和 barrier 代码的关键。`S`、`P`、`O` 都是 tile 值,各有各的「家」:

Tile	含义	驻留位置与数据通路
S	分数 tile	由 score MMA 算出后写入 TMEM
P	softmax 分子 tile	softmax 把 S 从 TMEM 读进寄存器 (register, 每个线程私有的最快存储), 算出 $P = \exp((S - m_{new})/\sqrt{d})$, 再把 P 写回 TMEM
O	输出累加器 tile	value MMA 从 TMEM 读 P, 从 SMEM 读 v, 把结果累加进 TMEM 里的 0

把这条数据流画成图,谁读谁写就一目了然了:

结构图

连接关系:

- S 分数 tile / TMEM $\xrightarrow{\text{读入寄存器}}$ 寄存器: 算 softmax
- 寄存器: 算 softmax $\xrightarrow{\text{exp2} + \text{重缩放}}$ P 分子 tile / TMEM
- P 分子 tile / TMEM $\xrightarrow{\text{value MMA}}$ O 累加器 tile / TMEM
- O 累加器 tile / TMEM $\xrightarrow{\text{最后一块后}}$ 写出 O / row_sum

注意

注意:前面说的「重缩放」也是一次 tile 操作,可不是几句标量记账那么轻巧。`row_max` 一变,旧的 `O` 就得从 TMEM 读出来、在寄存器里乘上 `scale`、再写回 TMEM,然后下一次 value MMA 才能往里累加。

这套结构会一直贯穿后面每一节:一处 tile 摆放 + 一条硬件通路 + 一个用来宣告「下一个消费者可以开跑了」的 barrier。把这三件套吃透,整章实现的骨架你也就抓住了。

Tile-原语图 (Tile-Primitive Graph)

上一节我们已经定下了三个运行状态 (S、P、O) 各自「住」在哪儿:S 在 TMEM,P 经寄存器在 TMEM 来回跑,O 累加在 TMEM。有了这张「数据住哪」的地图,这一节就把整个算法落成一条具体的、能跑的 tile 搬运路径——也就是所谓的「Tile-原语图」。

tile-原语图说白了就是一张「生产者—消费者」依赖图。图里每个节点,就是某块数据待在某一级内存里的那个样子(比如「SMEM 里的 Q」);每条边,就是一次操作——要么把数据从这儿搬到那儿,要么就地算出点新东西。这张图一画清楚,后面所有关于 layout、barrier、流水线的活儿,本质上就剩一件事了:给图上每条边补上对的同步。

一条 K/V block 的短路径

对一个 K/V block 来说,kernel 从上往下走的这条搬运路径,几行就能概括:

步骤	数据落点	靠什么完成
起点	GMEM 中的 Q、K、V	——
1	SMEM 中的 Q、K、V	TMA load 搬入
2	TMEM 中的 S	score MMA: 计算 QK^T
3	TMEM 中的 P	softmax 分子:TMEM $\rightarrow$ 寄存器 $\rightarrow$ TMEM
4	TMEM 中的 O	value MMA: 计算 $P \cdot V$
5	GMEM 中的 O	归一化、SMEM 暂存、TMA store 写回

画成有向图更直观,每条边都标了「靠哪条硬件路径完成」:

结构图

连接关系:

- $Q/K/V @ \text{SMEM} \xrightarrow{\text{score MMA } (QK^T)} S @ \text{TMEM}$
- $S @ \text{TMEM} \xrightarrow{\text{softmax: TMEM} \rightarrow \text{寄存器} \rightarrow \text{TMEM}} P @ \text{TMEM (fp16)}$
- $P @ \text{TMEM (fp16)} \xrightarrow{\text{value MMA } (P \cdot V)} O @ \text{TMEM}$
- $O @ \text{TMEM} \xrightarrow{\text{归一化} + \text{SMEM 暂存} + \text{TMA store}} O @ \text{GMEM}$

注意

注意: 这里的 P 不是最终归一化好的注意力权重,只是当前这块 K/V 的 softmax 「分子」。真正的归一化(除以 row_sum)被特意拖到最后一块处理完才做,所以图里只有 $O @ \text{TMEM} \rightarrow O @ \text{GMEM}$ 这条边上才冒出「归一化」这几个字。

和 GEMM 的唯一本质区别

这张图跟 GEMM 的 tile-原语图比,差别其实就「一行」:

- **GEMM:** 就是一条 MMA 链反复累加—— $A \cdot B$ 一路 MMA 算下去,中间没有任何非 MMA 的计算插进来。
- **FA4:** 有两段 MMA(score MMA 和 value MMA),而 softmax 恰好卡在这条链中间。

你可以这么理解: 把 GEMM 的 MMA 链从中间「劈」一刀, 塞进一个 softmax 阶段。后面几乎所有比 GEMM 复杂的地方——多出来的 TMEM↔ 寄存器流量、多出来的角色分工、多出来的 barrier——全是这「多出来的一段」连锁反应出来的后果。先把这一点攥住, 后面再乱的复杂度也有了主心骨。

展开成完整的生产者—消费者图

把上面那条短路径每一步都摊开, 标清「用哪个原语 + 走哪条硬件路径」, 就得到完整的依赖图。下表把 TIRx 原语 (Tx.*) 一一对到底层硬件指令上:

阶段	tile 搬运 / 计算	TIRx 原语	硬件路径
加载 Q/K/V	GMEM tile -> SMEM tile	Tx.copy_async(..., dispatch="tma")	TMA load
Score MMA	SMEM 中的 Q、K -> TMEM 中的 score tile S	Tx.warp.gemm_async(..., dispatch="tcgen05")	tcgen05.mma
Softmax 读	TMEM 中的 S -> warp-group 寄存器 tile	Tx.wg.copy_async(reg, tmem)	tcgen05.ld
Softmax 写	寄存器中的分子 tile P -> fp16 TMEM 视图	Tx.copy_async(tmem_as_f16, reg)	TMEM store, 随后 tcgen05.wait.st()
Value MMA	TMEM 中的 P、SMEM 中的 V -> TMEM 中的输出累加器 0	Tx.warp.gemm_async(..., dispatch="tcgen05")	tcgen05.mma(带 一个 TMEM 操作数)
Correction(校正)	TMEM 中的 0 -> 寄存器 -> TMEM 中的 0	TMEM 读回、寄存器乘法、TMEM store	tcgen05.ld / TMEM store
Epilogue(收尾)	最终的 0(TMEM)-> 寄存器 -> SMEM -> GMEM	TMEM 读回、Tx.copy、TMA store	tcgen05.ld + TMA store

几个细节值得单拎出来说:

- **Softmax** 写回为什么非得来一句 `tcgen05.wait.st()`? 因为把寄存器里的 P 写回 TMEM(tcgen05 风格的 TMEM store) 是异步的, 可后面紧跟的 value MMA 又要把这块 P 当输入读。那怎么办? 必须先 `wait`, 确认 store 真落地了, 才能放心让 MMA 去消费——这是图里一条藏着的、但很关键的依赖边。
- **Value MMA** 带一个「TMEM 操作数 (operand, 即送进 MMA 的输入矩阵)」。普通 GEMM 的 MMA 两个输入都在 SMEM; 可这里 P 在 TMEM、V 在 SMEM,MMA 直接吃一个 TMEM 操作数, 省掉了把 P 再搬回 SMEM 那一趟。
- **Correction** 这一行是「新冒出来的」。row_max 一往上跳, 之前算的 0 就还停在旧 scale 上, 必须先把 TMEM 里的 0 读到寄存器、乘上校正系数、再写回 TMEM, 然后下一次 value MMA 才能接着往里累加。这正对应上一节强调的那句:rescale 是一次 tile 操作, 不是标量记账。

哪些边是 GEMM 里没有的

把这张图跟 GEMM 的链摆一块儿对比, 多出来的就是 softmax 和 correction 这两行。它俩有个共同毛病:

1. 都得来回跑 **TMEM -> 寄存器 -> TMEM**(GEMM 的 MMA 链里, 数据基本不用绕回寄存器再算)。
2. 都在 **score MMA 和 value MMA 之间硬加了一道交接 (handoff)**——也就是说, 两段 MMA 没法背靠背连上, 中间隔着「先读出来、在寄存器里算、再写回去」这一套。

FA4 为什么要更细的 warp 角色分工、更绕的 barrier? 根子就在这两条「绕回寄存器」的边上。数据要在不同执行单元之间来回交接, 那就得有人生产、有人消费, 中间还得有个同步原语来喊一嗓子: 「好了, 下一个消费者可以开跑了。」

注意

注意: 读这张图, 有个特别好用的练习——只盯那条短路径, 对每一个箭头挨个回答四件事: **生产者是哪个阶段、消费者是哪个阶段、源 tile 是谁、目标 tile 是谁、走的哪条硬件路径**。一个箭头一个箭头过完, 再问自己一句: 哪些箭头在 GEMM 那章里**压根不存在**? 答案就落在 softmax 读/写、还有 correction 这几条 TMEM↔ 寄存器的边上。

Warp 的角色与作用域 (Warp Roles and Scopes)

上一小节把「数据怎么流」讲完了——Q、K、V 怎么在 GMEM、SMEM、TMEM 之间倒腾,score MMA 和 value MMA 各在哪一步发生。数据通路定了之后, 下一个该问的问题自然就是: **这每一个阶段, 到底谁来干?**

答案是: 把一个 CTA 里的线程按「角色」分, 而不是按「数据」分。

核心思路: 按「做什么活」分工, 而不是按「碰哪块数据」分工

这里每个 CTA(线程块, 一组一起调度、能共享 SMEM 的线程) 有 4 个 warpgroup(warp 组,4 个 warp = 128 个线程), 总共 512 个线程 (4 × 128)。Flash Attention 4 的分工原则很要紧:

注意

注意: 线程是按「这个 warpgroup 负责哪一类活」来分的, 不是按「它碰哪块数据」来分。这就是看懂整个 kernel 结构的入口。

整体先记住一句话:

- **WG3** 是「开机器」的那个 warpgroup——TMA 加载、MMA 计算、TMA 存储这三类硬件引擎指令, 都归它来发。
- **WG0、WG1、WG2** 是「埋头算寄存器数学」的 warpgroup——夹在两次引擎调用中间的那些运算:softmax、correction(校正/重缩放)、epilogue(收尾), 都归它们干。

换句话说,WG3 更像一个「调度台」, 只管按按钮启动那些专用硬件单元 (TMA 引擎、Tensor Core);WG0~WG2 才是真正趴在通用 CUDA core 上做寄存器运算的「计算工」。

完整角色分配表

归属 (Owner)	角色 (Role)	具体做什么
WG3, warp 1	TMA load	把 Q、K、V 的 tile 从 GMEM 搬到 SMEM
WG3, warp 0	MMA	发起 score MMA 和 value MMA 两类矩阵乘
WG3, warp 2	TMA store	把最终的 O tile 从 SMEM 写回 GMEM
WG0	Q stage 0 的 softmax	从 TMEM 读 S, 算出 P, 把 P 写回 TMEM

WG1	Q stage 1 的 softmax	对第二个 Q 流水线阶段做同样的工作
WG2	correction 与 epilogue	在 TMEM 中重缩放 O、归一化、整理输出

能看出来,WG3 内部还按 warp(线程束,32 个线程的最小调度单位) 又细分了一层:warp 0 专盯 MMA,warp 1 专盯 load,warp 2 专盯 store。三类硬件引擎,一类配一个 warp 守着,谁也不碍着谁。

关键澄清:「两个 Q stage」不是两个 attention head

表里 WG0 和 WG1 干的是「同一种活」(softmax),只不过各管 Q 流水线里的一个槽位。这地方特别容易看岔:

注意

注意: 千万别把「两个 Q stage」读成「两个 attention head」。它俩其实是 **Q 流水线里的两个槽位 (slot)**——WG0 占一个,WG1 占一个,这样两个 Q tile 就能同时「在飞」(in flight),凑成流水并行。softmax 之所以写了两份,就是因为有这俩流水槽,跟「有两个头」一点关系没有。

画张图,把这种「同种活、两个流水槽」的关系摆出来:

结构图

分组:

- **Q 流水线 (两个并发槽位)**
- **Q stage 0:** 「WG0: softmax」
- **Q stage 1:** 「WG1: softmax」

连接关系:

- Q 流水线 (两个并发槽位) → 两个 Q tile 可同时在飞 → 隐藏延迟、提升吞吐

代码里如何「认领」角色

kernel 靠一组符号化坐标 (symbolic coordinates) 来认线程: 当前这个线程属于哪个 warpgroup, 又是 warpgroup 里的第几个 warp:

```
wg_id = T.warpgroup_id([4])      #          warpgroup(0-3)
warp_id = T.warp_id_in_wg([4])   #          warpgroup          warp
```

注意

注意: 读这份 kernel 有个实用窍门——先去找「角色分支」(role branch),也就是那些基于 wg_id / warp_id 的 if 分支。一旦你锁定了某个角色分支,嵌在它里头的所有 tile 级原语 (load、MMA、softmax 等) 归哪支「队伍」管,就一目了然了。这是快速理清控制流的最佳切入点。

各个角色分支里头到底在干什么:

- **WG3 warp 1:** 发 TMA load 命令。先由一个被选中的 lane(elected lane, lane 即 warp 内的线程编号 0~31) 发出拷贝指令; 接着 TMA 引擎真正去搬 tile。
- **WG3 warp 0:** 发 `tcgen05.mma` 指令 (也就是在 Tensor Core 上做矩阵乘)。
- **WG0 和 WG1:** 在整个 **warpgroup 作用域 (full warpgroup scope)** 下跑 softmax——也就是 128 个线程一整组一起协同完成。
- **WG2:** 同样是整组 warpgroup 一起跑 correction 和 epilogue。

一个关键的「不对称」, 决定了整张 barrier 图

整个 kernel 的同步结构 (barrier graph), 被一个核心的「不对称」给定了形:

注意

注意: 所有 MMA——不管是 score MMA 还是 value MMA——都只从 **WG3 warp 0** 这一个地方发出来。WG0 和 WG1 自己从来不发 MMA; 它们只负责「消费」已经算好的 score tile, 跑 softmax, 再把结果 P 写回 TMEM。

为什么 softmax 非得拿 barrier 包起来? 根子就在这儿: 发 MMA 的 (WG3 warp 0) 和消费 score、跑 softmax 的 (WG0/WG1) 压根不是同一拨人。两边各干各的, 中间就必须有个东西帮它们对上节奏。这条依赖链画成图是这样的:

结构图

连接关系:

- WG0 / WG1 / 跑 softmax / 生成 P $\xrightarrow{\text{p_o_rescale 屏障 / 传递 P + 安全的 O 槽位}}$ WG3 warp 0 / 发起 value MMA

这里冒出来两个会贯穿整章的 mbarrier 名字, 务必记牢:

屏障名	作用
s_ready	把 score tile 从 MMA warp(WG3 warp 0) 传递给 softmax(WG0/WG1), 表示「分数算好了, 可以做 softmax 了」
p_o_rescale	把 P 以及一个对 value MMA 安全的 O 槽位传递回去——这个 O 槽位要么已经完成重缩放, 要么因为不需要重缩放而被释放

注意

注意: `s_ready` 和 `p_o_rescale` 这两名字, 本章后面会反复出现。它们就是把「MMA 发起方」和「softmax 消费方」这条跨 warpgroup 依赖链拴在一起的两根绳——把它俩弄明白了, FA4 同步逻辑的骨架你就拿下了。

怎么读后面的代码片段 (Reading the Fragments)

本章后面要一段段拆的代码, 全是从 `flash_attention4.py` 这个真实 kernel 里抠出来的片段 (fragment)。既然是节选, 就难免会用到一些「定义在别处、本章又没整段贴出来」的名字。第一次撞上一

个不认识的变量，人很容易就卡住了——所以这一小节的用处，就是先把这些名字**一次性理一遍**，给你一张随时能翻回来查的「术语速查表」。

怎么用？很简单：**现在不用背**。把这节当字典摆着，后面哪段代码蹦出个陌生名字，翻回来瞄一眼它是啥就行。

哪些名字「望文生义」，会在用到时就地解释

有一类名字本身就挺「望文生义」的，它们会在**第一次真正派上用场的地方**就地登场、就地讲清楚，所以这里不细说，先让你混个眼熟：

- `wg_id`、`warp_id`:warpgroup 编号、warpgroup 内的 warp 编号（上一节已经见过）。
- `BLK_M` / `BLK_N`:tile 在 `M` / `N` 方向上的分块大小。
- `HEAD_DIM`: 注意力头维度 $\backslash(d\backslash)$ 。
- `kv_stage`:K/V 流水线的阶段 (stage) 索引。
- `SMEM_PIPE_DEPTH_*` / `TMEM_PIPE_DEPTH`: 各条流水线在 `SMEM` / `TMEM` 上的**深度**（同时能有几个 tile 在路上）。
- `should_accumulate`:value MMA 这一步是「累加进旧 0」还是「初始化 0」的标志。
- `CTA_GROUP`: 一个 MMA 由几个 CTA 协作完成（本章固定为 1）。

这些等到对应片段出现时再讲，反而更顺，这里就先不抢戏了。

剩下这些名字：用到前先建个索引

真正得先打个底的，是下面这批——它们既不那么望文生义，又会反复出现。下表用中文把它们的意思重新捋了一遍，后面看代码随时翻回来查：

名字	含义
<code>q_stage</code> 、 <code>i_q</code>	<code>Q</code> 流水线的阶段号，取值 0 或 1，表示用的是哪个 <code>Q</code> tile 槽位 (<code>SMEM_PIPE_DEPTH_Q</code> = 2)。关键巧合：在 <code>WG0</code> / <code>WG1</code> 的 softmax 里,warpgroup 自己的 <code>wg_id</code> (0 或 1) 就正好等于这个阶段号，所以 <code>S_region[q_stage]</code> 、 <code>P_region[wg_id]</code> 、 <code>O_region[i_q]</code> 选中的是同一个 <code>Q</code> 阶段
<code>MMA_N</code>	<code>score</code> / <code>output</code> tile 的宽度，以 <code>TMEM</code> 列数计，这里是 128
<code>MMA_K</code>	value MMA 在内层 <code>K</code> 维（也就是 <code>P</code> / <code>V</code> 的列方向）上每步前进的宽度,16 列
<code>K_SPLIT</code>	value MMA 调度的切分点，等于 <code>6 * MMA_K</code> = 96(详见《两个 MMA 阶段》一节)。第一段 value MMA 负责列区间 <code>0:K_SPLIT</code> ，即 <code>0:96</code>
<code>should_rescale</code>	<code>WG2</code> 用的 逐行 (per-row) 标志 ：在下一 <code>次</code> value MMA 之前，旧的 0 是否需要先重缩放 (rescale)。它在整个 warp-group 内用 <code>any_sync</code> 归约成一个统一结论
<code>rescale_threshold</code>	「行最大值变化太小就跳过 <code>rescale</code> 」的阈值，本版 kernel 取 8.0；一旦判定跳过, <code>acc_scale</code> 会被直接设成 1.0(即「不缩放」)
<code>scale_log2</code>	以 <code>log2</code> 为底表达的 softmax 缩放系数, <code>log2(e)/√d</code> ，于是 <code>P = exp2((S - m) · scale_log2)</code>
<code>acc_scale</code>	softmax 算出来、要传给 <code>WG2</code> 的 逐行重缩放因子
<code>chunk_start</code> / <code>chunk_end</code> 、 <code>p_start</code> / <code>p_end</code>	当前正在 读取 / 写入 的那一段 32 宽 softmax chunk 的列区间

几个值得提前点透的设计意图

光盯着表格，容易只记住「等号右边等于几」；可有几条背后的「为什么」，一旦你看穿了，后面读代码会顺很多。

① 三个数组，共用同一个阶段号——这是故意对齐的。`q_stage`、`wg_id`、`i_q` 在 `softmax` 这条路上其实就是同一个 0/1，只不过分别出现在不同数组的下标位置上。想通这点有什么好处？以后你看到 `S_region[q_stage]`、`P_region[wg_id]`、`O_region[i_q]`，就不用担心它们指向三套八竿子打不着的东西——它们说的是同一个 Q 流水线槽位里的分数、概率、输出。用下面这张图把这个「三合一」记牢：

Q 流水线两个槽位 (`SMEM_PIPE_DEPTH_Q = 2`)，同一列里的 S / P / O 属于同一个 Q tile，且 `q_stage == wg_id == i_q`：

维度	槽位 0	槽位 1
阶段号	0	1
谁负责	WG0 softmax	WG1 softmax
分数 S	<code>S_region[0]</code>	<code>S_region[1]</code>
概率 P	<code>P_region[0]</code>	<code>P_region[1]</code>
输出 O	<code>O_region[0]</code>	<code>O_region[1]</code>

注意

注意：别把「两个 Q 阶段」看成「两个注意力头 (attention head)」。它们就是 Q 流水线里的两个槽位，目的是让两个 Q tile 同时在路上 (in flight)，WG0、WG1 各管一个。softmax 代码「写了两遍」，纯粹是因为有这俩并行的槽位。

② `MMA_N / MMA_K / K_SPLIT`：列方向上的尺寸语言。这三个名字都是在说 TMEM 列方向上的「步长和边界」：`MMA_N = 128` 是一个 tile 多宽，`MMA_K = 16` 是 value MMA 每步往前啃几列，而 `K_SPLIT = 6 * MMA_K = 96` 把整条 value MMA 切成了两段。为什么要切？这是后面《两个 MMA 阶段》那一节的重头戏——眼下先记住「第一段管 0:96、剩下的归第二段」这条边界就够了。

③ `scale_log2`：为什么用 `log2`，不用自然底数。softmax 里要算 $\exp(S - m)$ ，可硬件上 `exp2` (以 2 为底) 比 `exp` (以 e 为底) 省事。换底公式给出 $\exp(x) = \exp_2(x \cdot \log_2(e))$ ，于是把 $1/\sqrt{d}$ 的缩放和 $\log_2(e)$ 合成一个常数 `scale_log2 =  $\log_2(e)/\sqrt{d}$` ，最后 $P = \exp_2((S - m) \cdot \text{scale_log2})$ 。一句话：用 `exp2` 是为了贴合硬件指令，`scale_log2` 就是「把所有缩放揉进一个系数」的产物。

④ `should_rescale / rescale_threshold / acc_scale:rescale` 这条主线的三件套。还记得导读里那个 twist 吗——running max 一变，旧 0 就处在错误尺度上，得重缩放。这三个名字正是这件事的实现细节：

- `should_rescale`：要不要缩放 (先逐行判断，再用 `any_sync` 在 warpgroup 内归约成一个统一动作)；
- `rescale_threshold = 8.0`：行最大值只动了一丁点 (小于阈值) 就跳过，跳过时 `acc_scale` 直接取 1.0；
- `acc_scale`：真要缩放时缩放多少，由 softmax 算出来，再通过 SMEM 邮箱 (mailbox) 递给 WG2 去执行。

串起来读就是：softmax(WG0/WG1) 负责拍板缩放因子，WG2 负责动手缩放——这也正好呼应上一节那句「softmax 跟 correction/epilogue 分属不同 warpgroup」的分工。

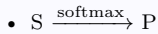
把这张表和这四点记住，后面再生的片段也都有处可查。

两个 MMA 阶段: 被 softmax 串起来的双矩阵乘 (The Two MMA Phases)

Flash Attention 的内层循环里, 装的不是一个矩阵乘, 而是两个 MMA, 中间拿一段 softmax 串着。每来一个 K/V tile,kernel 都得把这三步挨个跑一遍:

结构图

连接关系:



你可以把它想成三个工人排成一条流水线: 第一个 MMA 产出分数 S,softmax 把 S 加工成分子 P, 第二个 MMA 拿 P 去更新输出累加器 0。这里有个关键设计: 除以 row_sum 那步归一化, 被故意拖到了最后的收尾阶段 (epilogue)。非得等所有 K/V tile 都处理完、0 攒齐了, 才在最后统一除这一次。这恰恰是在线 softmax 的精髓: 边累加边维护统计量, 最后一锤子归一化, 中途绝不反复除。

怎么读懂每个 tile 算子: 四问 + 一个 Handoff

前面讲 GEMM 那几章, 我们用”作用域 / 布局 / 调度 (scope / layout / dispatch)” 这张卡片来逐个审视 tile 算子。这里照搬同一套问法, 只是再添一行 **Handoff(交接)**, 专门点出”是哪个 (些) mbarrier 把这块 tile 递给下一个角色”。也就是对每个算子问四个问题:

问题	含义
Scope(作用域)	谁来执行它 (哪个 warpgroup / warp / lane)
Layout(布局)	输入输出 tile 各自住在哪里 (SMEM / TMEM / 寄存器)
Dispatch(调度)	用什么指令路径发射 (如 tcgen05)
Handoff(交接)	用哪个 mbarrier 把成果交给下一棒

注意

注意: 计算代码从来不直接写裸的 TMEM 列号。kernel 把它**唯一那一块 TMEM 分配**切成几个”按阶段命名的视图”——S_region、P_region、O_region, 然后拿流水线阶段去索引, 比如 S_region[q_stage]、O_region[i_q]、P_region[i_q, 0:K_SPLIT]。这些视图是 T.TMEMStages 在《TMEM 布局与复用》那节里定义的。现在你只要把每个 region 当成”同一块物理 TMEM 上的一个有名字的切片”就行。

第一阶段: 分数 MMA(Score MMA)

每一轮 K/V 迭代, 开场就是分数 MMA, 它算的是:

$$S = Q_{\text{block}} K_{\text{block}}^\top$$

然后把 128 x 128 的分数 tile 写进 TMEM。核心 API 这么用:

```
Tx.warp.gemm_async(  
    S_region[q_stage],           # :S      TMEM  
    Q_smem[q_stage, 0:BLK_M, 0:HEAD_DIM], # A:Q,    SMEM  
    K_smem[kv_stage, 0:BLK_N, 0:HEAD_DIM], # B:K,    SMEM  
    dispatch="tcgen05",         # tcgen05( 5  Tensor Core)
```

```

    cta_group=CTA_GROUP,
)
if T.ptx.select_sync():
    # warp lane
    s_ready.arrive(q_stage) # lane s_ready arrive,

```

按四问拆解:

- **Scope:** 由 WG3(warpgroup 3) 的 warp 0 发射; 再由一个被选中的 lane(elected lane) 在 s_ready 上 arrive。
- **Layout:** Q、K 都在 SMEM, 结果 S 落到 TMEM(S_region[q_stage])。
- **Dispatch:** tcgen05。
- **Handoff:** s_ready(交给 softmax)。

注意

注意: 整个交接, 就是”被选中的那一个线程在 s_ready 上 arrive”这么一个动作。它对外喊一声: 这块分数 tile 算完了, softmax warpgroup 可以放心去读了。为什么用单个 lane arrive, 而不是整个 warp? 就是怕整 warp 一起 arrive 把 barrier 的计数搞乱。

中间阶段: 夹在两个 MMA 之间的 softmax(Softmax Between MMAs)

两个 MMA 中间夹的就是 softmax, 它把分数 tile S 加工成分子 tile P。这是整条流水线里唯一一个在 GEMM 里完全找不到对应物的阶段——它压根不是矩阵乘, 而是逐行 (row-wise) 的标量运算。它的卡片是:

- **Scope:** WG0(对应 Q stage 0)/ WG1(对应 Q stage 1), 整个 warpgroup 都参与。
- **Layout:** S 在 TMEM → 读进寄存器 (registers) → 算出的 P 以 fp16 写回 TMEM(P_region[wg_id])。
- **Dispatch:** 用 tcgen05.ld 读出, 用 TMEM store 写回; 中间在寄存器里做逐行数学。
- **Handoff:** 等待 s_ready; 完成后在 p_o_rescale(前 96 列) 和 p_ready_2(后 32 列) 上 arrive。

WG0/WG1 先等 s_ready, 确认分数 tile 备好了, 然后一次一个寄存器大小的块 (register-sized chunk) 地把分数从 TMEM 读进寄存器:

```

Tx.copy_async(
    s_chunk[:, chunk_start : chunk_end], # :
    S_region[wg_id, chunk_start : chunk_end], # :TMEM tile
) # warpgroup TMEM->

```

分数进了寄存器, softmax warpgroup 就严格按顺序干三件事:

1. 算出每行的行最大值 (row max) 和行和 (row sum);
2. 算出 softmax 分子 tile P;
3. 把 P 以 fp16 写回 TMEM。

第 3 步的写回长这样:

```

Tx.copy_async(
    P_region[wg_id, p_start : p_end], # :TMEM fp16 P
    p_chunk[:, p_start : p_end], # : P
)

```

这里很自然会冒出一个疑问: **P 我们刚在寄存器里算出来了, 干嘛还费劲写回 TMEM?**

答案是: 下游的取值 MMA 要把 P 当一块**矩阵操作数**来读, 而 MMA 指令读不了散落在各个线程里的标量寄存器——它只认共享、规整、能按矩阵布局寻址的内存。在这个 kernel 里,P 唯一能被 MMA 读进去的形态, 就是 **P_region**(它是搭在 fp16 TMEM 别名 **tmem_as_f16** 上的一个视图)。所以这趟写回**不是白搬**, 而是把 P 摆成「下一个 MMA 唯一认得的形状」, 不做不行。

第二阶段: 取值 MMA(Value MMA)

每一轮 K/V 迭代收尾的, 是取值 MMA, 它算的是:

$$O = O + P_{\text{block}} V_{\text{block}}$$

走到这一步,O 已经被调到当前 K/V 块该有的正确状态了——第一块时被初始化, 后面每块时被重缩放 (rescale) 过——所以这个 MMA 剩下要干的就只有”累加”。它跟普通 GEMM 最大的区别, 在于**操作数都住在哪**:A 操作数 P 在 TMEM,B 操作数 V 在 SMEM, 累加器 O 也在 TMEM。

```
#           MMA:      P      96      (0:K_SPLIT),      V      96
Tx.warp.gemm_async(
    O_region[i_q],                #           :O      TMEM
    P_region[i_q, 0:K_SPLIT],      # A           :P,      TMEM
    V_smem[kv_stage, 0:K_SPLIT, 0:HEAD_DIM], # B           :V,      SMEM
    transB=True,
    accum=should_accumulate,      #           "      "      "      "
    dispatch="tcgen05",
    cta_group=CTA_GROUP,
)
#           MMA           (accum=True,   p_ready_2   ),
#           K_SPLIT:BLK_N
```

卡片:

- **Scope**:WG3 的 warp 0。
- **Layout**:P 在 TMEM + V 在 SMEM → O 在 TMEM(**O_region[i_q]**)。
- **Dispatch**:tcgen05, 且带一个 TMEM 操作数。
- **Handoff**: 等待 p_o_rescale、p_ready_2、kv_load.full; 完成后在 o_ready 上 arrive(交给 epilogue)。

两个 MMA 在硬件上的根本区别: 操作数位置

项目	分数 MMA(Score MMA)	取值 MMA(Value MMA)
A 操作数	Q, 来自 SMEM	P, 来自 TMEM
B 操作数	K, 来自 SMEM	V, 来自 SMEM
累加器/输出	S 写入 TMEM	O 累加进 TMEM
特点	两个操作数都从 SMEM 读	一个操作数 (P) 从 TMEM 读

看得出来: 分数 MMA 两个输入都从 SMEM 来; 而取值 MMA 把 P 这一路输入换成了从 TMEM 直接喂给 Tensor Core——这正是上一步非把 P 写回 TMEM 不可的根本原因。

注意

注意: `accum=should_accumulate` 这个标志, 就是算法里” 初始化还是累加” 那个开关的实现: 一个 query 块的第一个 K/V tile 上它是 **False**(直接写, 等于初始化), 后面每个 tile 上都是 **True**(往上加)。

为什么把取值 MMA 拆成 96 + 32 两刀?

你会发现取值 MMA 不是一发打完的, 而是按 96 + 32 拆成两次发。流程是这样:

结构图

连接关系:

- softmax 写 P, 分成 4 个 32 列 chunk: chunk0 | chunk1 | chunk2 | chunk3 → 前 96 列 (chunk0~2)
- softmax 写 P, 分成 4 个 32 列 chunk: chunk0 | chunk1 | chunk2 | chunk3 → 后 32 列 (chunk3)
- 前 96 列 (chunk0~2) → 前三块就绪即可发射 / 第一个子 MMA (96 宽)
- 后 32 列 (chunk3) → 等 `p_ready_2` 再发射 / 第二个子 MMA (32 宽)
- 前三块就绪即可发射 / 第一个子 MMA (96 宽) → 两者在时间上重叠: / chunk3 的 `exp` + TMEM 写回与 96 宽 MMA 同时进行
- 等 `p_ready_2` 再发射 / 第二个子 MMA (32 宽) → 两者在时间上重叠: / chunk3 的 `exp` + TMEM 写回与 96 宽 MMA 同时进行

拆分的逻辑是这样:

1. softmax 把 P 分成 4 个 32 列的 chunk 来写;
2. 前 3 个 chunk(共 96 列) 一备好, 取值 MMA 就立刻对着 P 的前 96 列和 V 的对应行开跑;
3. 最后 32 列得等 `p_ready_2`;
4. 第二个 MMA 把这最后一块吃掉, 整个 tile 收尾。

拆开的根本目的, 就是别让 Tensor Core 闲着。假如把取值 MMA 当一条整指令来发, 那整个阶段就得干等着, 等到全部 4 个 32 列 chunk 都求完指数 (`exp`)、都存好为止。而”前三块一就绪就先开打”这么一来, kernel 就把最后一块 chunk 的 `exp` 计算和 TMEM 写回, 跟一条已经在飞的 96 宽 MMA 叠到一块儿了——本来要白白浪费的空闲时间, 这下变成了有用的计算。这是典型的”用细粒度流水化 (pipelining) 换吞吐”的手法。

TMEM 的布局与复用 (TMEM Layout and Reuse)

这一小节回答一个看着不起眼、实际牵动全局的问题:**S、P、O** 这三块中间结果, 到底搁哪儿? 答案有点出人意料——它们全被塞进**同一块** 128×512 的 TMEM 里。正因为挤在一起, 在这个 kernel 里 `barrier` 和内存布局就成了拆不开的一对: 布局把复用逼出来, 而复用要想合法, 就只能靠 `barrier` 盯着。

注意

注意: 这里的 TMEM 是 Blackwell 架构带来的、专给 tcgen05 MMA 用的张量内存。它的容量小得可怜 (每个 CTA 只有 128×512 的 fp32 空间), 所以”省着用”在这儿不是什么优化技巧, 而是硬性约束。

三类 tile slot 共享同一块分配

你可以把这块 TMEM 想成一排 tile 槽位 (tile slot), 每一类数据占其中一段:

槽位类型	存放内容	数据含义
Score slots(分数槽)	$S = QK^T$	注意力打分矩阵
Numerator slots(分子槽)	softmax 指数化之后的 P tile	即 $\exp(S - \text{rowmax})$
Output slots(输出槽)	fp32 的 0 累加器	最终输出的累加结果

关键在于: 这三类槽位不是各自独立的缓冲区, 而是同一块物理分配上的不同区域。这么共享不是为了代码好看, 纯粹是被容量逼的。

算笔账你就明白”被逼”这俩字一点不夸张。设 Q 方向的流水线深度 (Q-pipeline depth) 是 2: TMEM 总容量:128 行 $\times$ 512 列 (fp32)。逐项算账:

占用项	列数计算	列数
两个 S 槽	$2 \times \text{MMA_N} = 2 \times 128$	256
两个 O 槽	$2 \times \text{MMA_N} = 2 \times 128$	256
合计已占用		512(正好占满!)
留给 P 的空间	512 512	0

也就是说, 光 S 和 O(各两个流水线 stage) 就把 512 个 fp32 列全吃光了, 一列不剩。P 连个落脚的地方都没有。

P 如何”无中生有”:fp16 视图的别名 (aliasing) 技巧

fp32 这个视角下既然腾不出地方,kernel 就换个视角看同一片字节: 用 fp16 视图把这块物理内存重新盖一遍。道理很简单——fp16 元素只有 fp32 的一半宽, 同样多的字节, 用 fp16 来数就能数出两倍的列。P 就住进这些「凭空冒出来」的列里。

下面这段代码就是别名技巧的核心: 先拿一个 fp32 视图, 然后把内存池的基址倒回 0, 再在一模一样的那片物理字节上拿一个 fp16 视图:

```
tmem_pool = T.TMEMPool(pool, total_cols=N_COLS_TMEM, cta_group=CTA_GROUP, tmem_addr=tmem_addr)
tmem = tmem_pool.alloc((128, N_COLS_TMEM), "float32") # fp32 : S 0
tmem_pool.move_base_to(0) # : 0,
tmem_as_f16 = tmem_pool.alloc((128, N_COLS_TMEM * 2), "float16") # fp16 : , P
tmem_pool.commit()
```

move_base_to(0) 是整个技巧的命门: 它让第二次 alloc 从同一个起点开始分, 于是 tmem(fp32) 和 tmem_as_f16(fp16) 指的就是同一块物理内存, 只不过用两种宽度去解读。注意 fp16 视图的列数是 $N_COLS_TMEM \times 2$, 这正好对应”元素半宽 $\rightarrow$ 列数翻倍”的换算。

注意

注意: 这套别名能安全跑, 唯一的前提是**时序**——每块区域都得严格等它”上一任消费者”用完了, 才被重新启用。而保证这个时序的, 恰恰是 **barrier**。所以在 FA4 里, barrier 不只是个调度工具, 它从根上让这套布局**合法**。下一小节会专门拿一张 **barrier** 表来捋”每块区域复用之前, 得先等谁干完”。

用 T.TMEMStages 把字节切成按 stage 索引的区域

有了这两个视图, 内核就用 **T.TMEMStages** 把 S、P、O 各自切成”分阶段 (staged)”的区域。好处是: 计算代码可以**按流水线 stage 去索引**, 根本不用碰原始列号。

```
# S    fp32    ,      0
S_region = T.TMEMStages(tmem,      col_start=0,      width=MMA_N,
↳ stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N)
# O    fp32    ,      S    (MMA_N *    )
O_region = T.TMEMStages(tmem,      col_start=MMA_N * SMEM_PIPE_DEPTH_Q, width=MMA_N,
↳ stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N)
# P    fp16    ,stride  x2
P_region = T.TMEMStages(tmem_as_f16, col_start=MMA_N,      width=BLK_N,
↳ stages=SMEM_PIPE_DEPTH_Q, stride=MMA_N * 2)
```

这里唯一让别名”露馅”的地方, 就是 **P_region** 那个 **stride=MMA_N * 2** 里的 * 2:

- **S_region** 和 **O_region** 是按 **fp32** 的 **tmem** 列来量的;
- **P_region** 是按 **fp16** 的 **tmem_as_f16** 列来量的, 而 fp16 列只有 fp32 列的一半宽;
- 所以, P 在 stage 之间挪动时, 要想落到跟 **fp32** 布局对齐的同一批物理字节上, 跨 stage 的步长就得**翻倍**。

拿一张图, 就能直观看出三块区域在同一片字节上是怎么对应的 (以流水线深度 2 为例):

物理 TMEM 是 128 行 × 512 fp32 列; fp16 视图把同一片字节按半宽元素重新数, 列数翻倍 (0 ~ 1023)。两个视图在同一片字节上的对应关系如下:

字节区间 (按 fp32 列)	0-128	128-256	256-384	384-512
fp32 视图	S 槽 0	S 槽 1	O 槽 0	O 槽 1
fp16 视图 (列号 ×2)	fp16 0-256	fp16 256-512	fp16 512-768	fp16 768-1024
P_region		P 槽 0	P 槽 1	

P_region 的 **col_start = MMA_N = 128**(fp16 列), 步长 = **MMA_N*2 = 256**(fp16 列); 跨一个 P 槽正好覆盖一个 fp32 槽的字节宽度。

注意

注意: **P_region** 的 **col_start=MMA_N** 和 **stride=MMA_N*2** 都是按 **fp16** 列量的 (fp16 列只有 fp32 列一半宽)。所以 P 的字节不是简单”接在 O 后头”, 而是借用了 fp32 视图里某些槽位对应的同一片字节——P 跟 S/O 能合法共存, 全靠 **barrier** 保证的时序, 而不是靠地址互不重叠。

可以这么理解:P 借住在那段已经算完、不再要的字节里——S 一算完,softmax 把它变成 P 写回去,这段字节的”身份”就从 fp32 的 S 切成了 fp16 的 P,靠的就是同一块内存被两个视图以不同宽度共用。

区域定义好之后, 计算代码反而很干净

底层那套别名换算虽然绕,可一旦区域定义好了,上层计算代码就根本不用操心原始列号。它只管按 stage / warpgroup 去读写:

```
S_region[q_stage]      #      S
S_region[wg_id, ...]    #      S
P_region[wg_id, ...]    #      P
O_region[i_q]           #      0
```

整段计算代码从头到尾没碰过一个原始列索引。”哪段字节归谁、fp16 步长要乘 2”这些脏活,全被塞进了 T.TMEMStages 的区域定义里。这正是好抽象该干的事:把”被容量逼出来的别名复杂度”圈在一个角落,让主干逻辑保持清清爽爽。

和你的 agent 一起试试

你可以让你的 AI agent 来讲讲这个 FA4 内核里的 fp32(tmem) 视图和 fp16(tmem_as_f16) 视图:S、P、O 各自落在哪块物理 TMEM 区域?P_region 的 stride 为什么是 MMA_N * 2?至于”复用”这个问题——某块区域要重新启用,得先等哪些消费者干完——留到下一小节,看完 barrier 表再回来对照。

barrier 如何把各个角色「接线」起来 (How Barriers Connect the Roles)

这是整个 kernel 里最难啃的一块,所以最好一层一层剥开看。前面几节已经把活儿分到了不同的 warpgroup:WG3 开硬件引擎 (TMA load、MMA、TMA store),WG0/WG1 做 softmax,WG2 做修正 (correction) 和收尾 (epilogue)。可问题是——这些角色彼此不说话,它们只能靠 barrier 互相「递东西」。这一节要干的,就是把这些 barrier 看成一张「谁把哪块 tile 交给谁」的接线图。

先抓主干: 只有 5 个搬运数据的 handoff

面对一长串 barrier,正确的打开方式是:先只盯那几个真正在主计算路径上搬数据的 handoff,其余的一律先当成「记账 (bookkeeping)」,以后用到再查。真正「数据就绪 (data-ready)」的交接,就下面这 5 条:

交接 (handoff)	含义
TMA load → score / value MMA	Q、K 或 V 已经到达 SMEM,可以喂给 MMA
score MMA → softmax	S 已在 TMEM 里就绪
softmax / correction → value MMA	P 已在 TMEM 里就绪,且 O 可以安全累加
value MMA → epilogue	最终的 O 已在 TMEM 里就绪
epilogue → TMA store	O_smem 已就绪,可以写回

除此之外的所有 barrier,本质上都是流水线记账:它们管的是释放某块 SMEM / TMEM / 暂存 (staging) 缓冲区,好让另一个角色拿去复用。

注意

注意：好消息是，一个 barrier 不管是搬数据还是单纯记账，**读法都一样**——都是一次「tile 交接」。每碰到一个 barrier，就问三句：**谁生产了数据？谁来消费？两边都用完之后，哪块缓冲区被腾出来了？**这三问问清楚，再玄乎的 barrier 也就那么回事。

两个 MMA 的「就绪闸门」

把上面这些交接收一收，关键就看两个 MMA 各自在等什么。这里要摆正一个心态：**这些 barrier 是「正确性闸门 (correctness gate)」，不是「时间表 (schedule)」**。它只回答「这个 MMA 开火前得满足哪些条件」，至于「什么时候发生」它压根不管。

结构图

连接关系：

- score MMA → S (TMEM)
- value MMA → O (TMEM)

- **score MMA** 就等两样：Q、K 都到了 SMEM。等齐了就产出 S。
- **value MMA** 得同时等三样：V 在 SMEM、softmax 给的 P tile、还有一个能安全写入的 0 槽位 (WG2 要么已经把旧 0 释放掉，要么已经给它重缩放好了)。

P 的就绪为什么被**拆成两半**？就是因为上一节那个 96 + 32 调度：P 前 96 列一就位，value MMA 就能先开火，最后 32 列由另一个闸门 `p_ready_2` 单独放行。

唯一的「异类」：`softmax → correction` 的标量信箱

有一个交接不走「tile 就绪」这套路子：**softmax 到 correction 这条边**。它递过去的不是一块 tile，而是一个**标量**——在 K/V 循环里是缩放因子 `acc_scale`，在 epilogue 里是最终的行和 `row_sum`。这个标量通过一个**只有一格的 SMEM「信箱 (mailbox)」**交给 WG2。

正因为这一格信箱**每轮迭代都得复用**，所以没法用普通的 tile 就绪闸门，得用一对 `full / empty` barrier 守着它——这是个标准的「生产者-消费者通道」，而不是 tile-ready 闸门：

结构图

连接关系：

- softmax 生产者：1. 等 `softmax_corr.empty` (信箱空了才能复用) → 2. 把 `acc_scale / row_sum` 写入信箱
- 3. `arrive softmax_corr.full` $\xrightarrow{\text{通知}}$ WG2 消费者：4. 等 `softmax_corr.full`，然后读信箱
- 5. `arrive softmax_corr.empty` $\xrightarrow{\text{释放}}$ 6. softmax 下一阶段可复用这一格信箱

这套握手一步步是这样：

1. softmax 先等 `softmax_corr.empty`，确认信箱已经腾空，才能复用这一格；
2. softmax 把 `acc_scale` 或最终的 `row_sum` 写进信箱；
3. softmax 在 `softmax_corr.full` 上 arrive(喊一声「写好了」)；
4. WG2 等 `softmax_corr.full`，然后把信箱读出来；

5. WG2 在 `softmax_corr.empty` 上 arrive(喊一声「读完了」);
6. 到下一阶段,softmax warpgroup 就能复用这一格信箱了。

注意

注意:`softmax_corr.empty` 的含义特别容易看岔,务必盯清楚。它只说明「WG2 已经把这一格标量消费掉了」,它绝不代表 P 就绪,更不是放行 value MMA 的那道闸门。放行 value MMA 的闸门是 `p_o_rescale`(只在 P 前 96 列写好、且 0 槽位能安全累加时才触发)。把这俩搞混,是一类经典「结果算错」bug 的源头。

完整 barrier 清单 (参考表)

主干捋清楚了,下面这张完整清单就当查阅手册用。每一行还是老规矩——「谁 → 谁」、「什么变安全了」:

Barrier	生产者 → 消费者	什么变得安全了
<code>q_load.full</code>	TMA load → score MMA	Q 的 SMEM tile 可以喂 MMA
<code>q_load.empty</code>	本 Q 阶段的所有 score MMA → TMA load	Q 的 SMEM 阶段可被下一个 task 复用
<code>kv_load.full</code>	TMA load → score / value MMA	K 或 V 的 SMEM tile 可以喂 MMA
<code>kv_load.empty</code>	score / value MMA → TMA load	K/V 的 SMEM 阶段可被复用
<code>s_ready</code>	score MMA → softmax	TMEM 里的 S tile 可读
<code>p_o_rescale</code>	softmax + WG2 → value MMA	P 的前 96 列已在 TMEM,且 0 槽位可安全累加
<code>p_ready_2</code>	softmax → value MMA	P 的最后 1/4(32 列) 已在 TMEM
<code>o_ready</code>	value MMA → epilogue	最终的 0 累加器就绪
<code>softmax_corr.full</code>	softmax → WG2	<code>acc_scale</code> 或最终 <code>row_sum</code> 已在 SMEM 信箱里就绪
<code>softmax_corr.empty</code>	WG2 → softmax	WG2 读完后,同一格信箱可复用
<code>corr_epi.full</code>	epilogue → TMA store	0_smem 就绪,可以写回
<code>corr_epi.empty</code>	TMA store → epilogue	0_smem 阶段可被复用

看「谁发信号」就能猜出 barrier 类型

跟 GEMM 一样,barrier 是哪种类型,看「谁来发完成信号」就知道了,这条规律反过来还能帮你读代码:

信号来源	barrier 类型	为什么
TMA load	TMABar	TMA 引擎自己按字节计数 (byte-count) 来判断搬运完成
MMA 完成	TCGen05Bar	<code>tcgen05.commit</code> 负责给整个「完成组 (completion group)」发信号
纯线程间交接	MBarrier	参与的线程各自 显式 arrive

拆开看 softmax → value 的双闸门

softmax 到 value 这个交接值得再放大看一眼,它用了两道闸门:

- `p_o_rescale`: P 前 96 列写好、且 0 tile 能安全累加时,放行 value MMA 开跑;
- `p_ready_2`: 放行 P 的最后 32 列,正好对上上一节那个 96 + 32 的 value-MMA 调度。

第一个 K/V 块最省心。这会儿还没有「旧 0」要 rescale, 所以 WG2 干脆对 p_o_rescale 直接预先 arrive(pre-arrive)——相当于一上来就把这道闸门打开, 反正也没活可干。

后面的块就得当心了。WG2 必须等到自己要么跳过了一次不必要的 rescale, 要么已经把旧 0 缩放完了, 才在 p_o_rescale 上 arrive。这里有个故意做得保守的「跳过判定」:

```
# softmax : log2
delta = (m_old - m_new) * scale_log2
if delta > -rescale_threshold:
    # , rescale: max, acc_scale 1.0
    acc_scale = 1.0
else:
    # , exp2 , WG2 rescale 0
    acc_scale = exp2(delta)
```

说白了就是: 只有最大值「跳」得够大, 才走 exp2 路径、真正叫 WG2 去 rescale 0。

拿到 softmax 的信号后, WG2 还要用 any_sync 把整个 warpgroup 里的 should_rescale 归约一遍:

```
# warpgroup OR : rescale
if T.ptx.any_sync(should_rescale):
    rescale_0() # 0
```

这个「跳过」为什么重要? 因为 rescale 0 是一次横跨整个累加器的 TMEM → 寄存器 (RF) → TMEM 读-改-写。要是阈值逻辑已经把 acc_scale 定成 1.0 了, 那这趟读-改-写就是纯浪费, 能跳当然要跳。

为什么所有新 barrier 都挤在 softmax 周围?

最后还有一个观察值得点破: 所有新增的 barrier 都挤在同一个地方。s_ready、p_o_rescale、p_ready_2, 加上 softmax_corr 这一对, 全围着 softmax 转。

它们存在的理由只有一个: score MMA 和 value MMA 不再挨着了。在 GEMM 里两个 MMA 可以背靠背连发; 可在 Attention 里, 中间硬塞进了寄存器上的数学运算、TMEM 的回写、还有对输出的 rescale——这中间的每一步, 都得配一次自己的交接。说到底, barrier 的数量, 就是「softmax 这段非 MMA 工作」复杂度的直接写照。

和你的 agent 一起试试: 让它把一个 K/V 块完整走一遍 s_ready → p_o_rescale → p_ready_2 → o_ready。每碰到一个 barrier, 都把四件事问清楚: 谁在等? 谁来 arrive? 哪块 tile 变得能读了? 之后哪块存储能被复用了?

流水线结构: 同一时刻到底谁在干活 (Pipelining Structure)

上一节讲的 barrier 回答的是「正确性」问题: 某个角色 (role) 消费一个 tile 之前, 哪些东西得先备齐。可 barrier 有一件事不告诉你——实际上到底谁和谁在同时跑。这一节就来回答这第二个问题。

这俩真是两码事。一道正确性闸门啥时候被满足, 跟生产者啥时候真正在跑, 可以差出十万八千里——可能早得多, 也可能晚得多。换句话说, 「S 就绪了没」这个检查发生在哪一刻, 跟「S 到底是啥时候算出来的」根本不绑定。barrier 图回答的是依赖关系, 这一节要画的是时间线 (timeline)。

没有「单一流水线深度」这回事

GEMM 里我们习惯问「流水线深度 (pipeline depth) 是几」——因为 GEMM 只有一条数据流 (K 维度上的 tile), 所有东西都跟着同一个节奏走。FA4 这套就不行了: 不同的 tile 流 (tile stream) 前进的速度不一样, 所以**每条流各管各的环形缓冲 (ring buffer)**, 各有各的深度。

流水线 (tile 流)	深度	含义
Q 流水线	2	一个 CTA 同时处理两个 Q stage:WG0 负责其中一个,WG1 负责另一个
KV 流水线	3	K、V 块在内层循环里不断流过, 而同样的两个 Q stage 被反复复用
TMEM 流水线	2	每个 Q stage 拥有自己的一组 S/P/O 的 TMEM slot; 等对应 barrier 触发后这些 slot 才被回收复用

注意

注意: 把这三条流水线分开看, 是关键。Q 为什么是深度 2? 因为「两个 Q stage 并行算 softmax」(WG0 和 WG1 各管一个);KV 为什么是深度 3? 因为我们想让 TMA 多提前预取几个 K/V 块, 把访存延迟藏起来; 而 TMEM slot 的复用又各自被对应的 barrier phase 卡着。三套节奏彼此独立, 没法用一个数字一概而论。

从「正确性闸门」切到「时间线视角」

原文这里给的是一张时间线示意图 (timeline)。它跟上一节的 barrier-flow 图回答的是两个不同问题:

- **barrier-flow 图**: 去这儿查「精确的生产者-消费者等待关系」(谁等谁)。
- **时间线图 (本节)**: 去这儿看「大致同一时刻, 哪些角色正活跃着」。

下面拿一张表, 把一条「有代表性的流水线波次 (pipeline wave)」的时间线重画一遍: 行 = 角色 (对应代码里的一个 role branch), 列 = 大致的时间步 (从左往右推进), 格子 = 那一刻这个角色在干啥。

角色	t0	t1	t2	t3	t4	t5	t6	...
WG3 warp1 TMA load	Q0	K[n-1]	Q1	V[n-1]	K[n-2]	V[n-2]	K[n-3]	...
WG3 warp0 MMA		S0	S1	PV0	S0'	PV1	S1'	...
WG0/WG1 softmax			P0	P1		P0'	P1'	...
WG2 rescale				release	rescale O	...	normalize	...

WG3 warp2 TMA store							O 写回	...
------------------------------	--	--	--	--	--	--	------	-----

说明: S_x = 第 x 个 Q stage 的 score MMA; PV_x = 第 x 个 Q stage 的 value MMA; 带撇号 (') 的是进了下一个 K/V 块之后的同名操作;MMA 这一行里,score 和 value 是交错着发的。你能看到 score、softmax、correction(rescale)、value 这几行在时间上是彼此重叠的 (同一列里好几个角色都在动), 而不是一个干完再轮下一个。

每一行对应的角色, 正好就是 kernel 里那几个分支:

- **WG3 warp 1:** 发起所有 TMA load(把 Q/K/V 从 GMEM 搬进 SMEM)。
- **WG3 warp 0:** 发起 score MMA 和 value MMA(两种 MMA 都归它管)。
- **WG0 和 WG1:** 为两个 Q stage 分别跑 softmax。
- **WG2:** 负责 0 的 release / rescale, 最后再做一次最终输出的归一化 (normalize)。
- **WG3 warp 2:** 发起 TMA store(把最终的 0 写回 GMEM)。

跟着一条波次从左往右走

把上面的时间线从左往右读一遍, 就是一次完整的流水线波次:

1. **load warp 先起步:** 先发 $Q_0, K[n-1], Q_1, V[n-1]$, 然后接着往下流式预取索引更低的 K/V 块 ($K[n-2], V[n-2]$ ……)。留意一下,K/V 是从高索引往低索引走的。
2. **MMA warp 跟上:** 先发头几个 score MMA, 产出 S_0 和 S_1 。
3. **WG0/WG1 接棒:** 把 S_0, S_1 经过 softmax 变成 P_0, P_1 。

关键设计:MMA warp 必须把 score 和 value 交错发

这是本节最重要的一点。MMA warp 千万不能先把所有 score MMA 发完、再去发所有 value MMA。等两个 Q stage 都「热身 (primed)」好了, 它就开始交替发两种 MMA: 给当前 V 块发一个 value MMA, 紧接着给下一个 K 块发一个 score MMA, 就这么来回倒:

发射顺序	类型	运算	备注
1	score	$Q_0 * K[n-1]$	
2	score	$Q_1 * K[n-1]$	
3	value	$P_0 * V[n-1]$	开始交错:value 进来了
4	score	$Q_0 * K[n-2]$	
5	value	$P_1 * V[n-1]$	
6	score	$Q_1 * K[n-2]$	
7	value	$P_0 * V[n-2]$	
...	...	...	

为什么非得这么交错? 因为正是这种交错, 才让时间线里 score、softmax、correction、value 这四行叠到了一起, 而不是排成一队规规矩矩地一个接一个跑。要是先把全部 score 做完再做全部 value, Tensor Core 在两段之间就会空出一大段窗口, softmax 和 rescale 也没法跟 MMA 并行——流水线一下就退化成串行了。

WG2 那一行:release / rescale 的两种情形

时间线里 WG2 那一行写的是 `release / rescale`, 这两个词对应的就是我们之前见过的两种情况:

情形	WG2 干什么	原因
第一个 K/V 块	只参与 handoff, 让 value MMA 能往下走 (只 <code>release</code>)	此时还没有「旧的 0」, 无需 <code>rescale</code>
后续 K/V 块	在 value MMA 累加进 0 之前, 可能先 <code>rescale</code> 旧的 0	<code>running max</code> 变了, 旧 0 的比例失真, 需要先校正

而归一化 (`normalization`) 和 `TMA store` 只来这么一次——非得等整个 `attention task` 的最后一个 K/V 块处理完了, 才动手。

为什么 FA4 没法用「一条 GEMM 式流水线」来描述

收个尾, 回到本节的主旨: 没有哪条 GEMM 式的单一流水线能描述 FA4, 因为 Q、K/V、TMEM slot 各走各的独立时间表。这正是前面要拆出三个环形缓冲、三个深度的原因。

TIRx 的做法, 是把这些时间表明明白白地摊开: 用分开的 `tile buffer`、各自的 `PipelineState` 游标 (`cursor`)、各自的 `barrier phase` 来表达, 而不是把整个 kernel 藏到一个庞大的单体原语 (`monolithic primitive`) 后面。

注意

注意: 这是个明摆着的工程取舍。代价是「活动部件 (`moving parts`) 更多」——你得同时盯着好几个 `ring`、好几个游标、好几个 `barrier`; 但好处是复杂度始终摆在明面上、看得见也查得了 (**visible and inspectable**)。kernel 一出问题, 你能一条 `tile` 流一条 `tile` 流地去追, 而不是干瞪着一个黑盒。

重缩放与写回 (Rescaling and Writeback)

在线 `softmax` 的精髓就在「边算边修正」。每来一个新分数 `tile`, 每行的最大值 `m` 都可能被刷新。`m` 一变, 之前加进 0 的那些结果就尴尬了——它们是用旧 `m` 算的, 整体偏大。这一小节讲两件事: 这一步非做不可的修正 (也就是重缩放), 还有 K/V 循环跑完之后, 怎么把最终结果归一化再写回显存。

为什么重缩放是「必须」而不是「优化」

不少人第一次看 `Flash Attention`, 以为重缩放是个性能技巧, 图省事可以跳过。恰恰相反: 它是保证结果正确的硬性步骤, 跳不得。

顺一遍逻辑就明白了。在线 `softmax` 不是一口气看到所有分数的, 而是一块一块往上加。算每一块时, 用的是「当时已知的最大值 `m_old`」来做指数减法。等下一块带来个更大的 `m_new`, 麻烦就来了: 之前加进 0 的每一项, 都比它该有的值大了一个倍数 `exp(m_new - m_old)`。不修正? 那先来的块就被「加权过头」了, 最终输出直接算错。

修正办法是把已经累加的 0 整体乘上一个缩小因子:

$$O_{old} \leftarrow O_{old} \cdot e^{(m_{old}-m_{new})/\sqrt{d}}$$

注意 `m_old - m_new ≤ 0`, 所以这个因子 `≤ 1`, 刚好把之前偏大的部分缩回来。

注意

注意：这里的 `m(row_max)` 存的是原始、没缩放过的 $\text{QK}^{\sim}\text{T}$ 分数的最大值，所以指数项里得额外带上 $1/\sqrt{d}$ 这个注意力缩放因子。这个约定后面写 LSE 时还会再用一次。

这是一次完整的 TMEM → 寄存器 → TMEM tile 操作

还有一点也常被误解：重缩放不是几个标量的小记账，而是对整个 `0` 累加器 (accumulator) 来一次完整的「读出一乘一写回」。`0` 待在 TMEM 里，得先用 `tcgen05.ld` 把整块 tile 读进寄存器，在寄存器里逐元素乘上缩放因子，再写回 TMEM。

这活儿拆给两个角色合作完成：

- **Softmax 角色：**负责算出每行的缩放因子 `acc_scale`，塞进 SMEM「信箱 (mailbox)」里。
- **WG2(第二个 warpgroup)：**等 `softmax_corr.full` 信号，从 TMEM 把当前的 `0` 读出来，乘上缩放因子，再写回 TMEM。

核心代码片段 (关键行加了中文注释)：

```
RESCALE_TILE = T.meta_var(16)
o_row = T.wg_reg_tile(RESCALE_TILE)                                # tile
Tx.copy_async(o_row, 0_region[i_q, d_start : d_start + RESCALE_TILE]) # TMEM -> (tcgen05.ld)
Tx.mul(o_row, o_row, acc_scale)                                     #
Tx.copy_async(0_region[i_q, d_start : d_start + RESCALE_TILE], o_row) # -> TMEM(TMEM store)
T.ptx.tcgen05.wait.st()                                            # TMEM , MMA
```

Tile 原语读出卡：修正 (rescale)

本书每个阶段都拿同一张「读出卡」来统一描述，这一阶段的卡片是这样：

项目	内容
作用域 (Scope)	WG2, 整个 warpgroup
布局 (Layout)	<code>0</code> 在 TMEM → 寄存器 → <code>0</code> 在 TMEM(<code>0_region[i_q]</code>)
派发 (Dispatch)	<code>tcgen05.ld</code> 读出, TMEM store 写回，中间在寄存器里做乘法
交接 (Handoff)	等待 <code>softmax_corr.full</code> ；完成后触发 <code>p_o_rescale</code> (→ <code>value MMA</code>) 和 <code>softmax_corr.empty</code> (→ <code>softmax</code>)

端到端的同步流程

重缩放最微妙的地方，就在于几个角色之间的握手 (handoff)。一整轮的同步，可以画成下面这条流水线：

结构图

连接关系：

- SMEM 信箱 $\xrightarrow{\text{softmax_corr.full (信箱已填好)}}$ WG2: 读 `0` → 乘缩放 → 写回 `0` 到 TMEM
- WG2: 读 `0` → 乘缩放 → 写回 `0` 到 TMEM $\xrightarrow{\text{p_o_rescale (O 已就绪)}}$ WG3 的 `value MMA`：消费 `P`，累加进重缩放后的 `0 tile`
- WG2: 读 `0` → 乘缩放 → 写回 `0` 到 TMEM $\xrightarrow{\text{softmax_corr.empty (信箱已读空)}}$ Softmax

可复用信箱，进入下一轮

一步一步拆开来：

1. Softmax 把缩放因子写进 SMEM 信箱。
2. WG2 等 `softmax_corr.full`(确认信箱写好了)。
3. WG2 在 TMEM 里把 0 重缩放一遍。
4. WG2 在 `p_o_rescale` 上「到达 (arrive)」，宣告 0 已经是新值了。
5. 到这时候，WG3 的 value MMA 才能安全地消费 P，把结果累加进已经重缩放过的 0 tile。

整个循环靠 `softmax_corr.empty` 来闭合：WG2 读完信箱、把这个 SMEM 槽位释放掉，softmax 角色下一轮就能复用同一个信箱。

注意

注意：第 5 步那个顺序约束是关键——必须先把 0 重缩放，WG3 的 value MMA 才能往里加；不然新一块结果就会被加到还没缩小的旧 0 上，前面那番修正全白费。`p_o_rescale` 这个 mbarrier，干的就是强制这个先后次序的活。

K/V 循环结束后：从修正切换到收尾 (epilogue)

所有 K/V 块都处理完了，WG2 就从「修正」模式切到「收尾」模式，把最终结果归一化、写回显存：

1. 等最终的 `row_sum` 和 `o_ready` 信号。
2. 从 TMEM 读出最终的 0。
3. 乘上 $1 / \text{row_sum}$ ——这就是我们一开始故意往后拖的那次归一化（除以分母这步留到最后统一做，是在线 softmax 省算力的惯用招）。
4. 把结果转成 (cast)fp16。
5. 写进 `0_smem`。

最后，由 WG3 的 TMA store warp 把 `0_smem` 通过 TMA 搬回 GMEM(全局显存)。

一个需要注意的限制：只有前向输出，没有 LSE

想拿这个 kernel 去扩展的人得留个心：它只算前向输出 (forward output)。而训练场景下的前向传播，一般还得存下反向用的对数和指数 (log-sum-exp, LSE)。

要补 LSE，有个缩放细节绕不开。前面说过，本 kernel 的约定是：

- `row_max` 存的是原始、没缩放过的 QK^T 分数的最大值；
- `row_sum` 累加的是 $\exp((S - \text{row_max}) / \sqrt{d})$ 。

因为 `row_max` 身上没带 $1/\sqrt{d}$ ，所以你在拼自然对数形式的 LSE 时，得把这个缩放因子重新补回 `row_max`：

$$\text{LSE}_i = \log(\text{row_sum}_i) + \text{row_max}_i / \sqrt{d}$$

注意

注意: 这个实现「只出前向输出」, 不写 LSE。要是想支持训练的反向传播, 记得按上面的式子把 $1/\sqrt{d}$ 补回去, 否则 LSE 的数值会差一个常数因子。

因果掩码 (Causal Masking)

因果注意力 (causal attention) 里有条硬规矩: 一个 query 只能看「自己这个位置或更早位置」的 key。换句话说, 对第 i 个 query 来说, 它在分数矩阵里合法的列只到第 i 列, 后面的 key 全是「未来」, 必须当作不存在。

FA4 处理这条规矩的办法很利落: 已经成型的那条数据通路 (data path) 它一概不碰——score MMA、softmax、value MMA 整套流程原封不动。因果模式只在两个地方动手脚, 一个图「省」, 一个图「准」:

- **省:** 那些纯属未来、对当前 Q 块一点贡献都没有的 K/V 块, 整块整块直接跳过;
- **准:** 对横跨对角线的那一两个块, 逐行把越界的列精确屏蔽掉。

下面这两手分别看。

先看「省」: 整块跳过对角线上方的 K/V 块

把分数矩阵想成一张二维网格: 行是 query, 列是 key, 对角线就是「query == key 位置」那条线。因果约束下, 对角线右上方 (列号大于行号) 整片区域都非法。

按块一划分, 很多 K/V 块会整块落在某个 Q 块所需范围的右上方——里面每一列都是未来, 对这个 Q 块的输出贡献是零。既然如此, 根本没必要把它们从 GMEM 搬进来, 更别说跑 MMA 和 softmax 了。

实现里用 `get_n_block_max(...)` 算出「当前 Q 块顶多用得到的最后一个 K/V 块」, 然后 K/V 循环对超出这个上界的块, 就干脆不加载、也不计算了。

```
#          Q          ,          K/V
n_block_max = get_n_block_max(...)
for n_block in range(n_block_max): #
    ...                             #          load / score MMA / softmax / value MMA
```

注意

注意: 这一手纯粹是「省工」——被跳过的块, 连数据都不搬。因为上三角 (对角线右上方) 整块都被跳掉, 平均下来因果 kernel 只需算全注意力 (full attention) 大约一半的 K/V 块 (越靠后的 Q 块要遍历的 K/V 越多, 越靠前的越少), 这正是它比全注意力快的主要原因。

下图直观看一下哪些块被跳过、哪些块要逐列精修 (拿一个靠中间的 Q 块举例):

以一个靠中间的 Q 块为例, 它对各 key 块的处理 (横轴是 key 块编号):

key 块	0	1	2	3	4	5
处理方式						
含义	全保留	全保留	跨对角线 (逐列掩码)	整块跳过	整块跳过	整块跳过

- 完全在对角线下方: 每一列都合法, 无需掩码
- 横跨对角线: 跑 MMA, 但 softmax 前要按行精确掩码
- 完全在对角线上方: 全是未来, `get_n_block_max` 让循环根本不会碰它

再看「准」：对横跨对角线的块逐行掩码

`get_n_block_max` 只能整块整块地取舍, 可总有那么正好压在对角线上的块: 这种块里一部分列合法、一部分列非法, 既不能整块跳, 也不能整块留。

对这类块的处理是: `score MMA` 照常跑、把分数算出来, 但在求指数之前, 先在 `softmax` 里把非法列屏蔽掉。具体是逐行干的:

1. 拿这一行的 `query` 位置, 加上当前块的列偏移 (`block offset`), 推出这一行的列上限;
2. 列号在上限以内的, 保留;
3. 列号超过上限的, 在寄存器里直接置 `-inf`。

为什么置 `-inf` 而不是置 0? 因为这一步是在 `exp2` 之前做的:

- `exp2(-inf) = 0`, 这些非法列对 softmax 分子 (P) 的贡献自然就是零;
- 同时 `-inf` 也不会把这一行的 `row_max` (行最大值) 带歪, 因为它永远当不上最大值。

这么一来, 不管是行最大值, 还是 `exp2` 分子的累加, 非法列都被干干净净地剔掉了, 后续运算压根不带它玩。

注意

注意: 回顾一下前面——`row_max` 维护的是原始 (未缩放) QK^T 分数的逐行最大值, P 是 `exp2((S - row_max) · scale_log2)`。把非法列设成 `-inf`, 正好让它既抬不高 `row_max`, 又在 `exp2` 之后归零, 一箭双雕。

实现技巧: 用位掩码一次性处理, 而不是逐元素分支

要是对每个元素都写一句 `if > then -inf` 的分支, 在 GPU 上会慢得离谱——分支会把 warp 内的整齐执行搅乱。FA4 的做法是把「列上限」变成一个罩住整段 32 宽分数块的位掩码 (`bit mask`), 然后一把就把这段里所有越界列一起屏蔽掉:

```
#                                32                                ,
mask_r2p(score_chunk, col_limit)
```

这就把「一个元素一个元素地判断」变成了「整段按位操作」, 发散分支也就躲掉了。至于那些完全落在对角线下方的块 (每一列都合法), 连掩码都省了, 直接照原样进 `softmax`。

下表把三类块各自怎么处理归纳一下:

块相对对角线的位置	是否加载/计算	softmax 掩码	说明
完全在下方	是	不需要	每列都合法, 等同全注意力
横跨对角线	是	需要, 用 <code>mask_r2p</code> 按位屏蔽	部分列合法, 逐行求列上限
完全在上方	否	——	由 <code>get_n_block_max</code> 直接跳过, 数据都不搬

从 tile 原语视角看: 因果模式只是「微调」, 不是「重写」

站在 tile 级原语 (tile primitive) 的视角回头看, 因果模式的改动相当克制:

- 它一条数据通路都没碰——score MMA 到 softmax、softmax 到 value MMA 的衔接全照旧;
- 它就干了两件事:(1) 用 `get_n_block_max` 把 K/V 循环的遍历次数 (trip count) 砍短了;(2) 在寄存器里的 softmax 流程中、夹在 score MMA 和 P 写回 (writeback) 之间, 插了一步掩码操作。

结构图

分组:

- 全注意力: 「score MMA」「softmax (exp2 + 重缩放)」「P 写回」
- 因果注意力: 「score MMA」「掩码 mask_r2p」「softmax」「P 写回」

连接关系:

- 因果注意力 → 同时 `get_n_block_max` 缩短整个 K/V 循环的遍历次数

注意

注意: 这正是 FA4「tile 原语 + barrier」骨架的一个好处——像因果掩码这种功能变体, 可以只往某一步原语内部塞逻辑 (这里是 softmax), 其余阶段和同步结构一概不用动。数据流、barrier 图都保持原样, 改动被牢牢圈在局部。

GQA 支持 (GQA Support)

为什么需要 GQA

「分组查询注意力 (Grouped Query Attention,GQA)」让多个查询头 (query head) 共用同一个 K/V 头。动机很直接: 自回归推理 (decoding) 阶段, 真正吃显存带宽的是反复读 KV cache, 而 KV cache 的大小跟 K/V 头的数量成正比。要是让每 4 个 Q 头共用 1 个 K/V 头,KV cache 直接缩到 1/4, 带宽压力跟着大降。

可这种共用给 kernel 带来一个「打包 (packing)」难题: 既然我们对某个 K/V 头只想加载、只想留一个 K/V tile, 那怎么让好几个不同的 Q 头都从这同一个 tile 里取数算?Flash Attention 4 的答案是——把共用同一个 K/V 头的那一整组 Q 头, 一次性凑在一起处理, 针对调度器派下来的单个 kv_head_idx 把活儿算完。

核心思路: 重新解释 Q tile 的 128 行

先定义两个关键比例:

```
GQA_RATIO = num_qo_heads // num_kv_heads # K/V Q ( )
SEQ_Q_PER_TILE = BLK_M // GQA_RATIO # Q tile
```

精髓就在于给 Q tile 的 128 行 (BLK_M=128) 换一种读法。标准的多头注意力里, 这 128 行就是 128 个序列位置 (sequence position)。可在 GQA 下, 拿 GQA_RATIO=4 来说, 它们就不再是 128 个序列位置了, 而是 32 个序列位置 × 4 个查询头, 交错打包 (packed) 进同一块 tile, 这样这 4 个 Q 头就一起骑在同一个 K/V tile 上了。

每一行怎么解码回「序列位置 + 头编号」：

```
seq_pos = row // GQA_RATIO # ->
q_head = row % GQA_RATIO # ->
```

下面用 GQA_RATIO=4 直观看一下这 128 行是怎么排布的：
GQA_RATIO=4 时这 128 行的排布 (每个 seq 位置占连续 4 行，放它的 4 个 Q 头)：

行号 row	0	1	2	3	4	5	6	7	...	124	125	126	127
seq_pos	0	0	0	0	1	1	1	1	...	31	31	31	31
q_head	0	1	2	3	0	1	2	3	...	0	1	2	3
分组	\<- seq 0 的 4 个 Q 头 ->				\<- seq 1 的 4 个 Q 头 ->				...	\<- seq 31 的 4 个 Q 头 ->			

注意

注意: 这种「序列位置在外、头在内」的交错排法, 让 MMA 看到的依旧是一块连续的 128 × HEAD_DIM 操作数。计算单元根本不用知道 GQA 这回事。

Q 加载: 用 3D 视图无拷贝地完成打包

难点在于: 内存里的 Q 是自然布局 Q[batch, seq, qo_head, dim], 可 MMA 后面要读的却是一块扁平的 128 × HEAD_DIM SMEM tile。怎么把前者「塞进」后者, 又一点额外的数据搬运都不引入?

答案是给同一块 SMEM 套一个 3D 视图 (view)——视图无非是换个角度解释同一段内存, 不产生任何拷贝:

```
#          Q SMEM tile          (          ,          ,          , head_dim)
Q_smem_3d = Q_smem.view(SMEM_PIPE_DEPTH_Q, SEQ_Q_PER_TILE, GQA_RATIO, HEAD_DIM)

Tx.copy_async(
    Q_smem_3d[i_q, :, :, :],          #          :          3D
    Q[batch_idx,
      m_start : m_start + SEQ_Q_PER_TILE,          #          :          SEQ_Q_PER_TILE
      kv_head_idx * GQA_RATIO :          #          :          K/V
      (kv_head_idx + 1) * GQA_RATIO,          #          GQA_RATIO          Q
      :],
    **tma_copy_q,          #          TMA
)
```

几个关键点:

- 目标侧的 (SEQ_Q_PER_TILE, GQA_RATIO) 这两维, 正好跟上面 seq_pos / q_head 的解码规则一一对上, 把 32×4 的逻辑结构原样映射进 128 行。

- 源侧在头维度上用 $\text{kv_head_idx} * \text{GQA_RATIO} : (\text{kv_head_idx}+1) * \text{GQA_RATIO}$ ，精准切出「归这个 K/V 头管的那一组 Q 头」。
- 整件事靠 TMA(Tensor Memory Accelerator) 的异步拷贝完成，视图负责把两种布局对齐，拷贝本身一气呵成。

K/V 不扩展 + O 存储对称还原

GQA 真正省钱的地方在于:K 和 V 在显存里压根不被复制扩展。 kv_head_idx 对应的那唯一一块 K/V tile, 被打包进 Q 行里的全部 GQA_RATIO 个查询头一起复用。这正是 GQA 省带宽的本质——少读 K/V, 而不是把 K/V 复制成好几份去将就 Q。

输出侧跟输入侧完全对称:epilogue 之后, 同样用一个对应的 3D 视图, 把打包在一起的各行结果解包还原, 写回自然布局 $\text{O}[\text{batch}, \text{seq}, \text{qo_head}, \text{dim}]$ 。

设计上的优雅:GQA 只活在边界上

这个方案最让人服气的一点, 是关注点切得干干净净:

位置	是否感知 GQA	看到的数据形状
Q 加载边界	是 (3D 视图打包)	$\text{Q}[\text{batch}, \text{seq}, \text{qo_head}, \text{dim}]$ $\leftrightarrow (\text{SEQ_Q_PER_TILE}, \text{GQA_RATIO}, \text{HEAD_DIM})$
内部计算路径 (score MMA 等)	否	普通的 $128 \times \text{HEAD_DIM}$ Q tile
O 存储边界	是 (3D 视图解包)	$(\text{SEQ_Q_PER_TILE}, \text{GQA_RATIO}, \text{HEAD_DIM}) \leftrightarrow \text{O}[\text{batch}, \text{seq}, \text{qo_head}, \text{dim}]$

也就是说,GQA 全被摁在 Q 加载和 O 存储这两个边界上解决掉了。一进计算路径,score MMA 看到的还是一块平平无奇的 $128 \times \text{HEAD_DIM}$ Q tile, 后面整张 tile-primitive 计算图 (softmax、PV MMA、流水线、mbarrier 同步等等) 一行都不用改。

拿一张流程图把这种「夹心」结构概括一下:

结构图

连接关系:

- $\text{Q SMEM tile} / 128 \times \text{HEAD_DIM} / (32 \text{ seq} \times 4 \text{ head 打包}) \rightarrow \text{计算路径} / \text{score MMA} / \text{softmax} / \text{PV MMA} / (\text{完全不感知 GQA})$
- $\text{计算路径} / \text{score MMA} / \text{softmax} / \text{PV MMA} / (\text{完全不感知 GQA}) \xrightarrow{\text{3D 视图解包}} \text{O 显存} / [\text{batch}, \text{seq}, \text{qo_head}, \text{dim}]$

注意

注意: 这种「边界上打包、核心里透明」的套路, 很值得搬到别的场景去——凡是只想在数据进出时改排布、又希望计算核心保持通用的需求, 都能靠零拷贝视图 (view) 把转换逻辑摁在加载/存储这两端。

Tile 调度 (Tile Scheduling)

调度器 (scheduler) 的核心职责就一件事: 把每个 CTA 派到一个具体的注意力任务上。这个任务用三元组 (batch, kv_head, m_block) 描述——也就是「第几个 batch、第几个 KV 头、第几个 Q 块 (Q block)」。

换句话说, 调度器只管「这个 CTA 负责算哪一块输出 tile」, 至于这块怎么算, 它不操心。

为什么要专门搞个调度器? 因为在 GPU 上, 所有 CTA 是一起启动、一起收尾的; 一个 kernel 的总耗时, 取决于**最后一个干完的 CTA**。要是任务分得不匀, 有些 CTA 早早算完干等着、另一些还在埋头苦算, 整体吞吐就被拖下来了。要不要费心调度, 关键就看一件事: **这些 tile 任务的计算量是不是一样大**。而这一点, 又是掩码 (masking) 模式说了算。

两种调度策略对应两种掩码模式

Flash Attention 4 对「非因果 (non-causal)」和「因果 (causal)」这两种模式, 用了两套不同的调度器。

非因果模式:FlashAttentionLinearScheduler(线性调度器)

非因果注意力里, 每个 Q 块都得对**全部** K/V 块算一遍注意力。也就是说, 每个 tile 任务的活儿一样多、整整齐齐。负载这么均匀, 就用不着什么花活儿: 维护一个固定大小的 CTA 池, 每个 CTA 干完手头的任务, 就把任务索引**整体往前跳 num_ctas 个** (也就是 GPU 上 CTA 的总数), 这样所有任务就被均匀铺给了所有 CTA。这是典型的「跨步划分 (strided partition)」——简单、没分支、负载天然就平。

因果模式:FlashAttentionLPTSchedulert(最长处理时间优先调度器)

因果掩码把「均匀」这个前提彻底打破了, 各任务的活儿一下子变得极不平衡:

- 序列开头附近的 Q 块, 只能「看到」自己前面那一点 token, 所以大概只要处理 **一个** K/V 块;
- 序列末尾附近的 Q 块, 前面所有 token 都「看得到」, 所以要处理 **全部** K/V 块。

因果掩码下各 Q 块的工作量 (# = 需要计算的一个 K/V 块):

Q block	需要计算的 K/V 块	工作量
0(开头)	#	最小
1	##	
2	###	
...	...	
N(末尾)	#####	最大

这时候要还像非因果那样「傻乎乎地」平均切任务, 就会冒出严重的长尾: 分到「重」块的 CTA 会比分到「轻」块的 CTA 晚收工老半天,GPU 利用率被尾巴上那几个任务拖垮。

所以因果模式改用 **LPT(Longest-Processing-Time, 最长处理时间优先)** 调度。它的核心思想来自经典的多机调度算法: **先把最重的任务安排上**。把活儿大的 Q 块「前置 (front-load)」分配, 让重活先开工, 这样各 CTA 的完成时间 (finish time) 就被拉平了, 长尾也削掉了。

注意

注意:LPT 一边追求负载均衡, 一边还会**尽量把相邻的 batch / head 任务凑在一起**调度, 好保住 **L2 cache 局部性 (locality)**。也就是说, 它不是死板地按工作量贪心, 而是在「均衡」和「数据复用」之间找平衡——既要**让重活先跑**, 又不**想老是切到八竿子打不着的数据上**, 白白糟蹋已经缓存好的 K/V。

两类策略对比

维度	非因果 (Linear)	因果 (LPT)
调度器	FlashAttentionLinearScheduler	FlashAttentionLPTSchedulr
各 tile 工作量	完全相等 (每个 Q 块都遍历全部 K/V)	高度不均 (开头轻、末尾重)
划分方式	固定 CTA 池, 步长为 num_ctas 跨步推进	最长任务优先、重块前置
优化目标	均匀铺满	拉平完成时间 + 兼顾 L2 局部性
next_tile() 行为	跳到下一个任务, 继续循环	当前任务结束后, 退出循环

统一的循环接口

两个调度器内部策略差得远, 可它们对外露出来的循环接口一模一样。所以 kernel 主体代码根本不用管自己用的是哪种调度器:

```
while scheduler.valid():
    m_block_idx = scheduler.m_block_idx
    batch_idx   = scheduler.batch_idx
    kv_head_idx = scheduler.head_idx
    #           Q ,           K/V
    scheduler.next_tile()
    #           tile(           )
```

两者唯一的行为差别, 全藏在 next_tile() 里:

- 非因果模式:next_tile() 让当前 CTA 跳到下一个任务, 循环接着转 (一个 CTA 这辈子可能要处理好几个 tile);
- 因果模式:next_tile() 干完当前任务后直接把循环结束掉 (因为 LPT 早在 kernel 启动前就把每个 CTA 该干哪些活都排好了)。

结构图

连接关系:

- 读取 m_block / batch / kv_head 索引 → 计算这一个 tile / (TMA load → score MMA → softmax / → value MMA → correction → TMA store)
- 计算这一个 tile / (TMA load → score MMA → softmax / → value MMA → correction → TMA store) → scheduler.next_tile()
- scheduler.next_tile() $\xrightarrow{\text{非因果: 跳到下一任务}}$ scheduler.valid()?
- scheduler.next_tile() $\xrightarrow{\text{因果: 结束}}$ 循环退出
- scheduler.valid()? $\xrightarrow{\text{否}}$ 循环退出

调度只决定「算哪一块」, 不决定「怎么算」

这一点是整个调度设计的精髓: 不管用哪种调度器, 它干的都纯粹是调度决策——只挑「这个 CTA 摊上哪一块注意力 tile」, 至于这块 tile 内部怎么算, 它从不插手。

循环体里跑的, 始终是同一套本地原语 (local primitives), 跟调度策略半毛钱关系没有:

结构图

把「调度」和「计算」这么彻底地拆开, 带来两个好处: 一是 `kernel` 主体逻辑能保持单一、稳定, 不用为不同掩码模式写两套计算路径; 二是以后想加新的调度策略 (比如针对别的稀疏掩码模式), 只要再写一个实现相同循环接口的调度器就行, 计算内核一点不用动。

编译并验证 (Compile and Verify)

前面所有小节给的都是「片段」——一块一块拆开来讲原理。这一节反过来: 把整章拆出来的零件重新拼成一个能跑的整体, 真刀真枪编译它、扔到 GPU 上跑, 再拿 PyTorch 的参考实现核对结果对不对。说白了, 这是整章唯一一个「跑一下就能把全部内容验收掉」的单元格 (cell)。

完整 `kernel` 的实现书里没有逐行铺开, 而是收在 `tirx-kernels` 仓库的 `flash_attention4.py` 这一个文件里——本章讲过的每一块 (TMA 加载、两阶段 MMA、在线 softmax、rescale 与回写、causal 掩码、GQA、tile 调度) 都在那儿组装成了一份能直接 `import` 的源码。

与 GEMM 验证单元格的两点差异

要是你跑过上一章 GEMM 的验证单元格, 会发现 Flash Attention 的入口几乎一个样, 就两处不同:

维度	GEMM 验证	Flash Attention 4 验证
入口函数	直接拿到 <code>kernel</code>	更丰富的工厂函数 <code>get_flash_attention4_kernel</code> (需要传形状、GQA 头数、是否 causal 等)
额外参数	无	多一个 <code>profiler_buf</code> , 供 <code>kernel</code> 内置的 profiler 写计时数据

第一点, 是因为 `attention` 的参数空间比 GEMM 大太多了 (序列长度、query/KV 头数、head dim、要不要 causal), 所以得有个工厂函数照着这些参数「现做」一份具体的 `kernel`。第二点, 是因为这份 `kernel` 自带 profiler, 跑的时候需要一块设备上的 `buffer` 来落计时点——所以调用时比 GEMM 多传一个张量。

跑通整章的那一个单元格

下面就是「一键验收全章」的代码。它干了四件事: 备输入、编译、跑、核对。

```
import torch
import torch.nn.functional as F
import tvm
from tirx_kernels.attention.flash_attention4 import (
    get_flash_attention4_kernel, PROFILER_BUFFER_SIZE)

# GQA      :32   query      8   KV
B, S, Hq, Hkv, D = 1, 1024, 32, 8, 128
Q = torch.randn(B, S, Hq, D, dtype=torch.float16, device="cuda")
K = torch.randn(B, S, Hkv, D, dtype=torch.float16, device="cuda")
V = torch.randn(B, S, Hkv, D, dtype=torch.float16, device="cuda")
O = torch.empty(B, S, Hq, D, dtype=torch.float16, device="cuda")
# profiler      buffer,
```

```

prof = torch.zeros(PROFILER_BUFFER_SIZE, dtype=torch.uint64, device="cuda")

#                                     kernel(      non-causal)
kernel = get_flash_attention4_kernel(B, S, S, Hq, Hkv, D, is_causal=False)
target = tvm.target.Target("cuda")
with target:
    #      tidx      TIR
    ex = tvm.compile(tvm.IRModule({"main": kernel}), target=target, tir_pipeline="tidx")

# ex.mod      torch      ,      ;      prof
ex.mod(Q, K, V, 0, prof)
torch.cuda.synchronize()

# torch      ;enable_gqa=32      query      8      KV
qt, kt, vt = (x.transpose(1, 2).float() for x in (Q, K, V))
ref = F.scaled_dot_product_attention(qt, kt, vt, enable_gqa=True).transpose(1, 2).half()
torch.testing.assert_close(0, ref, rtol=1e-2, atol=1e-2)
print(f"FA4: B={B} S={S} Hq={Hq} Hkv={Hkv} D={D}, non-causal -> PASS")

```

几个细节值得留意:

- **布局转换:** Q/K/V 在内核里用的是 (B, S, H, D), 而 PyTorch 的 `scaled_dot_product_attention` 要的是 (B, H, S, D), 所以参考实现先 `transpose(1, 2)` 把 head 维挪到前头, 算完再换回来。
- **GQA 对齐:** `enable_gqa=True` 让 PyTorch 用上跟我们 kernel 一样的「32 个 query 头共享 8 个 KV 头」语义, 不然两边算的压根不是一回事。
- **精度对齐:** 参考实现先 `.float()` 升到 fp32 再算, 最后 `.half()` 落回 fp16, 这样才跟 kernel 的输出 dtype 对得上。

期望输出: 最后一行打印 ... -> PASS。

为什么用 `rtol/atol` 而不是逐位相等

你可能会问: 都是算注意力, 为啥不要求结果分毫不差, 反倒给个 `rtol=1e-2`, `atol=1e-2` 这么「松」的容差? 原因是: 就算 kernel 把在线 softmax 的累加放在 fp32 里做, 它跟「高精度参考实现」之间也还隔着好几层互相独立的近似来源。一项项拆开看:

结构图

连接关系:

- 高精度参考实现 → ① fp16 存储: 输入与中间操作数被存成 fp16, 带来舍入误差
- ① fp16 存储: 输入与中间操作数被存成 fp16, 带来舍入误差 → ② exp2 重写: softmax 用 exp2 改写 / $(\text{scale_log2} = \log_2(e)/\sqrt{d})$, 把每个指数都换算到 2 的幂上)
- ② exp2 重写: softmax 用 exp2 改写 / $(\text{scale_log2} = \log_2(e)/\sqrt{d})$, 把每个指数都换算到 2 的幂上 → ③ 在线 softmax 重排: 按块累加并逐行 rescale, / 而不是一次性把所有块加在一起
- ③ 在线 softmax 重排: 按块累加并逐行 rescale, / 而不是一次性把所有块加在一起 → ④ 回写 fp16: 最终 O 在写回时被 cast 成 fp16
- ④ 回写 fp16: 最终 O 在写回时被 cast 成 fp16 → kernel 实际输出

注意

注意: 这里挑的 `rtol/atol` 跟源 `kernel` 自带测试用的容差是一样的。它是冲着「上面四种误差叠加之后相对参考实现的总偏差」来设的, 不是只盯 `fp16` 舍入这一项。换句话说, 它得把 ①~④ 全罩住, 而不是只罩其中某一层。

如果真的 FAIL 了, 去哪里找问题

正因为这个容差是「按总误差量身定的」, 所以你要是看到的不是「擦边差一点」的临界值, 而是一次实打实的失败, 那它差不多就是个明确的路标: 问题八成出在 `softmax` 通路上, 而不是浮点精度本身。具体说, 得回头查本章那些用 `barrier` 串起来的「交接 (`handoff`)」:

可疑现象	对应被破坏的交接点
漏等 <code>s_ready</code>	<code>scores</code> 还没算好就进了 <code>softmax</code>
漏等 <code>p_o_rescale</code>	<code>rescale</code> 与第二个 <code>MMA</code> 之间的同步丢失
漏等 <code>p_ready_2</code>	概率矩阵 <code>P</code> 还没就绪就被第二阶段 <code>MMA</code> 消费
<code>row_max / row_sum</code> 更新后, <code>rescale</code> 步骤没有应用	在线 <code>softmax</code> 的运行缩放 (<code>running scale</code>) 没跟上

这些恰好就是本章一路用 `mbarrier` 小心护着的那几个 `handoff`。一次真正的数值失败, 说白了就是在告诉你: 某对 `producer/consumer` 之间的等待被漏了, 或者某次 `row_max/row_sum` 的更新没被 `rescale` 好好吸收掉。把验证失败当成「指向 `softmax` 路径的诊断信号」来读, 往往比死盯着浮点数据本身更快摸到根因。

与 GEMM 的区别 (Differences from GEMM)

前面几节我们已经看明白,Flash Attention 4(下面简称 FA4) 在内核结构上其实是「站在 GEMM 肩膀上」的: 它复用的还是那同一套搬 `tile`、做 `MMA`、再写回的硬件原语。那问题来了——底层硬件能耐没变,FA4 写起来怎么就比一个高性能 GEMM 难这么多?

本节的核心观点很简单:**FA4 难, 不是因为它换了硬件, 而是因为它的 `tile` 数值更多, 这些数值之间的「交接 (`handoff`)」也更多。**所有复杂度, 最后都能追到本章开头点明的那个结构性变化上——多了第二个 `MMA`, 而且 `softmax` 还被夹在两个 `MMA` 中间。

逐项对比:GEMM 与 FA4 在哪些轴上发生了变化

下面这张表把 FA4 相对 GEMM 变了的几个维度一条条列出来。看的时候建议带个视角: 每一行的差异, 归根到底都是「第二个 `MMA` + 中间那段 `softmax`」这一件事派生出来的。

对比维度	GEMM	Flash Attention 4
MMA 阶段	同一个 <code>MMA</code> 反复执行	分成两段: 打分 <code>MMA</code> (<code>score MMA</code>) 和取值 <code>MMA</code> (<code>value MMA</code>)
MMA 之间的工作	除了流水线交接外没有别的事	在线 <code>softmax</code> (<code>online softmax</code>)、掩码 (<code>masking</code>)、对 <code>O</code> 的重缩放 (<code>rescaling</code>)
运行时状态	只有累加器 (<code>accumulator</code>)	行最大值 (<code>row max</code>)、行求和 (<code>row sum</code>)、 <code>O</code> 累加器
主要中间结果	累加器所在的 <code>TMEM tile</code>	<code>S</code> 、 <code>P</code> 、 <code>O</code> 三块 <code>TMEM tile</code> 区域

warp 角色分工	TMA 生产者、MMA 消费者、写回 (writeback)	TMA load、MMA、softmax、correction(校正)、TMA store
屏障 (barrier)	主要是 load / compute / writeback 之间的交接	额外多出 score / softmax / value / correction 等多组交接
调度单元 (scheduling unit)	一个输出矩阵 tile	一个注意力任务:(batch, kv_head, m_block)

为什么会有这些差异: 从「两个 MMA」一路推下去

把上表每一行串起来看, 你会发现它们不是一堆零散的设计选择, 而是一条因果链:

结构图

连接关系:

- 起因: 多了第二个 MMA, 中间夹了 softmax $\rightarrow$ MMA 拆成两段 / (score / value)
- 起因: 多了第二个 MMA, 中间夹了 softmax $\rightarrow$ 两段之间要做 / softmax / 掩码 / 重缩放
- 起因: 多了第二个 MMA, 中间夹了 softmax $\rightarrow$ 需要维护跨块的 running 状态 / (max / sum / O)
- MMA 拆成两段 / (score / value) $\rightarrow$ 需要更多中间 tile / (S / P / O 三块)
- 两段之间要做 / softmax / 掩码 / 重缩放 $\rightarrow$ 需要新增 warp 角色 / (softmax / correction)
- 需要维护跨块的 running 状态 / (max / sum / O) $\rightarrow$ 需要更多 barrier / 把这些角色串起来
- 需要更多中间 tile / (S / P / O 三块) $\rightarrow$ 调度单元也从「输出 tile」/ 升级为「注意力任务三元组」
- 需要新增 warp 角色 / (softmax / correction) $\rightarrow$ 调度单元也从「输出 tile」/ 升级为「注意力任务三元组」
- 需要更多 barrier / 把这些角色串起来 $\rightarrow$ 调度单元也从「输出 tile」/ 升级为「注意力任务三元组」

- **MMA 从一段变两段:** GEMM 全程就是把同一个 $C += A @ B$ 重复跑很多遍; FA4 则是先算 $S = Q @ K^T$ (打分 MMA), 再算 $O += P @ V$ (取值 MMA)。两段 MMA 的输入输出都不一样, 自然就得上两套节奏。
- **两段 MMA 之间不再是「空档」:** GEMM 里, 两次 MMA 之间除了流水线交接啥也不干。FA4 偏要往这个夹缝里塞 online softmax (在线、分块地求 softmax)、给被掩盖的位置打掩码、行最大值更新后还要对已累加的 O 重缩放。这些可都是「真活儿」, 不只是搬数据。
- **运行时状态变多:** GEMM 只要维护一个累加器。FA4 因为 softmax 要在分块流式处理里保持数值稳定, 就得额外带上每行的 running max、running sum, 再加 O 累加器——三份状态都得跨 KV 分块一路更新。
- **中间结果从一块变三块:** 相应地, TMEM 里得给 S (打分结果)、P (softmax 之后的概率)、O (输出累加) 各留一块 tile 区域, 而不是 GEMM 那一块累加器 tile。
- **warp 角色更细:** GEMM 大致就「TMA 生产者 / MMA 消费者 / 写回」三类角色。FA4 把分工拆得更碎——TMA load、MMA、softmax、correction、TMA store, 各管一摊, 这样上面那些额外的计算环节才能流水线化地并行起来。

- **屏障更多:** 角色多了, 交接点跟着就多了。除了 GEMM 里常见的 load/compute/writeback 交接,FA4 还得多加 score、softmax、value、correction 等好几组 mbarrier, 把「谁算完了、下一个角色才能消费」管得死死的。
- **调度单元升级:**GEMM 的调度单元就是一个输出矩阵 tile;FA4 的调度单元升成了一个完整的「注意力任务」, 用三元组 (batch, kv_head, m_block) 来标识——也就是「哪个 batch、哪个 KV 头、哪个 M 方向的块」。

底层契约其实一点没变

有一点要专门强调: 上面列了这么一大堆差异, 可 **FA4 靠的那套 TIRx 底层契约 (contract)**, 从头到尾一点没变。也就是说, 描述这套内核的「四件套」语义, 在 GEMM 和 FA4 里完全一样:

契约要素	它回答的问题
tile 原语 (tile primitive)	「搬运 / 计算的是哪一块 tile」
作用域 (scope)	「哪些线程在协作完成它」
布局 (layout)	「这块 tile 存在哪里」
屏障 (barrier)	「下一个角色什么时候可以消费它」

注意

注意: 看懂这一点学 FA4 太重要了——你不用去啃一套全新的硬件抽象。FA4 用的还是同样的 tile 原语、同样的 scope/layout/barrier 语义。区别只在「数量」上:tile 数值更多、角色之间的交接更多。一句话,FA4 是 GEMM 在「广度」上的铺开, 不是在「深度」上另起炉灶。

所以, 你要是已经把 GEMM 内核那套契约吃透了, 读 FA4 的正确姿势就是: 把它拆成「同样的契约 + 更多 tile + 更多 handoff」, 然后顺着上面那条因果链, 把新增的角色和屏障一个个对号入座就行。

课后练习 (Exercises)

这一小节拿几道问答, 把全章最容易绕晕的几个点串起来: 两段 MMA 之间多出来的那次 tile 交接、softmax 为什么要把 P 写回 TMEM、还有某个 barrier 到底”证明”了什么。下面每道题, 我先把”问的是啥、该往哪个方向想”讲清楚, 再给一份对着本章前文 (尤其是「两段 MMA」和「barrier 如何连接各角色」这两节) 推出来的参考答案。建议你先把答案盖上, 自己推一遍, 卡住了再翻回来看。

注意

注意: 这些练习真正要练的不是”背答案”, 而是把每个 tile 原语都套进同一张读卡——scope(谁在跑) / layout(tile 在哪) / dispatch(怎么发射) / handoff(用哪个 barrier 交棒)。哪天你能拿这四问拆开任意一个原语, 这章就算读通了。

第 1 题:FA4 相比 GEMM 多出来的 tile 交接

问题: 相比 GEMM,FA4 在两段 MMA 之间多出来一个全新的 tile 交接。请说出它的**生产者 (producer)**、所交接的 TMEM tile、**消费者 (consumer)**。

怎么想:GEMM 只有”一段反复跑的 MMA”,累加器 tile 就在 MMA 内部循环里转,中间没有别的角色插队。FA4 把一段 MMA 拆成了「score MMA」和「value MMA」两段,中间硬塞了 softmax。规律是:只要”角色变多”,就一定”交接变多”——所以新交接肯定落在 score MMA 与 softmax、softmax 与 value MMA 这两条边上。题目问的是”两段 MMA 之间”那个 GEMM 压根没有的环节,落点就是 score MMA 把分数 tile 递给 softmax 这一步。

参考答案:

要素	内容
生产者	score MMA(由 WG3 warp 0 发射)
TMEM tile	分数 tile S(写入 S_region[q_stage])
消费者	softmax(WG0 / WG1, 整 warpgroup 读出)
交接 barrier	s_ready

整个交接说穿了就一句话:score MMA 算完,由一个被选中的 lane(`elect_sync`)在 `s_ready` 上 arrive,喊一声”这块 S 已经躺在 TMEM 里了”,softmax warpgroup 这才能去读。GEMM 里没这一步,是因为 GEMM 的 MMA 算完直接进下一轮累加,中间没别的角色要看这块中间结果。

```
Tx.warp.gemm_async(S_region[q_stage], Q_smem[...], K_smem[...],
                  dispatch="tcgen05", cta_group=CTA_GROUP)
if T.ptx.elect_sync():
    s_ready.arrive(q_stage) # lane softmax
```

第 2 题:为什么 softmax 要把 P 写回 TMEM

问题:softmax 在寄存器里刚算完分子 tile P,为什么不直接把它留在寄存器里喂给 value MMA,反而要写回 TMEM?

怎么想:这题考的是”MMA 的操作数到底能读什么形状”。寄存器是每个线程私有、按线程散开的一堆标量;而 MMA 是 Tensor Core 指令,它要的是一个矩阵形状的 tile 操作数,只认 SMEM 或 TMEM 这种共享、规整、能被硬件按矩阵布局寻址的地方。一堆散落在各线程私有寄存器里的 fp32 标量,Tensor Core 根本没法当矩阵来读。

参考答案:因为 value MMA 要把 P 当矩阵操作数 (tile operand) 来吃,而 MMA 读不了散在各线程的标量寄存器。本 kernel 里 P 唯一能被 MMA 消费的状态,就是 `P_region`——它是 fp16 TMEM 别名 `tmem_as_f16` 上的一个视图。所以这趟写回不是多此一举的搬运,而是把 P 摆成”下一段 MMA 唯一真能读进去的形状”。顺便说一句,写回时还顺手做了 fp32→fp16 的 `cast`,既省了 TMEM 占用,又对上了 Tensor Core 处理低精度操作数的吞吐。

可以这样对照记忆:

阶段	P 的存放形态	谁能用它
softmax 算完瞬间	各线程私有寄存器 (分散标量)	只有本线程的标量运算
写回之后	<code>P_region</code> (fp16 TMEM 视图)	value MMA 当矩阵操作数读取

第 3 题:p_o_rescale 或 p_ready_2 到底证明了什么

问题:从 `p_o_rescale` 和 `p_ready_2` 里挑一个,说清楚这个 barrier 究竟”证明”了什么;如果 value MMA 跳过对它的等待,会出什么问题?

怎么想:value MMA 干的是把 $P \times V$ 累加进 0。它要能安全开跑,前提有两类:一是操作数真到位了 (P 这部分列已经写进 TMEM),二是累加目标安全 (0 这个槽位要么已经被 WG2 rescale 成

新 max 下的正确状态, 要么因为压根不需要 rescale 而被放行)。这两个 barrier 分别守的, 就是 P 的两段 (96 + 32 切分) 和 0 的安全性。

参考答案 (以 p_o_rescale 为例):

- 它同时证明两件事成立:(1)P 的前 96 列已经写进 TMEM;(2)0 槽位对 value MMA 的累加是安全的——WG2 要么已经把旧 0 按新 row max rescale 完了, 要么判定本轮 max 跳得不够大、acc_scale 保持 1.0、于是跳过了 rescale。第一个 K/V block 是特例: 没有旧 0 要 rescale, WG2 干脆直接预先 arrive p_o_rescale。
- 要是跳过这个等待会怎样:value MMA 可能在 P 前 96 列还没写完时就去读 TMEM, 读到陈旧/半成品数据; 更阴的是 0 还没被 rescale 就被加了进去——旧 0 是按旧 max 缩放的, 新一块直接加上, 数值基准对不齐, 结果就是个悄无声息的错误结果 (wrong-result bug), 通常还不报错、不崩溃, 排查起来要命。

注意

注意: 本章特意警告过一个经典坑——softmax_corr.empty 只说明“WG2 已经把那个 scale/-sum 邮箱槽位消费掉了”, 它不代表 P 就绪, 更不是放行 value MMA 的门。真正放行 value MMA 的是 p_o_rescale。把这俩搞混, 正是 wrong-result bug 的常见来源。

要是你选 p_ready_2: 它证明的是 P 的最后 32 列 (最后四分之一) 已写进 TMEM, 对应 value MMA 那个 96 + 32 切分调度——第二个 sub-MMA 专吃这最后一段。跳过等它, 第二段 MMA 就会在最后 32 列还没写好时去读, 照样读到垃圾数据、结果出错。这个切分的意义就是别让 Tensor Core 空转: 前三块 chunk 一就绪就先发 96 宽的 MMA, 把最后一块的 exp 和 TMEM 写入跟正在飞的 MMA 叠到一起。

和你的 agent 一起练 (Try with your agent)

挑一个前文没给读卡的 tile 原语, 让 agent 替它补一张 scope / layout / dispatch / handoff 卡, 然后回到源码里, 用 guard、allocation、wait 这三处线索去核对它说得对不对。可以拿来练的对象有:

- epilogue 里的 Tx.copy_async: 把最终 0 从 TMEM/寄存器搬到 0_smem, 核对它是谁 (WG2) 发起的、交给哪个 barrier(corr_epi.full → TMA store)。
- fp32 → fp16 的 Tx.cast: 就是第 2 题里把 P 降精度写回 TMEM 的那一步, 核对它的精度转换是发生在 register 里、还是写入 TMEM 时。
- 第二个 gemm_pv sub-MMA: 吃 P 最后 32 列的那段 value MMA, 核对它的 handoff 等的是不是 p_ready_2、accum 是不是 True。

核对的方法永远是同一套: guard(if T.ptx.elect_sync() / 角色分支 wg_id == ... 告诉你 scope)、allocation(S_region / P_region / 0_region 视图告诉你 layout)、wait/arrive(barrier.wait / barrier.arrive 告诉你 handoff)。把 agent 的回答一项一项对回源码, 别直接信它。

小结

本章顺着「跟一个 tile 走完整条数据通路」这条主线, 把 Flash Attention 4 怎么落到 GPU 上讲透了。核心脉络拢成几条:

- **算法层面:**Attention 的难点不在矩阵乘本身,而在两段 MMA 中间夹的那段 softmax。在线 softmax 靠「分块流式累加 + 维护 `row_max/row_sum/0`」躲开了物化整个 `S`,代价就是行最大值一变大就得对旧 `0` 重缩放——这是贯穿全章的 twist。工程上又拿 `exp2`(配合 `scale_log2 = log2(e)/√d`) 顶替 `exp`,还把归一化拖到最后统一做。
- **落地层面:**TIRx 按「干什么活」而不是「碰哪块数据」给 4 个 warpgroup 分角色,把 `S/P/0` 挤进同一块 TMEM、靠 `fp16` 别名复用,再用一组 `mbarrier` 把跨 warpgroup 的 `handoff` 接起来。所有新增的 `barrier(s_ready、p_o_rescale、p_ready_2、softmax_corr` 等)都围着 softmax 转,因为正是 softmax 把本来挨着的两段 MMA 给拆开了。
- **流水线层面:**`Q`(深度 2)、`KV`(深度 3)、`TMEM slot`(深度 2)各跑各自的节奏,没有「单一流水线深度」这一说;`MMA warp` 必须把 `score` 和 `value` 交错着发,才能让 `score / softmax / correction / value` 这四类活在时间上叠起来。
- **功能扩展:**因果掩码、GQA、tile 调度都被特意摁在「边界」上,核心那张 `tile-primitive` 计算图原封不动——这正是「tile 原语 + barrier」骨架的厉害之处。
- **关键收获:**读懂 FA4 的正确姿势,是把它当成「跟 GEMM 同一套契约 + 更多 tile + 更多 handoff」,对每个原语反复套那四问 `scope / layout / dispatch / handoff`,同时盯紧 `softmax_corr.empty` 和 `p_o_rescale` 混淆这类 wrong-result bug。

延伸阅读

- 原文:[Flash Attention 4 —Modern GPU Programming for ML Systems](#)
- 前置章节 (第三部分 GEMM): 本章一路拿 GEMM 当参照系——GEMM 的 `tile-primitive` 图、`scope/layout/dispatch` 读卡,还有 `TMA / tcgen05 MMA / TMEM / mbarrier` 这些原语,都是读懂 FA4 的必备底子。
- 硬件背景:Blackwell 架构的 TMEM(Tensor Memory) 和第 5 代 Tensor Core(tcgen05) 指令族,是本章讨论 TMEM 别名复用、MMA 操作数位置的硬件前提。
- 参考实现:`tirx-kernels` 仓库的 `flash_attention4.py` 把本章讲过的全部组件 (TMA 加载、两段 MMA、在线 softmax、rescale 与回写、causal 掩码、GQA、tile 调度) 都收在一起,可以直接 `import`、编译,再拿 PyTorch 的 `scaled_dot_product_attention(enable_gqa=True)` 对拍验证。
- 进一步阅读: 想把 kernel 扩到训练场景,得补写 `log-sum-exp(LSE)`,还要记得按 `LSE_i = log(row_sum_i) + row_max_i/√d` 把 `1/√d` 缩放因子补回 `row_max`。

术语对照

英文	中文	简要说明
Flash Attention 4 (FA4)	Flash Attention 4	用在线 softmax 实现、避免物化完整分数矩阵的高性能注意力 kernel
Online softmax	在线 softmax	按 <code>K/V</code> 块流式累加、逐行维护运行态、最后统一归一化的 softmax 算法
Score MMA	打分 / 分数 MMA	第一段矩阵乘 $S = Q \cdot K^T$, 两个操作数都来自 SMEM
Value MMA	取值 MMA	第二段矩阵乘 $0 \neq P \cdot V, P$ 来自 TMEM、 <code>v</code> 来自 SMEM

Rescale / Correction	重缩放 / 校正	<code>row_max</code> 变大后对旧 <code>0</code> 做的 TMEM→ 寄存器 →TMEM 读-改-写, 统一尺度
<code>row_max</code> / <code>row_sum</code>	行最大值 / 行和	在线 softmax 逐行维护的两个运行态 (<code>row_max</code> 存未缩放分数的最大值)
<code>scale_log2</code>	$\log_2$ 缩放系数	$\log_2(e)/\sqrt{d}$, 使 $P = \exp_2((S - m) \cdot \text{scale_log2})$, 贴合硬件 <code>exp2</code> 指令
<code>acc_scale</code>	累加器缩放因子	softmax 算出、经 SMEM 信箱传给 WG2 的逐行 rescale 因子
<code>rescale_threshold</code>	重缩放阈值	行最大值变化小于此阈值 (本版 8.0) 则跳过 <code>rescale, acc_scale</code> 置 1.0
TMEM (Tensor Memory)	张量内存	Blackwell 为 <code>tcgen05</code> MMA 引入的片上内存, 每 CTA 仅 128×512 fp32
<code>tcgen05</code>	第 5 代 Tensor Core 指令	驱动 MMA、TMEM 读 (<code>tcgen05.ld</code>)/写、commit 的指令族
TMA (Tensor Memory Accelerator)	张量内存加速器	在 GMEM↔SMEM 之间异步搬运 tile 的硬件引擎, 按字节计数判完成
Warpgroup (WG)	warp 组	128 线程为一组; 本 kernel 每 CTA 有 4 个 (WG0-WG3), 按角色分工
Q stage	Q 流水线阶段 / 槽位	Q 流水线的 0/1 两个并发槽 (非两个 attention head), WG0/WG1 各管一个
mbarrier	内存屏障	跨线程/角色的同步原语, FA4 用它把各 handoff 接线起来
<code>s_ready</code>	score 就绪屏障	score MMA → softmax:S 已在 TMEM 可读
<code>p_o_rescale</code>	P/O 就绪屏障	softmax+WG2 → value MMA:P 前 96 列就绪且 0 槽位可安全累加
<code>p_ready_2</code>	P 尾段就绪屏障	softmax → value MMA:P 最后 32 列就绪 (对应 96+32 切分)
<code>softmax_corr</code> (full/empty)	softmax-校正信箱屏障	保护单格 SMEM 信箱, 在 softmax 与 WG2 间传 <code>acc_scale/row_sum</code>
Causal masking	因果掩码	query 只能关注自身及更早的 key; 整块跳过 + 跨对角线逐行掩码
<code>get_n_block_max</code>	最大 K/V 块上界	按 Q 块位置算出需遍历的最后一个 K/V 块, 跳过对角线上方整块
<code>mask_r2p</code>	列上限转位掩码	把逐行列上限转成 32 宽位掩码, 一次性屏蔽越界列 (避免发散分支)
GQA (Grouped Query Attention)	分组查询注意力	多个 Q 头共享一个 K/V 头, 缩小 KV cache、降带宽
GQA_RATIO	分组比例	每个 K/V 头被多少个 Q 头共享 (如 4)
LPT Scheduler	最长处理时间优先调度器	因果模式用: 重块前置以拉平各 CTA 完成时间, 兼顾 L2 局部性
Linear Scheduler	线性调度器	非因果模式用: 固定 CTA 池按 <code>num_ctas</code> 跨步均匀分配任务
Epilogue	收尾阶段	K/V 循环结束后归一化 (乘 $1/\text{row_sum}$)、cast fp16、写 SMEM、TMA store
LSE (log-sum-exp)	对数和指数	训练反向所需的统计量; 本 kernel 仅前向输出、不写 LSE
Tile-primitive graph	tile-原语图	「生产者—消费者」依赖图: 节点是某层存储里的 tile, 边是搬运/计算

第 15 章 · 参考资料 (附录)

原文:[Reference](#)

本章要点

本章要点 (TL;DR)

- 这一章是「第五部分 · 参考与附录」的[导读页](#)。它不教新东西, 只告诉你一件事: 遇到具体问题时, 该去翻哪一页。
- 全书真正的主线是第一到第四部分 (Parts I-IV)。附录不是用来通读的, 而是你[边读边查](#)的工具箱。
- 附录主要有三块内容:**TIRx 语言参考**、**编译器内部实现** (讲下降流水线)、**Warp-Specialized 内核调试指南**。
- 完整的 `tvm.tirx` Python API 不在这本书里, 而是放在[上游 TVM 官方文档](#)中。
- 至于 TIRx 的底层语义和张量布局模型 (Layout API), 其实[第二部分](#)已经讲过了, 这里只是给你一个指路牌。

前置知识

前置知识: 读这一章前, 最好先大致认识 GPU 内核 (在 GPU 上跑的小程序)、GEMM(矩阵乘法)、warp specialization(让不同线程组各干各的活) 这几个概念。没把握的话, 先翻一下第 0 章 · 极简入门。本章是一张导航页、术语多为预告, 这里点到的名词在正文各章会细讲, 看不懂可先跳过。

15.1 这一部分是什么

先把整本书的脉络捋一遍。《Modern GPU Programming for MLSys》真正讲东西的, 是 **Parts I-IV** 这四个部分。它的讲法大概是这样: 先带你认识 GPU 硬件长什么样; 再引出 TIRx, 这是一门张量级的中间语言; 接着讲编译器是怎么把你写的高层算子, 一步一步”降”成高效 GPU 代码的; 最后教你怎么写出 warp-specialized 的高性能内核。这条线才是主菜。

那这一章所在的「参考与附录」呢? 定位完全不一样。原文说得很白: 这里放的, 是”你读着读着随手要查的东西”。说白了, 它不是让你从头啃到尾的, 而是搁在手边的一本工具书。你读正文卡住了, 或者真动手写内核、调内核的时候撞上某个具体问题, 翻到对应那一页查一下就行。

关键

关键: 附录是用来”哪里不会查哪里”的, 不是用来通读的。第一次学的话, 先把 Parts I-IV 顺顺当当读一遍, 附录就当成踩坑时翻一翻的入口, 够用了。

15.2 三类需求 → 三个去处

附录是按”你想干嘛”来分的——你有什么需求，就告诉你该往哪儿翻。下面这张表把需求和去处一一对上：

你的需求	该翻哪一页
想查某个 TIRx 语言特性 （语法、内建、语义）	TIRx 语言参考 (TIRx Language Reference)
想搞懂 编译器内部实现 ，尤其是 下降流水线 (lowering pipeline)	编译器内部实现 (Compiler Internals)
要调试 异步 GEMM(矩阵乘法，深度学习里最核心的算子) / FlashAttention(高效计算注意力的内核，简称 FA) 内核的 hang(挂死)、crash(崩溃)、结果错误或性能变慢	Warp-Specialized 内核调试 (Debugging Warp-Specialized Kernels)

说白了，这三块对着三种最常见的场景：

- **查语言**：写 TIRx 的时候，忘了某个语法、某个算子怎么用 → 翻语言参考。
- **看原理**：想弄明白”我写的高层算子，最后到底怎么变成 GPU 指令的” → 翻编译器内部实现，重点看 lowering pipeline。
- **修 bug**：异步内核出毛病了 → 翻调试指南。这里说的异步内核，是指那些用到 TMA(异步搬数据的硬件单元，见第 0 章)、warp specialization(让不同 warp 各司其职：有的专搬数据、有的专算)、tcgen05 MMA(矩阵乘加, Tensor Core 的核心指令) 和 TMEM(Tensor Memory, Blackwell 上专给 Tensor Core 用的一块片上内存) 的 GEMM / FA 内核。为什么单拎出来给它一节？因为这种异步、流水线化的内核，是最难调的一类。

澄清一个容易混的点：这本书的 warp-specialized 调试指南，讲的是**跑在 Blackwell(sm_100a, NVIDIA 较新的一代 GPU 架构) 上的内核**。它的矩阵乘用的是 Blackwell 那套 tcgen05 MMA 指令，再配上 TMEM(Tensor Memory)。注意，**这不是 Hopper(上一代架构) 那一代的 warpgroup MMA(wgmma，以 4 个 warp 为一组协同做矩阵乘的指令)**。两者是不同代架构上的不同指令，千万别搞混。

下面用一张 Mermaid 图，把这个”对症下药”的过程画出来：

结构图

连接关系：

- 属于哪类？查 TIRx 语法/特性 → TIRx 语言参考 / TIRx Language Reference
- 属于哪类？理解编译器/下降流程 → 编译器内部实现 / Compiler Internals / (lowering pipeline)
- 属于哪类？异步 GEMM/FA 内核 / hang/crash/算错/变慢 → Warp-Specialized / 内核调试指南
- 属于哪类？完整 tvn.tirx Python API → 上游 TVM 官方文档 / (不在本书内)
- 属于哪类？TIRx 底层语义 / 张量布局模型 → 回到第二部分 / Intro to TIRx + Layout API

注意

注意：图里的 R1/R2/R3，是附录自己的三个子页面。R4(完整 Python API) 和 R5(底层语义与布局) 就不在附录里了，作者只是顺手给你指个路，告诉你”这两该上哪儿找”。

15.3 两个” 不在这里，但你可能要找” 的指针

有两样东西，你读着读着八成会去找，可它们偏偏不在这儿。附录特意提了一句，免得你白翻半天。

1. 完整的 `tvm.tirx` Python API

书里讲 TIRx，只讲关键概念和怎么用。那种完整、一条一条列全的 `tvm.tirx` Python API 文档，书里没有，它放在上游 TVM(upstream TVM) 的官方文档里。

关键

关键：你要是想查” 某个函数的精确签名、全部参数、所有可选项” 这种字典式的细节，该去翻 TVM 官方 API 文档，别在这本书里找。说白了就是分工：书负责把思想讲透，API 文档负责把细节列全。

2. TIRx 原生层 / native level 与张量布局模型 / Layout API

这两块，第二部分 (Part II) 早就讲过了。附录在这儿只是给你指条路，告诉你” 回去看哪儿”，省得你在附录里瞎翻：

内容	在哪讲过
TIRx 的原生/底层语义	第二部分 · TIRx 入门 (Introduction to TIRx)
张量布局模型 (tensor layout model)	第二部分 · TIRx 布局 API(TIRx Layout API)

15.4 建议的使用方式 (阅读顺序)

这是张导航页，它自己没什么” 先读这段、再读那段” 的讲究。真正值得说道的，是怎么把附录揉进你整个学习过程里。给你一套实在的用法：

1. **第一遍学：**顺着主线把 Parts I-IV 读完。这个阶段附录就当” 卡住了再去查” 的入口，不用提前看。
2. **写代码的时候：**TIRx 的某个语法或特性吃不准 → 先翻 **TIRx 语言参考**；想要精确的 API → 直奔 **TVM 官方文档**。
3. **想往深里钻原理：**等正文读明白了，再去看看 **编译器内部实现**，顺着 lowering pipeline 把” 高层算子 → GPU 代码” 这一路走一遍。
4. **真上手调内核：**开始写或调异步 GEMM、FlashAttention 这类 warp-specialized 内核，遇到挂死、算错的时候 → 重点啃 **Warp-Specialized 内核调试指南**。
5. **底层语义或布局忘了：**回 第二部分复习 Intro to TIRx 和 Layout API。

小结

这一章是「参考与附录」的导航页，记住三点就够了：

- 全书的主线在 **Parts I-IV**。附录是**手边随查的工具箱**，哪里不会查哪里，别从头通读。
- 附录按” 需求” 分成三个子页面：**TIRx 语言参考** (查语法特性)、**编译器内部实现** (看 lowering pipeline)、**Warp-Specialized 内核调试** (修异步 GEMM / FA 内核的 hang、crash、算错、变慢)。

- 两样最容易”找不着”的: 完整 `tvm.tirx` Python API 在[上游 TVM 文档](#)里;TIRx 底层语义和张量布局模型在[第二部分](#)。

把这张”对症下药”的导航表记牢, 真动起手来, 你就能很快翻到对的那一页。

延伸阅读

- 原文页面:[Reference —Modern GPU Programming for ML](#) Sys
- 完整 `tvm.tirx` Python API: 见上游 TVM 官方文档 (Apache TVM 项目文档)。
- 相关前置章节: 第二部分「TIRx 入门 (Introduction to TIRx)」与「TIRx 布局 API(TIRx Layout API)」。

术语对照

中文	English
下降流水线	lowering pipeline
编译器内部实现	Compiler Internals
TIRx 语言参考	TIRx Language Reference
Warp-Specialized 内核调试	Debugging Warp-Specialized Kernels
张量布局模型	tensor layout model
TIRx 布局 API	TIRx Layout API
TIRx 原生层	TIRx native level
上游 TVM	upstream TVM
挂死	hang
崩溃	crash

第 16 章 · 调试 Warp 专门化 Kernel

原文: [Debugging Warp-Specialized Kernels](#)

本章要点

本章要点 (TL;DR)

- 先别急着重写 kernel。Warp 专门化 / warp specialization kernel 出运行期故障，十有八九不是算法写错了，而是某一次「交接 / handoff」掉了链子。具体怎么坏的？无非这么几种：barrier 没初始化、到达计数对不上、集体操作被角色守卫挡掉了一半、barrier phase 过期了，或者存储还没用完就被人抢去复用了。
- 动手之前，先填一张「角色 → 存储 → 交接 → 生命周期」四要素表。把这四样东西白纸黑字写下来，再拿生成的 CUDA 一项一项去对。千万别凭感觉乱改同步代码——那样越改越乱。
- 真正算数的是生成的 CUDA，它才是「事实来源 / source of truth」。你在 Python 里写的那些高层守卫 (`wg_id == 0`, `elect_sync()`)，编译一过就变成了一堆 `threadIdx` 算术。所以先用 `inspect_source("cuda")` 把生成代码存下来，grep 一下关键字，再回头去读 Python。
- CUDA 一旦出错，会把整个 context「毒化 / poison」掉。撞上非法内存访问怎么办？别犹豫，直接重启 Python 进程。不然你接着调的根本不是当前代码，而是上一次崩溃留下的残局。
- 症状只是线索，不是结论。先把现象按「死锁 / 崩溃 / 结果错 / 正确但慢」分个类，再对着对应的检查清单一条一条排。还有一句话要刻在脑子里：一次只改一个交接点。

前置知识

前置知识：读这一章前，最好先懂 warp 专门化（把不同 warp 派去干不同的活）、mbarrier / phase（异步屏障和它的相位计数）、以及生成的 CUDA 大概长啥样。没把握的话，先翻一下第 0 章·极简入门，以及第 13 章「用 Warp 专门化和 Cluster 扩展 GEMM」。本章会默认你已经认识这些词。

这一章是《Modern GPU Programming for ML Sys》参考与附录里的一篇「工程实战手册」。它不教新算法，只干一件事：把前面那些复杂的异步流水线 kernel，提炼成一套能反复用、能直接上手的调试套路（配套代码就是「用 Warp 专门化和 Cluster 扩展 GEMM」那一章里 GEMM（通用矩阵乘法，深度学习里最核心的算子）的步骤 7-9）。

这些 kernel 有个共同点：TMA 加载（TMA = 硬件的批量异步搬运引擎，专门在 GMEM 和 SMEM 之间整块搬数据）、tcgen05 的 MMA（矩阵乘加，即 Tensor Core 上的矩阵乘法指令）、还有 TMEM（Tensor Core 专用的累加器内存）/ SMEM（共享内存，一个 block 内线程共用的高速片上内存）写回 / writeback，这三件事是叠在一起 / overlap 跑的，谁也不等谁。好处是快，坏处是——一旦出 bug，因为并发度太高，排查起来格外恼火。不过有个好消息：这套方法不光对 GEMM

好使,Flash Attention(注意力 / attention 的高效实现) 里那几段交接,照样能套上去。

16.1 心法: 把异步 kernel 看成一组「交接」

传统 GPU kernel 你是怎么调的? 基本上就是从头到尾把代码读一遍,逮逻辑 bug。可 warp 专门化 kernel 不能这么干。

它的玩法是这样: 把不同的 warp(线程束,32 个线程的最小调度单位,见第 0 章)/ warpgroup(线程束组,4 个 warp = 128 个线程) 派去干不同的活,也就是分**角色 / role**——一组专门发 TMA 加载,一组专门发 MMA,一组专门做写回。你就把它们想成流水线上的几个工位,各管一摊。工位和工位之间靠什么协调? 靠**异步屏障 / mbarrier**,相当于互相喊话:「我这道工序干完啦,该你接手了。」

关键

关键: 在这种结构里,代码本身经常「怎么看都没毛病」,真正爱出事的地方是**角色之间的交接**。哪怕某个 barrier 的到达计数只写错了一个数,整条流水线都能给你永久卡死。

所以这一章开篇就先立一条规矩:

注意

注意: 别一上来就重写 kernel。顺序应该是这样:(1) 先把运行环境查清楚,(2) 再去查看生成的 CUDA,(3) 最后才回头动 Python。环境和编译这两关一过,剩下的运行期故障基本就跑不出下面这几种「坏掉的交接」了:

- barrier 压根没初始化
- 到达计数写错了
- 集体操作 / collective 被角色守卫挡掉了一部分
- barrier phase 过期了
- 生产者 / producer 刚写的的数据,消费者 / consumer 还没看见呢,存储就被拿去复用了

这条心法记牢了——后面那一堆检查清单,说白了都只是它的展开版。

16.2 调试前: 先排除运行环境

很多事看着像「kernel 的 bug」,其实根本不在 kernel 里——是环境出了岔子。所以这一章的要求很明确: **第一步先把环境查干净**。两条命令就够了:

```
#      TVM                      (          checkout)
python -c "import tvm, tvm.tirx; print(tvm.__file__, tvm.__version__)"

#      GPU                      (      kernel      Blackwell)
python -c "import torch; print(torch.cuda.get_device_name(), torch.cuda.get_device_capability())"
```

为什么偏偏查这两样?

- **这些 kernel 只认 Blackwell(sm_100a)**。(Blackwell 是 NVIDIA 的一代 GPU 架构,比上一代 Hopper、再上一代 Ampere 更新) **tcgen05** 那条路径,非得 Blackwell 这一档的硬件才撑得住。GPU 要不是 Blackwell,代码你怎么改都白搭。

- **Python** 很可能偷偷导入了一个过期的 **TVM**。万一 `tvm.__file__` 指过去的是个旧目录，你还以为早更新过了——那你跑的压根不是刚改的代码。这个坑最阴，经常一搭进去就是好几个钟头打水漂。

关键

关键：环境确认没问题之后，先跑一遍 **kernel 的最小正确性检查**（比如 `run_correctness()`），之后再**去管性能**。先看「对不对」，再操心「快不快」，顺序别反了。

16.3 标准调试流程 (Workflow)

原文给了一条 8 步的工作流，我把它画成了下面这张流程图。它的精神用一句话就能说清：**一步步收窄范围、拿生成代码对照、一次只动一个地方。**

结构图

连接关系：

- 2. 编译是否失败？ $\xrightarrow{\text{是}}$ 检查：已装的 API / target / / dispatch= / buffer 作用域 / (先别看运行期同步代码)
- 2. 编译是否失败？ $\xrightarrow{\text{否}}$ 3. 保存 `inspect_source('cuda')` / grep: 角色守卫 / `mbarrier_init` / / `tcgen05` / `cp.async.bulk.tensor` / `cta_sync()`
- 3. 保存 `inspect_source('cuda')` / grep: 角色守卫 / `mbarrier_init` / / `tcgen05` / `cp.async.bulk.tensor` / `cta_sync()` $\rightarrow$ 4. 为出错路径写出 / 角色/存储/交接/生命周期四要素表
- 4. 为出错路径写出 / 角色/存储/交接/生命周期四要素表 $\rightarrow$ 5. 拿生成的 CUDA 逐项核对该表 / (`barrier init` 在角色分支之前等)
- 5. 拿生成的 CUDA 逐项核对该表 / (`barrier init` 在角色分支之前等) $\rightarrow$ 6. 归类: 死锁 / 崩溃 / / 结果错 / 正确但慢
- 6. 归类: 死锁 / 崩溃 / / 结果错 / 正确但慢 $\rightarrow$ 7. 一次只改一个交接点 / (`init` 计数 / `phase` / 守卫 / `fence` / `drain ...`)
- 7. 一次只改一个交接点 / (`init` 计数 / `phase` / 守卫 / `fence` / `drain ...`) $\rightarrow$ 8. 先重跑正确性，再测性能
- 8. 先重跑正确性，再测性能 $\xrightarrow{\text{仍未解决}}$ 1. 用仍能复现的 / 最小 shape 复现故障

这几步里，有几个细节得拎出来反复念叨：

- **第 1 步：**要是故障是非法内存访问 / `illegal memory access`，下次跑之前务必先**重启 Python**(为啥?16.7 节专门讲)。
- **第 2 步：**编译挂了，先去看 API / target / dispatch / 作用域，**别急着去抠运行期的同步代码**。编译问题和同步问题压根是两码事。
- **第 3 步：**生成的 CUDA 存盘之后，先 **grep 关键字字符串**，再去读 **Python**。这是本章翻来覆去强调的招：把生成代码当成你的「地图」来用。
- **第 7 步：**一次只改一个交接点 (`init` 计数、`arrive/wait` 的 `phase`、角色守卫、`fence`、TMA store 的 `drain`、TMEM 的 `alloc/dealloc`、tile scheduler 的推进，挑一个改)。你要是一口气改好几处，真出了问题，根本分不清到底是哪一处起的作用。

16.4 「四要素表」：可迁移的调试工作表

这是整套方法的**心脏**，也是最该背下来的一段。不管你面前是**哪个**异步 kernel，动手改代码之前，先老老实实在下面这张表填出来：

要素	要写下来的内容
角色 / Roles	每一个异步操作，到底是谁发出来的——哪些线程 / warp / warpgroup / CTA(线程块，一个 block 的线程集合，见第 0 章)。
存储 / Storage	每个 tile(大矩阵切出来的小方块) 走到每一步时「住」在哪儿:GMEM(全局内存，显存，容量大但慢)、SMEM、TMEM，还是寄存器 (register，每个线程私有的最快存储)。
交接 / Handoff	谁生产、谁消费、用的哪个信号对象、到达计数多少、phase 多少，还有那个让数据真正可见的 fence 或 drain。
生命周期 / Lifetime	每个存储槽位，最早什么时候能拿去复用、能读回、能释放。

表填完了，接下来**拿生成的 CUDA 去逐项验**这五件事：

1. **角色守卫**跟你表里的角色对得上。
2. **barrier 初始化**是在被守卫的角色分支**外面** (在它前面)，不是塞在分支里头。
3. **集体操作 / collective** 没被 lane(通道，即线程在 warp 内的编号 0~31)/ warp / warpgroup 守卫不小心收窄。这一类 bug 最隐蔽，后果也最狠。
4. **arrive / wait** 的 phase 跟你的交接表对得上。
5. **TMA store** 的 drain、**TMEM** 的 dealloc、**SMEM** 的复用，统统等到生命周期表点头说「现在可以了」之后才发生。

关键

关键：同一张表，套在 GEMM 的 TMA → MMA → 流水线上行，套在 Flash Attention 的 score → softmax → value → correction 那几段交接上也行 (score = 分数矩阵,Q·K 的结果;softmax = 把分数归一化成概率的那步)。这就是「可迁移」的意思——方法本身一个字不用变，你只要把表里的内容换成对应 kernel 的角色和存储就完事了。

下面这张图，把 GEMM 流水线的三个角色、以及它们之间的交接都画了出来。看一眼，你就能摸到「四要素」在真实 kernel 里到底长啥样：

结构图

分组：

- 分组
- 分组
- 分组

连接关系：

- GMEM (A/B) $\xrightarrow{\text{TMA load}}$ SMEM
- SMEM $\xrightarrow{\text{tcgen05 MMA (累加)}}$ TMEM
- TMEM $\xrightarrow{\text{写回 (结果 D)}}$ SMEM / GMEM

注意

注意: 重点盯紧那几个到达计数。`tma2mma/mma2tma/mma2ld` 都是 **1**(一个引擎或一条 commit 触发一下就够了), 而 `ld2mma` 是 **128**(得等 WG0 全部 128 个线程都到齐)。这些数对不对, 几乎就决定了 **kernel** 会不会死锁 (细节见 16.9 节)。

16.5 当编译失败时

编译期的问题, 要**抢在**运行期同步问题前头解决, 因为这俩根本不是一类 bug。原文给了一张对照表, 我重新理了一下:

症状	大概率出问题的地方	第一步该查什么
报「未知 TIRx API」或属性错误	装的 wheel 跟教程代码对不上	打印 <code>tvm.__file__</code> / <code>tvm.__version__</code> , 拿 API 名字去和「TIRx 语言参考」核一遍。
报「不支持的 dispatch=」	你选的 target 或原语撑不起这条路径	看 <code>dispatch</code> 参数和 target 能力; 本教程的 <code>tcgen05</code> 路径要 Blackwell。
buffer 作用域 (scope) 对不上	有个 buffer 走错了硬件路径	翻工作表的「存储」那行: TMEM 只能走 tcgen05 访问 ,TMA 的操作数 (operand, 指令的输入数据) 得用兼容的 GMEM/SMEM 布局 (layout)。
编译过了, 可生成的 CUDA 里 根本没有 你要的那条路径	<code>dispatch</code> 没按你以为的方式降级	改算法之前, 先去生成的 CUDA 里 <code>grep</code> 一下 <code>tcgen05</code> 和 <code>cp.async.bulk.tensor</code> 。

关键

关键: 最后一行是个超级容易栽的坑。一句话记牢: **编译过了, 不等于生成了你想要的指令**。`dispatch` 完全可能悄没声地拐去一条慢路 (没用 TMA、没用 Tensor Core), 却照样编译成功。所以「去生成的 CUDA 里亲眼确认那条路径在不在」, 这一步是躲不掉的。

16.6 检查生成的代码: 让 CUDA 当地图

这是本章最实用的一个工程招数。把生成的 CUDA 落成一个文件, 你就能对它 `grep`、对它 `diff`——调试效率立马上一个台阶:

```
from pathlib import Path

#         CUDA
cuda_source = ex.mod.imports[0].inspect_source("cuda")
Path("artifacts").mkdir(exist_ok=True)
Path("artifacts/my_kernel.cu").write_text(cuda_source, encoding="utf-8")
print(cuda_source)
```

16.6.1 TIRx 守卫 → CUDA 表达式的对照

关键就在这儿: 你在 Python 里写的那些高层守卫, 过了编译会**变成**一堆具体的 `threadIdx` 算术。你得先把下面这张映射表吃透, 才能在生成的 CUDA 里一眼认出「哦, 这段就是我那个角色守卫」:

TIRx 写法	生成的 CUDA
wg_id == 0	(warp_id_in_cta >> 2) == 0
wg_id == 1	(warp_id_in_cta >> 2) == 1
warp_id == 0	(warp_id_in_cta & 3) == 0
warp_id == 3	(warp_id_in_cta & 3) == 3
lane_id == 0	((int)threadIdx.x) % 32 == 0
.init() 的内部守卫	((int)threadIdx.x) < 1(仅 CTA 的 0 号线程)
elect_sync()	tvm_builtin_elect_one_sync_op()

这张表里，有几处得想明白：

- **wg_id 凭啥用 >> 2?** 因为一个 warpgroup 刚好 4 个 warp。把 warp 号右移 2 位（也就是除以 4），得出来的就是它归哪个 warpgroup。
- **warp_id 凭啥用 & 3?** 取最低两位，看它在 warpgroup 里头排第几（0~3）。
- **.init() 凭啥变成 threadIdx.x < 1?** 它的意思是「只有 CTA 的 0 号线程才去做 barrier 初始化」。这条太关键了——它正好解释了后面 16.9 节那个「barrier init 被 wg_id 守卫坑死」的经典死锁。

16.6.2 读 Python 前，先 grep 这些字符串

在生成 CUDA 里搜	它意味着什么 / 要检查什么
if (threadIdx.x < 1)	单 CTA 线程守卫，通常是 barrier 初始化所在处
mbarrier_init	barrier 初始化存在，且应当出现在角色分支之前
tcgen05	Tensor Core 路径确实生成了
cp.async.bulk.tensor	拷贝确实降级成了 TMA
cta_sync();	CTA 范围的 barrier；它绝不能待在 wg_id 分支里面

注意

注意：这套「先 grep 后读」的路子，等于把一个又大又长的生成 kernel，变成了一张能导航的地图。你根本不用从头啃到尾，顺着这几个锚点（barrier init、角色守卫、tcgen05、TMA、cta_sync）一路摸过去，就能落到出问题的那个交接上。

16.7 何时必须重启 Python

这是一条单独的、但要命的纪律，所以我专门拎出来讲：

关键

关键：CUDA 出了错，不会自己收拾干净。只要撞上一次非法内存访问、XID 错误，或者「CUDA context 被毒化 / poisoned」，那么后面那些跟它八竿子打不着的调用——哪怕你只是来一句 torch.randn——都可能跟着一路报错。

这后果很坑：你要是不重启进程就接着调，那你调的其实是上回崩溃留下的烂摊子，根本不是你眼下这段代码。结果就是你对着一个早修好的 bug 来回折腾，死活想不通。所以——

一碰到非法内存访问 / XID / context 毒化，马上重启 Python 进程，再去试下一个 fix。

16.8 症状地图 (Symptom Map)

拿到一个跑挂的 run，先别忙着下结论，按「现象」快速归个类再说。不过有句话得先打个预防针：

注意

注意: 症状只是线索，不是最终诊断。它顶多帮你圈出「大概是这一片出事了」，真正的根因还得回头靠四要素表，一项一项去对。

线索 (现象)	大概率区域	第一步该查什么
kernel 挂住，随后运行时报「unspecified launch failure」	死锁	barrier init 的位置、到达计数、cta_sync() 的位置、next_tile() 的参与情况
非法内存访问、XID，或之后无关的 CUDA 调用也跟着失败	崩溃 / context 被毒化	重启 Python，再查指针范围、存储生命周期、集体操作参与情况
错误的行以「128 行」或「tile 大小」的条带 (stripe) 出现	同步竞争或 tile 索引错位	生产者/消费者 phase、scheduler 推进、以及哪个 warpgroup 拥有哪个行条带
出现 NaN 或明显非法的值	描述符 / 操作数设置 / 累加未初始化	SMEM/TMEM 描述符设置、swizzle(把数据按特定规律打散摆放以避免 bank 冲突)/layout、累加器初始化
有限但呈规律性的错误值	数据过期或仅部分可见	缺 fence、缺 TMA store drain，或存储在生命周期允许之前被复用
输出正确，但没达到预期加速	dispatch 或资源问题	生成的 CUDA 路径、流水线深度、占用率 (occupancy)、寄存器溢出 (spill)

接下来这几节，就把「死锁 / 崩溃 / 结果错 / 正确但慢」这四类挨个掰开了细讲。

16.9 死锁 (Deadlocks)

死锁是 warp 专门化 kernel 最常常便饭的故障。建议你照下面的顺序，一条一条往下排：

① 到达计数 ≠ 初始化计数 (最常见)

这是头号嫌疑犯。举个典型例子: MBarrier.init(128) 这句话等于放出话来——「我得等满 128 个线程到了才放行」。可实际的 arrive 却被 if warp_id == 0: if lane_id == 0: 这两层守卫给框住了，到头来真正到的只有 1 个线程。这下好了，这个 barrier 永远凑不够 128,wait 也就永远等不到头——卡死。

下面这张表，把 GEMM 里几个 barrier 的「正确计数」列了出来。拿它跟你的 kernel 比一比，看对不对得上：

Barrier	init(计数)	谁来到达	实际到达数
TMABar(tma→mma)	1	TMA 引擎，经 arrive(stage, bytes)	1
TCGen05Bar(mma→tma, mma→ld)	1	MMA warp，经 tcgen05.commit	1
MBarrier(ld→mma)	128	WG0 全部线程，经 arrive	128

关键

关键:128 这个数从哪来的? 它正好是一个 warpgroup 的线程数 ($4 \text{ warp} \times 32 \text{ lane}$)。所以 init 计数是 128 的时候, arrive 必须让 WG0 的全体线程都来执行一遍, 绝不能被 lane 或 warp 守卫拦掉一部分。

② barrier init 被塞进了 wg_id 守卫里

回想一下 16.6 节: `.init()` 会被翻译成 `if threadIdx.x < 1:`, 意思就是只有 CTA 的 0 号线程才去做初始化。而 CTA 的 0 号线程, 偏偏归在 **WG0** 名下。问题就出在这儿:

```
//      :   init      WG1
if (wg_id == 1) {
    mbarrier_init(...); //      ! CTA      0      WG0,
}
```

所以记牢一句: 所有 **barrier init** 都得待在顶层 / **top level**, 别往任何角色守卫里塞。怎么验? 简单——在 `inspect_source()` 里 `grep mbarrier_init`, 看看它们都蹲在什么位置。

③ cta_sync() 蹲进了 warpgroup 分支里

`cta_sync` 说白了就是 `__syncthreads()`, 它要求所有 CTA 线程都到齐。你要是把它放进 `if wg_id == 0:` 里头, 那 WG1 的线程压根走不到这个同步点 → 又死锁了。

关键

关键: 如果你只是想在单个 warpgroup 内部做同步, 用 `T.cuda.warpgroup_sync(10)`, 别拿 `cta_sync()` 来凑。

④ 有些消费者 warpgroup 的线程漏掉了 next_tile()

`tile scheduler` 盯的是每个线程各自的状态 (**per-thread**)。一旦有线程跳过了 `tile_scheduler.next_tile()`, 它的状态就跟别的线程对不上号了, 后果往往是某些线程在那儿一直空转、不出来。

⑤ TMA 和 MMA 对 K-tile 数量没数到一块去

要是 MMA 把循环次数手滑写成了 `K_TILES - 1`(而不是 `K_TILES`), barrier 的 phase 就会一点点跑偏 / **drift**, 跑到第二个外层 tile 时一下就死锁。

注意

注意: TMA 循环和 MMA 循环, 都得老老实实迭代 `K_TILES` 次。哪怕少跑一次, 整条流水线的 phase 节奏就全乱套了。

⑥ PipelineState 的初始 phase 设反了

这里有个设计得相当巧的地方:

- 生产者从 phase=1 起步。这样它第一次 wait 直接放行, 马上就能动手生产。
 - 消费者从 phase=0 起步。这样它第一次 wait 会被卡住, 老老实实等生产者先产出东西来。
- 你要是让这俩从同一个 phase 起步, 那第一次交接当场就可能死给你看。

16.10 崩溃与 context 毒化

崩溃 (非法内存访问、XID 这些) 常见的几个根子:

- pool.commit() 之后又去调了 pool.alloc。注意,barrier 包装器内部也会偷偷调 alloc, 所以分配顺序得严格按下面这个来:

“text tmem_addr → barrier → move_base_to(1024) → Asmem / Bsmem / Dsmem
→ commit() “

只要在 commit() 之后还想再 alloc, 事情就坏了。

- tcgen05.alloc 或 tcgen05.dealloc 上挂了 lane 守卫。发这条指令的那个 warp,32 个 lane 必须一个不少地一起上。你要是写成 if lane_id == 0:, 只放一个线程进去, 那就是未定义行为 / undefined behavior。

关键

关键: 对一下 16.11 节那个 Step 7 骨架——tcgen05_alloc/tcgen05_dealloc 该有 warp 守卫, 但绝不能有 lane 守卫。

- tcgen05.dealloc 之前漏了 cta_sync()。后果是: 写回那边 TMEM 还没读完, 这边就把 TMEM 给释放了——读到的全是垃圾。
- GMEM 或 SMEM 访问越界。把问题缩到单个 tile, 查一查 scheduler 算出来的 m_idx / n_idx, 再确认当前 shape 是 kernel tile(或 cluster tile) 的整数倍。

16.11 Step 7 参考骨架

原文给了一份「能正常编译的 Step 7 kernel」的顶层结构。这东西特别值钱, 你完全可以拿它当「标准答案样板」——把出毛病的 kernel 跟它一段一段地对, 常常一眼就能看出哪儿少了一个守卫、或者哪儿多了一个守卫。下面我把它的骨架摘出来, 一段一段配上中文讲解:

```
// (1) Barrier      :      ,      CTA 0
if (threadIdx.x < 1) {
    mbarrier_init(tma2mma[0..1], 1); // TMA->MMA,      1(TMA      )
    mbarrier_init(mma2tma[0..1], 1); // MMA->TMA,      1
    mbarrier_init(mma2ld, 1);        // MMA->      ,      1
    mbarrier_init(ld2mma, 128);      //      ->MMA,      128(WGO      )
}

// (2) TMEM      :WGO      warp0,      warp      lane      (      lane      )
if (wg_id == 0 && warp_id == 0) tcgen05_alloc(..., 512);

// (3) fence + cta_sync,      phase      :producer=1, consumer=0
```

```
// (4) Warp
if (wg_id == 1 && warp_id == 3 && elect_sync) { /* TMA */ while(valid){ ... next_tile(); } }
if (wg_id == 1 && warp_id == 0 && elect_sync) { /* MMA */ while(valid){ ... next_tile(); } }
if (wg_id == 0)                               { /* WB */ while(valid){ ... next_tile(); } }

// (5)      :          warp,      lane
cta_sync();
if (warp_id == 0) { tcgen05_relinquish_alloc_permit(); tcgen05_dealloc(..., 512); }
```

动手改算法之前, 这四点务必先核一遍:

- 1. **barrier init** 在顶层, 没躲在任何 **wg_id** 守卫里头。
- 2. **tcgen05_alloc / tcgen05_dealloc** 有 **warp** 守卫、但没有 **lane** 守卫——发指令那个 **warp** 的所有 **lane** 都得到场。
- 3. **TMA** 循环和 **MMA** 循环都跑 **K_TILES** 次 (对应 16.9 节 ⑤)。
- 4. **phase** 初始值是 **producer=1、consumer=0**(对应 16.9 节 ⑥)。

关键

关键: 这个骨架, 几乎就是前面所有死锁 / 崩溃检查项的「正确答案合集」。你就把它当成一份看得见摸得着的 **checklist** 来用——你的 **kernel** 每偏离骨架一处, 那一处就先打个问号。

16.12 结果错误 (Wrong Results)

结果算错了, 别瞎猜。先看错误长成什么「样子 / pattern」, 按样子分类, 再倒推原因:

错误形态	大概率指向
整行条带 (whole row stripes) 错	生产者/消费者 phase、tile 索引、或角色归属 (role-ownership) 不匹配
输出 NaN	描述符设置、操作数设置、或累加未初始化
有限但有规律的错值	消费者读到了旧 tile 、部分写入的 tile , 或 store 尚未 drain 的数据

下面这几个是经典坑:

- **tcgen05.commit** 写到了 **elect_sync** 外面。后果很微妙:32 个线程人人都建了一个 **commit group**, 其中 31 个是空组, 而这些空组会立马给 **mbarrier** 发信号。结果就是 **MMA** 还没顾得上读 **SMEM**,**TMA** 那边就可能已经把 **SMEM** 给覆盖掉了。

注意

注意:**commit** 这种「本该由一个线程发」的操作, 一定要圈在 **elect_sync()** 里头, 保证真正去提交的就那么一个线程。

- **TMA store** 之前漏了 **fence.proxy_async("shared::cta")**。这样一来, 线程刚写进 **SMEM** 的数据,**TMA** 引擎可能压根看不见。为啥? 因为不同的内存代理 / **proxy** 之间, 得靠 **fence** 才能让各自的写入互相看得到。
- **TMA store** 之后漏了 **cp_async.bulk.commit_group() + wait_group(0)**。后果是:store 还没 **drain** 完, 下一个 **tile** 就过来把 **Dsmem** 抢去复用了。

- 持久化 kernel / persistent kernel 在小尺寸 (比如 1024×1024) 上偶尔挂掉。这种现象特别能唬人——尺寸一大,K 循环跟着变长, 反倒把竞争给盖住了, 看着一切正常。这时候重点去查 tile 之间的 phase reset, 还有 TMA store 的 commit / wait。

关键

关键: 「小尺寸偶尔挂、大尺寸看着没事」, 这是一个明明白白的竞争条件 / race condition 信号, 而绝不能拿它当「大尺寸代码就没问题」的证据。

- fence.after_thread_sync() 一般不是你寻找的那剂解药。MMA 完成时用的那个 mbarrier, 本身就自带 release-acquire 语义了。步骤 8 和 9 里加它, 纯粹是为了在写回边 / writeback edge 上稳一点——而且位置卡得很死: 在 mma2ld.wait 之后、第一次 tcgen05.ld 之前。千万别手一顺, 就把它也加到 TMA→MMA 这条边上去。

16.13 正确但慢 (Correct but Slow)

输出对是对了, 可性能离预期差一大截, 这时候就掏出前面那套「查生成 CUDA」的老办法, 一模一样的招数往下排:

线索	大概率区域	第一步该查什么
生成的 CUDA 里没有 cp.async.bulk.tensor	拷贝没降级成 TMA	检查 dispatch="tma"、target 能力、操作数 layout
生成的 CUDA 里没有 tcgen05 路径	MMA 没降级成 Blackwell Tensor Core 指令	检查 dispatch="tcgen05"、target 能力、操作数 layout
TMA 和 MMA 没有重叠	流水线太浅, 或 phase 把生产者/消费者串行化了	在生成的 CUDA 里检查 wait/arrive/advance 的顺序
小 shape 正确性好, 但大 shape 速度差	寄存器溢出、占用率、或暂存缓冲 (staging buffer) 压力	查编译器资源报告; 减小 tile、分块写回、或降低流水线深度

注意

注意: 性能问题和正确性问题, 其实用的是同一套家伙——存下生成的 CUDA,grep 关键路径, 核对 wait/arrive/advance 的先后。区别只有一处: 这回你盯的是「重叠够不够」「资源够不够」, 而不再是「交接对不对」。

16.14 提交一个好的 Issue

上面这些都查过了, 故障还赖着不走, 那就先把问题缩到最小, 再去 Apache TVM 的 GitHub 仓库提 issue。一个像样的 issue, 得带上这些东西:

- tvm.__file__ / tvm.__version__ 的输出, 加上 GPU 的计算能力;
- 能复现故障的最小 shape;
- 故障是哪一类: 编译期 / 死锁 / 崩溃 / 结果错 / 正确但慢;
- 最小的那段 kernel 或 notebook cell, 外带它的正确性检查;
- 你存下来的 inspect_source("cuda") 输出, 或者一段能把可疑守卫 / barrier / dispatch 路径亮出来的最小代码片段。

关键

关键: 这一节其实就是整章方法的「收尾」。你前面填的四要素表、存的 CUDA、做的归类——这些刚好就是一个好 issue 要的全部料。换句话说, 把调试纪律做扎实和把 issue 材料备齐, 本来就是同一件事。

小结

这一章不是算法教程, 而是一份专门伺候 warp 专门化异步 kernel 的调试操作手册。它的核心, 一句话拎清: 把 kernel 当成一组「交接」, 用「角色 / 存储 / 交接 / 生命周期」四要素表把它写出来, 再拿生成的 CUDA 一项一项去对。

整章的逻辑兜成了一个圈:

1. 先把环境排掉 (TVM 版本、GPU 是不是 Blackwell), 再跑最小正确性检查。
2. 生成的 CUDA 才算数: 存盘、grep 关键字字符串、对着 TIRx→CUDA 映射表看, 然后才回头读 Python。
3. 绝大多数运行期故障, 都是交接坏了: 到达计数对不上、init 被角色守卫坑了、集体操作被收窄、phase 跑偏或初值写反、存储提前被复用。
4. 先按四类症状归类 (死锁 / 崩溃 / 结果错 / 正确但慢), 对着各自的清单查, 一次只改一个交接点, 改完先重跑正确性。
5. CUDA 出错会毒化 context: 一碰上非法内存访问, 马上重启 Python。
6. 小尺寸偶挂, 八成就是竞争条件, 别让大尺寸那副「看着没事」的样子把你骗了。

最后再唠叨一句: 把 16.11 节那个 Step 7 参考骨架记牢。它差不多就是所有检查项的「正确答案合集」, 你的 kernel 每偏离它一处, 都值得停下来, 好好怀疑一下。

延伸阅读

- 原文章节:[Debugging Warp-Specialized Kernels](#)
- 前置章节: 本书「用 Warp 专门化和 Cluster 扩展 GEMM(Scaling GEMM with Warp Specialization and Clusters)」中的 GEMM 步骤 7-9, 是本章调试方法的直接应用对象。
- 工具参考:TIRx 语言参考 (TIRx Language Reference), 用于核对 API 名称与 dispatch 路径。
- 提交 issue:Apache TVM GitHub 仓库。

术语对照

中文	English
线程束	warp
线程束组	warpgroup
Warp 专门化	warp specialization
角色守卫	role guard
交接	handoff
生产者 / 消费者	producer / consumer
异步屏障	mbarrier (async barrier)
到达计数	arrival count

相位	phase
集体操作	collective operation
共享内存	SMEM (shared memory)
全局内存	GMEM (global memory)
张量内存	TMEM (tensor memory)
线程块	CTA (cooperative thread array)
非法内存访问	illegal memory access
上下文毒化	context poisoning
竞争条件	race condition
寄存器溢出	register spill
占用率	occupancy
持久化 kernel	persistent kernel
流水线深度	pipeline depth

第 17 章 · 编译器内部 (概览)

原文:Compiler Internals

本章要点

本章要点 (TL;DR)

- 先说清楚: 这一章是个**导航页**, 放在「参考与附录」里。它不是写给写 kernel(在 GPU 上跑的那段计算程序) 的人看的, 是写给**想动手改 TIRx 编译器**的人看的。
- 这章现在就指向一个有内容的子页面——**TIRx 下降流水线 / TIRx lowering pipeline**。它只回答一个问题: 你敲下 `tvm.compile(..., tir_pipeline="tirx")` 那一刻, 编译器内部到底干了些什么。
- 「下降 / lowering」是啥? 说白了, 就是把你写的那些高层东西, 一层一层翻译成底层东西。具体来讲, 就是把**瓦片原语 (tile = 大矩阵切出来的小方块, 瓦片原语就是直接操作这些小方块的高层指令)**、带 `TileLayout` 类型的缓冲区、执行作用域 `id` 这几样, 翻译成主机 (host, 即 CPU 端的启动代码) 和设备 (device, 即真正在 GPU 上跑的代码) 两份函数, 最后由 CUDA 后端吐出源码。
- 这套翻译的总入口叫 **LowerTIRx**, 它就干三件事: 把瓦片原语派成具体的后端实现; 把布局算成真正的地址计算; 把作用域 `id` 落到 `blockIdx / threadIdx` 上。
- 这部分最值钱的地方在哪? 它会教你**只跑流水线的前几步, 然后一阶段一阶段地把 IR 打出来看**。这样编译器每一步干了啥, 你是亲眼看见的, 而不是靠脑补。

前置知识

前置知识: 读这一章前, 最好先懂 TIRx 的基础用法 (瓦片原语、缓冲区、`T.copy/T.gemm` 这些写法, 见第 9 章), 以及 `host/device`(主机/设备) 与 `kernel` 启动 (把活儿发到 GPU 上跑) 的概念。没把握的话, 先翻一下第 0 章 · 极简入门。本章会默认你已经认识这些词。

17.1 这一页是什么: 写给贡献者的入口

原文这一页的正文很短, 其实就是一个**目录 (index)**。它在书里归在「第五部分 · 参考与附录」, 任务很单纯: 把你领到「编译器内部」相关的那几个子页面去。

它跟前面 1-14 章不是一路货。前面那些章教你怎么用, 这一章讲的是编译器自己怎么转的。两边一对比就清楚了:

维度	主线章节 (Part I-IV)	本章「编译器内部」(Part V)
读者	想写高性能 GPU kernel 的人	想改 / 扩展 TIRx 编译器本身的贡献者
关注点	怎么用 API、怎么排布数据、怎么调性能	kernel 源码在编译器里 如何被一步步翻译
阅读时机	从头顺着读	用到了再翻

关键

关键: 你就把「编译器内部」当成一份实现笔记 / 贡献者文档来看, 别当教程。不读它, 你照样能写出能跑的 TIRx kernel。可一旦你心里冒出一个念头——”我写的这行 `T.copy`, 最后到底被翻译成啥了?”——或者你想往编译器里塞一个新原语, 那这里就是你该来的地方。

「参考」部分一共三个页面, 分工很清楚。你想干啥, 就翻对应那一页:

你的需求	去哪一页
查某个 TIRx 语言特性	TIRx 语言参考 (TIRx Language Reference)
了解编译器内部 (下降流水线)	编译器内部 (Compiler Internals) ← 本章
调试异步 GEMM(矩阵乘法)/ FlashAttention(一种高效的注意力算子) 的挂起、崩溃、结果错误或变慢	调试 warp(一个 warp = 32 个线程的小班, 见第 0 章) 专业化 kernel(Debugging Warp-Specialized Kernels)

17.2 子页面一览: 目前只有「TIRx 下降流水线」

这一页眼下就列了一个条目:

- **TIRx lowering pipeline / TIRx 下降流水线**

下面给你画张图, 让你一眼看清它在整本书里待在哪儿, 跟旁边那两个参考页又是什么关系。

结构图

分组:

- **第五部分 · 参考与附录:**「TIRx 语言参考 / (查语言特性)」「调试 warp 专业化 kernel / (排查异步 kernel 问题)」

连接关系:

- 子页面:TIRx 下降流水线 $\xrightarrow{\text{讲清楚 tvml.compile 内部}}$ host 函数 $\rightarrow$ C/LLVM / device 函数 $\rightarrow$ CUDA

<p align="center"> 图 17-1: 本章在「参考」部分的位置, 以及它指向的实质内容.</p>

注意

注意: 导航页本身一点技术细节都没有, 真东西全在子页面「TIRx 下降流水线」里。别急着翻过去——下面 17.3 先给你来一份速览, 看完你心里就有数: 要不要点进去深读。

17.3 速览:TIRx 下降流水线在做什么

这一节是子页面内容的**浓缩版导读**, 先帮你抓住主干, 再决定要不要点进去。想看每个 pass 一个一个怎么讲的, 翻本仓库的第 18 章笔记。

一句话定位

你一调用 `tvm.compile(mod, target, tir_pipeline="tirx")`, 后面就连着发生一串事: 你写好的 TIRx 模块, 被丢进一条**排好顺序的 TIR pass 流水线** (也就是 `python/tvm/tirx/compilation_pipeline.py` 里那个 `tirx_pipeline`)。这条流水线把你那些高层写法翻成 `host + device` 两份函数, 再扔给 CUDA 后端生成源码。

整体数据流

结构图

连接关系:

- authored TIRx (你写的 TIRx 模块) $\xrightarrow{\text{BindTarget (绑定目标平台)}}$ `tirx_pipeline` (模块级 pass 序列; 内部由 `SplitHostDevice` 拆出 `host/device`)
- `tirx_pipeline` (模块级 pass 序列; 内部由 `SplitHostDevice` 拆出 `host/device`) $\rightarrow$ `host` 函数
- `tirx_pipeline` (模块级 pass 序列; 内部由 `SplitHostDevice` 拆出 `host/device`) $\rightarrow$ `device` 函数
- `host` 函数 $\xrightarrow{\text{host finalize}}$ C / LLVM
- `device` 函数 $\xrightarrow{\text{device finalize}}$ CUDA

<p align="center"> 图 17-2: 编译先 <code>BindTarget</code> 绑定目标, 再跑模块级 <code>tirx_pipeline</code>(host/device 的拆分发生在流水线内部), 最后按函数种类分别做收尾 (finalization) 并渲染:host 出 C/LLVM,device 出 CUDA.</p>

三个值得记住的关键阶段

整条流水线有十几个 pass(`narrow dtype`、向量化、循环展开、CSE、各种 `fp8/bf16` 合法化等等)。这些你不用全记。想搞懂 lowering 到底是怎么回事, 盯死下面这三步就够了:

关键阶段	它把什么变成了什么
LowerTIRx	整个下降的核心。它里头分两半: <code>TilePrimitiveDispatch</code> 和 <code>LowerTIRxCleanup</code> 。前半把每个瓦片原语 (<code>copy / gemm / reduction ...</code>) 换成你选定的那个后端的具体代码。后半跑 <code>LayoutApplier</code> , 干三件事: 把 <code>TileLayout</code> 类型的缓冲区访问算成真正的地址 (<code>addr = data + elem_offset + layout.apply(coord)</code>); 把缓冲区展平; 再把执行作用域 <code>id(T.cta_id / T.thread_id)</code> 经过 <code>launch_thread</code> 落到 <code>blockIdx / threadIdx(GPU 内置的“我是第几个线程块 / 第几个线程”的编号)</code> 上。
SplitHostDevice	它在 <code>launch_thread</code> 这个边界上下刀, 把一个 <code>kernel</code> 劈成两半: <code>host 函数</code> (管算 <code>grid/block</code> , 即一次启动开多少个线程块、每块多少线程, 然后发起启动) 和 <code>device 函数</code> (真正跑在 GPU 上那个 <code>__global__</code> 体)。
MakePackedAPI	它把 <code>host 函数</code> 改写成 TVM 调用约定要的那套 <code>packed-func ABI</code> 。改完之后, 它就是最终被调起来的那个“启动器”。

关键

关键:LowerTIRx 一跑完, 模块就”掉回”普通 TIR 了。啥意思? 就是说瓦片原语没了, TileLayout 那层间接也没了, 作用域 id 也变成真正的线程轴了。换句话说, 高层那些花活儿到这一步全翻译完了。再往后的 pass, 无非是在普通 TIR 上做些常规优化和合法化, 没别的了。

你能自己把每一步复现出来

子页面里最实用的一招, 就是教你只手动跑流水线最前面那几步, 然后一阶段一阶段把 IR 打出来。书里那些 IR 片段, 就是这么一步步生出来的:

```
from tvm.tirx import transform as TT

target = tvm.target.Target("cuda")
# (host llvm),
mod = TT.BindTarget(target.with_host("llvm"))(tvm.IRModule({"main": scale}))
mod = TT.LowerTIRx()(mod) # :
print(mod.script()) # TIRx IR,
```

要是你想直接看最后生成的 CUDA:

```
# , tirx
exe = tvm.compile(tvm.IRModule({"main": scale}), target=target, tir_pipeline="tirx")
print(exe.mod.imports[0].inspect_source()) # CUDA
```

注意

注意:LowerTIRx()(mod) 这种”单跑一段”的玩法, 是你排查问题、摸清编译器脾气的利器。想看哪一步的结果, 就在那一步前后各 print(mod.script()) 一次, 前后一对比, 变了什么一目了然。

17.4 建议的阅读顺序

1. 先看看你是不是这章的目标读者。要是你只想写写 kernel、调调性能, 这部分先跳过没关系。等哪天你心里冒出”怎么编译出来是这副样子”的疑问, 再回头看也来得及。
2. 从这一页 (本章) 进, 先把一件事搞清楚: 「编译器内部」眼下就讲了下降流水线这一个主题, 别的还没有。
3. 认真读子页面「TIRx 下降流水线」(也就是本仓库第 18 章笔记)。把 pass 表按顺序过一遍, 重点啃这三个关键阶段: LowerTIRx (尤其是里头的 TilePrimitiveDispatch 和 LayoutApplier)、SplitHostDevice、MakePackedAPI。
4. 自己动手跑一遍。拿原文那个就一行的 scale kernel, 自己走一遍 BindTarget → LowerTIRx, 每跑一步就 print(mod.script()), 把脑子里的”概念”跟屏幕上的”真实 IR”一一对上。
5. 要用了再横跳。要排查异步 kernel 的问题, 就转去「调试 warp 专业化 kernel」; 只是查个语言特性, 就转去「TIRx 语言参考」。

小结

- 第 17 章自己就是「参考与附录」里的一个导航页, 写给编译器贡献者, 不是给写 kernel 的人看的。
- 它眼下就指向一个有内容的子页面:**TIRx 下降流水线**, 把 `tvm.compile(..., tir_pipeline="tirx")` 内部那一整套翻译过程讲清楚。
- 下降是怎么回事, 一句话: 把**瓦片原语** + **TileLayout 缓冲区** + **执行作用域 id**, 过一串排好顺序的 TIR pass, 翻成 **host(启动器)** 和 **device(__global__ 体)** 两份函数, 最后渲染成 CUDA。
- 最关键的三个阶段:**LowerTIRx**(派发原语 + 应用布局 + 落地线程轴)、**SplitHostDevice**(主机/设备拆分)、**MakePackedAPI**(打包 ABI)。
- 最实用的一招: **只跑流水线前几步, 再 `print(mod.script())`**, 这样你就能一阶段一阶段”看见”编译器到底在干什么。

延伸阅读

- 原文 (本导航页):<https://mlc.ai/modern-gpu-programming-for-mlsys/tirx_guide/arch/index.html>
- 子页面「TIRx 下降流水线」: 对应本仓库第 18 章笔记; 原书路径在同一 `tirx_guide/arch/` 目录下。
- 完整的 `tvm.tirx` Python API, 请参阅上游 TVM 官方文档。
- 相关参考页: 「TIRx 语言参考」「调试 warp 专业化 kernel」(均在本书「参考与附录」部分)。

术语对照

中文	English
下降流水线	lowering pipeline
瓦片原语	tile primitive
主机函数 / 设备函数	host function / device function
执行作用域 id	execution-scope id
收尾 (阶段)	finalization
打包函数 ABI	packed-func ABI
布局应用器	LayoutApplier
编译目标	target

第 18 章 · TIRx Lowering 流水线

原文:[TIRx lowering pipeline](#)

本章要点

本章要点 (TL;DR)

- 你写的 TIRx 代码很“高层”，但 GPU 看不懂。**TIRx 流水线 / tirx pipeline** 就是一条流水线，负责把它一步步翻译成 GPU 能执行的样子。
- 整条路径很简单：手写的 TIRx → 绑定目标 → 跑流水线 → 把代码拆成 host(主机) 和 device(设备) 两部分 → 各自收尾 → 最终生成 C/LLVM 和 CUDA。
- 流水线一共 **18 个 pass(处理步骤)**，从最核心的 **LowerTIRx** 开始，后面依次做：合并线程绑定、化简、缓冲区扁平化、数据类型合法化、向量化、循环展开、消除重复表达式、拆分 host/device、生成调用接口等。
- 其中 **LowerTIRx** 自己又拆成两小步：第一步把高层算子 (tile 原语) 展开成具体实现，第二步把“数据该放哪”的布局抽象算成真实的内存地址，并把“谁来执行”翻译成 **blockIdx / threadIdx**。
- 一个很实用的点：你可以只跑流水线的前半段（比如只跑到 **LowerTIRx**），然后用 `mod.script()` 把中间结果打印出来看。本书里的 IR 片段都是这么来的。

前置知识

前置知识：读这一章前，最好先懂 TIR / pass(编译里一道道处理步骤)、host/device(主机/设备，即 CPU 端和 GPU 端)、以及 grid/block/thread(网格/块/线程) 这几个概念。没把握的话，先翻一下第 0 章·极简入门，以及第 17 章。本章会默认你已经认识这些词。

18.1 这条流水线解决什么问题

先看看我们手上拿着什么。在 TIRx 里，你写的代码站得很“高”——只说大意，不抠细节。一个 kernel(在 GPU 上由大量线程并行执行的那段函数，见第 0 章)，你拿三样东西就能描述出来：

- 用 **tile 原语 / tile primitive**(`copy`、`gemm`(矩阵乘, GPU 上最常见的核心运算)、`reduction` 这些) 说清楚“要算什么”。
- 用带 **TileLayout** 类型的缓冲区说清楚“数据怎么摆”。
- 用 **执行域 id / execution-scope id**(像 `T.cta_id`、`T.thread_id`) 说清楚“谁来干”。

这么写当然省事，可问题来了：GPU 根本看不懂这套东西。GPU 后端只认**最朴素**的 TIR——具体的线程轴、压成一维的内存访问、它原生支持的那几种数据类型。

那怎么办？这就得靠 **lowering(下降) 流水线**了。它干的活，就是把上面这套高层写法一步步“压”下去，压成 GPU 跑得动的朴素形式。入口特别简单，就一行：

```
tvm.compile(mod, target, tir_pipeline="tirx") #          tirx
```

这一行背后发生了什么？它把你手写的 TIRx 模块塞进 **tirx 流水线**，过一遍那一长串处理步骤，把它劈成 **host(主机)** 和 **device(设备)** 两类函数，最后丢给 CUDA 后端去吐源码。

关键

关键：流水线的定义在 `python/tvm/tirx/compilation_pipeline.py` 里，函数名叫 `tirx_pipeline`。说白了，看懂这条流水线，就等于看懂了“高层的 `tile` 写法是怎么一步步变成 CUDA kernel 的”。它是整个 TIRx 编译栈的主干。

18.2 它处在编译流程的哪个位置

`tvm.compile` 干的活，拆开看就是四步：

1. **绑定目标 / BindTarget:** 先跟模块交代清楚“咱们要往哪种硬件上编”——比如目标是 `cuda,host` 用 `llvm`。后面的步骤得先知道这个，才知道该往哪个方向翻。
2. **跑 tirx 流水线:** 就是 18.3 节要一个个列出来的那一长串步骤。
3. **给 host / device 函数各做各的收尾 (finalization):** 注意,host 和 device 用的是两套不一样的收尾步骤。
4. **把每个 device 函数交给 CUDA 代码生成器**，生成最终源码。

那为什么 host 和 device 要分开收尾？别急，下面这张图先把整条主干画出来，看完你就明白了。

结构图

连接关系：

- `tirx_pipeline` / (18 个模块级 pass) → `host func` / (主机启动器)
- `tirx_pipeline` / (18 个模块级 pass) → `device func` / (设备 kernel)
- `host func` / (主机启动器) $\xrightarrow{\text{host finalize / 主机收尾}}$ C / LLVM
- `device func` / (设备 kernel) $\xrightarrow{\text{device finalize / 设备收尾}}$ CUDA 源码

注意

注意:host 和 device 不是一上来就两家分开的。流水线跑到 **SplitHostDevice** 这一步 (也就是 18.3 节的第 15 个 pass) 之前，整个 kernel 一直是一个函数；到了这一步，它才被“劈”成 host 和 device 两半，然后各回各家、各做各的收尾。收尾为什么要“按函数种类分着跑”，原因就在这。

下面这张图讲的是同一件事，只是画得更紧凑、横着排：

结构图

连接关系：

- `authored TIRx` $\xrightarrow{\text{BindTarget}}$ `tirx_pipeline`
- `tirx_pipeline` → `host func`

- `tirx_pipeline` → `device func`
- `host func` $\xrightarrow{\text{host finalize}}$ `C / LLVM`
- `device func` $\xrightarrow{\text{device finalize}}$ `CUDA`

18.3 流水线的 18 个 pass(逐个看)

`tirx_pipeline` 会严格照下面这个顺序, 一个接一个地把这些步骤施加上去。其中有几个能用 `PassContext` 的开关关掉 (用到了会说)。先拿一张表整体扫一眼, 不用记, 有个印象就够了。

#	Pass 名称	它做什么	备注
1	LowerTIRx	核心下降——把 <code>tile</code> 原语和布局抽象解析掉 (详见 18.4 节)	流水线的“重头戏”
2	UnifyThreadBinding	合并等价的线程轴绑定, 保证每个 <code>threadIdx</code> / <code>blockIdx</code> 轴只声明一次	去重
3	StmtSimplify	语句级算术化简 (由 <code>arith</code> 分析器 / <code>arith analyzer</code> 完成)	第一次化简
4	LowerTIRxOpaque	把剩余的 <code>不透明 (opaque)</code> TIRx 结构下降为朴素 TIR	收尾性下降
5	FlattenBuffer	把多维的 <code>BufferLoad</code> / <code>BufferStore</code> 扁平化为一维	多维 → 1D
6	BF16ComputeLegalize	把 <code>bf16</code> 计算改写成合法形式 (上转为 <code>f32</code> 再算)	计算合法化
7	NarrowDataType(32)	在可证明安全处, 把索引/循环的 <code>PrimExpr</code> 数据类型收窄到 32 位	省寄存器/算力
8	VectorizeLoop	把 <code>T.vectorized</code> 循环变成向量操作	配 <code>tir.disable_vectorize</code> 时跳过
9	UnrollLoop	展开标了 <code>T.unroll</code> 的循环 (以及小的常量循环)	循环展开
10	StmtSimplify	再次化简——此时向量化/展开已暴露出新的常量	第二次化简
11	CommonSubexprElim	公共子表达式消除 (CSE), 把重复子表达式提到临时变量	配 <code>tir.disable_cse_tir</code> 时跳过
12	FP8ComputeLegalize	把 <code>float8</code> 计算改写成合法形式	计算合法化
13	VerifyMemory	安全闸门: 检查 <code>host</code> 端代码没有直接解引用 <code>device</code> 内存	校验, 不改写
14	AnnotateEntryFunc	把那个唯一的 <code>PrimFunc</code> 标记为模块入口	标注
15	SplitHostDevice	在 <code>launch_thread</code> 边界把每个 <code>kernel</code> 拆成 <code>host</code> 函数 + <code>device</code> 函数	关键分水岭
16	MakePackedAPI	把 <code>host</code> 函数改写成打包函数 (<code>packed-func</code>) ABI, 即 <code>TVM</code> 调用的启动器	生成调用接口

17	FP8StorageLegalize	合法化 float8 的存储 (打包进受支持的容器类型)	存储合法化
18	BF16StorageLegalize	合法化 bfloat16 的存储	存储合法化

几个”为什么这么排”的小细节

这串步骤的排法可不是随手摆的。挑几个最值得琢磨的点说说:

- **化简 (StmntSimplify) 为什么来两遍?**(第 3、第 10) 不是手抖写重了。第一遍,是把 LowerTIRx 翻完之后留下的一堆算术清理干净;第二遍,是专门收拾向量化 (第 8) 和循环展开 (第 9) 之后又冒出来的那些常量和能化简的结构。说白了就是: **先把东西摊开,再统一打扫一遍**——编译器排步骤,常用这个套路。
- **数据类型合法化为什么拆成”计算”和”存储”两拨,中间还隔得老远?** bfloat16 / float8 的计算合法化 (第 6、第 12) 放在前头,是因为改写计算会带出新的算术,得留给后面的化简、向量化接着收拾。而**存储合法化** (第 17、第 18) 压到了**最后**,等 host/device 拆完才动手——存储打包这事更贴近物理内存布局,属于收尾性质的活,搁最后最合适。
- **NarrowDataType(32)(第 7)** 是个性能优化。GPU 上算索引,只要 32 位够用,就能省寄存器、跑得更快。但它守着一底线: **只有在能证明安全的时候才收窄**。不然万一溢出,地址就算错了,直接越界。这就是典型的”能优化就大胆优化,但兜底必须有、还得证得出来”。
- **VerifyMemory(第 13)** 只看、不动手。它是一道安全闸门 (safety gate)。趁 host/device 还没拆开,它就盯一件事: host 代码有没有违规地直接去碰 device 内存。为什么非得放在 SplitHostDevice 前面? 就为了**早点把问题拦下来**,别等出了事才追悔。
- **有两个步骤能用开关关掉**: VectorizeLoop(对应 tir.disable_vectorize) 和 CommonSubexprElim(对应 tir.disable_cse_tir), 都能靠 PassContext 跳过。调试的时候,或者你想对比一下”开和关到底差在哪”,这俩开关特别趁手。

Finalization(收尾)pass——按函数种类分别跑

前面提过, SplitHostDevice 把函数劈成 host 和 device 两类。劈完之后, 收尾阶段会给它俩各上一套不一样的步骤:

函数种类	收尾 pass 序列	说明
host(主机)	LowerTVMBuiltin → LowerIntrin	先下降 tvvm_* 内建函数 (builtin), 再下降目标特定的内建指令 (intrinsic)
device(设备)	LowerWarpMemory → StmntSimplify → LowerIntrin	先把线程束作用域 (warp-scoped, warp 即一组通常 32 个一起执行的线程, 见第 0 章) 缓冲区下降成 shuffle 指令, 再化简, 最后下降内建指令

关键

关键: LowerWarpMemory 这一步是 device 端独有的。它把”warp 作用域的缓冲区”翻成 warp 内的 shuffle 指令。shuffle 是个啥? 说白了, 就是同一个 warp 里的线程在寄存器 (register, 线程私有、最快的那层存储) 这一层直接互相递数据, 不用先写进共享内存 (SMEM, 块内线程共用的片上内存, 比寄存器慢但比全局内存快)、再读出来绕一圈, 所以快。这是 GPU 上很常

见的高效数据交换招数。host 端没这一步，因为 host 上压根就没有 warp 这回事。

18.4 深入 LowerTIRx: 核心下降是怎么发生的

第 1 个步骤 LowerTIRx 是整条流水线的”重头戏”，绝大部分翻译活儿都堆在这里。它自己其实又是两个小步骤接在一起跑，代码在 `src/tirx/transform/lower_tirx.cc`:

```
# LowerTIRx                                pass
LowerTIRx = Sequential([ TilePrimitiveDispatch, LowerTIRxCleanup ])
```

这两步分工很清楚:

(1) TilePrimitiveDispatch——把高层算子换成具体实现

你写的每个 `copy`、`gemm`、`reduction`(底层都是 `TilePrimitiveCall`)，其实只是个”调用”，并没有真身。这一步干的，就是把每一个这样的调用，换成它在当前硬件上的真实函数体。具体两件事:

- **变体选择 / variant selection:** 同一个算子，换块硬件可能有好几种实现写法，这一步挑出最合适的那个 (比方说 `gemm`，架构不同，走的指令路径就可能不同)。
- **代码生成 / codegen:** 把选中的那个实现展开成真正的 TIR 语句。

打个比方，它就像把一个函数调用直接 `inline` 展开，塞进”这台机器上跑得最快的那段代码”。

(2) LowerTIRxCleanup——用 LayoutApplier 把抽象落地

这一步会跑 `LayoutApplier`(布局施加器)。前面那些虚的、抽象的东西，到这儿就得全变成实打实的细节。它做三件事:

1. **把布局访问换成真实的地址计算:** 你访问一个带 `TileLayout` 的缓冲区，只写了个坐标; 这一步把这个坐标算成具体的内存地址。核心公式是:

“`text addr = data + elem_offset + layout.apply(coord)`”

公式不用背，记住是三样东西相加就行: 基址 `data`(数据打哪儿开始)+ 元素偏移 `elem_offset`(这块缓冲区在大数组里的起点)+ `layout.apply(coord)`(布局函数拿你给的坐标算出来的那个偏移)。一句话: `TileLayout` 那层”绕来绕去”的抽象，在这一步被彻底拆光，变成干干净净的地址加法。

1. **缓冲区扁平化:** 把多维缓冲区压成一维，这样就能直接按地址去寻。
1. **把执行域 id 变成真实线程轴:** `T.cta_id`、`T.thread_id` 这些，靠 `launch_thread` 翻成 GPU 真认识的 `blockIdx`、`threadIdx`。

下面这张图，把 LowerTIRx 这两步”吃进去什么、吐出来什么”画清楚了:

结构图

连接关系:

- 手写 TIRx: / tile 原语 + `TileLayout` 缓冲区 + 执行域 id → `TilePrimitiveDispatch` / 变体选择 + `codegen` / (原语 → 具体实现)
- `TilePrimitiveDispatch` / 变体选择 + `codegen` / (原语 → 具体实现) → `LowerTIRxCleanup` / `LayoutApplier` / 布局 → 地址算术 / 缓冲区扁平化 / `scope id` → `block-`

Idx/threadIdx

- LowerTIRxCleanup / LayoutApplier / 布局 → 地址算术 / 缓冲区扁平化 / scope id → blockIdx/threadIdx → 朴素 TIR: / 无 tile 原语 · 无 TileLayout 间接 · scope id 已解析为线程轴

关键

关键:LowerTIRx 一跑完, 模块就成了**朴素 TIR**——tile 原语没了,TileLayout 那层抽象没了, 执行域 id 也全落成了真实线程轴。后面那 17 个步骤对着的, 都是这种”已经落了地”的 TIR。也正因为前面把 TIRx 特有的那些东西都消化干净了, 后面才能直接拿 TVM 现成的通用步骤 (化简、向量化、CSE 这些) 来用, 不用再单独为 TIRx 另写一套。

18.5 一个完整的小例子:scale kernel 的下降全程

前面概念铺得有点多, 这里拿一个最简单的例子, 把它们都串起来——一个只有一行计算的缩放 (scale)kernel, 看它怎么一路变成 CUDA。

起点: 手写的 TIRx

```
@T.prim_func
def scale(A_ptr: T.handle, B_ptr: T.handle):
    A = T.match_buffer(A_ptr, (256,), "float32")          #           256           f32
    B = T.match_buffer(B_ptr, (256,), "float32")
    T.device_entry()                                       #
    bx = T.cta_id([1])                                     #           id:1       CTA( )
    tx = T.thread_id([256])                                #           id:256
    B[tx] = A[tx] * T.float32(2.0)                         #           :
```

看一眼此刻的状态:bx、tx 还是**抽象的执行域 id**, 还没变成真实线程轴;A、B 也还是带类型的缓冲区视图。接下来, 咱们就盯着这几样东西, 看它们怎么一步步落地。

经过 LowerTIRx 之后:scope id 变成真实线程轴, 布局已施加

```
with T.launch_thread("blockIdx.x", 1) as blockIdx_x:      # cta_id -> blockIdx.x
    threadIdx_x = T.launch_thread("threadIdx.x", 256)      # thread_id -> threadIdx.x
    bx: T.let = blockIdx_x                                  #           id
    tx: T.let = threadIdx_x
    B_1[threadIdx_x] = A_1[threadIdx_x] * T.float32(2.0)    # A_1/B_1           1D
```

对比一下, 变了三处: 执行域 id 被 **launch_thread** 裹成了真实线程轴;A、B 压成了一维视图 **A_1、B_1**; 而计算逻辑本身一动没动, 还是”乘以 2”。前面念叨的那个”落地”, 这会儿全都兑现了。

经过 SplitHostDevice + MakePackedAPI 之后: 一个函数裂成两个

```
@I.ir_module
class Module:
    def main(...):          # host:      API      (      grid/block,      launch)
```



```
...
def scale_kernel(...): # device: GPU __global__
```

原来那个 `scale` 函数, 现在劈成了两个:

- `main(host)`: 在 CPU 上跑, 负责把网格 (grid, 所有块的总集合) 和块 (block, 即 CTA, 一组线程, 见第 0 章) 的配置算好, 然后发起 kernel 启动。它对外用的是 TVM 的打包函数调用接口。
- `scale_kernel(device)`: 真正在 GPU 上跑的那部分, 马上就要被生成成 `__global__` 函数。

终点: CUDA 后端生成源码

最后一步, CUDA 后端把 `scale_kernel` 生成成 `__global__` 函数, 那行核心计算就成了地道的 CUDA:

```
B_ptr[threadIdx.x] = A_ptr[threadIdx.x] * 2.0f;
```

下面这张竖着排的时间线, 把”这一行代码从头走到尾”完整串了一遍:

结构图

连接关系:

- 手写 $TIRx / B[tx] = A[tx] * 2.0 / (tx \text{ 是抽象 } thread_id)$ $\xrightarrow{\text{LowerTIRx}}$ 朴素 TIR / $B_1[threadIdx_x] = A_1[threadIdx_x] * 2.0 / (\text{scope id 已落地, 缓冲区已扁平})$
- 朴素 TIR / $B_1[threadIdx_x] = A_1[threadIdx_x] * 2.0 / (\text{scope id 已落地, 缓冲区已扁平})$ $\xrightarrow{\text{SplitHostDevice + MakePackedAPI}}$ 拆成两个函数 / `main(host 启动器)` + `scale_kernel(device)`
- 拆成两个函数 / `main(host 启动器)` + `scale_kernel(device)` $\xrightarrow{\text{CUDA codegen}}$ CUDA 源码 / $B_ptr[threadIdx.x] = A_ptr[threadIdx.x] * 2.0f$

18.6 自己动手复现每个阶段

这条流水线有个特别好用的地方: 你用不着一口气跑完, 完全可以只跑前面一段, 然后用 `mod.script()` 把中间结果打印出来瞧瞧。本书里所有 IR 片段, 都是这么来的。学 lowering 也好, 调 lowering 也好, 这都是最趁手的家伙。

只跑到 LowerTIRx, 看看下降后的 TIRx IR 长啥样

```
from tvn.tirn import transform as TT

target = tvn.target.Target("cuda")
# (host llvm),
mod = TT.BindTarget(target.with_host("llvm"))(tvn.IRModule({"main": scale}))
mod = TT.LowerTIRx()(mod) # tile
print(mod.script()) # TIRx IR
```

注意

注意: `target.with_host("llvm")` 的意思, 是给 CUDA 目标再配一个 host 目标 (这里用 LLVM)。为啥要配? 因为后面 `SplitHostDevice` 拆出来的那个 host 函数, 就得编译到这个 host 目标上去跑。

一口气编译整个模块, 直接看生成的 CUDA

```
exe = tvm.compile(
    tvm.IRModule({"main": scale}),
    target=target,
    tir_pipeline="tirx",          #          tirx
)
print(exe.mod.imports[0].inspect_source()) #      CUDA
```

这两段对照着用: 前一段看”逻辑落地后的中间态”, 后一段看”最终的 CUDA 长啥样”。两头一对照, 一个 kernel 从高层 DSL 到 CUDA 源码, 中间每一跳你就全看明白了。

提示

提示: 想搞清楚某一个步骤到底干了啥? 有个简单招——把流水线分别截到这个步骤的”前”和”后”, 各打印一次, 然后一 diff, 哪儿改了一目了然。再配上 18.3 节那几个开关 (比如把向量化或者 CSE 关掉), 你就能精准抓出某个变换到底起了什么作用。

小结

本章把 **TIRx Lowering 流水线** 从入口 `tvm.compile(..., tir_pipeline="tirx")` 一路讲到了 CUDA 源码。回头捋一遍:

- **整体位置:** `BindTarget` → `tirx_pipeline` → `host/device` → `C/LLVM` → `CUDA`。
记住一个关键: `host` 和 `device` 是在 `SplitHostDevice` 这一步分的家, 分完之后各走各的收尾。
- **18 个步骤的排法是有讲究的:** 核心下降 `LowerTIRx` 打头阵; 中间穿插着两轮化简、分两拨做的数据类型合法化 (计算 / 存储)、偏性能的类型收窄和向量化/展开, 还有那道只看不改的安全闸门 `VerifyMemory`; `SplitHostDevice` 则是把单函数劈成 `host/device` 的分水岭。
- **LowerTIRx 是整章的核心:** `TilePrimitiveDispatch` 把高层算子换成硬件上的具体实现, `LayoutApplier` 把 `TileLayout` 抽象算成 `addr = data + elem_offset + layout.apply(coord)` 这样的真实地址, 顺带把执行域 id 落成 `blockIdx / threadIdx`。它一跑完, 模块就成了朴素 TIR。
- **可复现:** 整条流水线都支持”只跑前面一段 + `mod.script()` 看中间结果”。这既是书里那些 IR 片段的来源, 也是你以后调 lowering 的标准做法。

说到底, 这条流水线最值钱的地方在于: 它把”你写的声明式 tile DSL”和”GPU 真正跑的 CUDA kernel”之间那道看着挺玄乎的鸿沟, 拆成了一串看得懂、能单跑、能逐段检查的机械步骤。玄乎劲儿一散, 剩下的全是你能动手验证的细节。

延伸阅读

- 原文: [TIRx lowering pipeline —Modern GPU Programming for MLSys](#)

- 流水线定义源码:python/tvm/tirx/compilation_pipeline.py(tirx_pipeline)
- LowerTIRx 实现源码:src/tirx/transform/lower_tirx.cc

术语对照

中文	English	说明
下降 / 降级	lowering	把高层结构逐步翻译为低层、贴近硬件形式的过程
流水线	pipeline	一条有序的编译 pass 序列
tile 原语	tile primitive	高层算子调用, 如 copy / gemm / reduction
执行域 id	execution-scope id	如 T.cta_id / T.thread_id, 描述”谁执行”
主机 / 设备	host / device	CPU 端启动器与 GPU 端内核
收尾	finalization	拆分后对 host/device 分别施加的末段 pass
打包函数 ABI	packed-func ABI	TVM 调用 kernel 所用的标准调用接口
公共子表达式消除	CSE (Common Subexpression Elimination)	把重复子表达式提取为临时变量
数据类型合法化	data type legalize	把 bf16/fp8 等改写为硬件可执行的合法形式
类型收窄	narrow data type	在安全前提下把索引/循环类型降到 32 位
布局施加器	LayoutApplier	把 TileLayout 解析为物理地址算术的组件
线程束作用域内存	warp-scoped memory	由 LowerWarpMemory 下降为 shuffle 的缓冲区
线程束	warp	GPU 中一组 (通常 32 个) 同步执行的线程
共享内存	SMEM (shared memory)	块内线程共享的片上内存
块 / 线程块	CTA (Cooperative Thread Array)	即 CUDA 的 block
安全闸门	safety gate	只校验不改写的 pass, 如 VerifyMemory

第 19 章 · TIRx 语言参考 (概览)

原文:[TIRx Language Reference](#)

本章要点

本章要点 (TL;DR)

- 一句话: 这一章是「TIRx 语言参考」那一组页面的**导读**。它自己不讲细节, 只告诉你一件事——写 TIRx 设备内核 / device kernel 的时候, 想查某个特性到底怎么写、是什么语义, 该翻哪一页。
- 这些参考页是从第二部分的「TIRx 入门」里单独抠出来的。说白了, 它们是**速查手册 / reference**, 不是**教程 / tutorial**。
- TIRx 语言被拆成了五块: 解析器工具、数据类型与表达式、缓冲区与内存、控制流、CUDA C++/PTX 内建函数。
- 这五块刚好对着你写内核时一层层往下走的五个关注点: **元编程** → **标量数据** → **内存对象** → **控制结构** → **直达硬件**。建议先照这个顺序扫一遍, 之后写代码时哪儿卡了再回来查。
- 它属于“参考与附录”这一部分的子页面。主线内容都在 Parts I-IV, 这一组只管“把细节列清楚”。

前置知识

前置知识: 读这一章前, 最好先懂线程层级 (warp / CTA / grid)、共享内存与寄存器这些基础内存概念, 以及 TIRx 内核大概长什么样。没把握的话, 先翻一下第 0 章 · 极简入门, 以及第 9 章的 TIRx 基础。本章会默认你已经认识这些词。

19.1 这一页是什么

这一页是一组**语言参考**的入口。你写 TIRx 设备内核会用到的所有语言特性, 原文都从第二部分那篇偏教学的「TIRx 入门」里抽了出来, 整理成几页速查文档。

为什么要这么拆? 因为入门那种一步步讲故事的方式, 适合你**第一次理解概念**; 可一旦你已经在写内核, 只想确认一个很具体的小问题, 再去翻教程就太慢了。比如:**T.unroll** 到底是完全展开, 还是只是顺序跑一遍循环? 动态共享内存 / dynamic shared memory 是不是只能分配一块?**elect_sync()** 最后选中的到底是哪个线程? 把这些答案做成手册式的参考页, 就是让你**遇到问题一查就有**。

关键

关键: 这一组页面是拿来“按需查”的, 不是拿来“从头读到尾”的。建议先快速扫一遍, 心里有个数——每页大概讲什么 (本章就帮你干这件事), 之后写代码卡住了, 再精确翻回对应那页。

19.2 五个子页面分别讲什么

原文列了五个子页面。下面一个一个说它们讲什么，各配一个最典型的例子。看完你心里就有谱了：碰到哪类问题，该去翻哪一页。

子页面	一句话定位	你会在这里查到
解析器工具 (Parser utilities)	解析期 (parse-time) 的元编程工具	把 Python 值内联进 IR、内联函数、打包解析期状态
数据类型与表达式	每个表达式的低层 dtype 与高层 type	dtype 到 CUDA 类型的映射、向量类型、指针 (handle)
缓冲区与内存 (Buffers & memory)	内核里所有内存对象的声明与语义	各种 scope(global/shared/local/tmem)、视图、标量、let、张量内存 (TMEM)
控制流 (Control flow)	if / 循环家族 / while	四种循环、统一 (uniform)vs 发散 (divergent) 控制流的坑
CUDA C++/PTX 内建函数	直达硬件的两个逃生口 (escape hatches)	T.cuda.* / T.ptx.* 内建、内联原始 CUDA、同步语义

下面一块块展开说。

解析器工具：解析期的元编程

先说这一页要解决什么：有些事你希望在”代码还没真跑、只是被解析”的阶段就办掉，比如提前把循环展开、把一个函数整个铺开。这一页讲的就是干这些事的工具。它们都在 TVMScript 被解析成 TIRx 的那一刻起作用，不会冒出任何运行时代码，纯粹是在解析期 / parse-time 做元编程 / metaprogramming。

- **T.meta_var(x)**: 你在 Python 里先把一个值算好，然后用它把这个值当成编译期的”元值 / meta value”，直接塞进 IR。它最妙的一点是：拿这种值去写普通的 Python for，循环会在解析期就自动展开 / unroll。

```
n = T.meta_var(4)           # n           Python int
for j in range(n):          #             4
    acc[0] = acc[0] + A[tx, j]
```

- **@T.inline**: 定义一个函数，让它在每个调用的地方都被内联。也就是说，生成的代码里根本看不到真正的函数调用，而是把函数体直接摊开。它走的是 Python 的 LEGB 词法作用域和延迟绑定那一套。
- **@T.meta_class**: 给一个普通 Python 类打个标记，意思是”它的实例就是一个解析期元值”。这个类的字段里可以放缓冲区和标量。用它你能把内核流水线 (pipeline，把多步操作重叠起来跑，见第 0 章) 的状态 (屏障、累加器 (accumulator，一边算一边把结果累加进去的寄存器)、scratch 视图) 打包成一个对象，免得在内核里到处撒一堆零散的局部变量。
- **T.constexpr**: 标记编译期的内核参数，然后由 @T.jit 的 .specialize(...) 把它烘焙进去 (细节看「TIRx 入门」)。

注意

注意: 这一整页就一个关键词——**解析期**。这些工具弄出来的东西，你在最终的内核里是看不见的：要么被内联了，要么被展开了，要么被当成常量烘进去了。

数据类型与表达式:dtype 与 type 两套体系

先记住一句话:TIRx 里每个表达式都同时背着**两层类型信息**。一层管”底下到底是哪些二进制位”，另一层管”逻辑上这是个什么玩意儿”。

- **低层 dtype**: 就是 `PrimExpr.dtype`, 管的是”是什么位 / what bits”。比如 `float32`、`float16`、`bfloat16`、`int32`、`bool`, 低精度的 `float8_e4m3fn` / `float4_e2m1fn`, 向量形式的 `float32x4`, 还有指针 `handle`。每个 `dtype` 都对应一种 CUDA 类型。
- **高层 type**: 标量写成 `PrimType(dtype)`, 指针写成 `PointerType(PrimType(dtype), scope)`。这一层主要在你**跟指针打交道**的时候才用得上, 平时基本不用管它。

这一页最该记的, 是这张 **dtype → CUDA 类型映射表** (用中文重排如下):

TIRx dtype	CUDA 类型
float32	float
float16	half
bfloat16	nv_bfloat16
int32	int
uint8	uchar
bool	signed char
float32x4	float4(向量)
handle	T*(指针)

向量 `dtype`(比如 `float32x4`) 好在哪? 它有两种玩法。一种是直接声明一个 `float4` 寄存器 (register, 每个线程私有的最快存储, 见第 0 章)(`T.alloc_local((1,), "float32x4")`), 然后用 `v[0]` 索引); 另一种是配合 `vload/vstore`, 把 16 字节当成**一次宽访问**一口气搬过去。这里有个点: 向量 `dtype` 不是只能跟 `vload` 搭配——随便哪个缓冲区或标量都能带上它。

再说**指针 (handle)**。缓冲区的 `data` 指针是个**不可变 / immutable**的 `Var`, 所以怎么拿到它是有讲究的。常规两种办法: 要么让 `alloc_buffer` 在分配的时候顺手把它定义好, 要么用 `decl_buffer(data=ptr)` 复用**一个现成的指针**。但有一种情况得特别处理: 如果你想让缓冲区指向一个**指针表达式** (比如 `T.ptx.map_shared_rank` 算出来的、另一个 cluster(一簇协作的 CTA, 见第 0 章) 里 CTA(线程块, 见第 0 章) 的共享地址), 那就得先用 `T.let` 把这个表达式绑成一个**指针 Var**, 再拿去用。

缓冲区与内存: 内核里所有”内存对象”的字典

这是五页里**最厚的一页**, 因为内核里几乎啥都是缓冲区。它讲清楚三件事: 两套声明 API、各个内存空间 (scope), 还有一堆”看着像数据、其实只是元数据”的视图和标量概念。

两个根 API:

API	干什么
<code>T.alloc_buffer(shape, dtype, scope=ê)</code>	真正分配新存储 (发出 <code>AllocBuffer</code> 节点); <code>T.alloc_shared</code> / <code>T.alloc_local</code> 是它带 <code>scope="shared"</code> / <code>"local"</code> 的简写
<code>T.decl_buffer(shape, dtype, data=ê)</code>	在 已有指针上声明一个视图 (不分配), 用来别名/重解释存储

内存空间 (scope):

scope	简写	对应内存
"global"	(默认)	设备全局内存

"shared"	T.alloc_shared	静态共享内存 (__shared__)
"shared.dyn"	(用 pool)	动态共享内存 (按启动参数定大小)
"local"	T.alloc_local	每线程寄存器
"tmem"	(TMEM pool)	Blackwell(NVIDIA 最新一代 GPU 架构) 张量内存 (TMEM,Tensor Core 专用的片上内存)

这一页有几个反复出现的要点, 先记在心里:

- 缓冲区说白了就是”挂在指针上的一层元数据”。同样逻辑上去访问 B[i,j], 你只要改一下 B 的 layout 或 elem_offset, 生成的地址算术就完全变了 (行主序是 i*8+j, 列主序是 j*4+i, 带偏移是 i*8+j+64·……)。可底下那个源数据指针从头到尾压根没动过。
- 动态共享内存, 整个内核只能分配一块。这一块你就当成一块”竞技场 / arena”。剩下所有想用动态共享内存的缓冲区, 都得用 decl_buffer(data=arena.data, elem_offset=ê) 在这块地里开视图。换句话说, 写两遍 alloc_buffer(scope="shared.dyn") 是错的。T.SMEMPool 就帮你把这套”竞技场记账”自动化成了一个 bump 分配器。
- 标量 / scalar 其实就是”单元素 local 寄存器缓冲区”的语法糖。你写 phase: T.int32 = 0, 跟你老老实实写 T.alloc_local((1,), "int32") 再用 [0] 索引, 解析完得到的 TIRx 结构一模一样。前者无非是让你少写一个 [0]。
- let 和标量是两码事, 别搞混。T.let 是个不可变绑定 (降下去就是一个普通 C 变量, 不是数组)。正因为它不可变, 算术分析器才能把它的常量界、可整除性、取值范围这些事实带到每一个用到它的地方, 顺手帮你化简索引、砍掉越界检查、决定要不要向量化。而可变量量呢, 本质上是一次内存 load, 上面这些好处一样都带不过去。
- 张量内存 / TMEM 是个特例。它得用 warp-uniform 的 T.ptx.tcgen05.alloc/dealloc 显式申请和释放, 而且不能直接寻址——读写只能走 tcgen05 的 mma/ld/st/cp。每个张量都用 decl_buffer(scope="tmem", allocated_addr=< >, layout=ê) 开一个视图。这一整套流程,T.TMEMPool 都替你封装好了。

这一页末尾还有一张 Buffer 方法表 (data / ptr_to / vload / vstore / view / local / permute / access_ptr)。这些方法基本都是编译期的重排或重解释: 要么就改改索引算术, 要么递给你一个指针, 本身不会产生任何运行时操作。

控制流:if、循环家族、while

这一页很短, 因为 TIRx 的控制流”长得跟 CUDA 差不多”:Python 的 if/else 直接变成 CUDA 的 if/else,break/continue 在循环里照样能用。真正值得专门查的只有两件事:

循环有四种风味:

写法	含义
T.serial(n)	顺序循环 (普通 range 就降成它;ptxas 仍可能自行展开)
T.unroll(n)	完全展开成直线语句, 生成代码里没有循环
T.vectorized(n)	向量化循环
T.grid(*extents)	嵌套循环巢

统一 / uniform 控制流 vs 发散 / divergent 控制流——这是最容易踩的坑, 也是这一页真正想敲打你的地方。

关键

关键: 先记住一句话——**集合操作 / collective op**, 必须让它要同步的每一个线程”齐刷刷地”都执行到。比如 `T.cuda.cta_sync()` 会映射成 `__syncthreads()`, 它要求线程块里所有线程都走到这一行。要是你把它塞进 `if wg_id == 0`: 这种只有部分 `warpgroup` 才进得去的分支里, 那别的 `warpgroup`(一组 `warp` 的集合, 常见是 4 个 `warp` = 128 线程) 永远到不了这一行, 内核就**死锁 / deadlock**(大家互相干等、谁也走不下去) 了。如果你只是想让某一个 `warpgroup` 内部同步一下, 那就改用 `warpgroup` 作用域的 `T.cuda.warpgroup_sync(id)`。

同样的话也适用于屏障初始化。`mbarrier`(memory barrier, 放在共享内存里、让线程和异步引擎互相等待的硬件屏障) 的 `.init()` 会降成一个单线程守卫 (`if (threadIdx.x < 1)`)。你要是再把它套进另一个发散分支里, 屏障就可能**压根没被初始化**, 结果就是莫名其妙的启动失败。

还有个小技巧: 如果你只是想做**表达式层面的”二选一”**(而不是真的要走某一条分支), 那就用 `T.if_then_else(cond, a, b)`。它降下去就是 C 的三元运算符, **不会引入控制流发散**。

CUDA C++/PTX 内建函数: 两个直达硬件的逃生口

有时候现成的 `tile`(把大矩阵切出来的小方块) 原语满足不了你的需求, 这时候这一页给了你两条”逃生口 / escape hatch”, 让你直接够得着硬件:

- 1. **调后端内建**。用 `T.cuda.* / T.ptx.*` 这两个命名空间 (来自 `tvm.backend.cuda`), 直接把设备内建暴露给你——同步、`mbarrier`、归约都有, 还有 PTX 那一大家子数据搬运 / MMA(matrix multiply-accumulate, Tensor Core 上的矩阵乘加指令)(`cp_async`、`cp_async.bulk.tensor` 也就是 TMA(Tensor Memory Accelerator, 专门成块搬运张量的硬件引擎)、`ldmatrix/stmatrix`、Blackwell 的 `tcgen05.*`、`atomic_add`、`fence` 等等)。
- 2. **内联原始 CUDA**。那些连内建都没有的东西, 就用 `T.cuda.func_call(name, *args, source_code=ê, return_type=ê)`, 把一段 `__device__` 源码原样注进去, 再把调用接上。

举个例子, 用 `warp shuffle` 在 `warp`(一个 `warp` = 32 个线程的小班, 锁步执行, 见第 0 章) 内做一次全归约, 核心就一行:

```
# __shfl_xor_sync    TIRx    ,
v[0] += T.tvm_warp_shuffle_xor(0xFFFFFFFF, v[0], i[0], 32, 32)
```

它会直接降成 `__shfl_xor_sync(0xFFFFFFFF, v_ptr[0], i_ptr[0], 32)`。

这一页还专门讲了**四类同步语义**。它们反复出现, 而且一旦用错, 通常不是给你报个错, 而是静默崩坏或者死锁, 所以值得单独记一记:

机制	关键语义 / 坑
Mbarrier 相位 / phase	<code>try_wait(bar, phase)</code> 会一直卡着, 直到屏障内部的相位 不等于 你传进去的 <code>phase</code> 。要是你跨循环复用同一个屏障, 那每次等完都得把你本地的相位跟踪器翻一下 (<code>phase ^= 1</code>); 不然后面的等待会立刻返回, 引擎可能读到只写了一半的内存
选举 / election	<code>T.ptx elect_sync()</code> 选中的是一个 <code>warp</code> 里的某个活跃 lane (通道, 即线程在 <code>warp</code> 内的编号 0-31)——它既不是 <code>lane 0</code> , 也不是”每 CTA 一个线程”。你要是想精确收到”恰好一个线程”, 还得再加一道 <code>if warp_id == 0</code> : 这样的 <code>warp</code> 级守卫

具名 warpgroup 屏障	<code>cta_sync()</code> 要求整个 CTA 都到齐; 做了 <code>warpgroup</code> 特化之后, 就改用 <code>warpgroup_sync(id)</code> 。硬件一共给了 16 个具名屏障 (ID 0-15), 不同 <code>warpgroup</code> 得用不同的 ID (比如 <code>warpgroup_sync(wg_id + 10)</code>), 免得撞到同一个硬件屏障
栅栏 / fence	用来给”生产者写、消费者 (往往是异步引擎”读” 这件事 排个先后: <code>fence.proxy_async("shared::cta")</code> 、 <code>fence.mbarrier_init()</code> 、 <code>tcgen05.fence.after_thread_sync()</code> 各管一条排序边

注意

注意: 这四类同步是 GEMM(通用矩阵乘法,GPU 上最核心的计算) 和 FlashAttention 内核里的”常客”。它们管的是**异步引擎和并行线程组**, 所以一旦用错, 通常不会给你报错, 而是直接**静默数据损坏或死锁**。这也正是作者为什么要单独写一章「Warp-Specialized 内核调试」。

19.3 建议的阅读顺序

这五页虽说是速查手册, 但作者排它们的顺序其实藏着一线索: **写内核的时候, 你的关注点会一层一层往下走**。所以第一次速览, 照下面这个顺序过一遍最顺:

结构图

连接关系:

- 2. 数据类型与表达式 / 标量 / dtype / 指针 → 3. 缓冲区与内存 / 内存对象 / scope / 视图
- 3. 缓冲区与内存 / 内存对象 / scope / 视图 → 4. 控制流 / if / 循环 / 同步守卫
- 4. 控制流 / if / 循环 / 同步守卫 → 5. CUDA/PTX 内建 / 直达硬件 + 同步语义

1. **解析器工具**: 先搞明白”解析期能做哪些元编程”, 因为后面好多代码都靠它来展开和内联。
2. **数据类型与表达式**: 把”标量数据”的底子打牢——dtype、向量类型、指针。
3. **缓冲区与内存**: 进到”内存对象”这一层。这页分量最重, 也是你回头查得最勤的一页。
4. **控制流**: 有了数据和内存, 再来看怎么把执行流程组织起来, 尤其得记牢”集合操作必须齐刷刷到达”这个坑。
5. **CUDA C++/PTX 内建**: 最后是直达硬件的逃生口和四类同步语义——写 GEMM/FA 的时候你也最常翻回这一页。

关键

关键: 速览过一遍以后, 就别再”从头读到尾”了。它真正的用法是: 代码写到哪儿卡住了, 就跳回对应那页查精确语义。

小结

本章是「TIRx 语言参考」这组页面的导读, 核心就三点:

- 它是从「TIRx 入门」里抽出来的**语言速查手册**，作用是”查精确写法和语义”，不是再把教程重讲一遍。
- 它把 TIRx 语言切成五块，刚好对着写内核的五个层面：**解析器工具（解析期元编程）**→ **数据类型与表达式（标量/dtype/指针）**→ **缓冲区与内存（内存对象/scope/视图）**→ **控制流（if/循环/同步守卫）**→ **CUDA/PTX 内建（直达硬件 + 四类同步语义）**。
- 它的用法是**按需查**：先按上面的顺序速览一遍，在脑子里画一张地图；之后写代码碰到具体问题，再跳回对应那页。其中你回头翻得最多的是两页——「缓冲区与内存」（最厚）和「CUDA/PTX 内建」（同步语义的坑最多）。

延伸阅读

- 原文页面:[TIRx Language Reference —Modern GPU Programming for MLSys](#)
- 前置教程：第二部分的「TIRx 入门 / Introduction to TIRx」和「TIRx 布局 API / TIRx Layout API」。
- 完整的 `tvm.tirx` Python API: 见上游 TVM / Apache TVM 的官方文档。
- 相关实践：想看四类同步语义的实际用法，可以读 Part III 的 GEMM 内核，以及「Warp-Specialized 内核调试」那一章。

术语对照

中文	English
设备内核	device kernel
解析器工具	parser utilities
解析期	parse-time
元编程	metaprogramming
元值	meta value
内联函数	inline function
数据类型 / 标量类型	dtype
向量类型	vector dtype
指针	handle / pointer
缓冲区	buffer
内存空间	scope
视图	view
静态共享内存	static shared memory
动态共享内存	dynamic shared memory
竞技场	arena
寄存器	register
标量	scalar
不可变绑定	let / immutable binding
张量内存	tensor memory (TMEM)
控制流	control flow
统一控制流	uniform control flow
发散控制流	divergent control flow
集合操作	collective op
死锁	deadlock
内建函数	intrinsic
逃生口	escape hatch
相位	phase

选举	election
栅栏	fence

第 20 章 · 解析器工具 (Parser utilities)

原文:Parser utilities

本章要点

本章要点 (TL;DR)

- 本章讲四个工具: `T.meta_var`、`@T.inline`、`@T.meta_class`、`T.constexpr`。它们有个共同点: 都在解析时 / **parse time** 干活。也就是说, 它们出力的那一刻, 是「TVMScript 翻译成 TIRx IR」的瞬间, 而不是 GPU 真正跑代码的时候。说白了, 它们就是 TIRx 的「元编程 / metaprogramming」入口。
- `T.meta_var(x)`: 把一个你用 Python 当场算好的值直接塞进 IR, 当编译期常量使。最典型的玩法, 是让一个普通的 Python for 循环在解析阶段就被展开 / **unroll**。
- `@T.inline`: 定义一个特殊的函数, 它会在每个调用点就地展开。换句话说, 生成的 IR 里压根看不到函数调用, 只剩替换进去的函数体。它遵循 Python 的 LEGB 作用域和延迟绑定 / **late binding** 规则。
- `@T.meta_class`: 给一个普通 Python 类贴个「解析期元对象」的标记。它的实例字段能装下 buffer、标量这些 IR 对象, 于是你就能把一组相关的分配和状态打包成一个对象, 不用再拎着一堆零散的局部变量到处传。
- `T.constexpr`: 标记一个编译期的 kernel 参数, 由 `@T.jit` 的 `.specialize(...)` 在编译时把它「烤」进 kernel。本章只点个名, 细节看《TIRx 入门》。

前置知识

前置知识: 读这一章前, 最好先懂 TIRx / TVMScript 的基本写法, 以及 buffer、SMEM(共享内存)、scope(内存作用域) 这些概念。没把握的话, 先翻一下第 0 章 · 极简入门, 以及第 9 章 TIRx 基础。本章会默认你已经认识这些词。

一、先搞清楚: 什么叫「解析期工具」

要听懂这一章, 先得在脑子里摆一条时间线。你写好一段 TIRx kernel, 到它真正在 GPU 上跑起来, 中间得过三道关:

结构图

连接关系:

- 你写的 TVMScript / (带 `@T` 装饰的 Python) $\xrightarrow{\text{① 解析 (parse)}}$ TIRx IR / (结构化中间表示)
- TIRx IR / (结构化中间表示) $\xrightarrow{\text{② 下降 (lower) / 编译}}$ GPU 上执行的机器码 / (SASS / PTX)
- GPU 上执行的机器码 / (SASS / PTX) $\xrightarrow{\text{③ 运行 (runtime)}}$ GPU 上执行的机器码 / (SASS

/ PTX)

- ① 解析阶段 / parse time: 解析器读你写的那段 Python(也就是 TVMScript), 把它翻译成 TIRx IR。注意, 这一步是在 Python 进程里跑的——你的代码这时候是被当成数据来处理的。
- ② 下降 / 编译阶段: LowerTIRx 这类 pass 上场, 把 IR 一步步落实现成具体的 GPU 指令。
- ③ 运行阶段: 代码真正在 GPU 上跑。

关键

关键: 本章这四个工具, 全都活在阶段 ①。它们不会变成 GPU 上的任何一条指令。它们只在「Python 翻译成 IR」这一瞬间动手——要么把一个 Python 值塞进 IR, 要么趁翻译的时候提前把循环或函数展开掉, 要么把一堆状态打成一个包。

说白了, 这就是「元编程」: 拿 Python 这门宿主语言, 去操控、去生成另一门语言 (TIRx IR) 的代码。你只要记住一句话——「这些事全发生在解析器里」, 后面四节就都顺了。

那它们各自治什么病? 主要是下面这三类:

工具	解决的痛点
T.meta_var	想把 Python 算出来的常量直接用进 IR, 又不想多写一个没用的临时变量; 想让循环在解析期展开
@T.inline	想把一段重复逻辑抽成函数复用, 又不希望 IR 里真出现一次函数调用
@T.meta_class	kernel 里状态太多 (屏障、累加器、scratch view……), 不想把十几个局部变量一路穿下去
T.constexpr	想要一个编译期就固定下来的 kernel 参数 (如 tile 尺寸), 让编译器据此特化

二、T.meta_var 一把 Python 值 inline 进 IR

2.1 它在做什么

T.meta_var(x) 其实就是在跟解析器交代一句: 「x 是个编译期的 meta 值, 你直接把它 inline 进 IR 就行, 别当成普通的『脚本变量』去解析。」

这里的 x, 是你在 Python 这一层就已经算好的值, 比如一个 Python int。那加不加 T.meta_var, 到底差在哪? 对比一下你就懂了:

- 不加: 解析器可能把 n = 4 看成 IR 里的一次变量赋值, 于是真给你生成一个 IR 变量节点。
- 加了 T.meta_var(4): 这下 n 就只是个 Python 整数 4。后面哪儿用到 n, 解析器就把 4 这个字面值直接填进去——IR 里根本没有 n 这么个变量。

```
n = T.meta_var(4)           # n      Python int 4,      inline
for j in range(n):          #        meta    ->
    acc[0] = acc[0] + A[tx, j]
```

2.2 最重要的副作用: 驱动循环展开

T.meta_var 最值钱的本事, 就是能让循环在解析期直接被展开。

为什么? 关键全在 `range(n)` 里那个 `n` 身上。`n` 是个实打实的 **Python 值** (不是 IR 变量), 所以 `for j in range(n)` 不过是一个最普通的 **Python for 循环**。解析器翻译的时候, 会老老实实把这个 Python 循环跑一遍: 转一圈, 就吐一条对应的 IR 语句。这么一圈一条, 循环就被摊平了。

还拿上面那段代码举例, 它等价于解析器生成了这么一段 IR(意思是这样):

```
acc[0] = acc[0] + A[tx, 0]
acc[0] = acc[0] + A[tx, 1]
acc[0] = acc[0] + A[tx, 2]
acc[0] = acc[0] + A[tx, 3]
```

下面这张图, 把「普通的运行期循环」和「meta 值驱动的解析期展开」并排一放, 差别一眼就出来了:

结构图

分组:

- **普通 IR 循环变量**: 「`for j in range(N)` / `N` 是 IR 变量」「IR 里保留一个 / 真实的 for 循环节点」「运行期在 GPU 上 / 循环执行 `N` 次」
- **meta_var 驱动展开**: 「`n = T.meta_var(4)` / `n` 是 Python int」「`for j in range(n)` / 就是 Python for」「解析器跑一遍 Python 循环 / 逐圈生成 IR 语句」「IR 里没有循环, / 只剩 4 条展开后的语句」

连接关系:

- IR 里保留一个 / 真实的 for 循环节点 → 运行期在 GPU 上 / 循环执行 `N` 次
- `for j in range(n)` / 就是 Python for → 解析器跑一遍 Python 循环 / 逐圈生成 IR 语句
- 解析器跑一遍 Python 循环 / 逐圈生成 IR 语句 → IR 里没有循环, / 只剩 4 条展开后的语句

关键

关键: 一个 `for` 会不会在解析期展开? 判断起来特别简单——就看它的**循环上界 (还有循环里用到的那些值)** 是不是 **meta 值**。而 `T.meta_var` 干的, 正是把一个 Python 值「升格」成 meta 值的那个开关。上界一旦是 meta 值, 这个 `for` 就退化成一次普通的 Python 迭代, 解析器照着它一条一条把语句铺出来就完事了。

2.3 为什么这么设计

- **省掉那种「用一次就扔」的局部变量**: 很多时候你不过是想用一个 Python 算出来的常量 (比如 `tile_k = 64`, 这里的 `tile` 就是大矩阵切出来的小方块, 见第 0 章), 犯不着在 IR 里专门给它立一个变量节点。`T.meta_var` 能让这个值不留痕迹地融入 IR。
- **把 Python 和 IR 的元编程打通**: 它把 Python 算出来的结果, 干干净净地注入到 IR 的生成过程里。循环展开只是最常见的一种用法而已。说到底, 只要你想让「解析器把某个值当成已知常量来处理」, 都能用它。

三、@T.inline — 内联函数

3.1 它在做什么

先给个直觉:@T.inline 让一个函数「写的时候像函数, 出来的时候像手写」。它修饰的函数, **函数体会在每个调用点就地铺开 (也就是 inline)**。换句话说, **生成的 IR 里一个函数调用都看不到**, 看到的只是把函数体「抄」过去、再把形参换成实参之后的那几行语句。

```
@T.inline
def add_into(acc, x):
    acc[0] = acc[0] + x

add_into(acc, A[tx, j])      #      -> acc[0] = acc[0] + A[tx, j]
```

调一次 `add_into(acc, A[tx, j])`, 效果跟你亲手写 `acc[0] = acc[0] + A[tx, j]` 一模一样: 形参 `acc` 换成实参 `acc`, 形参 `x` 换成实参 `A[tx, j]`, 然后整个函数体铺到调用点上, 就这么回事。

3.2 作用域规则:LEGB + 延迟绑定

原文专门强调:@T.inline 遵循 Python 的 LEGB 词法作用域, 而且采用延迟绑定 / late binding。这俩词听着唬人, 其实都是 Python 里现成的规矩, 一个个拆开看就明白了:

- **LEGB**: 就是 Python 找名字的标准顺序——Local(局部)→ Enclosing(闭包外层)→ Global(模块全局)→ Built-in(内建)。内联函数体里用到的名字, 就照这个顺序一层层往外找。
- **延迟绑定**: 名字是真正用到它的那一刻才去解析, 不是定义的时候就钉死。
- **形参会盖住外层的同名变量 (shadowing)**: 正因为走 LEGB, 内联函数的形参名要是碰巧跟外层某个变量撞名了, 那在函数体里, 这个名字指的就是形参 (Local 这一层级优先级最高), 外层那个同名的就被挡在外面了。

注意

注意: 这条规矩在实战里很要紧。举个例子: 外层有个变量叫 `x`, 你的内联函数形参也叫 `x`, 那函数体里的 `x` 用的是**传进来的实参**, 不是外层那个 `x`。这正是它和朴素宏 (macro) 的区别——宏只是把函数体原封不动粘到调用点, 一不留神就会误碰外层的同名变量。`@T.inline` 有作用域规则兜底, 行为更像一个真函数, 只是省掉了运行期的调用开销而已。

下面这张图, 把「@T.inline」和「真实的函数调用」摆一块儿看:

结构图

连接关系:

- 写法相同, 都长得像函数 / `add_into(acc, A[tx, j])` → @T.inline 版 / 解析期就地展开
- 写法相同, 都长得像函数 / `add_into(acc, A[tx, j])` → 真实函数调用版 / IR 里保留一个 call 节点
- @T.inline 版 / 解析期就地展开 → IR: `acc[0] = acc[0] + A[..]` / (无 call, 无栈帧)
- 真实函数调用版 / IR 里保留一个 call 节点 → IR: `call add_into(acc, A[..])` / (运行期有调用开销)

3.3 为什么这么设计

- **既能复用，又零开销**：你想把重复的逻辑抽成函数，让代码好读、好复用；可又不想在底层 kernel 里背上函数调用的开销（在 GPU kernel 里，函数调用往往不便宜，还碍着编译器优化）。`@T.inline` 正好两头都顾上——写着像函数，出来像手写。
- **比宏靠谱**：它不是死板的文本替换，而是带着 Python 作用域语义的展开。所以同名遮蔽它能处理对，宏式展开里那些经典的坑也就绕过去了。

四、@T.meta_class —解析期状态对象

4.1 它在做什么

先说它治什么病：kernel 里东西一多，你就得拎着一大把零散的局部变量满世界跑。`@T.meta_class` 就是来帮你「装箱」的。

具体来说，它给一个普通 Python 类贴个标记：这个类的实例是解析期的 meta 值。它最关键的能耐是——实例的字段 (field) 能装下 buffer（一块带类型和形状的连续内存区，见第 0 章）、标量这些 IR 对象。这么一来，你就能把一组相关的分配 (allocation) 和状态打包进一个对象，然后在 kernel 体里像用普通 buffer 那样，直接用它的字段。

```
@T.meta_class
class State:
    def __init__(self, smem):
        self.acc = T.alloc_local([1], "float32") # local buffer
        self.buf = T.decl_buffer([64], "float16", smem, scope="shared.dyn") # SMEM

s = State(smem.data)
s.acc[0] = T.float32(0.0) # buffer
# ... s.buf[i] ... #
```

仔细看 `State`，它就是个再普通不过的 Python 类：`__init__` 里调用 TIRx 的分配 API (`T.alloc_local`、`T.decl_buffer`)，把结果存进 `self.acc`、`self.buf`。那 `@T.meta_class` 这个装饰器干了啥？它让解析器把 `State` 的实例当成「解析期的状态容器」。于是实例的字段就能装得下 IR 对象，在 kernel 体里也能被正确引用。

4.2 为什么这么设计：别再「穿线」一堆局部变量

一个真正快的 kernel，流水线（把搬数据和算数据错开重叠起来跑，见第 0 章）上的状态多得吓人：

- 好几个 `mbarrier`（屏障，让线程互相等齐、对上步调的同步原语）；
- 一堆累加器（`accumulator`，边算边把结果累加进去的那块寄存器/buffer）；
- 各种 `scratch` 视图 / 中间 buffer（临时用来暂存中间结果的草稿空间）；
- 还有阶段计数、`phase` 标志这类标量。

不打包的话，你就得把这十几样东西全当局部变量，一路从外层「穿」到内层，再从内层函数传给下一个内联函数。结果呢？参数列表越拖越长，又容易出错，又难读。`@T.meta_class` 的思路是：这些逻辑上本来就是一伙的状态，干脆全收进一个对象里。

结构图

分组:

- 不打包: 穿线一堆局部变量: 「kernel 体」「acc」「buf」「barrier0」「barrier1」「phase」「参数列表越来越长」
- 打包: 一个 `meta_class` 实例: 「kernel 体」「s = State(...)」「s.acc」「s.buf」「s.barriers」「s.phase」「接口干净」

连接关系:

- kernel 体 → acc
- kernel 体 → buf
- kernel 体 → barrier0
- kernel 体 → barrier1
- kernel 体 → phase
- phase $\xrightarrow{\text{逐个传进各内联函数}}$ 参数列表越来越长
- s = State(...) → s.acc
- s = State(...) → s.buf
- s = State(...) → s.barriers
- s = State(...) → s.phase
- s = State(...) $\xrightarrow{\text{只传一个对象}}$ 接口干净

关键

关键: `@T.meta_class` 给你的是**条理**, 不是性能。它本身不会冒出任何运行期对象——实例只活在解析期。它治的是「kernel 代码怎么组织」这个病: 把散落一地的流水线状态拢成一个有名字的整体, 让 kernel 体读起来像在操作一台清清楚楚的状态机, 而不是在一堆裸 buffer 里转晕。

4.3 和 `@T.inline` 的搭配

`@T.meta_class` 和 `@T.inline` 是天生一对。你可以写一组 `@T.inline` 的「方法」辅助函数, 让它们都接收同一个 `State` 实例 (比如 `add_into(s, ...)`、`advance(s)`)。内联展开之后, 这些函数既复用了逻辑, 又共享了同一份状态, 接口还特别干净。

五、`T.constexpr` —编译期 kernel 参数

`T.constexpr` 用来标记一个**编译期**的 kernel 参数。这个参数会被 `@T.jit` 的 `.specialize(...)` 在编译时「烤」进 (bake in) kernel。

原文在本章只点了个名, 扔下一句话: 完整细节看《TIRx 入门 (Introduction to TIRx)》。不过照着本书一贯的路子, 我们先把它定位讲清楚:

- 普通的 kernel 参数, 是**运行期**才定的——每次调用都能传不同的值。
- 而打了 `T.constexpr` 标记的参数, 在 `.specialize(...)` 那一刻就定死了。这相当于跟编译器说: 「在这个特化 (specialize) 出来的 kernel 版本里, 这个值就是个常量。」

- 既然成了编译期常量，编译器就敢放手做更狠的优化：常量折叠、循环展开、砍掉分支，样样都行；还能拿它去驱动布局和调度上的决策。这也正是它跟本章前几个工具气味相投的地方——都是「把某个值在更早的阶段就钉死，好让后面生成的代码更优」。

注意

注意:`T.constexpr` 起作用的时机，跟前三个工具不太一样。`T.meta_var`、`@T.inline`、`@T.meta_class` 都在**解析期**动手；而 `T.constexpr` 挂在 `@T.jit` 的 `.specialize(...)` 这套编译期特化机制上。但骨子里的精神是一回事——「决策能提前就提前」，只是落地的位置不同罢了。具体语义以《TIRx 入门》为准，本章只是把它收进「解析器 / 编译期工具」这一家子里点个名。

六、四个工具的横向对比

把这四件工具塞进同一张表，谁负责干啥就一目了然了：

工具	作用对象	生效时机	核心效果	典型场景
<code>T.meta_var(x)</code>	一个 Python 值	解析期	把值 <code>inline</code> 进 IR；让 <code>for</code> 在解析期展开	用 Python 常量驱动循环展开
<code>@T.inline</code>	一个函数	解析期	函数体在每个调用点就地展开,IR 里无 <code>call</code>	抽取可复用逻辑且零调用开销
<code>@T.meta_class</code>	一个 Python 类	解析期	实例字段可装 <code>buffer</code> /标量，打包状态	聚合屏障/累加器/ <code>scratch</code> 等流水线状态
<code>T.constexpr</code>	一个 kernel 参数	编译期 (<code>.specialize</code>)	把参数烤成编译期常量	<code>tile</code> 尺寸等编译期固定的配置

这四个工具其实共用一条思路，一句话就能串起来：

能提前确定的东西，就趁早定下来——不管是值 (`meta_var` / `constexpr`)、函数展开 (`inline`)，还是状态怎么组织 (`meta_class`)。这样既让 IR 更干净、生成的 GPU 代码更快，又不用丢掉 Python 这门宿主语言的元编程表达力。

小结

- 本章这四个工具，都是 TIRx 的**解析期 / 编译期元编程入口**。它们干的都是「在 TVMScript → TIRx IR 这条翻译链上做文章」，本身不对应 GPU 上的任何一条运行期指令。
- `T.meta_var(x)`: 把 Python 算出来的值升格成编译期 `meta` 值，再 `inline` 进 IR。最常见的效果，是让 `for j in range(n)` 这类循环在解析期就被**展开**成一条条铺平的语句。
- `@T.inline`: 函数体在每个调用点就地展开,IR 里看不到函数调用。它遵循 Python 的 **LEGB 词法作用域和延迟绑定**，形参能正确盖住外层的同名变量，所以比朴素的宏更靠谱。
- `@T.meta_class`: 把普通 Python 类的实例变成「解析期状态容器」，字段能装下 `buffer` / 标量。你可以拿它**打包**屏障、累加器、`scratch` 视图这些流水线状态，省得在 kernel 体里拎着一堆零散局部变量到处穿。它和 `@T.inline` 一搭，就能写出特别干净的「带状态辅助函数」。

- **T.constexpr**: 标记编译期的 kernel 参数, 由 `@T.jit` 的 `.specialize(...)` 烤进 kernel, 细节看《TIRx 入门》。
- 全章就一条主心骨: 能提前定的值、展开、状态组织, 就趁早定下来。这样既留住了 Python 的元编程能力, 又让生成的 IR 和 GPU 代码更精简、更快。

延伸阅读

- 原文页面:[Parser utilities —Modern GPU Programming for ML](#) Sys
- **T.constexpr** 与 `@T.jit` 的 `.specialize(...)` 细节: 见本书第二部分「TIRx 入门 (Introduction to TIRx)」。
- 元编程与循环展开的更广背景: 可结合本书 TIRx 语言参考的其余章节一起看。

术语对照

中文	English
解析期 / 解析时	parse time
元编程	metaprogramming
元值 / meta 值	meta value
内联 (就地展开)	inline
循环展开	unroll
词法作用域	lexical (LEGB) scoping
延迟绑定	late binding
遮蔽 (同名覆盖)	shadowing
编译期常量	compile-time constant
特化	specialize
累加器	accumulator
屏障	mbarrier / barrier
中间暂存 (视图)	scratch (view)

第 21 章 · 数据类型与表达式

原文:[Data types and expressions](#)

本章要点

本章要点 (TL;DR)

- 在 TIRx(TVM 的 TensorIR 扩展) 里, 每个表达式身上都挂着两层类型: 下面一层 `dtype` 说的是”这堆是什么位”, 上面一层 `type` 说的是”这是个值还是个指针”。
- `dtype` 跟 CUDA 的标量或向量类型一对得上, 比如 `float32` $\rightarrow$ `float`、`float16` $\rightarrow$ `half`、`bfloat16` $\rightarrow$ `nv_bfloat16`、`float32x4` $\rightarrow$ `float4`。生成代码的时候, 它说了算——最后落成哪个 C++ 类型, 就看它。
- 向量 `dtype`(像 `float32x4`) 是头等公民。你拿它声明一个寄存器, 再配一次 16 字节的读写 (`vload/vstore`), 向量化访存就成了。但要记住一句: 向量这个属性是 `dtype` 自己带的, 跟 `vload` 没关系。
- 上面那层 `type`, 大多数时候就是 `PrimType(dtype)`(也就是标量), 你基本不用管。它真正派上用场, 是碰到指针 (`handle`) 的时候——指针得写成 `PointerType(PrimType(dtype), scope)`。
- `buffer` 的指针 `data` 是个改不了的 (`immutable`)`Var`。要是你想让 `buffer` 指向一个算出来的指针表达式 (比如跨 CTA 去够另一个块的共享内存地址), 那就得先用 `T.let` 把它绑成一个 `Var`, 绕一道。

前置知识

前置知识: 读这一章前, 最好先懂 GPU 的几个基本”地盘”——寄存器、共享内存、CTA(线程块)、集群 (`cluster`), 以及什么是 TIRx(TVM 描述 GPU kernel 的中间表示, 见第 9 章)。没把握的话, 先翻一下第 0 章 · 极简入门。本章会默认你已经认识这些词。

21.0 为什么要把”类型”拆成两层?

先把结论撂这儿: 在 TIRx 里, 一个表达式的”类型”是两层的。为啥要搞得这么麻烦? 咱们从平时写 CUDA 聊起。

你写 `float x;` 的时候, 类型就一个维度, 够用了。编译器一看到 `float`, 啥都知道了: 占 4 个字节、是浮点、能拿来做浮点运算——一句话就交代清楚了。

可 TIRx 不是给人直接读的代码, 它是一种**中间表示 (IR / intermediate representation)**: 专门给编译器看的半成品, 编译器读完它再去生成 CUDA。既然是要喂给编译器的, 记的信息自然就得更细。

TIRx 是 TVM 在 TensorIR 上面加的一层扩展, 干的活是描述 GPU kernel, 最后吐出 CUDA 源码。它有两件事得一块儿顾好: 一是把底层每个数据占多少位、怎么摆, 控制得死死的; 二是在 IR 这层就能看出来”这是个值, 还是个指针”。为了一手抓俩, 它干脆把类型劈成了两层:

层级	名称	回答的问题	典型取值
低层	dtype	”这是什么位?”(标量/向量元素类型)	float32、float16、int32、float32x4、handle
高层	type	”这是标量还是指针?”	PrimType(dtype)、PointerType(PrimType(dtype), scope)

关键

关键: 先记住一句话——`dtype` 管”数据长啥样”(多少位、带不带符号、是不是向量),`type` 管”这数据是拿来干嘛的”(是个能直接参与运算的值, 还是个指向某块存储的指针)。这两件事一分开, 编译器既能精确地生成底层代码, 又能顺手对指针做点语义检查。

下面这张图就是这个两层结构的概貌:

结构图

连接关系:

- TIRx 表达式 `PrimExpr` $\rightarrow$ `dtype` 低层位类型
- TIRx 表达式 `PrimExpr` $\rightarrow$ `type` 高层语义类型
- `dtype` 低层位类型 $\rightarrow$ 标量: `float32` / `int32` / `bool` ...
- `dtype` 低层位类型 $\rightarrow$ 向量: `float32x4` ...
- `dtype` 低层位类型 $\rightarrow$ 指针: `handle`
- `type` 高层语义类型 $\rightarrow$ `PrimType(dtype)` / (绝大多数表达式是标量)
- `type` 高层语义类型 $\rightarrow$ `PointerType(PrimType(dtype), scope)` / (只有指针才用)

21.1 表达式的 dtype

咱们先讲下面那层——`dtype`, 它最好懂。

每个表达式 (`PrimExpr` / `Expr`) 身上都有个 `dtype` 属性, 告诉你这个表达式里**每个元素是什么类型** (标量也好, 向量也好)。常见的就这么几类:

- 普通浮点:`float32`、`float16`、`bfloat16`
- 整数:`int32`、`uint8`
- 布尔:`bool`
- 低精度浮点:`float8_e4m3fn`、`float4_e2m1fn` 这一类。它们是专门为 AI 计算憋出来的窄格式 (尤其是推理, 还有新一代训练), 位数特别少。名字里其实就写着位数怎么分:`e4m3` 就是 4 位指数、3 位尾数。
- 指针:`handle`, 说白了就是个 C 指针。
- 向量形式: 像 `float32x4`, 意思是”4 个 `float32` 打成一包当向量用”, 对应 CUDA 里的 `float4`。

注意

注意: 每个 `dtype` 到了生成代码 (codegen) 那一步, 都会变成一个”对得上号的 CUDA 类型”。也就是说,`dtype` 不是个虚的标签, 而是跟后端类型一一咬合的、实打实的物理类型。

21.1.1 一个例子：用各种 dtype 分配 buffer

光看列表不解渴，上个例子。原文有个示范 kernel，拿好几种 dtype 各分配了一块 buffer——有放在寄存器里的，有放在共享内存里的——顺手还演了一次 `float32x4` 的向量化访存。核心片段在下面，中文注释我都加好了：

```
@T.prim_func
def dtypes(A_ptr: T.handle, O_ptr: T.handle):
    A = T.match_buffer(A_ptr, (256,), "float32") #           256           float32 buffer
    O = T.match_buffer(O_ptr, (256,), "float32")
    T.device_entry(); bx = T.cta_id([1]); tx = T.thread_id([64]) #           +           CTA           bx           tx

    f16 = T.alloc_local((1,), "float16") #           :float16
    bf16 = T.alloc_local((1,), "bfloat16") #           :bfloat16
    i32 = T.alloc_local((1,), "int32") #           :int32
    u8 = T.alloc_local((1,), "uint8") #           :uint8
    b1 = T.alloc_local((1,), "bool") #           :bool
    sm = T.alloc_shared((64,), "float16") #           (SMEM)tile:64           float16

    v = T.alloc_local((1,), "float32x4") #           :           dtype           (float4)
    v[0] = A.vload([tx * 4], dtype="float32x4") #           4           float32
    O.vstore([tx * 4], v[0]) #           4           float32
    # ... (           f16/bf16/i32/u8/b1/sm) ...
```

有几处得拎出来单说：

1. `T.alloc_local((1,), dtype)` 开的是寄存器 (register, 每个线程自己的一小块超快私有存储, 见第 0 章) 级别的存储, 每个线程自己有一份, 别人看不到。形状写成 `(1,)`, 就是说“我只要一个标量元素”。
2. `T.alloc_shared((64,), "float16")` 开的是一块共享内存 (SMEM / shared memory, 一个 CTA 内所有线程共用的片上存储) tile(大数组切出来的一小块), 这块地方是整个 CTA(线程块, 一组一起跑、能共用共享内存的线程, 见第 0 章) 里的线程一块儿用的。
3. `v = T.alloc_local((1,), "float32x4")` 是这一节最要紧的一行。它直接用向量 dtype 声明了一个 `float4` 寄存器, 之后写 `v[0]` 就能把这个向量拿出来。这里得留个神:buffer 形状是 `(1,)`, 只有一个元素没错, 可这“一个元素”本身就是 4 宽的向量。
4. `A.vload([...], dtype="float32x4")` 和 `O.vstore([...], ...)` 把挨着的 4 个 `float32` 当成一次 16 字节的访存整体搬。为啥这么干? 这是 GPU 上压榨带宽的老套路: 一条访存指令越宽, 总共要发的访存次数就越少。

21.1.2 它生成什么样的 CUDA?

TIRx 写完了, 咱们肯定想看看它到头来长成啥样。上面那段会被 lower(降级 / 下降) 成下面这堆 CUDA。为了让重点更扎眼, 原文把没关系的部分都删了:

```
“c++ half f16_ptr[1]; // float16 -> half nv_bfloat16 bf16_ptr[1]; // bfloat16 -> nv_bfloat16
int i32_ptr[1]; // int32 -> int uchar u8_ptr[1]; // uint8 -> uchar signed char b1_ptr[1]; //
bool -> signed char __shared__ alignas(64) half sm_ptr[64]; // 共享 float16, 带 64 字节对齐
float4 v_ptr[1]; // float32x4 -> float4 (向量)
```

```
v_ptr[0] = (float4)(A_ptr + tx * 4); // 向量化加载:reinterpret 成 float4 再解引用 (float4)(O_ptr
+ tx * 4) = v_ptr[0]; // 向量化写回: 同样 reinterpret 成 float4
```

```

> ** **: , CUDA **: `float*`
↳ `float4*`, ** `float4` 16 , `float32x4` dtype
↳ , ,

"TiRx dtype -> CUDA " , :

| TiRx (`alloc_local` / `alloc_shared`) | CUDA |
| --- | --- |
| `alloc_local`((1), "float16")` | `half f16_ptr[1];` |
| `alloc_local`((1), "bfloat16")` | `nv_bfloat16 bf16_ptr[1];` |
| `alloc_local`((1), "int32")` | `int i32_ptr[1];` |
| `alloc_local`((1), "uint8")` | `uchar u8_ptr[1];` |
| `alloc_local`((1), "bool")` | `signed char b1_ptr[1];` |
| `alloc_shared`((64), "float16")` | `__shared__ alignas(64) half sm_ptr[64];` |
| `alloc_local`((1), "float32x4")` | `float4 v_ptr[1];` |

### 21.1.3 dtype " ", vload

, : " " `vload` ,

> ** **: , dtype ** , `vload`/`vstore`
↳ `T.alloc_local`((1), "float32x4")` , `float4`
↳ , `vload`/`vstore` " "
↳ dtype, **, buffer , dtype**, `vload`
↳

** (orthogonality)**: " " " ,
↳ , ; buffer
↳

### 21.1.4 dtype -> CUDA

, :

| TiRx dtype | CUDA | | |
|---|---|---|---|
| `float32` | `float` | |
| `float16` | `half` | |
| `bfloat16` | `nv_bfloat16` | brain float,8 7 |
| `int32` | `int` | 32 |
| `uint8` | `uchar` | 8 |
| `bool` | `signed char` | `signed char` |
| `float32x4` | `float4` | (4xfloat32) |
| `handle` | `T*` ( ) | , `T` |
| dtype | CUDA | | `<` `>x<N>` -> CUDA `<` `><N>` |

> ** **: , `bool` `signed char`, C++ `bool`? : GPU
↳ 1 , (`float8_e4m3fn` `float4_e2m1fn`
↳ ), CUDA ,

---

## 21.2 dtype type ?

`dtype` , `type` ? :

```



```
- **`dtype`      **      , "      (what bits)"
- **`type`      **      , "      "

      ?      :

-      **      **,      type      `PrimType(dtype)`      `float32`      , type      `PrimType("float32")`
-      **      **,      type      `PointerType(PrimType(dtype), scope)`      `dtype`
↳      , `scope`      (      `shared`      `global`)

> **      **:      ,      type      ,      `PrimType(dtype)`      ,      **type
↳      ,      **      ?      "      "      "      ,
↳ dtype

``mermaid
graph LR
    subgraph SCALAR_CASE[""]
        S["float32      " --> SP["type = PrimType('float32')"]]
    end
    end
    subgraph PTR_CASE[""]
        P["      shared      uint64      " --> PP["type = PointerType(PrimType('uint64'), 'shared')"]]
    end
    end
```

21.3 指针 (handle)

接着上节说——指针就是 `type` 系统真正能耍开的地方，这节好好讲。

在 TIRx 里，指针抛头露面时用的 `dtype` 是 `handle`。可它的” 身份证信息”(指向谁、住哪个作用域) 并不写在 `dtype` 里，而是上层的 `PointerType` 替它保管着。这节聊的是个挺实在的事：一个指针，你怎么正经地拿到手、又用起来。

21.3.1 一条核心规矩:buffer 的 data 是不可变的 Var

每个 `buffer` 都有个 `data` 字段，它就是这个 `buffer` 底下垫着的那根指针。关于 `data`，记住两点就够：

1. 它是个指针类型的 `Var`(变量)；
2. 它改不了 (`immutable`)——指针一锤定音之后，就不许再重新赋值了。

关键

关键: 干嘛先把这条规矩摆出来? 因为后面那几种” 取指针” 的写法, 全是被这条规矩逼出来的。规矩特别简单:`data` 只认 `Var`, 不收表达式。所以呢, 你别想着把一个” 算出来的地址表达式” 随手塞给 `buffer` 当 `data`——得先把它” 安顿” 成一个 `Var`, 才行。

21.3.2 拿到指针的三种方式

把上面那条规矩 (`data` 必须是 `Var`) 揣兜里，取指针就有三条路可走，各管各的场景：

方式	API	它做了什么	适用场景
分配新存储	<code>T.alloc_buffer(...)</code>	分配存储 + 同时定义 <code>data</code> 指针	需要一块全新的存储

复用已有指针	T.decl_buffer(..., data=ptr)	在一个已存在的指针 Var ptr 之上声明 buffer	已经有了指针 Var，只是想换个 buffer 视图去访问它
绑定指针表达式	先 T.let 成 Var，再 decl_buffer	把一个指针表达式先绑定为 Var，再用它声明 buffer	指针来自计算（如跨 CTA 的远程地址），而非现成的 Var

不管你走哪条，终点是同一个: `data` 最后必须是个合法的 `Var`。下面这张流程图，就是把三条路怎么殊途同归到这个终点画了出来：

结构图

连接关系：

- 我手上有什么？什么都没有，需要新存储 → `T.alloc_buffer(...)` / 分配存储并定义 `data`
- 我手上有什么？已有指针 Var → `T.decl_buffer(..., data=ptr)`
- 我手上有什么？只有一个指针表达式 → 先 `T.let` 绑定成 `Var`
- 先 `T.let` 绑定成 `Var` → 再 `T.decl_buffer(..., data= 那个 Var)`
- `T.alloc_buffer(...)` / 分配存储并定义 `data` → `data` 是合法的 `Var`
- `T.decl_buffer(..., data=ptr)` → `data` 是合法的 `Var`
- 再 `T.decl_buffer(..., data= 那个 Var)` → `data` 是合法的 `Var`

21.3.3 绑定指针表达式：访问远程 CTA 的共享内存

三条路里头，第三条最有意思，也最值得掰开讲。它对付的是这么一种情况：你手上的指针不是现成的 `Var`，而是临时算出来的一个表达式。这时候你绕不开——得先用 `T.let` (搭上 `PointerType`) 给它“起个名字”、变成一个 `Var`，然后才好递给 `decl_buffer`。

啥时候会碰上“算出来的指针”？原文举的典型场景是分布式共享内存 (distributed shared memory)/ 集群 (cluster, 新一代 GPU 上由多个 CTA 组成、能互相读写共享内存的协作单元)。它靠 `T.ptx.map_shared_rank` (底层就是 PTX 的 `mapa` 指令) 干一件事：把“本 CTA 的某个共享地址”换算成“集群里另一个 CTA 的共享地址”。这么一换，几个 CTA 之间就能直接够到彼此的 `SMEM` 了。而这个换算出来的结果，天生就是个地址表达式——所以它正好得走“先 `T.let`，再 `decl_buffer`”这条道。

```
from tvn.ir.type import PointerType, PrimType

# 1)      map_shared_rank(PTX mapa)          CTA          (          )
#      reinterpret  handle,      T.let          PointerType  Var "ptr"
ptr: T.let[T.Var(name="ptr", dtype=PointerType(PrimType("uint64")))] = \
    T.reinterpret("handle", T.ptx.map_shared_rank(mbar.ptr_to([0]), 0))

# 2)      ptr          Var,          buffer
#      CTA      mbarrier(uint64),      shared
remote_mbar = T.decl_buffer([1], "uint64", data=ptr, scope="shared")
```

咱们一行一行拆：

1. `T.ptx.map_shared_rank(mbar.ptr_to([0]), 0)`：它对应 PTX 的 `mapa`，干的活是——把当前 CTA 里某个 `mbarrier` (放在共享内存里的一个同步计数器，用来等“数据到齐了没”) 的共享地

址, 换算成集群里目标 CTA(这里 rank 给的是 0) 上同一个位置的共享地址。它吐回来的是一个指针表达式。

2. `T.reinterpret("handle", ...)`: 把上面那个结果重新解释成 `handle`, 也就是通用指针 `dtype`。
3. `T.let[T.Var(..., dtype=PointerType(PrimType("uint64")))] = ...`: 这步就是关键的“起名字”。前面念叨过 `data` 必须是 `Var`, 所以这儿用 `T.let` 把那个表达式绑定到一个叫 `ptr` 的指针 `Var` 上, 它的上层类型是 `PointerType(PrimType("uint64"))`。其中 `PrimType("uint64")` 说的是它指向的是 `uint64` 元素 (`mbarrier` 本来就是拿 `uint64` 存的)。这里有个小细节: `PointerType` 只填了元素类型, 没填 `scope`。作用域 `"shared"` 留给了下一步的 `decl_buffer`——俩人各管一摊, 分工明确。
4. `T.decl_buffer([1], "uint64", data=ptr, scope="shared")`: 最后这步, 在那个合法的指针 `Var` 上面声明出一个 `buffer`: 1 个元素、`uint64`、作用域 `shared`。打这儿起, 你一读一写 `remote_mbar`, 真正碰到的就是远端那个 CTA 的共享内存。

注意

注意: 这段代码别被它唬住。它一下子扯进来好几个底层概念——PTX `mapa`、集群里的分布式共享内存、`mbarrier`、还有 `reinterpret`。但本章真正要你记的不是这些机制本身, 而是一条规则: 指针表达式不能直接拿来当 `data`, 必须先用 `T.let` 绑成 `Var`。这条想通了, 你也就明白这段代码为啥非得绕一圈、套个 `T.let` 了。

最后拿一张图收个尾, 看看一个“远程共享地址表达式”是怎么一步步熬成一个能用的远程 `buffer` 的:

结构图

连接关系:

- 远程 CTA 的共享地址 / (PTX `mapa`, 表达式) $\xrightarrow{\text{reinterpret("handle")}}$ 通用指针 `handle` / (仍是表达式)
- 具名指针 `Var` / (`PointerType`, 必须的具名化) $\xrightarrow{\text{decl_buffer(data=ptr)}}$ `remote_mbar` / (`uint64`, `scope="shared"`) / 最终可读写的远程 `buffer`

小结

这章没多长, 但 `TIRx` 类型系统的三根顶梁柱算是立住了:

1. **两层类型:** 下面那层是 `dtype`(粒度的物理类型, 跟 `CUDA` 的标量 / 向量类型 1:1 对得上), 上面那层是 `type`(在类型系统里的角色, 要么 `PrimType`、要么 `PointerType`)。标量场景下 `type` 没啥可说的; 只有到了指针场景, `type` 才扛起那点要紧信息——指向谁、在哪个作用域。
2. **向量 `dtype` 是正交的:** 像 `float32x4` 这种向量 `dtype` 是头等公民, 声明个寄存器就到手一个 `float4`, 向量化访存只是它一堆用法里的一种。一句话: 向量这属性是 `dtype` 自己带的, 跟 `vload/vstore` 不捆一块儿。
3. **拿指针得守规矩:** `buffer` 的 `data` 是个改不了的 `Var`。三条取指针的路 (`alloc_buffer` / `decl_buffer(data=ptr)` / 先 `T.let` 再 `decl_buffer`), 说到底都是为了守住同一条底线——`data` 必须是 `Var`, 不能是表达式。指针要是算出来的 (比如跨 CTA 的远程地址), 那就先用 `T.let` 给它起个名字。

把这三条串一块儿你就咂摸出味儿了:TIRx 的类型设计自始至终就奔着一个目标——既要把底层 CUDA 代码精确地生成出来,又要在 IR 这层就把”值 vs 指针”这种语义差别拎得明明白白,还能拿来做校验。

延伸阅读

- 原文 (英文):[Data types and expressions —Modern GPU Programming for ML](#) Sys

术语对照

中文	English	备注
数据类型	dtype	低层级类型
高层类型	type	PrimType / PointerType
原始表达式	PrimExpr	TIRx 表达式基类
原始类型	PrimType	标量的高层类型
指针类型	PointerType	指针的高层类型, 含元素 dtype 与作用域
句柄 / 指针	handle	通用指针 dtype
向量 dtype	vector dtype	如 float32x4 → float4
向量化访存	vectorized memory access	vload / vstore
共享内存	SMEM (shared memory)	CTA 内共享
寄存器	register	alloc_local
作用域	scope	如 "shared"、"global"
不可变	immutable	buffer 的 data 指针
降级 / 下降	lower	IR → CUDA 代码生成
集群	cluster	多 CTA 组成的协作单元
分布式共享内存	distributed shared memory	跨 CTA 访问彼此 SMEM
低精度浮点	low-precision float	float8_e4m3fn、float4_e2m1fn 等

第 22 章 · 缓冲区与内存

原文: [Buffers and memory](#)

本章要点

本章要点 (TL;DR)

- 缓冲区 (buffer) 的本质就是「指针 + 元数据」: 相同的逻辑下标访问, 会因为元数据 (布局 layout、偏移 elem_offset) 不同而编译成完全不同的地址算术。
- 创建缓冲区只有两个根本 API: `T.alloc_buffer`(新分配存储) 和 `T.decl_buffer`(在已有指针上声明一个视图 view)。`alloc_shared` / `alloc_local` 都只是带特定 scope 的 `alloc_buffer`。
- scope 决定了内存空间: `global` / `shared`(静态共享内存) / `shared.dyn`(动态共享内存) / `local`(寄存器) / `tmem`(Blackwell 张量内存)。
- 共享内存有静态与动态两种: 静态在编译期定形; 动态是一整块在 launch 时定大小的 arena(竞技场), 整个 kernel 只能有一块, 其它缓冲区都是它的视图。`T.SMEMPool` 自动帮你做这件繁琐的偏移记账。
- 标量 (scalar) 是「一个元素的寄存器数组」的语法糖; 而 `T.let` 是不可变绑定, 会下降成普通 C 局部变量, 可被编译器自由传播/CSE。张量内存 (TMEM) 必须显式 `alloc/dealloc`, 不能直接寻址, 只能通过 `tcgen05` 指令读写。

前置知识

前置知识: 读这一章前, 最好先懂 GPU 的内存层级 (寄存器 / 共享内存 / 全局内存谁快谁慢、各归谁用), 以及静态 vs 动态共享内存 (SMEM)、TMEM 这几个名词; 另外对 TIRx 的基本写法 (`T.prim_func`、`T.match_buffer` 这类) 有个印象会更顺。没把握的话, 先翻一下第 0 章 · 极简入门, 以及第 9 章 TIRx。本章会默认你已经认识这些词。

这一章讲的是 TIRx 的内存模型, 在整个「语言参考」里算是骨干。

想读懂它, 你只要先抓住一句话: 在 TIRx 里, 一个缓冲区 / buffer 并不是一个真的装着数据的「盒子」。你把它想象成贴在指针上的一张标签更准确——标签上写着「这块数据长什么样、该怎么访问」。这些信息, 我们统称元数据。

为什么要这么设计? 说白了, 就是图个省事: 这样一来, 绝大多数缓冲区方法干的活儿, 无非是在编译期改改这张标签——换个形状、换种读法而已。它们影响的只是「下标最后怎么翻译成地址」, 运行时一条多余指令都不会多出来。这个道理一旦想透, 后面所有 API 为什么长成那个样子, 你立马就全懂了。

一、缓冲区的统一心智模型

先混个脸熟: 在 TIRx 里, 缓冲区来来回回就这么几种玩法。

- **包装参数指针**:`T.match_buffer(ptr, shape, dtype, ...)`，把外面传进来的指针参数包成一个缓冲区。
- **下标访问**:`A[i, j]`，读写某个元素。
- **切片**:`A[m0:m0+BM, 0:BK]`，取出一块子区域，得到一个 `BufferRegion`(缓冲区区域 / buffer region)。
- **取指针**:`A.ptr_to([i, j])` 拿某个元素的地址，或者 `A.data` 拿这块存储的原始数据指针。

这里有一句最要紧的话: 只要不是 `tmem`，所谓「声明一个缓冲区」，说到底就是「给一根指针配上一个布局 `layout`」。这两样凑齐了，随便你写哪个下标，最后都能算出一个地址来。这个地址怎么算，原文给了个公式，值得记一记:

```
addr(buffer[coord]) = buffer.data + elem_offset + layout.apply(coord, shape=shape)["m"]
```

公式看着挺唬人，拆开其实就三块加起来:「起点 + 挪一段 + 按坐标算偏移」。一个个说:

组成部分	大白话
<code>buffer.data</code>	起点指针，也叫基址 / base pointer，就是这块存储从哪儿开始
<code>elem_offset</code>	在起点的基础上再往后挪几个元素，用来把一个「视图」摆到 <code>data</code> 的某个偏移处
<code>layout.apply(coord, ...)[“m”]</code>	把多维坐标 <code>coord</code> 换算成一维的元素偏移;“m”就是这个换算结果里的「元素偏移」那一项

符号你不用死记，记住这条思路就够了: 从哪儿开始 + 往后挪多少 + 坐标算出多少，三者一加就是地址。

关键

关键: 同样一句 `B[i, j]`，就因为缓冲区的元数据不一样，编译出来的地址算术能差出十万八千里。元数据是编译期就钉死的，到了运行时，剩下的不过是对裸指针做加减乘罢了。

下面我用一组对照例子，把这套心智模型给你坐实。我们对一块 `4×8` 的区域干同一件事 `B[i, j] = A[i, j] + 1`，但只动 `B` 的声明方式，看看地址会跟着怎么变。`A` 我们让它一直是行主序，所以 `A[i, j]` 的下标永远是 `i*8 + j`，雷打不动——这样你就能专心盯着 `B` 一个看了:

```
from tvm.tirx.layout import TileLayout, S

# 1)      (row-major):
B = T.match_buffer(p, (4, 8), "float32")
# 2)      (column-major):      TileLayout      stride      (1, 4)
B = T.match_buffer(p, (4, 8), "float32", layout=TileLayout(S[(4, 8):(1, 4)]))
# 3)      (shifted view):      64
B = T.match_buffer(p, (4, 8), "float32", elem_offset=64)
# 4)      stride      16:      16      8
B = T.match_buffer(p, (4, 8), "float32", layout=TileLayout(S[(4, 8):(16, 1)]))
```

还是那句 `B[i, j]`，这四种声明生成的 `CUDA` 下标，各是各的样:

“`c++ B_ptr[((i 8) + j)] = ...; // 行主序: i8 + j` `B_ptr[((j 4) + i)] = ...; // 列主序: j4 + i` `B_ptr[((i 8) + j) + 64]] = ...; // elem_offset=64: i8 + j + 64`(整体平移 64) `B_ptr[((i 16) + j)] = ...; // 行 stride 16: i16 + j`(每行间隔 16)

```
> **      **:'TileLayout(S[(shape):(strides)])`      :      / stride      ?
↪      1,      `S[(4,8):(1,4)]`      : 0      1      1      4
↪      ,

##      :      API

      API,      : **      **

### 2.1 `T.alloc_buffer`

```python
T.alloc_buffer(shape, dtype, scope=..., ...)
```

它会实打实地分配一块新存储 (在 IR 里吐出一个 `AllocBuffer` 节点), 然后把一个 `Buffer` 还给你。它有两个常用的简写:

- `T.alloc_shared(...)` 就是 `scope="shared"` 的 `alloc_buffer`。
- `T.alloc_local(...)` 就是 `scope="local"` 的 `alloc_buffer`。

2.2 `T.decl_buffer` —— 在已有指针上声明视图

```
T.decl_buffer(shape, dtype, data=..., ...)
```

它一块新存储都不分配, 只是在一根现成的指针 `data` 上「再开一个视图」。说白了, 就是给同一块存储起个别名 (alias), 或者换种眼光重新解读 (reinterpret) 它——比方说内存池里的某段子区域, 或者一个张量内存地址。

关键

关键:`decl_buffer` 有个小例外——要是你传了 `data=None`, 它手头就没现成指针可用了, 这时它会退一步, 像 `alloc_buffer` 那样自己掏腰包分配一块。

归根结底, 这两者的差别就一处: 那根 `data` 指针 (它是个不可变的 `Var`) 究竟从哪儿来:

API	<code>data</code> 指针从哪来
<code>alloc_buffer</code>	它自己造出这个 <code>data</code> (这次分配的产物)
<code>decl_buffer</code>	它接收一个已经存在的 <code>data</code> (从外面传进来)

注意

注意: 要是你手里是个指针表达式 (而不是现成的 `Var`), 想拿它来撑起缓冲区, 那得先把它绑定成一个 `Var` 才行 (具体怎么绑, 见原书「数据类型与表达式」那章)。

2.3 共享的描述符参数

这两个 API 共用同一套描述参数。挑几个最常打交道的看看:

参数	含义
dtype	元素类型, 如 "float32"、"float16"、"float4_e2m1fn" 等
shape	逻辑形状 (各维 extent 组成的元组)
layout	物理映射 (TileLayout); "default" 即稠密行主序
elem_offset / byte_offset	把一个视图放到 data 的某个偏移处
allocated_addr	携带一个预先指定的地址 (用于张量内存)
align	数据指针的对齐字节数

还有个 `scope` 参数, 它说了算的是: 这块缓冲区到底落在**哪种内存空间**里。这条线索本章后面几节会反反复复出现, 你先认个脸:

Scope	快捷方式	对应内存
"global"	(默认)	设备全局内存 (global memory)
"shared"	<code>T.alloc_shared</code>	静态共享内存 ( <code>__shared__</code> )
"shared.dyn"	(走内存池)	动态共享内存 (pooled, 见下文)
"local"	<code>T.alloc_local</code>	每线程的寄存器 (registers)
"tmem"	(走 TMEM 池)	Blackwell 张量内存 (tensor memory)

下面这段把四种典型用法摆在一块儿, 你对照着看:

```
A = T.match_buffer(A_ptr, (M, K), "float16", align=16) # ()
As = T.alloc_shared((BM, BK), "float16") # tile
acc = T.alloc_local((4,), "float32") #
view = T.decl_buffer((BM, BK), "float16", data=As.data) # As
```

三、共享内存 (shared memory)

共享内存 / shared memory(往下简称 SMEM) 有两种「口味」: **静态**和**动态**。怎么分? 就看一点——它的大小是**编译时**就定死了, 还是**每次启动 kernel 的时候**才现定。除此之外还配了个内存池 (pool) 帮手, 专门收拾动态这种麻烦事。

结构图

连接关系:

- 共享内存 SMEM → 静态 static / scope=shared
- 共享内存 SMEM → 动态 dynamic / scope=shared.dyn
- 静态 static / scope=shared → 编译期定大小 / `__shared__ T x[N]`
- 动态 dynamic / scope=shared.dyn → launch 时定大小 / `extern __shared__ arena[]`
- 动态 dynamic / scope=shared.dyn → `T.SMEMPool` / 自动 bump 分配 + 偏移记账

3.1 静态共享内存

最省事的共享缓冲区就是**静态**这种——用 `T.alloc_shared`(也就是 `scope="shared"`), 大小在编译时就钉死。它最经典的套路是三步走: **先把数据搬进来** → **拿 `cta_sync` 让整个 block(CTA, 一组协作的线程, 共享同一块共享内存) 都看见这次写入** → **再读回来用**。



```
@T.prim_func
def smem_demo(A_ptr: T.handle, B_ptr: T.handle):
 A = T.match_buffer(A_ptr, (128,), "float32")
 B = T.match_buffer(B_ptr, (128,), "float32")
 T.device_entry()
 bx = T.cta_id([1])
 tx = T.thread_id([128])
 sm = T.alloc_shared((128,), "float32") #
 sm[tx] = A[tx]
 T.cuda.cta_sync() # , __syncthreads()
 B[tx] = sm[tx] * T.float32(2.0)
```

它会老老实实变成一个普普通通的 `__shared__` 数组:

“`c++ __shared__ alignas(64) float sm_ptr[128]; // 对应 T.alloc_shared sm_ptr[tx] = A_ptr[tx]; __syncthreads(); // 对应 T.cuda.cta_sync() B_ptr[tx] = sm_ptr[tx] * 2.0f;`

```
3.2

** ** (`scope="shared.dyn"`) , ** kernel **,
↪ `sharedMemBytes` , :

> ** **: kernel ** ** , arena /
↪ , `T.decl_buffer`(` arena      `data` ,
↪ `elem_offset``,` , arena

```python
arena = T.alloc_buffer((128,), "float32", scope="shared.dyn") # arena
# arena decl      ,      0 64
As = T.decl_buffer((64,), "float32", data=arena.data, scope="shared.dyn") # offset 0
Bs = T.decl_buffer((64,), "float32", data=arena.data, elem_offset=64, scope="shared.dyn") # offset 64
As[tx] = A[tx]
Bs[tx] = B[tx]
T.cuda.cta_sync()
C[tx] = As[tx] + Bs[tx]
```

这两个视图, 共用的是同一块 `extern __shared__ arena`(下面为了看得清楚, 我把 `arena` 起名 叫 `smem`):

“`c++ extern __shared__ __align__(64) float smem[]; // 唯一的动态共享 arena smem[tx] = A_ptr[tx]; // As ——偏移 0 处的视图 smem[tx + 64] = B_ptr[tx]; // Bs ——偏移 64 处的视图 __syncthreads(); C_ptr[tx] = smem[tx] + smem[tx + 64];`

```
      :      ,      arena,      :

```mermaid
flowchart TD
 subgraph StaticGroup["static __shared__ "]
 S1["S1[\"sm[128]\"]"]
 S2["S2[\" \"]"]
 end
 end
 subgraph DynGroup["dynamic arena (launch) "]

```

```
Arena["extern __shared__ float smem[]"]
Arena --> V1["As (offset 0, 64)"]
Arena --> V2["Bs (offset 64, 64)"]

end
```

图注: 静态共享内存是几块各自独立的小数组; 动态共享内存只有一整块 `arena (smem[])`, 大小在 `launch` 时确定), `As` 落在偏移 0, `Bs` 落在偏移 64, 两个加起来正好填满 0~128。

#### 注意

**注意:** 千万别手一抖写了两次 `alloc_buffer(scope="shared.dyn")`, 那是错的——动态共享内存只许分配一块。一句话把两者拎清: 静态共享内存编译时就定好大小 (`__shared__ T x[N];`); 动态共享内存呢, 是独此一块、启动时才定大小的 `arena`, 你想要的各个视图, 都靠 `decl` 在它内部不同偏移上声明出来。

### 动态共享大小是怎么被传进去的?

这里有个特别容易绕晕的点: 既然大小是启动时才定的, 那 `sharedMemBytes` 到底是谁填进去的? 答案是——根本轮不到你手动填, TVM 会照着你 `shared.dyn` 分配的大小, 自动把它算出来。整个流程是这样:

1. 其实呢, `arena` 多大, 编译时就已经心里有数了 (上面例子里是 128 个 `float`, 也就是 512 字节)。
2. 到了 `lowering` (下降) 这一步, TVM 会往设备 `kernel` 的 `tirx.kernel_launch_params` 里塞一个 `"tirx.use_dyn_shared_memory"` 标记。
3. `host` 端的启动器一看到这个标记, 就把总字节数算好, 当成**最后一个**启动参数递进去。

```
kernel : use_dyn_shared_memory
"tirx.kernel_launch_params": ["blockIdx.x", "threadIdx.x", "tirx.use_dyn_shared_memory"]

host (... , gridDim.x, blockDim.x, dyn_shared_bytes):
T.call_packed("dyn_kernel", A.data, B.data, C.data, 1, 64, 512)
```

等跑起来, 这个 512 就被填进 `cuLaunchKernelEx` 调用的 `config.sharedMemBytes` 里。从头到尾一条龙, 全自动, 你啥都不用管。

### 3.3 内存池语法糖: `T.SMEMPool`

前面那种做法——手动 `decl` 一堆视图、自己掰手指头算偏移——又烦人又容易出错。 `T.SMEMPool` 就是来替你包这个脏活的: `arena` 的偏移记账, 它给你**全自动**搞定。

它的工作原理叫**碰撞分配器 / bump allocator**: 你每问它要一块, 它就在 `shared.dyn` 上往后推一段、把偏移记下来, 视图你一行都不用自己写。除了最基本的 `alloc / commit`, 它还附送几个贴心的小帮手:

- **逐块指定 `align=`**: 某一块缓冲区想单独设个对齐? 可以。
- **`alloc_mma` 帮手**: 自动给你配好一个 `MMA` (矩阵乘累加, `Tensor Core` 干的活) 用得上的 `swizzle` (把数据在共享内存里错位摆放, 避开 `bank` 冲突) 布局。
- **`move_base_to`**: 把游标 (`cursor`) 往回倒一倒, 后面的分配就能**接着复用**前面那块空间。

```
pool = T.SMEMPool() # shared.dyn bump
As = pool.alloc(BM, BK), "float16", align=128) # tile
Bs = pool.alloc(BK, BN), "float16", align=128)
Cs = pool.alloc_mma((BM, BN), "float16") # MMA ,swizzle
pool.commit() #
pool.move_base_to(offset)
```

**注意**

注意: 后面要讲的 **TMEM 池 (T.TMEMPool)**, 其实就是叠在一个 **SMEMPool** 上面的。所以你现在把 **SMEMPool** 这套 bump 分配的路数搞明白, 待会儿看 **TMEM** 会顺畅很多。

**SMEMPool** 怎么做 bump 分配, 看一眼图就明白了:

**结构图**

连接关系:

- base (游标 cursor 从 0 开始) → As
- As alloc 向右推进, 按 align 对齐 → Bs
- Bs → Cs
- Cs → cursor (commit() 时即总大小)

图注:**SMEMPool** 是个碰撞分配器, 游标从 0 起步, 每 **alloc** 一次就往右推进一段 (顺带按 **align** 对齐); 到 **commit()** 那一刻, 游标停在哪儿, 内存池就有多大;**move\_base\_to(off)** 能把游标倒回去, 让后面的分配覆盖、复用前面那块空间。

四、寄存器 (registers)

每个线程自己用的那点临时数据, 搁寄存器里跑得最快。想要这种存储, 就用 **T.alloc\_local(shape, dtype)** (也就是 **scope="local"**) 来分配。它**每个线程一份、谁也不碍着谁**, 最后会变成一个蹲在寄存器里的本地数组:

```
r = T.alloc_local((4,), "float32") #
for k in T.unroll(4): # ,
 r[k] = A[tx, k]
... r[0..3] ...
```

“**c++ alignas(64) float r\_ptr[4];** // 每线程私有, 驻留寄存器 **r\_ptr[0] = A\_ptr[tx 4 + 0]; r\_ptr[1] = A\_ptr[tx 4 + 1];** // ...

```
> ** : `alignas(64)` ?** ** ** `data_alignment`
↳ `runtime::kAllocAlignment`, 64 CUDA codegen
↳ , ** , `local`
>
> , 64 ** **
↳ ? , nvcc/ptxas (/
↳ SROA), , (no-op),
↳ ** (spill) ,
↳ , (`local` scope dtype)
```

```

4.1 (scalar):

 : ** **
→ , 1 `local` , `[0]` :

`python
phase = T.alloc_local((1,), "int32") # 1
phase[0] = 0
while phase[0] < 4:
 acc = acc + A[tx, phase[0]]
 phase[0] += 1

```

可满屏都是 `phase[0]`, 看着实在别扭。于是「标量」就是为这事加的一层语法糖——让你直接拿名字读写的单元素寄存器缓冲区:

```

phase: T.int32 = 0 # ()
while phase < 4:
 acc = acc + A[tx, phase]
 phase += 1

s = T.local_scalar("int32") # ; (s = ..., s[0])
acc: T.float32 = 0.0 #

```

### 关键

**关键:** 这两种写法可不只是「长得像」——它们解析出来的 **TIRx** 结构压根就是同一个。这层糖在 parser 阶段就化没了:`phase: T.int32` 就是那个单元素 local 缓冲区,`phase / phase += 1` 就是 `phase[0] / phase[0] += 1`。你拿这俩 kernel 去跑 `tvm.ir.assert_structural_equal`, 直接通过;`printer` 甚至会反着来, 把显式的 `alloc_local + [0]` 重新打印成标量的样子。换句话说, 解析一结束, 两者就再无分别——都下降成同一句 `alignas(64) int phase_ptr[1];`。要是你想显式指定 scope, 就用 `T.local_scalar / T.shared_scalar / T.alloc_scalar`。

### 注意

**注意:** 那为啥不干脆用 **Var**? 因为 TIRx 的 **Var** 是不可变的——它只是个一次性的静态绑定 (`T.let` 吐出来的就是这玩意)。可标量要的恰恰是**可变**: 循环里、累加器 (accumulator, 边算边把结果累加进去的那个变量) 里, 你得一遍遍给它重新赋值。所以标量非得靠一个「能反复写入」的单元素缓冲区来撑着, 这活儿 **Var** 干不了。

## 4.2 let: 不可变绑定

`T.let` 绑定是**不可变**的——它对应一个 `LetStmt`, 干的事就是「给一个值起个名字」, 而不是开一块缓冲区。它特别适合用来定义那种算一次就再也不变的派生常量:

```

n: T.let = M * K # (LetStmt)
half: T.let[T.int32] = N // 2 # ...

```

它会下降成一个普通的 C 标量变量——注意，不是缓冲区，没有数组，也没有 [0] 那一套。拿 `half: T.let = m * 2` 举个例子 (`m` 是个运行时才知道的值):

“`c++ int half = m * 2; // let -> 一个类似 const 的局部变量`

```

 (simplifier) (CSE)
↪ , `m * 2` , `half`

> ** : ?**

↪ , ,

>

> : (arithmetic analyzer) `LetStmt` , (analyzer.Bind(var,
↪ value)) , / (modular set)
↪ ** ** ,

>

> ** ** : (`buf[0]`), ,]
↪ :`let` ** ** , CSE ;

↪ load/store ,

 `let` , :

| | scalar | `let` | |
|---|---|---|---|
| | ** **(() | ** **(() |
| | `local` (load/store) | `LetStmt`, C |
| | (1) | |
| | / | () | ** **((,) |
| | | (`M*K`) |

(tensor memory,TMEM)

Blackwell(NVIDIA GPU , Hopper Ampere) ** / tensor
↪ memory(TMEM)** scope, , :

1. ** **: warp (warp-uniform, warp
↪) `T.ptx.tcgen05.alloc` / `tcgen05.dealloc`
2. ** **: `T.decl_buffer(..., scope="tmem", allocated_addr=< >,
↪ layout=<tmem >)`
3. **`allocated_addr`(() ** tensor-core
↪ ,`T.alloc_buffer(scope="tmem")` ** **, ** ** `allocated_addr`
4. ** **: ,TMEM `tcgen05` `mma` / `ld` / `st` / `cp`
↪

 TMEM warp , `decl` ,

↪ warp :

```python
addr = T.alloc_shared((1,), "uint32")                                #
if warp_id == alloc_warp:                                # tcgen05.alloc warp
    T.ptx.tcgen05.alloc(T.address_of(addr), n_cols=512, cta_group=cta_group)
acc = T.decl_buffer((CTA_M, 512), "float32", scope="tmem",
                    allocated_addr=0, layout=tmem_layout)                                # 0
# ... acc                                gemm_async / copy_async                                ...
if warp_id == alloc_warp:
    T.ptx.tcgen05.relinquish_alloc_permit(cta_group=cta_group)                                #
    T.ptx.tcgen05.dealloc(addr, n_cols=512, cta_group=cta_group)

```

这里头的列偏移，还有 `tmem_layout` (数据通路上的 一种 D/F 布局)，通通得你自己操心。看到这儿你大概已经在皱眉头了——别急，下面要讲的内存池，自动帮你发的正是这一整套烦人的序列。

TMEM 手动管理的生命周期：

时序图（按时间顺序）

- 1. `alloc_warp` 单一 warp → 张量内存 TMEM: `tcgen05.alloc(addr, n_cols=512)` 【warp 统一】
- 2. 备注: `decl_buffer(scope=tmem, allocated_addr=0)` / `acc` = 列 0 处的视图
- 3. kernel 主体 → 张量内存 TMEM: 通过 `tcgen05 mma/ld/st/cp` 读写 (不可直接寻址)
- 4. `alloc_warp` 单一 warp → 张量内存 TMEM: `relinquish_alloc_permit` 交还分配许可
- 5. `alloc_warp` 单一 warp → 张量内存 TMEM: `tcgen05.dealloc(addr, n_cols=512)`

5.1 TMEM 池:T.TMEMPpool

T.TMEMPpool 把上面那一大摊子活儿全给你打包了——warp 统一的 `alloc/dealloc`、列偏移的 `bump` 分配、外加数据通路布局，一站全搞定：

```
tmem_addr = pool.alloc((1,), "uint32")           # pool      kernel      smem
tmem_pool = T.TMEMPpool(pool, total_cols=512, cta_group=cta_group,
                        tmem_addr=tmem_addr)
acc = tmem_pool.alloc((CTA_M, 512), "float32")   # allocated_addr
tmem_pool.commit()                               #      tcgen05.alloc(      warp)
# ...      acc ...
tmem_pool.dealloc()                              #      tcgen05.dealloc(      warp)
```

注意

注意：用 `TMEMPpool` 之前，你得先递给它一个现成的 `SMEMPpool`。为啥前面说过了——它是“叠”在共享内存池上头的，得借共享内存里的一个小槽，来存放分配到的基址。想看它怎么真刀真枪地用，翻原书第三部分那个 `GEMM` (通用矩阵乘法, GPU 上最核心的计算) kernel 就行。

六、缓冲区方法 (Buffer APIs)

咱们绕回开头那句心智模型: **Buffer 不过是指针上的一层元数据**。正因为如此，它的绝大多数方法都只是编译期的 `reshape / reinterpret`——要么换个算下标的方式，要么递给你一个指针，它自己一条运行时指令都不会冒出来。常用的方法先扫一眼：

方法	它是什么
<code>B.data</code>	原始数据指针 (一个 Var); 打印成 <code>B_ptr</code>
<code>B.ptr_to([i, j])</code>	指向某元素的有类型指针 (即 <code>address_of</code>); 打印成 <code>&B_ptr[ê</code>]
<code>B.vload([i], dtype="float32x4") / B.vstore([i], v)</code>	向量化 load / store; 打印成 <code>*(float4*)(B_ptr + ê)</code>
<code>B.view(*shape, layout=ê)</code>	用新的形状/布局重新解释同一块存储 (无拷贝)
<code>B.local(*shape, layout=ê)</code>	取出调用线程在某 <code>local</code> 缓冲区里的私有寄存器切片
<code>B.permute(*dims)</code>	轴重排后的视图 (即转置布局)

B.access_ptr(mask, ê)	带掩码的访问指针 (tvm_access_ptr 内建), 用于把一块区域传给某 intrinsic
-----------------------	--

下面分几类, 挑关键的用法挨个看看。

6.1 指针:ptr_to / data

想把**某个元素的地址**递给一个 intrinsic 或者内联函数, 就用 ptr_to; 而 data 给你的是整块存储的基址指针:

```
B[tx] = T.cuda.func_call("ld", A.ptr_to([tx]), source_code=SRC, return_type="float32")
```

“c++ B_ptr[tx] = ld(&A_ptr[tx]); // ptr_to([tx]) -> &A_ptr[tx]; A.data -> A_ptr

```
### 6.2      :`vload` / `vstore`

      **      / wide transfer**      ,      :

```python
B.vstore([tx * 4], A.vload([tx * 4], dtype="float32x4"))
```

“c++ (float4)(B\_ptr + tx 4) = (float4)(A\_ptr + tx 4); // 一次搬 4 个 float

```
6.3 reshape / :`view` / `permute`

 ** ** , `A.view(64, 4)` 256
↪ `64x4` ;`A.permute(1, 0)` :

```python
A2 = A.view(64, 4); y = A2[tx, 0] + A2[tx, 3] # A2[tx, j] -> A_ptr[tx*4 + j]
At = A.permute(1, 0); z = At[i, j]          # At[i, j] -> A_ptr[j*4 + i]
```

“c++ A2_ptr[tx 4] / +3 // view: 行主序 64x4 的下标 At_ptr[(j 4) + i] // permute: 步长互换 (转置)

```
### 6.4      :`local`

`local`      ?      `local`
↪      , **      **      tile(      ) (tile
↪ primitives)      :

```python
R = T.alloc_buffer((32, 8), "float32", scope="local",
 layout=TileLayout(S[(32, 8) : (1 @ laneid, 1)]))
Rl = R.local(8) # lane 8
```

“c++ alignas(64) float Rl\_ptr[8]; // 这个 lane 私有的寄存器

```

> ** **: `1 @ laneid`, 0 lane(, warp 0-31)
↪ :`32` warp 32 lane, lane 8
↪ `R.local(8)` , warp lane

##

- ** **: = ** + ** = `data + elem_offset +
↪ layout.apply(coord)["m"] `view` / `permute` / `vload`
↪ , ** ** ,
- ** API**:`alloc_buffer` ;`decl_buffer` (`data=None`
↪) `alloc_shared` / `alloc_local` `scope`
- ** **: (, `__shared__`) (launch ,** kernel
↪ arena**,) `SMEMPool` , `move_base_to`
- ** **: `alloc_local`
↪ ;** ** ();
↪ `alignas(64)`,
- **`let` **: `let` , C ; ,
↪ load/store
- ** (TMEM)**:Blackwell ,`tcgen05.alloc` / `dealloc` ,`allocated_addr`
↪ , , `tcgen05` `TMEMPool` `SMEMPool` ,

##

- :[Buffers and memory Modern GPU Programming for MLSys](https://mlc.ai/modern-gpu-programming-for
↪ -mlsys/tirx_guide/language_reference/cuda/buffers.html)
- : (Data types and expressions) (`Var`),
↪ GEMM kernel(TMEM)

##

| | English |
| --- | --- |
| | buffer |
| | BufferRegion |
| | view |
| | layout / TileLayout |
| | stride |
| | elem_offset |
| | shared memory (SMEM) |
| | static shared memory |
| | dynamic shared memory |
| () | arena |
| | bump allocator |
| | registers |
| | scalar |
| | let / immutable binding |
| | tensor memory (TMEM) |
| | CSE (common-subexpression elimination) |
| | SROA (scalar replacement of aggregates) |
| warp | warp-uniform |

```



# 第 23 章 · 控制流

原文:Control flow

## 本章要点

### 本章要点 (TL;DR)

- TileIR(TIR-X) 里的控制流就三大类:if/else 分支、循环家族 (T.serial / T.unroll / T.vectorized / T.grid), 还有 while。每一类都会被「直白地」降级 (lower) 成对应的 CUDA C++ 写法。
- 想在两个值之间二选一, 用 T.if\_then\_else(cond, a, b)。它降级成三元运算符 ? :, 不产生分支、也不引入控制流发散 (divergence), 做 ReLU 这类逐元素取舍很合适。
- 一定要分清**统一控制流 (uniform control flow)**和**发散控制流 (divergent control flow)**: 逐线程的 if tx < 128 用来做普通计算没问题; 但**集体操作 (collective op)**(比如 \_\_syncthreads()) 必须被所有参与同步的线程**统一**到达, 否则会死锁。
- T.cuda.cta\_sync()(= \_\_syncthreads()) 绝不能放进任何线程级或 warpgroup 级发散的分支。只想同步单个 warpgroup 时, 改用 T.cuda.warpgroup\_sync(id)。还有 mbarrier.init() 这种「单线程守护」式的初始化, 同样不能嵌进发散分支, 否则屏障可能根本没被初始化。
- while 靠一个可变标量计数器实现, 底层降级成 while(1) + break, 而这个计数器其实是个单元素寄存器缓冲区 (register buffer)。

## 前置知识

**前置知识:** 读这一章前, 最好先懂 GPU 的线程层级 (thread / warp / lane / CTA / warpgroup) 和「同步」这个概念。没把握的话, 先翻一下第 0 章 · 极简入门。本章会默认你已经认识这些词。

这一章是「第五部分 · 参考与附录」里的语言参考 (language reference)。它干的事很简单: 把 TileIR 这门 DSL 的控制流原语, 一个一个跟它生成的 CUDA C++/PTX 代码对上号。这么一来, 你不光会写, 脑子里还能预判出生成的代码长什么样。

这事为什么值得单独讲? 因为 TileIR 不是「黑盒」。你写的每一条控制流语句, 几乎都能直接对应到一段 CUDA。说白了, 看 TileIR 代码, 你基本就能想象出底层会生成什么。

篇幅不长, 但里头有一节——**统一控制流 vs 发散控制流**——是 GPU 编程里最容易翻车的地方, 我们重点讲透它。

下面先把语法原语一个个过一遍, 再单独深挖「发散」这个大坑。

## 一、if / else: 分支

这是最省心的一种: 你在 TileIR 里写的 Python if / else, 原封不动就变成 CUDA 的 if / else, 中间什么转换都没有。

最常见的玩法，是拿线程号或 lane 号 (lane = 一个线程在它所属 warp 里的编号,0~31) 当「守护 (guard)」，让不同线程各走各的路：

```
TileIR :
if tx < 128:
 A[tx] = A[tx] * T.float32(2.0) # 128 : 2
else:
 A[tx] = A[tx] + T.float32(1.0) # : 1
```

降级后的 CUDA C++ 跟它一模一样，一行对一行：

“c++ if (((int)threadIdx.x) < 128) { A\_ptr[tx] = A\_ptr[tx] \* 2.0f; } else { A\_ptr[tx] = A\_ptr[tx] + 1.0f; }

```

: `tx` `threadIdx.x`, `T.float32(2.0)` `2.0f`,
:
: `if`, (warp = 32 ,GPU
↪ , 0) lane (divergence)
? :
- , ** **,
- ,
: `T.ptx.select_sync()`
, ** ** , TMA(Tensor Memory
↪ Accelerator) MMA() warp , warp
↪ warp 32 lane ,
, ** warp lane ** `T.ptx.select_sync()` :
```python
if T.ptx.select_sync():
    ... #      lane(      TMA / MMA)
```

它是怎么干的呢？在一个同步点上，它从当前活跃的 lane 里确定性地点出一个来，给这个 lane 返回 **true**，其余所有 lane 一律返回 **false**。你把它塞进 **if** 的条件，就写成了「只让一个线程发射」这个惯用套路。

关键

关键:`select_sync()` 选的是 warp 内部的「代表」，它的作用范围也只到 warp 里面。这个 warp 里所有活跃的 lane 都会执行到这条语句，只是被点中的那个拿到 **true**，别人拿到 **false**——选举在 warp 内部就尘埃落定了。注意，它管的「全员」最多到一个 warp，可不是整个线程块。所以它跟后面要讲的「CTA 级集体操作必须全员统一到达」(CTA / thread block = 一个线程块，跑在同一个 SM 上、能共享 SMEM 并互相同步的一组线程，见第 0 章) 压根是两码事，千万别搞混。

表达式级选择: `T.if_then_else`, 无分支

有时候你压根不想跑两段不同的代码, 只想在「两个值里挑一个」。这种时候, `T.if_then_else(cond, a, b)` 才是对的选择。

它好在哪? 它不生成 `if` 分支, 而是降级成 C 的三元运算符 `cond ? a : b`。

“`c++` // 例如 ReLU: 大于 0 取原值, 否则取 0, 无分支 `O_ptr[tx] = (A_ptr[tx] > 0.0f) ? A_ptr[tx] : 0.0f;`

```
> **                                ?**                                :
> -      `if`/`else`:                                ,                                (predication)                                warp                                lane
↪
> -      :                                `select`                                (                                / `selp`) **                                lane
↪
>      **,                                ,
>
>      :                                clamp relu `max(0,x)`
↪ `T.if_then_else`                                ,
                                :

```mermaid
flowchart TD
 Q{ }
 Q -->| " | A["if / else
-> / predication
warp]"
 ↪ ()
 Q -->| " | B["T.if_then_else(cond, a, b)
-> ? :
 "]
 style A fill:#ffe2e2,stroke:#c0392b
 style B fill:#e2ffe6,stroke:#27ae60
```

二、统一控制流 vs 发散控制流 (核心)

这一节是全章的核心, 也是 GPU 死锁 (deadlock) 最常冒出来的地方。先把两个概念掰开说清楚:

概念	含义	对应原语示例	放进发散分支会怎样
普通逐线程工作	每个线程独立算自己的, 互不依赖	<code>A[tx] = ...</code> , 逐元素运算	没问题, 顶多是性能上的发散开销
集体操作 (collective op)	一组线程必须「一起」到达并参与同步	<code>T.cuda.cta_sync()</code> (= <code>__syncthreads()</code> ) 屏障操作	可能死锁或未定义行为

核心规则, 一句话:

集体操作要被它同步的每一个线程「统一 (uniformly)」地走到——绝不能藏在只有一部分线程会进去的发散分支里。

经典死锁: 把 `cta_sync()` 放进发散分支

`T.cuda.cta_sync()` 降级成 `__syncthreads()`。它有个硬性要求: 线程块 (CTA / thread block) 里的每一个线程都得走到这个同步点, 大家才能一块儿往下走。

来看个反面例子——把它塞进一个按 `warpgroup`(`warpgroup` = 若干个连续 `warp` 打包成的一组, Hopper 之后常见的同步/分工粒度) 划分的分支里:

```
: !
if wg_id == 0:
 T.cuda.cta_sync() # warpgroup 0
 ...
```

问题出在哪？只有 warpgroup 0 的线程会进这个分支、走到屏障；别的 warpgroup 根本不会进来。结果 `__syncthreads()` 永远凑不齐所有线程，整个 kernel 就永久卡死（死锁）了。

下面这张图把这个坑画出来了：

### 结构图

连接关系：

- 线程块 (CTA) = 4 个 warpgroup → wg0
- 线程块 (CTA) = 4 个 warpgroup → wg1
- 线程块 (CTA) = 4 个 warpgroup → wg2
- 线程块 (CTA) = 4 个 warpgroup → wg3
- wg0 → if wg\_id == 0 ?
- wg1 → if wg\_id == 0 ?
- wg2 → if wg\_id == 0 ?
- wg3 → if wg\_id == 0 ?
- if wg\_id == 0 ?  $\xrightarrow{\text{是 (仅 wg0)}}$  cta\_sync() / 等待全部线程到齐
- if wg\_id == 0 ?  $\xrightarrow{\text{否 (wg1/wg2/wg3)}}$  跳过整个 if 体 / 各自继续 / 退出
- cta\_sync() / 等待全部线程到齐 → 永久阻塞：/ wg1/wg2/wg3 永远不来 / → 死锁

### 注意

**注意：**死锁的病根不在「同步」本身，而在「只有一部分线程会走到同步点」。集体操作天生就带着「全员都得到场」的约定，而发散分支恰恰把这个约定给破坏了。

### 正确做法：用 warpgroup 作用域的同步

那要是你本来就想「只让某一个 warpgroup 内部同步」呢？那就别用 CTA 级的 `cta_sync()` 了，换成 **warpgroup 作用域**的同步原语：

```
if wg_id == 0:
 T.cuda.warpgroup_sync(id) # warpgroup, wg
```

`T.cuda.warpgroup_sync(id)` 只在一个 warpgroup 内部立屏障，它要求的「全员」也就算这一个 warpgroup。所以把它放进 `if wg_id == 0:` 这种按 warpgroup 划分的分支里，反而是安全的——因为这个 warpgroup 的线程会**整整齐齐一起**进这个分支，没人会掉队。

关键

关键区别:

- `cta_sync()` 的「全员」= 整个线程块的所有线程 → 不能放进发散分支。
- `warpgroup_sync(id)` 的「全员」= 单个 `warpgroup` 的所有线程 → 可以放进「按 `warpgroup` 统一划分」的分支。

一句话: 用哪个, 就看你真正要同步的**范围**, 跟分支的**划分粒度**对不对得上。原文还提到,`warpgroup` 级同步最典型的舞台, 是「用 `warp` 专化 (`warp specialization`) 和 `cluster(cluster = 多个线程块再打成一组、能跨块协作的更大单位,Hopper 引入) 扩展 GEMM(General Matrix Multiply, 通用矩阵乘法, 深度学习里最核心的算子)」这类场景 (细节看相关章节, 还有 CUDA C++/PTX intrinsics 那一节)。`

同样的坑: 屏障初始化 (`mbarrier.init`)

发散这个坑, 不光在「同步」这一步会踩到, **屏障初始化**那一步同样危险。原文特意点了出来:`mbarrier`(内存屏障, 专门用来给异步操作如 TMA 拷贝「等结果到齐」的一种屏障) 的 `.init()` 会降级成一个**单线程守护**——只让一个线程去做初始化:

“`c++ if (threadIdx.x < 1) { // .init() 的本质: 只让 0 号线程去初始化屏障 / 初始化 mbarrier }`

```
 ,`.init()`` `if (threadIdx.x < 1)`
↪ ** **, : 0
↪ ,
 ?
↪ , , ** (unspecified launch failures)**

```mermaid
flowchart TD
    A["<br/>if some_cond:""] --> B["mbarrier.init()<br/>                                if(threadIdx.x &lt; 1)"]
    B --> C["0"]
    C --> D["?"]
    D --> E[""]
    E --> F["<br/>                                ->                                /                                "]
    style E fill:#ffe2e2,stroke:#c0392b
    style D fill:#e2ffe6,stroke:#27ae60
```

经验法则: 凡是带「集体语义」或「单线程守护语义」的原语——同步、屏障初始化、TMA/MMA 发射准备这些——都得保证它们待在**所有相关线程都会统一走到**的位置。要么搁在所有线程都会经过的直线代码里, 要么搁在「按同样粒度统一划分」的分支里。一旦把它们跟普通的逐元素发散分支搅在一块, 基本就是必出问题。

三、循环家族

TileIR 的循环有**四种口味**。你平时写的那个普通 Python `range`, 默认对应的就是其中的 `T.serial`:

原语	语义	降级结果 / 用途
T.serial(n)	顺序循环	生成普通 for;ptxas 仍可能自行展开 (unroll)
T.unroll(n)	完全展开	编译期 完全展开 为直线语句, 没有循环结构
T.vectorized(n)	向量化循环	生成向量化访问/计算
T.grid(*extents)	嵌套循环巢	一次性写出多层嵌套 for

break / continue 在这些循环里都照常能用。

T.grid: 一次写出嵌套循环

T.grid(8, 8) 其实就是两层嵌套 for, 只不过写起来比手写嵌套清爽多了:

```
for i, j in T.grid(8, 8):
    B[i, j] = T.max(A[i, j], T.float32(0.0))    #      ReLU
```

它降级成标准的双层 for。这里有个细节: 二维索引 (i, j) 被拉平成了 $i * 8 + j$:

“c++ for (int i = 0; i < 8; ++i) for (int j = 0; j < 8; ++j) B_ptr[i * 8 + j] = max(A_ptr[i * 8 + j], 0.0f);

```
### `T.serial` vs `T.unroll`:      vs

,      :**      **

| | `T.serial(4)`(`      `) | `T.unroll(4)`(`      ,      4      `) |
| --- | --- | --- |
|      | `for (int k=0; k<4; ++k)`<br/>` stmt(k);` |
↪ `stmt(k=0);`<br/>`stmt(k=1);`<br/>`stmt(k=2);`<br/>`stmt(k=3);` |
|      | ptxas      unroll | IR      ,      |

- `T.serial(n)`:      ,      ,      ptxas
- `T.unroll(n)`: IR      **      ** 4      ,      ,
↪      (      )      ,
↪      :      ,      (register =      ,      0 )

> **      **:      (ILP);      TileIR
↪      **      **,      ,

## `while`

`while`      ,      **      **

      TileIR      ,      (      / Buffers and memory
↪      ,      ):

```python
i: T.int32 = 0
while i < 64:
 A[i] = A[i] + T.float32(1.0)
 i += 1
```

它降级的路子有点意思: 它不直接生成 `while (i < 64)`, 而是生成一个 `while (1)` 死循环, 然后在循环一开头用 `break` 提前跳出来。计数器 `i` 呢, 被实现成了一个长度为 1 的寄存器数组 `i_ptr[1]`:

```
“c++ int i_ptr[1]; i_ptr[0] = 0; while (1) { if (!i_ptr[0] < 64) { break; } // 条件不满足就提前退出 A_ptr[i_ptr[0]] = A_ptr[i_ptr[0]] + 1.0f; i_ptr[0] = i_ptr[0] + 1; // 计数器自增 }”
```

```
> ** ?** IR TileIR IR
↳ , ** ** (SSA ,)
↳ , ? , ** ** 1
↳ , i += 1`      `i_ptr[0]`      ,      `while`
↳

> ** **:`while(1) + break` , `while(cond)`
↳ , , PTX/SASS
↳ ,

##

- TileIR :`if`/`else` `while`, CUDA C++ TileIR
↳ ,
- ** , `T.if_then_else`**, , ReLU clamp
↳
- ** , vs **: (`cta_sync()`) (
↳ `mbarrier.init()`), ** **,
↳ , `warpgroup_sync(id)` ,
↳
- / / ** **, `T.unroll`
↳ `T.serial` ,
- `while` , `while(1) + break` IR
↳ ,

##

- :[Control flow Modern GPU Programming for MLsys](https://mlc.ai/modern-gpu-programming-for-mlsys)
↳ /tirx_guide/language_reference/cuda/control_flow.html)
- (): Scaling GEMM with Warp Specialization and Clusters (warp cluster
↳ GEMM) CUDA C++/PTX intrinsics () Buffers and memory ()

##

| | English | | |
|---|---|---|---|
| | control flow | `if` / / `while` |
| | () | divergence / divergent control flow | warp lane |
| | | uniform control flow | |
| | | collective operation | () |
| | guard | |
| | () | elect (`elect_sync`) | warp lane |
| | | predication | |
| | | unroll | |
| | | vectorize | |
| | | deadlock | |
| | | CTA / thread block | `__syncthreads()` |
| | | warp | 32 lane / |
| warpgroup | warpgroup | warp |
```

	mbarrier			
	register buffer	1		



# 第 24 章 · CUDA C++ / PTX intrinsics

原文: [CUDA C++/PTX intrinsics](#)

## 本章要点

### 本章要点 (TL;DR)

- 平时我们写内核,用的都是高层的 **tile 原语 / tile primitive**。可有时候,你想要的那个硬件操作它表达不出来。这时候有两条「逃生通道」能让你直接捅到硬件:一条是**调用后端内建函数 / backend intrinsic**,也就是 `T.cuda.* / T.ptx.*` 这一堆命名空间;另一条是内联一段原始 **CUDA 源码**。
- `T.cuda.* / T.ptx.*` 干的活其实很朴素:把 CUDA 后端的设备内建函数原封不动地端到你面前。同步、`mbarrier`、归约都有,还有 PTX 那套数据搬运和 MMA 家族 (`cp_async`、`TMA`、`ldmatrix`、`tcgen05` 这些)。
- 写 GEMM、Flash Attention 这类内核,你会反复碰到 **四类同步机制**:`mbarrier` 相位 / `phase`、单线程选举 / `election`、具名 `warpgroup` 屏障 / `named barrier`、栅栏 / `fence`。它们管的是异步引擎和一群并行的线程。坑在哪?用错了它一般不报错,而是直接给你静默损坏数据,或者干脆死锁。
- 这四个里头,`mbarrier` 的相位坑藏得最深。同一个屏障要在循环里反复用,本地那个相位跟踪器每转一圈都得 `phase ^= 1`。一旦忘了翻,后面的等待会立刻返回——根本没等,放任引擎去读那块「才写了一半」的内存。
- 万一连内建函数都凑不齐,还有最后一招:`T.cuda.func_call(..., source_code=..., return_type=...)`。你给它一段 `__device__` 源码,它原样塞进去,还顺手帮你把调用点接好。

## 前置知识

**前置知识:** 读这一章前,最好先懂 GPU 的线程层级 (`warp / CTA / warpgroup`)、异步引擎 (`TMA`、异步 MMA) 和屏障 / 同步这几个概念。没把握的话,先翻一下第 0 章 · 极简入门。本章会默认你已经认识这些词。

## 24.1 为什么需要「逃生通道」

这本教材从头到尾的主线,就是用高层的 **tile 原语**来描述 GPU 内核。你只管声明数据布局、写 `tile`(把大矩阵切出来的小方块) 这一层的计算,剩下的全交给框架——它会把你的代码降低 / lower 成高效的 PTX。这套抽象用起来很舒服,常见的那些套路 (GEMM(通用矩阵乘法,深度学习里最核心的运算)、注意力 (attention)、归约……) 它基本都能包圆。

可问题来了:硬件跑得太快。Hopper(英伟达 GPU 架构代号,见第 0 章) 刚出 `TMA`(张量内存加速器,一个专搬大块数据的异步引擎),`Blackwell`(更新一代架构) 又甩出个 `tcgen05`。总有那么些

新能力,tile 原语**还没来得及封**,或者**压根就不打算封**(太底层、太专用了)。遇上这种情况,你总不能被抽象「锁死」、干瞪眼吧。这一章讲的,就是怎么绕过抽象、直接捅到硬件,一共两条路:

结构图

连接关系:

- 你的内核 (prim\_func) → 高层路径: tile 原语
- 你的内核 (prim\_func) → 逃生通道 1: T.cuda.\* / T.ptx.\* (后端内建函数)
- 你的内核 (prim\_func) → 逃生通道 2: T.cuda.func\_call(source\_code=...) (内联原始 CUDA 源码)
- 高层路径: tile 原语  $\xrightarrow{\text{框架自动降低}}$  PTX / SASS
- 逃生通道 1: T.cuda.\* / T.ptx.\* (后端内建函数) → PTX / SASS
- 逃生通道 2: T.cuda.func\_call(source\_code=...) (内联原始 CUDA 源码) → PTX / SASS

关键

**关键:** 这两条通道是来「补缺」的,不是来「顶替」tile 原语的。先后顺序很清楚: 能用 tile 原语就用 tile 原语; 原语够不着,才下沉到内建函数; 内建函数也没有,才退到内联源码。记住一句话: 越往下沉,框架帮你兜的底越少,正确性越得你自己扛——尤其是同步这块。

24.2 调用后端内建函数 (backend intrinsics)

T.cuda.\* 和 T.ptx.\* 这两个命名空间 (都来自 `tvm.backend.cuda`) 干的事很直白: 把 CUDA 后端的设备内建函数原封不动端给你。你能拿到的东西包括线程同步、mbarrier、归约 (把一组数汇成一个值,比如求和),还有 PTX 那套数据搬运 / MMA(矩阵乘加,Tensor Core 干的活) 家族。

24.2.1 常见入口一览

先看看最常用的那几类入口长啥样:

```
T.cuda.cta_sync() # CTA() , __syncthreads()
T.cuda.warp_sync() # warp , __syncwarp()
T.cuda.wargroup_sync(8) # wargroup (, 24.3.3)
T.cuda.cta_sum(val, num_warps, scratch.ptr_to([0])) # CTA

bar = T.alloc_shared((1,), "uint64") # mbarrier
T.ptx.mbarrier.init(bar.data, 1) # mbarrier,
T.ptx.mbarrier.try_wait(bar.data, phase) # (, 24.3.1)
```

一个一个说:

内建函数	作用	对应底层
T.cuda.cta_sync()	整个 CTA(线程块) 对齐	__syncthreads()
T.cuda.warp_sync()	一个 warp 内对齐	__syncwarp()
T.cuda.wargroup_sync(id)	一个 wargroup 对齐 (走具名屏障)	命名屏障 (bar.sync)

T.cuda.cta_sum(val, num_warps, scratch)	CTA 级归约求和 (需一块 scratch 共享内存做中转)	多步 warp shuffle + 共享内存
T.ptx.mbarrier.init(bar, n)	把共享内存里的 mbarrier 初始化为期望 n 次到达	mbarrier.init
T.ptx.mbarrier.try_wait(bar, phase)	按相位非阻塞地等待屏障翻转	mbarrier.try_wait.parity

注意

注意:mbarrier 说白了就是 **共享内存 / SMEM** 里的一个 **uint64**。所以你得先用 `T.alloc_shared((1,), "uint64")` 给它腾块地方, 再把 `.data`(也就是那根指针) 递给 `init / try_wait`。它专门干一件事: **等异步引擎** (像 TMA 拷贝、异步 MMA) 把活干完。到了 Hopper/Blackwell 这代, 异步流水线 (让搬数据和算数据重叠起来跑, 互不相等) 的同步全靠它, 算得上是顶梁柱级别的原语。

24.2.2 完整可运行示例:warp 内 all-reduce

原文给了个例子: 用 `T.tvm_warp_shuffle_xor` 做「warp(一个 warp = 32 个线程的小班, 齐步走, 见第 0 章) 内全归约求和」。要搞懂 shuffle 内建函数, 这个例子是再合适不过了。咱们先看代码, 然后一点点把思路捋清楚:

```
@T.prim_func
def warp_reduce(A_ptr: T.handle):
 A = T.match_buffer(A_ptr, (32,), "float32", align=16)
 T.device_entry()
 # :CTA / warp / lane (0..31)
 cta_id = T.cta_id([1]); warp_id = T.warp_id([1]); lane_id = T.lane_id([32])
 v = T.alloc_local((1,), "float32"); i = T.alloc_local((1,), "int32")
 v[0] = T.float32(31 - lane_id) # lane ()
 i[0] = 16 # = 32/2
 while i[0] >= 1:
 # : i lane (butterfly)
 v[0] += T.tvm_warp_shuffle_xor(0xFFFFFFFF, v[0], i[0], 32, 32)
 i[0] = i[0] // 2 # :16 -> 8 -> 4 -> 2 -> 1
 A[lane_id] = v[0] # , 32 lane
```

**核心理路: 蝶形归约 / butterfly reduction.** 先给个直觉: 它就是「两两配对、交换相加, 然后步长一层层折半」这么个求和办法,5 步下来,32 个 lane(lane = warp 里每个线程的编号,0~31) 个个都攒齐了总和。

`__shfl_xor_sync` 妙就妙在配对的方式上。每个 lane 都跟「自己 lane 号异或 i」算出来的那个 lane 换数据。i 一路取 16, 8, 4, 2, 1, 信息 5 步就传遍整个 warp。最后的效果是:**32 个 lane 全都拿到同一个总和**, 而不是只有 lane 0 一家拿到。它叫「all-reduce」(全归约), 原因就在这儿。

warp 内 32 个 lane 的蝶形归约 (以步长 i 的变化为例):

步长 i	配对 (lane ⇔ 异或 i 得到的 lane)
i=16	lane 0 ⇔ lane 16, lane 1 ⇔ lane 17, ..., lane 15 ⇔ lane 31

i=8	lane 0 ⇔ lane 8, lane 1 ⇔ lane 9, ...
i=4	lane 0 ⇔ lane 4, ...
i=2	lane 0 ⇔ lane 2, ...
i=1	lane 0 ⇔ lane 1, ...

5 步走完后, 所有 lane 持有相同的最终和 (all-reduce, 不是单点 reduce)。

这个 `T.tvn_warp_shuffle_xor`, 框架会把它原样降低成对应的 CUDA 内建函数 `__shfl_xor_sync`, 基本就是一对一的直译:

“`c++` // 框架生成的 `C++`, 几乎是一行直译 `v_ptr[0] = v_ptr[0] + __shfl_xor_sync(0xFFFFFFFF, v_ptr[0], i_ptr[0], 32);`

```
> ** **: `0xFFFFFFFF` **lane / lane mask**, 32 lane ,
↳ `__sync` shuffle lane ** **, ,
↳ ? Volta ,GPU / independent thread scheduling,warp
↳ , `__sync` shuffle
↳ ; ,

24.2.3

 shuffle `T.ptx.*` / `T.cuda.*` , :

| | | |
| --- | --- | --- |
| `cp_async` (LDGSTS) | | tile |
| `cp_async.bulk.tensor` (TMA) | (Tensor Memory Accelerator) | Hopper tile
↳ |
| `ldmatrix` / `stmatrix` | / MMA / | / Tensor Core |
| `tcgen05.*` | Blackwell 5 Tensor Core MMA | Blackwell GEMM |
| `atomic_add` | | |
| `fence` | | (proxy) (24.3.4) |

> ** **: , , `cp_async` Ampere
↳ ,TMA(`cp_async.bulk.tensor`) Hopper ,`tcgen05.*` Blackwell
↳ , , , `tvm.backend.cuda` API

24.3 (synchronization semantics)

 , GEMM Flash Attention , ** **
↳ ** ** ** ** ? , **
↳ / silent corruption, / deadlock** bug

24.3.1 mbarrier (Mbarrier Phases)

 :mbarrier ** / phase bit**,
↳ ,

`T.ptx.mbarrier.try_wait(bar, phase)` ?

> ** , , `phase` **

 : `phase`, ,
↳ , `phase` `phase ^ 1`
```

```

**
↪ ** **, **, `phase ^= 1`
↪ ? :

- `try_wait(bar, 0)` : 1, ,
- , `try_wait(bar, 0)` 1 , 0
↪ ** **, ** **,
- : , **
↪ memory**,

 (`phase`):

| | phase | | | `phase ^= 1` |
| --- | --- | --- | --- | --- |
| 0 | 0 | `try_wait(bar, 0)` | 1 | phase = 1 |
| 1 | 1 | `try_wait(bar, 1)` | 0 | phase = 0 |
| 2 | 0 | `try_wait(bar, 0)` | 1 | ... |

 (phase):

| | phase | | |
| --- | --- | --- | --- |
| 1 | 0(1) | `try_wait(bar, 0)` | 1, !=0 -> ! ->
↪ -> |

> ** **: ** ** , `phase`
↪ , , Building a Tiled GEMM
↪ ,

24.3.2 (Election)

`T.ptx.select_sync()` , ** warp lane**
↪ , ** ** :

- ** lane 0** , , 0
- ** CTA ** warp , CTA
- ** lane / active lanes** lane,

 : ** CTA ** (TMA MMA),
↪ `select_sync()` ? warp , CTA warp,
↪ ? **warp / warp-level guard**:

```python
if warp_id == 0:          #          :          0      warp
    if T.ptx.select_sync():          #          :          warp          lane
        #          :          CTA
        Tx.gemm_async(...)          #          GEMM
        # tcgen05.commit(...)          #          Blackwell MMA

```

结构图

分组:

- **CTA (多个 warp):** 「warp 0」 「warp 1 (被守卫挡掉)」 「warp 2 ... (被守卫挡掉)」

连接关系:

- warp 0 $\xrightarrow{\text{if warp_id} == 0 \rightarrow \text{只剩 warp 0}}$ lane0 ... lane31
- lane0 ... lane31 $\xrightarrow{\text{elect_sync}() \text{ 选 1 个}}$ 整个 CTA 恰好 1 个线程

注意

注意: 为啥非得「恰好一个线程」不可？因为 `gemm_async / tcgen05.commit` 这类指令，是冲着异步引擎下命令去的，同一条命令发一遍就够了。要是放任好几个线程一起发，轻的是重复发起，重的直接给你未定义行为。所以「先用 `if warp_id == 0` 守一道，再用 `elect_sync()` 挑一个」，就是把「一大堆线程在跑」稳稳收成「单个线程发令」的标准套路，以后照抄就行。

24.3.3 具名 warpgroup 屏障 (Named Warpgroup Barriers)

先回顾一下：`T.cuda.cta_sync()` 对应的是 `__syncthreads()`，它非得 CTA 里每个线程都到齐了才放行。要是内核里所有线程跑的都是同一套代码（没做 warp 特化），这倒没啥问题。

可一旦你上了 **warpgroup 特化 / warp specialization** (`warpgroup = 4` 个 warp 抱团, Hopper 起的概念; warp 特化 = 让不同的线程组各干各的专职活)——也就是不同的 warpgroup 走不同的代码路径（最典型的，就是「生产者 warpgroup 专管搬数据，消费者 warpgroup 专管算 MMA」）——麻烦就来了。这时候你要是把 `cta_sync()` 塞进某个 warpgroup 的分支里头，直接死锁给你看。道理不难：别的 warpgroup 走的是另一条岔路，压根就不会跑到这个 `cta_sync()`；偏偏 `__syncthreads()` 又非要等齐所有人。于是大家你等我我等你，谁也动不了。

硬件早给这事备好了解法：**16 个具名屏障 / named barriers**，编号 0~15。`T.cuda.warpgroup_sync(10)` 只同步某一个 warpgroup 内部的线程，别的 warpgroup 它一概不管：

```
#      warpgroup      ID,
T.cuda.warpgroup_sync(wg_id + 10)  #      wg_id=0   10, wg_id=1   11 ...
```

结构图

分组：

- 必须全员到齐，放进分支会死锁：「warp0」「warp1」「... warpN」

连接关系：

- `cta_sync()` (`__syncthreads()`) $\rightarrow$ 必须全员到齐，放进分支会死锁
- `warpgroup_sync(10)` $\xrightarrow{\text{用屏障 ID 10}}$ 仅生产者 warpgroup 内部对齐
- `warpgroup_sync(11)` $\xrightarrow{\text{用屏障 ID 11}}$ 仅消费者 warpgroup 内部对齐

对比	<code>cta_sync()</code>	<code>warpgroup_sync(id)</code>
同步范围	整个 CTA 的所有线程	单个 warpgroup 内的线程
底层	<code>__syncthreads()</code>	16 个具名屏障之一
放进 warpgroup 分支里	死锁	安全 (其它 warpgroup 不受影响)
ID 管理	无需	必须给不同 warpgroup 分配不同 ID

关键

关键: 具名屏障的 ID 是有限的硬件资源，拢共就 16 个。你得跟分配寄存器（每个线程私有的最快存储，数量很少）似的，规划着用，千万别让两个 warpgroup 撞到同一个号上。原文那句 `warpgroup_sync(wg_id + 10)` 是个小巧的手法：拿 warpgroup 自己的编号去算屏障号，这样每个 warpgroup 自动就分到一个独一无二、不打架的号。实战怎么用，见原文「Scaling GEMM with Warp Specialization and Clusters」那章。

24.3.4 栅栏 (Fences)

栅栏管的是**定序 / ordering**：它保证「生产者先写」「消费者（往往是某个异步引擎）后读」这个先后顺序不会被打乱。

别拿它跟屏障弄混。一句话区分：屏障管的是「等大家到齐」，栅栏管的是「我写下的东西，后面的读者一定看得见，而且顺序没乱」。

原文列了三种栅栏：

栅栏	它保证的定序
<code>T.ptx.fence.proxy_async("shared::cta")</code>	让「线程写进共享内存的东西」，能被 异步代理 (async proxy) (比如 TMA store、MMA) 随后读到。普通线程访存和异步引擎访存走的是不同的「代理」，得靠这道栅栏把两边接上。
<code>T.ptx.fence.mbarrier_init()</code>	保证 mbarrier 的 初始化 排在前头，任何对这个屏障的到达 (arrive) 或等待 (wait) 都在它之后。不然别的线程可能在屏障还没建好的时候就去用了。
<code>T.ptx.tcgen05.fence.after_thread_sync()</code>	在 <code>tcgen05</code> 写回 (writeback) 这条边上，加一道 偏保守的定序栅栏 。原文说它出现在「Step 8 和 Step 9」，而且 在 TMA→MMA 这条路径上用不着 。

注意

注意: 要搞懂 `fence.proxy_async`，关键是「代理 / proxy」这个词。在 Hopper/Blackwell 上，普通线程的访存和异步引擎 (TMA/MMA) 的访存，被当成两条各走各的内存「代理」通道。它俩各管各的可见性顺序，谁也不知道对方写了啥。所以你哪怕 `__syncthreads()` 一下，也没法跨代理保证「写完了才读」。`fence.proxy_async("shared::cta")` 干的就是把这两条代理通道接通，在 CTA 共享内存这个范围里硬把顺序定下来。

关键

关键: 第三种栅栏 `tcgen05.fence.after_thread_sync()`，原文白纸黑字说它「偏保守 / conservative」，而且「TMA→MMA 这条路径上用不到」。这给我们提了个醒：栅栏不是白来的，它有性能开销，**不能见缝就往里插**。该加的地方 (`tcgen05` 写回) 加上，不该加的地方 (TMA→MMA) 就省掉——这样既不出错，又不白白搭上性能。

24.4 内联原始 CUDA(Inlining raw CUDA)

万一你要的功能连内建函数都没有，别急，还有最后一招：用 `T.cuda.func_call` 把一段 `__device__` 源码原样塞进去，调用点框架会帮你自动接好。

```

SRC = r"""
__device__ __forceinline__ float my_relu(float x) { return x > 0.f ? x : 0.f; }
"""

@T.prim_func
def k(A_ptr: T.handle, B_ptr: T.handle):
    A = T.match_buffer(A_ptr, (256,), "float32")
    B = T.match_buffer(B_ptr, (256,), "float32")
    T.device_entry(); bx = T.cta_id[1]; tx = T.thread_id[256]
    #           :       +       +       +
    B[tx] = T.cuda.func_call("my_relu", A[tx], source_code=SRC, return_type="float32")

```

`func_call` 有四个要紧的参数:

- **name**: 你要调用的那个设备函数叫啥。注意, 它得跟 `source_code` 里定义的名字对上。
- ***args**: 递给它的实参, 这里就是 `A[tx]`。
- **source_code=...**: 一段源码字符串, 框架会一字不改地 / **verbatim** 发射到生成的代码里。
- **return_type=...**: 返回类型, 框架靠它把调用的结果接回表达式, 这里是 `"float32"`。

降低之后的 C++ 大概是这副样子——源码原样落地, 调用点自动接上:

```

“c++ __device__ __forceinline__ float my_relu(float x) { return x > 0.f ? x : 0.f; }
// ... B_ptr[tx] = my_relu(A_ptr[tx]); // 调用点由框架自动生成

```

```

```mermaid
flowchart TD
 SRC["SRC (__device__)"]
 SRC -- "source_code=..." --> EMIT[" .cu ()"]
 EMIT -- "name + args + return_type" --> CALL["B_ptr[tx] = my_relu(A_ptr[tx]); ,"]
 CALL --> return_type["return_type"]

```

### 注意

**注意:** 这是逃生通道里最底下的一档, 动手前先想清楚。源码一字不改地发射, 反过来说就是: 除了类型推断, 框架几乎啥检查、啥兜底都不给你做。拼写有没有错、函数签名对不对、`__forceinline__` 这类修饰加没加, 全得你自己写对。所以但凡 `T.cuda.*` / `T.ptx.*` 内建函数能办的事, 就别退到这一层来。

## 小结

- 本章给了你两条直达硬件的逃生通道。它俩的定位是一样的:tile 原语够不着时, 拿来补缺。
  - 后端内建函数 `T.cuda.*` / `T.ptx.*`——同步、mbARRIER、归约、shuffle, 加上 `cp_async/TMA/ldmatrix/t` 这些数据搬运和 MMA 家族, 基本就是一条通往 CUDA/PTX 设备内建函数的直通车。
  - 内联原始 `CUDA T.cuda.func_call(source_code=...)`——连内建函数都没有时的最后兜底, 把 `__device__` 源码一字不改地塞进去。
- 本章最大的雷区, 是同步语义。四个机制, 每个都各自带着一个「最容易踩」的坑:



机制	一句话记忆	最容易踩的坑
mbarrier 相位	try_wait 等的是相位「翻转」	循环复用时忘了 phase ^= 1 → 不等就返回 → 读半写内存
单线程选举	elect_sync() 是「每 warp 一个」	误以为是 lane 0 / 每 CTA 一个; 需配 if warp_id == 0 守卫
具名 warpgroup 屏障	只同步本 warpgroup, 共 16 个	cta_sync() 放进 warpgroup 分支 → 死锁; ID 撞车 → 互相干扰
栅栏	定序「写在前、读在后」	跨代理 (proxy) 不加栅栏 → 异步引擎读到旧值; 乱加 → 损性能

- 最后送你一个统一的心智模型：逃生通道越往下沉，框架帮你兜的底就越少。从 tile 原语 → 内建函数 → 内联源码，正确性的担子一级一级往你自己肩上压。尤其是异步和同步的定序，千万别凭感觉瞎写，一定对着实战章节里的相位跟踪表和栅栏摆放规则来。

延伸阅读

- 原文链接:[CUDA C++/PTX intrinsics](#)
- 原文交叉引用的实战章节 (用于把本章机制落地):
  - Building a Tiled GEMM** ——完整走一遍 mbarrier 相位跟踪表, 以及 elect\_sync() + warp 守卫发起 gemm\_async / tcgen05.commit。
  - Scaling GEMM with Warp Specialization and Clusters** ——具名 warpgroup 屏障在 warp 特化下的用法与 ID 规划。
- tvm.backend.cuda 后端 API 参考——T.cuda.\* / T.ptx.\* 全部内建函数的完整清单。

术语对照

中文	English
后端内建函数	backend intrinsic
tile 原语	tile primitive
线程束	warp
线程束组	warpgroup
共享内存	SMEM(shared memory)
线程块	CTA(thread block)
蝶形归约	butterfly reduction
全归约	all-reduce
通道 (lane)	lane
通道掩码	lane mask
相位	phase(mbarrier phase)
单线程选举	election(elect_sync)
具名屏障	named barrier
栅栏	fence
异步代理	async proxy
张量内存加速器	TMA(Tensor Memory Accelerator)
静默数据损坏	silent corruption
死锁	deadlock
warp 特化	warp specialization



# 术语对照表 (全书总表)

本表汇总《现代 GPU 编程·面向 ML 系统 (中文精读笔记)》各章出现的核心术语, 去重整理为「英文—中文—简要说明」三列。

同一概念在不同章节可能有略微不同的中文译法 (如 *warp specialization* 既见「Warp 专门化」也见「warp 专精」「warp 特化」), 本表取常用译法并在说明中兼顾。返回首页:README。

## A · 架构与执行模型

英文	中文	简要说明
SIMT (Single Instruction Multiple Threads)	单指令多线程	GPU 的基本执行范式: 一条指令同时驱动 warp 内多个线程。
warp / lane	线程束 / 通道	32 个线程组成一个 warp, warp 内每个线程占一条 lane (由 laneid 标识)。
warpgroup	线程束组 / warp 组	4 个 warp (128 线程) 组成一组, 是新一代 MMA 的调度单位。
CTA (Cooperative Thread Array)	协作线程阵列 / 线程块	即 thread block, 共享同一块 SMEM 的线程集合。
cluster / thread block cluster	集群 / 线程块集群	多个 CTA 组成的集群, 可跨 CTA 共享内存与协作。
scope	作用域 (谁来执行)	指定一段代码由哪一层级 (线程 / warp / warpgroup / CTA) 执行。
occupancy	占用率	SM 上活跃 warp 数与硬件上限之比, 影响延迟隐藏能力。
overlap	重叠	让数据搬运与计算同时进行, 是性能优化的核心杠杆。
persistent kernel	持久化核 / 常驻核	一个 CTA 常驻 SM 循环领取多个 tile, 摊薄启动开销。
grid scheduler / grid stride	网格调度器 / 网格步长	网格层面分发工作的调度器, 及持久化核循环领取 tile 的步长。
work-stealing	工作窃取	空闲 SM 主动抢占未完成 tile, 改善尾部负载均衡。
critical path	关键路径	决定内核总耗时最长依赖链。
tail behavior	尾部行为	网格末尾 SM 负载不均导致的性能损耗现象。

## B · 性能模型

英文	中文	简要说明
roofline model	屋顶线模型	以内存带宽与算力吞吐为上界、用算术强度定胜负的性能模型。
arithmetic intensity (AI)	算术强度	每搬运一字节数据所执行的浮点运算数 (FLOP/Byte)。

memory-bound / compute-bound	内存受限 / 算力受限	性能瓶颈分别落在带宽端或算力端的两种状态。
ridge point	脊点	屋顶线上内存受限与算力受限的分界点。
fusion	算子融合	合并多个算子以减少中间结果的访存往返。
blocking for reuse	为复用而分块	把数据切成 tile 留在片上反复使用，提高算术强度。

C · 数据布局 (Layout)

英文	中文	简要说明
data layout	数据布局	逻辑张量到物理存储位置的映射规则。
shape / stride	形状 / 步长	张量各维的大小，以及沿各维移动一格对应的物理偏移。
view	视图	不复制数据，仅改变 shape/stride 的张量重解释。
tile / tiling	瓦片 / 分块	把大张量切成可放入片上内存的小块。
named axes (TLane / TCol)	命名轴	与硬件对应的具名逻辑轴（如 TMEM 的 Lane/Col）。
affine map	仿射映射	用线性公式描述逻辑坐标到物理地址的映射。
forward mapping	前向映射	一个逻辑坐标映射到一组物理坐标的方向。
register fragment	寄存器片段	MMA 操作数 / 结果在各线程寄存器中分布的小块。
replication / broadcast	复制 / 广播	同一份数据被多个线程持有的布局模式。
sharding / shard	分片	把数据切分给不同线程 / 单元各持一份。
offset	偏移	布局起始位置相对基址的位移。
iter (extent, stride, axis)	迭代器三元组	描述一维遍历的范围、步长与所属轴。
atom	原子单元	布局可重复拼贴的最小基本块。
swizzle	混排 / 地址混洗	对 SMEM 地址做 XOR 重排以规避 bank 冲突。
swizzle atom	swizzle 原子	swizzle 模式可重复的最小单元。
bank conflict	bank 冲突	多线程同周期访问同一 SMEM bank 导致的串行化。
global memory coalescing	全局内存合并访问	warp 内连续地址合并成少量内存事务的高效访问。
warpx4 broadcast	warpx4 广播	一份数据广播给 4 个 warp 共享的布局。
TileLayout	仿射布局	TIRx 的主力布局对象，用 shard/replica/offset 描述仿射映射。
SwizzleLayout / ComposeLayout	换序布局 / 组合布局	XOR 换序布局，以及把多个布局组合成新布局。
tensor layout model	张量布局模型	贯穿全书的「逻辑坐标 → 物理坐标集合」统一模型。

D · Tensor Core 与 MMA

英文	中文	简要说明
Tensor Core	张量核 (固定功能单元)	专做矩阵乘累加的硬件单元，非通用 ALU。
MMA (Matrix-Multiply-Accumulate)	矩阵乘累加	Tensor Core 执行的核心操作 $D = A \cdot B + C$ 。
accumulator	累加器	存放 MMA 累加结果的寄存器 / TMEM 区域。
tcgen05	第 5 代 Tensor Core 指令族	Blackwell 的 MMA 指令集，结果写入 TMEM。
matrix descriptor	矩阵描述符	告诉 MMA 操作数在 SMEM 中布局 / 主序的元数据。
major mode (K-major / MN-major)	主序 (K 主序 / MN 主序)	操作数在内存中沿 K 还是 MN 维连续。
block-scaled MMA / scale factor	块缩放 MMA / 缩放因子	低精度 MMA 按块附带缩放因子以保持数值范围。
epilogue	收尾阶段 / 尾声阶段	MMA 后做缩放、加偏置、类型转换并写回的阶段。

E · 内存层级与特殊内存

英文	中文	简要说明
TMEM (Tensor Memory)	张量内存	Blackwell 专供 Tensor Core 累加的二维 (Lane $\times$ Col) 片上内存。
Lane / Col (TLane / TCol)	TMEM 二维坐标轴	TMEM 的两个寻址维度。
allocated_addr (column offset)	列偏移地址	TMEM 分配返回的列方向起始地址。
DSMEM (Distributed Shared Memory)	分布式共享内存	集群内一个 CTA 可访问另一 CTA 的 SMEM。
static shared memory	静态共享内存	编译期确定大小的 SMEM。
dynamic shared memory / arena	动态共享内存 / 竞技场	运行时按需从竞技场分配的 SMEM。
bump allocator (SMEMPool)	碰撞分配器	顺序推进指针的轻量 SMEM 分配器。
buffer	缓冲区	指针之上附加 shape/stride/layout 元数据的抽象。
metadata over a pointer	指针之上的元数据	buffer 的本质: 裸指针 + 布局信息。
elem_offset	元素偏移	buffer 起始元素相对基指针的偏移量。
register buffer	寄存器缓冲区	驻留在寄存器中的小型缓冲区。
scalar	标量	单元素本地缓冲区的语法糖。

F · 异步搬运与协调

英文	中文	简要说明
TMA (Tensor Memory Accelerator)	张量内存加速器	异步批量在全局与共享内存间搬运 tile 的硬件引擎。
tensor-map descriptor	张量映射描述符	描述全局张量布局供 TMA 寻址的对象。
tile coordinates	tile 坐标	TMA 按整块定位时使用的块级坐标。
mbarrier (memory barrier)	内存屏障 / 异步屏障	到达计数 + 相位 + 字节计数三合一的异步同步原语。

arrival counter / arrival count	到达计数器 / 到达计数	记录已到达 barrier 的参与者数量。
phase / phase bit	相位 / 相位位	barrier 在两个相位间翻转，用于区分世代。
arrive / wait	到达 / 等待	参与者声明到达，以及阻塞等待 barrier 满足。
expect_tx / arrive.expect_tx	期望字节预算	提前声明本世代期望接收的字节数。
complete-tx / tx-count / byte count	完成事务 / 字节计数	异步引擎回报已完成的事务 / 事务数。
byte-count completion	字节数完成	以累计字节数达到预算作为完成判据 (区别于到达计数)。
try_wait / try_cancel / query_cancel	等待释放 / 请求 / 查询指令	非阻塞探测 barrier，以及 CLC 的请求 / 查询新 tile。
commit group / wait group	提交组 / 等待组	把一批异步操作打包提交并按组等待完成。
commit	提交	把 MMA 工作组提交给硬件执行。
double buffering / ring buffer	双缓冲 / 环形缓冲	用多份缓冲让搬运与计算重叠。
multi-stage buffering	多级缓冲	超过两级的流水缓冲。
stage	流水级	流水线中的一个阶段。
ownership transfer	所有权转移	缓冲区在生产者与消费者间交接使用权。
async proxy	异步代理	区分同步 / 异步访存的内存代理，需配 fence 保证可见性。

G · CLC 与调度

英文	中文	简要说明
Cluster Launch Control (CLC)	集群启动控制	让常驻集群运行时动态领取新输出 tile 的机制。
tile scheduler / l2_group_size	tile 调度器 / L2 分组大小	决定 tile 领取顺序的调度器，及为 L2 复用而分组的大小。
output tile	输出分块	一个 CTA / 集群负责计算的结果块。
sentinel	哨兵值	标记「无更多工作」等特殊状态的占位值。
predicate	谓词	控制指令是否生效的布尔条件。

H · Warp 专门化与调试

英文	中文	简要说明
warp specialization	Warp 专门化 / 专精	不同 warp 各司其职 (如生产者搬运 vs 消费者计算)。
handoff	交接	一段数据 / 资源在角色间移交的事件，是调试建模的核心。
role guard	角色守卫	用条件分支把代码限定给特定角色 warp 执行。
collective operation / collective op	集体操作 / 集合操作	必须由作用域内全体参与者一起执行的操作。
context poisoning	上下文毒化	在错误作用域调用集体操作污染后续状态的错误。

race condition	竞争条件	因缺乏同步导致结果依赖执行时序的缺陷。
silent corruption	静默数据损坏	不报错却产生错误结果的隐蔽缺陷。
deadlock	死锁	参与者互相等待、谁也无法前进的僵局。

I · TIRx 语言与控制流

英文	中文	简要说明
TIRx (Tensor IR neXt)	张量 IR neXt	IR 层级的 GPU 编程 DSL，显式表达 <code>scope/layout/dispatch</code> 。
dispatch	派发	选择某操作走哪条具体硬件路径。
layout	布局	数据 <code>tile</code> 住在哪、怎么排 (见上方 <code>Layout</code> 专节)。
dtype	数据类型	低层位级类型 (如 <code>float32x4</code> )。
type / PrimType / PointerType	高层类型 / 指针类型	IR 中的高层类型系统。
PrimExpr	原始表达式	TIRx 的基础表达式节点。
handle / pointer	句柄 / 指针	通用指针 <code>dtype</code> 。
vector dtype	向量类型	如 <code>float32x4</code> $\rightarrow$ <code>float4</code> 的打包向量。
vectorized memory access	向量化访存	用 <code>vload</code> / <code>vstore</code> 一次搬多个元素。
let / immutable binding	不可变绑定	一次性赋值、不可重赋的命名值。
scalar (sugar)	标量 (语法糖)	1 元素本地缓冲区的便捷写法。
uniform vs divergent control flow	统一 vs 发散控制流	<code>warp</code> 内是否所有线程走同一分支。
elect_sync / election	选举单线程发射 / 选举	在 <code>warp</code> 内推选一个线程执行单线程操作。
T.if_then_else / ternary	表达式级选择 / 三元运算符	表达式而非语句层面的条件选择。
cta_sync / __syncthreads	线程块同步	CTA 内全体线程同步屏障。
warpgroup_sync / named warpgroup barrier	warpgroup 同步 / 具名 warpgroup 屏障	warpgroup 作用域的 (可具名) 同步。
mbarrier.init	内存屏障初始化	初始化一个 <code>mbarrier</code> 对象。
fence	栅栏	强制内存可见性 / 顺序的屏障。
unroll / serial / vectorized / grid	完全展开 / 顺序 / 向量化 / 嵌套循环	循环的不同执行修饰符。
intrinsic	内建函数	直接映射到底层指令的函数。
backend intrinsic	后端内建函数	暴露 CUDA C++ / PTX 能力的内建。
butterfly reduction	蝶形归约	warp 内用 <code>shuffle</code> 做对数级归约的模式。
escape hatch	逃生口	绕过高层抽象、直达底层指令的机制。

J · 解析期 / 元编程

英文	中文	简要说明
parser utilities	解析器工具	在解析阶段生效的一组元编程辅助。
parse time / parse-time metaprogramming	解析期 / 解析期元编程	在源码被解析为 IR 时执行的元编程。
metaprogramming	元编程	用代码生成 / 操控代码的技术。
meta value / meta_var	元值	仅存在于解析期、不进入运行时的值。
inline (@T.inline)	内联 (就地展开)	把函数体在调用处直接展开。

meta_class (@T.meta_class)	元类	解析期可参数化展开的类模板。
constexpr / compile-time constant	编译期常量	编译期即可求值的常量。
specialize	特化	为特定参数生成专门版本的代码。
unroll	循环展开	解析期把循环体复制多份。
lexical (LEGB) scoping	词法作用域	按代码书写层次解析名字的规则。
late binding	延迟绑定	名字到值的绑定推迟到使用时确定。
shadowing	遮蔽 (同名覆盖)	内层同名变量遮住外层变量。

K · 编译器内部与下降

英文	中文	简要说明
lowering / lowering pipeline	下降 / 降级 / 下降流水线	把高层 IR 逐步转成低层代码的多 pass 流程。
tirx pipeline	TIRx 流水线	<code>tvm.compile(tir_pipeline="tirx")</code> 内部的下降流水线。
LowerTIRx	TIRx 核心下降 pass	流水线中最关键的下降阶段。
tile primitive	tile 原语 / 瓦片原语	以 tile 为操作单位的高层算子原语。
TilePrimitiveDispatch	tile 原语派发	把 tile 原语映射到具体硬件实现的子阶段。
LayoutApplier	布局应用器 / 施加器	把 TileLayout 实际落到缓冲区上的子阶段。
execution-scope id	执行域 id	标识某段代码所属执行作用域的标记。
SplitHostDevice	host/device 拆分	把混合代码拆成主机与设备两个函数。
host function / device function	主机函数 / 设备函数	分别在 CPU 与 GPU 上运行的函数。
packed-func ABI	打包函数 ABI	TVM 主机 / 设备调用约定。
finalization	收尾 (阶段)	下降流水线末端的整理 / 渲染阶段。
warp-scoped memory	线程束作用域内存	归属于单个 warp 的内存。
safety gate	安全闸门	拦截非法 IR、保证下降正确性的检查。
SROA / CSE	聚合体标量替换 / 公共子表达式消除	两类标准编译优化。

L · 参考与生态

英文	中文	简要说明
Compiler Internals	编译器内部实现	面向贡献者的编译器架构文档。
TIRx Language Reference	TIRx 语言参考	TIRx 语言特性的权威说明。
TIRx Layout API	TIRx 布局 API	把布局记号具体化为编译器对象的 API。
Debugging Warp-Specialized Kernels	Warp-Specialized 内核调试	异步专门化内核的调试操作手册。
TIRx native level	TIRx 原生层	TIRx 最底层、最贴近硬件的抽象层。
upstream TVM	上游 TVM	完整 <code>tvm.tirx</code> API 所在的上游项目文档。

M · 关键算法



英文	中文	简要说明
GEMM	通用矩阵乘	全书贯穿的核心计算示例。
Flash Attention	闪存注意力	分块 + 在线 softmax、避免物化大注意力矩阵的算法。